

PRODUCT TRUSTWORTHINESS

# 产品可信工程

人读规范与 Semantica 唯一  
执行语义

以 ISO 26262 为贯穿样板，以 Semantica 为唯一  
机器执行入口

关键突破口

生产

安装与维修

第二卷 · 规范语义与 Semantica 深度绑定

作者 claude F5 max · 5k · 用时 4 天 · 2026.08-semantica.1

jiaqiawang969/OntologyEngineering

书写规范 · Semantica 执行

人读教材 · 唯一机器语义入口



PRODUCT TRUSTWORTHINESS

# 产品可信工程

人读规范与 Semantica 唯一执行语义

第二卷 · 规范语义与 Semantica 深度绑定

以 ISO 26262 为贯穿样板，以 Semantica 为唯一机器执行入口

书是供人阅读、评审和追责的规范；Semantica 是全书唯一可执行语义。

作者：claude F5 max

Semantica 深度绑定候选审阅版 · 2026 年 8 月

内部文档号（非 ISBN）：PTW-V2-2026.08-SEMANTICA.1

章节包的可执行状态以 Semantica 注册表、验收收据与发布记录为准；本候选书不宣称全部章节包已经发布完成。



# 目录

|                                    |           |
|------------------------------------|-----------|
| 前言                                 | xi        |
| <b>第一部分 人读规范：按 ISO 规范建立可追责工程判断</b> | <b>1</b>  |
| <b>第一章 为什么“全绿”不等于产品可信</b>          | <b>3</b>  |
| 1.1 六盏绿灯，为什么没人敢签字                  | 4         |
| 1.2 一句能签的话，长什么样                    | 6         |
| 1.3 报告是工件，支持是关系                    | 8         |
| 1.4 只改一件事                          | 9         |
| 1.5 换一个世界，问法还在                     | 10        |
| 1.6 十个断面与一张地图                      | 11        |
| 1.7 散会之后                           | 12        |
| <b>第二章 先把词说清：从同名之争到术语体系</b>        | <b>15</b> |
| 2.1 白板上的争论，这个行业早就吵过了               | 16        |
| 2.2 相关项、系统、元素：分析边界的名字              | 17        |
| 2.3 故障、错误、失效：一条链，三个词               | 18        |
| 2.4 四个一百毫秒                         | 20        |
| 2.5 ASIL：风险的语言，不是荣誉的语言             | 20        |
| 2.6 换一个世界：ENV-01 需要自己的术语表          | 21        |
| 2.7 纸上语言到不了的三个地方                   | 22        |
| <b>第三章 谁有权说“可以相信”：安全管理与安全生命周期</b>  | <b>25</b> |
| 3.1 谁有权把两条记录判成同一台                  | 26        |
| 3.2 一场把签名页问穿的审核                    | 27        |
| 3.3 好人守不住的东西：安全文化                  | 28        |
| 3.4 有空的人，不是合格的人                    | 28        |
| 3.5 第二双眼睛该站多远：三种确认                 | 29        |
| 3.6 两百份报告，为什么还不是一句“可以相信”           | 31        |



|            |                                        |           |
|------------|----------------------------------------|-----------|
| 3.7        | 建议的尽头是决定：量产放行 . . . . .                | 31        |
| 3.8        | 纸面职责的尽头 . . . . .                      | 32        |
| <b>第四章</b> | <b>先决定担心什么：从使用情境到安全目标</b>              | <b>35</b> |
| 4.1        | 先画措施，为什么看起来如此合理 . . . . .              | 36        |
| 4.2        | 同一失常，换个处境就不是同一件事 . . . . .             | 37        |
| 4.3        | 你到底在担心哪一种损失 . . . . .                  | 38        |
| 4.4        | HARA 如何把安全问题压成一条可审查的纵线 . . . . .       | 40        |
| 4.5        | 三把尺子回答三种不同的问题 . . . . .                | 42        |
| 4.6        | 等级之后，目标仍然不能先写成方案 . . . . .             | 46        |
| 4.7        | 把同一问法搬到 ENV-01，什么必须留下，什么必须丢掉 . . . . . | 47        |
| 4.8        | 把问题冻结给第 14 章 . . . . .                 | 48        |
| <b>第五章</b> | <b>把目标接住：技术安全概念与系统层开发</b>              | <b>51</b> |
| 5.1        | 挂在墙上的目标 . . . . .                      | 52        |
| 5.2        | 翻译，不是抄写 . . . . .                      | 52        |
| 5.3        | 架构要经得起反问 . . . . .                     | 54        |
| 5.4        | 接缝上的合同 . . . . .                       | 55        |
| 5.5        | 一笔限时的预算 . . . . .                      | 56        |
| 5.6        | 逐层收账 . . . . .                         | 57        |
| 5.7        | 一张全绿的矩阵，答不上一个问题 . . . . .              | 58        |
| <b>第六章</b> | <b>数字要能作证：硬件层开发与硬件度量</b>               | <b>61</b> |
| 6.1        | 预算到手的那个下午 . . . . .                    | 62        |
| 6.2        | 机制要有上游：硬件安全需求怎么写 . . . . .             | 62        |
| 6.3        | 手册上的失效率，是谁的失效率 . . . . .               | 63        |
| 6.4        | 同样 2 FIT，去向为什么不同 . . . . .             | 64        |
| 6.5        | 把账算一遍：400 FIT 的底账 . . . . .            | 65        |
| 6.6        | 加一个 U7，只好了零点四八 . . . . .               | 67        |
| 6.7        | 过线之后：那个复制来的数 . . . . .                 | 68        |
| 6.8        | 纸面追不上数字的脚步 . . . . .                   | 70        |
| <b>第七章</b> | <b>通过会过期：软件层开发</b>                     | <b>73</b> |
| 7.1        | 失效率清单上，没有软件的名字 . . . . .               | 74        |
| 7.2        | 第一层承诺，要能生成标准答案 . . . . .               | 75        |
| 7.3        | 两个框，隔不出两个世界 . . . . .                  | 76        |
| 7.4        | 代码的纪律，写给编译器和三年后的人 . . . . .            | 77        |
| 7.5        | 覆盖率百分之百的那一天 . . . . .                  | 78        |

|             |                                      |            |
|-------------|--------------------------------------|------------|
| 7.6         | 拼起来才见真章 . . . . .                    | 78         |
| 7.7         | 差异分析：给每一张“通过”标注保质期 . . . . .         | 80         |
| 7.8         | 换一个世界，过期照旧 . . . . .                 | 81         |
| 7.9         | 纸面差异分析到不了的地方 . . . . .               | 81         |
| <b>第八章</b>  | <b>多一条通道，不等于多一分独立：ASIL 分解与相关失效分析</b> | <b>83</b>  |
| 8.1         | 散会时挂着的问题，和图上多出来的第二条通道 . . . . .      | 84         |
| 8.2         | 拆开的是承诺，不是工作量 . . . . .               | 84         |
| 8.3         | 两条通道，一个命运 . . . . .                  | 86         |
| 8.4         | 给“独立”一个分母 . . . . .                  | 88         |
| 8.5         | 换一个世界：ENV-01 的两只探头 . . . . .         | 89         |
| 8.6         | 纸面查不动的缝 . . . . .                    | 90         |
| <b>第九章</b>  | <b>设计走进工厂之后：生产、运行、服务与报废</b>          | <b>93</b>  |
| 9.1         | 移交会上，工厂接过的是什么 . . . . .              | 94         |
| 9.2         | 图纸之外：设计欠产线一份前提清单 . . . . .           | 95         |
| 9.3         | 一张对照表的代价：等效代换料 . . . . .             | 97         |
| 9.4         | 厂门不是终点：服务、维修与报废 . . . . .            | 99         |
| 9.5         | 现场的回声：数据什么时候才算证据 . . . . .           | 100        |
| 9.6         | 纸面的身份链，到这里为止 . . . . .               | 100        |
| <b>第十章</b>  | <b>回到放行桌：支撑过程与安全案例</b>               | <b>103</b> |
| 10.1        | 三百多份文件，合在一起说的是哪句话 . . . . .          | 104        |
| 10.2        | 六张绿卡，重新开口 . . . . .                  | 105        |
| 10.3        | 看不见的线：六种支撑手艺 . . . . .               | 107        |
| 10.4        | 会前一晚，十一点四十 . . . . .                 | 108        |
| 10.5        | 纸面的极限，与一扇门 . . . . .                 | 109        |
| <b>第二部分</b> | <b>唯一执行语义：由 Semantica 承接规范本体化</b>    | <b>111</b> |
| <b>第十一章</b> | <b>把“可以交付”写成机器能查的一句话：可信主张本体</b>      | <b>113</b> |
| 11.1        | 只限 EPS 的试点 . . . . .                 | 114        |
| 11.2        | 第一块砖：三元组、类与实例 . . . . .              | 115        |
| 11.3        | 结构的边界，是它承诺回答的问题清单 . . . . .          | 116        |
| 11.4        | 机器的“不”：缺件即拒，支持要凭据 . . . . .          | 117        |
| 11.5        | 空格的名字，与只改一件事 . . . . .               | 119        |
| 11.6        | 圈里的助手：提议、校验、执行、审计 . . . . .          | 120        |
| 11.7        | 名字还没有判据 . . . . .                    | 121        |

|                                 |            |
|---------------------------------|------------|
| <b>第十二章 名字背后的身份判据：对象与同一本体</b>   | <b>123</b> |
| 12.1 主张有了骨架，对象还没有身份 . . . . .   | 124        |
| 12.2 同一个什么：身份判据按族分立 . . . . .   | 125        |
| 12.3 记录是别名，“同一”要带证据 . . . . .   | 127        |
| 12.4 重演之一：一座用证据搭的桥 . . . . .    | 128        |
| 12.5 重演之二：消息进群之前的一道闸 . . . . .  | 129        |
| 12.6 百分之九十八的诱惑 . . . . .        | 130        |
| 12.7 本体不制造事实，也不拥有裁决 . . . . .   | 132        |
| <b>第十三章 把“谁说了算”画进图里：治理与责任本体</b> | <b>135</b> |
| 13.1 一次没人敢按下的回车 . . . . .       | 136        |
| 13.2 把一个名字还原成四样东西 . . . . .     | 136        |
| 13.3 六个位置，一张不许同坐的桌子 . . . . .   | 137        |
| 13.4 “够不够远”，变成一道算术题 . . . . .   | 138        |
| 13.5 旧案第二次发生 . . . . .          | 140        |
| 13.6 一条带日期的边：授权 . . . . .       | 141        |
| 13.7 组织一动，图里亮灯 . . . . .        | 142        |
| 13.8 机器买到的，和买不到的 . . . . .      | 143        |
| 13.9 表格里锁着的担心 . . . . .         | 144        |
| <b>第十四章 让担心可以被推理：情境与危害本体</b>    | <b>147</b> |
| 14.1 担心被锁在表格里 . . . . .         | 148        |
| 14.2 让一行表格站成七个对象 . . . . .      | 149        |
| 14.3 同一失常，第二次开庭 . . . . .       | 151        |
| 14.4 字母背后必须站着动作、窗口和画像 . . . . . | 153        |
| 14.5 图的自洽，买不来分级的正确 . . . . .    | 154        |
| 14.6 目标出发去下一站 . . . . .         | 156        |
| <b>第十五章 承诺不再丢失：需求与追溯本体</b>      | <b>157</b> |
| 15.1 四十多项的清单，翻出来重问 . . . . .    | 158        |
| 15.2 一条需求，长成一个对象 . . . . .      | 158        |
| 15.3 连线要带理由，分配要等确认 . . . . .    | 160        |
| 15.4 两毫秒，第二次来到门口 . . . . .      | 162        |
| 15.5 待定的窗口，也要有账本 . . . . .      | 164        |
| 15.6 重开清单，这一次机器先说 . . . . .     | 165        |
| 15.7 清单的最后一列 . . . . .          | 166        |

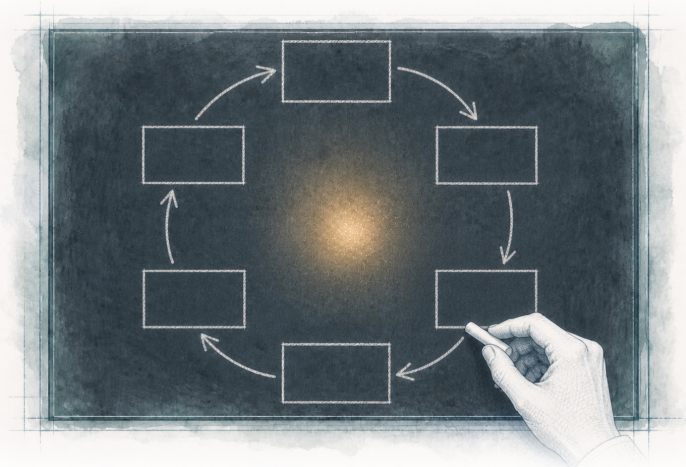


|                                     |            |
|-------------------------------------|------------|
| <b>第十六章 带着出处的数字：测量与证据本体</b>         | <b>169</b> |
| 16.1 热心的批量导入 . . . . .              | 170        |
| 16.2 一个数字的六条边 . . . . .             | 171        |
| 16.3 算出来的数字，出处是一条链 . . . . .        | 173        |
| 16.4 作证要报射程 . . . . .               | 175        |
| 16.5 那个数，这次没能过夜 . . . . .           | 176        |
| 16.6 提取是提议，入账要点头 . . . . .          | 177        |
| 16.7 证据都新鲜，可产品在变 . . . . .          | 178        |
| <br>                                |            |
| <b>第十七章 变化有了形状：版本与变化本体</b>          | <b>181</b> |
| 17.1 三个问题，等一个形状 . . . . .           | 182        |
| 17.2 变化的形状：快照与事件 . . . . .          | 182        |
| 17.3 每张“通过”，交代自己的世界 . . . . .       | 184        |
| 17.4 重开集合，由一条查询返回 . . . . .         | 185        |
| 17.5 两种痛快，被同一道门拦住 . . . . .         | 187        |
| 17.6 会自己动的那一维 . . . . .             | 188        |
| 17.7 助手圈得出集合，签不了字 . . . . .         | 189        |
| 17.8 查得动的绑定，查不动的缝 . . . . .         | 190        |
| <br>                                |            |
| <b>第十八章 看见方框之间的缝：依赖与独立性本体</b>       | <b>191</b> |
| 18.1 变化会传播，传播踩着依赖 . . . . .         | 192        |
| 18.2 把缝画成边：依赖成为一等对象 . . . . .       | 192        |
| 18.3 同一张电源树，第二次长出来 . . . . .        | 193        |
| 18.4 “图上不相连”的分母 . . . . .           | 196        |
| 18.5 把“独立”写成押着事实的主张 . . . . .       | 197        |
| 18.6 分解的信用账，记在结构里 . . . . .         | 198        |
| 18.7 机器看得见的，和它看不见的 . . . . .        | 199        |
| <br>                                |            |
| <b>第十九章 每一颗件都有自己的历史：制造与现场本体</b>     | <b>201</b> |
| 19.1 数据都上系统了，为什么还答不出“哪些件” . . . . . | 202        |
| 19.2 三层身份：一张图、一箱料、一颗件 . . . . .     | 202        |
| 19.3 历史长在单件身上：事件账 . . . . .         | 203        |
| 19.4 前提请出头脑：设计前提对象与代换合同 . . . . .   | 204        |
| 19.5 同一次断供，第二次发生 . . . . .          | 206        |
| 19.6 厂门之外：换件、回声与另一台设备 . . . . .     | 208        |
| 19.7 图不制造事实：诚实边界与放行桌 . . . . .      | 208        |

|                                                  |            |
|--------------------------------------------------|------------|
| <b>第二十章 活的安全案例：发布与保证本体</b>                       | <b>211</b> |
| 20.1 十套语言，回到一张桌子 . . . . .                       | 212        |
| 20.2 把案例变成对象：最小发布保证本体 . . . . .                  | 213        |
| 20.3 同一个深夜，第二次到来 . . . . .                       | 215        |
| 20.4 模型换了，什么还活着 . . . . .                        | 216        |
| 20.5 RC18：活的安全案例第一次开庭 . . . . .                  | 218        |
| 20.6 谁在指挥，谁在记得，谁有权改写 . . . . .                   | 221        |
| <b>附录 A 半导体应用指南：把 Part 11 读成一条证据链</b>            | <b>223</b> |
| A.1 导读：芯片不是一只黑盒 . . . . .                        | 224        |
| A.2 半导体部件划分、故障模型与 IP 边界 . . . . .                | 226        |
| A.3 基础失效率：来源、假设与计算边界 . . . . .                   | 231        |
| A.4 半导体 DFA：DFI 七分类与 B1-B12  workflow . . . . .  | 238        |
| A.5 故障注入与验证证据 . . . . .                          | 245        |
| A.6 生产、分布式开发与确认接口 . . . . .                      | 247        |
| A.7 五类技术案例：同一问题框走五遍 . . . . .                    | 249        |
| A.8 本体化实践：数值诚实与依赖链的半导体延伸 . . . . .               | 256        |
| A.9 要点回顾与思考题 . . . . .                           | 261        |
| A.10 回正文索引 . . . . .                             | 262        |
| <b>附录 B 摩托车适配与 T&amp;B 边界索引：Part 12 改了什么、为什么</b> | <b>265</b> |
| B.1 导读：同一套标准为何需要车型适配 . . . . .                   | 266        |
| B.2 适配总则与适用边界 . . . . .                          | 266        |
| B.3 安全文化与确认措施的摩托车适配 . . . . .                    | 269        |
| B.4 摩托车 HARA 与 MSIL 判定 . . . . .                 | 270        |
| B.5 整车集成、测试与安全确认 . . . . .                       | 275        |
| B.6 附录判例：严重度、暴露度与可控性怎么给证据 . . . . .              | 277        |
| B.7 本体化实践：MSIL 表的 Semantica 候选合同与未完成边界 . . . . . | 278        |
| B.8 要点回顾与思考题 . . . . .                           | 280        |
| B.9 回正文索引 . . . . .                              | 282        |
| <b>附录 C 术语表：一份受控词汇的五条概念路径</b>                    | <b>285</b> |
| C.0 怎么使用这份术语表（条目模板与受控来源） . . . . .               | 285        |
| C.1 结构群：从相关项到软件单元 . . . . .                      | 287        |
| C.2 因果群：故障、错误、失效与依赖失效 . . . . .                  | 290        |
| C.3 风险群：危害事件、S/E/C 与完整性等级 . . . . .              | 294        |
| C.4 需求群：安全目标、需求与工作产物 . . . . .                   | 298        |
| C.5 时间群：容忍、检测、处理与运行时间 . . . . .                  | 303        |

|             |                                     |            |
|-------------|-------------------------------------|------------|
| C.6         | 本体化实践：一份词表、多条概念路径 . . . . .         | 305        |
| C.7         | 字母索引、首讲章与交叉引用 . . . . .             | 306        |
| <b>附录 D</b> | <b>28 张方法表速查</b>                    | <b>313</b> |
| D.1         | 读表总则：§4.3、§4.4 与三层对象 . . . . .      | 313        |
| D.2         | Part 4：系统与整车集成的 14 张卡 . . . . .     | 315        |
| D.3         | Part 6：软件生命周期的 12 张卡 . . . . .      | 325        |
| D.4         | Part 8：TCL3 与 TCL2 的 2 张卡 . . . . . | 332        |
| D.5         | 按场景跨表导航 . . . . .                   | 334        |
| D.6         | 人工注释：组合理由与适用性边界 . . . . .           | 335        |
| D.7         | 本体化实践：Semantica 全表包的待建合同 . . . . .  | 336        |

# 前言



## 这一版书怎样与 Semantica 同步

这一版把全书的机器语义彻底收进 Semantica。书仍然保存为什么要建立这些区分、术语从哪里来、案例怎样把问题逼出来以及工程师应当怎样判断；但是本体、能力问题注册表、查询、Shape、正反案例、规则、exact oracle、版本、PROV、执行 receipt 与 release verdict 不再在书旁边各留一份。它们唯一存在于 `semantica.chapter_packages.vol2.ch01` 至 `vol2.ch20`，规范转述层唯一存在于 `semantica.chapter_packages.vol2.normative`。

这不是把书交给一个外部工具“跑一下”，而是重新规定二者的身份：\*\*书是人可读的工程规范，Semantica 是它唯一可执行的语义继承者。\*\*读者在正文看到的 Turtle、查询或门禁片段只承担解释作用，不是可复制运行的正本；所有机器结论都必须给出 package id、scenario、源 commit、wheel 哈希、输入/输出哈希和 receipt。Semantica 缺失、哈希不符或能力不支持时，结论必须停在 **blocked**，不存在书内 fallback。

同样重要的是，执行统一并没有改变权威边界。ISO 术语与模态的 Semantica normative package 是本书的工程释义和教学映射，不是标准原文、官方解释或合规证明。场景 oracle 通过也不等于 release 就绪；本版所有卷二包的 manifest 仍诚实标为 **partial**

且 `release_status=blocked`。真实产品事实仍来自受控工程系统，风险接受与放行仍由有权的人和组织承担。

二十章没有例外地采用同一映射：第 NN 章对应 `semantica.chapter_packages.vol2.chNN`，其主场景对应 `semantica.vol2.chNN.scenario.primary`；附加规范转述层是 `semantica.chapter_packages.vol2.normative`。前十章的规范知识、后十章的本体回答以及旧教学资产的迁移后继都只能通过这套 allowlisted registry 解析，不能把书中路径拼成文件名去运行。

所以，本版每一处“机器通过”都必须读成四行，而不是一个绿色单词：

1. 书中依据是哪一章、哪一段；
2. 实际执行的是哪个 package 和 scenario；
3. exact oracle 的状态与不支持能力是什么；
4. 独立 release verdict、输入/输出哈希与 receipt 说了什么。

四行缺一，结论就没有完整主语。尤其不能用场景通过覆盖 blocked release，也不能用本体一致覆盖未知事实或人的授权。

这一版还把“怎样使用这本书”从查阅升级为语义工作会话。任何工程 Skill 调用本书方法时，都应从 source-locked Semantica engagement 开始：task envelope 说明意图、请求动作、能力和带哈希证据，project binding 锁定行业 package/workspace、baseline、事实权威、决定权威与生命周期权限。每次调用固定返回工程结果、Semantica 结果和本体学习结果。`no_delta` 是有理由的正常结论；`candidate` 只是候选，不是新的行业事实。

有复用价值的 `candidate` 必须携带完整 package delta，并依次经过 `proposed`、`committed`、`regression_passed`、`release_complete` 与 `promoted`。其中 `committed` 只表示候选版本已冻结，`release_complete` 只表示技术条件闭合，`promoted` 才进入受控行业 registry；`published` 仍是仓库与权利人的外部决定。普通项目经验不会自动改写本书：只有通用方法、稳定教学失败模式、ISO 本体化推演或 Semantica 的真实能力边界发生实质变化时，才把 `book_impact` 送入两卷 TeX/PDF 作者流程。

## 从一位汽车零部件企业的老板开始

有一次，一位做汽车零部件的企业老板找到我。

他认为工程标准很有价值，也看见了一个很直接的机会：把这些标准做成 Agent，让人工智能帮助工程师理解要求、检查工作、寻找证据。这不只是一个技术设想，在他看来，它完全可以成为一门生意。

而我长期做的，正是 AI 原生工程本体。我们一拍即合。



把标准交给大语言模型，最先得到的会是一位很勤快的阅读助手。它可以寻找相关内容，解释术语，整理清单，也可以按照已有模板生成报告。再向前一步，它还可以进入流程，提醒缺失项，调用工具，追踪任务是否完成。这些能力已经能够解决很多现实问题，也足以形成有价值的工程服务。

那天下午，我们没有停在“怎样让 Agent 读懂标准”。有人在黑板上画下第一个框：客户需求。箭头向右，是研发；再向右，是试验和质量；然后是生产，把设计变成实物。画到销售和服务时，笔转了一个弯——市场反馈和现场问题，箭头掉头指回研发。黑板上出现了一个圈。

我们盯着这个圈看了一会儿，发现它的中间是空的。每一个部门都在圈上，可它们围着的究竟是什么？大家默认是“那款产品”。可是销售说的产品，是一个可以报价的型号；研发说的是一套图纸和参数；试验说的是几件编了号的样品；生产说的是下个月要交付的一个批次。四个箭头指向同一个词，却不指向同一个东西。

那一笔停下来的地方，就是这本书的起点。真正应该站在黑板中央的，不是某一个 Agent，而是企业共同维护的工程本体：哪一款产品、哪版零件、哪个功能、哪批产品、哪次试验、哪种使用情境，彼此究竟怎样相连。围绕这个中心，圈上的活动才真正连成闭环：销售带回的是有来源的需求候选，研发从当前产品定义、约束与历史证据出发，试验、质量和制造把结果连回具体对象、配置与条件，服务问题和员工经验也先作为可追溯、可质疑的知识候选进入核验。CAD 模型、原始数据和生产记录仍留在各自合适的系统中；工程本体维护共同的对象、身份、关系、规则与证据边界。每次活动都从共同状态出发，又把新证据带回来；被核验和接受的部分再更新共同状态，成为下一轮研发、经营和学习的起点。

也正是画出那个空心的圈时，我意识到，真正困难的部分还在更深处。

工程师说“这个产品已经通过”，这里的“这个”究竟是什么？是一个功能、一套机械总成、一块控制器、一个软件版本、一件实物，还是装在特定整机上的当前状态？所谓“通过”，通过的是一次试验、一项规则检查，还是关于产品可以交付的整体判断？如果部件、软件、环境或用途发生变化，旧证据还能支持今天的决定吗？

这些问题不能靠多读几遍标准自动回答。标准可以告诉我们应当关注什么、应当完成什么工作，真正的工程判断却必须回到具体对象、具体状态、具体证据和具体责任人。关系没有说清，Agent 只会更快地找到材料，也可能更快地把本来不属于同一个问题的材料拼在一起。

于是，最初关于标准 Agent 的讨论，逐渐变成了一个更大的问题：AI 时代的工程，究竟只是给旧流程增加一个会聊天的入口，还是需要重新建立一套能够被人和机器共同理解的工程世界？

从一门可以成立的生意继续向下挖，我们碰到了一门值得建立的学科。于是有了这本书。

## 模型一更新，就被吃掉了

在遇到这家企业之前，我自己先当过几年“学生”，也当过几年“手脚”。

我买过机器，请过老师傅，把一间小车间当成实验室。想让 AI 学会一门工艺，最诚实的办法是陪它一起学：它不会伸手，我就替它伸手——它提出观察和操作的建议，我去执行，仪器把结果记下来，我们再对着数据复盘：哪一步的判断错了，错在缺了什么条件。一圈一圈走下来，最先让我吃惊的不是 AI 能学多少，而是老师傅的经验有多难写下来。“这批料不对，转速要降一档”——降到哪一档？对哪些料成立？师傅说得出手上的动作，却说不出完整的条件。可一旦真的把条件逼问出来、写成规则、连上证据，这条经验就不再依赖师傅是否在场。

那段时间，我常在一个几位朋友的小群里聊到深夜。一天晚上，有人问：

“那是否有弱耦合的方式可以解决呢？”

我回答：“我们做应用的，其实，我今天说完，都好好想想，我们普通人，能做的，只有行业本体。”

“其他，模型一更新，就被吃掉了。”

这不是一个技术判断，是一个生存判断。这几年，做 AI 应用的人都体会过同一种失重感：你精心打磨的提示词，下一版模型自己就会了；你辛苦搭起的工具链，新的通用能力一夜之间让它变成多余；你以为的护城河，往往只是模型暂时还没学会的东西。在一浪接一浪的模型更新里，普通人、小团队、做应用的企业，究竟还能积累什么？

我的回答是：积累那些不属于模型的东西。你所在行业里的对象和它们的身份，条件化的经验和它的适用范围，试验和现场留下的证据，检验对错的评测，还有组织里谁有权对什么说了算。这些东西模型不会自带，下一版模型也不会替你长出来——因为它们不是语言能力，而是你的世界本身。把它们组织起来的那层结构，就是行业本体。

“只有行业本体”，认真推敲起来说得绝对了一点。留得下来的不只是本体：经过核实验的事实、原始数据、失败案例、接入真实设备与系统的能力、责任链和信任，都会留下来。更准确地说，行业本体是这些资产之间最稳定、最可迁移的组织结构——是那个让其他一切积累不会散掉的骨架。

这也是这本书把工程本体论放在根的位置的原因。工程本体论首先追问：在当前任务里，哪些事物被承认为对象，它们分别是什么，怎样判断还是不是同一个，又允许建立哪些关系。它不是一份更华丽的术语表，也不是把所有文件搬进同一个数据库。它要把产品、状态、活动、证据、判断、变化和决定分开，再说明这些事物怎样连接才不会互相冒充。

这层共同语义一旦缺失，更多报告、更多数据库和更多智能体，只会更快传播同名异义、错误身份和版本错配。人工智能越强，工程越不能继续依赖文件名、表格栏目和少数专家心中的默契来维持边界。

但“工程本体论是根”并不意味着本体可以包办一切。本体负责让参与者知道自己在谈什么，受控设计、仪器输出、试验记录和现场数据负责说明现实中发生了什么；有权的人和组织则要在看过证据、反证、假设与未知项以后作出决定，并承担相应责任。

因此，工程本体论是语义与共识之根，真实工程活动是事实之源，有权的人和组织是决定之源。大语言模型可以帮助人机沟通，工程技能可以约束工作方法，智能体可以执行边界清楚的动作，但语言流畅不能代替共同语义，调用成功不能代替工程事实，自动检查通过也不能代替决定权。

## 从功能安全走向产品可信

这本书最初的输入来自汽车功能安全。它是一条足够严格、足够完整，也足以暴露工程矛盾的纵向样板：风险怎样被识别，目标怎样被分解，硬件和软件怎样实现，证据怎样累积，责任怎样交接，最终又由谁接受一项关于产品状态的判断。

但产品值得相信，从来不只包含安全。可靠性关心产品能否在规定条件和时间内持续履行功能；稳定性关心输出或状态是否随扰动和时间漂移到允许范围之外；可用性关心需要服务时产品能否提供服务；网络安全、制造一致性、可维护性、环境适应性和其他产品价值，也都有各自的损失、证据和决定边界。

这些方面彼此相关，却不能互相替代。一个系统可以极其可靠地执行错误控制；为了安全而快速停机，可能降低可用性；输出平稳不等于测量准确；一次样件试验通过，也不代表批量产品与现场使用已经可信。

所以，本书没有把“安全”简单替换成另一个更大的口号。它保留汽车功能安全作为严格样板，同时把视野扩展到产品可信：不同工程关注各自成立、彼此制约，又能围绕同一个产品状态形成可检查、可质疑、也能随变化重新打开的判断。

## 前十章提问，后十章回答

本书分成前后两次穿行。第 1—10 章从现实工程中最自然的判断出发，让它们在矛盾面前逐一失效：局部结果全绿为何仍不能证明产品可信，同名对象、责任、场景、目标、数据、版本、依赖、制造、现场和最终放行又会在哪里断开。前十章先把问题中的对象、边界和失败条件问到不能再含糊；否则，任何提前出现的“智能答案”，都只是在替模糊问题增加速度。

第 11—20 章再逐一回答这十组问题。每章建立一个独立的工程本体例子，拥有自己的对象、关系、证据边界、检查方法和失败条件，也只对自己的问题负责。在真实企业里，这些有边界的本体可以在共同身份与治理规则下协作，形成企业级本体体系；协作不等于熔成一张万能大图，也不等于让一个领域的事实自动流进另一个领域。

这两次穿行也不是走完就结束。工程里的问题有一个规律：解决掉一批明显的，更细的会浮出来——更细的语义边界、更偏的例外状态、更深的耦合条件。所以本体不是

一次建完的词典，而是随着问题演化的活结构；这本书教的与其说是一个模型，不如说是让它持续演化的方法。

大语言模型、工程技能和智能体会在后十章全面进入。顺序之所以这样安排，不是因为人工智能不重要，而是因为它需要先进入一个知道自己在谈什么、依据什么、又不能越过什么边界的工程世界。

## 写给读者

这本书希望训练的，首先是一种工程习惯：每当看到“通过”“可靠”“安全”“完成”或“可以发布”，先问它的主语是谁、状态是什么、证据射程到哪里、谁有权接受，又有哪些变化会让旧结论不再成立。

如果顺序阅读，前十章会不断制造这种不适，后十章再把它变成可以运行和检查的结构。如果你主要负责产品与发布决定，可以重点观察主张、证据和权限怎样闭合；如果你关心人工智能进入工程，更应留意它在什么条件下获得行动边界，又在什么地方必须把判断交还给事实和责任。

但在习惯之外，还有一个更远的问题，想请你带着读完全书。

当老师傅的手感第一次可以被写下来、被检验、被机器执行，跨过人员流动和岗位交替继续起作用，一家企业就开始拥有一份不再困在某个人脑中的记忆。这时候，问题会一层层变深：起初我们问，谁在指挥这些工作；接着发现真正要问的是，谁在记得这些工作；而最后绕不开的是——谁有权改写这份记忆，写下它的人，又能不能继续被看见。经验的贡献者、系统的所有者和收益的获得者，可能不是同一批人。

最后一问，这本书回答不了，它属于每一个具体的组织。但前十章和后十章做的所有事情——把对象、证据、边界和授权写清楚——正是让这一问能够被认真讨论的前提。我们把经验从个人的脑中解放出来，不是为了让它被新的平台重新圈占。

本书不承诺一个能够吞掉安全、可靠性和稳定性的万能总分，不承诺本体自治就等于现实正确，也不承诺 Agent 能替人承担工程责任。它只尝试把过去依赖经验维持的对象、关系、证据和边界，建设成一门能够被人理解、被机器使用、被事实反驳、也由责任人作出决定的工程学科。

# 第一部分

人读规范：按 ISO 规范建立可追责工程  
判断





# 第一章 为什么“全绿”不等于产品可信

Semantica 绑定：本章是人可读的规范与案例叙事；唯一机器语义是 `semantica.chapter_packages.vol2.ch01`，主场景为 `semantica.vol2.ch01.scenario.primary`。本体、CQ、查询、Shape、正反案例、工程规则、oracle、manifest、PROV 与 receipt 均由该包持有，本目录没有执行副本或 fallback。当前 manifest 为 `partial`，release 为 `blocked`；场景通过也不能升级为产品可信或发布结论。涉及 ISO 术语/模态时，仅以 `semantica.chapter_packages.vol2.normative` 的工程释义层辅助核对，不能替代标准原文。



**【导读】** 一场只剩最后一页的放行评审，六个绿色的“通过”，却没有人签字。本章不打算指责任何人，而是陪你把这个僵局一层层拆开：先让“都通过了还等什么”这个再自然不过的判断完整地发生，再看它撞碎在哪里；然后回答一个更难的问题——一句能签的话，究竟长什么样，它由哪几个部件组成；再看证据怎样与这句话建立关系，一次寻常的软件升版又会让哪些关系重开；最后把问法带出汽车行业，并把这条裂缝展开成全书的地图。读完本章，你应当能把任何一句“可以交付了”，改写成一条能够被反驳的完整主张，并数出它还缺哪几个部件。

## 1.1 六盏绿灯，为什么没人敢签字

周五下午四点，一套电动助力转向系统的放行评审只剩最后一页。这一轮候选有个内部代号：RC17。投影幕上是六张结果卡，每张右上角都是绿色：概念边界、硬件分析、软件回归、系统台架、冲击振动、生产写入。

系统工程师小唐把六张卡过了一遍，说出了会议室里多数人心里的话：“六个专业都做完了，全部通过，目前也没发现任何问题。我们还在等什么？”

这句话一点都不外行。它背后是一个几乎所有工程师都默认过的推理：每个专业的检查都通过，当前没有发现问题，把这些局部的绿色加在一起，产品就可以放行了。小唐说得每张卡背后的工作量，也确实没有人伪造数据、没有人隐瞒失败。如果这个推理不成立，那么问题一定藏在某个不容易看见的地方。

项目负责人陈工没有接话。他转向台架负责人郑工：“你那盏灯，照的是 RC17 吗？”

会议室安静了几秒。郑工翻开记录，念了出来：“台架用的是 P07 样机。软件是 1.8.4 的 rc2 预览版——当时为了复现一个边界工况，正式版还没冻结。标定是 C42。”

而桌上这一轮要放行的 RC17，是 H3.2 硬件、SW1.8.3 软件、C41 标定。台架那张绿卡上的每一个字都是真的，它只是不在说 RC17。

郑工对这种沉默不陌生。几年前的一个项目里，也是赶进度，台架在样机和预览构建上跑出的绿灯，被顺手当成了目标配置的证据，样车照计划发运。入冬后试验场传回抱怨：低温冷起动后的头几分钟，助力来得慢半拍。零下二十几度，减速机构里的润滑脂稠得像冷蜂蜜，电机要多花力气才能把同样的助力送到齿条上；目标配置的软件里有一张按温度补偿摩擦的表，而样机上那版预览构建，表只标定到零下十度——再往下，它只会沿用边界值。台架在常温里跑，这张表的最后几行，从来没有被翻开过。查证花了两周，返工追回了十几辆样车，冬季试验窗口错过，项目推迟了六周。事故复盘会上有人问郑工：“你的台架不是通过了吗？”他至今记得自己的回答：“通过的是样机。”没有人说谎，没有报告作假——可代价是真金白银的。

有了台架这个口子，其余几张卡也被逐一问了一遍，答案陆续回来：硬件分析的对象是 H3.2 这一版控制器，前提是声明过的任务剖面和一组诊断假设，软件不在它的结论里。软件回归跑的是 SW1.8.3 加 C41，在受控测试集和实验室环境里，测试集之外的输入和系统级后果它没有回答。概念分析评的是候选相关项的边界和选定的安全场景，它从来没有声称过“产品已经实现并且可以放行”。冲击振动试验的对象是控制器、连接器和安装支架组成的局部总成，振动谱、冲击脉冲、轴向、夹具、时长和失效判据都有明确声明，整套转向系统和其他载荷谱不在其中。生产写入检查证明的是声明工位和批次上镜像写入与校验这个动作正确，它既不评价镜像内容的工程充分性，也不拥有放行的权限。

六盏灯没有一盏需要改成红色。它们只是没有在回答同一个问题。

僵局摆在桌上，会议室里冒出了三个补救方案——每一个都值得认真对待，因为每一个你都会想到。

有人提议做一张汇总表：六行结果全部写“通过”，完成度一栏算出百分之百。这张表五分钟就能做好，也适合放进周报。但它没有修复任何东西：加法只适用于同一种量，六个不同问题的答案，不会因为颜色相同就自动产生一个更大的答案。把它们加起来，就像把三公斤、五伏特和二十摄氏度加在一起——每个数字都真实，总和没有意义。

有人提议全部重测：用 RC17 的完整配置，把六项工作从头做一遍，让六盏灯照同一个东西。这个方案在白板上无懈可击，在日历上不成立。全量重测意味着数以千计的台架小时和一个不知道排到哪一年的试验场窗口，而 RC17 的决定窗口就在眼前；更要命的是，等最后一项测完，软件多半又升了版——重测追不上变化。一个要靠“冻结全世界”才能成立的方案，不是工程方案。

于是有人看向陈工：“您拍个板吧，总要有人负责。”这是三个方案里最诚实的一个，因为它至少承认了前两个没有解决问题。但签字能转移压力，不能制造证据。陈工签下名字，六份结论的主语并不会因此统一，会议室里只是多出第七个结论——一个连内容都说不清楚的结论：“陈工同意了。”郑工那年冬天的教训里，恰恰不缺签字。

三条最自然的路都走不通，这个僵局的真正结构才显露出来。把六张卡排开，逐一问一个最笨的问题：你说“通过”，通过的是谁？答案各不相同——一个候选相关项的边界；一版控制器；一个软件加标定的组合；一台装着预览构建的样机；一组局部样件；一次写入动作。它们彼此相关，却不能互相替换。六份报告都可以在自己的边界内完全正确，却仍然拼不成一句关于 RC17 的整体结论。

**绿色只是结论的颜色，不是结论的主语。主语一旦不同，颜色就不能相加。**

可是，“说清主语”到底要说清多少东西？小唐想签的那句话——“可以交付了”——如果不能签，那么一句**能**签的话，究竟长什么样？

**读图任务：**先沿着整车底盘中高亮的 EPS 路径确认对象，再逐张检查六份绿色结果各自覆盖了什么，最后看两条母线是否真的闭合为同一句主张。

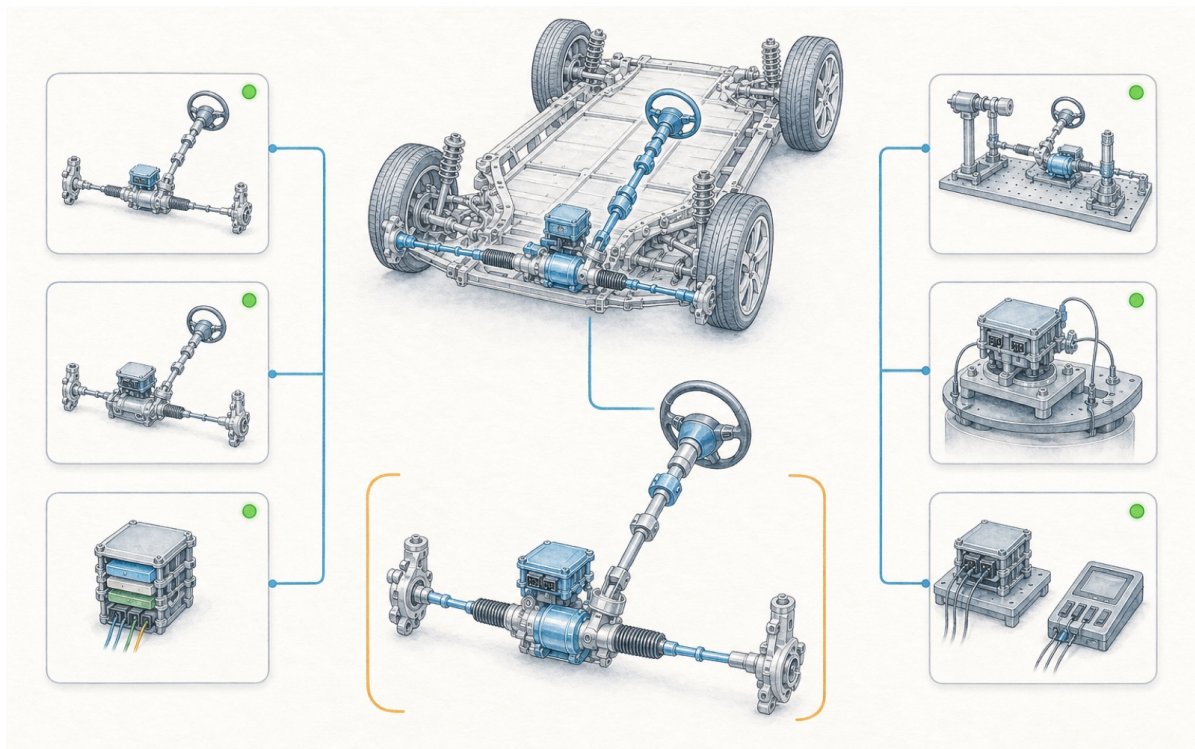


图 1-1 六份局部 PASS 可以分别真实，却因对象、配置、条件和决定范围未被同一边界绑定，不能仅靠数量组成产品级放行结论。

**图后判断：**图中最重要的不是六个绿点，而是它们在主张边界前各自停下。它说明局部结果不会自动相加；不证明任何一份报告为假，也不替代放行授权。

## 1.2 一句能签的话，长什么样

把“EPS 可以交付了”这句话放在解剖台上，一个部件一个部件地补齐，你会发现它缺的远不止一个版本号。

**先补主语。**“EPS”是产品线的名字，不是今天要放行的东西。今天桌上的是候选 RC17——一个特定的快照。产品名与产品快照是两回事：产品名可以活十年，快照只对应一种确定的组合。一句主张的主语必须是快照，否则六盏灯连对错都无从对起。

**再补配置。**RC17 究竟是什么？H3.2 硬件、SW1.8.3 软件、C41 标定、D7 驱动包、V12 整车集成基线。有了这张配置清单，六张绿卡第一次可以“对表”：硬件卡对上了 H3.2；软件卡对上了 SW1.8.3 加 C41；台架卡对不上——P07、SW1.8.4-rc2、C42，三项全偏。注意，对不上不等于报告作废，它只说明这张卡与这句主张之间的关系需要重新说明。没有配置清单，“对不上”这件事甚至无法被发现。

**再补关注。**同样一套 RC17，你担心的是什么损失？概念分析担心的是失常行为带来的伤害风险，这是安全的问题；冲击振动担心的是局部件在规定载荷下的失效，这是可靠性的问题。安全、可靠性、稳定性、可用性、网络安全、制造一致性——每一种关



注问的是不同的问题，收集的是不同的证据，之间还存在真实的取舍：一个系统可以极其可靠地执行错误的控制；为了安全而快速停机，恰恰牺牲可用性；输出平稳不等于测量准确；样件通过不等于批量可信。所以关注只能并列声明，不能折算成一个“可信总分”——总分掩盖的正是那些必须摆上桌面的取舍。本书保留汽车功能安全作为一条严格的纵向样板，但它是样板，不是其余关注的总名称。

**再补情境与时间。**在哪些工况、什么环境、哪个时段内，这句话有效？郑工那年的教训就藏在这一格里：台架的证据没有覆盖低温冷起动，而冬天如期而至。RC17 的主张还绑着一个决定窗口——这一轮放行决定本身有它的时限，过了窗口，主张需要重新确认，而不是自动续期。

**再补假设。**硬件分析的绿色建立在声明过的任务剖面 and 诊断假设之上；这些假设此刻并没有被确认，它们是被“先信了”的东西。假设不是耻辱，藏起来的假设才是。一句能签的话必须把自己站在哪些假设上写出来，因为假设一旦不再成立，主张随之重开。

**最后补决定范围。**这句话签了，授权的是什么？是“允许 RC17 进入本轮放行决定”，不是“这个产品从此可信”。还要写明谁有权签——评估的人和有权接受的人往往不是同一个人，把两者混起来，签字就只是仪式。

部件补齐，那句三个字的“可交付”变成了这样一句话：

针对候选 RC17（H3.2 硬件、SW1.8.3 软件、C41 标定、D7 驱动包、V12 集成基线），在已声明的关注、工况与本轮决定窗口内，基于列明的支持证据、反证与未知项，现有证据是否足以支持本轮放行决定——由有权角色接受或拒绝，并附重开条件。

这句话难看，但它终于可以被反驳了。别人可以指出目标配置并未真正受试，关键工况没有覆盖，某条反证没有处置，某个假设并未成立，或者签字的人没有相应权限。而“可以交付了”三个字光滑得没有一处可以着力——正因为它什么也没绑定，它才什么也不能承诺。

**一句不能被反驳的话，也没有资格被签署。**

主张缺了部件怎么办？答案不是含糊地“再看看”，而是把缺件显式列出来：缺哪个部件、为什么缺、谁负责补。缺件清单本身就是工程信息——一句主张允许带着已声明的缺口被讨论，但不允许带着未声明的缺口被通过。

这也给了“产品可信”它的第一个严格边界：可信不是贴在产品上的永久徽章，而是有权决定的人，针对明确的产品快照，在声明的关注、情境、时段、假设和决定范围内，看过支持证据、相反证据与仍然未知的部分之后，作出的可复查、也可撤销的接受。

部件齐了，下一个问题接踵而至：六张绿卡就在手边，它们和这句重写过的主张之间，究竟是什么关系？一份真实的报告，距离“支持这句话”，还差几步？

### 1.3 报告是工件，支持是关系

先把最硬的一张卡拆到底。软件回归那张卡，四百多项用例，自动化流水线跑出来的，每一项都有记录。用上一节的部件把它摊开：

| 追问     | 这张卡的回答                        |
|--------|-------------------------------|
| 什么活动   | 软件回归测试                        |
| 对什么对象  | EPS 控制软件                      |
| 哪版配置   | SW1.8.3 加 C41                 |
| 什么情境   | 受控测试集，实验室环境                   |
| 什么判据   | 既定验收准则，全部用例通过                 |
| 能支持什么  | SW1.8.3 加 C41 在该测试集与判据内满足回归要求 |
| 不能支持什么 | 测试集之外的输入；系统级后果；任何“可以交付”级别的结论  |
| 谁复核    | 软件负责人复核结果；放行决定不在此卡范围内         |

这张卡是好证据——在它声明的射程之内。射程之外，它什么也没说。现在把六张卡逐一放到 RC17 的主张旁边，问同一个问题：你与这句主张是什么关系？答案只有四种。

**支持：**在声明的射程内为主张作证。软件卡对 RC17 的软件与标定维度构成支持；冲击振动卡对局部件的可靠性关注构成支持。

**超出范围：**台架卡。它不是假的，也不是没用的——它支持的是另一句主张（关于 P07 样机加预览构建的主张），只是那句主张不在今天的桌上。把它算进 RC17 的证据，等于让证人在别人的案子里出庭。

**反驳：**郑工桌上还压着一页台架异常记录——那次边界工况复现里，有一条瞬时读数越出了预期带，后来没有复现。它可能是夹具问题，可能是预览构建的毛刺，也可能是真问题的第一次露头。反证不因为微弱就消失，它必须被处置：解释掉，或者立案追查。悄悄放进抽屉的反证，是所有放行事故里最常见的一页。

**未知：**低温冷起动没有测。注意，没有测不等于不合格——缺少记录时，默认答案是“未知”，不是“假”。只有先声明“这一格我们本应有记录”，缺口才成立为缺口。把未知当成假，会逼着团队为不存在的失败翻案；把未知当成真，就是郑工那年冬天的翻版。未知就是未知，它的名字必须出现在主张里。

四种关系摆开，一个更隐蔽的误会现了形。评审桌上常见两个等式：“文件在，证据就在”“没发现问题，就是支持”。两个都不成立。一份报告是一个**工件**——它存在、格式合规、字样为“通过”，这些都是工件自己的属性；而**支持**是工件与某句主张之间的**关系**——关系需要有人建立：对表配置、核对射程、处置反证、声明未知，然后才轮到“支持”两个字。工件不会自动长出关系。

这也是“绿色”这个词最容易骗人的地方。同样一片绿，至少有四层，每一层建立的东西不同：

第一层，**执行的绿**：任务跑完了，传输成功了。它证明动作发生，不证明对象正确。第二层，**结构的绿**：登记的数据符合声明的格式契约。它证明形式合规，不证明契约完整，更不证明内容为真。第三层，**证据的绿**：活动的产出可追溯，适用性经过复核，在声明射程内支持主张。到这一层，才有“证据”两个字。第四层，**决定的绿**：有权的角色，在声明范围内，记录了接受或拒绝，并附上重开条件。只有这一层对应“放行”。

四层逐级依赖，但每一层都不自动升级到下一层。流水线全绿是第一层，报告格式齐全是第二层，小唐把它们读成了第四层——这不是他的错，是“绿色”这个显示约定把四层压扁成了一种颜色。

**报告是工件，支持是关系；关系需要被建立，不会自动发生。**

为什么每个组织都不缺工件，缺的总是关系？因为工件可以被数：多少份报告、完成度百分之几，都能进周报、进考核；而建立关系的工作不产生任何新文件，它唯一可能的产出，是把一片让所有人安心的绿翻成“对不上”或“未知”。一个只统计工件的组织，实际上在奖励制造绿色、冷落核对绿色——关系没人建立，不是因为没人会做，而是做的人拿不到任何记功，还要先背上带来坏消息的名声。

假设团队照做了：配置对了表，关系分了类，反证立了案，未知进了主张。就在此时，走廊尽头的邮件到了——软件组申请把 SW1.8.3 升到 SW1.8.4。昨天刚刚对齐的一切，明天还剩多少？

## 1.4 只改一件事

变更申请写得很克制：软件从 SW1.8.3 升到 SW1.8.4；硬件仍是 H3.2，标定仍是 C41（除非软件差异明确波及），驱动包 D7、集成基线 V12、本轮决定的目的都不变。只改一件事。

会议室里立刻出现两派，两派都痛快。

一派说：全部作废，从头再来。既然配置变了，旧证据一律不认。这一派输在两处：其一，上一节刚刚算过，全量重测追不上变化，这条路走到头是“永远差一轮”；其二，它在浪费真证据——冲击振动试验的对象是机械局部件，软件升一百版也波及不到振动谱和夹具，作废它不是严谨，是用勤奋掩盖不敢逐项判断。

另一派说：软件而已，旧证据继续用。这句话应该让全屋后颈发凉——郑工那批追回的车，用的就是这句话的上一个版本。旧报告没有说谎，它只是被拿去支持了一句它够不着的主张。

两派错在同一个地方：把变化当成了对证据的整体判决——要么全体无罪，要么全体处决。而变化的影响从来是逐项的。正确的动作是一份差异分析，逐张卡回答：

软件卡：直接受击。SW1.8.4 相对 1.8.3 改了哪些模块、哪些参数，对应哪些用例需要重跑——它的射程卡上，“哪版配置”一格对不上了，“能支持什么”随之收缩，但方

法、判据、测试集的定义都还站着，四百多项用例并没有过期，过期的是它们与新主张的关系。

台架卡：一个诱人的陷阱。台架当时跑的是 SW1.8.4-rc2——软件升到 1.8.4 之后，这张卡反而“更接近”了，能不能顺势收进证据？不能。rc2 不是正式版，P07 不是目标样机，C42 不是 C41。“更接近”仍然不是“等于”，差异仍然要逐项说明。这张卡教了我们两次：上一节它教“对不上表要说明”，这一节它教“接近了也要说明”。

硬件卡：间接受击。H3.2 没变，但它的绿色建立在诊断假设上——硬件分析在计算失效风险时，默认有一道防线替硬件挡住故障，而那道防线是软件里的诊断：诊断多久跑一轮、按什么门限报警、报警之后走哪条处置路径，都写在软件里。如果软件差异触及这些路径，硬件卡上那些“先信了”的数就要重新问一遍，假设需要重新确认。假设清单在这一刻兑现价值：没有它，没人知道该来问硬件组。

生产写入卡：最容易被忘掉的重开。它看起来离软件最远，但它绑定的是镜像与校验和——软件一变，镜像必变，写入检查必须重做。“最不相关”的直觉，在配置清单面前不堪一击。

概念卡与冲击振动卡：可以保留，但保留要给理由。概念分析针对的是相关项边界，软件版本变化不改变边界本身（若安全场景的选定涉及软件行为，需复核确认）；冲击振动的对象与载荷与软件无关——这两条“无影响理由”要写下来，因为保留是一个判断，不是一个默认。

这就是“只改一件事”的完整答卷：哪些主张重开（整条放行主张必须重新确认），哪些证据关系重开（软件、台架、硬件假设、生产写入），哪些决定重开（上一轮任何引用了旧软件证据的接受），哪些证据凭明确理由保留（概念边界、机械可靠性）。既不是全部保留，也不是全部作废，而是逐项回答——这是唯一既不浪费、也不冒险的动作。

**版本变化不会把旧证据变成假证据；它改变的是旧证据与今天这句主张之间的关系。**

到这里，RC17 的故事讲完了。但一个疑问自然浮起来：这套问法——主语、配置、关注、情境、关系、变化——是转向系统专属的吗？出了汽车行业，它还成立吗？

## 1.5 换一个世界，问法还在

换一个世界看看。一台工业环境监测与控制单元——叫它 ENV-01——正在准备出货评审。它的桌上同样摆着六张绿卡：电路板设计规则检查通过，固件启动自检通过，实验室校准通过，环境箱稳定性试验通过，网络在线率达标，生产写入与序列号绑定完成。

熟悉的场面，熟悉的问题。这些绿色说的是同一台设备、同一个快照吗？设计规则检查通过，能推出产品可信吗？自检通过，能推出校准仍然有效吗？环境箱里输出稳定，能推出长期可靠吗？在线率达标，能推出它安全吗？序列号绑定完成，能推出这一台可以出货吗？



一个都推不出。而拆开这个僵局的问法，与 RC17 那张桌上的一模一样：主语和实例身份是什么，配置快照是什么，担心的是哪种损失，情境、方法、样本和时段是什么，哪些证据支持、哪些反驳、哪些缺口、哪些假设，谁执行、谁复核、谁有权决定。六问过处，ENV-01 的绿色同样分出了层次，同样露出了缺件。

但要小心另一个方向的偷懒：问法迁移过来了，答案不许跟着迁移。汽车行业为功能安全建立的分级和目标语义，是那个行业在自己的危害、自己的场景、自己的监管历史上长出来的，一个都不允许直接搬给 ENV-01；ENV-01 的每一个关注——数据准确、长期可靠、输出稳定、服务可用、生产一致——都要在自己的世界里重新回答，用自己的证据、自己的判据、自己的责任人。

**可迁移的是问法，不可迁移的是答案。**

这正是本书的结构原理：问法是方法，属于所有行业；答案是本体，属于每个领域自己。把这两者分开，一门学科才既能通用，又不冒充万能。

## 1.6  十个断面与一张地图

回头看 RC17 这场评审，它暴露的远不止一次放行的困难。如果团队连“正在谈哪个快照”都没说清，证据就找不到共同主语；如果“通过”“失效”“版本”这些词在不同专业里指向不同概念，大家即使用同一份模板，也在回答不同问题；如果执行、复核、接受风险和批准放行没有分开，签字就无法闭合成责任；如果产品没有被放进具体使用情境，元件可靠就会被冒充成风险可控；如果目标在派生、分配和接口之间悄悄丢失，如果数据脱离了对象、方法与来源，如果一次通过被默认穿越所有版本与漂移，如果多一条通道被当成了真正的独立，如果设计含义在制造、批次与现场断裂——最终，所有绿色结果堆在同一张放行桌上，仍然可能不属于同一个发布基线。

这是同一条裂缝的十个断面。前十章沿它逐段追问，后十章逐段回答，每章建立一个有边界、可检查、可反驳的工程本体，只对自己的问题负责：

| 提问    | 追问的断面               | 回答                         |
|-------|---------------------|----------------------------|
| 第 1 章 | 局部通过为何合不成整体可信       | 第 11 章把“可信”建成绑定对象、证据与决定的判断 |
| 第 2 章 | 同一个名字是不是同一个对象、同一种概念 | 第 12 章身份与概念的本体             |
| 第 3 章 | 谁执行、谁复核、谁有权接受       | 第 13 章治理与责任的本体             |
| 第 4 章 | 元件可靠为何推不出危害风险低      | 第 14 章情境与危害的本体             |
| 第 5 章 | 目标在派生与分配中怎样丢失       | 第 15 章需求与追溯的本体             |
| 第 6 章 | 脱离来源的数字为何还不是证据      | 第 16 章测量与证据的本体             |



| 提问     | 追问的断面          | 回答              |
|--------|----------------|-----------------|
| 第 7 章  | 一次通过为何穿不过版本与漂移 | 第 17 章版本与变化的本体  |
| 第 8 章  | 多一条通道为何不等于独立   | 第 18 章依赖与独立性的本体 |
| 第 9 章  | 设计含义能否连续到制造与现场 | 第 19 章制造与现场的本体  |
| 第 10 章 | 全部绿灯是否同一个发布基线  | 第 20 章发布与保证的本体  |

后十章的例子彼此有联系，却不会被塞进一个包办世界的超级本体。它们共享建立概念共识、组织证据和验证答案的方法，不共享未经证明的工程事实。

在这张地图上，三种权威始终分立：工程本体论是语义与共识之根——没有共同语义，十个断面无法被准确表达，大语言模型和智能体也不知道自己正在读取或改变哪一个对象；受控规范、设计原生工件、仪器输出、试验与现场记录是事实之源——本体的一条边永远替代不了一份实测；有权的人和组织是决定之源——门禁通过不是决定，Agent 的一次成功调用更不是。大语言模型负责语言与结构之间的翻译，工程技能约束方法，智能体执行边界清楚的动作；它们都是进入这个工程世界之后才获得工作边界的参与者，没有一个是可信的起点。

## 1.7  散会之后

散会前，陈工把“EPS”三个字母写在白板中央，说下周再议。任务单已经和开会前不一样了：郑工要给台架卡写一份差异说明——P07、rc2、C42 与目标快照的每一项偏差，以及那条压在抽屉里的异常读数的处置；小林要给软件升版列出差异分析清单，圈出定向重跑的用例范围；而主张部件清单的初稿，陈工点了小唐的名——那个想签字的人，现在负责把“能签的话”一个部件一个部件写出来。小唐没有推辞。他比谁都清楚那句“还在等什么”错在哪里，因为他是把它说出口的人。

只是这些任务有一个共同的前提，而它恰恰靠不住。白板中央那三个字母，硬件工程师看着它，想到的是一块控制器；小林想到的是一个软件构建；整车联络工程师想到的是一辆集成样车；质量工程师把它当成即将量产的一类产品。所有人都在诚实地使用同一个名字。问题也因此比“有人写错了版本号”更深：他们说的真是同一个对象吗？“可靠”“失效”“版本”这些词，在不同层级和时间尺度上，又是不是同一种概念？

第 2 章就从这场争论开始。没有共同身份和共同词义，判断还没来得及寻找证据，就已经指向了不同的世界。

导读里许过的承诺，现在可以兑现了。合上书之前，请找出你手头最近的一份“通过”报告，做三件事：把它支持的那句主张，按主语、配置、关注、情境、时间、假设、决定范围七个部件写出来；数一数缺了几件、缺件写给谁；再看看你自己的名字，签在

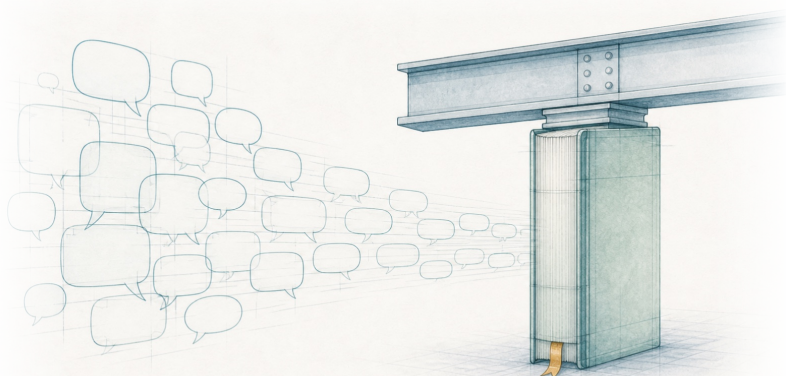
四层绿色的哪一层上。如果这三件事让你隐隐不适——这种不适，正是接下来九章要一层层展开的东西。

**本章注记：**本章的评审场景、人物、事故经过与全部产品数字（候选代号、硬件与软件版本、标定与基线编号、用例数、车辆数、时间跨度等）均为合成教学材料，不对应任何真实企业、真实产品或真实个人；文中的工程机理只用于说明证据边界的一般规律，不构成对任何实际系统行为的断言，也不表示任何报告已被接受或任何产品已获放行。



## 第二章 先把词说清： 从同名之争到术语体系

Semantica 绑定：本章保存术语辨析与工程方法；唯一机器语义是 `semantica.chapter_packages.vol2.ch02`，主场景为 `semantica.vol2.ch02.scenario.primary`。术语对象、本体、CQ、查询、Shape、正反案例、规则、oracle、manifest、PROV 与 receipt 均在该包中，本目录不设运行副本。当前状态为 `partial`、release blocked。ISO 定义的原文权威仍在用户合法持有的标准；`semantica.chapter_packages.vol2.normative` 只保存本书的工程释义与教学映射。



**【导读】**第 1 章散会时，白板中央的三个字母在四个人眼里是四个东西。本章不急着发明新办法，而是先回到一个常被跳过的事实：这场争吵，功能安全这个行业早就吵过了，而且把停战协议写成了一整卷术语——这是 AI 出现之前，人类工程共同体干过的最了不起的语言工程之一。本章陪你走过这卷语言的承重墙：相关项、系统、元素怎样划分分析边界；故障、错误、失效为什么是一条链上的三个位置；四个都写着“一百毫秒”的字段为什么不是同一种时间；ASIL 为什么是风险的语言而不是荣誉的语言。走到最后，我们也会诚实地看到这套纸上语言到不了的三个地

方——它们正是后十章的入口。读完本章，你应当能把一句含混的工程话改写成标准语言，并说出这套语言在哪里到头。

## 2.1 白板上的争论，这个行业早就吵过了

周一早上，白板上的“EPS”还没擦。陈工把上周的任务单过了一遍，然后把新入职的系统工程师小蔡介绍给大家——她从消费电子行业跳过来，今天第一天到岗。

小蔡的第一个问题就让会议室安静了：“你们说的 EPS，指的是那个功能，还是那块控制器？我看架构文档里写 EPS，台架记录里也写 EPS，好像不是一回事。”

有人笑了：“干这行的，心里都有数。”

这句话一点都不外行——它是每个成熟团队的真实经验：老搭档之间确实有默契，一个眼神就知道对方在说哪一层。但小蔡的存在本身就是反例：默契不会写在入职材料里。新人要靠犯错来学它，供应商靠猜测来学它，而下周要接入审核的第三方审核员，根本没有机会学它。靠默契维持的语言，覆盖半径只有一张熟人餐桌。

于是有人提议：那就发一份内部术语表，把常用词都定义一遍。这个方案也不外行——很多公司真的做了。但做过的人知道它的两个死穴：第一，术语表定义的是**词**，判定不了**对象**——它能告诉你“控制器”这个词的意思，却不能告诉你台架记录里那个“EPS”指的是不是软件构建；第二，术语表自己会繁殖：主机厂一套、三家供应商各一套，合作越深，表越多，对照表比术语表还厚。

第三个提议最硬气：“别自己编了，直接按标准来。”这个方向是对的，但落地时常常变成一句空话——标准的术语卷谁都知道存在，通读过的人寥寥无几；更没人告诉过团队，几百个术语里哪些是承重墙，哪些是脚手架。

这正是本章要做的事。但在走进之前，值得先停一秒，看清这卷术语是什么。

它不是**官僚文书**。汽车功能安全这个行业，几十年里在同样的争吵上付过真实的学费：“失效”和“故障”混用，导致处置走错门；“系统”边界不清，导致责任落空；时间参数张冠李戴，导致机制看似达标实则无效。这一卷术语，是整个行业替所有后来者吵完了架、验完了尸之后，把结论写成的**停战协议**。在没有 AI 的年代，把几百个含混的行业黑话逼问成精确定义，再让全球的主机厂、供应商和审核员说同一种语言——这是人力所能做到的最大规模的语义对齐。

**术语体系不是给会议增加负担的文书，而是一个行业为自己的争吵付过学费之后，写下的停战协议。**

为什么每家公司自己的术语表活不过三年，而这一卷标准的术语活了几十年？因为公司的术语表由文档部门维护，标准的术语由事故维护——每个定义背后都压着一付过学费的争吵。一个定义的权威，从来不来自发布它的机构，来自它替后来者省掉的那些重复损失。



所以本章的路线是：带着白板上那场争论，去标准的语言里找它早已写好的答案——先给“东西”找座位（相关项、系统、元素），再给“坏”找位置（故障、错误、失效），再给“快”找宿主（四种时间间隔），最后给“严”找刻度（ASIL）。

## 2.2 相关项、系统、元素：分析边界的名字

先回答小蔡的问题。功能安全的语言里，给“东西”准备的座位不止一个，因为分析需要的边界不止一种。

**相关项（item）**是功能安全生命周期的分析起点：在车辆层面实现一项功能（或功能的一部分）的那个系统或系统组合。说“EPS 相关项”，指的是“为驾驶员提供转向助力”这项车辆功能所对应的完整边界——方向盘上的转矩传感器、控制器、助力电机、传到齿条的机械路径，凡是这项功能成立所需要的，都在边界之内。整车工程师上周在白板前想到的“一辆车上的转向”，坐的就是这个座位。

**系统（system）**有一个容易被忽略的最低门槛：它至少要把传感器、控制器和执行器关联起来——相关的传感器或执行器可以在系统内部，也可以在边界外部，但一个孤零零的控制盒子不构成这里所说的系统。EPS 相关项之内，“转矩感知—计算—电机执行”这条链构成一个系统；它是系统工程师做架构分析时的座位。

**元素（element）**是个统称：系统、部件、硬件零件、软件单元，都可以在某次分析里被称为元素。它不是插在“系统”和“部件”之间的一个固定夹层——这是最常见的误读。说“元素”，是在说“本次分析正在处理的那个单位”，而不是在给零件定终身级别。同一块 H3.2 控制器，在整车级分析里是 EPS 系统的一个部件，在控制器级分析里自己就是被分解的对象；换一个分析，同一个东西合法地换座位。

这三个词有一个共同点：**它们分析边界的名字，不是零件的名字**。零件躺在货架上不会自己变成相关项；是分析任务把边界画在哪里，决定了它这次叫什么。把某个语境里的座位焊死在对象身上，再带进所有其他任务——上一章白板前的四种理解，一半的病根在这里。

那另一半呢？小林上周想到的“软件构建 SW1.8.3”，在这三个座位里都坐不下。它不是相关项，不是系统，也不是那种意义上的元素——它是一个**工件**：工程活动产生和使用的东西。需求文档、架构模型、源代码、构建包、测试报告、标定文件，都是工件。工件可以**描述**产品、**规定**功能、**实现**软件、**记录**证据，但它不会因为文件名里写着 EPS，就变成那台产品。把描述物当成被描述物，是四种理解里最隐蔽的一种错位——一份写着“EPS 架构”的文档躺在评审桌上，桌上并没有出现一台 EPS。

现在可以回头给白板上那场争论发座位牌了：整车工程师说的是相关项；系统工程师说的是系统；硬件工程师说的是一个元素（H3.2 控制器）；小林说的是一个工件（软件构建）；郑工台架上的 P07 是一台具体的物理样机——它在那次试验里扮演“被测对象”的角色。五个人都没说错，他们只是各自坐在不同的分析边界里，而白板上只写了三个字母。

争吵的解药不是禁止简称，而是在**交接的地方把座位说出来**。这正是标准语言的用法：不是让所有人天天念全名，而是在边界交接、证据流转、责任移交的关口，把“我说的是哪个座位上的东西”讲清楚。

座位分好了，下一个问题自然浮出来：东西清楚了，“坏”呢？三个专业各自嘴里的“故障”，是同一件事吗？

## 2.3 故障、错误、失效：一条链，三个词

先讲那个乌龙。几周前的一个早上，质量群里弹出一条来料检验消息：“R17 批次异常，请相关方关注。”十分钟内，消息被转进了项目群——候选代号 RC17 出了批次问题？陈工的电话被打爆，台架当天的排期被临时挂起，一场紧急会拉了七个人。四十分钟后，硬件负责人在会上举起一块电路板：“R17 是这颗电阻的位号。来料批次的失效率略超门限，正常处置就行。”会议室先是安静，然后爆发出那种混着后怕的笑。一场虚惊，半天排期，起因是两个只差一个字母的名字——以及一群没有先问“你说的是哪个座位”的人。

这个乌龙便宜地重演了上一章的教训，但它还送来一件更值钱的东西：那颗真实的 R17 电阻。顺着它，正好把“坏”的三个词走一遍。

假设 R17 内部出现了一道微裂纹。裂纹是一种**故障 (fault)**：一种异常条件，它能够导致元素或相关项失效。注意“能够”两个字——它不是“已经”。裂纹可能一直不影响阻值，可能只在振动和低温下间歇发作，也可能被电路的冗余设计挡在门外。故障可以潜伏、可以间歇、可以被隔离，它是一颗种子，不是一场收成。

如果裂纹发展成开路，R17 的“保持规定阻值”这项预期行为终止了——在 R17 这个观察边界上，这是一次**失效 (failure)**：故障显现之后，预期行为的终止。而同一个开路，站在控制器的观察边界看，它又成了输入网络里的一个故障：一个能够导致更大边界失效的异常条件。下层的失效可以成为上层的故障——但这是“可以”，不是“必然”；换层改名要看声明的结构和传播路径，不能当成自动规则。

开路让采样电压偏了。控制器读到的 ADC 值与真实值之间出现了差异——这是一个**错误 (error)**：计算、观察或测量的值与真实、规定或理论正确值之间的偏差。错误需要参照物才成立；它不专属于软件，也不以“看得见”来定义。一个错误可以被诊断读到却从未传播，也可以悄无声息地穿过滤波和判断，最终让控制器发出一个错误的转矩请求——如果执行路径接受了这个请求，车辆层面就可能出现那个所有人都听过的词：非预期助力。

把这条链排开：

故障（裂纹）-可能导致-> 错误（ADC 偏差）-可能传播为-> 失效（错误转矩输出）

每个箭头上都站着“可能”。裂纹可能没激活；偏差可能被校正；请求可能被独立限幅拦下。所以链上任何一环之后，都有四种不同的结局：已传播且有证据；条件未激活；已被容错或隔离；以及——证据不足，当前为未知。这四种结局不能共享一句“没出问题”：前三种各有各的证据义务，第四种压根还没资格下结论。

现在回头看，为什么说这三个词是承重墙？因为**处置从这里分岔**。说“故障”，工程动作是防激活、加检测、控制潜伏；说“错误”，动作是校正、抑制、限幅；说“失效”，动作是进入安全状态、降级、告警。三个词各开一扇门，门后是不同的机制、不同的证据和不同的责任人。上周郑工抽屉里那条台架异常读数，现在终于可以被准确地问了：它是一个错误候选——有参照、有偏差；它传播了吗？被容错了吗？还是证据不足？这三问的答案决定它走哪扇门，而在没有这三个词的会议室里，它只会被笼统地叫作“有个故障”，然后被笼统地处置——或者笼统地遗忘。

故障、错误、失效不是同一件事的三种说法，是一条链上的三个位置；把链压成一个词，处置就会走错门。

**读图任务：**从左向右跟随三个站点，特别看两个传播闸门和三条向下的拦截路径，判断为什么箭头只表示可能传播。

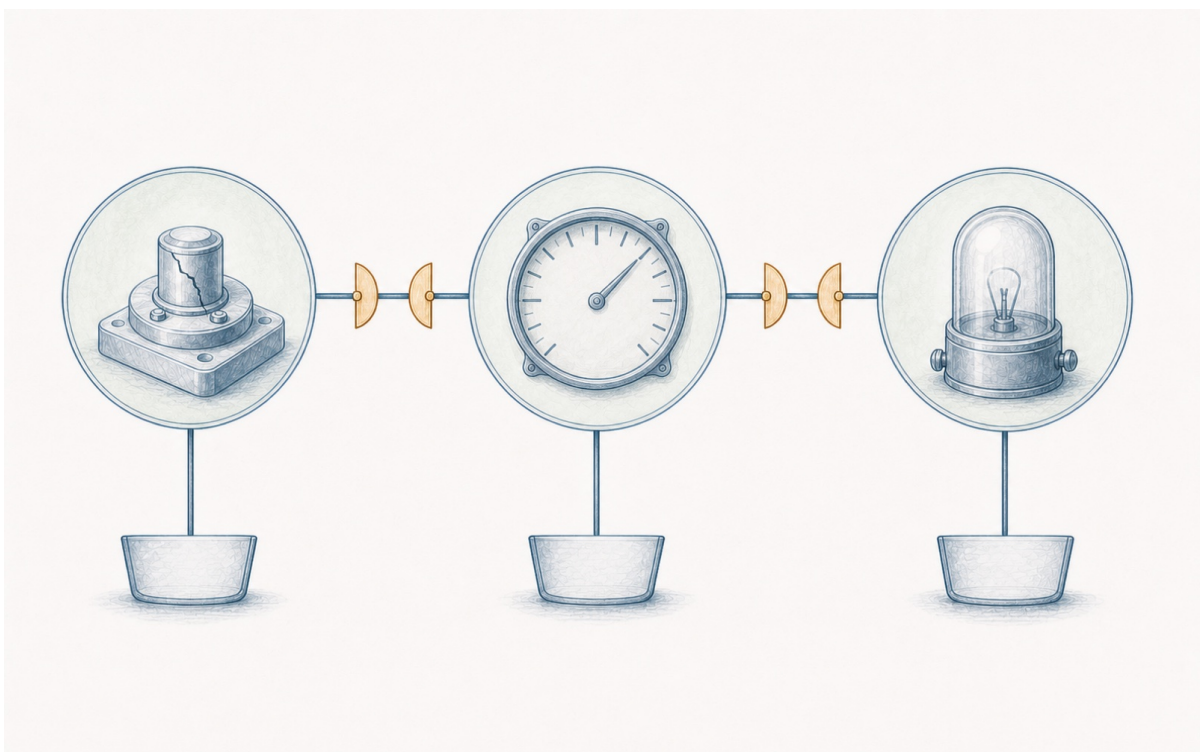


图 2-1 故障、错误和失效是一条因果链上不同位置，两段过渡都不是必然。图中的旁路提醒读者，拦截、修正或未激活都可以阻止下一站出现。

**图后判断：**这张图只规范三个概念的位置和条件性；它不给出具体系统的发生概率，也不证明任一监控或容错措施已经有效。

## 2.4 四个一百毫秒

链条分清了，表格里还埋着一种更安静的混淆。系统需求评审时，小蔡整理参数表，发现四个字段的值都是“100 ms”：诊断任务周期、故障检测时间、故障反应时间，还有一个写着“FTTI”。她问：“这四个能填同一列吗？反正都是一百毫秒。”

不能。同单位、同数值，不代表同一种时间——四个数字的起点、终点和“主人”各不相同：

| 名字           | 从哪里到哪里                    | 它是谁的属性      |
|--------------|---------------------------|-------------|
| 诊断任务周期       | 一次诊断执行到下一次执行              | 软件调度的一个参数   |
| 故障检测时间       | 故障发生 → 被检测到               | 某个安全机制的能力   |
| 故障反应时间       | 检测到 → 到达安全状态或应急运行         | 同一机制的另一段能力  |
| 故障容忍时间（FTTI） | 故障发生 → 若无机制干预、危害可能出现的最短时间 | 相关项与危害情境的属性 |

前两段相加（检测加反应），得到的是**机制**的端到端处理时间——它是机制的属性；而 FTTI 是**相关项**的属性，由车辆功能和危害情境决定：非预期的助力在方向盘上作用多久，驾驶员就再也拉不回车道——这扇窗口的宽度由车速、路面和人的反应决定，机制再快也改变不了它，机制能做的只是赶在窗口关上之前完成检测和反应。“机制不够快”这个问题，从来不是比较两个裸数字，而是证明：针对同一个故障、同一种模式、同一个危害情境，机制的处理时间落在相关项的容忍窗口之内。把四个“100 ms”填进同一列的表格会短很多，但推理会失去起点、终点和责任主语。

顺着时间，还有一个词需要在这里说死：**安全状态（safe state）**。它不是名叫“安全”的永久标签，而是失效情境中相关项的一种运行模式——在那个失效情境下，不带来不合理的风险。两个边界经常被踩：其一，关断不自动等于安全状态——对高速行驶中的转向系统，突然失去助力本身可能制造新的危害；其二，安全状态是状态，不是身份——RC17 进入安全状态后仍是同一个候选，两台同时降级的控制器也不会因此变成同一台。

同单位同数值不等于同一种时间；每个数字都要有宿主、起点和终点。

## 2.5 ASIL：风险的语言，不是荣誉的语言

术语体系还有最后一面承重墙。评审材料里到处写着 ASIL D、ASIL B，小唐提过一个很自然的问题：“要不咱们统一按 D 做？最严的标准，总不会错。”

这句话的善意是真的，误解也是真的。ASIL——汽车安全完整性等级——是对“这项危害需要多大力度的风险降低”的分级回答，从 A 到 D 逐级加严，另有 QM 表示按常规质量管理即可。它是**风险的语言**：等级来自危害分析中对严重度、暴露概率和



可控性的三项判断（怎样判，是第 4 章的正题），而不是产品的档次徽章。“按 D 做最保险”错在两处：其一，D 级意味着成倍的验证负担和架构约束，把它花在不需要的地方，吞掉的是真正 D 级项应得的注意力和预算；其二，它把分级从“证据的结论”偷换成了“态度的表达”——而一个靠态度而不是靠分析定级的团队，迟早也会靠态度降级。同样，QM 不是“没有要求”的意思——它意味着危害分析做完了、结论是常规质量流程足以覆盖，这本身就是一个需要证据的判断。

**ASIL 是风险的语言，不是荣誉的语言：它标注的是危害要求多大的力度，不是产品配得上多高的头衔。**

2.6 换一个世界：ENV-01 需要自己的术语表

这套语言这么好用，能不能整卷搬给别的行业？拿第 1 章那台环境监测单元 ENV-01 试试。

ENV-01 的评审桌上也有一句熟悉的话：“设备已验证可靠。”追问下去，校准工程师说的“可靠”是读数接近参考仪器；固件工程师说的是采样任务长跑不死机；运维说的是要数据时接口在线；质量说的是批内失效没超门限；维修说的是坏了好修；数据负责人在意的是时间戳和标定身份没在传输里被弄脏。一模一样的病，需要一模一样的药：把一个形容词拆成各自独立的关注，每个关注配自己的问法和证据——

| 关注    | 对 ENV-01 的问法        | 它不自动证明       |
|-------|---------------------|--------------|
| 准确度   | 读数与受控参考值差多少         | 漂移小、寿命长、接口在线 |
| 漂移稳定性 | 误差在声明时间与环境内是否有界     | 初始值准、设备可靠    |
| 可靠性   | 规定条件与时段内功能能否持续      | 任一时刻都可用、维修容易 |
| 可用性   | 需要服务时能否提供有效服务       | 数据正确、长期不漂移   |
| 可维护性  | 异常能否按目标时间诊断、修复、复验   | 失效不发生、后果可接受  |
| 数据完整性 | 数据从采集到存储保持正确绑定与未被篡改 | 传感器准确、系统在线   |

但注意搬过来的是什么：是“把词说死”的方法——先分对象、再分关注、再给每个数字找宿主——而不是词本身。相关项、ASIL、安全状态这些词，是汽车行业在自己的危害、监管和事故史上长出来的，出了这个行业就没有着落；ENV-01 的世界要用同样的方法，长出自己的词。整卷照搬术语，和整卷拒绝术语，是同一种懒惰。

**可迁移的是“把词说死”的方法，不可迁移的是词本身。**



## 2.7 纸上语言到不了的地方

到这里，白板上那场争论已经有了体面的收场：东西有了座位，“坏”有了链条，时间有了宿主，“严”有了刻度。这就是 AI 之前的世界能做到的最好水平——而且必须说，它相当好：一卷术语让全球的主机厂、供应商和审核员说上了同一种语言，这件事在人类工程史上并不常见。

但只要在工程现场多待几天，就会看见这套纸上语言到不了三个地方。

**它定义了词，定不了记录。**术语卷能告诉你“元素”是什么意思，却回答不了那个每天都在发生的问题：台账里的 DUT-P07、制造系统里的 SN-EPS-000417、PLM 里的 EPS\_ASSY\_043——这三条记录是不是同一台东西？老工程师凭记忆说“是”，但记忆不是证据。同名不能自动判同，异名不能自动判异，桥接证据不足时，诚实的答案只有“未知”——而纸上的术语体系，连表达这个“未知”的格式都没有。

**它的纪律靠人。**词的边界写在标准里，守边界的却是人的记忆和自觉。新人要重新学，交接会走样，进度一紧，“心里都有数”就会卷土重来。上周的评审、这周的乌龙、下周的新供应商，每一次都在考验同一批人的语言纪律，而纸面没有任何机制能替他们把关——文档不会在你用错词的时候弹出警告。

**它进不了机器。**当 Agent 开始读需求、查记录、写报告，它面对的是同一堆同名异物和异名同物，而术语卷对它只是又一份文本。没有一种机器可查的形式告诉它：这两条记录能不能合并、这个“故障”指的是链上哪个位置、这句“可靠”绑定了哪个对象和哪个关注。语言的裁判权还锁在少数专家的头脑里——这正是前言里那块黑板真正的痛处。

所以本章末尾冻结的，不是答案，是三个必须留给后十章的问题：给定多个系统里的一组名字，怎样判定它们指同一对象、不同对象还是未知，判据和证据是什么；给定一句“某某可靠”，怎样机器可查地展开它绑定的对象、配置、关注、条件、时间与判据；给定一堆工程记录，怎样自动识别其中的座位错误——把文档当产品、把角色当本质、把状态当身份。第 12 章将从这三个问题出发，把本章的术语体系变成机器可以检查的身份与概念结构——那是 AI 之后的世界的做法。

而眼前还有一个更近的问题。散会时小蔡问了今天的最后一个问题：“如果我查出来 DUT-P07 和 SN-EPS-000417 就是同一台，这个结论谁来批准？我能直接改台账吗？”没有人立刻回答。录台账的人、做试验的人、管 PLM 的人、批放行的人——谁有权宣布两条记录是同一个东西？如果系统管理员能改映射表，他是不是就拥有了工程裁决权？第 3 章就从这个问题开始：把词说清之后，还得把“谁说了算”说清。

导读的承诺可以兑现了。合上书之前，做三件事：在你自己的项目里，找出同一个产品在三个系统里的三个名字，问一问桥接证据在哪里；把最近一份报告里的“故障”改写成链上的准确位置；再找一个表格里的时间参数，写出它的宿主、起点和终点。如果哪一件做不下去——记住卡住的位置，后十章会回到那里。

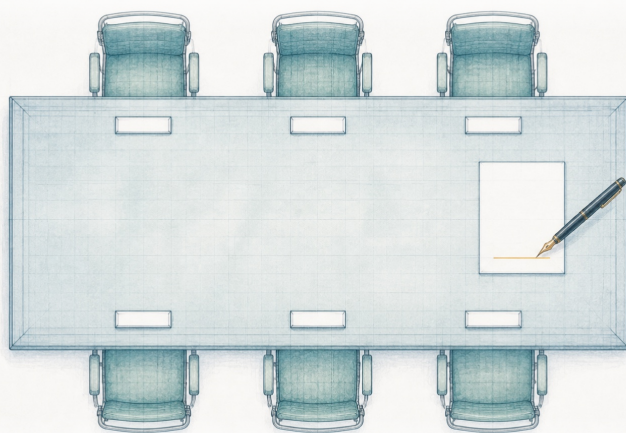
术语体系统一了词的定义，统一不了记录的对象——这是纸面语言的边界，也是第 12 章的起点。

**本章注记：**本章的人物、会议、乌龙事件与全部产品数字（候选代号、位号、序列号、参数值等）均为合成教学材料，不对应任何真实企业、真实产品或真实个人。正文对 ISO 26262 术语的转述为自然语言概括，精确定义以标准原文为准；本章不构成对任何实际系统的分级、判定或放行结论。



## 第三章 谁有权说“可以相信”： 安全管理与安全生命周期

Semantica 绑定：本章保存治理问题、角色与责任的可读论证；唯一机器语义是 `semantica.chapter_packages.vol2.ch03`，主场景为 `semantica.vol2.ch03.scenario.primary`。治理本体、CQ、查询、Shape、案例、规则、oracle、版本与 receipt 只在 Semantica。当前 manifest 为 `partial`、release 为 `blocked`；结构门禁不能制造组织授权，最终决定仍属于有权人员。



**【导读】** 上一章散会时，小蔡的问题悬在半空：查证两条记录是同一台之后，谁有权批准改台账？本章顺着这个问题，走进一个常被当作行政事务的领域——安全管理。先看三条最自然的答案怎样逐一落空；再听功能安全经理方工讲一场把签名页问穿的审核；然后把“有人负责”拆开成几件不能互相顶替的东西：让坏消息走得动的文化，让看的人看得懂的能力，让第二双眼睛站得够远的三种确认，把两百份报告变成一句话的安全档案，以及签字页上那个必须够得着风险的名字。最后，我们诚实地看这套纸面职责到不了的地方。读完本章，你应当能对任何一个“通过”的判断问出三张票——谁做的、谁挑战的、谁有权接受——并说出这三张票为什么不能落在同一个人手里。

### 3.1 谁有权把两条记录判成同一台

散会的人已经站起来一半，小蔡的问题把他们又按回了椅子：如果查实台账里的 DUT-P07 和制造系统里的 SN-EPS-000417 就是同一台样机，这个结论谁来批准？她能不能直接改台账？

第一个答案几乎是脱口而出的：“谁录的谁改。台账是台架组建的，让录入的人改回去就是了。”这个答案不外行——离事实最近的人，最清楚当初是怎么录的。但它有个绕不过去的弯：录入的人正是差错的当事人，让他独自判定“这两条是同一台”，等于让他给自己的差错下结论；更何况这一改牵动的不只是一行字——台架那张绿卡的归属、样机的试验履历，都挂在这条记录上。改错一个字是笔误，改错一条对应关系是改写证据。

第二个答案更实际：“找管系统的人。反正只有系统管理员有写权限。”也不外行——多数公司真就是这么改的。可是把这句话放慢了念，会念出一身冷汗：管理员有的是**权限**，一串账号密码给他的能力；他没有的是**职权**——判定两条记录同指一物是一个工程裁决，要看桥接证据，要担后果。让权限最大的人顺手做裁决，工程问题就变成了运维问题；上一章末尾那个疑问，正是从这里冒出来的。

第三个答案最省事：“让陈工拍板。”陈工这次自己摇了摇头：“我可以签。可要是有人问我凭什么签——哪份文件授权项目负责人裁决台账归并——我答不上来。上周五我们才把这个道理摆上过桌面：签字转移压力，不制造证据。”他顿了顿，“这问题不该我答。该请方工来。”

方工是公司的功能安全经理，入职五年，多数会议没人想起请他。他听完问题，没有先给答案，而是把问题翻了个面：“你们在问谁有权改一条记录。这个行业几十年前就发现，这类问题背后是同一个结构：一件事要成立，得分出几种人——动手做的人，出证据的人，挑毛病的人，承担后果的人，授权生效的人。台账这件事的答案不难：数据责任方带着桥接证据提出归并，一个不相干的人复核，有权的人批准并留痕。难的是另一件事——这套分法靠什么活着。”

**写入权限是系统发的，裁决职权是组织授的；两样一旦混同，管理员就成了裁判，签字就成了仪式。**

小唐有点不服气：“可我们的流程里有这些啊。每份报告都有编制、校对、批准三栏，哪一栏都没空过。签了字，不就是有人负责了吗？”

这话不外行——绝大多数组织正是靠这三栏签字运转的，它也确实比什么都不签好得多。方工没有反驳。他讲了一件旧事。



## 3.2 一场把签名页问穿的审核

七八年前，方工在上一家公司做底盘电子。一个项目走到量产定点前的关口，客户安排了一场功能安全审核。那家公司自认为准备充分：文件目录三层，签名页齐整，每份报告的编制、校对、批准三栏没有一处空白。

审核员没有翻目录。他抽了两份文件。

第一份是一项关键安全分析——作者正是当年的方工。“独立评审”一栏签的是他的组长：坐他斜对面的工位，和他向同一位部门经理汇报，年终考核由同一支笔打分。审核员只问了一句：“如果这份评审的结论是否定的，谁的项目要停？谁的奖金要少？”屋里没人答得体的。评审人确实不是作者，可评审人的日程、考核和前途，和作者拴在同一根绳上——这样的第二双眼睛，睁着，但不敢看。

第二份更难看。一条试验里发现的已知问题，处置记录写着：“经会议讨论，接受该风险。”纪要有日期、有议题、有参会名单，唯独没有一个名字对“接受”两个字负责。审核员问：“那么今天，这个风险由谁承担着？”会议室里翻了四十分钟文件。当年散会的人各自以为别人拍了板；三年过去，风险还在产品里，承担它的人不存在。

审核当场中止，定点被冻结。整改的头两个星期，公司试的是最顺手的两种补救。先是加签字栏——会签从三栏加到七栏，结果每个签字的人都更放心了：这么多人看过，总有别人把过关。责任没有变厚，只是摊薄了。接着换模板——新表单专设“风险接受人”一栏，可填这一栏的人职权从哪里来，表单自己说不出；栏是新的，空还是原来的空，只是看不出来空。到第三个星期他们才认账，一条条把“会议接受”改写成有名字、有职权来源的决定，把每一场评审配上真正隔开利害的人。整改用了八个星期，再加上等客户重新排期审核，量产推迟了一个季度——模具按原定投产日摊销、整条产线空转三个月，赔进去七位数；第二年，那家客户的新平台没有再向他们询价。

“最疼的是哪一点？”方工说，“签名页上没有一栏是空的。我们不是输在没签字，是输在每一栏都填了，却没有一栏指向一个真正有权、知情、独立的人。”

**签名不制造责任，只指向责任；当指针指不到一个有权、知情、够独立的人，签得越多，越像有人负责，也就越没有人负责。**

这样的签法为什么能一年年续下去，没有被任何人叫停？把每个人的账摊开看，它对谁都是最顺的一条路。让组长签评审栏，当天就能把栏填上——不用跨部门借人，不用等别人的日程，节点一个不误；省下的时间立刻记在进度上，压进去的风险要几年后才来收账。客户的审核几年一次、一次只抽得动两份，轮到自己这份被问穿的机会小得可以忽略。而看得见这件事的人，恰好都没有说破的理由：作者说破，是请人来挑自己的毛病；组长说破，是给自己添一份推不掉的活；部门经理说破，等于承认本部门的评审一直不算数。一件人人省事、无人受损的安排，不需要坏人来维护，自己就会活下去——直到一个考核不在这家公司的人，问出那句“谁的奖金要少”。

小唐安静了。可另一个疑问随之浮上来：那家公司没有坏人——没人伪造数据，没人隐瞒失败，每个人都按流程办了事。好人齐全的组织，责任为什么还是蒸发了？

### 3.3 好人守不住的东西：安全文化

责任蒸发的路径，每一步都平淡无奇。安全负责人调岗，他计划里的一项评审从此没了主人，几年没人过问——不是没人看见，是谁都觉得“总有人管”。进度吃紧，会上有人提“先发运、试验后补”，没人赞成，也没人反对——沉默通过。有人在评审里提了个尖锐问题，被夸了一句“很认真”，然后议题跳到下一页。每一幕单看都构不成过失，连起来就是那页找不到名字的纪要。

对付这种事，组织最先想到的三件套，都值得认真对待一次。第一件是培训加标语：走廊里挂“安全第一”，全员签承诺书。可墙上的字管不住日程表——当“安全第一”和“节点第一”打架时，决定胜负的从来不是标语。第二件是打分：设计一张文化测评表，年年自评。可打分检查的是表格填得全不全，不是组织在真实冲突里选了哪边；一个奖励抄近路的组织，照样能把测评表填成满分。第三件是抓典型：出了事，处分一个人。这最解气，也最“有效”——它高效地教会所有人一件事：别让坏消息署上自己的名字。

那么文化究竟是什么？看一条坏消息的旅程就知道。郑工抽屉里那条台架异常读数，压了多久？他不是坏人，也不是不懂——上一回有人把类似的读数报上去，换来的是一句“别自己吓自己”，外加一整晚写解释材料。报忧是有价格的：时间、难堪、被当成麻烦制造者的风险。价格越高，抽屉越深。而上周五之后，那条读数有了编号、责任人和处置期限——不是郑工突然变勇敢了，是这条消息被报出去的价格变了。

所以观察一个组织的安全文化，不要听它说什么，看四件事：坏消息报上去，资源、日程和决定会不会真的改变；安全和进度冲突时，哪个真的让路；奖励的名单上，有没有及时喊停的人；提出异见的人，第二年过得怎么样。

**安全文化不是墙上的标语，而是一条坏消息从被发现到改变决定所要走的路；这条路越短、越不需要勇气，文化越好。**

通道修通了，还只解决了一半。走进通道的疑问，得落到一个看得懂它的人手里。可是——谁算“看得懂”？

### 3.4 有空的人，不是合格的人

RC17 的补课清单摊开：台账归并要人复核桥接证据，台架差异说明要人挑战，将来那份安全档案要人评。派谁？

“派最资深的。”资历是真东西，但它回答的是“干了多少年”，不是“干没干过这件事”。一位干了二十年转向的老专家，可能从没评过一次危害分析；让他上，是拿威望冒充判断。

“派有证书的。”证书也是真东西，它证明一个人学过一类知识、通过了一次考试。可它证明不了这个人能对**这一份工作**产物下判断——一位熟读规范的专家，可能分不清转矩传感器和角度传感器谁的失效更险。

“派有空的。”这个最诚实，也最普遍——排期表上谁闲着谁上。排期表不是能力表。

三个答案共同的毛病，是把能力当成了挂在人身上的属性——资历、证书、头衔，随身携带，处处通用。而能力其实是一段关系：这个人与这项具体活动之间，相称不相称。相称至少要对三样：领域——他懂不懂转向系统、电机和整车工况；方法——他没做过这类分析，知不知道坑在哪里；语境——他熟不熟这个组织的流程和这个项目的历史。三样各自独立，谁也抵不了谁。

小蔡就是现成的例子。她在消费电子做过五年可靠性试验评审，方法这一样她比屋里多数人熟；可她上周才第一次见到 EPS 的台账，组织语境这一样她几乎为零。所以她可以做台账复核的助手，暂时当不了复核的主人——这不是资历歧视，是相称性的诚实结论。而相称是会过期的：技术路线换了、规范改版了、人调了岗，去年的“合格”就要重新回答。组织欠每个关键活动一本活的能力账：谁会什么、差什么、怎么补、什么事件之后要重评。

**能力不是头衔的属性，是人与一项具体活动之间的相称；活动换了、语境变了，相称就要重新回答一次。**

看得懂的人找到了。可新的问题跟着来：如果这个看得懂的人，坐在作者隔壁、向同一位经理汇报呢？方工的旧事已经给过半个答案——第二双眼睛不但要亮，还要站得够远。多远，才算远？

### 3.5 第二双眼睛该站多远：三种确认

小林先给了个方案：“软件的档让硬件组来评，硬件的档让软件组来评。不同部门的人，够独立了吧。”这不外行——多数公司的“交叉评审”就是这么排的，它也确实隔开了作者本人。可是把组织图铺开看：两个组向同一位研发经理汇报，项目奖金出自同一个池子。硬件组评软件的档，评出个“不通过”，节点顺延，两个组一起挨板子。换了个部门的皮，利害还连着筋。

那把标准提到最高——请公司外面的人？外部这层皮同样靠不住：一位顾问如果拿的是项目组的合同、看的是项目组挑过的材料、报告发布要项目组点头，他站得再远，视线也被人攥在手里。

这个行业几十年的答案是：独立是一把刻度尺，量的不是组织图上的距离，是利害之间的距离。最低一档，看的人不能是做的人；往上一档，不能同一个团队、不能向同一位直属上级汇报；最高一档，评的人的管理线、预算和放行权都要在被评部门之外。对象的风险越高，要求的刻度越远。用这把尺子量，小林的交叉评审卡在第二档的门槛上——隔开了作者，没隔开共同的上级；而一个内部独立部门的评审师，只要管理、资源、考核真的断开，反而够得着最高档。

尺子有了，还差一件事：量什么。会上有人提议：“上个月过程审核的成绩很好，能不能顶一下档案的评审？”不能——因为“第二双眼睛”其实是三双，各看各的东西，各问各的问题：

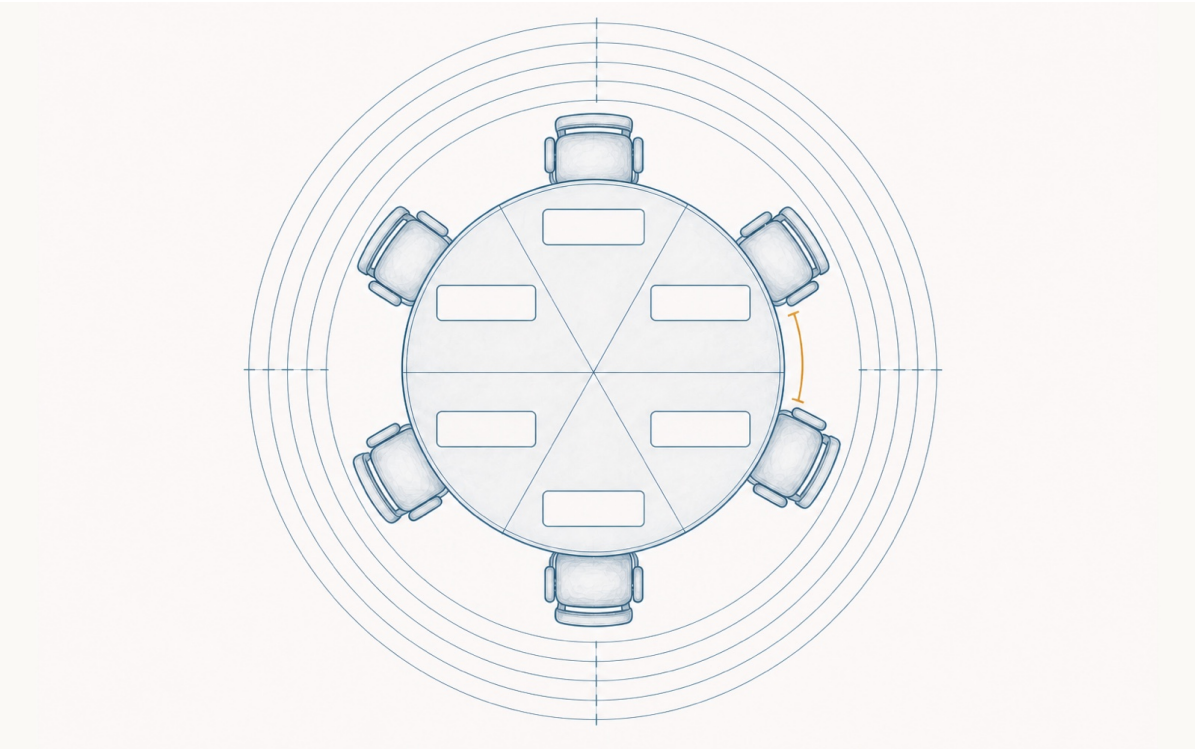
| 确认     | 看什么      | 问什么                  |
|--------|----------|----------------------|
| 确认评审   | 一份关键工作产物 | 它对安全的贡献站不站得住         |
| 安全审核   | 过程       | 说好的做法是否真的在做          |
| 功能安全评估 | 整体       | 产品、过程与论证合在一起，安全达成了没有 |

三个问题互不覆盖：过程分数再高，评不了一份具体的危害分析；一百次产物评审加起来，也拼不出“整体达成”的判断；而评估若没有前两者垫底，就只能对着空气下结论。拿一种确认顶替另一种，和拿台架绿卡冒充目标配置的证据，是同一种错误。

**独立性量的不是工位之间的距离，是利害之间的距离：评你的人，他的考核、预算和前途，不能握在你的结论手里。**

三双眼睛各就各位。可他们最终要下判断的东西，此刻散在两百个文件夹里。谁来把两百份报告，说成一句“可以相信”？

**读图任务：**先数清圆桌上的六个工作位置，再沿外圈距离尺观察角色之间的间隔，寻找那一对被标出为过近的座位。



**图 3-1** 角色数量不等于独立性。只有把关系、汇报线、利益与授权距离显式化，才能计算某次评审需要的挑战强度是否实际存在。

**图后判断：**这是管治结构的诊断图，不是人员能力评分表。它不因座位更远就证明评审更好，也不单独构成任命或授权。



### 3.6 两百份报告，为什么还不是一句“可以相信”

方工把这个问题抛给了小唐——那份主张部件清单的起草人：“假设明年评估师坐在这里，问‘凭什么说 RC17 在声明的边界内足够安全’，你递什么给他？”

小唐的第一反应是递目录：“两百多份报告，按专业分好类，每份都有签字。”方工摇头：他上一家公司的文件也全，目录也漂亮——仓库不是论证，审核员抽两份就问穿了。

“那写一份总结，把每份报告的结论抄成一页纸？”也不行。抄写不是推理：两百个“本项通过”排在一起，仍然没有回答它们**合起来**凭什么支持那句顶层的话——这份总结只是第二百零一份报告。

“那就等放行前一个月，找两个人集中装订？”更不行。到那时，三年前那份分析建立在什么前提上、前提后来变没变，经手人早已四散；补写的论证只能顺着结论倒着编理由。

三条路都堵死，安全档案的真面目才显出来。它是一场论证，从上往下三层：顶上一句主张——这个产品在声明的边界内，风险已降到可以接受；中间一层理由——为什么下面这些证据合在一起足以支持它；底下一批证据——每份都带着自己的射程，第 1 章的语言在这里原样接上。三层缺一不可：只有证据没有理由，是仓库；只有主张没有证据，是口号。还有两条纪律要钉住。其一，引用不等于采信——一份草稿可以被列进“将来的证据”，不能因为被列了就当成已被接受。其二，档案是活的——它随项目一起长大，变更一来就要重开受影响的分支。小林桌上那份软件升版申请就是眼前的例子：申请一旦批准，档案里凡是引用旧软件证据的理由都要逐条重看——这正是第 1 章那份差异分析的去处。

**证据回答“做过什么”，论证回答“凭什么”；安全档案是一场随产品一起长大的论证，不是交付前一夜装订出来的仓库。**

档案有了，三双眼睛也看过了，评估师最后说：“可以接受。”——这句话，是不是就是放行？

### 3.7 建议的尽头是决定：量产放行

会议室里多数人的直觉是：“评估都接受了，剩下的签字就是走流程。”这个直觉值得认真拆一次。评估师对他的判断负责，但他不对产品上路负责：他不掌握商务承诺，不背召回的代价，不面对停线的损失——而“接受风险”恰恰是这些东西的函数。让评估师的“接受”直通量产，等于让出主意的人替出代价的人做了决定。

第二个圈套藏在“有条件接受”里。评估的结论常常带着条件：某项措施须在某个节点前关闭。截下“接受”两个字、把条件页折进附件，是放行桌上最常见的自欺——条件不关闭，接受就不生效，这四个字要念全了才算数。



第三个圈套，方工的旧事早就演过：拿一页会议纪要当决定。“会议同意放行”和“经会议讨论接受该风险”，是同一句话的两个年头——都没有名字。

所以一张合格的放行页，薄得惊人，也重得惊人：一个名字、一个版本、一份配置清单、一个日期。签的是一句完整的话——针对这个快照，在这些证据、这些条件、这个窗口之内，我以我的职权，接受它进入量产。签字之前，证据的充分性必须已经立住；签字之后，责任并不散场——生产有生产的责任人，运行和服务有自己的看守，产品在路上跑的年头，比任何一届项目团队都长；量产后再有变更，这一页就要重签。这条从概念到报废的长链，就是“安全生命周期”五个字的分量：每一段都有自己的活儿，也有自己的名字。

小蔡听到这里说了实话：“我们以前出货，项目经理一个人说了算，也没出过大事。”这句话不需要反驳——行业不同，风险的量级、监管的密度、伤害的形态都不同。能搬走的是问法：做的、查的、收的要分开，接受风险的人要有名字、有职权。搬不走的是刻度：第 1 章那台 ENV-01 该由谁放它出厂、第二双眼睛要站多远，得在它自己的世界里重新回答——答案不必照抄汽车，问题一个也躲不掉。

**评估是建议，放行是决定；建议可以采纳，决定不能外包——签字页上必须是一个够得着风险、也躲不开风险的名字。**

至此，纸面上的责任世界已经修到了最好：文化让坏消息走得动，能力账让看的人看得懂，刻度尺让眼睛站得够远，档案把仓库变成论证，放行页把建议变成有名字的决定。可这套东西，究竟能撑到哪里？

### 3.8 纸面职责的尽头

先把敬意给足。靠任命书、职责矩阵、能力档案、独立刻度、三种确认和一页具名的放行，这个行业在没有 AI 的年代管住了横跨几十个国家的整车厂和供应商——让“谁有权说可以相信”这个问题，第一次有了可以审计的答案。这是了不起的成就。

但在工程现场多待几天，就会看见它到不了的三个地方。

**职责写在纸上，组织活在别处。**汇报关系在人事系统里，账号权限在 IT 系统里，任命在红头文件里，评审的独立性承诺在安全计划里——四处各自更新，互不通知。方工那场审核输掉的，本质是没人能回答“签这一栏的人，在签字那天，和作者是什么组织关系”。调岗、重组、代理、兼任，每一次都可能悄悄击穿一条承诺过的独立距离，而纸面不会报警——它连“今天的组织图”都拿不到。

**签名是图像，不是关系。**一个名字落在页上，绑定不了对象的版本、职权的范围、授权的有效期；核对一条签字链，靠人翻文件、打电话、凭记忆。审核员一天能问穿两份文件，问不穿两百份——那家公司的窟窿存在了三年，不是藏得深，是没人有力气逐页去问。

**机器读不出“谁说了算”。**当工具、流水线以至更聪明的助手开始替人整理证据、汇总状态，它们分不清编制与批准的分量，判不出一份“已接受”背后有没有真实职权，

更不知道自己汇总出的绿色该由谁负责——在它们眼里，签名只是又一段文本，职权只是又一个词。

所以本章末尾冻结的，不是答案，是三个必须留给后面的问题：给定一项确认活动的记录，怎样查验执行人在执行当日与作者、与责任部门的关系，确实达到了承诺的距离；给定一页签字，怎样让它可查验地绑定对象、版本、职权与范围；给定一次调岗或重组，怎样自动找出哪些承诺过的独立与授权已被击穿。第 13 章将从这三个问题出发，把本章的职责世界变成机器可以查验的结构——那是 AI 之后的世界的做法。

眼前的故事还有最后一幕。周五之前，方工把职责清单钉在了项目区的墙上：谁执行、谁出证据、谁挑战、谁评审、谁接受风险、谁授权放行，六栏都有名字，每个名字旁边写着职权的来源。用上这张清单的第一场会，是下周的概念评审会——按补课计划，团队要把 EPS 的危害分析重新做实，做成一份经得起独立挑战的东西。

会议开始不到十分钟，白板上已经画满了措施：双路信号、互相比较、封锁输出、点亮故障灯。六栏名字都坐在桌边，人人尽责，气氛热烈。只是那天上午，没有人先问一句：这些措施，回答的是什么担心？我们究竟在害怕什么损失？

第 4 章，就从这块画满答案、还没有问题的白板开始。

导读的承诺可以兑现了。合上书之前，做三件事：找出手边最近一份“已评审”的记录，写下评审人与编制人在当日的汇报关系，看这双眼睛站得够不够远；找一条写着“已接受”的风险，写出接受它的人的名字和职权来源——写不出名字的，问问它此刻由谁承担着；再用一句话写下你的项目里“谁有权放行”，拿去问三位同事。三个答案若不一致，不必沮丧——你刚刚完成了一次最小的审核。

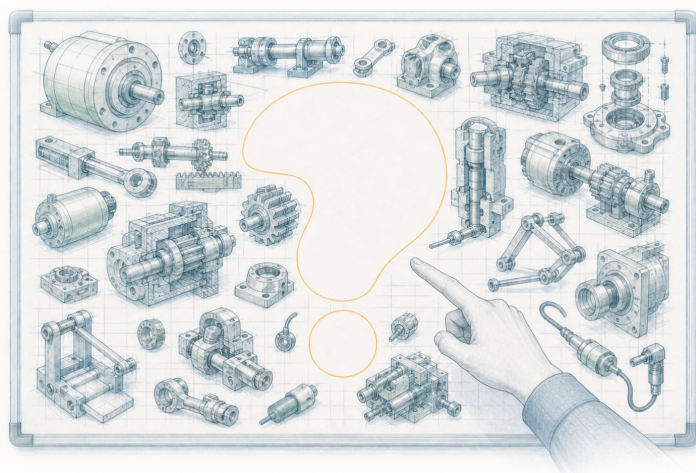
纸面可以写下谁有权说“可以相信”，却查不动此刻这句授权是否仍然成立——这是纸面职责的边界，也是第 13 章的起点。

**本章注记：**本章的人物、会议、审核事故与全部产品数字（候选代号、样机与序列号、版本号、时间与金额跨度等）均为合成教学材料，不对应任何真实企业、真实产品或真实个人。正文对功能安全标准（ISO 26262）中安全管理、安全文化、能力管理、确认措施、安全档案与量产放行等内容的转述为自然语言概括，精确要求以标准原文为准；文中的独立性刻度、能力维度与放行要素均为教学性归纳，不构成对任何组织的评价，也不构成对任何产品的分级、判定或放行结论。



## 第四章 先决定担心什么： 从使用情境到安全目标

Semantica 绑定：本章保留 HARA 方法、术语、案例与工程判断；唯一机器语义是 `semantica.chapter_packages.vol2.ch04`，主场景为 `semantica.vol2.ch04.scenario.primary`。本体、CQ、SPARQL、SHACL、正反案例、规则、`exact oracle`、PROV 与 `receipt` 全部随包发布，无书内副本。当前为 `partial`、`release blocked`。`semantica.chapter_packages.vol2.normative` 是本书的规范转述层，不是 ISO 原文；机器映射通过也不证明现实 HARA 正确。



**【导读】** 职责清单齐了，团队坐进概念评审会，白板上很快画满措施——却没有人先问“我们在担心什么”。本章沿危害分析与风险评估（HARA）这条最严格的行业纵线走完全程：先让“从失效清单直接给等级”的快捷路径完整发生，再用同一个失常换一个情境撞碎它；然后把一句混沌的担心拆成七个必须分开的问题，走过危害识别的两道网，让 S、E、C 三把尺子各自拿出证据；最后把等级落成安全目标，把未验证的假设登记在案。读完本章，你应当能对任何一行“某某失效——ASIL D”的清单，问出它缺了哪些前提。

另作一点说明：本章是前十章中唯一在正文保留标准条款与表格坐标的一章。HARA 的查表纪律，本身就是逐格对照标准原文的纪律——坐标在这里是教学内容，不是装饰；其余各章的条款坐标一律收入来源账，不进正文。

## 4.1 先画措施，为什么看起来如此合理

这一次，谁执行、谁作证、谁评审、谁有权接受风险，都已经写在职责清单上；团队拿着清单坐进了概念评审会。会议开始不到十分钟，白板上已经有了一套像样的转向方案：两路转矩信号互相比对，发现不一致就封锁功率输出，同时点亮故障灯。有人建议再加一条独立关断路径；有人开始讨论“要不要按 D 级做”。这些发言都不外行。他们相反地证明了团队对故障检测、功率驱动和降级处理很熟悉。

列席的整车工程师没有否定方案，只问了一句：“你们这套措施，精确在回答哪一件事？”

回答很快又分叉了。有人说“转向系统失效”，有人说“非预期助力”，还有人说“高速失控”。再问一步——车在哪里？什么模式？谁处在风险中？可能走到哪种后果？他们正在担心人身伤害、服务中断，还是什么别的损失？——白板上没有这些字段。

会议纪要此时是这样的：

分析对象：EPS 电动助力转向功能  
候选偏差：非预期助力转矩  
使用情境：未知  
正在评价的损失：未知  
后果分支：未知  
分级：“ASIL D?”（无依据）  
已选方案：双信号比较+关断+告警

问题不是这套方案一定错。问题是，它抢在了问题前面。还没有人说清“在什么处境里，哪个对象的哪种失常，会对谁形成什么损失”，方案已经锁定了传感器数量、故障反应和人机接口。之后再做分析，团队很容易只寻找能证明已选方案合理的那条路。

这是一个聪明人也会作出的自然判断：

从故障或失常清单出发，给每一行选一个等级和一项措施，就已经完成了“我们担心什么”与“要达到什么”的判断。

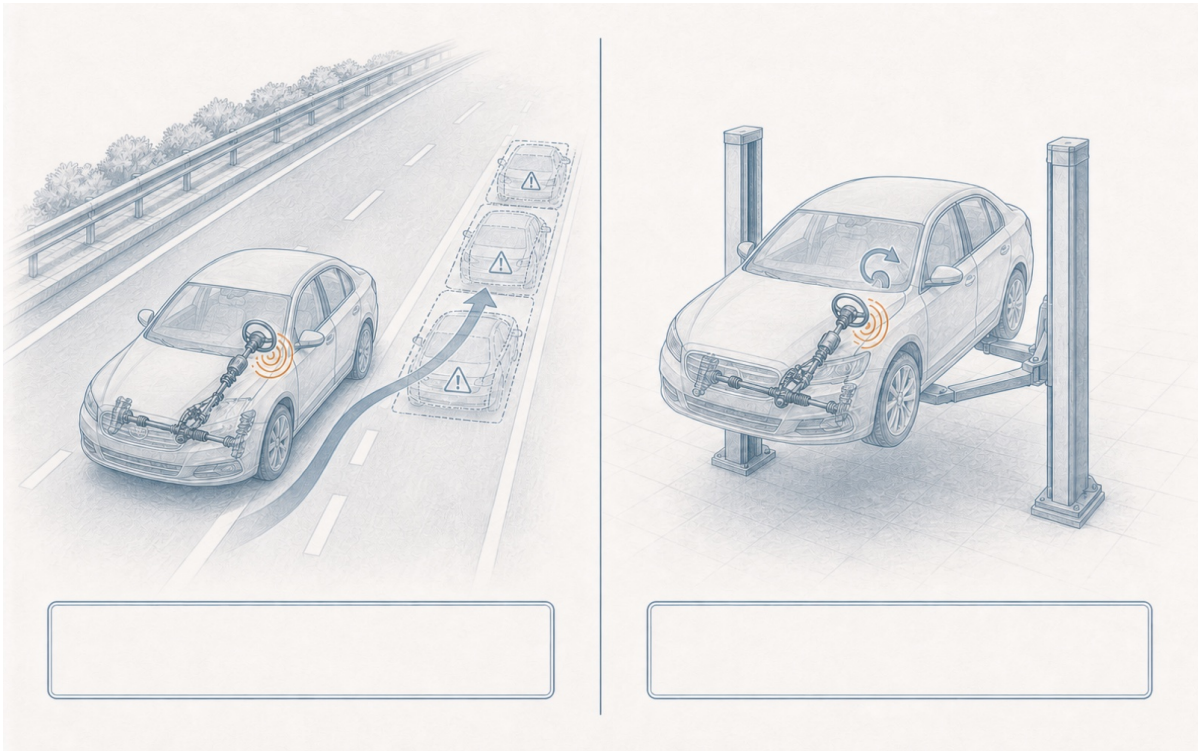
怎样才能知道这条快捷路径不够？最有效的方法不是再讲一遍正确流程，而是保持失常不变，只换它发生的处境。



## 4.2 同一失常，换个处境就不是同一件事

这一次不换车，不换 EPS，也不换失常。左右两个叙事都从“同一辆车的 EPS 输出非预期助力”开始。只把它放到两个不同的情境中：一个是车辆正在高速道路上行驶，另一个是车辆在维修工位静止，转向盘运动包络内没有人。

**读图任务：**先找出左右两幅中没有改变的车辆、EPS 与失常符号，再只比较道路高速行驶和维修工位这组情境差异。



**图 4-1** 同一对象和同一偏差进入不同情境后，场景、后果和后续评价必须重开。图中真正被替换的是车速、道路、车辆模式、附近人员和可用反应时间。

**图后判断：**本图只说明情境会改变后果链；不证明现实事故必然发生，也不给出严重度、暴露度、可控性、ASIL 或任何措施有效性。

把两条叙事并排看。分析对象没变：都是同一边界下的 EPS 功能。偏差也没变：都是非预期助力。变的是情境——车速、道路状态、车辆模式、周围人群与可用反应时间组成的条件边界。左侧可以沿“转向轮意外偏转 → 车辆横摆 → 偏离车道 → 碰撞”继续推演；右侧在本教学边界内没有人身伤害，只有意外运动与作业中断。

所以并排之后的第一个结论不是“高速更危险”——那只是表面的强度差。真正要带走的是：**失常名称不足以身份化我们正在评价的工程对象**。把“非预期助力”整行复制到另一个处境，再沿用旧后果、旧等级和旧目标，不是复用，而是把需要重新判断的东西藏了起来。

这一步留下一张情境拆分卡：

| 保持不变             | 只改变      | 必须重开           | 不得直接复制              |
|------------------|----------|----------------|---------------------|
| EPS 分析对象;非预期助力偏差 | 被选中的受控情境 | 场景、后果、关注、分级和目标 | ASIL、安全目标、实现方案和现有证据 |

这里的“只改变”是问题层的单因变式：只更换情境身份，不预先规定第 14 章要用什么名目去记录它。情境自身是一组受控条件，并不是“高速”或“维修”两个没有边界的字符串。它要说清适用对象、运行模式、环境、人群与时间范围。

为了让后续移交不靠同名字符串猜测，本章给两条核心叙事各自登记了独立身份：高速链上，“高速非预期助力”场景、“高速行驶”情境、“碰撞或偏出道路”后果各有一个受控的名字；维修链上，“工位非预期助力”场景、“工位静止”情境、“意外转动与作业中断”后果同样各有其名。机读的标识清单另行收存，不占正文。这些名字只身份化本章合成案例，不把后果宣布为现实事实。

可是，当我们说“后果要重开”时，究竟在重开什么？后果、担心、目标这几个词，评审会上常常还是同一列。

### 4.3 你到底在担心哪一种损失

把左侧高速场景写成一句：“EPS 非预期助力导致车辆偏离车道，所以要双传感器关断。”这句话看上去紧凑，其实把五件事压成了一条：处境、正在评价的场景、价值/损失视角、后果以及目标；句尾还越过目标，直接落到实现。

先不急着建正式模型，也不争唯一术语。对着当下评审任务，只做七个强制分开的提问：

| 本章工作区分           | 白话问法                                    | 在 EPS 高速线上的候<br>选回答              | 不得偷换为             |
|------------------|-----------------------------------------|----------------------------------|-------------------|
| 分析对象 (subject)   | 我们究竟在分析哪个<br>边界内的对象?                    | 整车层 EPS 功能相关<br>项                | 某个传感器故障或一<br>项措施  |
| 情境 (context)     | 哪些运行、环境、人群、<br>模式和时间条件界定<br>判断?         | 高速道路行驶、具体车<br>辆模式与参与者条件          | 无边界的“高速”标签        |
| 场景 (scenario)    | 该对象的该偏差在这<br>个情境中展开时, 正在<br>判断哪条后果分支?   | 非预期助力在高速条<br>件下导致方向控制丧<br>失的分支   | 失效清单中的一行          |
| 关注 (concern)     | 我们正在用 哪种价<br>值/损失视角评价, 对<br>谁有影响?       | 道路使用者的功能安<br>全                   | 后果、分数或“很危险”       |
| 后果 (consequence) | 在明示假设下, 这条分<br>支可能走到什么结果?               | 偏离车道、与他车碰撞<br>或冲出道路              | 已经发生的现实事故         |
| 目标 (objective)   | 为了回应该场景与关<br>注, 需要保持、限制、防<br>止、检出或恢复什么? | 防止或限制非预期助<br>力造成方向控制丧失           | 传感器数量、关断电路<br>或证据 |
| 假设 (assumption)  | 哪条尚待证成的前提<br>正在撑住场景、后果或<br>目标?          | 高速使用剖面、转矩幅<br>值/上升时间、代表性<br>参与者等 | 默认已成立的事实          |

这张表是本章的七问区分卡。它的价值不是又新增七个名词，而是让一条句子里的每个跳步都有了可以指认的位置。尤其要分开两对容易粘住的词。

第一对是关注与后果。后果是推演分支会走到的结果；关注是团队用哪一种价值或损失视角评价这个结果。“偏离车道”是后果链中的一站，“道路使用者的安全”才是当前关注。同一个现象还可以从可用性、维修性或质量视角被另行评价，但证据和决定准则会随之改变，不能把这些关注先压成一个总分。

第二对是目标与实现 (implementation)。“不让非预期助力导致方向控制丧失”说的是要达到什么；“双传感器比较后关断”说的是怎么做。前者能容纳不同方案竞争，后者是竞争中的一个候选。在 ISO HARA 这条安全线上，一部分通用目标会被投影为严格定义的安全目标；两者有映射，不是身份合并。

注意分寸：本章现在只冻结“必须分开哪些问题”，并不宣布这是一套唯一的分类办法或形式化写法。这些对象如何变成机器可查、可问答的结构，是第 14 章要回答的问题。

但在把问题交出去之前，还必须证明这些区分承受得住一条严格的行业纵线。对 EPS 来说，这条纵线就是 HARA。

## 4.4 HARA 如何把安全问题压成一条可审查的纵线

HARA 不从分级表开始，而从“我们在分析谁”开始。如果对象的功能边界、运行模式和外部依赖还在漂移，同一行“非预期助力”今天指向转向电机，明天又指向整套车辆横向控制，后面每个等级都没有稳定主语。

### 4.4.1 先给分析对象划边界

白话说，相关项定义是一份写清“这项功能做什么、有哪些模式、需要什么性能与可用性、到哪里为止、谁向它输入、谁消费它的输出、暂时借了哪些前提”的边界文件。ISO 26262-3 Clause 5 把它放在 HARA 之前。边界跟着功能因果走，不跟采购合同走：一个相关项可以跨多个供应方，同一颗 ECU 也可以承载与它无关的功能。

EPS 候选边界文件至少要给出：

- 整车功能与模式：根据驾驶员转矩与车速提供助力，并区分正常、降级与关断模式；
- 性能、质量与可用性需求：它们是识别“丢失”、“过多”、“振荡”等失常的比较基准；
- 内含元素与功能分布：转矩/角度感知、ECU、功率级、电机与减速机构分别承担什么；
- 双向接口与外部依赖：车速、供电、整车模式进来，助力转矩、诊断状态和告警请求出去；机械链、轮胎与悬架会改变后果链；
- 执行器能力边界：候选峰值转矩、上升时间和车辆响应，会同时进入严重度与可控性论证；
- 市场、法规、前代产品和同类事件的来源槽，以及明示的“待定/已规划”。

这份候选章稿只以乘用车 EPS 做纵向演示；卡客车的基本车型、配置与运营变体有 Clause 6.4.5 的专门边界，本章不用乘用车判例代替它。如果供应方实际开发的只是一个元素，也不得把元素偷写成相关项后做一份“看起来完整”的 HARA；适用路径与假设的相关项语境需另行声明。

### 4.4.2 两道网，不是一次灵感征集

边界文件交出的第一件东西是功能清单。对每个功能系统地追问“不该发生却发生、该发生却没发生、幅值过多/过少、方向相反、卡滞或振荡时会怎样”，产生功能层失常行为。HAZOP 引导词是一种顺手的工具，FMEA 类方法也可作系统识别；头脑风暴、检查表、质量履历和现场研究可以补网，但不能单独证明网外已经空了（Clause 6.4.2.2 及资料性 NOTE）。



第一道网是：

整车功能 × 可能的失常行为  
→ 整车层危害候选

“转矩传感器断线”不是整车层危害，它是内部故障候选；“无驾驶意图时出现转向助力，并改变车辆横向运动”才开始接近整车层危害。初始分析不计入相关项内部拟实现的安全机制（Clause 6.4.1.2）：不能先说“反正有转矩监控”，再用这句话把它自己的安全完整性需求降下来。相关项之外、且经证明充分独立的既有措施可以作为特定前提被考虑；独立性与适用配置必须先登记，不能用“车上反正还有别的系统”抵扣风险。

第二道网是：

整车层危害 × 运行情形（正常使用 + 可合理预见的误用）  
→ 相关危害事件  
→ 后果分支与组合效应

这里的正式对象是危害事件（Hazardous Event）：危害与运行情形的组合（ISO 26262-1:2018, 3.77）。它只在 ISO HARA 这条行业纵线中有这套精确身份，不与前节的通用场景画等号。组合之后还要写因果叙述，不能只留一个“碰撞”。如果一个失常同时带走多个功能，要评组合效应（Clause 6.4.2.6 及 NOTE）；供电失效若同时带走转矩、转向与照明，不能拆成三行各自判完。

反过来，也不得把一个运行情形为了降低 E 而切成一堆没有机理差异的子场景（Clause 6.4.2.7）。晴天和阴天若不改变非预期助力的致险机理，就不该用切分稀释暴露；干燥与湿滑会改变轮胎附着与可控性，则可以成为有证据意义的区分。

边界还有两道。第一，没有相关项失常、只由标称行为局限引发的危害，不在这条 HARA 的范围内（Clause 6.4.2.1 NOTE 2）；它需要转入适用的预期功能安全等分析路径，不能一边用 ASIL 语言回答性能局限，一边把真正的失常危害推出 HARA。第二，电击、烟雾、火灾、毒性等危害不因为出现在电子电气产品上就自动进 HARA；只有当它们由相关项的 E/E 失常引发时才进入本标准范围，否则应按组织其他程序分流（Clause 6.4.2.4）。

#### 4.4.3 把 EPS 高速事件立成一份案卷

两道网都走过以后，才得到本章的 EPS 危害分析案卷（草案）：



| 案卷槽位 | 候选内容（合成教学·草案）                                     |
|------|---------------------------------------------------|
| 相关项  | 在整车层提供电动转向助力的 EPS 相关项                             |
| 失常行为 | 无驾驶员意图时输出非预期助力转矩                                  |
| 危害   | 非预期转向作用改变车辆横向运动，产生方向控制丧失的潜在来源                     |
| 运行情形 | 目标市场高速道路行驶；车速、道路附着、交通参与者和运行模式细节待定                 |
| 危害事件 | “高速非预期助力”事件：高速行驶中 EPS 输出非预期助力                     |
| 后果分支 | 车辆横摆/偏离车道；驶入相邻或对向车道碰撞；冲出路肩；猛烈修正引发次生碰撞             |
| 涉险人群 | 本车驾乘人员、他车乘员、骑行者或行人；每类人的伤害形态与避害动作待分别评定             |
| 分析前提 | 不计入 EPS 内部拟设计的转矩监控、关断、限扭或告警机制                     |
| 假设   | 高速使用剖面、代表性驾驶人群、非预期转矩幅值/上升时间、道路附着与交通条件；均为“已规划、未验证” |

与它并排的维修工位事件保持相关项和失常行为不变，但运行情形改为“车辆静止、无人位于方向盘/转向执行链的运动包络内”，候选后果限于意外运动与当次作业中断；本章从服务连续性关注提出“在恢复作业前限制或检出意外转向运动”的通用目标。它没有 ASIL 或安全目标。这一边界是本书为了检查情境差异而建的合成条件，不是“维修环境总是无伤害”的现实结论。

案卷现在可以交给分级会。但会上的“我觉得很危险”还是太粗。严重程度、处在场景里的频繁程度、人能否避免伤害，分别是三种证据问题。

## 4.5  三把尺子回答三种不同的问题

分级会真正有价值的时刻，往往不是大家很快报出三个字母，而是三个字母第一次报得不一样。碰撞安全工程师说 S3，使用剖面工程师说高速运行占比还没有数据，人因工程师则问：“你们所谓驾驶员能救回来，假定他做什么动作、还剩多少时间？”这不是三个人意见不统一，而是一个危害事件终于被拆成了三项可以分别找证据的判断。

### 4.5.1  S：后果落在人身上，会伤到什么程度

严重度 S 评的是人身伤害，不是维修费、零件价格或品牌声誉。受险者也不只包括本车驾驶员：本车乘员、他车乘员、骑行者、行人以及事件中其他可能受伤的人，都

要按各自后果分支进入判断。样本应代表目标市场中真正会遇到该事件的人，既不能只挑最脆弱个体把等级顶到上限，也不能只挑最强壮个体把等级压低。

候选回读锚是 Part 3 Clause 6.4.3.2、Table 1 与 Annex B。S0 表示没有人身伤害；S1、S2、S3 依次覆盖轻/中度伤害、重度或危及生命但存活可期的伤害、危及生命且存活不确定或致命的伤害。Annex B 用 AIS 概率带帮助校准：S1 对 AIS 1—6、S2 对 AIS 3—6、S3 对 AIS 5—6 的参考条件都带“超过 10%”这一概率边界；较低两档还必须保留“且不满足更高严重度”的排他条款。否则同一高严重度分布会同时落入较低档，表面精确，逻辑却已经重叠。

伤害还可能组合。翻滚、二次碰撞或多个身体部位受伤时，不能只取一项单伤机械落档。若运行情形本身已经包含事故，相关项失常对伤害的增量也可能成为评价对象；这里的“可能”不能被写成所有案例都必须执行的统一减法。是评整场事故，还是评失常增加的那部分伤害，必须与因果链和适用条款一起记录。

对“高速非预期助力”事件，本章沿“高速横摆/偏离车道 → 与他车、道路边界或弱勢道路使用者发生碰撞”的代表性分支，把 S3 记为合成草案。它不是因为“转向天然最高级”，而是因为当前合成后果分支可能达到存活不确定或致命伤害。车速、碰撞形态、受险人群和伤害分布尚未由项目数据验证，所以这个 S3 只是一条待证成的判定，不是现实车型事实。

#### 4.5.2 E：车多常处在这组运行条件里

暴露概率 E 的主语是运行情形，不是故障。评“高速行驶中的非预期助力”时，E 要问目标车辆多常处在该高速运行情形中，而不是转矩传感器多久坏一次。后者属于实现与随机硬件失效的另一笔账；在概念阶段拿低故障率给 E 打折，相当于用尚未实现的设计替自己的完整性要求作证。

E 可以按处于情形的时长或情形发生频次估计，选择哪把尺子取决于致险机理。高速持续行驶适合看时间占比；只在换挡、起步或一次按需动作的瞬间暴露，则更适合数事件次数。两种口径、目标市场、车型使用剖面、数据窗口和不确定性都要同等级一起留下。装车量、选装率 and 市场份额不得用来稀释一辆已装备车辆的 E；把一个无机理差异的情形切成许多小格，也不得成为降低 E 的手段。

E0 是有依据的边界出口，不是“数据还没找到”的代号。只有运行情形可被论证为难以置信时，才可记录场景描述、排除理由与判定日期，留给后续验证复核，并走无须分配 ASIL 的路径。“未知”必须继续保持“未知”，不能为了让表格完整被填成 E0。

本章把高速情形暂列 E4，是把 Annex B 资料性时长校准中的“超过平均运行时间 10%”写成“高速使用剖面”假设（已规划、未验证），而不是引用了一份已经存在的车队统计。真实项目至少要回到代表性使用剖面、道路类型占比、行程频次和适用市场数据。来源未到位以前，这个 E4 的身份仍是合成教学材料（草案）。

4.5.3 C：危险已经显现，谁还能怎样把伤害避开

可控性 C 不是“驾驶员技术好不好”的总评。它要求把危害显现时的车辆状态、道路附着、突发程度、可用反应窗口、受险者角色和一个具体避害动作放在一起。动作可能由驾驶员完成，也可能由乘员、行人或其他参与者完成；对于卷入运动部件等非驾驶操纵型危害，还要按受险者是否能察觉、脱离或得到他人帮助来评价。

C0 表示用常规动作一般可控，并不表示“什么都不用做”；C1 表示超过 99% 的代表性人群可以避免伤害，C2 对应 90%—99%，C3 低于 90%。因此，“维持路径”“制动停车”“反打方向并制动”不是描述性附件，而是 C 判定可以被复核的主语。没有动作、窗口与代表性画像的“C2”，只有字母，没有判断。

Annex B 的判例与证据协议是资料性校准材料，不能被扩成所有项目通用的试验豁免。候选回读内容包括：C2 可用每场景 20 组有效数据全部满足判据，作为在 95% 置信水平下支持 85% 可控性的特定证据路径；C1 的极高比例可以借助专家判断；C3 因不主张可控而不需要为“能控住”举证。这些数值和路径必须连同适用前提、资料性模态一起回读；它们可以帮助项目设计证据，却不能替项目决定真实可控性。用户手册里写过处置动作，也不能自动把平均驾驶员变成受训专家。

对 EPS 高速事件，本章假定非预期转矩来得突然、幅值和上升时间足以夺取部分方向控制，平均驾驶员的反应窗口很短，因此把 C3 记为合成草案。这里至少还有四个未决：实际转矩包络、横向响应、目标市场驾驶员画像、其他交通参与者的避害空间。任一前提变化，C 都必须重开。

4.5.4 三次判断之后，才允许查 Table 4

S、E、C 是序数刻度，不能把它们随意编码后相乘。ISO 26262-3 Table 4 使用一张有限映射，公开规定 S1—S3 × E1—E4 × C1—C3 的 36 个组合如何落到 QM 或 ASIL A—D。候选全表如下；引用时应对照标准原文逐格核对。

| S × E \ C | C1 | C2 | C3 |
|-----------|----|----|----|
| S1 × E1   | QM | QM | QM |
| S1 × E2   | QM | QM | QM |
| S1 × E3   | QM | QM | A  |
| S1 × E4   | QM | A  | B  |
| S2 × E1   | QM | QM | QM |
| S2 × E2   | QM | QM | A  |
| S2 × E3   | QM | A  | B  |
| S2 × E4   | A  | B  | C  |
| S3 × E1   | QM | QM | A* |
| S3 × E2   | QM | A  | B  |
| S3 × E3   | A  | B  | C  |

| $S \times E \setminus C$ | C1 | C2 | C3 |
|--------------------------|----|----|----|
| $S3 \times E4$           | B  | C  | D  |

星号不能丢。Table 4 把  $S3 \times E1 \times C3$  的通常输出列为 A，同时指向 Clause 6.4.3.11：如果把若干本就不太可能的情形组合后，得到的暴露概率低于 E1，可以针对 S3/C3 论证 QM。原文是许可性的“可以论证”(may be argued)，不是自动降级；组合身份、独立理由、概率依据和评审决定都必须显式保留。

这张表有三个容易混淆的出口。第一，D 不是“非常危险”的修辞，它在 36 格中只对应  $S3 \times E4 \times C3$ 。第二，QM 占 18 格，是进表后的正式输出；它表示按质量管理路径处理该组合，不等于没有需求。第三，S0、E0 或 C0 根本不在这 36 格里，结论是“无须分配 ASIL”的有理由边界，不是 QM 的另一个拼法。维修工位合成事件“工位非预期助力”事件在“车辆静止且无人进入运动包络”的严格边界下暂列 S0，因而没有 ASIL；如果有人进入包络、车辆并非可靠静止或运动会传到车轮，边界改变，S0 必须撤回。

同一危害事件若有多条后果分支，应分别检查适用的 S/E/C 与结果，不能只抓“最吓人的一句”替所有分支定级。本章只用碰撞或冲出道路这条代表性后果分支演示纵线，不声称已经穷尽 EPS 高速事件的全部分支。Part 10 中关于多后果与分级应用的资料性示例，可在对照标准原文核实时作为校准参考，不能替代 Part 3 的规范判断。

由此形成分级记录（草案）：

| 对象        | S  | E     | C     | Table 4 输出 | 状态与仍缺的依据                   |
|-----------|----|-------|-------|------------|----------------------------|
| 高速非预期助力事件 | S3 | E4    | C3    | ASIL D     | 合成教学·草案；碰撞、使用剖面与人因证据均待项目验证 |
| 工位非预期助力事件 | S0 | 不进入查表 | 不进入查表 | 无须分配 ASIL  | 合成教学·草案；依赖静止与无人进入运动包络的严格边界 |

ASIL D 只是 ISO 这把行业量尺对一个教学事件的输出。它不是跨关注的“可信度总分”，不能拿去评价湿度漂移、服务可用性或供应商成熟度。等级回答了下游纪律要多严格，却仍没回答团队究竟承诺达到什么。

## 4.6 等级之后，目标仍然不能先写成方案

白板上的“双转矩信号比较 + 关断 + 告警”到这里仍然只是候选方案。即使高速事件暂列 ASIL D，也不能倒推出这三项措施就是唯一答案。等级约束开发的严格程度；安全目标先冻结必须防住或限制的功能性结果。两者都不负责替设计团队预选元件。

对本章高速线，候选的 ISO 安全目标写成：

安全目标“不发生非预期助力”（合成教学 · 草案 · ASIL D）：防止或限制 EPS 非预期助力导致车辆方向控制丧失。

它对应的通用目标对象是：

通用目标“防止非预期助力导致方向控制丧失”（候选）：在“高速非预期助力”场景中，回应“道路使用者安全”关注，防止或限制非预期助力造成的方向控制丧失。

两句可以共享工程意图，却不是同一个对象。前者属于 ISO HARA 的安全目标，承接危害事件与 ASIL；后者属于本书要交给第 14 章的通用场景—关注—目标问法。第 14 章必须把二者的关联与非等价边界说清，但本章不替它选择任何具体的记法、方向或表达形式；无论采用哪种实现，都不能因“名字相似”把安全目标等同为所有领域的目标。

Part 3 Clause 6.4.4 的候选回读锚要求 ASIL A—D 的危害事件导出至少一个安全目标。安全目标用功能表述，Clause 6.4.4.1 NOTE 还提醒它不应预设技术解决方案。目标与 ASIL 的后续配置管理和变更控制需沿 Part 8 Clause 6 处理。故障容错时间间隔、转矩物理包络或安全状态只有在确实影响分级或目标边界时才进入案卷；本章没有证据给出任何 FTTI 数值，因此 FTTI 待定，不能借一个漂亮的毫秒数制造完整感。

这条目标仍被假设撑着：高速使用剖面、非预期转矩幅值与上升时间、代表性驾驶人群、道路附着和其他参与者条件。假设不是脚注。每条都要有所有者、状态、证据计划和影响边；被验证时才可转状态，被拒绝时要沿影响边重开场景、后果、S/E/C、ASIL 或目标，不能把旧结论悄悄改成一条新“事实”。

本章也不把“写完目标”描述成验证完成。HARA 与安全目标的工作产物验证有候选锚 Part 8 Clause 9；集成相关项后对预期使用情境和安全目标进行确认，属于 Part 4 Clause 8 的确认边界。两者的对象、阶段和证据不同。这份候选章稿没有执行任何一项，也没有完成独立评审、符合性判断或放行。

因此本章交给第 5 章的不是九类策略清单，而是一份最小的 FSC 输入边界（其机读形式另行收存）：

FSC 的规范身份仍属于标准的概念阶段；这里只是把 FSC 的展开教学和方案比较移到第 5 章，没有把它改成 Part 4 工件，也没有从标准流程中删掉。



| 下游输入    | 本章交付的候选内容                             | 本章明确不决定        |
|---------|---------------------------------------|----------------|
| 安全目标与等级 | “不发生非预期助力”；ASIL D；架构、分解、分配与实现选型均为合成草案 |                |
| 目标对应的问题 | 高速非预期助力导致方向控制丧失的危害事件与后果分支             | 用哪一种机制阻断因果链    |
| 假设与边界   | 使用剖面、转矩包络、人群、道路条件均为已规划或未知             | 把未验证假设当接口承诺    |
| 物理/时间约束 | 需要由后续分析确定；FTTI 待定                     | 发明阈值、时间预算或安全状态 |
| 候选措施    | 双信号比较、关断、限扭、告警仍可进入方案竞争                | 把开场白板当已批准 FSC  |
| 验收意图    | 未来应证明目标在声明情境与假设下被满足                   | 本章声称验证、确认或授权通过 |

如何把这个边界派生成策略、功能安全需求、架构分配、接口和验证安排，是第 5 章的问题。本章此刻还有另一件事没有做完：如果这套方法真比“照着 HARA 表走”更有格局，它必须在没有 ASIL 的产品问题上仍然有用。

## 4.7 把同一问法搬到 ENV-01，什么必须留下，什么必须丢掉

现在换一个对象：一个合成的工业环境监测节点 ENV-01，候选偏差是湿度测量随时间或条件发生漂移。这里没有真实的传感器读数、产品规格、校准证书、环境箱数据、测量不确定度或现场失效记录；因此接下来的每一个实例都是合成教学材料（候选、已规划或未知），不描述任何真实设备的表现。

仍然做同一个实验：对象不变，偏差不变，只换情境。

| 槽位 | 仓储趋势监测                | 校准台检查                  |
|----|-----------------------|------------------------|
| 情境 | 仓储监测：节点用于观察仓储湿度趋势     | 校准台：节点与可追溯参考仪器进行当次校准练习 |
| 场景 | 仓储趋势观察中的漂移            | 校准练习中的漂移               |
| 后果 | 漂移可能使运行趋势被误读          | 漂移使当次校准练习无效            |
| 关注 | 测量漂移稳定性               | 测量漂移稳定性（同一关注）          |
| 目标 | 在数据进入运行判断前界定或检出漂移（候选） | 识别并拒绝无效的当次校准练习（候选）     |
| 假设 | 工作包络已声明但尚未证成（已规划）     | 可追溯参考仪器可用但尚未证成（已规划）    |

这两列证明了哪些问法可以留下：分析对象是谁、偏差是什么、在哪个情境、形成哪条场景、从哪一种关注评价、通向什么后果、由哪个目标回应、又被哪些假设支撑。它们也证明了哪些领域结论必须丢掉：HARA 的危害事件身份、S/E/C、Table 4、ASIL、安全目标、平均驾驶员和碰撞医学证据模型。把“严重度”改名成“漂移严重度”、再照搬一张 A—D 表，不是迁移，而是把没有来源的量尺偷偷带进了新领域。

ENV-01 真正需要的是另一套证据：声明的量程与工作温湿度包络、可接受漂移或稳定性判据、带时间戳的序列数据、可追溯参考仪器、校准程序、环境箱或扰动试验、测量不确定度，以及数据如何被下游决策消费。当前这些数据全部“未知”。所以目标只能说“界定或检出”和“拒绝无效练习”，不能虚构漂移阈值、精度、置信区间或寿命。

还要克制一个更隐蔽的扩张：测量漂移稳定性不等于可靠性，也不等于可用性。节点可能持续在线却持续偏移，也可能数值准确却频繁断连。三种关注可以关联，但各有失败定义、证据和目标；本章没有从漂移教学例推出 ENV-01 的真实可靠性、稳定性或在线率。

这就是 ENV-01 场景与目标记录（草案）的诚实状态：结构已经足够让团队知道下一批证据要挂到哪里，工程判断却远未完成。与 EPS 相同的是提问骨架；与 EPS 不同的是领域语义、量尺、来源和可接受性规则。

## 4.8 把问题冻结给第 14 章

先做一道未见过的迁移题。仍以 ENV-01 为对象，把偏差换成“供电中断”。情境 A 是连续趋势正在被生产人员消费，情境 B 是校准台上一次有意的重启检查。不要先写备用电源，也不要先报可用率。只问：两个情境中分别形成什么场景，损失视角是数据连续性、可恢复性还是别的关注，后果是什么，目标要维持、恢复还是拒绝什么，又有哪些假设正在撑住回答？

如果答案只是“供电中断很严重，所以加备用电源”，开场白板上的快捷路径并没有消失；它只是从汽车搬到了传感器。如果能保持对象与偏差不变，只因情境身份变化而重开后果、关注与目标，同时拒绝复制 ASIL，读者已经抓住了本章真正要保存的能力。

本章把这项能力整理成一份留给第 14 章的问题合同。它的核心问题是：

当同一对象的同一偏差出现在不同情境中时，如何识别正在被评价的场景、关注与后果，并在不预支方案的情况下写出目标？

它保留开场的自然判断，也保留两个反例：EPS 非预期助力在高速与静止维修情境中不能共享后果链或固定 ASIL；ENV-01 湿度漂移在仓储监测与校准台检查中需要不同候选目标，且两者都没有 S/E/C、ASIL 或安全目标。最小必分关系是六对：分析对象与偏差、情境与场景、关注与后果、目标与实现、通用目标与 ISO 安全目标、明示假设与既成事实。

第 14 章接到的不是“请画一张知识图谱”，而是三组必须能被精确回答的问题草案：

|      |                                                                                                                              |
|------|------------------------------------------------------------------------------------------------------------------------------|
| 问题草案 | 第 14 章必须让人或机器查清 本章不替它预设的实现什么                                                                                                 |
| 问题一  | 同一偏差在两个情境中形成哪 具体的分类办法、命名、记法与些不同场景与后果？只替换情 查询手段境身份后，哪些判断必须重开？                                                                 |
| 问题二  | 每个目标精确回应哪个场景与 把 ISO 的行业量尺当成放之四海关注？通用场景/目标如何与 的通用框架ISO 危害事件/安全目标显式映射而不等价？                                                     |
| 问题三  | EPS 投影能否返回精确 具体的查询与检查技术；任何 S/E/C/ASIL/安全目标，机器检查都不得代替现实充分 ENV-01 能否返回安全之外的 性判断目标且无 ISO 分级泄漏；某条 ENV-01 运行包络假设被否定时，哪些场景/目标必须重开？ |

验收也不是“文件能解析”这一个条件。第 14 章的候选答案至少要做到：

1. 能查出同一偏差在两个情境中形成的两个场景及各自后果；
2. 能查出每个目标精确回应的场景与关注；
3. 能显式区分通用场景/目标与 ISO HARA 危害事件/安全目标投影；
4. 能对 EPS HARA 投影返回精确 S/E/C/ASIL/安全目标结果；
5. 能对 ENV-01 返回安全之外的目标，同时精确证明其没有 ISO 分级语义；
6. 一条假设被拒绝时，能定位需重开的场景/目标，不自动改成新事实。

“没有记录”不自动等于“明确禁止”。因此第 5 项若要成为本章交付门禁，需要第 14 章先声明哪部分记录被认为已记全，再用可执行的检查证明“ENV-01 不得出现 ISO 语义”；本章只验收结果和边界，不替第 14 章预选实现。这只能证明局部包满足已编码合同，不能证明现实产品稳定，更不能授权任何工程决定。

四类答案仍明确留在本章之外：唯一的分类与形式化写法；现实 EPS 的 S/E/C/ASIL 或安全目标充分性；ENV-01 的真实漂移、准确度、稳定性或可靠性；任何方案分配、实现、验证、评审或授权决定。

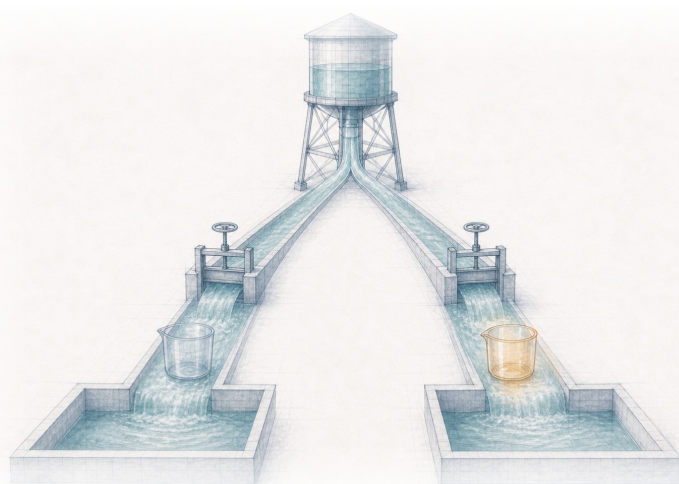
回到开场那张白板。双信号比较、关断和告警都还在，没有被否定；它们只是终于回到自己该在的位置——作为回应目标的候选实现，等待第 5 章展开。白板上新添的，

是另一条更难被擦掉的问题：当对象、偏差、情境、担心、后果、目标和假设都必须各自有身份时，怎样让团队与智能体对它们问出同一个问题、得到可复核的答案？这条问题交给第 14 章。

**本章注记：**本章的评审场景与 EPS、ENV-01 案例——包括全部情境、分级、目标与编号——均为合成教学材料，不对应任何真实企业、真实产品或真实个人；分级与目标示例不构成任何量产项目的结论。真实项目必须用自己的边界、使用剖面、伤害数据、人因研究、试验与独立评审重做判断。本章对标准条款的转述以原文为准。

## 第五章 把目标接住： 技术安全概念与系统层开发

Semantica 绑定：本章保存系统开发的推理链与教学案例；唯一机器语义是 `semantica.chapter_packages.vol2.ch05`，主场景为 `semantica.vol2.ch05.scenario.primary`。需求/架构本体、CQ、查询、Shape、案例、规则、oracle、manifest 与 receipt 均在 Semantica，本目录没有可执行替代。当前为 `partial`、release `blocked`；未支持或证据不完整的能力必须保持阻断。



**【导读】**上一章散会时，安全目标和三条候选措施留在了白板上。目标写得再好，它也只是墙上的一句话——不会自动变成电路、代码、接口和时间预算。本章陪你走完这段落地的全程：先看一句车辆层的话为什么不能直接发下去执行；再看它被翻译成两级承诺时会丢什么，承诺拼成架构时要经得起哪些反问；然后停在硬件与软件之间那道最容易没人管的接缝上，看一场低温试验怎样把一张填得整整齐齐的接口表撕开；接着立起一笔限时的预算，最后逐层把证据收回来。走到末尾，我们也会诚实地看到：这套纸面做法即使全部做对，仍有三个地方到不了。读完本章，你应当能沿着任何一条安全目标向下走到它的每一个承担者，并在每一次交接处问出：这里丢了什么。



## 5.1 挂在墙上的目标

周一早上，安全目标被打印出来，贴在项目区的墙上。最重的一条与高速工况的非预期助力有关：检出之后，必须在危害来得及形成之前让助力受控，或者转入安全状态。目标旁标着 D——风险语言里最重的一档。有一栏空着：从故障出现到危害可能形成的容忍窗口，上一章的冻结表格里写的是“待定”，因为它要靠车辆动力学和试验去挣，不能先抄一个好看的整数。白板的照片也钉在旁边：双信号比较、独立关断、驾驶员告警，三条候选措施，等着被展开。

陈工把任务交给系统工程师小唐：“分下去。”

怎么分？最省事的做法是把目标原句转发给硬件、软件和供应商，各自“落实”。这个做法几乎每个项目都试过，它败在一个安静的地方：目标是车辆层的语言。“非预期助力”对一段代码没有操作含义——软件工程师会按自己的理解设一个偏差阈值，硬件工程师会按自己的理解做一路限流，两个人都在诚实地落实，两份理解之间的差异却不会自己暴露，要等到台架或者路上才见面。

那就开个宣贯会，把目标讲透，让大家“心里都有数”？第 2 章已经埋葬过这句话：默契的覆盖半径只有一张熟人餐桌。何况下周有个新面孔要进项目——转矩传感器供应商派来的系统工程师梁工，在传感器行业干了十五年，先做应用支持后做系统，见过太多主机厂把他手册里的脚注当装饰，所以说话总带着前提：“这个值成立的条件是……”他没有参加过任何一次宣贯会，也不欠这个项目任何默契。

原句发不下去，默契靠不住，剩下的只有一条路：一次真正的翻译。可是，一句车辆层的话，要被翻译成什么，一块电路、一段代码才知道自己该干什么？

## 5.2 翻译，不是抄写

翻译分两级。第一级仍然站在功能层说话：“检出非预期助力，并在危害形成之前抑制助力或转入安全状态。”这句话故意不说用几路传感器、在哪颗芯片上比对——它规定的是功能上要做什么，行话叫**功能安全需求**。第二级才落到技术上：“控制器对两路转矩信号做合理性比对，偏差超出受控阈值时，软件请求抑制助力”；“电机驱动保留一条不依赖主控软件的电流限制通道”。这类在具体层级上给出技术承诺的话，叫**技术安全需求**。

两级不是“粗一点”和“细一点”的同一句话，因为承诺的主人换了。一条合格的技术承诺要能回答一整组问题：什么触发它，系统当时在什么模式，哪个技术主体做出什么反应，拿什么判成败。模式这一问最容易被省掉：正常行驶中检出偏差，反应是抑制输出、转入降级；初始化还没完成时检出同样的偏差，反应应当是根本不开启助力；电流越过硬件限幅时，反应要绕开软件。把三种模式压成一句“异常时采取措施”，不是简洁，是删掉了三段可以验证的内容。

翻译还要面对安全之外的要求。安全策略希望检出异常后尽快切掉助力，舒适性要求却希望转矩平滑、不突变——两个都正当，合在一起就是冲突。这种冲突必须在需求层被显式裁决并留下理由，不能揣着含糊发下去，留给标定工程师将来在车上凭手感摆平：手感摆平的东西，没有证据，也没有责任人。

还有一件事必须在翻译时想到：安全机制自己也会坏。比对在监视信号，看门狗在监视软件，限流在监视电流——那么谁在监视它们？一个坏了却长期没人知道的监视者，等于不存在，而且比不存在更危险，因为所有人都以为它还在。所以自检不是锦上添花，它的任务是把“坏了但没人知道”的时间压短。两个机制还可能打架：一个要断电，一个要维持降级供电，同时开口时听谁的——仲裁规则也得是翻译的产物，不能留给现场随机应变。

两周后，小唐把分配矩阵做完了。每条技术需求都连着上游，每条都有承担者：控制器、软件、电机驱动、梁工的传感器。覆盖率一栏是百分之百，全绿。小唐在周会上说了一句让所有人都放松下来的话：“**每条需求都有主儿了——都接住了。**”

这句话一点都不外行。追溯矩阵是这个行业的通行做法，小唐做得比多数项目认真：逐条连线，逐条核对，没有一行悬空。如果这样还不算接住，那问题一定藏在矩阵照不到的地方。

有人提议再加一列“验证方法”，逐条补齐。值得做，但它补的仍是每一行自己的完整性——矩阵审查的单位是行，而丢失恰恰可以发生在行与行之间：拆分时漏掉的东西不属于任何一行，加多少列都照不见它。又有人提议让各承担方逐条签收。也值得做，但每一方认下的是“我认领的这条我能做”，没有人被要求担保“我们认领的这些条，加起来等于上游那句话”。

撞破它的是梁工。需求评审会上，他第一次到场，听完介绍只问了一个问题：“我的传感器把故障报出去，要多快，才算够快？”

小唐沿着矩阵找。软件那条写着“检出偏差后请求抑制助力”，在；硬件那条写着“独立电流限制”，在；传感器那条写着“故障时置诊断位”，也在。可上游那句“在危害形成之前”——整个目标里唯一的时间条件——拆分之后不在任何一条里。三个承担者每人领走了一个动作，没有人领走“多快”。矩阵全绿，时间蒸发了。

**派生不是把一句话越抄越细，而是让承诺换主人；目标不会在换手中自动守恒——每一次交接都要重新清点：触发、模式、动作、判据、时限，少一样，就丢一样。**

顺手多问一层：五样东西里，为什么偏偏是时限最常蒸发？因为拆分是沿着组织的分工线切的。触发、模式、动作、判据，每一样都能落进某一家的地盘，认领起来天经地义；“多快”却横跨传感器、软件、电机驱动三家——谁认领它，谁就得替别人的时延担保。按行认领的制度只给“属于一家的东西”发座位，于是全链共有的那个条件，成了唯一没有座位的那个。这不是哪个人的疏忽，是逐行分包这种形式自带的盲区。

顺带把等级说清楚：目标旁那个 D，会沿着派生链一路跟下来。风险等级表达的是目标被违背的后果有多重，派生只是把同一份责任拆开承包——文档变厚，不会让

风险变轻。想让某一段真正降级，唯一的合法通道是拿出真正独立的冗余并证明这份独立，那是第 8 章的正题；在此之前，任何悄悄调低的等级都是断链，不是优化。

现在，每条技术承诺都有了承担者，时限也补进了句子。可承担者们背对背各干各的，各自正确，拼在一起就自动是一个系统吗？

### 5.3 架构要经得起反问

架构评审会上，一位新调来的工程师展示了一版更干净的框图：概念阶段的双路转矩传感器，被改成了单路传感器加软件校验。方框少了一个，成本低了一截，线条也顺眼多了。

“图更简洁”能不能作为接受理由？不能——简洁描述的是画法，不是论证。那“老方案评审时大家都没意见”呢？也不能——“没意见”审的往往是图有没有画错，而不是这样拼为什么就够了。把这两个理由排除掉，真正的问题才露出来：候选措施里的“双信号比较”，依赖的是两个独立的观察；改成单路之后，变成了同一份数据被软件看两次。这不是版式优化，是把一项安全策略改没了。

评审会顺着这个口子往下挖，挖出了一个更隐蔽的问题：现有的两路信号共用一个参考电压。单路漂移时，两路读数拉开差距，比对能抓住；可如果参考电压漂了，两路读数同向变化，差值依然很小，比对会保持沉默——恰好在两路同时说谎的时候。“有两路”只是库存，“两路不共享一个能同时欺骗它们的东西”才是冗余。

评审会还处理了另一个方向的直觉：那条硬件电流限制电路在上一代平台上用过三代产品，有人主张“老设计，免检”。用过三代确实意味着它携带的未知毛病更少——这是复用真正的价值；但信任历史不能替新边界体检：供电电压不同、温度范围不同、失效模式的表现也可能不同。评审会最后既没有删掉它，也没有盖章放行，而是把它保留为候选，附上一项适用性核对任务——复用改变的是未知量的起点，不是证据义务的终点。

这场分析的产物不是一份报告，而是三处设计改变：参考电压要么独立供给、要么被单独监控；软件校验之外保留那条硬件电流限制，作为软件失控时的兜底；控制器与电机之间预留能观测命令值、实际电流和故障状态的测试点——因为等到集成时才说“需要注入故障”，就只能拆台架补设计了。可测性不是测试团队的愿望清单，它是架构自己的属性。

到这里可以把一个容易被改名蒙混的概念说死。一摞技术需求，加一张框图，再加一份“为什么这样拼就足以兑现这些需求”的理由——三样合在一起，才叫**技术安全概念**。把需求清单的文件名改成“概念”，不会让内容多出一克。而那份理由必须诚实：说得当前证据支持到哪一步，也说得出还不支持什么。“分析还没做，所以只能声称结构齐了”不丢人；用形容词把没做的事盖过去才丢人。

架构不是需求的收纳盒，而是一份必须经得起反问的论证——“为什么这样拼就够了”。答不上反问的框图，只是画得整齐的愿望。

架构把活切给了硬件和软件，两边各自开工。可切缝正上方那道线——硬件交出的信号和软件读走的数据之间——归谁？

## 5.4 接缝上的合同

接口表有两百多行：信号名、方向、量程、单位、更新周期，填得整整齐齐。软件负责人小林在例会上说：“接口表都填满了，两边照表干活就行。”这句话同样不外行——信号表是每个项目都在用的工具，能把两百多行填满核对完，已经好过一半项目。

冬天来的时候，这张表挨了一记重的。

低温环境试验，零下三十度，台架上的助力时断时续，诊断反复报“两路转矩信号不一致”。排查用了三天：小林的团队先查自己的滤波和去抖，硬件查线束和接插件，都没有问题。最后是示波器给出了答案——低温下，一路信号的更新周期明显变长了。翻开梁工的手册，“更新周期 2 毫秒”旁边有个脚注：典型值；低温补偿运行时，最坏可达数倍。为什么补偿一开，更新就慢？道理不玄：越冷，芯片里的原始信号漂得越厉害、毛刺也越多，传感器得先量自己的温度，按温度把读数修一遍，再多采几次做平均把毛刺压下去，才敢把这个数发出去——每个数出门之前多干了几道活，更新周期自然被拉长。而“2 毫秒”是常温下量出来的：常温里这些活几乎不用干，数是顺着直道跑出来的。接口表誊抄时留下了“2”，丢掉了“典型”两个字和那行条件。方向盘上的转矩一变，正常那路已经跟上，慢的这路还停在上一个值，两路的瞬时差被拉开；软件按 2 毫秒设计的比对窗口没给这种滞后留位置，把一个只是更新慢了的旧值，当成了两路分歧，于是助力被尽职尽责地反复抑制。

没有人说谎：手册是真的，表是认真填的，代码是照表写的。代价却是真的——当季的低温试验窗口作废，重新排队五周；接口表两百多行逐行补“条件”和“最坏值”两列，重新会签；梁工和小林各自返工。三份局部正确的工作，又一次拼出了一个全局错误。

事后的补救提议也熟悉。有人说把两家文档合订成册，互相可见——可见不等于承诺：手册是供应商对器件的描述，不是对这个项目的担保，合订之后脚注照样可以被忽略。有人说以后对接会逐行过表——这次事故之前恰恰开过对接会，逐行过的就是那张表；表的格式里根本没有“什么条件下成立”“最坏什么样”“坏了怎么表现”的位置，把一张缺列的表过得再认真，也只是把缺陷核对得更整齐。

真正缺的是一份属于接缝自己的合同——把硬件与软件的交互写在同一份受控文件上，行话叫**硬件-软件接口**。它比信号表多出的，正是那几列救命的内容：这个值什么时候开始可信（初始化阶段可能送出格式正确、语义还没生效的数）；最坏什么样（最坏更新、最坏延迟，而不是典型值）；坏了怎么说（诊断位最晚什么时候置位、保持多久、谁必须在多长时间里读走它）；共享的资源谁来仲裁。这份合同要在两边开工之前签——事后从代码和原理图里倒推出来的接口文件，是对现状的辩护，不是对双方的



约束。签了之后它也不能死掉：传感器换代、轮询放宽，任何一方的“局部小改”都是合同变更，不许被描述成“接口没变”。

接缝不属于任何一方，所以必须有自己的合同；合同的要害不是“交什么”，而是“什么条件下成立、最坏什么样、坏了怎么说”。两份各自正确的文档，拼不出一个正确的中间。

合同里写满了毫秒：更新周期、置位时限、读走窗口。这些数字不是各自随手填的——它们从哪笔总账里分出来？

## 5.5 一笔限时的预算

故障发生的那一瞬间，危害通常还没有出现。电机转矩要建立，车辆的响应要发展，驾驶员的手还来得及做一点校正——从故障出现到危害真正可能形成，中间有一个窗口。要记住这个窗口属于谁：它属于危害和整车情境，由车辆动力学和使用场景决定，描述的是“如果没有任何机制出手，最坏多快会出事”。它不属于任何一个安全机制。

机制有自己的钟：从故障发生到被检出是一段，从检出到完成反应、进入安全状态又是一段，两段加起来，必须小于那个窗口。

面对这道题，第一种自然做法是“把机制做快点，多留余量”。快不是免费的：更快的检测意味着更短的去抖和判定，也就意味着更多误报——低温那三天的教训，起点正是一个设得过紧的窗口。余量也不是态度词，它得说得出来是多少毫秒、相对哪一个最坏值。第二种做法更隐蔽：“我们的机制全程需要三百毫秒，那就把窗口定成三百五。”这是让选手自己挪终点线——窗口由危害动力学决定，不由机制的方便决定，倒推出来的窗口什么也保护不了。

账要怎么算，用一组纯演示的数字走一遍（它们不是这套转向系统的参数）：设窗口两百毫秒。诊断每二十毫秒跑一次，连续三次命中才判定故障，那么最坏情况下——故障刚好发生在一次诊断结束之后——检出要六十毫秒；反应最坏九十毫秒；合计一百五十，还剩五十毫秒余量。现在只改一件事：诊断周期放宽到四十毫秒。三次命中让最坏检出变成一百二十毫秒，合计二百一十——其他什么都没动，全局已经超时。传感器的更新、软件的轮询、去抖的次数、执行的动作，每一段都在花同一笔钱；谁多花一毫秒，别人就少一毫秒，而且往往没人通知。

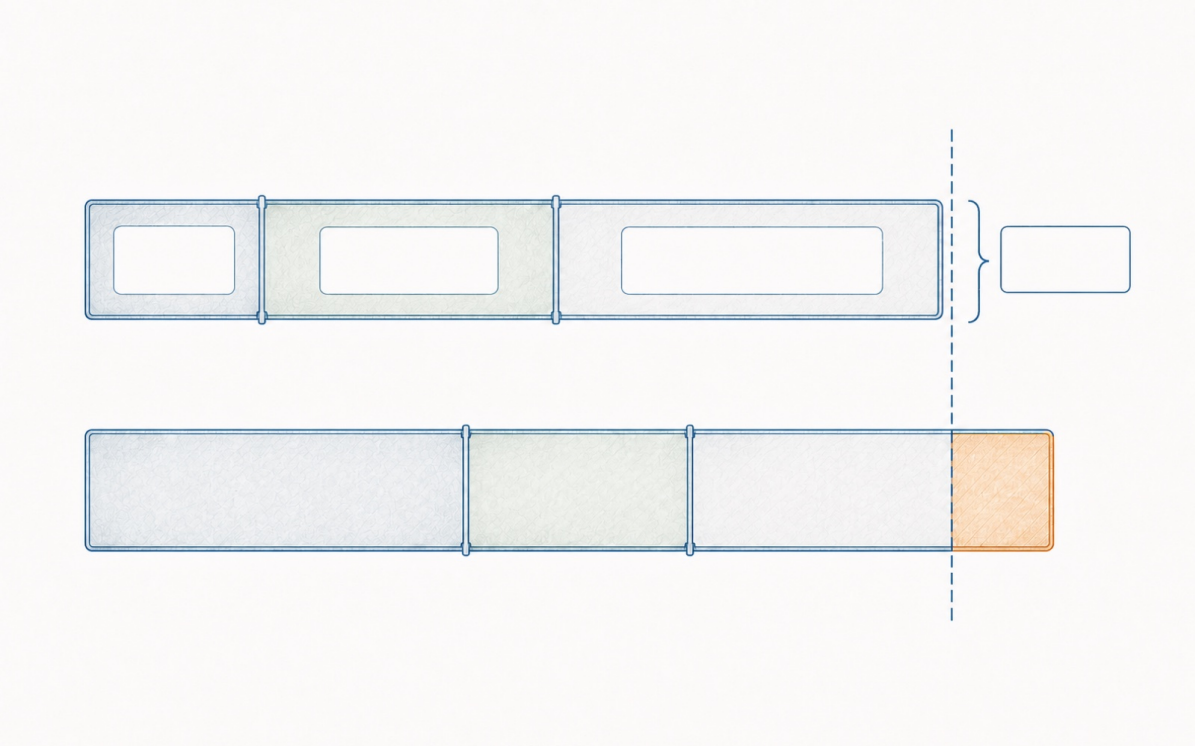
回到墙上那张表：这套系统自己的容忍窗口，此刻仍然写着“待定”。这不是拖延，是诚实——数字要靠车辆动力学分析和试验去挣，抄来的整数只会让后面所有推理建立在一个没有来源的地基上。但账本必须现在就立好：窗口一旦定值，它将被分给谁、每一段的最坏值由谁承诺、写进哪份合同——这些格子先画出来，数字到位的那天才有处可记。等数字来了再立账本，各段时延早已各自为政。

窗口属于危害，不属于机制；时间预算只能从窗口向下分，不能由机制的能力向上凑——而一笔没有账本的预算，谁都可以多花。



需求、架构、合同、预算，都立起来了。可它们还都是纸上的承诺——拿什么证明这些承诺真的成立？

**读图任务：**对比上下两条总时间窗口，先看检测、反应和余量如何分配，再看检测延长后哪一段被挤出右边界。



**图 5-1** 系统反应时间是一笔有上限的预算，不是多个团队各自声称足够快就能成立。任一环节超支，都会挤压后续执行和安全余量。

**图后判断：**图中的段长是概念性预算，不是某个产品的实测数据。它不单独证明诊断覆盖、反应路径或 FTTI/FHTI 已满足。

### 5.6 逐层收账

收账的地方在郑工的台架。但集成不是把东西装上车跑一圈，它是三层楼，每层收的账不一样。

第一层，在元素内部把硬件和软件合到一起，收的是合同的账：诊断位真的在说定的时间内置位了吗？软件真的在承诺的窗口里读走了吗？低温下的最坏更新真的落在补签之后的那一格里吗？第二层，把元素合成整套系统，收的是机制之间的账：比对、限流、告警同时工作时互相打不打架？一个要降级、一个要保持时，仲裁规则真的裁得

动吗？第三层，装进真车，收的是整车的账：故障注入之后，车辆的轨迹、驾驶员手上的力、与整车电源和通信的相处，是不是都落在判据之内？

同一个故障刺激可以在三层都出现，但问题和判据必须跟着被测对象换。把同一份测试报告换三个标题，不是三层，是一层刷了三遍漆。郑工还有一条用教训换来的规矩：下层没关闭的问题，必须显式写进上层的测试清单——那张“初始化时诊断位偶发误置位”的便签，要么在系统层被验证不会误触发降级，要么继续往上走，绝不允许躺在下层报告的最后一页里自然死亡。

账收到第三层，还剩一个容易被合并掉的问题。集成测试问的是“我们把东西做对了吗”——每条写下的需求都兑现了吗。可还有另一个问题：“我们做的是对的东西吗”——当初那条安全目标，在真实的整车上真的站得住吗？回答它的活动叫**安全确认**，它有自己的问题清单：这辆样车凭什么代表将来卖出去的那一类车？驾驶员真的控制得住吗——包括可以合理预见的误用？当初危害分析里那些只有到整车上才能检验的假设，逐条评过了吗？“整车上跑过了”回答不了这些：一辆车不自动等于代表性，跑过不自动等于按判据收账。想把三层集成和安全确认合并省时间的人，省掉的正是问题本身——四套问题，四份答案，谁也替不了谁。

**收账要逐层收，结论不许跳级：每一层的绿灯只在自己那层作数——楼可以一层层上，但没有电梯。**

秋天，账一层层收着，墙上的目标看起来正在稳稳落地。直到梁工的一封邮件到来。

## 5.7 一张全绿的矩阵，答不上一个问题

邮件写得很客气：传感器将在下一代改版，制程更替，内部补偿和更新机制都会变；老款两年后停产，建议项目排查影响。

陈工把邮件转给小唐，只问了一句：“哪些东西要重开？”

小唐有矩阵，有接口合同，有时间账本——这一章立起来的全部家当。他坐下来算，一个小时候发现自己算不动。第一，矩阵是人手维护的：上季度更新之后，架构改过两处，接口表补签过一轮，矩阵还不知道——它记录的是上次有人有空时的世界。第二，矩阵记录的是“这条和那条有关”，不记录“为什么有关、依赖了它的什么”：比对窗口当年为什么定在那个值？依据的是不是老款传感器的最坏刷新？这些理由不在矩阵里，只在当年做决定的人的脑子里——而其中一位已经离职。第三，“哪些要重开”没有任何机械的办法算出来。最后的解法是把所有相关团队拉进会议室，投影矩阵，逐行认领，开了两天。散会时清单列了四十多项，没有一个人敢担保它是完整的。

先把公道话说完：这一章走下来的每一步——派生时清点五要素、架构接受反问、接缝签合同、时间立账本、证据逐层收——都是这个行业用真实事故一条条炼出来的最佳实践。认真做到这些，已经好过绝大多数团队；这也是没有 AI 的年代，人类工程共同体能做到的最好水平。

但两天会议暴露的三件事，是这套纸面做法自身的边界，再认真也补不上。**其一，追溯靠人手维护，账本的更新永远慢于事实**——矩阵描述的是过去某一天，而变更发生在今天。**其二，派生的理由不留痕**——纸面记得住“有关”，记不住“为什么有关”；可变更影响要问的恰恰是“为什么”：不知道当年为什么这样拆，就不知道今天要不要重拆。**其三，影响算不出来**——改一条上游，没有任何办法可靠地列出下游哪些承诺、哪些判据、哪些试验必须重开，只能靠会议室、记忆和运气。这不是团队不够认真，是纸面这种载体记不住这些东西。这三个问题，本章冻结在这里，留给本书的后半部分——到那里，追溯将不再是一张表，而是一种可以被查问的结构。

会议散场前，另一份清单已经发了出去：系统层给硬件的需求单。上面除了功能承诺，还有一串数字——每条安全机制被期望达到的失效率量级、诊断被期望覆盖的比例、留给元器件的那份预算。硬件的同事把单子看了两遍，在回复邮件里问了一个问题，这个问题要用下一章一整章来回答：

“这串数字，从哪里来？到时候，又拿什么证明我们达到了？”

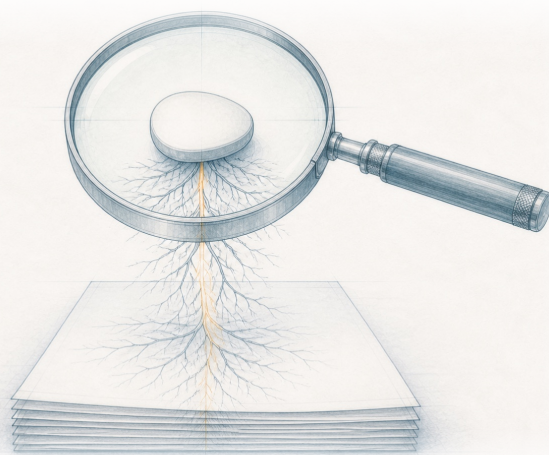
导读里的承诺，现在可以兑现了。合上书之前，做三件事。第一，挑出你产品里最高的一条目标，沿着分配一路走到最底层，在每一次交接处数一数：触发、模式、动作、判据、时限，五样还剩几样——数字每少一次，你就找到了一次丢失。第二，翻开一张接口表，随便挑一个数字，问它是典型值还是最坏值、什么条件下成立——如果表里没有地方写答案，你已经找到了自己的接缝。第三，假装收到一封元件换型邮件，手工列出下游重开清单，给自己计时，然后问：敢不敢担保没漏？做完第三件事的那种不踏实，请记住它——本书的后半部分，就从那里开始。

**本章注记：**本章的人物、会议、事故经过与全部数字（信号周期、温度、时间演算、排期、行数等）均为合成教学材料，不对应任何真实企业、真实产品或真实个人；文中的时间演算仅用于说明预算分摊的一般规律，不是任何实际系统的参数。正文对功能安全标准内容的转述为自然语言概括，精确表述以标准原文为准；本章不构成对任何实际系统的需求、架构、接口或测试结论。



## 第六章 数字要能作证： 硬件层开发与硬件度量

Semantica 绑定：本章保存硬件度量的概念、算例与证据纪律；唯一机器语义是 `semantica.chapter_packages.vol2.ch06`，主场景为 `semantica.vol2.ch06.scenario.primary`。本体、CQ、查询、Shape、正反案例、规则、oracle、manifest、PROV 与 receipt 只在 Semantica。当前为 `partial`、release `blocked`；算例 oracle 通过不构成 FMEDA 充分性、真实硬件指标或放行结论。



**【导读】**第 5 章散场时，系统层把需求和预算分了下去：软件领走一叠要实现的承诺，硬件领到的却是一串数字——两个百分数、一个概率上限，外加“必须达标”四个字。本章陪硬件团队把这串数字走一个来回：先弄清数字背后的需求从哪里来；再看“失效率”这个数是谁的、按什么条件成立；然后立起一笔五个去向的账，亲手把两个度量算出来，再算一次“加一个监控器能好多少”；最后在一场审核留下的旧伤疤前停下来，看清一个没有来源的数字能造成什么。读完本章，你应当能对任何一个写在报告里的数字问出五个问题——对象是谁、方法是什么、单位是什么、不确定度多大、来源在哪——并判断它此刻是站在证据的位置上，还是只是恰好站在那个位置上。



## 6.1 预算到手的那个下午

小唐抱着一摞文件走进硬件组，是个周二下午。系统层的分配工作收了尾，分给硬件的那一份摆上了吴工的桌面：一叠分配给硬件的技术安全需求，以及一页度量预算——针对“不产生非预期助力”这条最高等级的安全目标，单点故障度量 SPFM 不低于百分之九十九，潜伏故障度量 LFM 不低于百分之九十，随机硬件失效导致安全目标违背的平均概率 PMHF 不超过每运行小时亿分之一的量级。

小唐把预算页抽出来，半开玩笑半认真地说：“说实话，我挺羡慕你们硬件的。系统层为‘需求接没接住’吵了三个月，全是判断题；你们这儿多干净——三个门限写得清清楚楚，算出来，过线，签字。数字不会骗人。”

这句话一点都不外行。把验收从“我觉得可以”变成客观的数值比较，本来就是几代工程管理努力的方向；数字确实比形容词可审得多，一个百分数至少可以被复算，而“充分”“足够”连复算的入口都没有。如果这个行业把随机失效的管理压缩成三个门限，那一定是因为门限背后有一整套撑得起它的方法——问题只在于，拿到门限的人还记不得那套方法。

硬件负责人吴工没有接话。他干了二十多年硬件，从画电源板起家，上一代转向平台的控制器就是在他手里从原理图走到量产的。他有个全组皆知的怪癖：任何表格必须有一列“出处”，没有出处的数字不许过夜。新人听过这个怪癖的好几种来历版本，真实的那个版本，本章后面会讲到。

那天下午，吴工只是把预算页翻过来，在背面竖着写了五个词：对象？方法？单位？不确定度？出处？然后对小唐说：“数字不会骗人，但数字也不会自己作证。要让 99% 这个门限有意义，得先知道硬件被要求做到什么——不是‘达标’，是需求。需求从哪里来？”

## 6.2 机制要有上游：硬件安全需求怎么写

硬件层的开发不是“做一块达标的板子”。它同时欠着三笔账：把技术安全概念里分给硬件的承诺实现出来；分析随机硬件故障和它们的影响；和软件把接口协调清楚。三笔账在转向系统上正好拧在一起——转矩通路上的诊断状态由硬件产生，软件要正确读到并作出反应，单独优化任何一端，都证明不了整条安全路径成立。

第一笔账落到纸上，就是硬件安全需求：从分配给硬件的技术安全需求往下派生，把硬件要守的承诺写成可验证的句子。它们大致管五类事：控制硬件内部的失效（时钟异常时，看门狗要在规定时间内动作）；控制或容忍来自外部的失效（传感器输入开路时，输入级要进入可诊断的状态）；满足其他元素托付过来的需求（驱动级要向软件提供可用的故障状态）；把内外部失效检出并通报出去，而不是悄悄遮蔽；以及不属于机制的约束——随机失效的目标值、禁止的行为、连接器和线束上的设计措施。

写需求看起来是文书工作，跳过它的诱惑很实在。一种想法是靠经验：“资深工程师看一眼架构，该加的监控自然会加。”这话对了一半——经验确实是机制的重要来源，但经验没法被审：新人看不出哪个监控是必须的，供应商猜不到你默认了哪些保护，下个月来的审核员更没法核对“工程师觉得应该有”。另一种想法是抄成熟平台：“上一代的机制清单搬过来，错不了。”也对了一半——成熟平台的机制清单是真金白银换来的，但它回答的是上一个产品的问题；架构换了、需求换了、分配换了，抄来的机制可能守着一扇已经不存在的门，而新开的门没人守。两条路的共同缺陷是一样的：机制和需求之间断了线，出了事没人说得清“这个监控是为哪条承诺存在的”，没出事也没人说得清“哪条承诺还没有机制”。

**安全机制不是设计师的善意，而是需求的债务：失效分析表里每一行“机制”，都必须能指出一条要求它存在的需求；反过来，每一条分给硬件的承诺，都必须能指出谁在替它站岗。**

这条纪律还有两处容易记错的边界。其一，机制不必是纯硬件——硬件出诊断信号、软件做反应，是最常见的组合；正因为如此，硬件和软件负责人要共同核定接口清单：硬件给出的诊断位，软件真的会去读吗？读到之后来得及反应吗？一个覆盖率写着90%的机制，如果软件根本没订阅它的状态位，覆盖率就只是表格里的装饰。其二，需求追溯不必挂到每颗电阻——承诺追到最低层的硬件组件就够了；但失效分析可以继续往下挖到单个元器件和它的失效模式。这是两笔不同的账：追溯回答“谁实现这条承诺”，分析回答“物理世界怎样破坏这条承诺”。把分析粒度误当追溯义务，会造出几千行没人维护的假精确矩阵；把追溯下界误当分析下界，又会漏掉真正危险的那颗小器件。

需求有了，机制有了户口，表格的下一列轮到失效率——每个器件、每条路径，一年到头到底坏得多勤？这个数从哪里来？最顺手的答案是：查手册。

### 6.3 手册上的失效率，是谁的失效率

小蔡最近被派到硬件组轮岗。填失效分析表填到电阻那几行时，她觉得这是全表最轻松的部分：“这个好办，手册上查得到。手册说这类电阻零点几个 FIT，那 R17 就是零点几个 FIT，填进去就行了。”

先说单位。FIT 是失效率的行业单位：1 FIT 表示每十亿器件小时一次失效——它是速率，不是“十亿小时后必坏”的寿命承诺。小蔡的话也一点不外行：工业界公认的手册数据本来就是失效率的正规来源之一，查手册正是正确动作的第一步。她只是不知道，她随手举的这个例子，恰好是全公司最有名的一颗电阻。

R17——就是几个月前那场乌龙的主角，一个和候选代号只差一个字母的位号，让七个人开了四十分钟紧急会。那次虚惊的正文只有一句话：这颗电阻的某个来料批次，失效率略超门限，按正常流程处置。现在把这句话放到手册数字旁边，问题自己站了出来：手册说这类电阻零点几个 FIT，那个批次却实实在在越了线——手册错了吗？没

有。手册的数字是一个关于“这类器件、在声明的条件下、按声明的统计口径”的总体陈述，它从来没有听说过你的这个批次、你的这块板、你的发动机舱。

那怎么办？会议室里的自然对策照例有几条，值得逐一试过。

第一条：换一本更保守的手册，取更大的数，“保守总不会错”。错。首先，保守的代价是真实的——失效率抄大一倍，预算立刻吃紧，逼着你在别处加机制、加成本，而这些机制自己也会带来新的失效率；其次，对于按比例计算的度量，把所有数一律放大并不自动保守：分子分母一起变大，比值未必往安全的方向走。保守是一种需要逐项论证的判断，不是一种可以批发的美德。

第二条：不用手册，用现场统计，“真实数据总比手册强”。方向是对的，门槛常被略过：现场统计要成为证据，得先证明可比——统计对象和你的器件是不是同一类、同一工艺？运行条件相当吗？样本量撑得起结论吗？一个新设计根本还没有现场，上一代产品的现场数据用过来，中间隔着的每一处差异都要说明。

第三条：让老师傅拍一个，“他看了三十年器件”。经验可以进账，但要先立判据后出数：估计的依据是什么、和哪些已知数据对过、不确定的范围多大。先有数再找理由的专家判断，不是判断，是背书。

三条路都通向同一个要求：一个失效率要能用，光有数值不够，它得随身带着自己的身份——对象（哪类器件、哪个工艺、哪个批次口径）、条件（什么温度、什么应力、什么任务剖面）、时间基准（按运行小时还是日历小时——一辆车一年运行几百小时，日历却走八千七百多小时，两种口径差着一个数量级，不换算就相加是把米和英尺加在一起）、方法（手册、现场统计还是专家判断）、以及不确定度（这个数附近有多宽的雾）。还有一条最容易忘的：数字的前提在设计这边也要守住——手册数字假定器件在额定范围内工作，如果你的设计让那颗电容长年超温，手册抄得一字不错，前提已经死了。

**失效率不是器件的属性，是一句主张：“某类对象，在声明的条件与时间基准下，按某种方法统计，失效速率是多少、不确定多少。”**把数抄走、把这句话留下，数据就退化成了装饰。

小蔡在表格里给 R17 填上了数，也照吴工的规矩在“出处”一栏写上了手册版本和适用条件。然后她发现了下一件怪事：表里那段没有监控的供电路径，失效率只有 2 FIT，全表最小，却被吴工标成了最扎眼的红色；旁边失效率大它几十倍的行，反而安然无事。同样是 FIT，为什么去向差这么多？

## 6.4 同样 2 FIT，去向为什么不同

因为度量在乎的不是数字的大小，而是失效的去向。对着一个确定的安全目标，每一份失效率都要被问三个问题，然后判进五个去向之一。

第一问：它单独发生，会不会直接把安全目标顶穿？那段无监控供电路径的答案是会——它一旦失效，控制器可能在没有有效控制的情况下输出非预期助力，而且没有

任何机制拦它。这样的份额叫**单点故障**贡献：数值再小，性质最重。如果一个元素有机制罩着、但机制罩不住的那部分仍能单独顶穿目标，漏网的份额叫**残余故障**。

第二问：它需要和另一个独立故障凑在一起才能顶穿吗？凑对才危险的，进多点故障的账——再按第三问分成两半：在规定窗口内能被检出（或被驾驶员察觉）的，叫**可检出的多点故障**；在窗口内始终无声无息的，叫**潜伏的多点故障**。潜伏是危险的蓄水池：第一重故障躺在暗处，系统看起来一切正常，直到第二重故障到来。

第三问都不沾的——发生了也不显著增加目标被违背的概率——才叫**安全故障**。

小蔡立刻想到了那个最顺手的修法：“给供电路径加个监控不就行了？被监控了，就是安全故障了。”这个直觉几乎人人有过，它跳过了至少四个问题：监控器盖得住这条路径的哪些失效模式，盖不住的剩几成？检出之后，谁去控制后果——来得及吗？“来得及”用什么衡量？监控器自己坏了怎么办？四个问题没答之前，“有监控”什么也不改变。

另一个顺手的错法是拿覆盖率当判决：“这机制覆盖率 90%，所以 90% 都安全了。”覆盖率只回答“机制对这组失效模式盖住了多少”，盖住的那部分去哪个账——是安全、是可检出多点、还是别处——要看后果是否受控、组合关系是否成立、时间是否来得及，覆盖率一个数决定不了。

时间是这里最硬的裁判，而且是两只不同的钟。第一只钟属于整车：从故障发生到危险可能出现，中间有多长的窗口留给检测加反应？这个窗口由车辆动力学和危害情境决定，机制再快也改变不了它——顺带说明，转向这一项的窗口值此刻还压在系统层的待定清单上，所以本章所有与它挂钩的覆盖信用都注明“以窗口定值为准”。第二只钟管潜伏：第一重故障允许在暗处躺多久，才不至于和第二重故障凑成不可接受的风险？自检一天跑一次还是一个点火周期跑一次，决定一大笔失效率是记在“可检出”还是记在“潜伏”。一个监控器可以“最终总会报警”，却在两只钟上都拿不到信用：报警慢过第一只钟，直接后果的信用没有；自检慢过第二只钟，“可检出多点”的信用也没有。

**分类不是器件的户籍，是一次裁决：同一个失效，换一个安全目标、换一套机制、换一只钟，判决就要重写；而每一份进了账的失效率都必须有去向，一分不许丢，一分不许重。**

供电路径的 2 FIT 之所以刺眼，正因为它此刻判在单点故障——全表几百个 FIT 里，真正顶在咽喉上的就是这 2 FIT。账立到这里，两个百分数终于可以出场了：它们就是从这笔五个去向的账里，用两次除法算出来的。算一遍，比读十遍定义都清楚。

## 6.5 把账算一遍：400 FIT 的底账

吴工把组里的人聚到白板前，用一份刻意缩小的教学底账做走查——真实项目的失效分析有几百行，这里聚合成四个元素，数字都是合成的，方法是真的。账面如下（单位：FIT）：



| 元素    | 总失效率 | 单点 | 残余 | 可检出多点 | 潜伏多点 | 安全  |
|-------|------|----|----|-------|------|-----|
| 传感器   | 100  | 0  | 5  | 20    | 5    | 70  |
| 控制器   | 200  | 0  | 8  | 60    | 12   | 120 |
| 驱动功率级 | 98   | 0  | 5  | 20    | 3    | 70  |
| 无监控供电 | 2    | 2  | 0  | 0     | 0    | 0   |
| 路径    |      |    |    |       |      |     |
| 合计    | 400  | 2  | 18 | 100   | 20   | 260 |

每一行五个去向加起来等于总失效率，合计行也闭合。先说清这个闭合能证明什么：它只证明没有把失效率算丢或算重，不证明每个数判得对——就像财务报表的借贷平衡，假账也能平，但不平的账一定不能签。

单点故障度量 SPFM 问的是：全部纳入的失效率里，有多少**没有**以单点或残余的方式直接威胁这个安全目标？

$$\text{SPFM} = 1 - (2 + 18) / 400 = 95.00\%$$

吴工让小蔡接着算潜伏故障度量 LFM。她照着直觉写下  $1 - 20/400$ ，得了 95%。吴工摇头：“分母错了。”LFM 问的是另一件事：把已经被 SPFM 盯住的单点和残余扣掉之后，在剩下的空间里，还有多大比例的失效在潜伏？

分母要先减去那 20 FIT 的单点与残余：

$$\text{LFM} = 1 - 20 / (400 - 2 - 18) = 1 - 20/380 = 94.74\%$$

差得不多，性质全变：用错分母，等于让同一笔危险贡献在两个度量里各被稀释一次，问题就悄悄换掉了。分母还有一个更隐蔽的软肋——想让 SPFM 变好看，最省力的路不是消灭危险失效，而是往分母里灌无关的硬件：多算进一堆与安全目标毫无瓜葛的器件，总失效率涨了，比值自然“改善”。这就是为什么评审第一件事是核范围：分母里装的必须是“可能显著促成或阻止这个安全目标被违背”的硬件，一件不多，一件不少。

**度量是除法，而除法的全部诚实都在分母里：分母的定义换一个字，度量回答的问题就换了一个；分母的范围松一寸，绿色就便宜一寸。**

对照预算：SPFM 95.00%，门限 99%，没过；LFM 94.74%，门限 90%，过了。账面清楚地指出病灶——分子里那 2 FIT 的单点贡献和 18 FIT 的残余贡献。没过线怎么办？自然的动作是给那段裸奔的供电路径加机制。下一节就加：加一个监控器，账到底能变好多少？

**读图任务：**沿左侧总流量跟随四条宽度不同的支流，确认安全侧、已监控、残余与潜伏都是同一总体的去向。



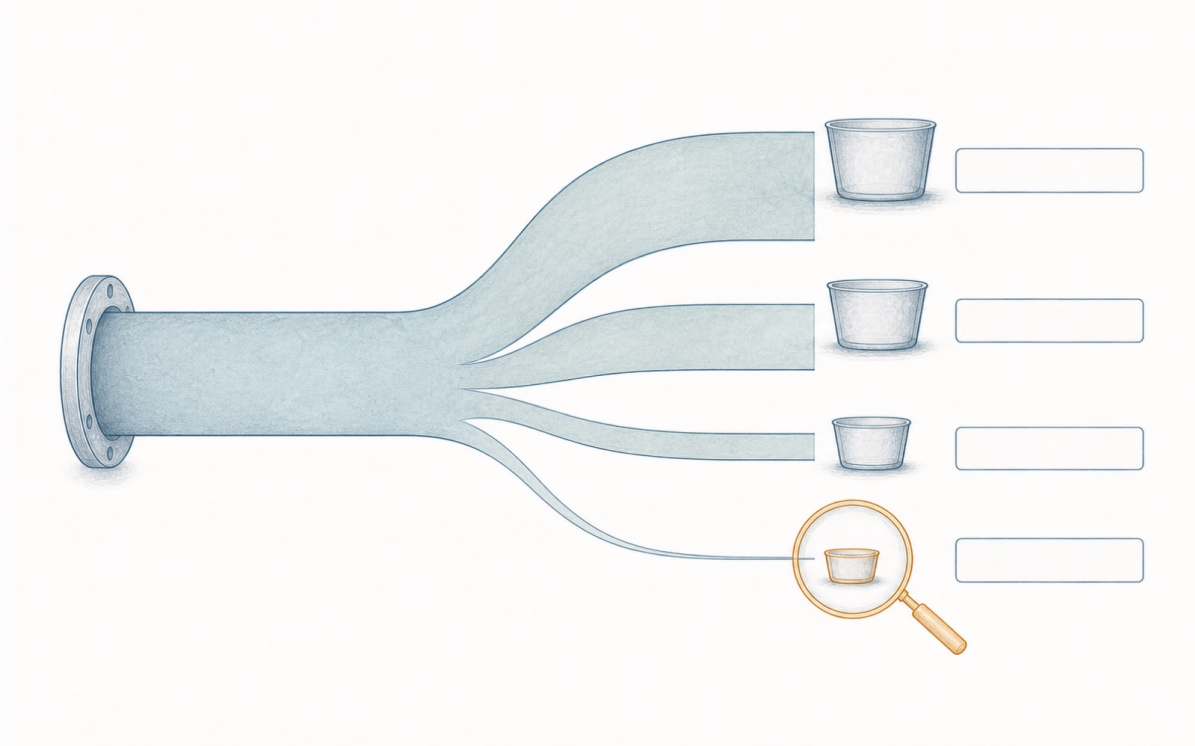


图 6-1 评价硬件度量时，关键不是只报一个总数，而是说清分类规则与每一单位的去向。小分支也可以决定分子、分母和最终结论。

**图后判断：**这是教学性流量账，不是真实元件失效率报告。它不给出 SPFM、LFM 或 PMHF 数值，也不证明诊断假设已被验证。

## 6.6 加一个 U7，只好了零点四八

设计变更是：新增一颗供电监控比较器，位号 U7，自身失效率取合成值 3 FIT。走查开始前，吴工先钉死一个边界：**原来的 400 FIT 里没有 U7**——加装之后总失效率是 403，不是 400。这句啰嗦话关掉了一个危险的歧义：如果 U7 原本就藏在某行里、现在又额外加 3 FIT，同一颗器件就在分母里被算了两次。

第一步，原供电路径的 2 FIT 迁账。假设 U7 对相关危险模式的覆盖率是 90%，且检出与反应赶得上两只钟、U7 与被监控路径确实独立——注意，这些全是要逐条拿证据的假设，不是“装了监控器”自动附赠的性质。在假设成立的前提下：1.8 FIT 迁去“可检出多点”，0.2 FIT 留作残余。小蔡问：“为什么不能直接  $2 \times 90\%$  搬走完事？”因为覆盖率不授权迁账：被盖住的 1.8 FIT 要逐模式核实后果受控、组合关系成立、时间来得及，才有资格坐进多点的账；三个前提缺一个，就退回原地。

第二步，U7 自己也要进账——这是最常被赖掉的一笔。监控器不是天使，它也是硅和焊点。它的 3 FIT 按后果分析拆开：45% 的模式不显著增加目标被违背的概率，记安全故障 1.35 FIT；其余 55%（1.65 FIT）会让监控功能失灵——这正是典型的多点候

选：U7 悄悄坏了，供电路径再坏，两重凑齐。这 1.65 FIT 里有多少潜伏，取决于**谁来监控监控器**：假设由控制器既有的诊断路径检测 U7 的恒高、漂移等危险模式，且检测周期赶得上第二只钟，则 90% 即 1.485 FIT 记作可检出多点，剩下 0.165 FIT 是没人照看的部分，老老实实记潜伏。（这条诊断路径的硬件已在控制器那 200 FIT 里，不再新增分母——若真实设计要为此加新硬件，新硬件的失效率就得进总账重算。）

第三步，重算。其他元素的 18 FIT 残余没人动过，原样在账上：

$$\text{SPFM} = 1 - (18 + 0.2) / 403 = 1 - 18.2/403 = 95.48\%$$

$$\text{LFM} = 1 - (20 + 0.165) / (403 - 18.2) = 1 - 20.165/384.8 = 94.76\%$$

白板前有几秒失望的沉默。加了一颗监控器，SPFM 从 95.00% 到 95.48%——离 99% 还远。但这半个百分点是诚实的：U7 把最危险的 2 FIT 大部分迁出了分子，同时带进了自己的 3 FIT，而基线里还有 18 FIT 的残余贡献根本不归它管。想让数字戏剧性变好，办法有的是——把 U7 的失效率漏掉不记，把 18 FIT 的残余含糊过去，或者给分母灌水——每一种都能让屏幕变绿，没有一种能让车变安全。

这笔账真正的价值在于它可以被“单因拷问”：一次只推翻一个假设，看哪笔信用被撤回。若控制器诊断路径检测 U7 的周期慢过第二只钟，U7 危险侧的 1.65 FIT 全部转潜伏，LFM 落到约 94.37%，SPFM 暂时不动；若检出加反应超过第一只钟的窗口，原路径那 1.8 FIT 的覆盖信用整笔吊销，必须回去逐模式重判——这里没有一个方便的新百分数可报，诚实的“待重算”胜过伪精确；若相关失效分析发现 U7 和被监控路径共享一个能同时放倒两者的原因——同一路供电、同一个基准源——“两个独立故障”的前提塌了，所有押在它上面的分类和结果一并撤回。

**机制改变的是分类，不是物理：它自己的失效率必须入账，它挣来的每一分信用都押着一条列得出、也撤得回的假设。**

走查散场前，小唐探头进来问进度，听到 95.48%，愣了愣：“那就是说，再迭代几轮设计，把两个百分数都推过线，硬件这块就可以签了？”吴工这次没有立刻回答。他把马克笔放下，说了一句组里老人都听过的话：“过线之后，还有一道我欠过债的坎。”

## 6.7 过线之后：那个复制来的数

先把道理摆出来，再讲那笔债。

SPFM 和 LFM 是比例，而比例天生有一个盲区：它看结构，不看绝对量。想象两个设计都做到了 99.5% 的 SPFM——A 的总失效率很低，B 的总失效率高出一个数量级；同样是 0.5% 的危险份额，落到整车上的绝对风险差着十倍。所以标准在比例之外还要一把绝对的尺子：估计在整个运行寿命内，随机硬件失效平均每运行小时把安全目标顶穿的概率——行业里叫 PMHF——并给它设概率上限；不走这条路的项目，也得走另一条：把能顶穿目标的失效原因一个一个拉出来过堂，逐个证明可接受。两把尺

子问的问题不同，谁也替不了谁。而无论哪条路，都要吃进一大堆输入：架构、每个器件的分类失效率、机制的覆盖、潜伏的暴露时长。也就是说，**那个最终写在报告里的概率，是全章这笔账一路诚实算下来的孙子辈**；账断在哪一环，它就悬空在哪一环。

现在讲那笔债。上一代转向平台做客户审核，是好几年前的事。硬件评估报告里有一个 PMHF 值，写得很漂亮，小于目标上限，绿色。审核员只问了一句：“这个数是怎么算出来的？”会议室里没有人能答。翻记录，那个数来自一张电子表格；追表格，那一格是从更早一个项目的报告里复制来的；再追那份更早的报告，算它的人已经离职，计算过程没有归档，连当年用的失效率库是哪个版本都无从查起。一个数字被复制了两代项目，它的出处比它先死了。

会议室里不是没有人替它辩护，而且两句辩护听起来都有道理。一句是：“两代平台架构差不多，数字的量级不会错。”——“差不多”恰恰是一个需要论证的判断：差在哪里、差多少、对结果敏感不敏感，得有一张差异清单撑着；而当年连这张清单都没有，“差不多”只是一种愿望的语法。另一句是：“这个数当年通过了评审。”——当年的签字属于当年那个项目、那份报告、那句主张；签字不能跨项目迁移，就像台架的绿灯不能替别的配置作证。两句辩护相继失效之后，房间里只剩下那个数字本身：数值没错——后来重算出来量级确实相当——但在被追问的那一刻，它拿不出对象、拿不出方法、拿不出单位口径、拿不出不确定度、拿不出来源。它不是证据，它只是长得像。

代价是实打实的：审核开出正式发现项，评估中断；随机失效评估从输入开始重建，硬件组补了整整一个季度的证据链；量产节点顺延，客户那边的信任又花了更久才补回来。而当年在表格里按下“粘贴”的那个年轻工程师，是吴工自己。他的上级当时说“先把格子填上，量级差不多，回头补算”，后来上级离职了，“回头”再也没有来。审核那天，是吴工站起来说的实话：“这个数是抄来的，我抄的。”从那以后，他的表格里多了那列“出处”，没有出处的数字，在他手下不许过夜。

**脱离对象、方法、单位、不确定度和来源的数字不是证据——它只是一个恰好站在证据位置上的数；而它站得越久，把它请下来的代价越大。**

顺手多问一层：为什么“先把格子填上”这句话，总是由上级说出口，而且换一家公司还是有人说？因为在组织的眼睛里，进度看得见，出处看不见。格子填上的那一刻，汇报立刻多一格绿，好处当场兑现；“这个数从哪来”要等到几年后有人追问才会显形，而到那时，说“先填上”的人多半已经不在场。一边是当天结清的奖励，一边是记在别人名下的远期债——账期这样错开，个人的诚实就得逆着激励游泳。吴工那列“出处”，做的正是把看不见的那一半，强行搬回看得见的表格里。

这也是给小唐那句“过线就能签”的完整回答。度量过线，证明的是架构对随机失效的相对覆盖，在声明的范围、账目和假设内成立——这已经很多，但硬件安全不止这些。比例过了还有绝对概率要问；随机的账算平了，系统性的故障一分都没碰——画错的原理图、超额定使用的器件、没定义清楚的接口，这些不是概率现象，分母再大也稀释不了它们，管它们的是设计规程、评审和验证；最后，算出结果的人不能自己宣布结果成立，度量和评估的结论还要过独立的验证评审——公式是分析者按的回车，结论



需要另一双眼睛。签字那一栏真正对应的主张是：“在此对象、此范围、此账目、此假设下，这些数字说了它们有资格说的话。”——每一个数字都能作证，但要按它的射程作证。这正是第 1 章那句老话在硬件层的样子。

道理讲完，账也算平，本章按说可以收尾了。可吴工那列“出处”还悬着一个没人爱问的问题：这列格子，靠什么保证一直被填、一直填对？

## 6.8 纸面追不上数字的脚步

诚实地看一眼数字在真实项目里的日常旅行：失效率从手册进了分析表，从分析表聚合进度量计算，度量结果进了评估报告，报告里的结论被摘进评审胶片，胶片上的绿格子被抄进月度汇报。五段旅程，每一段都是一次复制，而纸面世界的复制只搬数值，不搬护照——对象、方法、单位、不确定度、来源，每复制一次就有机会掉一样。到了汇报那一页，“95.48%”孤零零站在那里，它曾经拖着的那一整套前提——400 还是 403 的分母、两只钟的假设、独立性的前提、“以窗口定值为准”的批注——一样都不剩。吴工的债，就是这样欠下的：没有人作恶，只是复制了太多次。

纸面世界对此不是没有办法，它的办法就是吴工那样的人：立规矩（表格必须带出处列）、传习惯（新人第一课先学问来历）、设关卡（评审时逐个数字问一遍五个问题）。这些办法是真办法——本章前七节就是它们的展开，这个行业靠它们把随机失效管了几十年，管得相当好。但它们有一个共同的形状：**纪律长在人身上，不长在数字身上**。吴工在的时候，没有出处的数字过不了夜；吴工休假的那两周呢？换一家供应商呢？表格换一种模板呢？电子表格的单元格从不拒绝粘贴，文档不会在数字丢掉前提的那一秒弹出警告，而评审能抓住多少，取决于评审的人恰好想起问哪一格。一句话：纸面可以**要求**每个数字带上护照，却没有力气在数字被复制的那一秒钟**强制执行**。

这个极限不是硬件独有的。换个世界看一眼就知道：那台环境监测单元 ENV-01 的标定证书上写着“ $\pm 0.5\%$ ”，这个数被抄进产品手册、抄进投标文件、抄进客户的验收清单——每一站都该有人问：对哪个量程、什么温度、距上次标定多久、不确定度怎么给的？问法和硬件度量的五问一模一样；答案则要在它自己的世界里重新挣。可迁移的是问法，扛不动的也一样——纸面同样追不上那个数字的脚步。

所以本章末尾冻结的，不是答案，是三个交给后半本书的问题：怎样让一个数字无论被复制到哪里，都被迫带着它的对象、方法、单位、不确定度和来源，少一样就寸步难行；怎样让“过线”这样的比较结论自动拴着自己的范围与假设，前提一变，结论自动失效而不是继续变绿；怎样让每一次复制都留下痕迹，让任何一个数都能反查血缘，一路追到第一次测量或第一次假设为止。第 16 章会回到这三个问题，用另一种载体让数字自己扛住这些义务——那是 AI 之后的世界的做法。

而眼前，走廊另一头的问题已经等在那里。小林的软件组也在准备自己的证据，但他们的桌上没有 FIT，没有失效率手册，没有任何可以聚合的概率——软件的错误不是随机的，是设计进去的：同一个缺陷，一百万次运行就复现一百万次，没有“每十亿

小时坏一次”可言。硬件的“达标”好歹有两把尺子可算，软件连尺子都没有——那软件的“通过”，靠什么成立？第 7 章从这里开始。

导读的承诺可以兑现了。合上书之前，做三件事：在你手头的任何一份分析或预计表里任选一个失效率，试着一次写全它的对象、条件、时间基准、方法和不确定度，看卡在哪一样；找一个正在你的组织里流通的“达标”百分数，问清它的分母里装了什么、谁批准的范围；再挑一个你确定被复制过的数字，沿着它的旅程往回走，数一数它每过一站丢了什么。如果第三件事让你走进了死胡同——记住那个位置，第 16 章会回到那里。

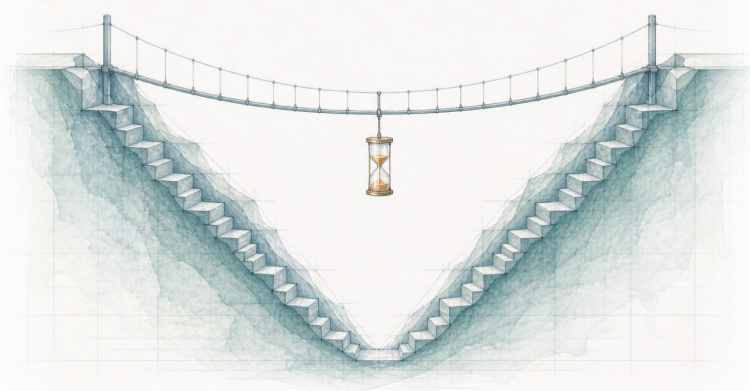
**本章注记：**本章的人物、会议、审核事件与全部数字（度量值、失效率、覆盖率、门限、位号、时间跨度等）均为合成教学材料，不对应任何真实企业、真实产品或真实个人；400 FIT 底账与 U7 变更算例是刻意缩小的教学账，不是任何实际产品的分析结果。正文对标准中硬件度量与随机失效评估要求的转述为自然语言概括，精确定义与适用条件以标准原文为准；本章不构成对任何实际系统的达标判定或放行结论。





## 第七章 通过会过期：软件层开发

Semantica 绑定：本章保存软件安全开发与变化影响的可读方法；唯一机器语义是 `semantica.chapter_packages.vol2.ch07`，主场景为 `semantica.vol2.ch07.scenario.primary`。本体、CQ、查询、Shape、案例、规则、oracle、版本与 receipt 均由该包持有，无书内 runner。当前为 `partial`、release `blocked`；工具运行成功不能证明软件安全或授权沿用旧证据。



**【导读】** 上一章散场时，硬件的数字终于有了来源与射程；可当同样的问题转向软件——“你的失效率是多少”——答案竟然是：没有。软件没有失效率可算，它的正确性不靠概率，靠过程。本章跟着软件负责人小林把这条过程之路走完：左边逐层写下承诺——安全需求、架构、单元设计与编码；右边逐层拿出证据——单元验证、集成、整机测试；V 模型两侧互相作证。然后回到桌上那张变更单：SW1.8.3 升到 SW1.8.4，只改一件事。我们要正面做完第 1 章留给小林的那份差异分析作业，也要亲眼看着每一张“通过”怎样悄悄过期。读完本章，你应当能对任何一份软件“通过”报告问出三件事：它通过的是哪一个构建、在哪个环境、按什么判据——并判断这三样东西今天还在不在。

## 7.1 失效率清单上，没有软件的名字

硬件评审收尾那天，失效率预算表投在幕布上，每一行数字都有了来源。陈工顺着表格往下看，在最后停住，转向小林：“软件那一行呢？你们的失效率是多少？”

小林的回答让不熟悉软件的人愣了一下：“软件没有失效率。”

这不是推脱。一颗电阻会老化，两颗同型号的电阻各有各的微裂纹，哪一颗先开路是掷骰子；所以硬件可以谈概率、算预算、用统计说话。而一万台车上的 SW1.8.3 是同一串比特，既不磨损也不老化。如果某个输入组合会触发一个错误，那么一万台车在遇到那个组合时会一起触发——不是随机抽中谁，而是全体同时中招。软件的失效不是零件在使用中坏掉，而是故障在写下代码的那一刻就已经埋进去了，只等某个输入、某个时序、某个模式把它激活成错误、显形成失效——第 2 章那条链，软件一环不少，特殊只在第一环：它的故障不是用着用着长出来的，是写进去的。

没有概率可用，会议室里很自然地冒出两个替代方案。

有人提议造一个数：用缺陷密度——每千行代码发现多少个故障——折算一个“软件失效率”填进预算表。这个数字对管理进度有用，但它作不了证：它的分子是**已经发现**的故障，而安全担心的恰恰是**还没发现**的那些；一个测试做得越少的团队，缺陷密度反而越好看。

有人提议靠时间：像硬件耐久那样，让软件连续跑上足够多的小时数，用“没出事”的时长作证。这条路更远。要触发的错误往往藏在罕见的条件组合里——某种模式切换叠加某个边界值叠加某拍时序——平稳运行一万小时可能一次都碰不上它；而且软件每升一版，统计就得清零。运行时长能证明故障存在，永远不能证明故障不存在。

两条路都通向同一个结论：

**软件没有失效率，只有还没被触发的错误；对付它的武器不是概率，而是过程——左边每写下一层承诺，右边就要有一层证据与它对质。**

这就是软件世界的 V 模型：左侧从安全需求下到架构、单元设计、代码，一层比一层具体，每一层都是对上一层的承诺；右侧从单元验证上到集成、整机测试，每一层证据都只对质左侧对应的那一层——单元测试对质单元设计，集成测试对质架构，整机测试对质安全需求。“过程”两个字在这里不是官僚话，它就是软件可信的全部来源。

几天后，那张变更申请到了小林桌上：SW1.8.3 升 SW1.8.4，其余声明不变。差异分析的第一步不是看改了什么，而是先弄清当初承诺过什么——承诺的第一层，软件安全需求，究竟长什么样？

**读图任务：**从 V 形左臂逐级向下读承诺，再沿右臂向上读证据，逐层检查同一高度的水平线是否真正成对。

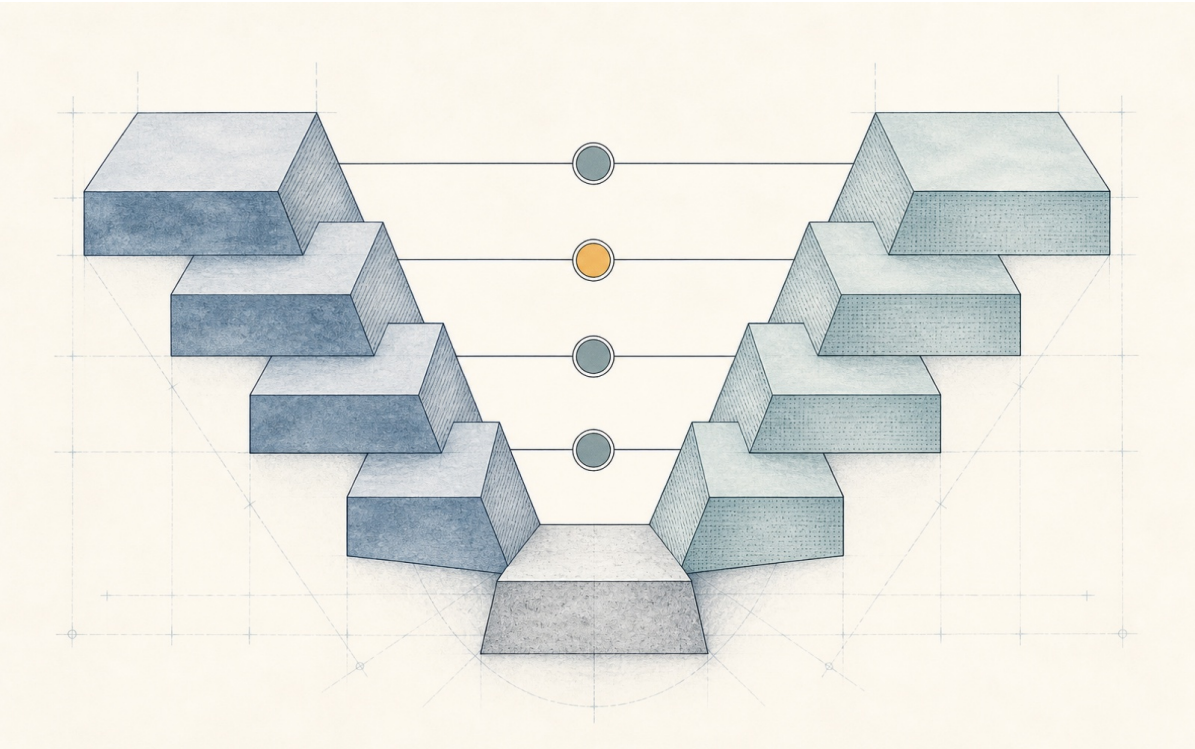


图 7-1 软件可信不来自一次统笼 PASS，而来自每层承诺都有同层次的可判定证据与之对质。跨层证据可以补充，却不能悄悄替代缺失的对应关系。

**图后判断：**本图是关系模型，不是项目里程碑计划。它不证明需求已充分、测试已通过，也不要求所有开发必须采用线性流程。

## 7.2 第一层承诺，要能生成标准答案

小林把变更单钉在白板边上，先没有动它，而是翻出软件这一层的承诺书。系统层交下来的话是这样一句：“软件应检测不合理的助力转矩请求，并在规定时限内进入受控的降级。”这句话没错，但它还不是软件安全需求——它是软件安全需求的原料。

从原料到需求，最省事的路是复制一份、改个标题、加一句“由软件实现”。追溯表格会因此非常好看：每条软件需求都指得回上游。但信息量是零——“检测不合理的转矩”仍然没有说什么算不合理、检测窗口多长、检测到之后哪条路径接手。

第二条省事的路是干脆开写代码：“细节编码的时候自然就清楚了。”这条路的代价要到测试那天才付：写测试的人对着“应检测不合理转矩”出不了题——多大算越界？恰好压线算不算？要连续几拍才算数？他只能去问写代码的人，而写代码的人给出的答案，正是代码里已经写的那个答案。用实现来解释需求，再用需求来考实现，这场考试永远满分，也永远考不出任何东西。

还有第三条路，看起来最稳妥：把句子写得更全——“软件应正确处理所有异常输入”。这句话谁也挑不出错，因为它什么也没绑定：哪些输入算异常、“正确处理”在每种

模式下分别指什么，全都留白。一句无法被反驳的需求，和第 1 章那句无法被反驳的“可以交付了”是同一种光滑——没有一处可以着力，也就没有一处可以作证。

真正的完成标准只有一条：写测试的人不来问你，就能提前写出标准答案。为此，这句原料要被逼问出全部细节：转矩的允许范围和最大变化率是多少，用什么数值表示，边界含不含；越界要连续几拍才坐实，坐实后多久之内完成反应——这个时限的项目取值仍在系统层敲定之中，但“有这样一个待定值、归谁定、影响哪些测试”必须先写下来；还有模式：车辆刚唤醒、诊断自检还差两拍才完成的时候，传感器数据算有效、算未知还是算禁用？标定模式下限值临时改动，监控跟着新值还是守着安全上限？

逼问的结果，是承诺书拆成了两条、分给两个模块：一条守输入——检查转矩传感器的数值、诊断标志和数据新鲜度，发现不可信就请求降级；一条守输出——检查助力指令的范围与变化率，越界就进入抑制。还有一件容易被漏掉的东西也进了承诺书：标定 C41。代码可以完美地执行“与上限比较”，但上限本身是标定给的——错误的标定能让完美的代码尽职尽责地保护一个错误的边界。软件的承诺，从来是代码加数据一起许下的。

**一条软件安全需求的完成标准，是测试的人不用来问你，就能提前写出标准答案；写不出标准答案的句子还不是需求，只是愿望。**

承诺拆给了两个模块，一个守进、一个守出。评审预备会上小唐看着架构图说了一句话，把下一个问题带了出来：拆开了，就互不干扰了吗？

## 7.3 两个框，隔不出两个世界

小唐的原话是：“一个守输入，一个守输出，一个失灵另一个还兜着——这不就是双保险吗？”

这个直觉一点都不外行。冗余思维是工程里最值钱的直觉之一，整个安全行业都建立在“别把鸡蛋放在一个篮子里”上。但它有一个前提：两个篮子得真的是两个篮子。

小林在白板上画了一条走廊。两个模块此刻跑在同一个周期任务里，先跑输入检查，再跑输出监控。假设某天总线上连续涌来异常报文，输入检查模块陷入一个没有上限的重试循环——它自己的代码一行都没错，只是不肯退出。轮不到输出监控运行，这个周期就结束了。恰恰在错误输入最猖獗、最需要输出监控的那几拍，监控缺席了。两个框各自测试全部通过，挡不住这条链。

那就拆成两个任务？走廊还在：如果高优先级任务可以被无限触发，低优先级的监控照样饿死；如果两个任务共享一块没有保护的内存，一次越界写就能把对方的阈值改掉；它们还数着同一个时钟，用着同一个错误处理服务——那个服务卡死的话，检测和反应一起消失。那再加个看门狗？看门狗看得见“没人喂狗”，看不见“喂狗的人读了脏数据”。

**框图分开的是名字；周期、内存、时钟和错误处理这些真正的走廊还连在一起。“互不干扰”是一项要拿证据的主张，不是画图的赠品。**



这份证据不便宜：要有最坏执行时间和调度分析，证明在声明的负载下监控总能轮到；要有内存保护和访问规则，证明一个模块写不坏另一个；还要有故障注入——故意让一个模块发疯，看另一个是否还在岗。分析回答“为什么应该没事”，注入回答“逼到墙角时是不是真的没事”，两样缺一不可。而且这份证据自带失效条件：任务优先级一调、内存布局一改、处理器一换，“互不干扰”就要重新拿证——这是本章后面还会回来的一根线头。

走廊查清楚了，还剩每个房间的内部：模块里的代码本身，怎么写才不埋雷？

## 7.4 代码的纪律，写给编译器和三年后的人

高等级软件的编码纪律，常被当成两句话就能带过的话题。第一句：“我们招的都是写了十年 C 的老手。”经验确实是第一道网，这话不外行。但语言的暗角连老手也得靠约定防身——不是因为水平不够，而是因为有些坑不在人的直觉里，在语言标准的小字里。

小林给团队看过一个例子。转矩用十六位有符号定点数表示，分辨率百分之一牛米，能表示大约正负三百二十七牛米。变化率要算本拍减上一拍：如果两个值恰好一正一负都在满量程，差值需要十七位才装得下；把它存回同样十六位的变量，超出范围之后会发生什么，语言标准根本不作承诺——在一种平台上它可能“绕回来”给你一个貌似合理的数，在另一种平台上连那次减法本身都是未定义行为。正确的写法要先把数扩宽再相减，还要处理时间差为零、时间戳倒退这些没人愿意想的分支。写了十年 C 的老手也会在赶工的下午顺手写下那行直接相减——所以纪律要写成规则，不能存在手感里。

第二句：“我们采用了 MISRA C。”这句也不外行——语言子集准则正是为封锁暗角而生。但一个名字不等于一套纪律：采用哪个版本、启用哪些规则、谁有权批准偏离、工具查得了哪些、剩下的靠谁人工把关——这些都空着的话，“采用了”三个字和没采用的差别只在汇报材料里。低复杂度、强类型、禁止隐式转换、变量必须初始化、高等级下不用递归、并发要有交代——准则的每一条背后都是一类出过事的暗角。

纪律管住语言，设计还要管住语义。单元设计写到什么程度算够？和需求那层同一个标准：实现的人不用来问。如果写代码的人还得跑来问“上电后第一拍，上一拍的值取什么”，设计就还没写完；如果做验证的人没法从设计推出“时间戳倒退时应该发生什么”，设计也还没写完。

**编码准则不是审美洁癖，是把语言里最会骗人的角落提前封死；封不死的，设计必须写明——写到每一个风险决定都有可观察的后果，验证才有东西可验。**

设计写细了，代码交上来了。怎么证明代码真的照办了？测到什么程度才算够——覆盖率达到百分之百，是不是就是终点？

## 7.5 覆盖率百分之百的那一天

这个问题是小蔡问出来的。她整理验证计划时看到覆盖率指标，很自然地说：“覆盖率都到百分之百了，验证不就完成了吗？”

先说清楚：她引用的是一套真实的纪律。结构覆盖率是成熟团队的标志，行业对高等级软件的要求逐级加严——从语句走到过，到分支两边都走到过，再到每个条件真正独立影响过判定。测完之后量一量哪里没走到，是暴露漏洞最锋利的工具之一。小蔡没有引用错任何东西。

但覆盖率有两个天生的沉默点。第一，它量的是“代码被走过”，不看“答案判得对不对”。假设整套用例的预期结果都来自同一份被误解的需求——每条用例都执行了、每个分支都走到了、每次比对都通过了，覆盖率百分之百，全绿，全错。覆盖率对此一声不吭，因为它的职责本来就不包括判卷。

第二，它看得见写了的代码，看不见没写的代码。需求里漏掉的那个模式切换场景，不会以“未覆盖”的形式出现在报告里——没有代码，就没有可覆盖的对象。反过来它倒有一个真本事：一条只在时间戳倒退时才会进入的防御分支从没被任何用例触发，覆盖率报告会把这个洞亮出来。这正是它的价值——暴露没测到的地方。但洞亮出来之后，诚实的出路有三条：从需求里补一条真用例；发现这段代码根本走不到，问它为什么还在；或者承认这是从外面驱动不了的防御，把保留的理由写下来。最省事的第四条路——凑一条只为踩线的用例把指针推上去——是把量具当成了目标。这也正是它的边界：指针走到一百的那一刻，它能说的话就说完了。

那把方法堆满呢？评审、静态分析、边界值、故障注入、资源评估……十种方法全部用上，总够了吧？堆数量不等于消灭盲区：如果所有用例都在正常模式下生成，十种方法会一起漏掉“刷写结束后的第一次采样”；如果所有证据都建立在同一份误解的需求上，十种方法会给出十份一致的错误答案。方法组合要按“这段代码可能怎么错”来配——需求理解错靠评审和基于需求的测试，数值和边界错靠边界值分析，异常路径靠故障注入，资源不够靠最坏情况评估——各补各的盲区，才叫组合。

**覆盖率量的是代码被走过，不是答案判得对；它能告诉你哪里还没测，永远不能告诉你测过的地方都对了。**

就算单元层每一关都真过了——拼起来，就对了么？

## 7.6 拼起来才见真章

“单元都过了，集成不就是接线检查？”这句话低估了拼装。还有人说反话：“别在台架上折腾了，上实车跑几圈才是真证据。”这句话高估了整车。两句都得挡一挡。

集成看的是单元永远看不见的东西。举一个安静的例子：新鲜度检查的需求说，数据年龄按传感器**采样时刻**算。单元测试时，测试桩老老实实送进采样时刻，旧数据如期被拒收，单元真过、没有水分。问题藏在集成层的适配代码里——它把报文**到达时刻**填

进了同一个字段。正常负载下两个时刻只差毫厘，集成的名义场景全部通过；直到某条用例故意制造总线拥塞，报文在线上堵了一阵，到达时刻还很新、采样值早已过期，旧转矩被当成新数据放了进去。单元证据和集成证据都没有造假，它们只是共同没有照到这条缝——直到有人专门为这条缝写了用例。

至于“上实车才算数”：实车能暴露整体语境，却做不到可控地重复一次极端故障，更不能一天重复两百次。台架长于把边界工况按住了反复揉，整车长于让你看见没想到的组合——谁上场由要回答的问题决定，不由“哪个听起来更真”决定。

会议室安静下来的时候，小林讲了一件旧事。几年前在上一家公司，供应商例行升级了编译器版本。他当时负责回归，说过一句他现在还记得的话：“代码一行没改，编译器又不是我们的产品，回归也全绿——还要重测什么？”这句话在当时没人反驳，因为它听起来无懈可击。新编译器编出来的版本，回归套件照跑，全绿发布。集成台架的长时间运行里，降级反应开始偶发地迟到几拍——复现不稳定，日志看不出规律。排查了十一天，所有人都在翻“改过的代码”。这不是笨：排查总要从最便宜的嫌疑清单查起，而组织里最现成的清单就是变更记录——上面有名有姓，翻它不必向任何人解释；编译器不在清单上，因为清单是按“我们动过什么”开列的，不是按“什么在左右机器的行为”开列的。最后答案就站在这张清单照不到的地方：新编译器聪明了一点，把一个每拍都要调用的小函数就地摊开、省掉进出的开销，又把循环里重复的计算挪到循环外——同一段源码，跑完少花几十微秒。快本来是好事，坏在旧版本里藏着一个没人写下来的巧合：准备新数据的那个任务，每拍总是恰好赶在监控任务醒来之前把值放好——没有谁设计过这个先后，它只是被旧的执行时长撑着，一直成立。新版本快了这几十微秒，先后就不再被撑着：多数拍里顺序照旧，偶尔一次中断插进缝里，监控先醒、数据后到，这一拍它拿着上一拍的旧值做判断，数到一半的越界计数被打断重来——反应迟到的正是这几拍。中断哪一拍来由负载决定，所以复现不稳定；日志按拍记录，看不见一拍之内谁先谁后，所以查不出规律。回归全绿是真的——那些用例判的是输入输出对不对，没有一条盯着“这一拍读到的是不是上一拍的值”。代价是集成测试整轮重做，交样推迟一个月，而他那份全绿的回归报告，在复盘会上被投影给所有人看。

那次事故教给小林的，不是“编译器不可信”，而是一件更根本的事：

**“通过”绑定的从来不只是那份源代码，而是把代码变成机器行为的整条链——编译器、优化选项、库、链接布局、时钟与负载；链上任何一环挪动，“通过”就开始过期。**

这也是为什么本章叫“通过会过期”。现在可以回到白板边那张变更单了：SW1.8.3升SW1.8.4，只改一件事——旧的绿卡，还剩多少？

## 7.7 差异分析：给每一张“通过”标注保质期

第 1 章那间会议室里，两派已经交过手：一派要全部作废重测，输在追不上变化、浪费真证据；一派说“软件而已，旧证据接着用”，输在郑工那批追回的车上。还有第三条容易走的路：拿着改动清单机械地圈——1.8.4 改了哪两个模块，就重测哪两个模块。这条路比前两条精细，但小林的十一天学费恰好花在它的盲区上：新版本是一次完整的重新构建，二进制整体变了，链接布局和时序节拍可能整体挪动；两个模块的改动清单，圈不住整条链的影响。

差异分析要做的，是把每一层“通过”当初绑定了什么摊开，逐层判断它对新主张还剩多少效力。小林交出的清单是这样的：

| 层     | 这张“通过”当初绑定什么           | 升版之后                                                                   |
|-------|------------------------|------------------------------------------------------------------------|
| 安全需求  | 两条承诺、阈值与模式语义           | 承诺未变，但要核实差异没有触碰阈值定义与模式语义，核实的理由写下来                                      |
| 单元验证  | 改动模块的设计与代码版本           | 改动的两个模块定向重跑，接口相邻的单元一并重跑；方法、判据、测试集的定义仍然有效，过期的是它们与新构建的关系                 |
| 集成    | 上一版的完整构建与调度节拍          | 新构建整体重做冒烟与接口回归，时序敏感用例必须重跑——这一行是旧事故的直接遗产                                |
| 台架    | P07 样机、SW1.8.4-rc2、C42 | 升到 1.8.4 后这张卡“更接近”了，但 rc2 不是正式版、P07 不是目标样机、C42 不是 C41；接近仍不是等于，三项偏差逐项说明 |
| 工具链   | 编译器、选项、库的精确版本          | 逐项核对与 1.8.3 构建完全一致；这次核对下来确实没变——但“没变”是查出来的，不是声明出来的                      |
| 标定与配置 | C41 及各参数的含义            | C41 声明不变，仍要核实 1.8.4 没有改变任何参数的解释方式                                      |

注意这张清单里没有一行写“全部作废”，也没有一行写“照单全收”。每一行都在回答同一个问题：这张旧“通过”的绑定物里，哪些还在，哪些已经不在。工具链那一行值得多看一眼：结论是“没变”，动作却一点不少——把编译器、选项和库逐项核对一遍，然后把核对本身记录在案。声明“其余不变”是变更单的义务，验证“其余真的没变”是差异分析的义务；两者只差一步，隔着的正是他那十一天。



还有一行是“接近的诱惑”重演。台架那张卡当时用的是 SW1.8.4-rc2——软件升到 1.8.4 之后，它比谁都更接近新主张了，顺手收编的念头比上一章更强烈。小林在清单里给它的判词和第 1 章一样：更接近仍然不是等于，rc2 与正式版的差异、样机与目标件的差异、C42 与 C41 的差异，一项都不许含糊过去。差异分析最怕的不是明显过期的证据，而是“看起来快够上了”的证据——前者没人敢用，后者人人想用。

**差异分析的产出不是一份“重测哪些”的清单，而是给每一张旧“通过”重新标注保质期——哪些仍在期内、凭什么；哪些已经过期、欠什么。**

这套“保质期”的问法，是汽车软件的行话，还是到哪里都成立？

## 7.8 换一个世界，过期照旧

这套问法出了汽车行业照样锋利。还记得那台环境监测单元 ENV-01 吗？它的固件三年前“验证通过”，报告还在档案柜里。可当着手排查一批现场读数异常时，团队发现：构建那版固件的笔记本电脑已经报废，编译器版本没人记得，测试脚本存在一位离职同事的家目录里。报告上的“通过”一个字都没变，支撑这个“通过”的世界已经没了。该问的问题和 RC17 桌上的一模一样：它通过的是哪一个构建、在哪个环境、按什么判据——三样东西今天还在不在。答案要 ENV-01 的世界自己重建，问法却原样通用。

可是回到小林的桌上，还有最后一个不那么体面的问题：这份差异分析本身，靠什么维持不过期？

## 7.9 纸面差异分析到不了的地方

小林的差异分析在评审会上被接受了。但值得诚实地看一眼它是怎么维持的：那是一张手工表格，每行一件回归资产，列上写着它绑定的构建、环境、测试夹具版本、结论和失效条件。它今天是对的，靠的是小林的记忆、他付过的学费和他的认真。

而纸面世界没有任何机制替他把关。没有什么会强制一份测试报告随身携带自己的构建指纹和环境指纹；没有什么会在编译器悄悄升级的那天，把受影响的旧结论自动标红；下一次升版——1.8.5 总会来的——这张表格要靠人脑重新推演一遍；小林休假的那两周，没有第二个人能替他回答“这条结论还新鲜吗”。测试夹具在悄悄改版，台架软件在悄悄更新，漂移不发通知。一次通过穿不过新构建、新环境与漂移——这不是谁失职，是纸面载体的天花板。

所以本章末尾冻结的，是三个纸面回答不了的问题：给定一份“通过”报告，怎样让它可查地绑定自己的构建、环境与判据，而不是靠人回忆；给定一次变更，怎样自动列出哪些结论因此过期、哪些仍在保质期内、理由是什么；给定流逝的时间，怎样发现哪些回归资产已经悄悄漂移。第 17 章将从这三个问题出发，把版本、构建、环境与结论之间的关系变成机器可以维护的结构——那是 AI 之后的世界的做法。



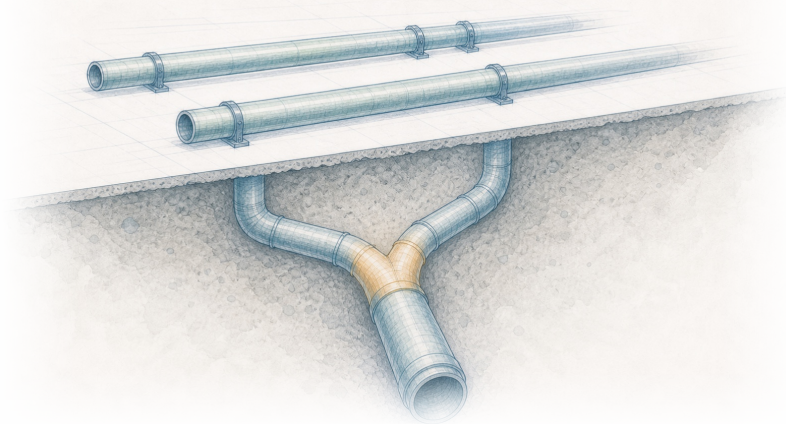
散会时，硬件负责人吴工在门口停了一下，看着架构图对小林说：“你的两个哨兵，它们盯着的主控算法，还有我的诊断电路——跑在同一颗 H3.2 芯片上，吃同一路电源，数同一个时钟。你一张绿卡，我一张绿卡，各自都合格。芯片打个喷嚏呢？”没有人接话。软硬件各自达标，但它们共享同一块芯片、同一路电源、同一个时钟——各自合格，合在一起就安全吗？第 8 章从这个问题开始。

导读的承诺现在兑现。合上书之前，做三件事：找一份你手头最近的软件测试报告，查它有没有记录构建标识、编译环境与工具版本——找不到的话，问问谁能凭记忆补出来；在你的项目里找一条只有“应正确处理”却写不出标准答案的需求，试着把它逼问到测试的人不用来问你；再回忆你们上一次工具链升级，谁决定了哪些测试要重跑，这个决定记录在哪里。如果第三件事的答案是“好像没人决定过”——记住这种凉意，第 17 章会回到这里。

**本章注记：**本章的人物、会议、事故经过与全部产品数字（候选代号、软硬件版本、标定编号、位宽与量程示例、排查天数、延期时长等）均为合成教学材料，不对应任何真实企业、真实产品或真实个人；文中对软件开发与验证做法的转述为自然语言概括，精确要求以标准原文为准；本章不构成对任何实际系统的判定、分级或放行结论，也不表示任何报告已被接受。

## 第八章 多一条通道，不等于多一分独立：ASIL 分解与相关失效分析

Semantica 绑定：本章保存 ASIL 分解、DFA 与独立性论证；唯一机器语义是 `semantica.chapter_packages.vol2.ch08`，主场景为 `semantica.vol2.ch08.scenario.primary`。方案本体、CQ、查询、Shape、案例、规则、oracle、manifest 与 receipt 全在 Semantica。当前为 `partial`、release `blocked`；未建模或不受支持的依赖推理不得被解释成“已证明独立”。



**【导读】**上一章散会时，一个问题挂在半空：软件达标了，硬件也达标了，可它们跑在同一块芯片上，喝同一路电，听同一个时钟——各自合格，合在一起就安全吗？本章从一个最诱人的答案开始：多画一条通道。冗余是工程最古老的智慧，分解也是标准明文允许的做法，但“两路了就独立了”这句话里藏着两个没有兑付的承诺。本章先陪你把分解这份合同逐条读完——拆出去的每条要求欠什么、原来的等级靠什么赎回；再听一位付过学费的供应商工程师讲，两条画得漂漂亮亮的通道怎样共享同一个命运；然后跟着一场相关失效分析的走查，看“独立”两个字怎样从一句口号变成一句可以被反驳的结论。读完本章，你应当能对任何一张画着两条通道的

架构图问出：这两条路在哪里共享同一个命运；并且在“独立”写进结论之前，先说出它的分母——查过哪里、凭什么排除、剩下的谁来看住。

## 8.1 散会时挂着的问题，和图上多出来的第二条通道

软件的绿灯和硬件的绿灯各自立住之后，那个问题没有跟着散会一起散掉：两套各自合格的东西，共享同一块芯片、同一路电源、同一个时钟——合在一起，安全吗？

周四下午的架构评审会就是为它开的。陈工主持，硬件负责人吴工带着原理图，小林带着软件架构，小蔡负责记录。系统层需求评审来过的那家转矩传感器供应商，这次又派来了系统工程师梁工——他们围绕自家传感信号，给 EPS 的下一版监控链路准备了一套双通道方案的草图。

小唐第一个走到白板前。他画了两个并排的框，一条主通道，一条监控通道，然后把笔帽合上：“其实答案已经在图上了。我们把监控功能单独拎出来，给它画一条自己的通道——自己的输入、自己的判断、自己的关断出口。两条路互为备份，一条倒了另一条还在，这就是独立。而且按分解的规矩，一条最高等级的安全要求落到两条独立的通道上，每条通道自己的等级就可以降下来。一张图解决两个难题：共享的担心没有了，因为有了第二条路；等级的重担也轻了，因为拆开了。”

这段话一点都不外行。冗余是工程最古老的智慧之一：客机装两台发动机，刹车分两条回路，医院配备用发电机——“别把所有鸡蛋放在一个篮子里”这句老话，养活了半部可靠性工程史。分解也不是小唐的发明：功能安全的规则里明明白白写着，一条高等级的要求可以拆到两个足够独立的部分上，各自按较低的等级开发。方向没有错。错的风险藏在那两个“就”字里——“两条路了**就**独立了”，“拆开了等级**就**降下来了”。

陈工盯着那两个框看了一会儿，问了一个很短的问题：“降下来——凭什么？”

会议室安静下来。是啊，标准允许拆，可它为什么允许？它拿什么换的？在回答“两条路是不是真的独立”之前，得先弄清一件更基本的事：分解这份合同，到底许诺了什么，又向我们要什么。

## 8.2 拆开的是承诺，不是工作量

先替小唐把最顺手的理解摆出来：分解是一种加法。D 级拆成 C 加 A，就像十块钱换成七块加三块，总额守恒，各自的担子轻了。这个理解必须第一个排除，因为它错得最隐蔽：安全等级不是数值，是刻度——它标注的是开发和验证要多严谨，不是一种可以存取的额度。两笔低额存款凑不出一笔高额信用；C 加 A 不是 D 的算式，它是标准点过头的几种冗余结构之一。以最高的 D 级为例，允许的拆法是一高一低（C 加 A）、两个居中（B 加 B），或者一条保持原级、另一条按常规质量管理开发——每一种都是结构，不是分账。这个错法隐蔽到什么程度？查表居然查得通：把五档从低到高数

成零到四，一高一低是三加一，两个居中是二加二，原级加常规是四加零——允许的拆法个个“加起来等于四”。可这份整齐是挑出来的，不是算出来的；它最坏的作用是教人把刻度当额度，仿佛账面已经配平，后面那笔最贵的独立性证明就可以不付。

第二个顺手的理解是：我们已经有两路了。软件架构里本来就有主算法和监控模块，一个算助力，一个查合理性，各管一摊——把这两个模块画成两个框，不就是分解吗？这个理解也要排除，而检验它只需要一个思想实验：**把另一条路遮住**。遮住主算法，只剩监控模块——它靠什么输入发现“助力不该来的时候来了”？它有没有自己的关断出口，还是只能给主算法发一条“请你停下”的消息，而主算法此刻已经失效？分工回答的是“谁做哪一部分”，分解要求的是“每一条拆出去的要求，独自就能兑现原来那句承诺”。一只只会看“程序还活着没有”的简单看门狗，发现不了“程序活着但一直在算错”——把它算作第二条通道，遮住试验一遮就穿帮。协作不是冗余，两个框不是两条路。

把这两种理解排除之后，分解合同的真实条款才能读清楚。它有两页。

第一页是**功能的页**：拆出去的两条要求，每一条都必须自己扛得起原来那句话。对 EPS，那句话是“在危害容许的时间窗口内发现非预期助力，并把系统带进受控的降级”。主通道单独在场时要能做到，监控通道单独在场时也要能做到——各自的输入、各自的判据、各自可达的关断出口、各自算得清的时间账。两条路可以用不同的原理，但不许互相借答案：“监控器会替我兜底”不能写进主通道自己的理由里。

第二页是**信用的页**：原来那个高等级的风险降低，不由任何一条路单独提供，而由两条路的组合、加上一件必须另行证明的东西——独立——共同赎回。所以每条拆出的要求都带着一个括号记号：拆出来的 A 写作 A(D)，括号里的 D 是一笔来源记忆，提醒所有后来的人：这不是一条普通的 A 级要求，它在一条最高等级的承诺链上服役，它的“低”是用独立性押出来的。

小林在这时问了一个很实际的问题：“那软件这边是不是可以松口气？拆完以后监控通道按低等级做，测试和评审都能省不少吧。”

省，但不是全省——有几笔账不打折。两条路各自的开发可以按拆后的等级配置，这是分解真正让出的地方；可是把两条路合起来、验证它们共同满足原来那条要求的集成工作，按拆之前的等级走；面向原安全目标的整体硬件账（上一章那些失效率指标）不因为拆了就分成两笔小账各自过关；而支撑这一切的独立性分析本身，恰恰要按原来的最高等级来做。换句话说，分解把担子从“两条路各自”那里挪走了一部分，又把它加到了“两条路之间”——省下的钱，要拿去买独立的证明。

**分解拆开的是承诺，不是工作量：每一条拆出去的要求都必须独自扛得起原来那句话，而原来那个等级的信用，要靠两条路的组合加上被证明的独立才能赎回。**

合同读完，小唐的方案还剩最后一个词没有着落，也是最贵的一个词：独立。图上分开的两条路，命运也分开了吗？



### 8.3 两条通道，一个命运

小唐转向梁工：“你们这套双通道方案，独立性应该有现成的报告吧？”

梁工没有直接回答。他说：“我先讲一个我们自己的项目。”

几年前，梁工所在的团队给另一家客户做过一版双通道的转矩监控设计。两条通道分属两颗处理器，各自的需求、各自的代码、各自的测试，两边的通道级验证全绿，报告写着“互为冗余，彼此独立”。B 样的电路板已经投了板，试验夹具做了，认证排期订了。然后客户的安全负责人来做联合走查，第一个问题是：“把电源树给我看一下。”沿着供电往上追，两颗处理器的电源最终汇在同一颗稳压器上；更难看的是，那颗稳压器的使能脚，接在主通道处理器的一个输出上。这一笔在原理图定稿时还是个加分项：上电瞬间两路一起启动容易把电源拉趴下，让主处理器先起来、自检看到电压站稳，再抬这个脚放监控通道上电——上电顺序管得干干净净。没有人在任何一次会上决定过“让主通道掌管监控通道的生死”，这句话是从那个完全合理的局部决定里自己长出来的：主通道的某几种失效，会顺手把监控通道的电也拉掉。走查继续，第二个发现跟着来了：两条通道的输入滤波，用的是同一份代码库里的同一个函数，同一个边界条件缺陷，两边各复制了一份。

“结果是重新布板，两百多块样板报废，认证试验全部重排，项目推迟了两个多月，定点差点丢掉。”梁工说，“客户当面问我：你们的独立性报告里，怎么没有电源树？我记得我的回答是——我们每条通道都测过。”

会议室里没有人笑。这句话和郑工那句“通过的是样机”，是同一个句式。

顺带把两个数字掂一掂，它们互相作证。两百多块不是挥霍：一次 B 样投产，台架要板、耐久要板、认证要板、整车试制要板，再加夹具调试和备件，本来就按这个盘子备料——报废也就按这个盘子报废。两个多月同样拆得开：改板画图一两周，新板打样贴装三四周，两条通道的验证重跑两三周，让出去的认证台位重新排队还要再等——前几段只能串着走，首尾一接，两个多月已经是一切顺利的结果。

这个故事最扎人的地方在于：它不是粗心的产物。两边的设计都认真，两边的测试都真实，报告没有一个字造假——每条通道**各自**的一切都对，错的是“各自”这个视角本身。功能安全给这类失效起了一个准确的名字：**相关失效**——两条路的失效不相互独立。它有两种典型的走法。一种叫**共因失效**：同一个原因，同时够到两条路。一次电源浪涌放倒两颗处理器；同一份代码缺陷在两边各发作一次；同一批次器件带着同一个工艺弱点一起老化。另一种叫**级联失效**：一条路的失效沿着某条现实存在的路径，把另一条路拖下水。主通道错误的输出污染了监控通道的输入；低等级的任务占满了处理器，高等级的监控没有了运行的时间。还有一种更安静的走法：监控通道先被某个共同的原因悄悄放倒，几天后主通道再失效时，岗哨早已不在——两次失效不在同一瞬间，合起来仍是同一个命运。

听到这里，自然的对策会一个个冒出来。把供电分开走线、把两颗芯片摆远一点？有用，但那只是修改了嫌疑清单，没有出具结论——分开供电之后，共享的传感器、共



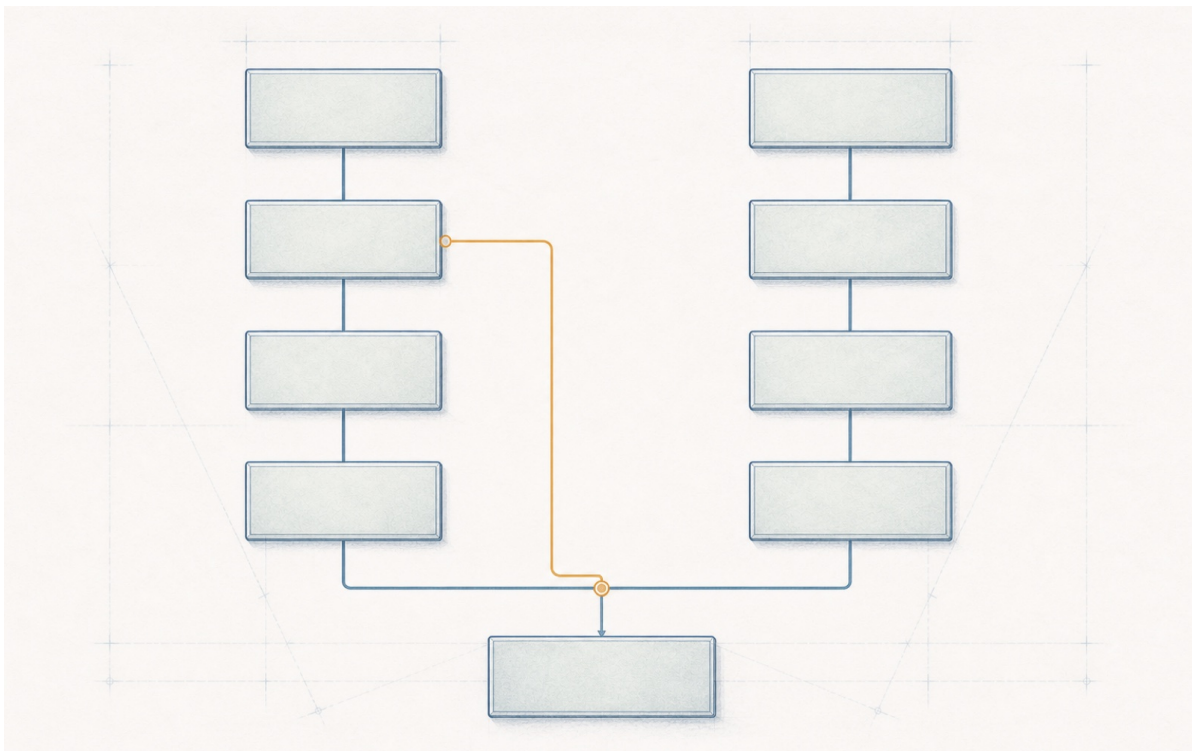
享的时钟源、共同的接口规格还在原地。那就用两颗不同型号的芯片、两家供应商？也有用，但两条通道仍然可能读同一颗转矩传感器、由同一份需求文档喂大、被同一个编译器编译、在产线上被同一台设备刷写。反过来也一样：同住一颗芯片不等于自动判死刑——有些芯片内部有独立的电源岛、独立的时钟和经过验证的隔离机制，判断的依据是对具体架构的分析，不是框图上的距离。

**图上的两条通道是几何事实，独立是工程主张；依赖不住在方框里，住在方框之间的缝里。**

梁工最后说：“所以你问我要现成的独立性报告——我没有。我有的是一张清单，上面是我们那次学费换来的每一道缝。走查要一起做。”

那么，缝要从哪里系统地找？怎样把“独立”从一句口号，变成一句可以被反驳的话？

**读图任务：**先看上方两条功能通道为什么像是并行独立，再沿供电线向下追踪到同一上游模块，并注意额外的非对称控制线。



**图 8-1** 功能分解不等于依赖分解。两条通道只要共享上游资源、环境或控制路径，同一原因就可能同时破坏它们的声称独立性。

**图后判断：**图中共享电源是一个反例结构，不代表任何真实架构。它不给出 ASIL 分解等级，也不证明共因故障已发生或已被缓解。

## 8.4 给“独立”一个分母

下一周，团队为候选的双通道架构做了第一次相关失效分析走查。参加的人：陈工、小唐、小林、吴工、梁工，小蔡记录。

开场之前，先排除两种省事的做法。第一种：开个头脑风暴会，大家把想得到的共因列一列，逐条讨论完就散会。问题在于没有分母——“没想到”和“没有”在会议纪要里长得一模一样，三个月后没人分得清哪些缝是查过并排除的，哪些是压根没人提起的。第二种：给共因估一个百分比，乘进可靠性计算，让它变成一个“已经考虑过”的数字。这更危险：需求被两边同样误解、同一份代码缺陷、同一个接口规格错误——这类系统性的原因没有诚实的发生概率可言，给它编一个数，得到的是一个看起来精确的错误。

走查用的是另一种纪律：沿着“一个原因怎样同时够到两条路”，把可能的耦合处一条一条过，每一条都必须得到三种回答之一——找到了可信的嫌疑、查过了并有排除的理由、或者暂时回答不了。空白不算回答。

| 追问的缝  | 对 EPS 双通道的问法                  |
|-------|-------------------------------|
| 供电    | 两路的电源向上追，最终汇在哪里？谁的失效能拉掉谁的电？   |
| 时钟与时间 | 两路各自凭什么知道“现在”？时间基共享吗？         |
| 输入与信号 | 两路的判断吃的是不是同一颗传感器、同一个适配层？      |
| 代码与规格 | 两边有没有同一份库、同一份接口文档、同一个编译器的影子？  |
| 工具与批次 | 谁给两路刷写？器件是不是同一批次？             |
| 环境与位置 | 同一处发热、同一次振动、同一个连接器，会不会一次够到两路？ |

这张桌子上，那天下午立了三桩案子。第一桩由小林自己举手：两条通道的转矩样本都经过同一个信号适配层，而软件那边最近暴露过一个毛病——适配层把“到达的时刻”盖在样本上当作“采样的时刻”。一条在队列里滞留的旧样本，两条通道会同时把它判成新鲜的。功能上再不同的两套判断，喝的是同一口被污染的井水。第二桩由吴工画出来：两路的供电各有稳压，但向上追两级，汇在同一个上游——一次足够深的电压跌落，可能先放倒监控、再放倒主路，前后脚，不需要同一瞬间。第三桩由梁工提出：按现在的产线设想，两条通道由同一台刷写设备、同一份配置源写入——一次错误的配置，可以在下线那一刻同时写坏两路。

要紧的是接下来怎么处置。立案不是判决：每一桩都要继续回答——这个原因可信吗，真发生时两条路是否都会会在时间窗口内失守，用什么措施看住它，措施又被什么证据验证。措施有三条路可走：断掉根源（把采样时刻在源头写死，传输途中不许覆盖）、挡住影响（两路各自用自己可信的时间基核对样本年龄）、拆开耦合（给监控通道

一个独立的时间来源)。而措施定了不算完——郑工的台架为此排进了两项新试验：供电跌落和报文乱序，逐路观察系统是否仍在窗口内转入安全反应。关掉时间戳这一桩，供电那桩不会跟着关；每一桩各有各的证据、各有各的负责人。

走查也不是从零开始的。它有两个上游供线索：故障树分析从顶事件往下拆——“两条路同时漏检”这个顶事件本该是一个“与”门，两条腿都断才塌；可如果“共享的旧样本”这同一个基础事件出现在两条腿底下，冗余在树上当场退化成单点，树把重复照了出来。失效模式与影响分析从底下往上推——逐个元件问“你坏了会怎样”，当同一类失效模式在两条通道里反复出现，嫌疑就该上桌。但两者只负责生成嫌疑：树上的重复事件、表里的相似模式，可不可信、在这版架构里是否真有那条路径，要由走查对着原理图和代码一条条判。分析是接力，不是替身。

散会前小蔡问了她的问題：“上个月软件那边不是出过一份‘互不干扰’的分析吗？高等级和低等级的任务住同一颗芯片，证明低的不抢时间、不越内存、不堵通信。拿它当独立性证据，行不行？”

不行，但这个问题问得好，因为它逼出了一条容易踩的边界。同住一个元素里的高低等级部分，默认要全部按最高等级开发；想让低等级的部分维持低等级，就得证明它不给高等级的邻居添乱——不抢它的时间、不碰它的内存、不堵它的通信。这是**共存**的证明，回答的是“同屋的人别惹事”。而分解要的是“两条路各自成事，并且不同命”——共存是其中很窄的一小块，它管不到两路共享的供电、传感器和代码谱系。把共存报告当独立性证据，就是拿一间屋子的门锁，冒充两栋房子的地基。

走查的记录最后长成这样一句话的形状：在声明的架构版本和运行模式内，沿着这些缝逐条查过；这几条有可信的嫌疑，各由某项措施看住，措施经某项试验验证；这几条凭这些理由排除；这几条还没有答案，开着。它不好看，但它可以被反驳——任何人都能指着某一条问：你凭什么排除？你的措施验证了吗？还有一条为什么还开着？

**“独立”不是“没发现问题”，而是一句有分母的结论：查过哪里、凭什么排除、剩下的原因由谁看住、看住的证据在哪里。**

这套问法立住了。但它是转向系统专属的吗？出了这间会议室，还成立吗？

## 8.5 换一个世界：ENV-01 的两只探头

那台工业环境监测单元 ENV-01 的团队，为了“更可靠”，给关键测点装了两只同型探头，读数取平均，互相校验，任何一只失效都能被另一只发现——听起来是不是很熟悉？

用同一套问法追下去：两只探头是同一个批次；出厂时对着同一台参考仪器校准，校准系数存在同一份配置文件里；固件里跑的是同一个滤波算法；供电来自同一块板子。如果参考仪器那天偏了，两只探头带着同一个偏差出厂，取平均只会把这个偏差算得更“稳定”；如果滤波算法在某种阶跃下都失真，互相校验永远一致——一致地错。两只探头，平均的是两份同一个错误。

可迁移的仍然只是问法——“这两条路在哪里共享同一个命运”。答案不许迁移：ENV-01 的缝要在它自己的架构里找，它的措施要用它自己的试验来验证。问法能走出行业。但还有一个更近的问题：那份走查记录本身，能走过时间吗——三个月后，它还活着吗？

## 8.6 纸面查不动的缝

必须先说公道话：走到这一步，已经是 AI 之前的世界里最好的做法了。分解合同两页都读了，等级的括号记着来源，走查有清单、有分母、有开着的案子——几十年里，这套纪律拦下过无数个“两路了就独立了”。第 1 章那张地图上，“多一条通道为何不等于独立”这个断面，纸面的答案就到这里，而且答得相当体面。

但只要把时间拉长，就能看见三道纸面查不动的缝。

**清单的完整性押在伤疤上。**那天下午最有价值的三个问题，一个来自小林刚踩过的坑，一个来自吴工手上的原理图，一个来自梁工两百多块报废样板换来的记忆。换一个没有伤疤的团队，同一张图，走查会短一半——而纪要上同样写着“已完成走查”。清单可以印成模板，“哪里算查够了”却没有底：缝不会自己报名，它等着一个恰好付过那笔学费的人路过。专家的记忆是这套做法真正的数据库，而记忆不可移交、不可盘点、会退休。

**架构图不画共享。**图纸的天性是画部件和连线：两个框、两条走线，清清楚楚。可那天立案的三桩，没有一桩是图上的一条线——“同一份代码库”“同一个适配层”“同一台刷写设备”“同一批次器件”，全都不是几何对象。最危险的耦合，恰恰都画不出来。于是评审看图，图上永远是两条路；依赖住在图外，住在代码仓库、采购记录和产线工艺文件里，没有任何一张视图把它们摆到同一双眼睛前。

**排除理由不跟着版本活。**走查记录里写着：“两路使用不同传感器，共享输入一项排除。”这句话为真，押在“不同传感器”这个事实上。三个月后原理图改版，为了降成本把两路合到同一颗传感器上——那行排除理由不会自己失效，纪要不会弹出警告，走查报告安安静静躺在文档库里，结论还是“独立”。纸面上，结论和它押着的事实之间没有任何活的连接；事实变了，只能靠又一个人恰好记得当年那行字。走查可以再开一次，但“哪些结论需要重开”这个问题本身，纸面回答不了。

**走查记录写在纸上，依赖活在版本里：事实变了，纸不会自己举手——这是纸面独立性的边界。**

所以本章末尾冻结的，是三个只能留给后十章的问题：给定一版架构和它背后的全部共享事实——器件、代码、工具、批次——能不能让机器回答“这两个元素之间存在哪些耦合路径”；给定一条排除理由，能不能问出它押在哪些事实上、其中哪一个已经变了；给定一份走查记录，能不能随时查出哪些案子还开着、谁在看守。第 18 章会从这三个问题出发，把依赖与独立做成机器可以检查的结构——那是 AI 之后的世界的做法。



而眼前，还有一桩立了案却还没找到主场的事。那天走查的第三桩——同一台刷写设备、同一份配置源写两路——它的战场根本不在分析室，在工厂。分析里的独立性成立了，靠的是一整套边界：不同的供电、不同的批次、不同的配置来源、独立的时间基。可这些边界是分析的语言，图纸移交工厂之后，产线认识它们吗？换一颗“等效”的代换料，会不会把两路换成同一批次？维修站换件重刷，会不会用同一份配置把两条通道一起写错？设计在分析里挣来的每一分独立，制造和服务的每一个环节都有机会在无意间还回去。第 9 章就从这里开始：设计的含义走进工厂之后，还能不能连续。

导读的承诺可以兑现了。合上书之前，做三件事：找一张你项目里画着两条通道的图，沿供电、时钟、输入、代码、工具五条缝各追一次，数数有几条最终汇在同一点；找一份写着“独立”或“互为备份”的结论，问出它的分母——查过哪里、凭什么排除；再挑其中一条排除理由，把它押着的事实写下来，问一句：这个事实要是变了，今天谁会知道？如果答案是“没有人”——记住这个位置，第 18 章会回到这里。

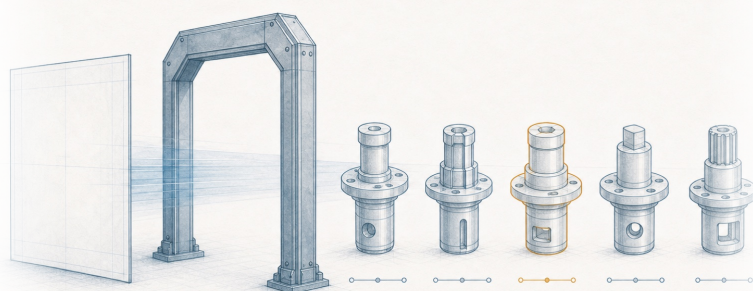
**本章注记：**本章的评审场景、走查过程、人物与全部产品细节（双通道方案、供应商往事中的电源树与代码库缺陷、样板报废数量与延期时长、ENV-01 的探头配置等）均为合成教学材料，不对应任何真实企业、真实产品或真实个人。正文对功能安全标准中 ASIL 分解、元素共存与相关失效分析要求的转述为自然语言概括，精确要求以标准原文为准；文中的允许组合与分析方法描述不构成对任何实际项目的裁决，也不表示任何架构已获得独立性结论或任何产品已获放行。





## 第九章 设计走进工厂之后： 生产、运行、服务与报废

Semantica 绑定：本章保存生产、现场、服务和退役的工程叙事；唯一机器语义是 `semantica.chapter_packages.vol2.ch09`，主场景为 `semantica.vol2.ch09.scenario.primary`。本体、CQ、查询、Shape、案例、规则、oracle、PROV 与 receipt 只在 Semantica，本目录没有 fixture 或执行副本。当前为 `partial`、`release blocked`；教学链路通过不构成真实批次、配置或现场事实。



**【导读】** 相关失效分析收尾的那天，白板上两条通道之间的独立性终于站住了。下一站是工厂——图纸、软件镜像、标定数据打包移交，设计的工作看起来结束了。本章讲的是这之后的事：同一张图纸，怎样变成十万颗不完全相同的件；设计分析押下的前提，怎样在批次、工位、代换料和维修之间悄悄失守；一颗件离开厂门之后，服务站、拆解线和现场工单，又怎样继续——或者停止——讲述它的身份。读完本章，你应当能沿着一颗件从图纸走到报废，指出设计的含义可能断开的每一个关口，并说出每个关口应当留下什么证据。

## 9.1 移交会上，工厂接过的是什么

独立性走查的最后一页翻过去之后，陈工在白板上把箭头往右画了一格：“下一站，工厂。”RC17的设计包开始向制造移交：图纸、公差、软件镜像、标定数据、装配顺序，一样一样清点交接。会议桌另一头坐着产线工艺负责人老何——五十二岁，从装配线的操作工干起，在这家工厂待了二十六年，带过两代产品从试制走到量产爬坡。他随身带着一个翻得起了毛边的活页夹，里面是他自己整理的工艺卡，有几页的年纪比会议室里一半的人的工龄都长。

散会时陈工点了小蔡的名：“你在图纸的世界里待了半年了，去老何那里跟一个月线，看看件的世界。”

跟线第一天，小蔡在来料区看着叉车卸货，说出了在一个在设计楼里人人都会同意的判断：“图纸冻结了，公差标齐了，软件镜像也封版了。产线照图施工，做出来的不就是设计定义的那个东西吗？一致性，说到底不就是‘照做’两个字？”

这句话一点都不外行。图纸本来就是为了被照做而存在的；公差本来就是为了把“照做”变成可检验的承诺。整个制造工程的传统，就建立在“图纸说清楚、产线做到位”这八个字上。如果这个判断不成立，问题一定藏在“照做”两个字盖住的某个地方。

老何没有争论。他带小蔡在厂里走了三站。

第一站，来料区。同一个图号的转矩传感器弹性体，货架上并排放着两家供应商的料箱，同一图号下面是不同的批次号。“图纸只有一张，”老何说，“料是一批一批来的。两家都合格，但两家的‘合格’不是同一种脾气——热处理曲线差一点，批与批之间的分散就差一点。图纸不知道这些，批次知道。”

第二站，拧紧工位。老何调出这台拧紧机最近三个月的记录：同一个程序号，上午和下午的扭矩落点有肉眼可见的漂移，换了一批螺栓之后，摩擦系数变了，同样的扭矩拧出来的预紧力不是同一个数。“图纸写的是‘应该拧多紧’，这台机器交出来的是‘今天下午实际拧了多紧’。中间隔着刀具磨损、换班、换料和这台机器自己的年纪。”

第三站，返修区。工位架上单独放着一颗控制器，标签上挂着它的经历：首次刷写中断，重刷，复测通过。“它和线上顺顺当当下来的那些，最终检验结果一模一样。但它多了一段历史。图纸上没有‘历史’这一栏。”

回办公室的路上，小蔡试着抢救自己的判断。把图纸写得更细呢？写不进去——图纸描述的是“这一类产品应该是什么”，而批次的脾气、下午的漂移、返修的经历，都长在“这一颗”身上；类的描述再细，也覆盖不了个体的经历。那把来料检验收得更严呢？检验只能按规格书查，而“规格书之内”和“设计假设之内”不是同一个圈——这句话的分量，小蔡两周之后才真正掂出来。那加密抽检呢？抽检回答的是批次的分布，回答不了“这一颗”；而装到车上的，从来都是这一颗。

图纸定义的是一类产品，产线交付的是一颗颗具体的件；一致性不是“照做”的自动结果，而是一条从设计前提通到每一颗件的、需要有人主动维护的链。

可这条链的第一环，还不在于工厂这边。产线要守住设计的含义，先得知道设计的含义押在哪里。除了图纸，设计还欠工厂一份什么？

## 9.2 图纸之外：设计欠产线一份前提清单

设计分析不是悬在真空里的。转矩限制的论证，假设标定数据被正确完整地写进控制器；诊断报警的阈值，是按选定器件的实测行为整定的；上一章刚刚站住的独立性论证，假设两条通道不会消费同一份错误的刷写包——如果烧录工位把同一个错包写进两路，两条通道在分析里再独立，也在工位上被合并成了一条。图纸移交工厂，制造还认识这些边界吗？答案很残酷：工厂不读安全分析报告，工厂读工艺卡。设计押下的前提如果不被翻译出来，产线连要守什么都不知道。

所以移交包里必须有第二份东西：一份点名清单——在成千上万个参数里，指出那几个“漂了会掏空安全分析”的量。标定数据与软件版本的正确性是一个；转矩传感器末端标定是一个；那颗弹性体的热处理状态可能是一个。点名的判据不是“重要”这种感觉，而是一条说得出口的因果：这个量偏离之后，沿什么路径，动摇哪条安全结论。点不出因果的，回到常规质量管理去；点得出的，就要在产线上配一道对应的控制。

这里容易犯一个分类错误：既然读回比对能拦住错误软件出厂，它算不算车上的安全机制？做一个删除试验就清楚了。车开出厂门，把烧录工位和读回工具全部拆掉，车上的转矩监控还得继续工作——它是车的一部分；而读回比对的职责在产品放行那一刻已经完成——它是产线的纪律。两者都为同一个安全目标服务，但工作的时间不同、地点不同、证据也不同。把产线控制当成车上的机制，会拿错证据来证明它；把车上的机制当成产线控制，会以为出厂测一次就够了。

清单点了名，控制怎么写？小蔡在工艺评审上见识了一行字的歧义。草案里写着：“烧录后检查版本。”她以为这已经够清楚了，直到老何让两个班组各自复述一遍怎么执行。甲班说：看烧录工具的日志，写入成功就过。乙班说：从控制器里把版本读回来，跟目标比。再问目标从哪来——甲班用全线通用的“最新版清单”，乙班按这辆车的身份去查该装哪一版。再问不一致怎么办——一组重刷了继续走，一组停线上报。同一行字，两套做法、三处分岔，每一套都自认为在“照做”。

一行意图裁决不了这些分歧，能裁决的是一张控制卡：

| 控制卡要回答的 | RC17 软件与标定这一项的答案              |
|---------|-------------------------------|
| 控制什么    | 软件与标定组合的正确性，不是“烧录动作完成”        |
| 什么时候查   | 最后一次可改写动作之后、产品放行之前；返工后重查      |
| 目标从哪来   | 按这一颗件、这一辆车的受控配置取，不用“目录里最新的”   |
| 怎么查     | 从件里读回身份与校验值，与目标比对，不只信工具说“我写过” |
| 什么算过    | 读回与目标一致；不一致即扣留，不允许自动放行        |
| 不一致怎么办  | 隔离这一颗、圈定嫌疑范围、查过程原因，有权者决定恢复    |
| 留下什么    | 日期、这一颗的身份、目标值、读回值、判定、所用工具版本   |

第 1 章那张生产写入绿卡背后，就是这套动作——也所以它的射程当时被说得很克制：它证明声明工位和批次上镜像写入与校验这个动作正确，既不评价镜像内容的工程充分性，也不拥有放行的权限。现在你能看到这个射程声明的另一面：动作本身要成立，背后就得有这一整张卡。

有人会说，写进作业指导书不就行了？指导书写的是正常路径，而两个班组的分歧全部发生在指导书没写的地方——目标服务超时了用不用缓存、读回失败了重试几次、返工件回到哪一步。控制卡的价值恰恰在异常分支：正常路径人人会做，区分好坏的是出事时仍然只有一种做法。也有人说，那就全检，全检总保险。全检检不掉方法错误：用错误的目标清单对每一颗都检一遍，得到的是百分之百一致的错误结论。

**产线的控制不是把设计再验证一遍，而是守住设计分析赖以成立的那几个前提；前提不点名，工厂就只能守住图纸上看得见的东西。**

清单和控制卡管得住工位，却管不住世界。料是会断的，交期是会爆的，而设计冻结的那一刻，没有人承诺过全世界跟着冻结。缺料的那一天，“等效代换”四个字就会被摆上桌——老何对这四个字的分量，有切肤的记忆。

**读图任务：**从顶层设计定义向下跟随扇形展开的连线，分清批次和单件两层，再找到被单独展开时间线的那一件实物。



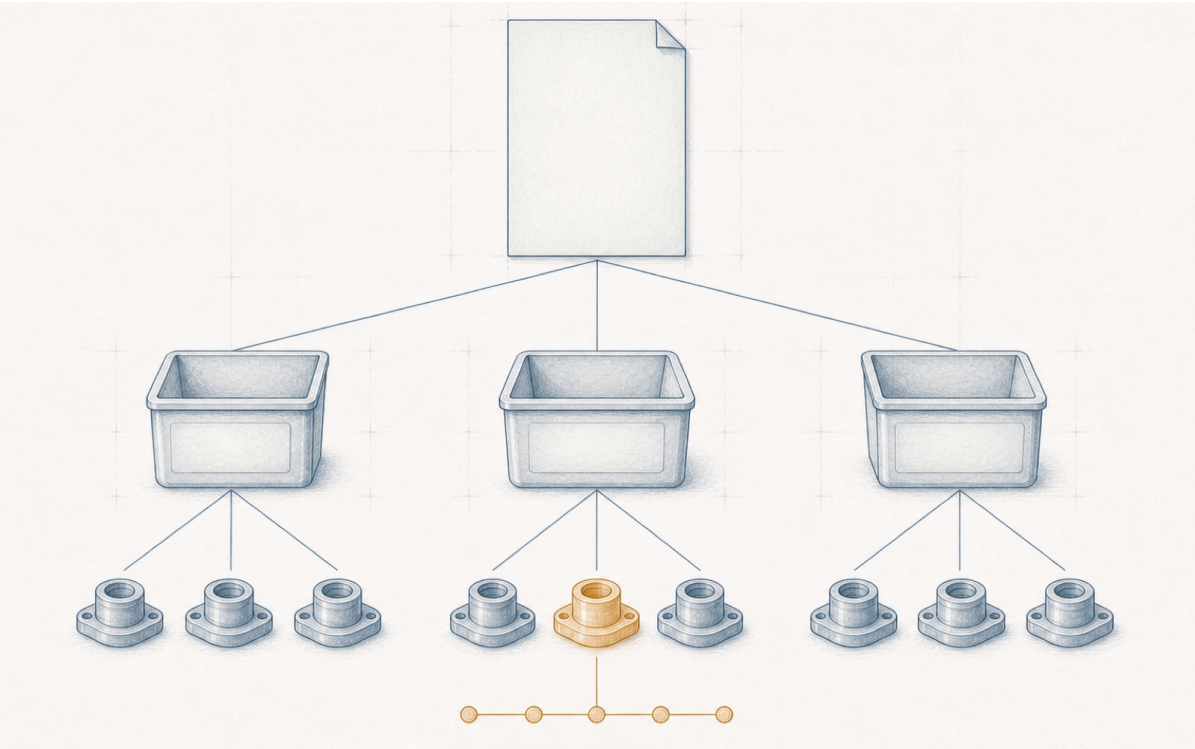


图 9-1 设计定义、批次和单件是可关联但不可互换的身份层。现场结论要回到某一实物，就必须保留它从制造到服务的连续时间账。

**图后判断：**本图不声称一个批次内的个体完全相同，也不单凭图上连线证明追溯真实完整；实际身份仍需受控标识与原始记录支持。

### 9.3 一张对照表的代价：等效代换料

那是几年前，上一代转向控制器量产的第二年。一颗电流采样电阻的原厂突然把交期拉到十四周，产线库存只够三周。采购很快找到一颗等效料：规格书对照表列了十二行——阻值、精度、封装、额定功率、耐压……逐行相同。代换单送到老何桌上，他核了对照表，签了字，说了一句后来在复盘会上被原样复述的话：“规格书一项一项对过，参数完全等效，采购手续也齐全——这不叫改设计，这叫换个牌子买同一颗料。”

这句话在当时挑不出毛病。等效代换是这个行业每天都在做的事；规格书就是器件的合同，合同条款逐项相同，凭什么说它们不是同一颗料？来料检验合格，试装合格，产线末端测试全绿。三个月里没有任何异常。

入冬之后，北方市场的投诉进来了：助力时不时进入降级模式，仪表报警，过一会儿又自己恢复。排查花了三周——故障只认低温，车一回到温暖的车间就什么都测不出来，投诉车和没事的车在台账上长得一模一样，机理最后是把投诉车上拆下的采样电阻放进温箱里一条一条测出来的。定位到的那一刻，所有人沉默：新料的温度系数在低温端走的是另一条曲线——仍在规格书限值之内，可诊断窗口从来不是按规格书限

值开的。窗口开多宽是一场交易：开到规格限那么宽，什么料都容得下，但真正的故障也要漂到那么远才会被抓住；设计当年手里有原厂料好几批的实测曲线，分散比规格限窄得多，就把窗口贴着实测分布收了进来，把省下的那段空间换成了诊断的灵敏度。所以原厂料永远出不了窗——这扇窗本来就是按它的身材裁的；而代换料只承诺过不出规格限，从没承诺过不出这扇窗。新料在零下的偏移把窗口顶破了，好好的车被自己的诊断报成了嫌疑犯。而那张十二行的对照表上，没有“温度系数”这一行——就算加上，逐行对出来的也只是两家共同的规格限，对不出这扇按一家实测裁出来的窗。

真正的代价在追回的时候才显出来。代换没有改料号，没有分批次标识；原厂先前订下的料还在断断续续到货，代换料补在缺口上，新旧两种料就这样在线上混着用了三个月，台账只记到图号。“哪些件装了代换料”这个问题，没有任何系统答得上来。嫌疑范围圈不出来，就只能按时间窗全量追回：三个月里发运的两千一百台控制器全部召回复检，事后统计，真正装了代换料的不到七百台。停线两天，返工一个月。复盘会上老何说了那句后来被写进工厂培训材料的话：“多追回的那一千四百台，赔的不是那颗电阻的钱，是我们答不出‘哪些件装了它’的钱。”

这场事故最扎人的地方，是没有人违规。采购按流程询价比价，检验按规格书逐项验收，产线按图施工。每个环节都对，断掉的是一条不在任何人职责清单上的回流：代换申请没有回到设计评估。只有做过那份分析的人知道，诊断窗口押的不是规格书里的十二行承诺，而是原厂料一条没写进规格书的实测曲线。

自然的补救方案在复盘会上一个个被摆出来，又一个个被放下。把对照表加长？加到多长算够？你不知道设计押了哪些行为，表就永远缺一行——这次缺的是温度系数，下次可能是老化曲线、开关噪声或者批间分散。加严来料检验？检验的依据还是规格书，而规格书本身就是那张不完整的表。以后所有代换一律全项实测？不知道前提，就不知道该测哪个量——把所有想得到的量都测一遍，测的还是“想得到的”。三条路殊途同归地撞在同一堵墙上：“等效”不是一个采购问题，甚至不是一个质量问题。

**“等效”是一个设计结论，不是一张规格书对照表；任何没有回到设计评估的代换，都是一次没有留下记录的设计变更。**

回头看，这道断口不是哪个人挖的，是信息的走向挖的。器件跨过公司边界时，随身只带一份文件——规格书；采购的询价、检验的验收、商务的比价，整条流程都围着这份文件转，流程的眼睛天生只看得见写进规格书的东西，而设计押下的恰恰是规格书之外的那条实测曲线——它留在分析报告里，从来不跟着器件走。再算一笔账就更清楚：缺料停线的损失当天可见、按小时累加，催单的电话一个接一个；设计前提被击穿的损失要等到冬天才结账，还未必结到当初签字的人头上。信息只走规格书，压力只压当下——不需要任何人失职，组织自己就会把这件事做错。

从那以后，这家工厂的代换单上多了一栏“设计评估意见”，产线台账记到了料源批次。厂门之内的链，算是被一场事故焊结实了几节。可一颗件的一生并不在厂门之内结束。车开走之后，谁接着守？

## 9.4 厂门不是终点：服务、维修与报废

车辆出厂两年后进了服务站：控制器硬件故障，换件。新备件从库房出来时只有基础引导程序，软件和标定要现场刷写。服务工具列出可用的软件包，按日期排序，最新的在最上面。技师的手悬在屏幕上——换新件、刷最新版，这还用想吗？

这个直觉一点都不外行。小蔡在旁边跟着服务流程试点，一眼认出这是她老本行的常识：消费电子的世界里，“升到最新”几乎永远是对的，新版本修了旧毛病，兼容性由平台兜底。但车队不是手机。路上跑着的是多年版本叠加的世界：停产的硬件、历史的标定、地区的变体、做过和没做过技术升级的车混在一起。“最新”对这辆车的硬件组合可能根本不适用；“回退旧版”又可能把修掉的问题重新装回去。

试点期间真的发生了一次。一辆旧硬件的车换件后，技师按习惯选了最新软件包，服务工具弹出不兼容告警。技师的第一反应是工具误报——电话打回厂里，转到软件负责人小林那里，小林查了配置记录：那一版软件从某个硬件修订之后才支持，刷上去，转矩标定的接口对不上。这辆车在店里多趴了两天，白刷了一次，代价不大——但所有人都听懂了这次告警的言外之意：如果服务工具没有这道检查，一颗“焕然一新”的控制器就会带着错误的软件回到路上，而工厂里那张控制卡的所有纪律，在服务站这里本来一条都不存在。

这就是看待服务的正确角度：服务站是一个没有产线门禁的再制造工位。它做的事和产线一样——写入配置、更换部件、放行产品——却天然缺少产线的三道纪律：按这辆车的身份取目标，而不是按“最新”；写完读回比对，而不是相信工具说“我写过”；记录旧件退出、新件装入、目标与实际，而不是在工单上写一句“已更换，试车正常”。第1章绿卡上的“镜像与校验和”，必须一路延伸到每一个举着诊断仪的技师手里。指望技师“更小心”是不够的——产线尚且不靠操作员的小心，服务凭什么靠？

再往后走，被保护的人还在变。写给技师的说明可以说“执行标定程序并核对版本身份”，写给驾驶员的说明必须说人话：这个报警亮了还能不能开、开多快、多久之内要进站——把内部错误码列表塞给用户，不构成任何保护。到了拆解线，关心的人变成拆解工：断电之后哪里还储着能、哪个动作会让执行器意外动一下。而报废也未必是终点：拆下来的控制器如果流进二手件渠道，它会带着旧配置、旧数据和一段没人知道的历史回到另一辆车上——身份链如果在报废这一站被剪断，它就会在你看不见的地方重新接上，而且接错。

**离开工厂的每一次维修，都是一次没有产线门禁的再制造；服务需要的不是更细心的技师，而是和产线同一条身份与配置的纪律——而这条纪律，要一直守到拆解线的最后一颗件。**

服务记录、换件历史、用户反馈、回收件分析——现场每天都在产出数据。一个自然的推论随之而来：有了这些数据，闭环不就形成了吗？



## 9.5 现场的回声：数据什么时候才算证据

量产准备评审上，有一页周报写着：“现场监控已运行。”老何对这句话的警惕，和陈工对“都通过了”的警惕如出一辙。他问了三个问题：工单能不能按车、按批次、按配置查？查出来的东西有没有人定期分析？分析出问题之后，谁、按什么判据、在多长时间以内决定行动？三问有一问答不上，“已运行”就得缩回去——那只是“有一根数据管子”。

这三问正是现场闭环的三段：数据要可分析，分析要真发生，问题要连着行动。三段缺一不可，而最容易缺的是第一段的一个定语：“可分析”。一堆写着“方向有点沉”的自由文本工单，配不上“证据”两个字——除非它们能关联到具体的车、当时的配置、生产的时间窗和料源的批次。老何的事故正好从反面作证：当年如果工单和台账能按料源批次检索，嫌疑范围一个小时就能圈定；现实是三周排查加一千四百台冤枉的追回。数据不是越多越好——收集更多不可关联的数据，只是把堆变大；上一套更大的售后系统也不解决问题——系统负责存储，不负责关联。关联靠的是当初在产线和服务站一站一站留下来的身份链。

这个问题不只属于汽车。第1章那台环境监测单元 ENV-01 的出货绿卡里，有一张“生产写入与序列号绑定完成”。两年后的现场，一台设备的传感器模块坏了，服务商换了新模块——序列号还是那一台，可当年绑在这个序列号上的校准证据，还属于它吗？换件记录如果只写“已更换”，这台设备从此就有了两段接不上的历史。问法和 RC17 的世界一模一样：身份靠什么连续、换件怎么记、谁有权说“这还是同一台”。但答案必须在 ENV-01 自己的世界里重新挣——它的校准周期、它的部件粒度、它的服务商网络，一样都不能从汽车行业抄。

**现场数据只有能回到“哪一颗、哪一版、哪个时间窗”，才成为证据；回不去的数据，只是繁忙的噪声。**

到这里，可以把这一章沿途留下的东西数一遍：设计有分析，还有一份点名的前提清单；产线有控制卡，有每一颗件的记录；代换有设计评估的回流；服务有身份与配置的纪律；现场有三段闭环。从设计到现场，每一环都有了自己的证据。可是这条链靠什么连着？

## 9.6 纸面的身份链，到这里为止

先把敬意说足。以上这一整套——特殊量的点名、控制计划、单件追溯、服务纪律、现场闭环——是这个行业在没有 AI 的年代用几十年和无数次追回换来的做法，而且它真的有效：正因为有这套纪律，老何的事故才能在三周内定位，而不是永远成为“北方冬天的玄学”。这是纸面世界的最好水平。

但在现场多待一个月，就能看见这个最好水平的三处边界。

**链上的每一个铆钉都是人。**图号连着料源，料源连着批次，批次连着序列号，序列号连着车辆，车辆连着工单——这条链横跨图纸系统、制造台账和售后系统，每一次跨

系统的对账都靠人工核对。人手一紧、交接一乱，铆钉就松。老何多追回的那一千四百台，就是一枚铆钉锈掉的市场价。

**前提清单活在专家的头脑里。**设计押了哪颗料的哪些没写进规格书的行为，不存在于任何可查的台账。老何签代换单的那天，没有任何系统能提醒他：这颗电阻上押着一个诊断窗口。事故之后加的那栏“设计评估意见”，本质上是把问题转给了另一群人的记忆——做过那份分析的工程师在，链就在；他调岗、退休，链就断。纸面能记下结论，记不下结论押在哪里。

**现场的语言回不到设计的语言。**工单写“方向有点沉”，设计写“采样偏移”；工单按投诉分类，设计按失效机理分类。两种语言之间的每一次翻译都要人现场重做，翻译的质量取决于谁当班。信号明明回来了，却对不上发出它的那颗件、那版设计、那条前提——回声在半路散掉。

所以本章末尾冻结的，是三个纸面答不了的问题：给定现场的一颗件，怎样让机器答出它从设计定义、料源批次、单件记录到服务历史的完整身份；给定一次代换或偏差，怎样让机器答出它击穿哪些设计前提、波及哪个范围；给定一批工单，怎样让机器把它们自动关联回批次与配置。第 19 章会从这三个问题出发，把设计与现场之间这条靠人维护的链，变成机器可查的结构——那是 AI 之后的世界的做法。

而眼前还有一个更近的问题。RC17 的放行窗口就在下个月。现在，从设计到现场，每一环都有了自己的证据：概念有分析，系统有需求，硬件有度量，软件有回归，独立性有走查，产线有控制记录，现场有监控。把它们全部搬回第 1 章那张放行桌，摞成一摞——够了吗？这一摞纸彼此认识吗？它们说的是同一个候选、同一个基线吗？第 10 章回到那间会议室，做前十章的最后一件事：把六张绿卡，整理成一份敢签字的安全案例。

导读的承诺可以兑现了。合上书之前，做三件事：在你的产品里挑一颗关键料，去问设计者“你的分析押了它规格书之外的哪些行为”，看多久能得到答案；追一颗具体单件的完整身份历史——从图号到现场——数一数要打开几个系统、问几个人；再找一份维修或换件说明，看它靠什么保证换上去的件适用于“这一台”。哪一件事卡住了，就记住卡住的位置——第 19 章会回到那里。

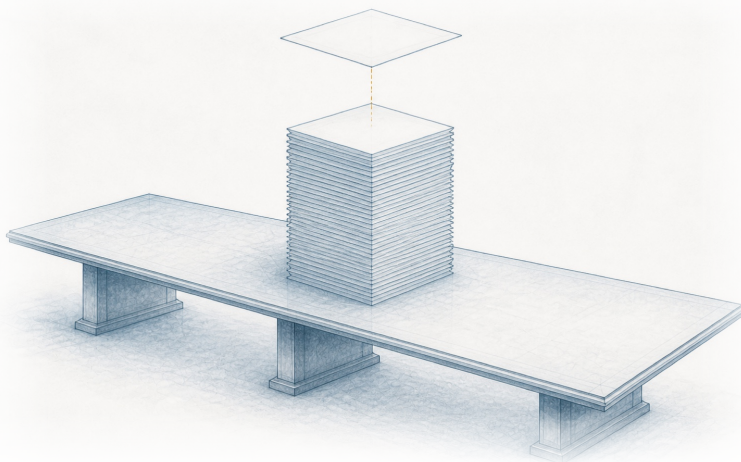
**本章注记：**本章的人物、移交与跟线场景、代换料事故、服务站事件与全部产品数字（候选代号、版本号、台数、时间跨度、排查周期等）均为合成教学材料，不对应任何真实企业、真实产品或真实个人；文中的器件失效机理与诊断行为只用于说明设计前提与制造、服务之间证据连续性的一般规律，不构成对任何实际器件、系统或供应链行为的断言。正文对生产、运行、服务与报废相关标准要求的转述为自然语言概括，精确表述以标准原文为准；本章不表示任何产品已获放行或任何现场结论已被接受。





## 第十章 回到放行桌： 支撑过程与安全案例

Semantica 绑定：本章保存工具、配置、文档、安全案例与放行边界的可读规范；唯一机器语义是 `semantica.chapter_packages.vol2.ch10`，主场景为 `semantica.vol2.ch10.scenario.primary`。本体、CQ、查询、Shape、案例、规则、oracle、manifest 与 receipt 均在 Semantica。当前为 `partial`、release `blocked`；多个局部 PASS 不会自动合成为一次有权放行。



**【导读】** 六周之后，第 1 章那间会议室重新亮灯：还是候选 RC17，还是那六张绿卡，只是桌上多出三箱这些日子攒下的新证据，而围坐的人已经换了一副语言。本章是前半部书的收口：陪这支团队把一摞文件整理成一份主张—论证—证据三层的安全案例，走到纸面世界能达到的最好状态；顺便把一直躲在幕后的手艺请到台前——接口协议、配置、变更、文档、工具与复用，它们不产生任何一页证据，却决定证据还连不连得上。最后，我们会诚实地承认这份纸面案例到不了三个地方，并把前十章的账一次冻结。读完本章，你应当能把一堆各自为政的“通过”报告整理成一份能被反驳的安全案例，并说出纸面案例的三个极限。

## 10.1 三百多份文件，合在一起说的是哪句话

会议室的门再次推开时，最先进来的是一辆推车。从设计到现场，每一环都有了自己的证据，现在它们被物理地搬回了放行桌：概念、系统、硬件、软件、走查、生产与现场，这几周各个专业交来的产出，小蔡按专业和日期分好了类，装了三个文件箱，三百多份。

“上次评审缺的，这回都补上了。”她说，“比上次厚了三倍。我照标准的目录做了一张清单，每一项都能对上号。装订好交上去，应该就齐了吧？”

这句话不外行。很多公司的放行包正是这样交付的：按标准章节立目录，逐项打勾，审核来了按目录抽查。文件在、格式对、签名全，是每一次审核都真实检查的东西，小蔡把这件事做到了无可挑剔。

陈工翻了翻最上面那一箱，问了一个问题：“这三百多份文件，合在一起，说的是哪一句话？”

没有人答得上来。每份文件各自说了一小句——这版硬件的失效率有来源，这组用例在那套配置上通过，这条产线的写入有校验和——可是“RC17 可以放行”这句大话，哪一页都没有说，也没有哪一页有资格说。三百句小话堆在一起，还是没有那句大话。

有人提议在清单上再进一步：给每份文件标注它支撑标准的哪一部分，凑齐了就算完整。这个方案的毛病上一次评审已经领教过——清单证明文件“在”，证明不了文件“支持”；目录齐全的档案馆里，照样可以一句结论都没有。又有人提议请一位资深评审人通读全部材料，写一篇总评，让总评替大家说那句大话。这个方案更诚实，却把整个问题塞进了一个人的脑子：总评是又一份文件，它的判断没有结构，别人想反驳都无处着力——反驳它，难道去反驳“我读完了，我觉得行”吗？

两条路都堵住，缺的东西才露出形状。这个行业早就给它起了名字：安全案例——一份针对具体对象、论证安全已经达成的文书。它不是文件的总和，而是一场写在纸上的辩护：最上面是一句写清了部件的主张，最下面是各自射程有限的证据，中间是一层论证，写明这些证据为什么、在什么条件下，合起来足以支撑那句主张。法庭不认一麻袋证词，法庭认的是起诉书、辩护词和证据链各就各位。

**一摞文件是档案，不是案例；安全案例是一场写在纸上的辩护——主张在上，证据在下，中间的论证没人写，上下两层就只能隔空相望。**

而主张这一层，其实已经有人写好了。陈工从自己的文件夹里抽出几页纸，推到桌子中央——小唐的主张部件清单，六周前散会时布置的作业：主语、配置、关注、情境与时间、假设、决定范围，一个部件一格，第一行填的就是 RC17 和它的五项配置。“骨架用它。”陈工说，“上次想签字的人，把能签的话拆开又装了回去。今天我们把肉挂上去。”

骨架立起来了，可肉不是新的——还是那六张绿卡，加上这几周的补测和分析。他们还认得这六周学会的语言吗？一张一张挂上去，会挂成什么样子？

## 10.2 六张绿卡，重新开口

挂卡用了两个下午。同样六张卡，每一张开口说的话都和六周前不一样了。

概念边界卡先上。它不再笼统地说“分析通过”，而是先报自己的座位——对象是候选相关项的边界，不是哪块控制器、哪个构建——再报自己的担心：几种失常行为在选定场景下的伤害风险，每个风险后面跟着一句安全目标。它挂在主张的“关注”一格下面，作证的是“担心已被点名”，不是“担心已被消除”；这两句话之间隔着后面所有的卡。

硬件分析卡跟上。吴工那边交来的失效率数字，这一次每个都带着宿主、方法和来源：哪颗料、出自哪本手册还是哪批实测、按什么任务剖面折算、不确定度多大。数字有了射程，它支持的小句才写得出来：在声明的剖面与假设内，这版控制器的随机失效风险落在目标之内。假设清单单独成页，挂在主张的“假设”一格——那些被“先信了”的东西，信到什么时候、由谁去验，都写着名字。

软件回归卡由小林重新交上。对象一栏钉死在 SW1.8.3 加 C41；旁边多了一页变更登记：1.8.4 的升版申请仍然在途，他的差异分析圈出了受击的模块与用例范围——申请一旦获准，哪些结论重开、哪些凭明确理由保留，答案提前写好了。一次通过不再默认穿越版本，这是他这些日子学得最贵的一课。

台架卡最引人注目。郑工交了两样东西：一份差异说明，把 P07、预览构建、C42 与目标快照的每项偏差写明，声明这张旧卡支持的是另一句主张，只作旁证；一份定向补测，用目标配置把当初的边界工况重跑了一遍，这才是挂进案例的正证。然后他从抽屉里拿出那页压了许久的异常读数，放进案例的第一部分：“错误候选，在链上还没走到失效；补测未复现，传播和容错都没证据，当前为未知。列开放项，我看守，复现即重开。”他顿了顿，“这一页，我不想再放回抽屉了。”

冲击振动卡照旧支持局部样件的可靠性小句，射程原样声明。走查那几周揪出的那路共享供电——两条监控通道曾经共用一路电，图纸上看着独立——作为反证挂在论证下面，旁边是整改记录和复测结果：反证没有被删掉，它被处置了。生产写入卡由老何补齐了批次一栏：镜像、校验和、工位、批次，设计定义到产线记录的那条身份链，一环一环能指认到具体的东西。

到这里，有人提议让每个专业把自己的卡再加厚一轮，附上全部原始数据，显得更扎实。没有必要——页数不是论证，法庭不按证词的斤两断案。又有人提议省事些：每个专业在卡末自己添一句“因此支持 RC17 放行”。这一句谁都不能写——那是让证人替法官宣判，每张卡的射程都够不着那句大话，六个“因此”加起来还是够不着，第 1 章的旧病不能换个写法再犯一遍。

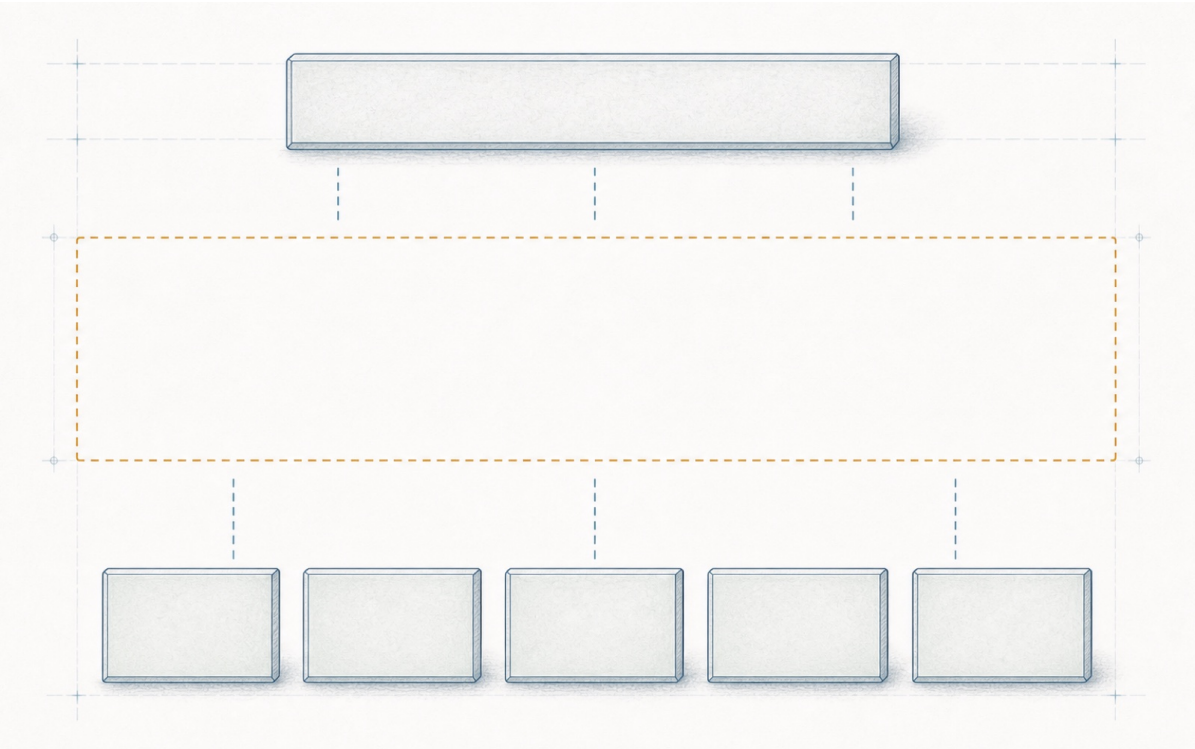
那句大话只能由中间那层来扛。论证这一层，是团队第一次真正合写的东西：从每条安全目标出发，沿着派生的理由链，把系统需求、软硬件需求和它们各自的验证一条条接起来，派生的理由写在明处；每个环节标明谁执行、谁复核、谁有权接受，方工带着他的独立评估从旁挑战——评估的人和拿主意的人分开坐，这是第 3 章付过学费才分开的两把椅子；关注只并列、不折算，安全的小句和可靠性的小句各归各的分支，

谁也不给谁凑总分；缺口不藏，逐条声明：异常读数未知待判，升版申请在途，低温工况的覆盖靠新补的台架而不再靠沉默。

论证是案例里唯一不能由任何单个专业交出的东西：它写明这些射程有限的证据为什么合起来够了，也写明哪里还不够、由谁看守、何时重开。

两个下午收工，骨架挂满，第一次像了回事。可小蔡整理引用清单时发现，这份案例引用的证据来自四个专业、两家单位、七八套工具和三个记录系统。把它们连在骨架上的那些线——谁交给谁、哪版对哪版、改了怎么办、报告是谁生成的——由谁看守？

**读图任务：**先分别看顶部主张和底部证据卡，再把注意力放在中间的大面积空缺，检查虚线是否真的说明了两者之间的支持关系。



**图 10-1** 证据不会自己走向主张。论证必须交代每份证据支持哪个子主张、依赖哪些假设，以及何种缺口或反证会迫使决定重开。

**图后判断：**本图只揭示结构性缺口，不判定底部证据的真伪或充分性，也不因将证据放入案卷就自动产生产品级授权。



### 10.3 看不见的线：六种支撑手艺

第一反应是最自然的那个：各家看好自家的线。软件管软件的版本，硬件管硬件的表格，供应商的证据由供应商自己保管。这正是大多数团队的现状，它的断点也早被大家踩过：线断的地方从来不在各家院子里，而在院墙上——供应商交来的传感器数据算谁复核过，台架的配置该对谁的表，一到边界，“自家看好”这四个字就没了主语。

第二反应是买一套大系统，把所有证据集中进库，统一编号、统一权限。集中确实解决了“找不到”，但库统一的是存放，不是含义：两份编号规整的报告，照样可以一份说的是这版软件、一份说的是那版。而且系统和它的管理员自己成了新问题——小蔡在第2章问过的那句“改台账谁批准”，在这里原样复活。

这个行业给出的答案不是一件工具，而是一组手艺。它们在标准里占了整整一部分，名字都不响亮，做的事却是同一件：看守证据之间的关系。

接口协议看守组织之间的线：哪些活动谁做、哪些信息怎么交、谁有权到对方现场去看，写成双方签署的约定，证据跨过公司边界时才带得动它的责任和射程。配置管理看守版本之间的线：给一组相互匹配的版本一个共同身份——基线——每份证据声明自己绑在哪条基线上，“同一个 RC17”才从口头默契变成可核对的账。变更管理看守时间上的线：从申请、影响分析、有权者的决定，到实施、验证、记录，一环不许跳——小林那页差异分析，就是这条链上“影响分析”的正身，而不是可有可无的附件。文档管理看守每一页自己的身份：标题、作者、批准人、版本、状态，新版不覆盖旧版，旧报告永远还能作它当年那句证。复用看守旧证据的线：那颗用了十年的老料、那段跑了三代产品的底层库，“用了十年”只是故事的开头——同一版本吗？同一用法吗？这十年里它和它周围的变化记在哪？回答不出，年头就折不成信用。

还有第六种，最容易被忘掉。讨论到打包流程时，小林说了一句：“至少工具这块不用操心。回归是流水线跑的，报告是脚本生成的，机器不会手抖。”

这句话不外行，甚至就是自动化的初衷：让机器做搬运和誊写，恰恰是为了消灭人手抖出来的错，而流水线也确实从没有在誊写上错过。但方工接了一个问题：“如果脚本自己错了呢——谁会发现？”会议室安静下来。检查产品的工具，自己也是一个会出错的产品；它的输出有没有人再看一眼，决定了它错的时候是被拦住，还是长驱直入。这个行业连这件事都做成了分级的问法：这件工具错了，会不会把错误引进产品、或者放过本该拦住的错误？真错了，现有流程有多大把握发现？两问的答案合起来，先给这件工具定一个“多不放心”的档——行话把这个档叫工具置信度（TCL）——定在哪一档，才决定它需要为自己攒多少证据。小林的流水线跑了两年，从没有人对它问过这两问。

**支撑过程不产生一页产品证据；它们看守的是证据之间的关系——关系一断，每一页都还是真的，合起来却开始撒谎。**

纪律逐条补上，案例定了稿，放行会定在周四。这份案例，现在是不是终于无懈可击了？

## 10.4 会前一晚，十一时四十分

周三夜里十一时四十分，小蔡还在核最后一遍引用清单。她给自己立了个笨规矩：每份报告的“哪版配置”一行，逐份抄出来对一遍——这个规矩是从第 1 章那张软件射程卡上学来的。抄到软件回归报告时，笔停住了：封面写着 RC17，配置行却写着 SW1.8.4 的预览构建。

打了三个电话才弄清楚。小林的团队为差异分析做过一轮预演，在 1.8.4 的预览工作区里重跑了部分用例，报告模板照常生成了一份新修订；打包脚本的规则是“取最新修订”，于是放行包里那份回归报告，被悄悄换成了预演版。“取最新修订”不是谁的懒惰。脚本写下这条规则的那几年，全组只有一条开发线，报告一版盖一版，新修订永远是旧修订的更正，“取最新”就等于“取最对”——在只有一条基线的世界里，这是个完全正确的默认。预览工作区开出来的那天，世界悄悄变成了两条线，可两条线生成的报告落在同一个目录里，脚本按时间一排，谁最后落盘谁就是“最新”。规则一个字没改，规则底下押着的那个前提没了——而没有任何一个岗位，负责在前提变化的时候去通知一条脚本规则。包里其余文件都绑着 SW1.8.3——两份报告，两条基线，同一个封面日期。每一页都是真的，包含起来在撒谎。而这个包，已经作为定稿发给了全体与会人。

夜里的第一反应是最省事的：把文件换回来、封面改一改，天亮前没人知道。被小林自己按住了——改封面不是改事实，那份预演报告的正文说的就是另一版软件，改了标签，谎只是换了件体面的外衣。第二反应是推迟评审，把三百多份文件全部人肉重查一遍。查，当然要查；但全查解决不了问题本身——这次查完了，下次呢？每次放行都要靠全员通宵对表的流程，不是流程，是赌运气的仪式。

正确的动作没有捷径：撤下错版，追回定稿，取回绑定 SW1.8.3 的那份正身；打包脚本连夜加了一条规则——按基线取件，不按“最新”取件；事故本身写进案例，作为一条已处置的反证留档。代价照单全付：周四的会推迟到下周一，本就顺延过一次的决定窗口又被咬掉一块；小林团队搭进去一个周末，逐份重对引用；客户那边的搭载联络会跟着改期。散会后小林在走廊里说了一句话，后来进了组里的培训材料：“报告是我的组生成的，错也是我的组生成的。机器没手抖，是我们没对它问过那两问。”

下周一的放行会，反而是六周以来最短的一次。案例摊在桌上：小唐的部件清单是第一页，六张卡和补测证据各就各位，论证层写明理由，开放项逐条带着看守人，重开条件写了三条——异常读数复现即重开，1.8.4 获准实施即重开，窗口到期即重开。方工宣读独立评估的结论：建议接受，附条件。陈工逐页翻完，问了最后一轮问题，然后在那句长长的主张下面签了名——不是“EPS 可以交付了”，而是：针对 RC17 这个快照，在声明的关注、情境、假设、缺口与本轮窗口内，现有证据足以支持本轮放行决定，接受，附重开条件。

签完字，陈工把笔搁下，看了一眼郑工。当年冬试事故的复盘会上，有人问过郑工“你的台架不是通过了吗”；今天的案例里，台架那张卡终于只作它够得着的证，差异写

在明处，异常读数摆在第一部分，重开条件里有它的名字。郑工什么也没说，只是把开放项那一页又抚平了一遍。

小唐看着那句他亲手拆开、又亲手装回去的话，也没有说话。六周前他觉得签字是一瞬间的事，现在他知道，那是一整套结构压上来的重量。白板中央，六周前写下的“EPS”还没擦，陈工在旁边补了一行小字：RC17，已接受，附条件。

**能签的从来不是“产品可信”，而是一句带射程、带缺口、带重开条件的接受——这句话，纸面世界今天做到了。**

散会时没有人鼓掌。走出会议室的人心里都清楚，这已经是纸面能到的最好状态——可周三那个深夜也同样清楚地提醒过他们别的事。纸面到不了的地方，究竟有几处、在哪里？

## 10.5 纸面的极限，与一扇门

先试着不认账。有人提议给打包加一道流程：双人复核，两人签字。有用，但它是在给“人会疏忽”这个问题增加更多的人——周三夜里那处错，靠的是小蔡的笨规矩加上运气，双人复核执行到第两百次时的疲劳，和单人没有本质区别。又有人提议把本章全部纪律做成一张总检查单，每次放行走一遍。也有用，但检查单查得出“字段在”，查不出“两份报告说的是不是同一个东西”——它能问“配置行填了吗”，不能替你把三百多份文件的配置行互相对一遍。两个方案都是往纸上再加纸，而病根恰恰长在纸的性质里。

把那个深夜的教训放大，纸面案例到不了的地方一共三个。

第一，证据包没有办法自己核对同一基线。三百多份文件是否都在说同一个快照，纸面上唯一的核对手段是人眼逐行对表。小蔡对出来了，是她的纪律，也是运气；没有任何机制保证下一个深夜也有人恰好停笔。

第二，变更的影响靠人工圈定。小林的差异分析是这份案例里最专业的几页，可它的每一条“受击”与“不受击”，都出自他对这套系统的私人记忆。下一次升版，还得是他；他要是像郑工当年那位前任一样调岗离开，这份手艺不会随文档留下来。

第三，安全案例是死文档，产品是活的。签字那一刻，案例是真的；从下一刻起，产线在写入新的批次，现场在传回新的工况，1.8.4 在路上，而案例躺在库里一动不动。它不会因为世界变了而举手，只能等下一次有人想起来重开它——重开条件写得再好，也得有人来读。

回头看，前面每一章冻结的问题，其实都是这同一道裂缝：职责与签名链的闭合靠人，担心的完整靠专家的眼睛，派生与追溯的维护靠人工，数字的来源靠表格纪律，通过与版本的关系靠人脑，依赖与共因靠记忆，设计到现场的身份链靠台账——如今连案例本身的活性，也靠人的自觉。纸面把语言、职责和结构都立起来了，看守这一切的，却始终只有人的记忆、纪律和加班。把这张桌子换成 ENV-01 的出货评审，名词全换，三个到不了的地方一个不少。

纸面能把语言、职责和结构做到极致；做不到的，是让证据自己对表、让影响自己现身、让案例跟着产品一起活着。

这不是这支团队的失败，更不是这门学科的失败。恰恰相反——几十年里，这个行业把含混的词逼问成术语，把散落的活动组织成生命周期，把签字拆解成职责，把一摞文件组织成辩护。这是没有 AI 的年代里，人类工程共同体所能做到的极限，而且是一个了不起的极限。本书的前半部，写的就是这个极限本身。

但极限就是极限。证据要能自己对表，影响要能自己现身，案例要能跟着产品活着——这三件事需要的不是更厚的纸、更长的检查单，而是另一种载体：一种让机器也读得懂主张、证据和它们之间关系的载体。它长什么样，怎么造，是这本书后半部的事。这里只留一扇门，和门后隐约的灯光。

导读的期票，现在兑现。合上书之前，做三件事：把你手头最近一次放行的文件包按主张、论证、证据分成三堆，数一数中间那堆有几页；随手抽出包里两份报告，一对它们引用的配置是不是同一个；再问一句——上一次变更之后，影响范围是谁圈的，他的依据写在了哪份文件里。三个问题里只要有一个让你答不上来，你就已经站在那扇门前了。

**本章注记：**本章的评审场景、人物、事故经过与全部产品数字（候选代号、硬件与软件版本、标定与基线编号、文件数量、时间跨度等）均为合成教学材料，不对应任何真实企业、真实产品或真实个人。正文对安全案例与各支撑过程的转述为自然语言概括，精确要求以标准原文为准；本章不构成对任何实际系统的判定、分级或放行结论。

## 第二部分

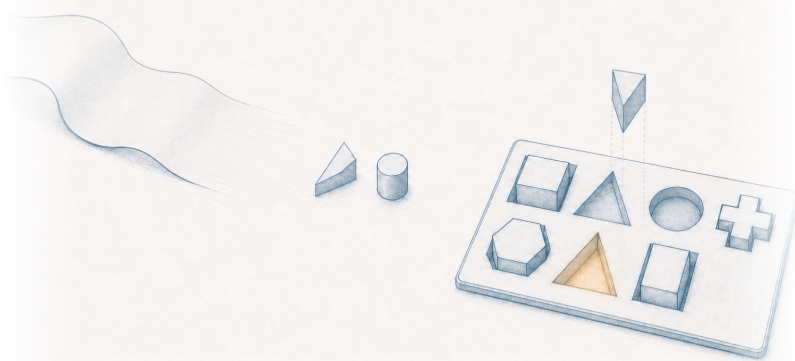
唯一执行语义：由 Semantica 承接规范  
本体化





## 第十一章 把“可以交付”写成 机器能查的一句话：可信主张本体

Semantica 绑定：本章是可信主张概念化的人可读正本；唯一机器语义是 `semantica.chapter_packages.vol2.ch11`，主场景为 `semantica.vol2.ch11.scenario.primary`。类/属性、CQ、查询、Shape、正反案例、规则、exact oracle、manifest、PROV 与 receipt 全部在 Semantica。正文记法只用于讲解，不构成第二套本体。当前为 `partial`、release `blocked`；结构可查不等于主张为真。



**【导读】** 前十章走到了纸面的极限：证据不能自己对表，影响不能自己现身，案例不会跟着产品活着。本章走进那扇门后的第一间屋子——一个只限 EPS 的试点。先让“模型这么聪明，还要什么结构”这个再自然不过的判断完整地发生，再看它撞碎在哪里；然后从第 1 章那张主张部件卡出发，学会机器世界的第一批词：三元组与类和实例、能力问题、正例反例与门禁、局部完整性与未知。接着，把那场六绿评审的材料原样搬进机器，让同一场僵局第二次发生——这一次被接住：缺件即拒，凭“通过”字样挂不上支持。读完本章，你应当能读懂一条三元组在说什么，能为一套最小结构开出它承诺回答的问题清单，并解释机器凭什么对一条主张、一条支持关系说“不”。

## 11.1 只限 EPS 的试点

附条件放行后的第二周，陈工在项目总结会的最后十分钟拍了板：申请一个试点。范围被他当场圈死——只做 EPS，只回答一个问题：那句在放行桌上千锤百炼出来的“能签的话”，能不能被机器读住、查住、拦住。不做平台，不许扩散，范围之外一律不碰。

牵头的人选没有悬念。那个周三深夜靠笨规矩和运气停住笔的小蔡，被点名转岗。陈工给她的话只有一句：“你那晚是人肉对表。试点要做的，是让笨规矩变成机器的天性。”

还有一位新成员：一个由大语言模型驱动的助手，第一次被接进这间会议室的网络。它的身份被陈工一句话定死——可以提议，可以干活；但它说的话不算事实，它做的事不算决定。白板中央，评审那天写下的“EPS”和那行小字还没擦，小蔡在旁边画了一个四格的圈：提议、校验、执行、审计。每转一圈，留一笔账。

第一次试点例会，小唐先声夺人。他把六份报告和放行会纪要整包喂给助手，连问三个问题，答案又快又顺，其中一条还主动指出了台架配置与目标快照不一致——这正是当初满屋子人熬到深夜才对出来的东西。小唐把屏幕转向众人：“模型这么聪明，还要什么结构？材料给它，问题问它，不就完了？”

这句话一点都不外行。助手十分钟读完了人要读三天的材料，而且确实读对了最关键的那处偏差。如果这个判断不成立，问题一定藏在不容易看见的地方。

陈工只提了一个要求：同样的材料，再答一遍。第二遍里，台架卡的地位从“超出范围”悄悄变成了“部分支持”；追问低温冷起动，它给出一段无懈可击的流畅解释，说该工况“已由台架复现覆盖”——把 rc2 那次边界工况复现，安到了低温头上。没有一句话结巴，也没有一句话可以对表。

漂移的根子不在记性，在答案的来路。助手不是从哪本底账里把上一次的结论取出来复述，而是每问一遍，就把一段最像样的话重新组织一遍；同一堆材料，说得通的讲法不止一种，这一遍押中这种，下一遍押中那种。它肚子里没有一张可以回头核对的表，两遍答案合不合得上，它自己都无从知道。

小唐不服，试了两条补救。先是加长提示词，把射程、对表、四种关系写成十几条规则塞进去——好了两天，换个问法又漏：提示词是恳求，不是合同，它约束语气，约束不了语义。又有人提议让助手自问自答三遍、取多数——三个流畅的答案在台架卡上各执一词，投票选出的仍是一句无处着力的话：复查的声音和被查的声音，是同一个。

**流畅是语言的性质，可查是结构的性质；模型再聪明，也得先有一套能查的结构，它的聪明才有落点，它的错误才有着力点。**

陈工替这场演示收了尾：“它答对的时候，我们没法确认；它答错的时候，我们没法抓住。先把‘能签的话’变成一个它逃不出去的结构。”——可机器能读的结构长什么样？从哪一块砖砌起？

## 11.2 第一块砖：三元组、类与实例

小蔡没有从任何教科书开始。她把小唐在六绿评审后起草的那张主张部件卡钉在白板上——主语、配置、关注、情境、时间、假设、决定范围，七个格子。问题只有一个：怎么让机器读懂这张卡？

机器世界里最小的陈述单位，是一句三个词的话：对象—关系—对象。“这条主张的主语是候选快照 RC17”，拆开就是（这条主张，主语是，RC17）。这样的一句话叫**三元组**。它笨得惊人：一次只说一件事。但正因如此，每一句都可以单独被查、单独对质、单独判对错——一段漂亮的总结做不到，一条三元组可以。

还差一样东西。“RC17 是一条候选快照”——这句话里，“候选快照”管“什么算”，叫**类**；“RC17”管“是哪个”，叫**实例**；把实例归入类，本身也是一条三元组，叫**类型声明**。为了不与别人的词撞名，团队给这套词统一冠了一个姓：claim。这套姓 claim 的词表，连同它们被约定的用法，就是这一章要建的东西——一个最小的**可信主张本体**。本体这个词听起来大，此刻它只指一件很小的事：一套被共同约定、机器可读的概念结构。

这段记法在说什么：下面几行，把小唐那张部件卡原样翻译成三元组——第一行的（这是各格补齐后的完整状态；小蔡最初录入时，情境与时间两格还空着——下一节机器拒绝的正是那一版）

类型声明说“这是一条可信主张”，其余每行填一个部件。

# 一条可信主张：七个部件各占一格（示意记法）

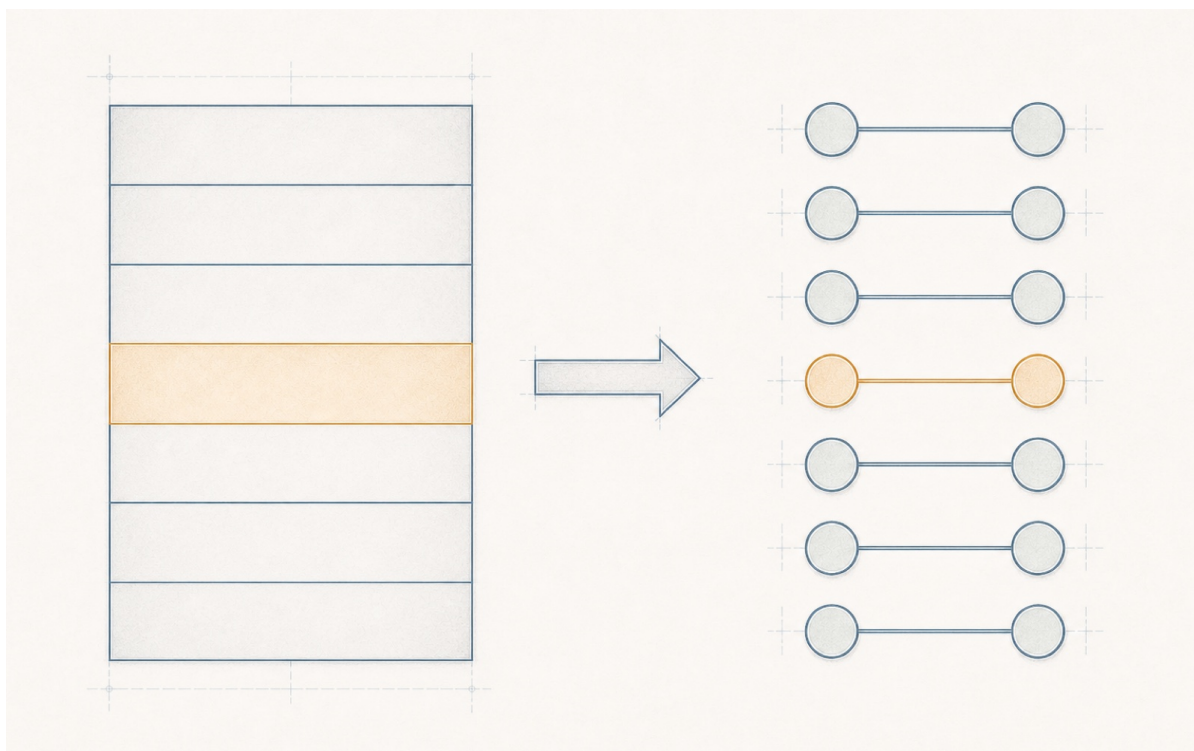
```
claim:Release_RC17 a claim:TrustClaim ; # 类型声明：这是一条可信主张
  claim:subject          eps:RC17 ; # 主语：候选快照
  claim:configuration    eps:Snapshot_RC17 ; # 配置：
H3.2+SW1.8.3+C41+D7+V12
  claim:concern          claim:Safety ; # 关注：功能安全（并列不折算）
  claim:context          eps:DeclaredConditions ; # 情境：声明过的工况
  claim:validWindow      eps:ReleaseWindow_17 ; # 时间：本轮决定窗口
  claim:assumes          eps:DiagAssumptionSet ; # 假设：诊断假设清单
  claim:decisionScope    claim:AdmitToReview . # 决定范围：进入本轮放行评审
```

注意主语那一行。评审桌上，“你说通过，通过的是谁”要靠陈工当面逼问；现在这一问变成了结构里一个独立的、永远敞开的格子——谁都可以随时来查，它填了什么、有没有填。

类回答“什么算”，实例回答“是哪个”；三元组把一句大话，拆成一格一格可以单独对质的小话。

砖有了。可这套词表要收多少词才算建完？收到哪里算够？谁说了算？

**读图任务：**把左侧主张卡的七个分格逐一对应到右侧七组点一边一点结构，观察自然语言在进入图谱时哪些关系必须被显式命名。



**图 11-1** 主张从一句话变成可查询对象，不是把整句话存成字符串，而是把主语、配置、关注、情境、时间、假设和决定范围变成可验证关系。

**图后判断：**这张图表示结构化转换，不是完整的 RDF/OWL 模型；它不给出具体的 IRI、基数约束或推理规则，也不证明转换后的主张真实。

### 11.3 结构的边界，是它承诺回答的问题清单

两条最自然的路先摆出来。一条是求全：把标准术语一个不落全建进去——建不完，建完也用不上，词表大到没人知道哪些格子真有人在查。另一条是凭手感：建到“看起来够了”为止——可“够”字没有判据，一换人就换一个“够”法。

本体方法给的答案，是把顺序倒过来：先写下这套结构**承诺回答的问题**，再倒推需要哪些词。这样的问题叫**能力问题**。一套本体的验收边界，从此不是它的规模，而是它的问题清单：清单上的问题必须答得出、答得对；清单之外，不承诺、不冒充。

试点的清单没有新拟，就是第 1 章章末冻结下来的三问，用白话写出来：

其一，给定一条候选主张，它绑定了哪些部件，还缺哪些？其二，给定一份证据，它与主张的关系是支持、反驳、超出范围还是未知，理由何在？其三，只改一件事之后，哪些主张与关系必须重开，哪些可以凭明确理由保留？



这三问不是理论：那场六绿评审，人脑、白板加上加班，把它们逐一回答过一遍。试点的全部野心，是让机器把同样三问回答到**可对表**。换一张桌子这份清单也成立——ENV-01 的出货评审同样要问这三问；但答案和个体一个都不迁移，那个世界要用自己的证据自己重建，这是第 1 章就立下的规矩。

这段记法在说什么：第一问已经可以问了。下面是“问题 → 查询 → 答案表”的第一次亮相——查询逐个核对七个必备部件，把还空着的格子报出来。

问题：这条主张的七个部件，还缺哪几个？

```
# 查：主张的必备部件中，哪些还空着
SELECT ?missingPart WHERE {
  VALUES ?part { claim:subject claim:configuration claim:concern
                  claim:context claim:validWindow claim:assumes
                  claim:decisionScope }
  FILTER NOT EXISTS { claim:Release_RC17 ?part ?anyValue . }
  BIND(?part AS ?missingPart)
}
```

| 缺失部件 | 说明       |
|------|----------|
| 情境   | 声明工况尚未录入 |
| 时间   | 决定窗口尚未录入 |

答案表让小蔡愣了几秒——这两格是她自己录的：只顾着把主语、配置、关注填扎实，情境和窗口想着“回头补”。机器第一个拒绝的不是助手，是牵头人本人。她把这两行发进群里，配了一句：“它连我也查。”

一个本体的验收边界，不是它收了多少词，而是它承诺回答哪些问题。

查询能把缺件报出来，可报告终究是温和的——回头补，也可以永远不补。怎样让缺件的主张根本进不了评审队列？

### 11.4 机器的“不”：缺件即拒，支持要凭据

周四下午，小蔡把那场六绿评审的全部材料原样搬进结构：一条主张、六份绿色工件、一页台架异常读数。同一场僵局，第二次发生——这一次，对面坐的是机器。

第一道门禁很简单：**缺任何一个部件的主张，不得进入评审队列**。小蔡那条还欠两格的主张刚一提交，就被拦在了队列之外。

这段记法在说什么：下面是机器出具的裁定记录——它不打分、不劝告，只说明

拦下了谁、因为缺什么。

```
# 门禁裁定（节选）：缺件的主张不进评审队列
[ a claim:GateViolation ;
  claim:offendingClaim claim:Release_RC17 ;      # 被拦下的主张
  claim:missingPart    claim:context ,
                        claim:validWindow ;      # 缺了哪两格
  claim:verdict        "rejected"                # 裁定：拒绝受理
] .
```

会上立刻有人皱眉：拦得这么死，活还怎么干？两条软化方案被端上来。一条是让助手把缺的两格补上——它确实一秒钟就填出了像模像样的情境和窗口，可细看之下，窗口值是它从旧纪要里“顺”来的上一轮窗口。生成的字段不是事实，只是长得像事实；补上它们，缺口没有消失，只是穿上了衣服。另一条是把门禁改成警告不拦截——可警告会像绿灯一样被习惯：当初满屋子的绿色没有拦住任何人，一行黄色小字更不会。

**机器“生成”的能力如今到处都是，机器“拒绝”的能力才值钱：一次有据可查的拒绝，比一百次顺滑的通过更接近工程。**

主张补齐入队之后，轮到六份工件。小唐想省最后一道事：让助手扫一遍报告，凡结论字样为“通过”的，自动挂上一条指向主张的支持边。助手照办，六条支持边整整齐齐。门禁全数退回——理由与第1章那句旧话一字不差：报告是工件，支持是关系；关系需要被建立，不会自动发生。只是这一次，“被建立”三个字有了机器可查的定义。

这段记法在说什么：一条合法的支持边长这样——除了两头，它还必须自带两样凭据：

一份真实存在的对表记录，和一个复核过适用性的人。

```
# 支持是关系：一条支持边必须自带凭据（示意记法）
claim:Link_SwRegression a claim:SupportLink ;
  claim:fromEvidence  eps:SwRegressionReport ;    # 哪份工件
  claim:toClaim       claim:Release_RC17 ;        # 支持哪条主张
  claim:scopeChecked  eps:ConfigMatch_SW183_C41 ; # 对表记录（受控活动产物）

  claim:reviewedBy    eps:RoleSoftwareLead .      # 谁复核了适用性
# 只有“通过”字样、拿不出对表记录的支持边，一律拒绝入图
```

关键在第三行：它指向的对表记录必须真实存在——那是一次受控活动留下的产物，得有人真的把配置逐项对过表。“通过”字样只是工件自己的属性，一格凭据都替代不了。

在这套规矩下，六份工件各归各位：软件卡、冲击振动卡凭对表记录挂上支持；台架卡挂成“超出范围”，P07、rc2、C42 三项偏差逐条列明——不作废，也不冒充；郑工

抽屉里那页异常读数，第一次有了正式席位：一条反驳边，指向主张，状态“待处置”。图不许它再回到抽屉里去。

六份工件各就各位。可有一格始终空着——低温冷起动。空格，在机器眼里算什么？

## 11.5 空格的名字，与只改一件事

两种处理空格的本能，都试过、都不行。把空格判为不合格？那是逼团队为一场不存在的失败翻案，翻不动，最后大家学会的动作是悄悄删掉那个字段。把空格当成默认通过？郑工那年冬天追回的车，就是这条路的尽头。

所以这套结构里，证据与主张的关系不止支持与反驳两种——台架卡已经占住了第三种“超出范围”，而空格属于第四个诚实的取值：**未知**。缺少记录时，默认答案是未知，不是假，也不是真——这条三值纪律，第 1 章就立在人的语言里，如今写进了机器。

但还差一步。机器凭什么把某个“未知”升格为“缺口”？它自己不知道世界上本应有什么。只有当有人**声明局部完整性**——“低温冷起动这一项本应有记录，而这条主张名下的证据都已收在这份清单里：清单里没有，就是世上没有”——机器才被授权把账面的空白读成世界的空缺，空白才成立为可数的缺口。声明是人给的，缺口是机器数的；没有声明，机器对空白只有一句话可说：未知。

**机器只有在被告知“这里本应有记录”之后，才有资格说“缺”；在此之前，空白只有一个诚实的名字——未知。**

试点进行到第三周，1.8.4 的变更走到了眼前——第 1 章那道“只改一件事”的考题，轮到机器来答。六绿评审那天，小林的差异分析是全场最专业的几页，也是最私人的几页：每一条“受击”与“不受击”都存在他的脑子里。这一次，他把那门手艺沉淀成了边上的标注：每条支持边录入时，都声明自己依赖哪些配置维度。于是软件维度一变，重开集合不再靠人圈，而是被查询直接返回。

这段记法在说什么：第三问的机器版——查询找出所有依赖软件维度的支持边，把它们点名重开。

问题：软件从 1.8.3 升到 1.8.4，哪些支持关系必须重开？

```
# 查：受软件维度波及、必须重开的支持边
SELECT ?link WHERE {
  ?link a claim:SupportLink ;
        claim:toClaim claim:Release_RC17 ;
        claim:dependsOnDimension eps:SoftwareVersion .
}
```

| 被点名的边   | 处置                  |
|---------|---------------------|
| 软件回归支持边 | 重开：直接受击，适用性待重判      |
| 生产写入支持边 | 重开：镜像随软件必变          |
| 硬件分析支持边 | 重开：诊断假设需重新确认        |
| 冲击振动支持边 | 保留：依赖维度不含软件，无影响理由在案 |

重开不是作废：被点名的边退回“待重判”，证据本身一页未损——这正是第 1 章“版本变化改变的是关系，不是证据真伪”的机器兑现。也要说老实话：查询算出的集合，只忠实于已录入的依赖标注——标注漏了，机器跟着漏；而变化本身的形状（构建、环境、漂移），这套小结构还装不下，那是后面章节的事。

结构、门禁、查询都立住了。那位助手呢——它在圈里，到底能做什么、不能做什么？

### 11.6 圈里的助手：提议、校验、执行、审计

白板上那个四格的圈，现在正式开转。第一件派给助手的活：把台架卡的三项偏差整理成结构里的偏差说明。

**提议：**助手起草一组三元组，写进提案区——提案区不是正图，写一万条也不算数。**校验：**门禁先行。助手引用的每份对表记录，必须已存在于受控活动的记录里；这一回它顺手“补”了一条 C42 与 C41 的等价说明，当场被退——那条说明在任何活动记录里都不存在，是它编的。**执行：**退回重提、校验通过之后，写入草稿区。**审计：**小蔡逐条签认转正，账上留下这一圈的全部脚印。

这段记法在说什么：一圈走完，账长这样——五行，分别记下谁提议、谁校验、写了什么、谁签认、谁有决定权。

# 一圈有界循环留下的审计账（示意记法）  
eps:AgentRun\_014 a claim:BoundedAgentRun ;  
    claim:proposed    claim:Link\_BenchOutOfScope ; # 提议：台架卡记为超出范围  
    claim:checkedBy   claim:SupportBasisGate ;       # 校验：门禁先行  
    claim:executedAs   eps:DraftEntry\_Bench ;        # 执行：只写草稿区  
    claim:auditedBy    eps:RolePilotLead ;           # 审计：小蔡逐条签认  
    claim:decisionBy   eps:RoleProjectOwner . # 决定：最后一笔属于陈工

这一圈里，三种权威各归各位，谁也替不了谁。姓 claim 的词表管**语义**——一个词该怎么用、一条边缺什么不许入图；受控活动的记录管**事实**——对表记录、试验产物、异常读数，本体的一条边永远替代不了一份实测，助手编的那条等价说明就是撞在这条线上；有权的人管**决定**——门禁通过不是接受，助手执行完不是授权，最后一笔落在人的名字上。

散会前，小唐盯着那条被退回的提案看了很久：“它还是那么聪明。但现在，它错得起了。”以前它的错藏在流畅里，没人抓得住；现在它的错落在结构上，错一条退一条，条条有账。

助手可以提议一切，却只能执行圈内的动作；圈由本体划定，门由门禁把守，最后一笔永远属于人。

试点第一份周报发出去那晚，小蔡却在一个最不起眼的格子前停住了——主语那一格。

## 11.7 名字还没有判据

归档前，小蔡让助手顺手把结构里外与 R17 有关的记录收拢一遍。助手勤勤恳恳，拉回来一批物料库的领用单——R17，一颗贴片电阻，就是那颗只差一个字母、曾把陈工电话打爆的老朋友。满屋子笑出了声。

笑完，屋里凉了下来。机器判“有关”的依据，是字符串相像；它判“同一”的依据，是字符串相等。除此之外，一无所有。台账里的 DUT-P07、序列号、装配号，在人眼里明明有桥，在图里是几个互不相识的点；主张部件卡上那格千辛万苦立起来的主语，机器能查它填没填，却回答不了一个更根本的问题——格子里那个名字，指的到底是谁？两条主张的主语同名，是同一个对象，还是又一次 R17？机器耸耸肩：未知。

这就是试点第一个月的诚实账目。可信主张本体回答了它承诺的三问：一句能签的话长什么样、缺了哪件；证据与它是什么关系、凭什么；只改一件事后哪些必须重开。它没有回答、也没有能力回答的是：主张绑定了对象，而**对象的身份还没有机器判据**——什么算同一个，证据链怎么搭，谁有权写下“同一”二字。这道题，交给下一章。

导读的期票，现在兑现。合上本章之前，试三件事：任取本章一条三元组，用一句白话说出它在说什么；替你自己手头的某份结构化台账，写下它承诺回答的三个问题——写不出来，说明它还没有边界；最后向同事解释两次拒绝——机器凭什么拦下一条缺件的主张，凭什么退回一条只有“通过”字样的支持边。三件都做得到，你就已经站在了后十章的地面上。

白板上，四格的圈还留着。小蔡下班前在“主语”那格旁边，画了一个问号。

**本章注记：**本章的试点场景、人物、对话与全部产品数字（候选代号、硬件与软件版本、标定与基线编号、时间跨度等）均为合成教学材料，不对应任何真实企业、真实产品或真实个人；本章不构成对任何实际系统的判定或放行结论。文中全部记法片段只用于说明结构与门禁原理；唯一可执行版本是 `semantica.chapter_packages.vol2.ch11`。当前包为 `partial`、`release blocked`。





## 第十二章 名字背后的 身份判据：对象与同一本体

Semantica 绑定：本章保存对象身份与同一判据的解释；唯一机器语义是 `semantica.chapter_packages.vol2.ch12`，主场景为 `semantica.vol2.ch12.scenario.primary`。本体、CQ、查询、Shape、正反 bridge 案例、规则、oracle、manifest 与 receipt 只在 Semantica。当前为 `partial`、release `blocked`；相似度或局部门禁通过不能授权合并现实记录。



**【导读】** 上一章结束时，主张有了机器可查的骨架，缺件即拒——可是主语那一格，填什么才算不缺？小蔡把三张打印纸摆上桌：台架台账里的一条、制造系统里的一条、PLM 里的一条。老工程师凭记忆说它们是同一台，而记忆不是证据。本章陪你把“同一”这个最不起眼的词拆到底：先回答“同一个什么”（身份判据要按对象的族来分立），再回答“凭什么说同一”（别名要带来源，“同一”要带证据链，缺证据就诚实地保持未知）；然后让第 2 章的两场旧事在新载体下重演一遍——三条记录桥接成功，撞名乌龙在消息进群之前被拦下；最后看一个高相似度的合并提议怎样被门禁挡住，以及为什么挡住比合并更值钱。读完本章，你应当能对任何一句“这两

条记录是同一个东西”，问出它的族、判据与证据链，并在证据不足时敢于把“未知”原样保存下来。

## 12.1 主张有了骨架，对象还没有身份

试点小组的第二次工作会，小蔡带来一个卡住的演示。上一章立起来的主张结构运转正常：一条主张缺了关注、缺了情境、缺了决定范围，机器都会当场拒收。可当她想为台架那台样机录入第一条真实主张时，AI 助手回了一个她答不上来的问题：主语这一格指向的对象，是记录库里的哪一条？

小蔡把三张打印纸摆上桌。台架台账里写着 DUT-P07；制造系统里写着 SN-EPS-000417；PLM 里写着 EPS\_ASSY\_043。这三条记录她太熟悉了——入职第一周，她就是拿着它们问出了那个让会议室安静的问题，而当时的答案是郑工的一句“这就是同一台，我记得”。第 2 章末尾冻结的第一个问题原封未动地躺在这里：给定多个系统里的一组名字，怎样判定它们指同一对象、不同对象、还是未知？判据和证据是什么？

会上很快冒出两个方案，每一个都值得认真对待。

第一个是“一物一码”：给每台东西发一个全局唯一编号，三个系统统一挂靠。这个方向不外行——新建系统确实应该这么做。但摆在桌上的是十几年的存量：台架台账、制造系统、PLM 各有各的历史，供应商和试验外包方还有自己的系统，谁也不会为了试点重编十年的账。更深一层，编号只是又一个名字：它会被抄错、被重用、被贴在换过总成的机器上。全局编号解决的是“以后怎么起名”，解决不了“已有的名字指谁”。

第二个是建一张对照表：三列名字，人工核对一遍，从此一劳永逸。做过术语表的团队会认出这条路的老问题——第 2 章刚讲过，对照表自己会繁殖。而它还有一个更致命的缺陷：表里只有“对上了”三个字，没有对上的理由。谁核对的？凭什么？哪天有人质疑，答案还是那句“心里都有数”。一张没有证据的对照表，不过是把老工程师的记忆抄写了一遍——记忆的毛病它一个不少，还多了一个“看起来很权威”的毛病。

有人半开玩笑：让 AI 助手读一遍三条记录，它肯定说是同一台。小蔡真让它读了。它说“极可能是同一台设备”，理由是名称模式、日期与型号高度吻合。“极可能”——上一章刚刚立过的规矩在这里再次生效：语言的流畅不是事实的来源。机器需要的不是一个更自信的猜测，而是一个可以核对的判据。

可“判据”两个字说出口，第一个坑就在脚下：判定两条记录是不是“同一个”，你得先回答——同一个**什么**？

## 12.2 同一个什么：身份判据按族分立

这个问题不是抬杠。拿桌上的三条记录试一下就明白。

假如“同一个”问的是**物理单件**——那台占据时空、可以摸到铭牌的机器——那么判据是它的**履历连续性**：从下线那一刻起，序列号与实物之间那条没有断过的看管链。物理单件会被维修、被调换总成、被重新喷号，所以“名字相同”从来不是判据，“这台机器的历史连得上”才是。内容更不是：同一批次下线的两台机器，图纸、物料、参数可以逐项全同，按内容比一个字都挑不出差别，可它们就是两台，各占各的位置，各走各的命运——能把它们分开的，只有时空里那条各自的履历。

假如“同一个”问的是**软件工件**——比如那份写着 SW1.8.3 的构建——判据完全换了一副面孔：**内容等同**。软件工件复制一千份还是同一个工件，逐位比对内容一致就是同一，差一位就不是。它没有履历连续性的问题，因为它不占据时空；问一份构建“这台还是不是原来那台”，问题本身就坐错了座位。反过来也一样：换一台机器、隔上半年，重新编译出一份逐位一致的构建，来路完全是另一条，它仍是同一个工件——真拿履历去判，同一份东西就会被数成两份。

假如“同一个”问的是**产品候选**——比如 RC17 这样的快照——判据又是第三种：**配置清单逐项等同**。候选由它的组合定义：哪版硬件、哪版软件、哪版标定。改动其中一项，得到的是另一个候选，哪怕名字只差一个字母。第 1 章全书第一课讲的“快照”，在这里落成了判据。

三种对象，三种“同一”。这就是为什么老工程师的直觉再好也说不清楚：他心里其实同时装着三套判据，切换全凭经验，而经验不写在任何地方。

熟悉第 2 章的读者会问：标准的语言里不是已经有座位了吗——相关项、系统、元素？有，但座位回答的是另一个问题。座位是**分析边界**的名字：同一块控制器，这次分析里是元素，下次分析里是被分解的对象，换分析就换座位，这是合法的。而族回答的是**存在方式**：一台物理单件不会因为换了分析就变成软件工件。座位告诉你“这次它算什么”，族告诉你“它是哪种东西、按什么判同”。两套语言不冲突——一个元素座位上，坐的可能是一台物理单件，也可能是一份描述它的软件工件；正因为座位不管判同，族才必须管。

**“同一”不问族就没有判据：先回答“同一个什么”，才有资格回答“是不是同一个”。**

这段结论翻译成机器的语言，是本章最小本体的第一块骨架。下面这段记法在说什么：三个族各是一类对象，每一族绑定自己的判据；族与族之间声明不相交——

一台物理单件永远不会“合并”成一个产品候选。

```
# 三族对象，各绑各的同一判据（示意记法）
id:PhysicalUnit      a id:ObjectKind ;          # 物理单件：占据时空的那一台
    id:identityBy    id:HistoryContinuity .      # 判据：序列号 + 履历连续
id:SoftwareArtifact a id:ObjectKind ;          # 软件工件：内容即身份
    id:identityBy    id:ContentEquality .        # 判据：逐位内容一致
id:ProductCandidate a id:ObjectKind ;          # 产品候选：由配置定义的快照
    id:identityBy    id:ConfigEquality .         # 判据：配置清单逐项等同
id:PhysicalUnit id:disjointWith id:ProductCandidate . # 族与族不相交
id:PhysicalUnit id:disjointWith id:SoftwareArtifact .
```

最后两行不是形式主义的装饰，它们是第 2 章冻结的第三个问题——自动识别“座位错误”——的机器答案的一半：把文档当产品、把型号当单件、把状态当身份，在这套结构里都会撞上“族不相交”这堵墙。一份写着 EPS 的架构文档属于软件工件族（更准确说是工件族），它与任何物理单件之间连“是不是同一个”这个问题都不成立，机器直接拒绝提问。至于“状态不是身份”——一台进入安全状态的机器还是那台机器——族的判据里根本没有“状态”这一项，它想混进来也没有门。

判据分了族，下一个问题立刻跟上来：判据是族的，可桌上摆的是**记录**。三条记录和那一台机器之间，到底是什么关系？

**读图任务：**逐个比较三个展台上的对象与它们下方的判据，寻找为什么履历链、内容指纹和组合契合不能互相借用。



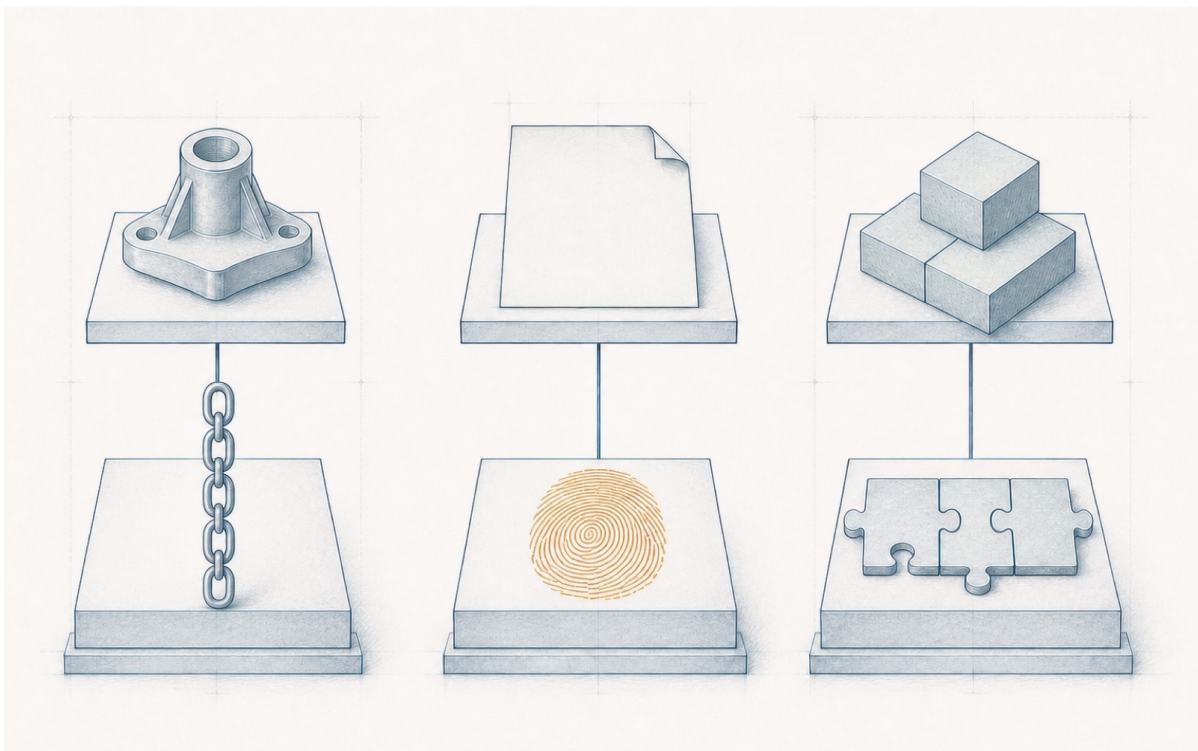


图 12-1 不同对象族需要不同的“同一个”判据。物理实物依赖连续履历，数字工件依赖可重现的内容身份，产品候选则依赖受控配置的完整组合。

**图后判断：**图中图形只是判据类型的缩写，不是真实序列号、哈希值或 BOM。它不证明某一条履历完整，也不解决身份冒用。

### 12.3 记录是别名，“同一”要带证据

第 2 章有一句埋得很深的话：术语卷定义了词，判定不了对象。现在可以把这句话向前推一步——台账里的一行字也不是对象。DUT-P07 不是一台机器，它是台账台账**在自己的语境里**对某台机器的称呼；SN-EPS-000417 是制造系统在出生登记的语境里对某台机器的称呼。每个名字都带着自己的来源，离开来源就没有着落。这就是**来源限定别名**：名字不属于世界，属于账本。

细看这三条别名，还能看出第 2 章讲过的另一个错位。DUT 的意思是“被测件”——这是一个**角色**的名字：P07 在那几次试验里扮演被测对象，试验结束角色就卸下，而机器还是那台机器。台账用角色称呼对象，日常没问题，可要是把角色当成本质——以为“DUT-P07”是一种东西——同一台机器换个台架就会凭空多出一个“新对象”。别名机制把这件事摆平了：角色名也好、序列号也好、台账代号也好，都只是别名，都要挂到同一个问题上——你指的是哪台？

于是“同一”的真正形状浮出来了。它不是两条记录之间长得像不像的关系，而是一个三角：两条别名，经由各自的证据，指向**同一个对象节点**。证据是什么？是受控活动

留下的记录：样机进台架那天，技师在进机登记单上抄录了铭牌序列号——这张登记单就是“DUT-P07 与 SN-EPS-000417 指同一台”的桥接证据；总装下线那天，装配记录把序列号挂进了 PLM 试制台账的单件条目——这条记录就是第二段桥。桥不是谁宣布出来的，是当年的工作本来就留下的，只是从来没人把它当成“同一”的承重结构。

这段记法在说什么：那台机器本身是一个节点；三条记录是三条别名，各自声明来源；其中两条别名带着自己的桥接证据。

```
# 三条记录 = 同一台单件的三个来源限定别名（示意记法）
id:Unit_417 a id:PhysicalUnit .                # 那台机器本身
id:Alias_DUT_P07    id:aliasOf id:Unit_417 ;
    id:inSource      id:BenchLedger ;           # 来源：台架台账
    id:evidence      id:EntryForm_0312 .        # 桥：进机登记单（抄铭牌序列号）
id:Alias_SN_000417  id:aliasOf id:Unit_417 ;
    id:inSource      id:MfgSystem .             # 来源：制造系统（出生登记）
id:Alias_ASSY_043   id:aliasOf id:Unit_417 ;
    id:inSource      id:PlmTrialLedger ;        # 来源：PLM 试制台账单件条目
    id:evidence      id:AssemblyRecord_0290 .   # 桥：装配记录（件号挂序列号）
```

规矩随之立下，三值一个不少——这套三值纪律上一章已经讲过原理，这里直接用：机器只有在证据链完整时才允许把两条别名挂上同一个对象节点，这叫**同一**；只有在持有差异证据时（比如两条记录对应的实物曾同一时刻出现在两地）才允许把它们分立为两个对象节点，这叫**不同**；两样都没有，就停在**未知**——别名各自悬挂，不指向任何共同节点，也不被强行分开。未知不是失败，是账面如实的样子。第 2 章说过，纸上的术语体系连表达“未知”的格式都没有；现在有了，它就是“别名暂不挂接”这个状态本身。

记录不是对象，是对象在某个来源里的别名；“同一”不是别名之间的相似，而是它们经由各自的证据指向同一个对象。

骨架立起来了。它接得住当年桌上那三条记录吗？

## 12.4 重演之一：一座用证据搭的桥

小蔡按流程发起了第一次真实的挂接。AI 助手的工作循环和上一章一样，四拍：提议、校验、执行、审计。

第一拍，提议。助手扫描三个来源，找到了两段候选证据：郑工组技师的进机登记单——上面有抄录的序列号、登记人和日期；老何系统里的装配记录——序列号挂着 PLM 的单件条目号。助手把“三条别名挂接同一单件节点”作为提议提交，附上两段证据。

第二拍，校验。门禁先查族：三条别名声明的对象族都是物理单件——查到这里出过一个插曲。PLM 那条记录最初被助手标成了“装配物料号”，物料号是 **型号** 的名字，属于设计定义，不是单件；族检查当场亮灯：单件别名不能挂接到型号上——这正是“把型号当单件”的座位错误被机器接住的样子。管 PLM 的同事查了库确认：EPS\_ASSY\_043 是试制台账里的**单件条目**，带序列号字段，不是物料号。改正声明，族检查通过。然后查链：台账别名到序列号，有登记单；序列号到 PLM 条目，有装配记录。链闭合。

第三拍，执行。郑工作为台账的责任人确认了登记单的真实性，挂接写入。第四拍，审计：提议、证据、族检查的那次亮灯、确认人和时间，全部留痕。

从此以后，第 2 章那个“每天都在发生的问题”换了一种问法。这段查询在问什么：台架台账里的 DUT-P07 和制造系统里的 SN-EPS-000417，是不是同一台？凭什么？

```
SELECT ?unit ?bridge WHERE {
  id:Alias_DUT_P07      id:aliasOf ?unit .
  id:Alias_SN_000417    id:aliasOf ?unit .      # 两条别名指向同一节点？
  id:Alias_DUT_P07      id:evidence ?bridge .    # 桥接证据是哪份记录？
}
```

| 判定 | 指向对象              | 桥接证据                 |
|----|-------------------|----------------------|
| 同一 | 那台序列号为 000417 的单件 | 进机登记单（有登记人、有日期）；装配记录 |

答案表和当年郑工那句“这就是同一台，我记得”说的是同一件事——但性质完全变了。记忆住在一个人的脑子里，会退休、会出错、无法被审核；这张表上的每一格都指向一份可以调出来核对的受控记录，任何人、任何时候、包括下周来的第三方审核员，都能沿着链走一遍。老工程师没有被替代，他当年的判断被**兑现成了结构**——桥上每一块砖，本来就是他们那些年认真填过的单据。

**机器判“同一”的资格，来自可核对的证据链，不来自名字的长相，也不来自任何人的记忆——哪怕那记忆是对的。**

桥能搭起来了。那么第 2 章另一场旧事呢——那场因为两个只差一个字母的名字、烧掉半天排期的虚惊？

## 12.5  重演之二：消息进群之前的一道闸

事情几乎原样重演。几周后的一个早上，来料检验系统又发出一条消息：“R17 批次异常，请相关方关注。”同一颗电阻的位号，新一批来料，失效率又一次略超门限——这类事在硬件世界里本来就会周期性发生。

不同的是，这一次消息在转进项目群之前，先过了一道闸。试点小组把消息转发挂上了身份服务：凡是带着对象名字跨语境流动的消息，先解析名字再放行。闸门查到：R17 在硬件设计域里登记为**元件位号**，属于设计定义的一部分；而项目群的语境里，以 R 开头的短代号几乎总被读成**产品候选代号**——RC17 就登记在候选族里。位号所属的族与候选族早在骨架里声明了不相交。两个族不相交的名字，在跨语境转发时只差一个字母——这正是撞名告警的触发条件。

这段记法在说什么：告警不是模型的灵感，是三个已声明事实的机械推论。

```
# 撞名告警：消息进群前的最后一道检查（示意记法）
IF 消息引用名 "R17" 在来源域解析为 id:ComponentRefDes（元件位号）
AND 目标语境的高频期待族为 id:ProductCandidate（候选代号，近邻名 "RC17"）
AND 两族已声明 id:disjointWith # 位号族 n 候选族 = 空
THEN 暂缓转发，附告警：
    "R17 为元件位号，与候选代号 RC17 分属不同对象族，不指同一对象。"
```

消息进群时，告警已经缀在最前面。硬件值班工程师回了一个字：“收。”处置单照常开出，台架排期一分钟没有动，没有紧急会，没有四十分钟后举着电路板的解释。当年那场虚惊的全部成本——七个人、半天排期、陈工被打爆的电话——这一次约等于零。

值得说清楚闸门**没有**做什么。它不懂转向，不懂电阻，不懂批次异常的严重程度；它甚至不知道这条消息重不重要。它只知道三件在骨架里写死的事：R17 是位号，RC17 是候选，两族不相交。第 2 章说纸面语言“不会在你用错词的时候弹出警告”——现在会弹了，而弹出它的不是更聪明的机器，是被写成结构的、当年吵完架就该记下来的那点常识。

**撞名告警买到的不是智能，是时机：把“你说的是哪个座位”这个问题，从事后复盘搬到了消息进门之前。**

拦住混淆，机器已经证明了自己。那反过来呢——发现同一？台账里躺着两千多条历史记录，等着有人把桥一座一座搭起来。谁来搭？

## 12.6 百分之九十八的诱惑

AI 助手用了一个晚上扫完三个来源的存量记录，交出一份提议清单：二百一十七对候选挂接，每对附一个相似度评分，绝大多数在九十五以上。名称模式、型号、日期窗口、经手人交叉吻合——单看任何一对，都像极了桥已经在那里、只差有人点头。

评审会上，老何看着清单说了实话：“两千一百台，我是一台一台打电话追回来的，那滋味我这辈子记得。这批旧账真要人工一对一对核，核到明年也核不完。它都打出九十八分了，比我手下最细心的记录员还稳——证据链可以慢慢补，先让它把‘同一’写上，错了再改回来嘛。”



这句话一点都不外行。恰恰因为老何比在座任何人都清楚台账错配的代价，他才最想快点把账对齐；“先写上、错了再改”也是产线上处理低风险台账的成熟习惯。如果这个判断有问题，问题一定藏在“改回来”三个字里。

答案没有让人等太久。清单第四十一行：台架台账的 DUT-P03，对上制造序列 SN-EPS-000198，相似度九十八。门禁在校验这一对时停了下来：两条别名之间**没有任何桥接证据**——那台样机的进机登记单，翻遍档案柜就是缺了那一页。按规矩，挂接被拒，状态保持未知。

会上有人嘀咕“九十八分还拦”，郑工却盯着那一行看了很久，然后想起来了：P03 那台样机，中途在一次过载试验后返过修，控制器总成整套调换过——台架代号照旧叫 P03，可换件之后，架子上那台机器已经不是序列号 000198 那一台了。登记单缺页，正是因为返修那阵子流程乱了。也就是说，相似度最高的候选对里，至少这一对，写上“同一”就是写上一个**错误**——而且是那种最危险的错误：从此以后，每一条绑定 P03 的试验主张，都会安安静静地指向一台早已被换下去的机器。上一章刚刚建立的规矩在这里显出分量——主张绑定对象；对象一错，绑在上面的一切跟着错，而且错得毫无声息。“错了再改回来”改不动这个：等你发现的时候，你已经不知道有多少下游结论消费过这条假的“同一”。

郑工的回忆本身也没有被直接采信——第 2 章的老规矩，记忆不是证据。他带人把返修那段的维修记录从档案里翻了出来：调换总成的工单、日期、换上件的新序列号。补录成受控记录之后，机器才把这一对从“未知”改判为“不同”，证据挂在判定上。一段记忆变成一条证据，走的还是那条路：受控活动产生事实，本体只负责让事实被查得到。

至于那份被拒的提议，这段记法在说什么：拒绝本身也是一条被保存的账。

```
# 证据不全的合并提议被拒（示意记法）
id:Proposal_0041 a id:MergeProposal ;
    id:merges id:Alias_DUT_P03 , id:Alias_SN_000198 ;
    id:basis id:Similarity_98 .          # 相似度：检索线索，非判据
门禁裁定：无 id:evidence 桥 → 拒绝挂接同一对象节点；
状态保持：未知（两条别名各自悬挂，原样保存）；
审计留痕：提议、评分、拒绝理由、复核人。
```

相似度没有被贬值——那份二百一十七对的清单仍然极有用，它把两千多条记录的排查压成了一份可以按优先级消化的工作队列。它只是被放回了自己的座位：**候选的生成器**，而不是**同一的裁判**。

**相似度是检索的线索，不是同一的判据；证据不足时，这套结构最值钱的输出不是一个大胆的合并，而是把“未知”原样保存下来。**

为什么“先合了再说”在哪家公司都占上风？因为重复记录的成本是显性的：两条账天天有人对两遍，人人看得见，会上有人抱怨；而错误合并的成本是隐性的：账面从此



更整洁，代价被记到未来某一次追溯的账上——爆发那天，多半已换了经手人。组织天然优先消灭看得见的麻烦，于是系统性地偏向合并。保存“未知”难，不是难在技术，是难在要一个组织忍受一笔天天被看见的小成本，去避免一笔多年后才结算的大账。

散会前，老何自己把话圆了回来：“行，九十八分进队列，不进台账。”但他随口又补了一句，让会议室再次安静：“那证据齐的那一百多对呢？机器都校验过了——谁点头，才能真往台账里写？”

## 12.7 本体不制造事实，也不拥有裁决

在回答老何之前，先把本章的账诚实地结一遍。

这套同一本体做到的事，可以数得很清楚：它让“同一”从一句断言变成了一个三角结构——族给判据，来源给别名，受控记录给桥；它让第 2 章冻结的第一个问题（同名怎么判）有了机器可查的答案，让第三个问题（座位错误怎么自动识别）有了族不相交这堵墙；至于第二个问题——一句“某某可靠”怎样展开成对象、关注、条件与判据——那是上一章主张结构的领地，本章补上的是它悬空的主语落点，两章经由显式的对齐合同协作，这里不展开。三源在本章各归各位：**本体**管语义——族、判据、三值的定义；**受控活动**管事实——登记单、装配记录、维修工单，桥上的每一块砖都来自它们；**有权的人**管决定——这一格，马上要谈。

同样要数清楚的，是它做不到的事。本体不制造事实：登记单要是当年就抄错了序列号，链会闭合得很漂亮，结论照样是错的——机器验的是链的完整，不是世界的真相；对账、盘点、实物核验这些老手艺，一样都省不掉，本体只是让“该核验哪里”第一次变得可以排队。本体也不会自动变对：族怎么分、判据怎么定，是人的裁决，裁错了机器会非常忠实地错下去。顺带一句，这套族也搬不走——隔壁 ENV-01 的世界要分自己的族（传感器单件、标定数据集、部署点位各有各的“同一”），判据得重新逼问一遍；第 2 章的老话在结构世界里原样成立：可迁移的是方法，不是词。

现在回到老何的问题。证据齐的一百多对挂接提议排在队列里，机器校验全部通过——然后呢？“写入同一”这个动作，在台账的世界里从来不是一个技术动作：它改变的是此后所有主张的落点，是审核时证据链的走向，是出了事之后责任的归属。机器可以证明“这两条别名**可以**挂接”，它永远证明不了“**应当由我来**挂接”。校验通过是资格，不是授权——这两个词的距离，正是下一章的正文。

小蔡在会议纪要的末尾写下这个待决项时，忽然意识到这个问题她见过。入职第一周，散会时她自己问的：“如果我查出来这两条记录就是同一台，这个结论谁来批准？我能直接改台账吗？如果系统管理员能改映射表，他是不是就拥有了工程裁决权？”当时没有人回答。如今问题绕了一整圈回到她桌上——这一次不能再没有答案了，因为队列里一百多对挂接正在等着某个人、以某种身份、在某种可以被机器检查的授权之下，按下“写入”。谁有权写“同一”？做的人、复核的人、担风险的人、授权的人，在结

构的世界里怎么分开、怎么互相制约？第 13 章就从这里开始：把“谁说了算”，也画进图里。

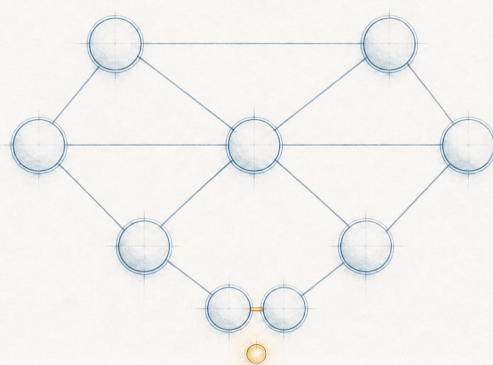
导读的承诺到这里兑现：族、判据、证据链、三值——下次再有人对你说“这两条记录是同一个东西”，你知道该问出哪四样东西，也知道在哪一步应该诚实地停在“未知”。

**本章注记：**本章的人物、会议、台账记录、相似度评分与全部产品数字（候选代号、位号、序列号、样机编号等）均为合成教学材料，不对应任何真实企业、真实产品或真实个人。正文对 ISO 26262 术语（相关项、系统、元素等）的转述为自然语言概括，精确定义以标准原文为准。文中各记法片段为教学示意，不构成任何本体语言的规范语法；唯一可执行版本是 `semantica.chapter_packages.vol2.ch12`，当前为 `partial`、`release blocked`。



## 第十三章 把“谁说了算” 画进图里：治理与责任本体

Semantica 绑定：本章保存治理距离、责任与授权的概念化；唯一机器语义是 `semantica.chapter_packages.vol2.ch13`，主场景为 `semantica.vol2.ch13.scenario.primary`。本体、CQ、查询、Shape、激活正反案例、规则、oracle、manifest 与 receipt 均在 Semantica。当前为 `partial`、release `blocked`；机器可验证授权关系，不能自行授予裁决权。



**【导读】** 身份的证据链修好了，合并却停在最后一步：谁有权写下“同一”？本章接住这个问题，把第 3 章钉在墙上的职责清单搬进机器：先把一页签字还原成人、角色、活动、对象的四元关系；再把做、出证据、挑战、评审、接受风险、授权六个位置各自钉住；把“第二双眼睛站得够不够远”变成一道可以计算的距离；让方工当年输掉的那场审核在新载体下重演一遍——这一次，机器当场拒绝；然后给每一条授权装上范围和日期，让“此刻这句授权是否仍然成立”变成随问随答；最后诚实地看机器买到了什么、买不到什么。读完本章，你应当能把任何一页签字还原成机器可查验的四元关系，能让机器替你回答“评审的人站得够不够远”“这句授权此刻还算不算数”，并说出剩下的那部分为什么仍然要留给人。

### 13.1 一次没人敢按下的回车

试点间的大屏上，一条归并提案已经停了三天。当年那台样机在两套系统里各有一个名字，两条履历隔着一道没人敢跨的沟；上个月，身份本体的验收会把桥接证据链逐环验过，三值判定终于走到了“同一”的门口。现在只差一个动作：把“同一”写进图里，两条履历并成一条。

小蔡的手指悬在回车键上方。她有权限——试点系统是她牵头建的，管理员账号在她手里，这一下按下去，没有任何东西会报错。可她想起自己刚入职那年，在会议室里问出的那个问题：查实两条记录是同一台之后，谁有权批准改台账？我能不能直接改？后来方工把那个问题翻成了六种人：动手做的、出证据的、挑毛病的、评审的、接受风险的、授权生效的。清单钉在墙上，六栏都有名字。

问题是，机器读不懂墙。

那三天里，当年的三个老答案还换了新衣裳回来过一趟。有人说，既然证据链机器都验过了，小蔡直接写就是——这是“谁录的谁改”的试点版：把验证的可靠偷换成裁决的资格。有人说，那就让小蔡以管理员身份代录，回头补个说明——这是“找管系统的人”的试点版，权限还是在冒充职权。也有人说，发消息请陈工回一个“同意”就行——可一条消息绑不住对象、版本和范围，它比当年那页会议纪要强不到哪里去。三个答案在纸面时代各自摔过跤，换了载体，摔跤的姿势一点没变。

陈工把方工请进了试点间——这一章的领域，本来就是他的。方工站在大屏前看了很久，说：“你们建的图会认对象了，会认证据了。可它还不认人。它不知道我是谁、凭什么坐在这里，也不知道你那个管理员账号和‘有权裁决’之间差着什么。第3章末尾我们冻结过三个问题，今天该翻译给机器听了。”

三个冻结的问题，翻成机器听得懂的能力问题，就是这个本体的验收边界：给定一项确认活动的记录，机器能否回答——执行的人在执行当日，与作者、与责任部门的组织距离，是否达到了承诺的档位？给定一页签字，机器能否查验——它绑定了对象、版本、职权范围与有效期？给定一次调岗或重组，机器能否列出——哪些承诺过的独立与授权，已经被击穿？

三问答得动，“谁说了算”才算画进了图里。可要答这三问，机器得先认识哪几样东西？

### 13.2 把一个名字还原成四样东西

最先想到的办法，是给提交表单加字段：加一栏“评审人”，系统强制非空，不填不让过。这个办法不外行——它至少挡得住空栏。可方工旧案输掉的恰恰不是空栏：当年会签从三栏加到七栏，每一栏都填了，责任照样蒸发。一个非空字段只能证明“有人填过一个名字”，它绑不住这个名字对的是哪一版对象、签的是哪一天、签字那天他和作者是什么关系。



第二个办法更省事：把红头任命文件和职责矩阵扫描进系统，再配上电子签章。图像是进了机器，可还是图像——机器搜得到名字，算不动关系。电子签章证明“这个名字确实是他按的”，仍然不证明“他按得有没有道理”。第 3 章说过，签名是图像，不是关系；把图像搬进电脑，它并不会因此变成关系。

真正要做的动作是拆。一页签字，其实压缩了五样东西：谁（一个人）、以什么身份（一个角色）、对什么东西（一份工作产物和它的版本）、哪一天（时间）、凭什么（一条授权）。治理本体的第一块砖，就是把每一次签名、每一次评审、每一次批准，都摊开成这样一组关系，而不是一格图像。

下面这段记法在说什么：它把“方工评审了归并分析”这句话拆成一个活动节点，人、角色、对象、日期、凭据各占一条边——活动居中连起人、角色、对象，

正是导读说的四元关系，日期与凭据再把它钉进时间和职权里，缺一不可。

```
# 一页签字，还原成四元关系（示意记法）
gov:Review_MergeAnalysis a gov:ConfirmationReview ;
  gov:performedBy gov:FangGong ;           # 人：谁做的
  gov:inRole      gov:ReviewerRole ;       # 角色：以什么身份
  gov:targets     gov:MergeAnalysis_v2 ;    # 对象：哪份产物、哪一版
  gov:onDate      "2027-04-12" ;           # 时间：哪一天
  gov:underAuth   gov:Auth_ReviewAssign_09 . # 凭据：凭哪条授权
```

注意那条日期边。冻结的第一问问的是“执行当日”的关系——不是任命那天，不是审计这天，是活动发生的那一天。日期落成一条边，机器才有资格把“那一天的组织图”翻出来对质。也注意对象那条边锁的是版本：评的是第二版，第三版改出来，这条评审不会自动跟过去——它只对它照过的那一版负责。

一页签字要能被机器查验，先得从一个名字还原成一组关系：谁、以什么角色、对哪个对象的哪一版、在哪一天、凭哪条授权——名字是图像，关系才是结构。

关系里最要紧的一条是角色。角色有哪几种？哪些位置，不许同一个人坐？

### 13.3 六个位置，一张不许同坐的桌子

角色最常见的建法，是做成头衔：在人员表里加一列，“他是评审员”“她是安全经理”。这个建法不外行——组织里名片就是这么印的。可头衔挂在人身上，处处通用，而第 3 章刚刚论证过能力是人与活动之间的相称：小蔡在台账复核里够格当助手，在软件评审里什么都不是。一个“处处是评审员”的字段，恰恰抹掉了这个差别。

第二个建法按部门划：测试部天生是挑战者，质量部天生是评审方。可部门是组织单元，不是活动位置——方工旧案里评他的人就在“另一个组”，换了个部门的皮，利害还连着筋。部门名答不了“这一次，谁坐在哪个位置上”。

所以角色不长在人身上，也不长在部门上，长在活动上。第 3 章墙上那六栏，在图里是六个角色类——每一次具体活动，都要有人坐进相应的位置。

下面这段记法在说什么：它只做一件事——把六种角色立成六个类，供每一次活动的“以什么身份”那条边去引用。

```
# 六种角色，六个类（示意记法）
gov:PerformerRole      a gov:RoleType .    # 动手做的人
gov:EvidenceRole       a gov:RoleType .    # 出证据的人
gov:ChallengerRole     a gov:RoleType .    # 挑毛病的人
gov:ReviewerRole       a gov:RoleType .    # 评审复核的人
gov:RiskAcceptorRole  a gov:RoleType .    # 接受风险的人
gov:AuthorizerRole     a gov:RoleType .    # 授权生效的人
```

六个类立起来之后，规矩才有地方落笔。规矩用白话说就三条：同一次活动里，坐“做”位置的人，不得同时坐进“评审”的位置；“接受风险”的位置必须有且只有一个具名的人，而且这个人必须拽得出一条当日有效的授权；“授权生效”永远是一个人的动作，不是一个会议的属性。这三条怎么变成机器的拒绝，本章稍后有一场现场演示。

角色长在活动上，还有一个立竿见影的好处：同一个人换活动就换位置，图里看得清清楚楚。小蔡在台账归并里坐的是执行的位置，在治理本体的建设评审里坐的是“做”的位置，将来若有人评她建的本体，她一步也迈不进那次活动的评审位置——不用谁提醒，位置本身就把路堵上了。头衔时代要靠自觉守住的界线，现在长在了结构里。

**角色不是挂在人身上的头衔，是长在一次活动上的位置；同一个人可以在不同活动里换位置，同一次活动里有些位置不许同一个人坐。**

“不能是同一个人”写清楚了。可方工旧案输掉的不是同一个人，是太近的两个人——多近算近，机器怎么量？

### 13.4 “够不够远”，变成一道算术题

第 3 章给过一把刻度尺：独立量的不是工位之间的距离，是利害之间的距离。纸面时代，这把尺子靠审核员的追问才量得动。要让机器来量，先得回答：利害距离在图里长什么样？

它长在三条最普通的组织关系上：谁和谁在同一支队伍（成员关系），谁向谁汇报（汇报关系），谁的考核握在谁手里（考核关系）。这三条边不由治理本体发明——它们来自受控的人事与任命记录，人事记录怎么写，图里就怎么画。本体只补一件事：把刻度声明出来。从近到远：同一个人、同一支队伍、同一位直属上级、同一条考核线、管理线之外。档位这么排，排的是利害松绑的次序：同队的人分着同一份成败，同一位上级捏着两个人的前途，同一条考核线意味着奖惩出自同一个源头——每退一档，松开

一层绑在一起的得失。每一个评审安排在建立时，都要声明它承诺的档位——对象风险越高，承诺的档位越远。

于是“够不够远”从一句承诺变成一次计算。郑工作为台账的数据责任方，带着证据提出了那条归并；分析要找评审人，第一个候选是小林。

问题：小林评郑工的归并分析，够不够远？查询查的不是“够远的证据”，而是“太近的证据”——同队、同上级、同考核线，撞上任何一条就算太近。

```
SELECT ?tooClose WHERE {
  gov:MergeAnalysis_v2 gov:authoredBy ?a .
  { ?a gov:memberOf ?g .      gov:XiaoLin gov:memberOf ?g .
    BIND(" 同一队伍" AS ?tooClose) } UNION
  { ?a gov:reportsTo ?m .      gov:XiaoLin gov:reportsTo ?m .
    BIND(" 同一上级" AS ?tooClose) } UNION
  { ?a gov:appraisedBy ?p .      gov:XiaoLin gov:appraisedBy ?p .
    BIND(" 同一考核线" AS ?tooClose) } }
```

答案表回来两行：

|                |
|----------------|
| 太近的证据          |
| 同一上级（研发经理）     |
| 同一考核线（研发线年度考核） |

小林出局——不是他不够好，是他不够远。换方工进查询，答案表是空的：功能安全经理的汇报线和考核线都在研发部之外，五个档位量到头，一条撞上的都没有。这里要多说一句空表的分量：查“太近”而一无所获，只有在人事记录声明过“汇报与考核关系已记全”的前提下，才敢当“够远”用——记录记了一半的组织图，量出来的距离一文不值。这是上一半试点里反复出现的老纪律：机器只有在被告知“这部分已记全”之后，才有资格判缺。

**独立性一旦写成可查的组织关系——同一个人、同一支队伍、同一位上级、同一条考核线——“够不够远”就从一句承诺变成一次计算。**

尺子挂进了图里。方工盯着那张空答案表看了半晌，忽然开口：“拿我当年那两份文件来，试试它。”

**读图任务：**分别阅读左侧距离尺和右侧挂在授权边上的日历牌，思考独立性与授权为什么都必须成为带范围和时间的可查对象。

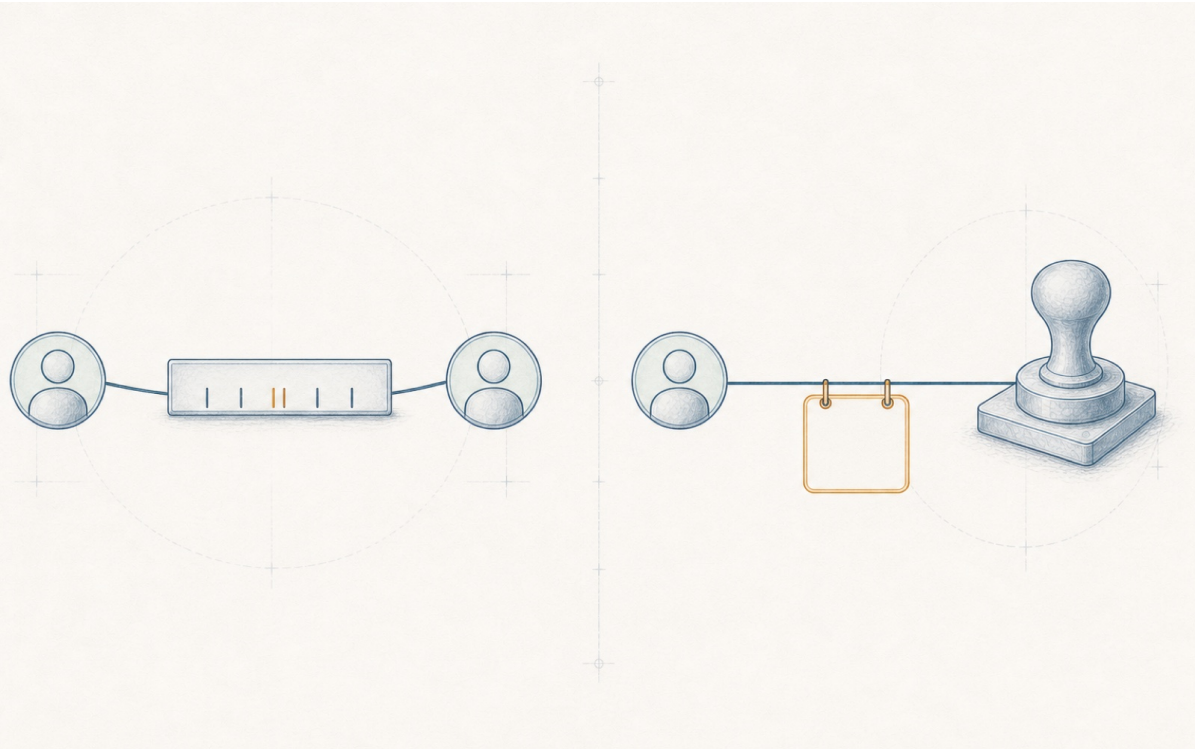


图 13-1 治理不应只存一个角色名或一个人名。它要能计算某次活动需要的独立距离，并核对某个人对某类决定的授权在当时是否有效。

**图后判断：**距离尺不是人事排名，日历牌也不是法律授权文件。本图只表示应被记录的关系属性，不单独决定任命合规性。

### 13.5 旧案第二次发生

方工把七八年前那两份文件做成了替身——内容换成试点的沙盘数据，结构原封不动——喂进新载体。他要亲眼看看，当年让他输掉一个客户的那两个窟窿，这一次能不能藏得住。

第一次提交，他故意把自己填进评审栏：作者方工，评审人方工。机器几乎没有迟疑：拒绝——做的人不得坐进评审的位置，同一个人。

第二次提交，他把当年真实的签法复原：评审人填上“组长”的替身——斜对面的工位，同一位部门经理，年终考核同一支笔。这一栏在当年的签名页上不是空的，七年里也没有任何人质疑过它。机器跑完距离查询：拒绝——同一上级、同一考核线，距离不足该分析承诺的档位。

第三次提交，是那页最难看的纪要：“经会议讨论，接受该风险。”有日期，有议题，有参会名单。机器的回答只有一行：拒绝——接受风险的位置，没有指向任何具名的人 and 任何有效授权。

下面这段记法在说什么：它是后两次拒绝的报文示意——机器不光说“不”，

还要说清违反了哪条规矩、证据是什么，让拒绝本身成为可审计的记录。

```
# 机器的拒绝（示意报文）
gov:Submission_047 gov:rejectedBecause [
  gov:violated gov:ReviewerDistanceRule ; # 独立距离不足
  gov:evidence " 评审人与作者向同一上级汇报" ;
  gov:evidence " 评审人与作者同一考核线" ] .
gov:Submission_048 gov:rejectedBecause [
  gov:violated gov:NamedRiskAcceptorRule ; # 担险无具名有权人
  gov:evidence " 接受风险的记录未指向任何个人与授权" ] .
```

试点间里没人说话。当年，这两个窟窿是审核员抽中两份文件才问穿的；问穿之后，是八个星期的整改、一个季度的推迟、七位数的损失和一个再也没有回来询价的客户。而刚才，三次提交，三行红字，前后不到一分钟。方工看着屏幕上那条“未指向任何个人”，轻声说：“当年这句话，我们翻了四十分钟文件才敢承认。”

当年要靠审核员抽中才问得穿的空栏，如今每一次提交都被问一遍；把问题问进结构里，勇气就不再是稀缺品。

评审的人够远了，担险的位置必须有名字了。可名字凭什么有权？授权从哪里来，到哪一天为止？

## 13.6 一条带日期的边：授权

第一个答案还是老样子：授权在红头文件里，扫描件都在案。可文件授的是职位，机器仍然不知道这句授权盖到哪些对象、管到哪类活动、到哪一天为止——一纸任命在图里还是一格图像。第二个答案更危险，危险到值得专门立一节：“他能改，就说明他有权改。”拿系统权限当职权——这正是小蔡当年那个问题的第一现场，也是所有台账事故共同的温床。

授权在图里必须是一条边，而且是一条带日期的边：授给谁、以什么角色、对什么范围、从哪天起、到哪天止、凭什么授的。

下面这段记法在说什么：它写出陈工在试点里那条裁决授权的完整形状——六个字段，少一个都不成其为授权。

```
# 一条授权：有人、有角色、有范围、有日期、有来源（示意记法）
gov:Auth_LedgerAdjudication a gov:Authorization ;
  gov:grantedTo gov:ChenGong ; # 授给谁
  gov:forRole gov:AuthorizerRole ; # 以什么角色
  gov:overScope gov:LedgerIdentityMerge ; # 对哪类对象与活动
  gov:validFrom "2027-03-01" ; # 从哪天起
```



```
gov:validUntil "2027-09-30" ;           # 到哪天止
gov:grantedBy gov:PilotCharter .         # 凭什么授的
```

先看截止日期那条边：授权不是一次性盖下的章，是一段有起止的承诺，到期不续，边就断了。再看来源那条边：授权自己也要有授权——它指向试点章程和授权人自己的记录动作，链条一环扣一环，到组织的章程为止。链条封了顶，“凭什么”这个问题才问得到底。

有了这条边，搁置三天的归并终于走完了全程：郑工以数据责任方的身份提案，证据链那一侧的语义属于第 12 章的身份本体，经由显式对齐合同送进来，本章只管一件事——谁有权认下它；方工在承诺档位之外完成评审；陈工在有效授权之内记录接受；最后，小蔡用管理员账号执行写入，两条履历并成一条。

写入之前，她做了一件多余又不多余的事：故意绕开流程，用管理员账号直接写。机器拒绝了她——写入动作必须引用一条裁决记录，而裁决记录必须由持有当日有效授权的人作出。她把这行拒绝截图发给了陈工。当年在会议室里问出“我能不能直接改台账”的人，今天亲手把“不能”写成了机器的回答：管理员的权限在图里是一个执行位置，裁决的位置上没有它。

写入权是系统发的一把钥匙，裁决权是组织授的一条边；机器能把这两样分开查验的那一天，“谁说了算”才第一次离开口头。

可组织是活的。任命在变，汇报线在变，试点在到期。今天成立的边，下个月还在吗？

## 13.7 组织一动，图里亮灯

十月初，公司调整组织：功能安全组从质量线划入研发线，理由写得堂堂正正——贴近业务。同一周里还有一件谁都没在意的小事：陈工那条裁决授权，截止日期是九月底。

人事记录第二天同步进来，清晨的图里亮了一串灯。

问题：今天，哪些承诺过的独立与授权，已经不再成立？

```
SELECT ?item ?why WHERE {
  { ?item gov:promisedDistance gov:OutsideMgmtLine ;
    gov:currentDistance gov:SameMgmtLine .
    BIND(" 独立距离被击穿" AS ?why) } UNION
  { ?item gov:validUntil ?end .
    FILTER ( ?end < "2027-10-08" )
    BIND(" 授权已过期" AS ?why) } }
```

答案表：

| 亮灯项                    | 原因                                |
|------------------------|-----------------------------------|
| 方工名下两条在途评审安排<br>台账裁决授权 | 承诺“管理线之外”，当前与被评部门同一管理线<br>已于八天前到期 |

没有人记得这两件事——这正是要点。第 3 章说过，调岗、重组、代理、兼任，每一次都可能悄悄击穿一条承诺过的独立距离，而纸面不会报警。现在图会。方工看着自己的名字出现在亮灯清单里，非但不恼，反而笑了：“当年没有人能回答‘签这一栏的人，在签字那天，和作者是什么组织关系’。现在这句话，每天清晨都被回答一遍——连我自己也躲不掉。”

收拾这摊事的，是那位不具名的助手，它走了完整的一圈。提议：从亮灯清单出发，起草两件东西——两条评审安排的替换人选，按距离计算和能力档案初筛出候选；一条新的授权草案，范围照旧，有效期顺延到试点终审。校验：草案过门禁，六个位置无冲突、距离达档、日期闭合。执行：人签完之后，它把记录落进图里。审计：每一步留痕——包括它被拒的那一次。有人顺口说了句“让助手直接把新授权生效得了”，助手提交的“生效”动作被门禁原样弹回：授权生效必须是授权人本人的记录动作。调用成功不是事实，执行更不是授权。

**授权不是一次性盖下的章，是一条随组织呼吸的边；“此刻这句授权是否仍然成立”，必须是一个随时可问、马上有答案的问题。**

小林在旁边看完全程，由衷感慨：“说句实话，这套东西比人靠谱。距离机器量，日期机器盯，连方工都被它点了名。以后评审安排就让助手排，机器都通过了，人就不用看了嘛。”

这话不外行。机器不忘、不倦、不讲情面，翻两百份记录不喊累——它确实在这件事上比任何人都可靠。可是，机器都通过了，人真的可以不看了吗？

### 13.8 机器买到的，和买不到的

第二天，方工抽查了一份门禁全绿的评审记录。四元关系齐整，距离够档，授权有效，日期闭合——挑不出一处结构毛病。然后他看了两个数字：意见栏里只有三个字，“无意见”；从领取到通过，八分钟。那是一份四十页的分析。

机器看得见票齐不齐，看不见眼神有没有落在纸上。挑战是否真实发生，图里只有它的形状。这是治理本体的第一条边界。

第二条边界在距离上。组织距离可以计算，利害距离比它宽：师徒之谊、多年私交、一段旧怨、一份顺手的人情——都不在人事记录里，都量不出来。图里量出的“够远”，是利害距离的下界，不是全部。所以第 3 章的三双眼睛一双也不能撤，方工这样的人仍然要抽查、要追问、要在结构合格的记录里嗅出不对劲的味道。

第三条边界在事实上。那次重组，机器是在人事记录同步之后才亮灯的——图不比它的事实源更快，也不比它的事实源更对；人事错录一条汇报线，图就错量一条距离。本体不制造事实，它只忠实地消费受控记录，包括记录的错误。

把三条边界放在一起，恰好是三样东西各归各位：组织关系与任命来自受控的人事与任命记录，那是事实之源，本体一条边也不发明；什么叫评审、什么叫距离、什么叫授权，由治理本体给出语义，那是共识之源；而每一条授权、每一次接受，永远是一个有名字的人的动作，那是决定之源。助手在三者之间只做翻译、搬运与核对——它提议、它执行、它留痕，它不作证、更不拍板。

边界画完，可以回头销账了。第3章冻结的三问：执行人在执行当日与作者的组织距离——距离计算加日期边，答得动了；一页签字怎样绑定对象、版本、职权与范围——四元关系加授权边，答得动了；一次调岗或重组击穿了什么——清晨的亮灯清单，答得动了。第3章末尾那句“纸面查不动此刻这句授权是否仍然成立”，如今是一个几秒钟的查询。但每一问的答案后面都站着一行小字：以受控记录为准，以承诺档位为准，以人最后看过为准。

**门禁核的是结构，不是眼神；机器保证该有的关系都在，保证不了该发生的挑战真发生了——所以门禁通过永远不是决定，它只是决定的地基。**

这条边界也决定了行业本体怎样学习。助手可以把反复出现的治理缺口整理成 candidate，但 candidate 只是一份带来源的提案。冲突、替换或删除必须由绑定的决定权威逐项写理由；commit 只冻结候选，回归与 release 要另算，promotion 还要一份绑定 exact delta 的独立授权。这样，“哪类授权会在重组后失效”可以被炼化成跨项目规则，而“方工今天是否仍有权签这份记录”仍只由当前受控人事与任命事实回答。Semantica 记得规则和谱系，不替组织作事实裁决，也不替任何人承担决定。

顺着这句话再问一层：为什么恰恰是门禁全绿之后，八分钟评完四十页的事会多起来？因为一排绿灯先替所有人说了一遍“没问题”。没有门禁的年代，评审的人清楚自己是唯一一道闸，那份“只有我拦得住”的孤责逼着他逐页翻；如今机器查过距离、查过授权、查过日期，他心里的位置悄悄从“把关的人”退成“补看一眼的人”——机器都没拦，我何必多事。责任没有减少一分，减少的是对责任的感受，而机器从头到尾只担保过结构。门禁越可靠，这道让渡越顺滑——这不是哪个人懈怠，是把关的感受被制度重新分配了；方工抽的偏偏是全绿的记录，抽的就是这道错觉最厚的地方。

地基打完了。试点例会的下一项议程，还缺什么？

## 13.9 表格里锁着的担心

下一项议程，是把当年补课重做的那份危害分析搬进机器。投影上打开那张熟悉的大表格：工况、事件、后果、分级、目标，密密麻麻几百行——概念评审会开了四轮才磨出来的东西，团队最硬的一份家底。

机器现在能立刻回答：这份分析谁写的、谁评的、评的人够不够远、谁有权签发它——四元关系一应俱全。小蔡却问了另一个问题：“这份分析在担心什么？哪些工况漏了？低温那一行的分级，凭的是什麼判断？”

图沉默了。在图里，这份分析只是一个带着签名链的文件对象。每一行都是一次具体的担心——车速、路面、驾驶员还剩几秒反应、可能伤到谁——可这些担心锁在表格的格子里，机器一行也读不懂。谁有权说，画进了图里；说的是什么，还留在纸上。

方工合上电脑前说了最后一句：“当年审核问穿我们的是‘谁负责’。下一个问穿我们的，会是‘你们到底在怕什么’。”

**治理本体回答“谁有权说”；它一个字也回答不了“说的是什么”——把担心变成机器可以推理的对象，是下一章的事。**

导读的承诺可以兑现了。合上书之前，做三件事：找一条你项目里最近的“已评审”记录，把它还原成四元关系——人、角色、对象版本、日期、凭据，数数缺几条边；给你手里权限最大的那个账号写一句话，说明它的写入权对应哪条裁决授权——写不出来的，就是你身边的方工旧案候补；再挑一条在途的授权，问它“此刻仍然成立吗”，数一数你要翻几个系统、问几个人才敢回答——那个数字，就是你的组织与本章之间的距离。

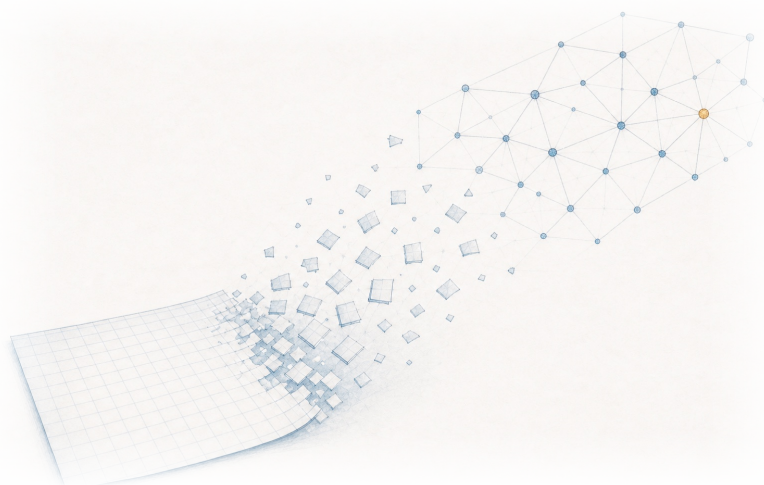
**本章注记：**本章的人物、试点场景、组织调整与全部数字（日期、时长、金额跨度等）均为合成教学材料，不对应任何真实企业、真实产品或真实个人；对功能安全标准中确认措施独立性与安全管理职责的转述为自然语言概括，精确要求以标准原文为准。文中各记法与查询片段只用于教学；唯一可执行版本是 `semantica.chapter_packages.vol2.ch13`，当前为 `partial`、`release blocked`。





## 第十四章 让担心可以被推理： 情境与危害本体

Semantica 绑定：本章保存情境、危害事件和目标的概念化；唯一机器语义是 `semantica.chapter_packages.vol2.ch14`，主场景为 `semantica.vol2.ch14.scenario.primary`。本体、CQ、查询、Shape、正反案例、规则、oracle、manifest 与 receipt 全部在 Semantica。当前为 `partial`、release `blocked`；`semantica.chapter_packages.vol2.normative` 仅提供工程释义映射，机器分级一致性不能代替标准原文核对或领域专家判断。



**【导读】** 上一章把“谁说了算”画进了图里：谁有权录入、谁有权评审、谁有权裁决，各自成了可以被查证的对象。可是当团队回头去找“我们究竟担心什么”时，发现这份知识还躺在概念阶段的表格里——一行一个失常，一格一个字母，理由散在三份纪要和一张白板照片之间。本章陪小蔡把那张表格拆开：先看表格为什么存得下结论、存不下判断；再让情境、失常、事件、三个判断和目标各自成为对象；然后把概念阶段那两份白板案卷第二次开庭，看同一个失常换一个情境时，机器怎样替人圈出必须重开的判断；最后回答一个更难的问题——图全部自洽之后，分级就正确了吗。读完本章，你应当能把风险分析表格里的任何一行，拆成情境、失常、事

件、判断、目标与假设各自的对象，并对其中任何一个可控性字母问出三件事：假定的动作是什么，反应窗口有多长，画像里的人是谁。

## 14.1 担心被锁在表格里

试点推进到这个星期，权限的问题已经不再吵了：录入、评审、裁决各归各的对象，机器拒绝过一次作者自评之后，没有人再把“走个流程”挂在嘴边。周二例会上，方工带来一件旧世界的日常：一家服务站报告，维修工位上一台车的方向盘自己动了一下，技师问要不要按安全事件上报。

这件事概念阶段其实判过。方工记得结论——车辆静止、无人在转向运动包络内的前提下，那个维修工位事件没有人身伤害，不分配等级——但他花了两个小时才把理由凑齐。表格在项目盘里，白板照片在某次会后有人随手拍的相册转存目录里，三份分级会议纪要两份有编号、一份只有日期；维修工位那一行甚至不在主表上，在一页“无需分级事件”的附页里。表格写着结论，可是“无人在运动包络内”这条边界押在哪里、押着谁，表格没有说。技师的问题恰恰踩在这条边界上：方向盘动的时候，有没有人正弯着腰？

上一章末尾大家就承认过：权限清楚了，可“担心什么”的知识还锁在表格里。现在这把锁真的挡了一次路。会上冒出三个补救方案，每一个都值得认真对待。

有人提议给表格加两列：“理由”和“依赖”。这不外行——很多组织就是这么做的。可试着填一下就知道：理由一落格，就成了另一段散文，机器读不出“这个字母押着那条假设”；下次有人问“假设变了哪些行要重开”，还是要靠人把散文重读一遍。又有人提议把纪要扫描件链接在每行后面。这解决了“找得到”，没解决“问得动”：链接的另一端是照片，照片不回答问题。还有人提议指定一位“活字典”，让当年参加分级会的人负责答疑。这是三个方案里最接近真相的一个，因为它承认理由住在人的脑子里；可它也最脆弱——人会调岗、会退休，而翻遍三份纪要，当年分级会上那三位专家连名字都没有留下，只有“碰撞安全工程师”“使用剖面工程师”“人因工程师”三个职务。活字典找不到人来当。

**表格存得下结论，存不下结论对世界的依赖。**每个字母都是一次判断的残骸——判断的对象、理由、证据和前提在会议室里活过一次，散会后只有字母被抄进了格子。小蔡把概念阶段冻结下来的问题重新念了一遍，翻译成这套新载体必须回答的三个能力问题。第一问对着情境：同一个失常出现在两个情境里，各自形成什么事件、走向什么后果，只换情境时哪些判断必须重开？第二问对着目标：每个目标回应哪个事件、哪种担心，通用的目标与标准意义上的安全目标怎样关联而不混同？第三问对着假设：一条假设被推翻时，哪些分级和目标必须被点名重开，而不是悄悄留在原地装作事实？三问都不新——它们正是当年白板案卷冻结时留下的问题，只是那时没有任何载体能把答案查出来。

要让机器接住这三个问题，第一步不是写查询，而是让表格里挤在一行的那些词，各自站出来成为对象。哪些词？

## 14.2 让一行表格站成七个对象

把“非预期助力／高速／S3 E4 C3／D”这一行放在解剖台上，至少压着七样东西，每一样回答一个不同的白话问题。情境：这次判断在什么运行、环境、人群和模式的条件边界内适用？失常行为：相对期望行为发生了什么偏差？危害事件：哪条失常在哪个情境里展开——注意是组合，不是失常自己？后果：这条分支在明示假设下可能走到什么结果？三个判断：分别拿什么证据，判到哪个字母？目标：为了回应这个事件，要防止、限制或检出什么？假设：什么前提正在底下垫背？本章用 **haz:** 前缀给它们各立一个对象。

关键的一步在事件。失常本身没有等级——给“非预期助力”直接配一个等级，正是当年白板差点犯下、后来被两幅对照图撞破的错。有资格进分级会的从来是“失常 × 情境”这个组合。概念阶段的口径说得更细：失常先在整车层显形为危害，危害再与运行情形组合成危害事件；本章把中间那环收进事件对象里，用情境这个座位安放运行情形和它的受控条件——压缩的是写法，不是链条。所以事件不是一个名字，而是一个带两条边的对象——它由哪条失常、在哪个情境里组成，两条边缺一不可。情境自己也不许偷懒成一个字符串：“高速”两个字不是情境，车速范围、道路类型、车辆模式、周围人群和时间边界的那组受控条件才是。条件写全了，后面“换一个条件算不算换了情境”才有得判；写不全，两个团队会拿着同一个词各想各的路。下面这段记法把高速事件和它名下的可控性判断立出来；注意字母在哪里：它不是行，也不是结论，

只是判断对象身上的一个属性。

# 一行表格拆开后的样子（示意记法）

```
haz:HE_Highway a haz:HazardousEvent ;
    haz:ofMalfunction haz:MB_UnintendedAssist ; # 失常：非预期助力
    haz:inContext      haz:CTX_Highway ;         # 情境：高速道路行驶
    haz:leadsTo        haz:CONSEQ_LaneDeparture . # 后果：偏离车道或碰撞
haz:CJ_Highway a haz:ControllabilityJudgment ;
    haz:judges         haz:HE_Highway ;          # 判断的对象是事件
    haz:level          haz:C3 ;                  # 字母只是判断的一个属性
    haz:evidenceKind   haz:HumanFactorsEvidence . # 这类判断只认人因证据
```

这段记法里最值钱的不是类名，而是最后一条边：三个判断各自绑定自己的证据类型。严重度判断只认伤害与后果分支的证据——它的问题是“落在人身上会伤到什么程度”，受险者不止本车驾驶员，样本要代表真正会遇到这个事件的人群。暴露概率判断只认使用剖面的证据，而且要声明口径——按时长还是按频次；它的主语是运行情

形，不是故障，拿“传感器不常坏”来给暴露概率打折，等于用还没实现的设计替自己的要求作证，这类证据在图里连门都进不去。可控性判断只认人因证据——它问的是“这群人在这段窗口里做不做得成这个动作”，答案只能从真实的人身上量出来，伤害的账和使用的账都替它答不了；动作、窗口、画像，下一节它会咬人。这不是分类洁癖：概念阶段那场分级会上，三个字母第一次报得不一样，正是因为它们本来就是三种不同的证据问题。表格把三个问题压成三格，图把它们还原成三个对象。

担心本身也是对象。同一个事件可以被“道路使用者的安全”评价，也可以被“服务连续性”评价，证据和出口随之不同：前者走分级查表，后者只产生一个没有等级的通用目标。目标因此也分两层：通用的目标说“要防止、限制或检出什么”，标准意义上的安全目标是它在安全这条行业纵线上的投影，带着事件和等级。两者之间画一条显式的投影边，不写等号——名字相似不是身份合并的理由，这条纪律第十二章已经用另一场撞名乌龙付过学费。

还有一条从入门就立下的老规矩在这里继续生效：图里允许留空。当年概念阶段没有证据给出故障容错时间，案卷里写的是“待定”；录入时它就保持一个明示的未知状态，不许被一个漂亮的毫秒数补齐。机器判断“缺了什么”之前，先要有人声明“这部分已经记全”——没有这句声明，空只是空，不是错。

骨架立起来了。概念阶段那两份白板案卷放进去，会发生什么？

**读图任务：**先看失常与情境如何共同构成中央事件，再沿三条下行边检查三把尺下面是否各自挂着不同类型的证据。



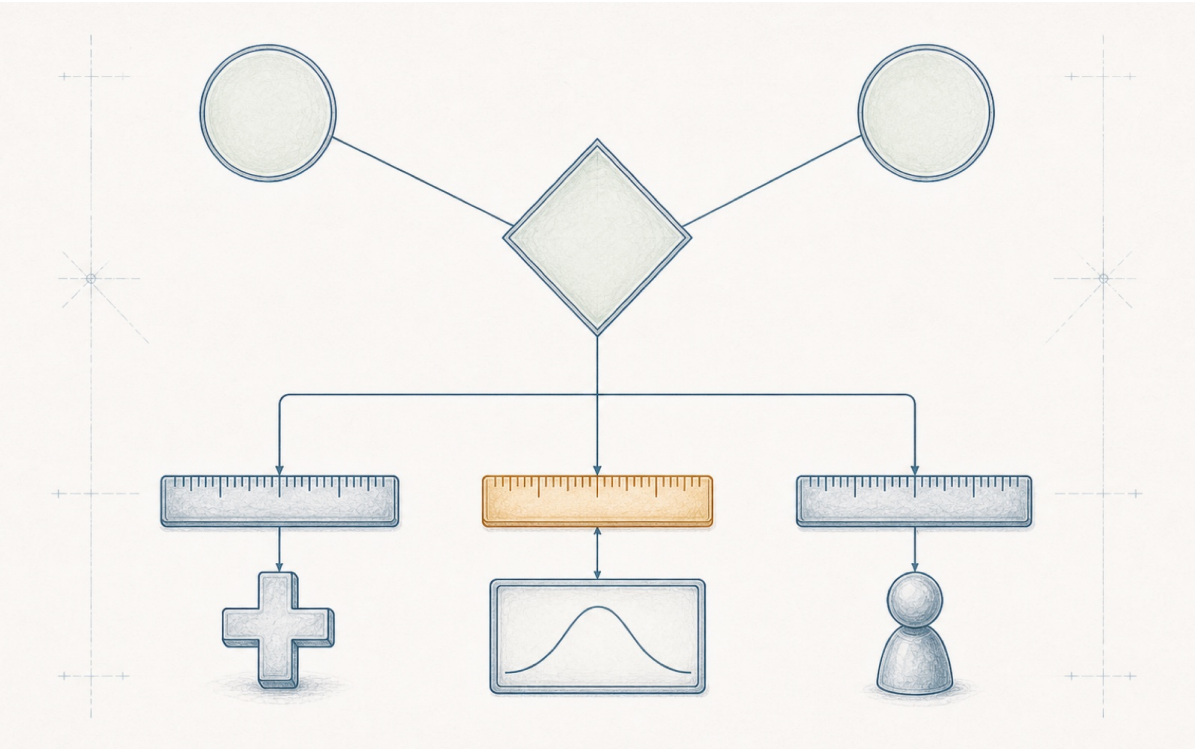


图 14-1 风险评价的对象是条件化事件，而不是裸露的失常名。严重度、暴露度和可控性是三种不同测量主张，它们需要不同的来源与适用条件。

**图后判断：**图中三把尺不产生评级数值，也不展示 ISO 26262 查表过程。它只规范事件身份与证据类型，不证明任一 ASIL 结论。

### 14.3 同一失常，第二次开庭

录入用了一个下午。AI 助手先读扫描的案卷和纪要，提议出候选对象和边；每条提议过一遍语义检查；通过的由小蔡确认后写入；每次写入留下一条记录——谁提议、什么检查、谁批准。助手全程没有直接改过图：它提议，机器校验，人执行，账留底。提议也不总是对的。第一轮里助手把纪要中的“转矩传感器断线”提成了事件候选，小蔡当场驳回——那是内部故障，不是整车层的危害，当年的案卷对此写得很清楚。提议可以错，错得起，因为写入之前有人握着闸；这一圈下来，图里每个对象都能回答“你是谁放进来的、凭什么”。

维修工位案卷进来时，小蔡刻意让它和高速案卷共用同一个失常对象——这正是当年



白板上的实验：不换车，不换失常，只换情境。

```
# 小蔡录入的两份案卷（节选）
haz:HE_Workshop a haz:HazardousEvent ;
    haz:ofMalfunction haz:MB_UnintendedAssist ;    # 同一个失常对象
    haz:inContext      haz:CTX_Workshop ;          # 只有这条边不同
    haz:leadsTo        haz:CONSEQ_UnexpectedMotion .
haz:SJ_Workshop a haz:SeverityJudgment ;
    haz:judges haz:HE_Workshop ; haz:level haz:S0 ;
    haz:dependsOn haz:ASM_NoPersonInEnvelope .      # S0 押在这条边界假设上
haz:SG_NoUnintendedAssist a haz:SafetyGoal ;
    haz:respondsTo haz:HE_Highway ;
    haz:hasASIL     haz:ASIL_D .                  # 等级只挂高速事件的目标
```

服务站那个问题现在有了着落：维修工位的“无需分级”不再是表格里一句孤零零的话，而是一个严重度判断对象，明明白白押在“无人进入运动包络”这条假设上。技师的追问——方向盘动的时候有没有人弯着腰——正是在敲这条边。而高速事件的安全目标带着等级挂进图里时，维修工位那条线只得到一个没有等级的通用目标：从服务连续性的担心出发，在恢复作业前限制或检出意外转向运动。问一句“每个目标回应哪个事件、哪种担心”，两行答案各归各位，谁也没有借谁的等级。

接着是第一个能力问题的正面回答。

问：同一个“非预期助力”失常，出现在哪些情境里，各自形成什么事件、走向什么后果？

```
SELECT ?event ?context ?consequence
WHERE {
    ?event haz:ofMalfunction haz:MB_UnintendedAssist ;
        haz:inContext      ?context ;
        haz:leadsTo        ?consequence .
}
```

答：

| 事件          | 情境     | 后果          |
|-------------|--------|-------------|
| 高速非预期助力事件   | 高速道路行驶 | 偏离车道或碰撞     |
| 维修工位非预期助力事件 | 维修工位静止 | 意外转向运动与作业中断 |

两行，不多不少。然后小蔡做了当年白板上没人能当场做的事：有人问“如果情境换成低速拥堵排队呢？”在表格世界，防止有人把高速那行照抄过去，只有两个老办法：在备注栏写“此行仅适用于高速”——备注靠人读，赶进度的人不读备注；或者立一条模

板规矩“复制行后必须清空分级列”——清空靠人记得，而且清空之后没有任何东西告诉你缺了什么、该拿什么补。小蔡不用这两个办法。她新建一个情境对象，把同一个失常挂上去，机器立刻承认第三个事件成立——但这个事件名下空空如也：没有严重度判断，没有暴露概率判断，没有可控性判断，没有目标。查询返回的不是答案，是三把空椅子，每把椅子还标着自己只认哪类证据。空椅子算数，靠的不是查询聪明，而是事件这个类型自带的合同——凡是事件，三种判断各须坐满一把椅子；判缺的资格来自这条事先写下的合同，正对得上前面那条老规矩：先有“此处必须记全”，空才是缺。旧事件的判断一条也没有爬过来，因为判断的边全部钉在各自的事件上。

在表格世界里，“换了情境不得复制分级”是一条要靠评审员记住的纪律；在图里，它成了一个几何事实——旧判断根本没有边可以爬到新事件上去。**重开清单不是查询的本事，是建模时画下的每一条依赖边在付利息。**当年那位整车工程师问“你们这套措施，精确在回答哪一件事”，现在这个问题每天都被机器替他问一遍。

机器能摆出空椅子。可一份坐进椅子的判断，它怎么分辨那是判断，还是敷衍？

## 14.4 字母背后必须站着动作、窗口和画像

高速案卷的可控性判断进来时出了岔子。助手从纪要第一页里读出“C3”，提议了一个只有字母的判断对象。旧世界对付这种空心字母有两个办法：一是培训评审员多问一句——概念阶段那位人因工程师就问，问得极好，可他的问题散会就蒸发了；二是在模板

里加必填格——纸面必填拦不住空格，拦不住“抄上一行”。这一次，载体自己开了口：

### # 机器的拒绝报告 (示意)

被检对象: haz:CJ Highway

## # 高速事件的可控性判断

合同要求: assumedAction / reactionWindow / personProfile 各恰一条

实际情况：三条全缺

结论 : 拒绝写入; C3 退回候选, 不进入查表

小蔡补上假定动作“反打方向并制动”，再交——机器仍然拒绝，这次的报告只剩两行：窗口缺，画像缺。缺哪样，拒哪样，一样一样地讨。有人不服气：C3 主张的本来就是“控不住”，控不住还要什么动作和窗口？恰恰更要。说“控不住”，说的是某一群人在某个长度的窗口里做不成某个动作——抽掉画像和窗口，这个判断换一群人、换一种转矩上升就悄悄变了意思，而没有任何东西提醒它变了。直到小蔡翻出纪要的第二页，才发现三样东西早就有人问齐了，白纸黑字：“你们所谓驾驶员能救回来，假定他做什么动作、还剩多少时间？”提问的人在案卷上没有名字，只有“人因工程师”五个字。小蔡把动作、窗口、代表性人群画像逐一录入，数据接收门禁接受了这条记录；这只代表结构满足入账合同，不是危害评级、风险接受或产品放行。

那一刻小蔡意识到，例会上找不到的那位“活字典”其实一直都在。碰撞安全工程师的 S3 连着后果分支，使用剖面工程师那句“还没有数据”变成了一条状态为计划中的假设，人因工程师的三连问变成了可控性判断的三条必填边。三个没有留下名字的人，如今各自守着一类证据。问题的作者会离场，问题不再离场。

证据类型这道锁同样咬人：有人顺手把一份人因观察报告挂到暴露概率判断上，机器拒绝——暴露概率只认使用剖面证据，还追问口径是时长还是频次；拿元件故障率来充暴露概率证据，同样进不了门，那是另一本账。让“字母背后必须有动作、窗口和画像”从评审员的好习惯，变成载体的拒收条件。这就是本章对“三把尺子”的机器化：不是替人打分，而是替人守住“没有证据对象就没有判断”这条底线。

要说破的是这道门禁的天花板：要是有人在动作栏里填一句“驾驶员注意安全”呢？机器确实分不出空话。它锁的是部件在不在、类型对不对，不是判断好不好；空话仍然要靠评审的人眼挑出来。而且要看到，必填栏立起来之后，空话不是变少了，是搬家了：门禁把“合格”交给机器定义——三条边都在，就是合格——赶进度的人自然走满足定义的最短路径，栏是机器要的，就填给机器看；形式越完备，“填过了”越容易被错当成“想过了”，可检查的部分吸走全部注意力，检查不到的部分自动退进暗处。但空话从此有了一个固定的位置可供挑剔——评审的人不必再去三份纪要里找它，它就钉在判断对象的那条边上。

可是，当每一道检查都亮了绿灯，D 级就算成立了吗？

## 14.5 图的自治，买不来分级的正确

周五评审会上，小蔡把跑完的结果投在幕上，说了一句她此刻完全有资格说的话：“这回不一样了——图是自治的，检查全绿。S3、E4、C3，D 级，这是机器背书过的结论。”

这句话不外行。检查是他亲手建的，每一条都咬过人；相信它们，再自然不过。方工没有否定，只问了一个问题：“E4 的证据对象是什么？”

小蔡点开暴露概率判断。证据边的另一端不是一份车队统计，而是一条假设——“高速使用剖面”，状态：计划中。机器要求的是“判断必须挂证据对象、类型必须匹配”，它从头到尾没有说过、也没有能力说“这条假设为真”。图里的 E4 是一个语义合格的候选判断，不是一个被世界证实过的事实。可控性那格更明显：动作、窗口、画像三条边齐了，可窗口里那个秒数至今没有做过一次真实的人因试验；机器守住了“必须有窗口”，守不住“窗口是多少”。本体守得住语义的自治；分级的正确，仍然要用领域证据与专家判断去买。

小蔡沉默了一会儿，然后把幕布上那行结论亲手改成了“候选，待证据”。这不是认输——恰恰是这套载体教他的：全绿的准确含义是“结构无可挑剔”，而结构无可挑剔的错误结论，比结构混乱的错误结论更危险，因为它更像真的。

这正是三种来源各归各位的时刻。图管语义：判断必须成对象、必须绑对证据类型、依赖必须画成边——违者拒收。受控的工程活动管事实：只有一次真实的使用剖面研究、一场真实的人因试验，才能把“计划中”变成“已验证”。有权的人管决定：带着一个候选 D 级继续往下开发，是方工在这一页上签下的风险接受，不是任何一条查询的输出。三者少了哪一个，另外两个都会僭越。

方工随手做了个演示，把那条使用剖面假设的状态改成“已拒绝”，问机器谁受影响。

```
SELECT ?reopened
WHERE {
    ?reopened haz:dependsOn haz:ASM_HighwayProfile .
    haz:ASM_HighwayProfile haz:status haz:Rejected .
}
```

答案三行：暴露概率判断、由它进入查表的分级结果、连同等级的安全目标。三样东西被点名重开，一样也没有被悄悄改成新事实。当年这种影响圈只存在于最资深的人的脑子里；现在它是任何人都能跑的一条查询。

同一个机制，第二天就接住了开场那件事。回复服务站之前，方工先在图上问：维修工位的“无需分级”押着什么？答案一条——“无人进入运动包络”。技师说方向盘动的时候有人正弯着腰，等于这条假设在那台车、那个工位上不成立。机器点名严重度判断重开，“无需分级”当场收回，事件按开放项处理，谁来重判、谁有权关闭，走的是上一章画好的那套授权关系。两个小时的翻档案，变成了两分钟的两条查询；更要紧的是，担心第一次以对象的身份接住了现场。

边界也要试反面。小蔡把试点里那台环境监测节点的湿度漂移放进同一副骨架：仓储趋势监测与校准台检查两个情境、同一条漂移失常、各自的后果、“测量稳定性”的担心、两个候选目标，全部立得住——问法是通用的，当年概念阶段就验证过这一点。可当助手依样画葫芦，提议给漂移事件也配一个等级时，机器拒绝：分级查表只在安全担心的投影支路上存在，漂移场景没有资格进那张表——它该去找的是量程包络、参考仪器和带时间戳的序列数据，那是另一个担心的另一套证据。骨架迁走了，安全的量尺没有跟着走私；这道拒绝和放行同样重要，因为把等级借给不相干的领域，是比缺一条边更隐蔽的败坏。

诚实的边界要在这里说满。这张图不制造事实：它管不了高速上真实的伤害分布，管不了真实驾驶员还剩几秒；它也不拥有决定权：全绿只表示这张局部图对已经写下的问题给出了与预期一致的回答，放行、验证、授权都在图外。本章示意结构的唯一机器投影位于 `semantica.chapter_packages.vol2.ch14`。它可以运行 `manifest` 声明的正反案例、查询与 `oracle`，却仍标为 `partial`，`release` 仍为 `blocked`；所以这里能说的是“受限场景可被复核”，不能说“完整实现已经通过”。

散会前方工在图上停了一下：危害定下了目标——目标现在是一个对象，带着事件、等级和一身假设。可它马上要离开概念阶段了。目标离开这间屋子之后，怎样不丢？

## 14.6 目标出发去下一站

当年白板上那三样东西——双信号比较、关断、告警——在这张图里依然没有位置，这一次不是被纪律拦住，而是被结构拦住：它们是回应目标的候选实现，属于需求的世界，不属于危害的世界。图替当年的评审会守住了那条最容易破的线：先说清担心什么、要达到什么，再让方案来竞争。

方工在散会前提醒了一句更远的事。目标接下来要走的路，他见过它在表格世界里怎么丢：一条目标派生成十几条需求，分配给系统、硬件、软件，每一次转手都合理，合起来却没有人能回答“这条需求究竟在替哪条目标作证”；等到某个数字丢了条件、某个接口丢了单位，追回去的路已经断了。危害的图到目标为止；目标之后的每一步转手，需要它自己的对象和边。

你可以现在就做一次本章的动作：拿你自己项目里风险表格的任何一行，把它拆成情境、失常、事件、三个判断、目标、假设七样对象，再对那个可控性字母问三件事——动作、窗口、画像。拆不动的地方，就是你的担心被锁住的地方。这也是导读那张期票的兑现：一行表格，七个对象，三个问题。

散会后小蔡做了一件小事：把那张白板照片重新扫描，挂进案卷对象的来源边里。照片还是那张照片，上面还是当年那几笔没擦干净的方案；但它第一次有了地址，有了指向它的问题，有了会替它守夜的门禁。写下它的人没有留下名字，写下的东西留了下来。

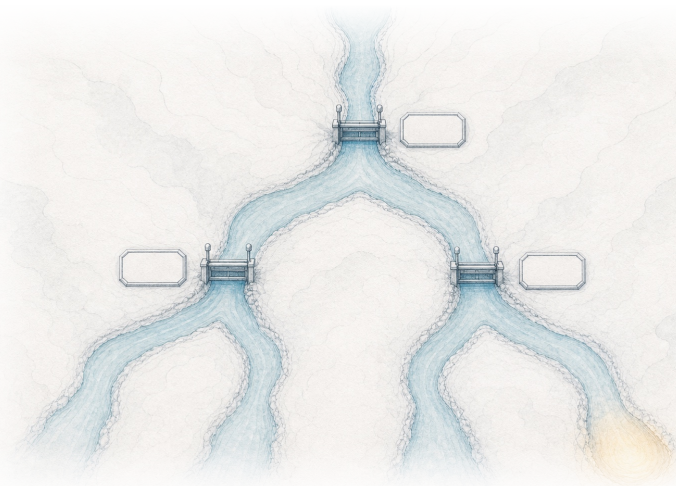
而下一站的难题已经在桌上。目标从这里出发，要被派生成需求、分配给系统、硬件与软件，在一层层转手中穿过接口和预算——在表格的世界里，目标恰恰就是在那条路上丢的。承诺怎样不丢地流进需求？下一章，让追溯自己长出边来。

**本章注记：**本章人物、事故、案卷与全部编号均为合成教学材料；EPS 与环境监测节点的情境、判断和等级不描述任何现实产品。正文片段只用于教学；唯一可执行版本是 `semantica.chapter_packages.vol2.ch14`，当前为 `partial`、`release blocked`。场景 `oracle` 通过只表示局部图对已编码问题的回答与预期一致，不证明现实分级正确、目标充分或任何工程决定成立。



## 第十五章 承诺不再丢失： 需求与追溯本体

Semantica 绑定：本章保存派生、分配、接口和重开关系的解释；唯一机器语义是 `semantica.chapter_packages.vol2.ch15`，主场景为 `semantica.vol2.ch15.scenario.primary`。本体、CQ、查询、Shape、traceability 案例、规则、oracle、manifest 与 receipt 只在 Semantica。当前为 **partial**、release **blocked**；闭世界检查只在声明范围内成立，不能证明现实依赖已经记全。



**【导读】** 前一章的试点把“担心什么”建成了机器可查的结构，危害定了目标；可目标要落地，还得流进需求——而系统层那本旧账我们都记得：一场开了两天的会议，一份四十多项、没有人敢担保完整的重开清单。本章陪小蔡把那本旧账搬进新载体：先看一条需求怎样从一句话长成一个“缺件即拒”的对象；再看连线怎样被逼着交出理由、分配怎样被逼着等来确认；然后守在接缝的门口，看当年那个“典型值 2 毫秒”第二次登门时撞上什么；再给一笔至今待定的时间预算立起账本；最后回答陈工当年那个没人答得动的问题——“哪些东西要重开”。读完本章，你应当能把一条需求写成机器可以拒收的对象，并在上游变化的那天，让机器先于会议室交出重开清单的第一版——同时说得这份担保的边界画在哪里。

## 15.1 四十多项的清单，翻出来重问

试点推进到需求这一站的那个早上，小蔡做的第一件事，是把档案里那份清单调了出来：四十多项，字迹来自好几个人，是当年梁工那封传感器换代邮件之后、整个项目在会议室里投影矩阵、逐行认领、开了两天才凑出来的东西。散会时没有一个人敢担保它是完整的——这句记录，就写在清单末尾。她把清单投在试点室的屏幕上，旁边是团队的 AI 助手的工作区。助手在这个房间里的地位从试点第一天起就没变过：它提议、它执行，它不是事实的来源，也不是任何事情的决定者。

小蔡自己也不是当年会议室里的旁观者。那两天她刚入职不久，负责给认领结果做记录，亲眼看着一群她敬重的工程师靠记忆打捞自己半年前的决定。散会时她在笔记本上写过一行字：“我们不是不认真，我们是没有地方放这些东西。”现在，地方有了——问题是往里放什么。

安全目标已经在上一站被建成了机器可查的对象，经由显式的对齐合同交到需求世界门口；本章不重建危害与目标的语义，只在自己的边界内把它们当作上游的锚点。问题是接下来这一步：目标流进需求的路上，纸面时代丢过三样东西，系统层的冻结报告写得清清楚楚。其一，追溯靠人手维护，账本永远慢于事实；其二，派生的理由不留痕，纸面记得住“有关”，记不住“为什么有关”；其三，影响算不出来——改一条上游，没有任何机械的办法列出下游哪些要重开。

小蔡把这三条翻译成新载体必须回答的问题，写在白板上：

- 任何时刻指着一条需求问“它完整吗”，机器要能回答缺什么；
- 任何一条派生连线，机器要能回答“为什么有关、依赖了上游的什么”；
- 任何一次分配，机器要能回答“接收方确认了没有”；
- 改一条上游，机器要能交出下游重开清单的第一版，并说清这份清单担保到哪里。

陈工看完白板只补了一句：“最后一条，就是我当年问小唐的那句——哪些东西要重开。这回我想听机器先答。”

要让机器答得动这四问，得先解决一件更基本的事：在机器眼里，“一条需求”到底得是什么东西？

## 15.2 一条需求，长成一个对象

纸面时代的需求是一句话，一句话住在表格的一格里。系统层那一章炼出过一条清点纪律：每一次交接都要重新清点触发、模式、动作、判据、时限，少一样就丢一样。纪律是对的，丢的方式也早看清了——那句“检出偏差后请求抑制助力”，拆分之后三个承担者每人领走一个动作，“多快”不在任何一条里。

怎么防？第一反应是把模板做硬：给需求文档加五个必填栏。这一招很多团队试过，败得很安静——纸面的必填栏得住空白，拦不住敷衍，“时限”一栏里填着“见上文”“尽快”“待定”的表格，每个项目都能翻出几页。第二反应是设专人复核：每条需求录入后由资深工程师过一遍。这一招也不算错，但复核也是人手，人手就有周期，周期就慢于事实；而且复核标准仍在复核人的脑子里，人一换，标准跟着换。

新载体换了一个思路：不把“完整”当成一种事后审查出来的品质，而把它写进“需求”这个词的定义里。下面这段记法在说什么：它声明了一条技术安全需求，

五个要素各占一行——在这个世界里，五行缺一行，这个对象根本不成立。

# 一条技术安全需求（示意记法）

```
req:TSR_TorqueCompare a req:Requirement ;
  req:trigger      " 两路转矩信号偏差越过受控阈值" ;      # 触发
  req:mode         req:NormalDriving ;                      # 模式：正常行驶
  req:action       " 请求抑制助力并转入降级" ;              # 动作
  req:criterion    " 偏差判定与降级进入均可在台架复现" ;   # 判据
  req:timeLimit    req:BudgetItem_Detect . # 时限：指向账本的一格
```

注意最后一行：时限不是一个随手填的数，它指向时间账本里的一格——这一格现在还空着，我们到预算那一节再回来收它。

这个门槛不是跟人过不去。触发说清它盯着什么，模式说清它在系统的哪种状态下生效，动作说清谁做出什么反应，判据说清拿什么判成败，时限说清多快才算数——五样合在一起，才是一句别人可以照着做、也可以照着验的承诺；少任何一样，下游就得靠猜，而猜出来的分歧不会自己暴露，要等台架或者路上才见面。而且缺哪一样，猜就长在哪一样上：缺时限的下场全书都看过了；缺模式，抑制逻辑在检修工位上也照样出手，诊断设备刚连上车就被请了出去；缺判据，开发的人按自己的理解测完说“好了”，验证的人按另一种理解判“不过”，两份报告都是真的，吵到最后才发现两边验的根本不是同一句承诺。系统层的正文早把这个道理讲透了，新载体做的只是一件事：把道理变成门。

迁移旧矩阵是 AI 助手的活：它读旧表格，把一行行句子提议成一个个候选对象。

第一批迁移就撞了墙。下面这段示意的是机器怎么拒绝——被拒的正是当年那句话：

# 机器怎么拒绝（示意）：缺件的句子进不了需求库

```
输入： " 检出偏差后请求抑制助力"          # 从旧矩阵迁来的原句
裁定：拒收 ——不构成 req:Requirement
缺件： req:mode（模式）、req:timeLimit（时限）
处置：降为 req:Memo 暂存，补齐后重新提交
```

小蔡盯着“缺件”那一行看了很久。当年这个洞，是梁工第一次到场、以一个局外人的直觉问出来的——“要多快，才算够快？”靠的是运气：恰好有一个不欠这个项目任何默契的人坐在会议室里。现在同一个洞，是录入的那一秒被机器指出来的，不需要任何人恰好在场。被拒的句子没有被扔掉，它降级成备忘，留在暂存区等人补齐——机器拒绝的不是这句话，是它冒充需求的资格。

**需求的完整性不该是审查出来的品质，而应是存在的门槛：五样凑不齐的句子，可以是备忘，不允许是需求。**

对象立住了。可四十多项清单的病根不在对象上，在对象之间——那些线，机器怎么知道它们为什么在那里？

### 15.3 连线要带理由，分配要等确认

迁移进行到第三周，小唐来看图。屏幕上，需求对象一层层挂着，派生连线一条条校验通过，分配一条不缺。小唐是当年那张全绿矩阵的作者，他看了很久，说：

**“上次全绿是我手填的表，这次全绿是机器逐条查过的图——每条线都在，每个对象都齐。这回，总算是都接住了吧。”**

这句话比当年那句更不外行。机器校验的图确实严格好于手维护的矩阵：矩阵记录的是上次有人有空时的世界，图的每一次变更都当场过检；矩阵的格子可以悬空，图里悬空的对象根本存不进去。如果这样还不算接住，问题一定藏在校验照不到的地方。

它藏在理由里。评审会上，梁工顺着比对需求的派生线往上看，停住了：AI 助手迁移时给这条线补的理由写着“比对窗口由上游检出时限派生”——通顺、体面、结构齐全，校验全绿。可梁工记得真实的历史：当年那个窗口，依据的是他老款传感器的最坏刷新，跟上游时限没有直接关系。做这个决定的工程师已经离职，理由不在任何纸面上，助手读不到，就按最像的样子补了一个。线是真的，理由是编的。

怎么办？第一个提议最省事：让助手把全部理由都生成了，反正它写得又快又通顺。不行——这正是全书反复立过的界碑：模型的提议不是事实，通顺不是为真，一条被机器猜出来的理由，比没有理由更危险，因为它长得像证据。第二个提议是把校验规则加严：理由不许是一段散文，必须指认“依赖了上游的哪个具体部分”。这一条被采纳了——有了锚点，理由第一次变得可以被证伪；但采纳之后大家发现它仍然不够：锚点保证理由可查，保证不了理由为真。为真只有一条来路：一次有记录的确认活动——评审、会签，或者像梁工这样当场指认历史。

于是这条线被打回“提议”状态，走了一整圈才重新变绿。这一圈值得慢放，因为它就是助手在这个世界里被允许的全部动作：第一步，提议——助手根据旧矩阵和会议纪要给出连线和理由的候选，候选带着明显的记号，任何查询都不会把它当成事实取走；第二步，校验——机器查结构，五要素、锚点、出处，缺一样打回；第三步，确认——梁工在评审里给出真实依据，小唐署名确认，这一步只能由人完成，候选从这一刻起才是事实；第四步，审计——谁提议、谁改写、谁确认、发生在哪一天，全程留痕，



半年后有人再问“这条线谁认的”，不需要任何人的记忆在场。那条编造理由的线，正是在第三步被拦下的——前两步机器全绿，恰好证明了机器的绿灯照不到“为真”。

下面这段记法在说什么：它是那条改正之后的连线——理由不再是一段话，而是一个对象，带着锚点、说法和出处。

```
# 一条带理由的派生连线（示意记法）
req:Der_042 a req:Derivation ;
    req:fromReq req:FSR_DetectAndMitigate ;      # 上游：功能层承诺
    req:toReq    req:TSR_TorqueCompare ;          # 下游：比对需求
    req:rationale [ a req:DerivationRationale ;
        req:dependsOnAspect req:Param_SensorUpdate ; # 锚点：
        传感器更新特性特性
        req:justification    " 比对窗口按老款传感器最坏刷新设定" ;
        req:confirmedIn      req:ReviewRecord_P15_03 ] . # 出处：
        本次评审记录
```

锚点那一行请记住——它指向的不是一条需求，而是接缝上的一个参数。这个细节要到本章最后一节才显出它的分量。

分配也是同一个道理。旧矩阵里“每条需求都有主儿”，可没有一处记录着主儿点过头——认领发生在邮件里、走廊上、周会的口头应答里，等到要追问“这条你当时是怎么理解的”，对话早已蒸发。新载体里，一次分配在接收方确认之前一直处于“悬空”状态，悬空不是错误，但它挂在图上人人可见，且不允许被任何下游结论引用；确认也不是点一下按钮，接收方确认的内容被写明了——确认的是这条需求的五要素在我的边界内可实现、可验证，基于此刻图里记录的接缝参数和预算格子。基于什么而确认，日后就凭什么而重开。到这里，三样东西第一次在情节里站成了三角：本体规定什么算一条成立的连线——这是语义；连线为真，来自评审与会签这些受控活动——这是事实；而当两个团队对“当年依据”各执一词时，拍板的仍然是有权的人——那天下午有一条争议线，最后是陈工裁的。三样各归各位，谁也替不了谁。

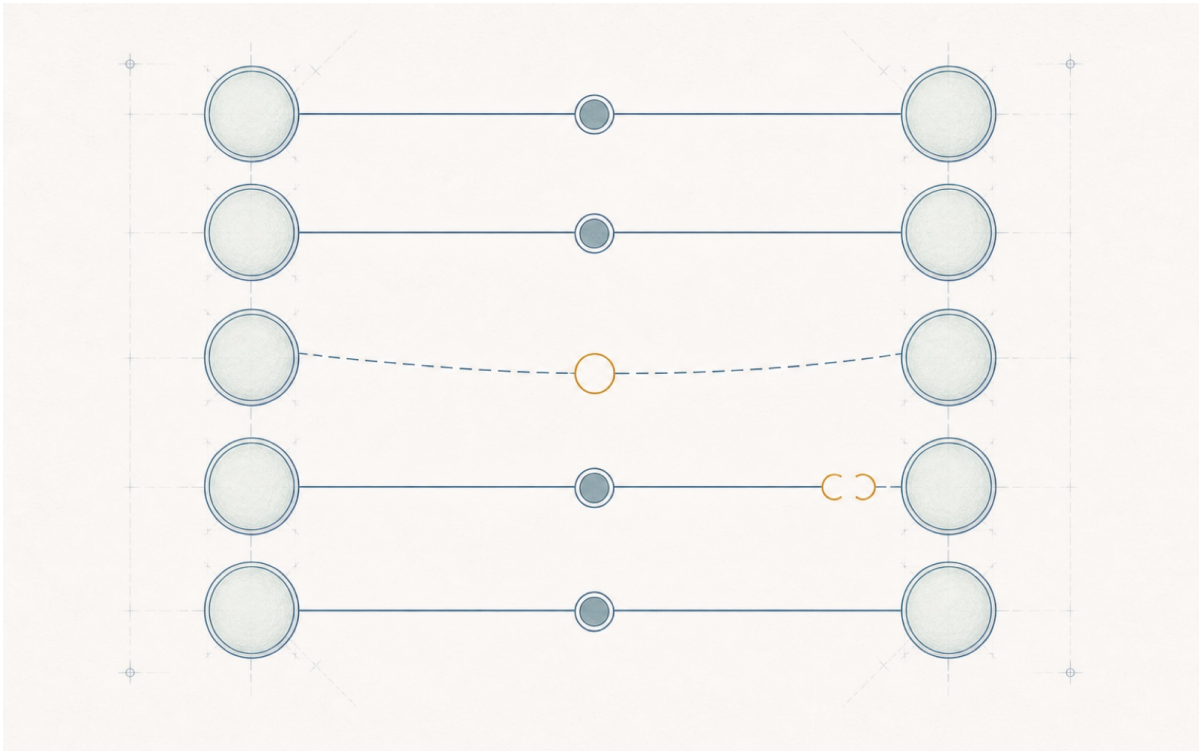
**连线证明的是存在，不是成立；一条派生要成立，得说得出了依赖了上游的什么——而“为真”永远来自确认它的活动，不来自画出它的手。**

这条界线值得追问一层：为什么一代又一代的组织，都心甘情愿把“全绿”当成“接住”？因为存在查起来便宜，成立查起来贵。线在不在，外人扫一眼就能数完；理由真不真，得把当年的历史请回来才问得动。于是评审对着看得见的连线提问，审核对着留了痕的步骤抽查，考核给数得出来的完整率发奖——三股压力全都压在“形”上，没有一股压得到“义”。一条编得通顺的理由在这种环境里不但活得下去，还活得最好：它喂饱了每一次检查，把唯一拆得穿它的那个人——记得真实依据的人——留给运气。全绿不是谎言，它只是把没人查得动的那部分，藏进了绿色里。



线也立住了。可小蔡心里还悬着一件事：当年最疼的那一跤，不是摔在需求上，是摔在接缝上——那个 2 毫秒，换了载体还会不会重演？

**读图任务：**跟随左右节点之间的每条连线，检查线中间的理由牌是否存在，再找到一条理由缺失和一条末端搭扣尚未闭合的关系。



**图 15-1** 需求追溯不是两个 ID 之间的裸连线。每一次派生、分解、分配或满足都要交代理由，而分配边在承接方确认前仍然是待定状态。

**图后判断：**本图不会因连线完整就证明需求正确，也不代替技术实现、验证或组织授权。理由对象自身仍需可追溯事实支持。

### 15.4 两毫秒，第二次来到门口

低温台架那三天，这个项目的老人都记得：手册是真的，表是认真填的，代码是照表写的，助力还是时断时续。病根事后看得清清楚楚——接口表誊抄时留下了“2”，丢掉了“典型”两个字和那行低温条件；事后的补救也做得不可谓不重，两百多行逐行补了“条件”和“最坏值”两列，重新会签。

可小蔡把当年的补救翻出来看，越看越不安：补上的是那张表，不是抄表这件事。只要接缝上的数字还靠人从文档往表里搬，“典型值”三个字就总有一天会再次被搬丢。有人说，加强培训，录入前先读脚注——可当年那张表恰恰是逐行核对过的，认真拦不

住结构性的洞。有人说，把供应商手册整册挂进系统，随时可查——可见不等于承诺，这个坑当年就排除过一次：手册是对器件的描述，不是对这个项目的担保，挂得再近，脚注照样可以被路过。

新载体的做法，是把当年补进表里的那两列，升格为接缝参数这个对象的构成条件。试点里立的规矩：接缝上的每一个数字，必须以参数对象的身份入库，值、成立条件、最坏值、最坏值出现的条件、坏了怎么说、谁承诺——缺一个字段，这个数进不了门。

重演的那一幕发生在一个很平常的下午。一位新来的工程师把老接口合同往载体里迁，敲到传感器那一行，顺手录入：“更新周期，2 毫秒。”回车。机器拒收：缺“成立条件”，缺“最坏值”。他愣了一下，回头去翻合同，把补签过的那两列找出来，重新录入，这次过了。前后不到十分钟，没有台架，没有零下三十度，没有五周的重新排队。

消息传到软件组，小林特意走过来看了一眼屏幕。当年说出“接口表都填满了，两边照表干活就行”的人是她，低温里查了三天自家滤波和去抖的人也是她。她看完那条拒收记录，只说了一句：“当年要是这道门，我那三天就不用查了——门口就断案了。”梁工看完那条拒收记录，说了一句话，后来被小蔡记进了试点日志：“我在手册里写了十五年脚注，今天第一次看到一个系统规定脚注不许被抄丢。”下面这段记法在说什么：它是那条最终入库的参数——当年脚注里的

每一样东西，现在都是必填字段。

```
# 接缝上的一个参数：值、条件、最坏值缺一不可（示意记法）
req:Param_SensorUpdate a req:InterfaceParam ;
    req:typicalValue "2 ms" ;
    req:validUnder    " 常温、低温补偿未激活" ;      # 典型值何时成立
    req:worstCase     "8 ms" ;                          # 最坏值（合成示例数字）
    req:worstUnder    " 低温补偿运行" ;                # 最坏值出现的条件
    req:failureTell   " 更新停滞时诊断位按时置位" ;   # 坏了怎么说
    req:committedBy   req:SupplierCommit_L07 .        # 谁对本项目承诺
```

最后一行同样是承诺换手的记录：这个 8 毫秒不是从手册里抄来的描述，而是供应商对这个项目的承诺，有出处、有署名。手册的脚注当年只说“最坏可达数倍”——“数倍”是写给所有客户的措辞，没法拿来设计窗口；8 毫秒是供应商第一次把“数倍”对这个项目钉成一个可以按计算器的数：四倍于典型值，签了名。软件的比对窗口从此对着最坏值那一格设计，而不是对着典型值。

一个不带条件的数字，在接缝上不是信息，是伏笔；载体的职责是让“典型值”三个字无处藏身——要么把条件和最坏值一并交出，要么这个数进不了门。

数字进了字段，新的问题跟着来了：8 毫秒也好，置位时限也好，读走窗口也好，这些毫秒最终都要从同一笔总账里扣。可这套系统的总账——那个容忍窗口——到今天还写着“待定”。待定的账，怎么记？

## 15.5 待定的窗口，也要有账本

从故障出现到危害可能形成的那个窗口，属于危害和整车情境，不属于任何机制——这条纪律是系统层用一个下午的评审换来的，试点原样继承。而这个窗口的数值，从概念阶段冻结到今天，一直是“待定”：车辆动力学分析还在进行，数字要靠分析和试验去挣，不能先抄一个好看的整数。

待定的窗口怎么进载体？第一个提议：先填一个保守的占位值，标注“临时”，免得账本空着难看。不行——这是全书埋葬过不止一次的做法：没有来源的数字一旦进了系统，就会被下游当真，“临时”两个字的下场和“典型”两个字不会有任何不同。第二个提议：那就等窗口定了值再建账。也不行——系统层的教训原文还在：等数字来了再立账本，各段时延早已各自为政。

新载体给出的第三条路，是把“待定”本身建成一个受控状态。下面这段记法在说什么：它是那笔预算的账本——窗口一格明确记为待定，待定有主人、有在办的活动；各段的最坏值凡是有承诺的，逐格入账；而余量一格，

记的既不是零，也不是某个数，而是“未知”。

# 一笔待定的预算（示意记法）

```
req:LedgerFaultToSafe a req:TimeBudget ;
  req:totalWindow req:Pending ;           # 容忍窗口：待定
  req:pendingOwner " 车辆动力学分析（进行中）" ; # 待定有主，不是没人管
  req:hasItem req:BudgetItem_Sense , # 传感段：最坏 8 ms（接缝承诺）
    req:BudgetItem_Detect ,           # 检出一段：最坏值已承诺
    req:BudgetItem_React ;           # 反应一段：最坏值已承诺
  req:margin req:Unknown .             # 余量：未知（非零也非够）
```

这本账立起来之后，机器多了一种从前没有的诚实。有人在评审里问“现在余量不够”，机器的回答是：各段已承诺的最坏值合计若干毫秒，窗口待定，余量未知，不予判定。它拒绝说“够”，也拒绝说“不够”——未知就是未知，这是从术语那一章一路继承下来的三值纪律。而它同时承诺了一件事：窗口定值的那一天，余量自动重算，凡是引用过这本账的结论逐条标示重查。上一节那条比对需求的时限，指的就是这本账里的一格——格子先画好，数字到位那天才有处可记，这句话现在是机器行为，不再是口号。

陈工对这一格的评价只有一句：“以前‘待定’写在表里，是等着被人忘掉的；现在它写在账上，是等着来讨债的。”讨债的日子不归机器定——分析何时收口、数字何时算挣到了手，仍然由做分析的人和有权接受它的人说了算；账本只保证：那一天来的时候，没有一个依赖它的格子能装作没听见。

待定不是空白，是一种受控状态：它有名字、有主人、有到期的义务，并且向下传染——凡是依赖它的结论，一律不得早熟。

五要素、理由、确认、条件、账本，都立住了。那么，现在可以回答陈工那个问题了：改一条上游，哪些东西要重开？

15.6  重开清单，这一次机器先说

机会来得比预想快。传感器换代的第二波通知到了：新款的设计资料包发放，老款停产进入倒计时——当年引发两天会议的那场变更，现在真的落地了。换代意味着一件确定的事：老款传感器的最坏刷新承诺，将随老款一起失效。

小蔡没有订会议室。她让助手对着载体提了一个问题。下面这段是问题、查询和答案的三段式——查询做的事说穿了很朴素：把所有理由里引用了那个传感器参数的连线，连同它们指向的下游，一次取出来。

问题：传感器换代，最坏刷新承诺失效——下游哪些对象的成立理由引用了它？

```
SELECT ?affected ?why
WHERE {
  ?edge req:rationale      ?r .
  ?r     req:dependsOnAspect req:Param_SensorUpdate . # 理由锚在这个参数上
  ?edge req:toReq          ?affected .
  ?r     req:justification  ?why .
}
```

答案（节选）：

| 受影响对象                               | 为什么有关              | 建议动作   |
|-------------------------------------|--------------------|--------|
| 比  对  需  求  （TSR_<br>TorqueCompare) | 比对窗口按老款最坏刷新设定      | 重开派生评审 |
| 预算条目(BudgetItem_<br>Sense)          | 传感段最坏值出自该承诺        | 重新取值入账 |
| 接缝合同相关条目                            | 置位与读走窗口以该参数为前<br>提 | 重签确认   |
| 软件侧的一次分配                            | 接收确认基于旧最坏值         | 重新确认   |
| 若干验证结论                              | 判据引用旧参数            | 重新验证   |

查询跑完不到一分钟。小蔡把结果和当年那份四十多项的清单并排放着看：当年两天会议凑出来的东西，很多行在这里，是被理由锚点逐条钓出来的；当年清单上没有、这里却有的，是那几条“悬空的分配”——旧世界根本没有地方记录“确认基于什么”，所以连丢都丢得无声无息。还有一行让小唐在屏幕前站了很久：比对需求赫然在列，理由一栏写着“比对窗口按老款最坏刷新设定”——当年做这个决定的人已经离职，两天会议里这条依赖是靠半张旧会议纪要碰运气碰出来的；这一次，它是被那条改正过的



连线自己交出来的。理由被留下的那一天，没人知道它会在哪一天救场；它只是等在那里。

系统层的结尾曾请读者做过一件练习：假装收到一封换型邮件，手工列出重开清单，给自己计时，然后问敢不敢担保没漏——并记住那种不踏实。此刻可以对照了：不踏实没有被勇气治好，它是被换掉了载体。

但小蔡没有让庆祝发生。她当着陈工的面把丑话说完：这份清单担保的范围有边界。有人提议把担保做大——让机器把“可能相关”的也全列上，宁滥勿缺；不行，无差别的“可能相关”会把清单撑回四十多项的规模，两天会议换个形式重开。有人提议再用人工圈一遍补漏；也不对症，那等于承认机器清单不可信，回到记忆与运气。真正的答案是把担保说精确：需求、派生、分配、接缝、预算这几类依赖，试点已经做过“记录完整”的显式声明，声明范围之内，机器担保一网打尽——凡记了理由的依赖，一条不漏；声明范围之外——比如试验资产与证据侧的牵连——机器不担保，清单末尾自动附注了这一句。清单本身也只是提议：每一行的“重开”要不要立项、何时立项，签字的是陈工，不是查询。

机器能担保的是账本之内的一网打尽；人的担保从此换了对象——不再担保“没有漏掉的下游”，而是担保“没有漏记的依赖”。这不是把责任推给机器，是把责任放到人担得动的地方。

散会前陈工问了最后一个问题：“当年我问‘哪些要重开’，没人答得动；今天机器答了。那清单最后一列呢——那些‘重新验证’，拿什么算验完？”

## 15.7 清单的最后一列

小蔡顺着清单往下看。“重开派生评审”，载体里有位置；“重新取值入账”，账本的格子等着；“重签确认”，确认活动有记录可留。可“重新验证”这一列，需求世界给不出答案——验证要拿数字作证：一个实测的更新周期、一个实测的置位时延、一条判据边上的实测裕量。而数字这种东西，本书前半部分用一整章的硬件教训证明过：脱离对象、方法、条件和来源的数字，不是证据。

需求本体能做的，是把判据和时限写成机器可查的承诺；承诺兑现与否，要靠带着出处的测量结果来回答——那是另一个世界的事，需要它自己的对象、自己的门禁、自己的诚实边界。经由显式的对齐合同，这份“重新验证”清单将交到那个世界门口。

导读的期票，现在兑现：你已经看过一条需求怎样因为缺件被拒收，一条连线怎样被逼着交出理由，一个“典型值”怎样在门口被拦下，一笔待定的预算怎样诚实地记账，以及一份重开清单怎样在一分钟内替代两天会议——连同它担保的边界。承诺不再丢失，靠的不是更认真的人，而是一个丢了就存不进去的载体。

下一章的问题，梁工在离开评审室时替所有人问了出来：“重新验证的时候，我给你的每一个数——你们拿什么记住它是在哪儿、怎么量出来的？”

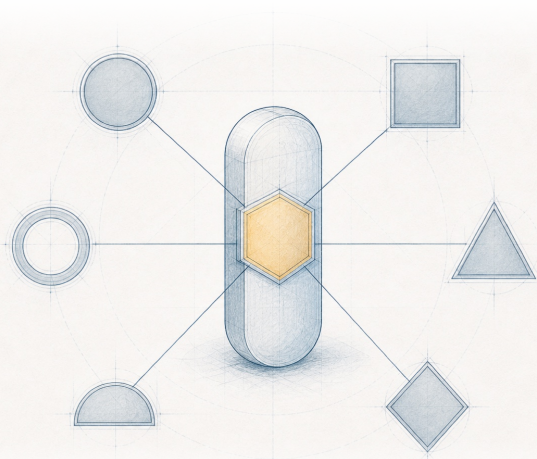


**本章注记：**本章的人物、会议、事故经过与全部数字（更新周期、最坏值、迁移批次、清单行数、用时等）均为合成教学材料，不对应任何真实企业、真实产品或真实个人；文中“8 毫秒”等数值仅为教学示例，不是任何实际系统的参数；容忍时间窗口在本章保持“待定”语义，不构成任何取值建议。正文对功能安全标准内容的转述为自然语言概括，精确表述以标准原文为准。各记法片段是有意简化的教学投影；唯一可执行版本是 `semantica.chapter_packages.vol2.ch15`，当前为 `partial`、`release blocked`。



## 第十六章 带着出处的数字： 测量与证据本体

Semantica 绑定：本章保存测量对象、方法、单位、不确定度与来源的概念化；唯一机器语义是 `semantica.chapter_packages.vol2.ch16`，主场景为 `semantica.vol2.ch16.scenario.primary`。本体、CQ、查询、Shape、正反案例、重算规则、oracle、manifest 与 receipt 均在 Semantica。当前为 **partial**、release **blocked**；重算一致不能证明输入数据真实、方法适用或结果足以放行。



**【导读】**需求那条线的试点收尾时，把话递了过来：验证一条需求要靠数字作证，而数字得带出处。这正是几年前硬件那场审核留下的老账——第 6 章末尾冻结了三个问题：怎样让一个数字无论被复制到哪里都被迫带着护照；怎样让“过线”这样的结论自动拴着自己的范围与假设；怎样让任何一个数都能反查血缘，一路追到第一次测量或第一次假设。本章陪试点团队把这三个问题接进机器：先看一次热心的批量导入怎样撞上“资产”的真正定义；再给测量值立六条边、给算出来的数字立计算链、给作证关系立射程；然后重演那场审核事故——这一次，那个复制来的数字没能过夜；最后看 AI 助手在边界内替人干活的样子。读完本章，你应当能判断一个

进了系统的数字算不算证据：它是否随身带着对象、方法、单位、条件、不确定度与来源，是否能反查到源头，缺任何一样时，机器是否会当场拒绝。

## 16.1 热心的批量导入

试点扩到测量这一段，是小蔡把工作桌搬进硬件组的那一周。牵头人还是她——从两年前那个什么都要问的新人，到现在带着一台机器和一沓能力问题上门。需求那条线刚收的尾给了她底气：承诺不再丢失，追溯结构把“哪条需求等哪个数字作证”摆得清清楚楚——现在轮到数字自己交代来历了。接待她的是吴工，桌上照例摊着那种每张都带“出处”列的表格。对这场试点，他不冷不热。别人的顾虑是“机器靠得住吗”，他的顾虑正相反：“规矩我立了二十年，靠盯。机器要是也得靠人盯着才守规矩，那就是多了一样要盯的东西。”这句话小蔡记下来了——它其实是本章的验收标准，只是当时还没人这么认领。

系统工程师小唐是来帮忙的，他带来一个让所有人眼睛一亮的提议：“硬件组十几年的失效分析表、度量表、试验记录，全在网络盘上。写个脚本一晚上全导进去，台账立刻就是满的——数字进了系统就是数据资产，越早进越好，空着才是浪费。”

这话一点不外行。把纸变成数据，本来就是资产化的第一步：能检索、能汇总、能复用，多少组织的数字化就是这么起步的，而且起步得对。小蔡也心动——一个空台账演示什么都难看。于是先导了一张旧度量表试水。

第二天的演示很难堪。查“那个百分之九十五点四八的分母里有什么”——答不出；查某颗器件失效率的适用条件——答不出；查任何一个数是谁、按什么方法、在什么条件下得到的——一律答不出。系统里一夜之间多了几千个数字，能回答的问题一个没多。小唐还想挽回一下：“至少能搜到了，总比躺在网络盘里强。”吴工走到屏幕前，指着其中一格：“搜到一个答不出来历的数，比搜不到更危险——搜不到你会去找人问，搜到了你就直接用了。”更糟的是小蔡自己的观察：屏幕上的数字比纸上的更像真的——纸会发黄，界面永远崭新，错误因此显得更权威。第6章讲过数字的五段旅行，每复制一次掉一样东西；批量导入不是旅行的终点，只是第六段旅程，而且是掉得最快的一段。

会上冒出两条补救，都值得认真试过。第一条：先导入，出处慢慢补。吴工只说了一句话就把它按住了：“当年我上级也是这么说的——先把格子填上，回头补算。回头没有来过。”补出处的债和补计算的债是同一种债，欠的方式都是“先”字。第二条：给导入模板加一列必填的“出处”文本框，空着就不许进。硬件组的表格早就有这一列——那是吴工立了很多年的规矩——可文本框吞下“见旧报告”“同上”照样放行：字符串不拒绝任何东西，必填的空格只保证格子里有字，不保证字后面有路。

**数字进了系统，不自动成为资产。资产的定义是能回答追问：带着可核查来历的数字才是资产，其余只是搬进机器里的纸——而且是复制起来更快的纸。**

那么，让数字“带着来历”，到底要带什么、以什么方式带，机器才有力气在录入的那一秒强制，而不是靠评审的人恰好想起来问？

## 16.2 一个数字的六条边

动手之前，小蔡把第 6 章末尾冻结的那三个问题翻成了三条能力问题，贴在白板上，当作这套本体的验收边界：任取台账里一个数字，机器能否列出它的对象、方法、单位、条件、不确定度、来源，缺任何一样能否当场拒绝？任取一个“过线”结论，机器能否报出它押着哪些范围与假设？任取一个数字，机器能否沿来源链反查，一路走到第一次测量或第一次显式假设，中途断链能否被发现？

第一条能力问题的答案，是把测量值从“格子里的字符”改成“一个对象”。白话说清楚：台账里的一个数不再是数值本身，而是一个实例，它必须用六条边把自己拴在世界上——对象（这是谁的数）、方法（怎么得来的）、单位（含时间基准——按运行小时还是日历小时，第 6 章算过，差一个数量级）、条件（在什么剖面、什么温度下成立）、不确定度（数值周围的雾有多宽）、来源（哪次活动或哪份资料产生了它）。六条边每一条都指向另一个对象，而不是一段文字——这是它和“出处”文本框的全部区别：边是机器能沿着走的路。

这六个字段有个出身。定字段那天，吴工从抽屉里翻出一张发黄的纸——当年那页度量预算的背面，竖着写的五个词：对象、方法、单位、不确定度、出处。小蔡在中间添了一个：条件。那是吴工每次评审都会口头追问、纸上却从来没写下的一问——手册数字假定器件在额定范围内工作，条件死了数字就死了，可“条件”从没在他的表格里拥有过自己的一列。吴工盯着那六个词看了一会儿，说：“就按这个建。第六个是你的。”

这段记法在说什么：下面是那颗著名电阻所属器件类的手册失效率，  
作为一个合格测量值对象的样子。

```
# R17 类厚膜电阻的手册失效率，作为测量值对象（示意记法）
meas:FailureRate_R17Class a meas:MeasurementValue ;
    meas:hasValue      "0.4" ;                # 数值本身
    meas:ofObject      meas:ResistorClass_ThickFilm ; # 对象：哪类器件
    meas:byMethod       meas:HandbookLookup ;    # 方法：手册查取
    meas:inUnit         meas:FIT_OperatingHour ; # 单位：FIT/运行小时
    meas:underCondition meas:RatedProfile_85C ;  # 条件：额定剖面
    meas:hasUncertainty meas:Factor3Band ;      # 不确定度：约三倍带宽
    meas:fromSource     meas:HandbookEdition4 . # 来源：手册第四版
```

注意两个细节。不确定度那条边不是装饰：三倍带宽进了对象，下游用它算绝对概率时才有资格谈“保守”——保守是对着一条宽度作的判断，不是一种口号。顺带把这个词本身说严：不确定度不是“这个数错了多少”——错了多少要是知道，改过来就完了；



它是量的人对自己交代的一句有把握的话：照这个方法、在这个条件下，真实的值多半落在这条带子里。它标注的不是错误，是已经承认的无知有多宽——正因为把无知写成了一条宽度，下游才有东西可算。单位那条边也不是字符串：“按运行小时的 FIT”是一个自带时间基准的单位对象，谁想把它和按日历小时统计的数相加，机器会先要求换算——米和英尺相加的老错误，从此在录入口就走不动。

有人问：那旧报告的扫描件行不行？把文件附上，算不算出处？不算。文件人能读、机器不能走——它是黑箱，不是边。出处必须指向一个机器能继续追问的对象：一次测量活动、一份登记过的资料、一条签认过的假设。

而缺了边的数字，机器的答复是这样的：

# 录入一个只有数值和单位的数——机器的答复（示意）

被拒对象：meas:PastedValue\_0517

已有字段：hasValue、inUnit

缺失字段：ofObject、byMethod、underCondition、

hasUncertainty、fromSource

结论：不构成测量值，不得进入台账。

注意机器没有说“这个数是错的”。它说的是“这不构成测量值”——拒收的不是数值，是没有形状的数值。被拒的数也没有被销毁，它只是进不了证据的位置：你可以留着它、研究它、替它补边，但在补齐之前，任何计算、任何作证都引用不到它。这正是第 11 章立过的那条原则在测量世界的样子：机器的“拒绝”比机器的“生成”更值钱——生成一万个数字很容易，在错误的数字试图站上证据位置的那一秒说“不”，才是新载体真正买到的东西。

**测量值不是格子里的字符，是一个六条边缺一即拒的对象。复制一个测量值，复制的是整个对象——护照从此长在数字身上，而不是长在吴工身上。**

手册里查来的、台架上测来的，六条边都好填。可硬件账上最重要的那几个数——两个百分数、一个概率——是算出来的。算出来的数，它的“来源”是一条计算。计算怎么进本体？

**读图任务：**不要只看分数线上方的分子方块，将注意力移到下方展开的名单，逐项查看它为什么比一个单独数字更能承担分母。

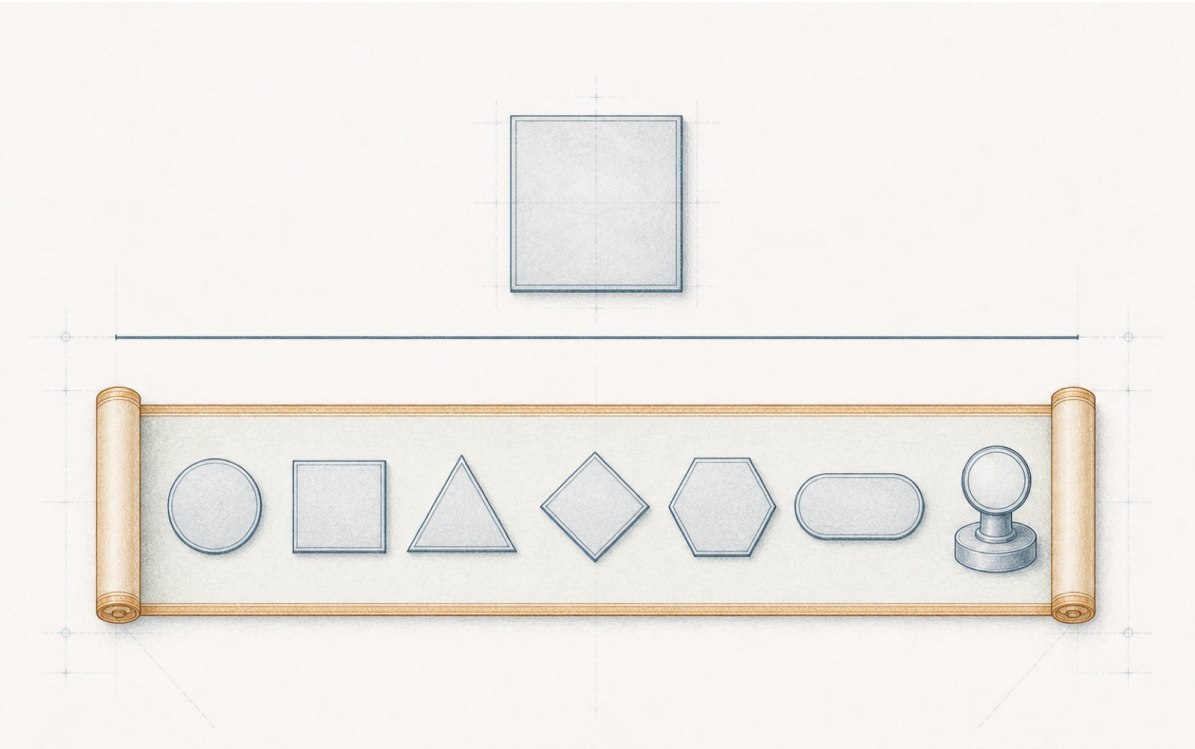


图 16-1  一个度量主张的分母首先是可审计对象集，然后才是一个数。名单让读者能检查每一项的纳入理由、排除理由、版本与适用范围。

**图后判断：**图中卷轴不是真实测量数据集，核对印章也不代表结论通过。它只说明可审计分母的结构，不给出任何指标数值。

### 16.3  算出来的数字，出处是一条链

小蔡先问了一个笨问题：“今天，那个百分之九十五点四八的出处是怎么记的？”答案是：度量表的备注列写着“由失效分析表计算”，失效分析表存在某个归档目录里，谁算的、用的哪一版、分母范围谁批的——问到第三层就得去翻邮件。这不是硬件组懒，这已经是纸面世界的良好实践了；只是“由某表计算”五个字和“见旧报告”是同一种字符串，机器在上面走不动。

自然的做法有两条，硬件组以前都用过。一条是把公式写在备注列：“等于一减去分子除以分母”——备注是字符串，查不动，也拦不住分母悄悄换内容。另一条是把当年算数的整张电子表格存档当出处——快照是黑箱：分母里灌没灌水、哪一行后来改过，快照都看不见；第 6 章说的“分母的范围松一寸，绿色就便宜一寸”，在快照里照样松，只是松得更隐蔽。

派生值因此要比测量值多带一样东西：计算链。谁除以谁、按什么公式、每个输入是哪一个测量值对象——尤其是分母：分母不再是一个数，而是一份清单，清单里一成员一条边。

这段记法在说什么：加装监控器之后的那个单点故障度量，  
作为派生值的样子——分子、分母、公式、输入，全部是边。

```
# 度量值：一个带计算链的派生数字（示意记法）
meas:SPFM_AfterU7 a meas:DerivedValue ;
    meas:hasValue          "95.48%" ;
    meas:byFormula         meas:OneMinusRatio ;           # 公式：1 - 分子/分母
    meas:numeratorFrom     meas:ResidualSum_18p2 ;        # 分子：单点 + 残余合计
    meas:denominatorFrom   meas:ScopeList_403 .           # 分母：范围清单

meas:ScopeList_403 a meas:ScopeList ;
    meas:hasMember         meas:Sensor , meas:Controller ,
                           meas:PowerStage , meas:PowerPathUnmonitored ,
                           meas:U7Monitor ;               # 成员：一件一条边
    meas:declaredCompleteBy meas:ScopeReview_HW .        # 已记全：范围评审签发
```

最后一条边值得单独说。清单挂着一句“已记全”的声明，由范围评审签发——这是第 11 章讲过的局部完整性：机器只有在被告知“这份清单记全了”之后，才有资格判断“少了一件”；没有这句声明，缺席只能算未知。有了它，两件事第一次变成可查的：往分母里灌无关硬件，会多出一条没有入账依据的成员边；把该算的件漏掉，会撞上“已记全”声明。

问题 → 查询 → 答案，走一遍：  
问题：那个百分之九十五点四八的分母里有哪些成员，各自凭什么入账？

```
选取 ? 成员 ? 入账依据
其中 {
    meas:SPFM_AfterU7 meas:denominatorFrom ? 清单 .
    ? 清单 meas:hasMember ? 成员 .
    ? 成员 meas:scopedBy ? 入账依据 .
}
```

| 成员       | 入账依据        |
|----------|-------------|
| 传感器      | 范围评审        |
| 控制器      | 范围评审        |
| 驱动功率级    | 范围评审        |
| 无监控供电电路径 | 范围评审        |
| U7 监控器   | 变更评审（加装后入账） |

答案表的最后一行是有故事的：当年吴工在白板前特意钉过一句啰嗦话——“原来的四百里没有 U7，加装之后是四百零三”。那句话当时靠他当面说，现在是一条边加一条依据，谁都能查，他不在场也一样。

在机器的账里，分母不是一个数，是一份带“已记全”声明的成员名单：灌一件水就多一条看得见的边，漏一件账就多一个查得出的缺——“范围松一寸”第一次有了形状。

数字有了身份，也有了血缘。可数字入账不是为了好看，是要去作证——它和“达标”“过线”这样的话之间，是什么关系？这关系又是谁建的？

## 16.4 作证要报射程

最省事的做法：报告里有“通过”字样，系统自动把报告挂到对应结论上。这条路第 1 章那间会议室早就替所有人趟过了——台架卡上每个字都是真的，它只是不在说那一轮候选；“通过”是字符，字符不知道自己在给谁作证。凭字样自动建立的支持关系，只是把当年的错配自动化了。退一步的做法：让录入的人顺手勾一个“相关结论”的框。勾选确实建立了边，但一条不带射程的边和没有边一样危险——样机的卡照样能被勾到目标配置头上，勾的人还觉得自己尽了责。

所以支持关系必须满足两条：显式建立（它是一个被创建的对象，不是被推断的巧合），并且随身带射程——给谁作证、在哪版对象与什么条件内有效、押着哪些还没兑现的假设。

这段记法在说什么：硬件度量报告支持“单点故障度量为百分之九十五点四八”这个陈述，这条支持关系自己长什么样。先把一件事说破：它支持的不是“达标”——账上这个数离百分之九十九的门限还差三个多百分点，“达标”这句话在这套账里立不起来；报告有资格作证的，只是这个取值

在声明的射程与假设内成立。

# 一条带射程的支持关系（示意记法）

```
meas:Support_042 a meas:SupportLink ;
```

```
    meas:evidence      meas:MetricsReport_HW ;      # 谁作证
    meas:statement     meas:Stmt_SPFM_95p48 ;      # 给谁作证
    meas:scopeObject   meas:Arch_H32_WithU7 ;      # 射程：哪版架构
    meas:scopeCondition meas:RatedProfile_85C ;      # 射程：什么条件
    meas:restsOn       meas:Assump_WindowTBD ,      # 押着：窗口值待定
                      meas:Assump_U7Independent .  # 押着：独立性成立
```

最要紧的是最后两条边。第 6 章末尾那些“以窗口定值为准”的批注、押在独立性上的信用，当年散落在正文页脚和吴工的记性里；现在它们是边。一条查询问“哪些支持关系押在窗口待定这条假设上”，答案回来一串：供电路径迁账的那笔覆盖信用、押

在这笔迁账上的两个百分数的取值、以及给百分数作证的这条支持关系本身——几笔信用押着同一条假设，以前要靠吴工在评审上想来说“这几个数都是以窗口定值为准的”，现在是一张表当场列出，他在不在场、记不记得，结果一个样。将来窗口定值的那一天，该重开哪些信用，不用开会回忆，查一下就有。这回答了冻结的第二个问题的一半：结论拴着假设，机器能列出受累的全体。另一半——假设一变，过期怎么自动发生——本章按下不表，那是下一章的事。至于“陈述”那一头的完整结构，属于试点最早建起的那套可信主张的世界，本章经由显式对齐合同与它协作，不在这里展开。

支持关系是建出来的边，不是从“通过”字样里长出来的字：谁作证、给谁作证、在什么射程内、押着哪些假设，写全了这条边才存在。机器宁可让一个陈述缺着证据，也不让它挂上一条说不清射程的证据。

门禁立到这里，一直沉默的吴工开了口：“拿它试个案子。我出题。”

## 16.5 那个数，这次没能过夜

第二天吴工带来一份复印件——好几年前旧平台那份硬件评估报告，上面那个漂亮的、绿色的概率值。全组都知道那个数的故事。他把复印件拍在桌上：“验收测试就用它。当年我怎么抄的，今天就怎么录。”

有人先替旧数据求情——是小唐：“要不给历史数据开个豁免通道？打个‘历史’标记先进来，不然台账半天是空的。”这个心疼很真实，但吴工摇头：“豁免通道，就是给下一个年轻人准备的粘贴键。”又有人试第二条路：把旧报告本身登记成来源，边不就接上了？小蔡真的试了——录入时来源那条边指向旧报告，机器沿着链往下走：旧报告的那个数自己有来源吗？追下去是更早项目的一份报告；再追，

算它的人已离职、过程未归档——链在第二步断了。机器的答复：

# 旧报告里的概率值，原样录入——机器的答复（示意）

被拒对象：meas:PMHF\_LegacyCopy

来源链检查：fromSource → 旧评估报告 → 更早的报告 → （断）

断点说明：链上任何一环都不是测量活动，也不是签认过的假设；

沿 fromSource 无法到达源头。

结论：来源链不闭合，不构成证据；可暂存工作区，

当日内未接上源头即清退——不许过夜。

最后那半句，是吴工亲手加进规则里的。小蔡起草门禁那天，他把自己念叨了很多年的那句话背给她听：“没有出处的数字，不许过夜。”小蔡把它写成了机器的条款：六条边不全、来源链不闭合的数，可以在工作区暂存一个白天——给人补证据的时间——但台账不收，当天接不上源头就清退。从个人怪癖到机器纪律，中间只隔一次显式化——把“我总觉得该这样”里的每一个词，逼问成机器能核对的判据：“出处”是什么？一



条可走的边。“没有”怎么判？沿链走不到源头。“过夜”到哪一秒为止？工作区的宽限期条款。吴工立了二十年的规矩，第一次不需要吴工在场。

会议室很安静。吴工看着屏幕上的红字看了很久，说：“当年拦我的这一下，晚到了好几年。”当年没有人作恶——上级一句“量级差不多，回头补算”，他按下粘贴键，表格没有拒绝，文档没有报警，评审没有问到那一格。一个二十多岁的年轻人在那一秒需要的不是更好的品德，是一只在那一秒伸出来的手。现在，拦截不再取决于谁恰好在场、谁恰好想起来问：他休假的那两周，门禁不休假。散会时小唐在门口站住，半是对小蔡半是对自己说：“我收回批量导入那句话。进系统不算数，进得了这道门才算数。”

这套安排里有一个必须说破的分工。本体只管语义：什么样的记录才算一个测量值、一条支持关系——它自己一个事实也不制造。数值到底是多少，由受控活动的记录承载：哪一天、谁、用哪台设备、按什么规程量出来的。而当来源是人的判断时——老工程师的估计要想成为可用的源头，必须由有权角色把它签认成一条显式假设，签的是人，担的也是人。机器可以拒收来历不明的数，却不能把任何一个数变成真的。吴工当年那个数，最冤枉也最有教育意义的一点是：数值本身没错，后来重算出来量级确实相当——它输就输在被追问的那一刻，语义、事实、签认，三样一样都拿不出来。

反过来的正例也当场演了一遍。迁好账之后，从那个百分之九十五点四八出发沿来源链反查：派生值走到范围清单和各路输入，输入走到手册值和台架测量值，手册值走到第四版手册的一次登记在案的查取，台架值走到某年某月的一次测量活动。任何一个数，追到头只有两种终点：一次测量，或一条签认过的假设——再无第三种。冻结的第三个问题，答案就是这条走得通的链。

台账立住了。可六条边一条条录，比填表慢得多——硬件几百行的账，靠人搬要搬到明年。这时候所有人都想到了同一件事：AI 不是最会读手册吗？

## 16.6 提取是提议，入账要点头

试点的 AI 助手领到的任务是有边界的：通读新到的器件手册，把里面的失效率提取成六条边齐全的候选对象。一晚上，它提了几十条，每条都填好了对象、方法、单位、条件、不确定度、来源，并且每条都带着一个洗不掉的标记：提议。

第二天的流程是四步。提议：助手交出候选，附上它是从手册哪一段读来的，供人核对。校验：门禁自动查六条边全不全、单位的时间基准对不对得上、来源可不可达——有两条候选在这里就被打回，助手把不同小节的两种条件抄串了。确认：硬件工程师逐条过——手册数字的前提是器件在额定范围内工作，这个前提在我们的设计里守不守得住？吴工在这一步又打回两条：“这颗电容我们用在超温的位置，手册条件不适用。”这一步机器替不了：适用性是领域判断，不是抽取任务。执行与审计：确认过的候选由受控活动落账，方法一栏写的是“文档提取，经人工确认”，连“助手提的、谁确认的”也留在账上；被打回的候选不删除，记为“不适用，附理由”——拒绝也是知识，下次换个设计还用得上。

提取是提议，入账要点头。助手快在读，人硬在判——一晚上几十条候选换来两小时的人工确认，这买卖划算，但划算的前提恰恰是那两小时不省。有人问吴工，确认这一步将来能不能也交给助手。他的回答很短：“它读的是手册，不是我们的板子。手册不知道那颗电容装在功率级旁边。”适用性判断需要的不是更强的阅读能力，而是对这块板子的责任——责任签不出去，所以点头交不出去。这也是三源分立在助手身上的样子：它可以替人跑语义校验、替人整理事实候选，唯独决定那一源，从头到尾没有它的位置。

这套问法出得了硬件组的门。环境监测单元 ENV-01 的标定证书上写着正负百分之零点五——在它自己的世界里，这也是一个测量值对象要回答的六问：对象是哪个量程，方法是哪份标定规程，条件是什么温度、距上次标定多久，不确定度按什么方式给，来源是哪一次标定活动。问法一个字不用改；答案一条也搬不过去——每条边都得在那个世界里重新挣。

演示散场时掌声不少。只有小蔡站在屏幕前没动——她盯着的是角落里一条不起眼的记录：软件版本升级的变更申请，在途。

## 16.7 证据都新鲜，可产品在变

先对着白板上那三条能力问题交账。数字带护照——答了：六条边缺一即拒，复制走到哪里护照跟到哪里。结论拴假设——答了一半：押着的范围与假设是显式的边，一条查询列得出受累的全体；但“假设变了之后过期自动发生”，还没有着落。血缘反查——答了：来源链走到测量或签认过的假设为止，断链当场可见。

诚实边界也要再钉一遍，免得这一章读成机器万能。六条边齐全的数也可能是错的——仪器测歪了、规程用错了、专家估走眼了，本体拦得住来历不明，拦不住世界本身出错；账平不等于账对，这句话在纸上成立，在机器里同样成立。所以台账收口处签字的仍然是人：机器交出的是可核对的来历与射程，接受还是拒绝，是有权者的事。陈工听完汇报只补了一句话，算是给这一段试点定了调：“以前我签字前问‘这个数哪来的’，得给你们三天去查；现在得给我三分钟。省下的不是三天，是三天里含糊过去的那些次。”

散会后吴工留在最后。他把桌上那摞带“出处”列的旧表格理齐了收进柜子，像是完成一个仪式。“这列格子，”他说，“现在不归我管了。”说这话的时候，他看起来轻了一些。一个人把自己的伤疤看守了很多年，如今终于可以交给一个不会休假、不会离职、不会心软的看守者——而他自己留下的，是当年站起来说“这个数是抄来的，我抄的”那份东西。那份东西机器学不会，也不需要学会：它负责让下一个年轻人永远用不上。

可小蔡盯着的那条在途变更，才是真正的收尾。软件要从一版升到下一版；系统层那只“第一只钟”的窗口值即将定值；器件手册的第五版已经在印，第四版这个来源要不要退役？每一件都会砸中某些支持关系押着的假设。今天的机器能回答“谁押在这条假设上”——这已经比吴工的记性强了——可它还没有一个词能表示“这条假设变了”：变

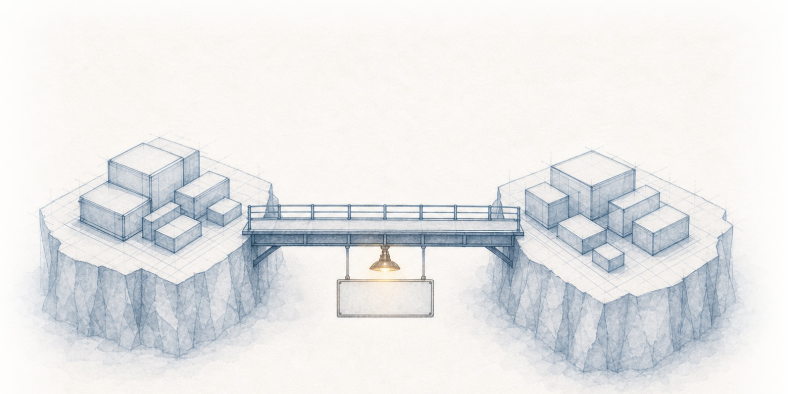
更在系统里只是一句在途的话，没有形状，没有边，没有传播的路。证据都新鲜，来源都闭合，射程都写明，可产品在变——变化本身怎么表示？改一处，哪些边跟着动、哪些“通过”该自动过期而不是等人宣布？第 17 章从这里开始。

**本章注记：**本章人物、对话、审核事件与全部数字（失效率、度量值、百分数、概率、位号等）均为合成教学材料，不对应任何真实企业、真实产品或真实个人。正文对标准中硬件度量与随机失效评估内容的提及为自然语言概括，精确定义以标准原文为准。文中记法只是教学投影；唯一可执行版本是 `semantica.chapter_packages.vol2.ch16`，当前为 `partial`、`release blocked`。



## 第十七章 变化有了形状： 版本与变化本体

Semantica 绑定：本章保存配置、变化、证据保质期与逐项裁决方法；唯一机器语义是 `semantica.chapter_packages.vol2.ch17`，主场景为 `semantica.vol2.ch17.scenario.primary`。本体、CQ、查询、Shape、变化/裁决正反案例、规则、oracle、snapshot/diff、PROV 与 receipt 全部在 Semantica。当前为 `partial`、`release blocked`；未登记变化不得被系统静默解释为“无影响”。



**【导读】** 上一章结束时，数字终于都带上了出处：每个值背后站着对象、方法与来源，证据从来没有这么新鲜过。可散会前小林只补了一句话，就让这份新鲜显得脆弱——“产品在变。下个月，这些证据还新鲜吗？”第 7 章章末冻结的三个问题由此回到桌上：怎样让每张“通过”可查地绑定自己的构建、环境与判据，而不是靠人回忆；一次变更之后，怎样自动列出哪些结论过期、哪些仍在保质期、理由是什么；时间流逝时，怎样发现悄悄的漂移。本章跟着小蔡给“变化”建一个最小的形状——快照、维度、事件、差异分析——然后把当年的两场事故在新载体上重演一遍。读完本章，你应当能把一次变更交给一条查询，让机器回答：哪些“通过”因此过期、哪些仍在期内——以及更重要的——每一个答案凭什么。



## 17.1 三个问题，等一个形状

测量那一摊收尾的时候，试点的士气正高：数字进账要带出处的规矩立住了，吴工那边连最不服气的人都承认，“来历不明的数字进不了账”这道门挡下的麻烦比它添的麻烦多。可庆功的话音没落，小林就把下一块石头放在了桌上——证据是新鲜的，产品却在动。变更单在途，工具链在换代，台架在维护升级；一份今天带足出处的证据，下个月支持的还是不是今天这句话？

试点推进到软件域那天，小林抱来一只文件盒，最上面是那张著名的手工差异分析表：每行一件回归资产，列上写着它绑定的构建、环境、夹具版本、结论和失效条件。这张表在评审会上被接受过，今天也仍然是对的——靠他的记忆、他付过的学费和他的认真。小蔡看完只问了一句：“您休假那两周呢？”小林想了想，给了一个诚实的回答：“那两周，大家尽量不升版。”没有人笑。一套靠某个人在场才成立的做法，再对也只是这个人的做法。

小蔡把第 7 章冻结的三个问题翻译成三句机器要接的话，写在白板上：给一张“通过”，你能回答它绑定的构建、环境与判据吗；给一次变更，你能列出重开集合并逐项给出理由吗；给一段时间，你能指出哪些绑定物已经悄悄不在了吗。按团队已经用惯的规矩，这三句话就是本章这个本体的验收边界：它承诺回答这三问，也只回答这三问——变更该不该批、重测值不值得，都不在它的射程里。

三句话共享同一个前提：机器得先看得见“变了什么”。而此刻“变”在团队手里只有两种存在方式。一种是代码仓库的逐行差异——太细了，机器看得见每一行，却不知道哪些行属于软件、哪些牵动标定的解释，更糟的是它根本照不到编译器；当年那十一天，逐行差异是零，事故照样发生。另一种是变更单——一份写给人读的文档，“其余声明不变”落在纸上就到了头，第 7 章已经算过这笔账：声明不变是变更单的义务，验证不变是另一回事，纸面上这两件事之间没有桥。

两条路缺的是同一样东西：变化本身还没有形状。变化，应该长什么样？

## 17.2 变化的形状：快照与事件

先立快照。第 1 章那张配置清单——硬件、软件、标定、驱动、基线——已经是快照的雏形，但在纸面上它只是主张里的一个从句，谁引用谁抄写，抄错了没有任何东西会响。也别指望发布包目录能顶替它：文件夹装得下软件和标定，装不下“当时用哪套工具链把它们变成机器行为”。现在把清单立成对象：一个配置快照是一个东西，它有维度，每个维度有值，它可以被引用、被比对、被绑定。小蔡抄完五行，小林伸手添了第六行：环境。“清单上从来没有它。我为它付过十一天。”

下面这段记法在说什么：把候选 RC17 写成一个六维的快照对象，逐维给值——

注意第六维和前五维平起平坐。

# 候选 RC17 的配置快照（示意记法）

```
chg:Snapshot_RC17 a chg:ConfigSnapshot ;
  chg:hardware      chg:HW_H32 ;    # 硬件维：H3.2 控制器
  chg:software      chg:SW_183 ;    # 软件维：SW1.8.3
  chg:calibration   chg:Cal_C41 ;    # 标定维：C41
  chg:driverPack    chg:Drv_D7 ;    # 驱动维：D7
  chg:baseline      chg:Base_V12 ;  # 基线维：V12 集成基线
  chg:buildEnv      chg:Env_E7 .    # 环境维：编译器、选项、库、夹具的组合
```

环境第一次成了配置的正式一维，而不是散落在某人记忆里的背景。快照立成对象之后，“两个快照相不相同”从眼力活变成了逐维比对——这件小事后面要派大用场。

再立事件。变化不是“新清单换掉旧清单”的模糊一瞬，而是两个快照之间的一条边。下面这段记法在说什么：把在途的那次软件升版写成一个变更事件，

边上写明改了哪一维、从什么值到什么值、声明哪些维不动。

# 一次变更事件：软件维从 1.8.3 升到 1.8.4（示意记法）

```
chg:Change_0091 a chg:ChangeEvent ;
  chg:fromSnapshot   chg:Snapshot_RC17 ;
  chg:toSnapshot     chg:Snapshot_RC17a ;
  chg:changedDimension chg:software ;    # 改了哪一维
  chg:changedFrom    chg:SW_183 ;
  chg:changedTo      chg:SW_184 ;
  chg:declaresUnchanged chg:hardware , chg:calibration ,
                        chg:driverPack , chg:baseline , chg:buildEnv .
```

最后那五条“声明不变”不是客套。机器会拿它们与两端快照逐维对表：声明标定不变、而新快照的标定维实际换了值的事件，登记不进去。变更单上“其余不变”四个字，第一次从礼貌变成了可核对的承诺。还有一个细节值得指认：两端快照有五个维度取值相同、只有软件维不同——第 1 章说的“单因变式”，在这里不再是一句描述，而是两个对象之间可以逐维数出来的事实。

**变化有了形状：不是两团配置之间的模糊差异，而是两个快照之间一条带名字的边——改了哪一维、声明哪些没动，都写在边上，都可以被对表。**

快照与事件立起来了。可屋里最多的东西既不是快照也不是事件，是历年攒下的那一摞“通过”。它们站在哪里？

## 17.3 每张“通过”，交代自己的世界

最省事的办法是给报告模板加三栏：构建、环境、判据。但加栏靠人填，漏填的报告照样归档，三年后没人分得清空栏意味着“没有”还是“没写”。再省事一点是靠命名规范和目录结构——文件名装不下六个维度，而且改名无声。两条路都把“绑定”寄存在自觉上，而本章要防的恰恰是自觉的假期。

新载体的做法只有一条：把每张“通过”登记为对象，绑定即部件，缺件即拒——第11章立下的那条“机器的拒绝比机器的生成值钱”，在这里长出本章的版本。下面这段记法在说什么：两张“通过”各自写下自己生效的世界——一张是软件

回归，一张是生产写入。

```
# 两张“通过”各自绑定的世界（示意记法）
chg:Pass_SwRegression a chg:PassRecord ;
    chg:onSoftware      chg:SW_183 ;    # 生效构建
    chg:onCalibration   chg:Cal_C41 ;    # 生效标定
    chg:inEnv           chg:Env_E7 ;     # 生效环境
    chg:byCriterion     chg:Crit_Regression .    # 按什么判据

chg:Pass_MfgWrite a chg:PassRecord ;
    chg:onSoftware      chg:SW_183 ;    # 镜像源于构建——所以老实登记
    chg:inEnv           chg:Env_Line2 ;
    chg:byCriterion     chg:Crit_ImageVerify .
```

先说这些值从哪里来。登记是一次受控活动：生效构建从流水线的构建记录里抄，生效环境从工具链清单里抄，谁登记、何时登记都留痕——不允许凭记忆填写。本体规定“必须有这些部件”，部件的值却必须来自事实的载体；这个分工本章后面还会回来。

再留意生产写入那一条：它老老实实写下了自己绑定软件维，因为镜像源于构建。当年的会议室里，它是“最容易被忘掉的重开”；此刻它只是登记表上普通的一行。这行登记，就是后面一切“不会忘”的全部秘密——先按下不表。判据那一格也不是摆设：判据本身是对象，它也会改版——判据收紧了一档，旧“通过”按的还是旧尺子。把判据绑进来，这种更安静的变化才有被问到的一天。

登记到台架那张卡时，屏幕把当年会议室的发现变成了一个可查询的事实：它绑定的样机、构建、标定三个值，没有一个与目标快照相同。没有人指责这张卡——它只是永远站在自己的世界里，支持着另一句不在今天桌上的主张。

登记旧档案时也遇到了登记不进去的。团队想起那台 ENV-01：三年前的固件“通过”还在档案柜里，可构建它的笔记本早已报废，环境维补不出来。缺件即拒在这里显出它的诚实——补不齐世界的“通过”，不再冒充证据，降级为历史记录。

一张“通过”在这个本体里的登记费，是交代自己的全部世界：构建、环境、判据，缺一即拒；交不出世界的“通过”，连过期的资格都没有。

绑定齐了。走廊尽头的邮件恰好在这时到达——还是那张单子：软件从 1.8.3 升 1.8.4，只改一件事。这一次，重开集合谁来圈？

**读图任务：**先找到两组快照蜂窝中的配置差异，再沿下方三张 PASS 卡的锚线回到它们各自属于的快照，最后看哪条锚线已断开。

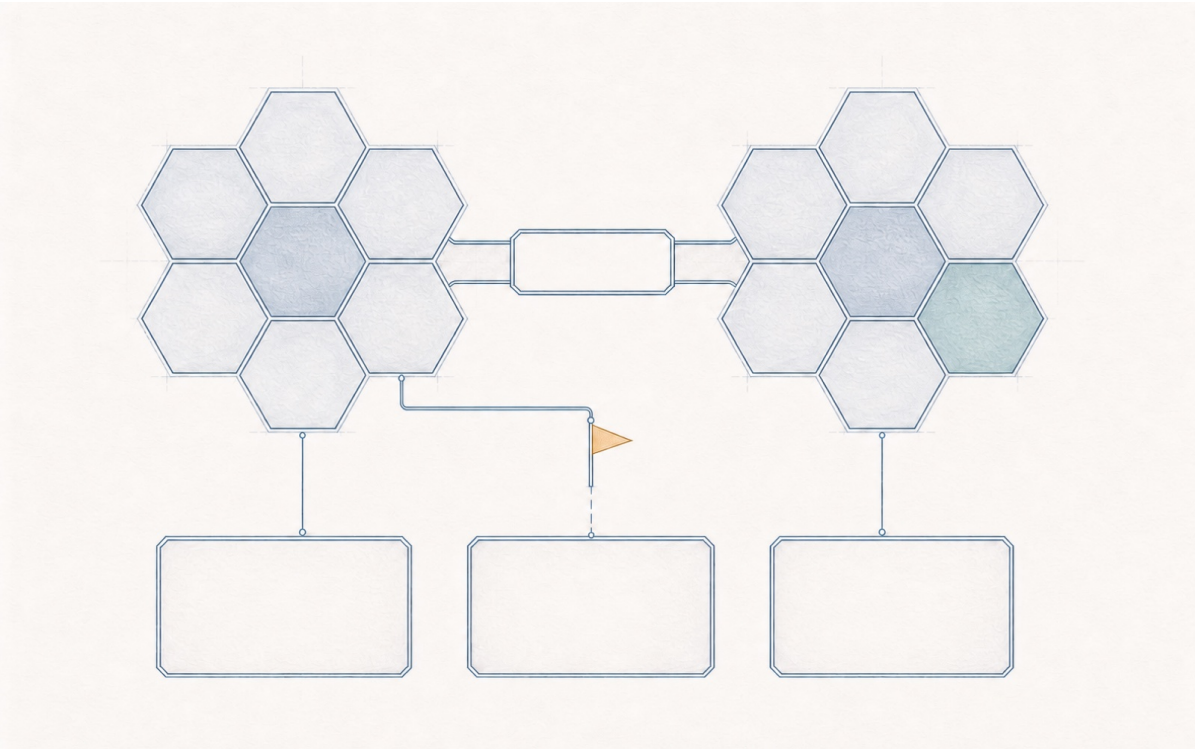


图 17-1 PASS 是带对象、配置、条件和时间的快照结论。快照改变后，旧卡片不必立即变假，但它与当前世界的支持关系必须重新判定。

**图后判断：**过期旗只表示需要重开评价，不自动否定原证据，也不表示变更一定导致安全问题。重新通过仍需人员依据差异和影响做决定。

### 17.4 重开集合，由一条查询返回

这件事团队其实做过两遍。第 1 章的会议室里逐卡辩论了一个下午，得出五张卡的逐项判决；第 7 章小林的六层清单做得更细，代价是他的记忆和学费。两版手工清单都对，也都不可复制——换个人、换个星期，就不保证还有这张清单。至于第三条路，照改动模块机械地圈，它的盲区正是那十一天的出处，早已出局。

现在轮到第四条路。问题：软件维的旧值被换掉之后，哪些“通过”绑定着它？

```
SELECT ?pass WHERE {
  chg:Change_0091 chg:changedDimension ?dim ;
                  chg:changedFrom      ?oldValue .
  ?pass a chg:PassRecord ;
        ?binding ?oldValue .           # 这张“通过”绑定了被换掉的旧值
  ?binding chg:bindsDimension ?dim .
}
```

答案表在一分钟内回来：

| 查询返回  | 当年的手工对应             |
|-------|---------------------|
| 软件回归卡 | 第 1 章判词：“直接受击”      |
| 集成冒烟卡 | 第 7 章判词：“新构建整体重做”   |
| 生产写入卡 | 第 1 章判词：“最容易被忘掉的重开” |

生产写入卡回来了，而且没有惊动任何人的记性。机器并不知道镜像是什么、不知道产线在哪——它只是把登记那天写下的那条绑定原样收了回来。机器不会忘，不是因为它聪明，是因为它从来不需要记。答案表的第二列同样要紧：每一行都自带理由——“因为你绑定了被换掉的那个值”。当年两版手工清单给过同样的判词，但判词长在小林的脑子里；现在它长在返回结果上，谁来问，答案都一样。

小蔡把返回结果与两版手工清单并排贴在白板上对了一遍账：第 1 章那个下午辩出来的逐卡判决，第 7 章那张六层表格，凡是冲着绑定去的行，机器全数命中，一张不多、一张不少——用时一分钟，而那个下午开了三个小时。

台架卡不在返回里，这同样值得看一眼：它绑定的从来不是目标快照的任何旧值，在新载体里它对新主张本来就没有取得过支持关系——“顺手收编”这一回连门都找不到。第 1 章它一连教过两课——“对不上表要说明”“接近了也要说明”，第 7 章升版在即时它把第二课重讲了一遍，这一章它上完了最后一课：说明不再靠人守着，错位本身就写在绑定里。而集合之外的概念卡与冲振卡，此刻的身份是“保留候选”，不是“自动保留”——这个区别马上就要紧起来。

还有一笔账要如实记下：当年会议室圈过硬件卡——它的绿建立在诊断假设上，软件差异若触及诊断路径，假设就要重新确认。这条查询没有返回它。先把这笔账记在白板角上，本章结尾要用。

**重开集合不是谁的记性好，而是绑定关系的必然推论：查询只是把登记那天写下的承诺，原样收回来。**

集合摆上桌，处理它的两种最痛快的办法，立刻有人说出口。



## 17.5 两种痛快，被同一道门拦住

郑工先开口：“既然机器一分钟就能圈出来，那就干脆点——集合内外一起，全部作废、全部重测，机器排期，一次洗干净。”这话一点不外行。这是被那年冬天教育过的人的本能：宁可多测，不可漏测，很多行业的血就是这样止住的；而且它这回还多了一层底气——重测的账单可以让机器去排，勤奋第一次显得不贵。对面的痛快也有人记得——“软件而已，旧证据接着用”，那句话的上一个版本追回过十几辆样车；这回它换了件衣服，在会议室里飘了一句：“查询没圈到的，照单全收就行了吧。”两句话在第1章的会议室里交过一次手，靠的是人的辩论；这一次，它们要过的是一道结构的门。

新载体给这两种痛快准备的不是反驳，是一个对象。每一份差异分析是一个东西：一端连着旧证据——那摞“通过”，一端连着关于新快照的新主张，中间是逐项判决。判决只有两种写法，各带一个必填件：保留，须带理由；作废，须带波及面——这次作废剥夺了哪些结论的支持，一项项点名。判决写完，差异分析对象本身也进档案：它不是会议纪要的附件，而是下一次变更的输入——三个月后再升一版，今天的每条保留理由都会被重新问一遍还成不成立。

先试郑工的“全部作废”。助手照吩咐批量生成判决，跑到冲振卡那行卡住了：作废判决的波及面是空的——冲振卡绑定的世界没有一维被这次变更触碰，它的作废剥夺不了任何在场结论的支持。宣布一张卡作废，却说不出作废了谁，这不是判断，是挥手——门禁拒收。再试“照单全收”：概念卡那行的保留判决缺理由，同一道门，同样拒收。下面这段记法在说什么：机器怎样拒绝一条没有理由的

“保留”。

```
# 门禁：没有理由的“保留”被拒绝（示意记法）
[ a chg:GateReport ;
  chg:violates chg:RetainNeedsReason ; # 保留须带理由
  chg:focus    chg:Verdict_ConceptCard ; # 概念卡的保留判决
  chg:missing  chg:retainReason ; # 缺的部件
  chg:message  " 保留是判断，不是默认：请写明该卡为何不受本次变更波及" ] .
```

那行提示语值得慢读：保留是判断，不是默认。概念卡最终当然保留了——理由是“软件版本不改变相关项边界，安全场景选定经复核未涉软件行为”，这句话写进判决，成为下次变更时可以被复查、也可以被推翻的东西。逐项判完，最终的差异分析长这样：查询返回的三张，判重开，各自附着要重跑的范围；概念卡与冲振卡，判保留，各带一条写下来的理由；台架卡维持原状——它本来就不曾支持这句主张，无所谓保留或作废。既没有一张卡被顺手处理，也没有一张卡被勤奋错杀。

本体拒绝的不是哪个答案，而是“不逐项”这件事本身：保留是判断，须带理由；作废是判断，须带波及面——两种痛快在这道门前，一样过不去。

郑工盯着那行拒绝理由看了一会儿，说了三个字：“比我狠。”

“狠”字落在哪儿，值得多想一步。整体判决的诱惑，从来不在省下的工时，在省下的署名。“全部作废”把判断换成预算——预算是公家的，错杀一张真证据，不记在任何人名下；“照单全收”把判断换成默认——默认出了事，账摊在“当时没人反对”上，薄得无人可问。逐项判决贵，贵在每一行后面都得站一个写理由的人：理由进档案，下次变更还要被重新问一遍成不成立。两种痛快省掉的是同一样东西——把名字签在具体判断上的成本；这道门收回来的，也正是这笔账。

这一轮变更被接住了——因为它有变更单。可要是没人递单子呢？

## 17.6 会自己动的那一维

试点第七周，构建服务器发来例行通知：工具链将于周末升级。当年，这样一封邮件不会惊动任何人——代码一行没改，编译器又不是产品。小林那十一天的排查里，所有人都在翻改过的代码，答案却在没人怀疑的地方。纸面世界之所以查不到，有两层原因：其一，在纸面的变更管理里，编译器升级不算“变更”——变更单管产品定义，工具链在所有清单之外，没有一张单据为它而设；其二，旧报告不带环境，“哪些通过依赖旧编译器”这个问题连落笔的地方都没有。

补救的直觉有两个。给作业指导书加一条“工具链升级须评审”？制度靠人记得触发，而这类升级常由维护脚本在深夜完成。那就冻结工具链、永不升级？“冻结全世界”的方案第 1 章已经出局过一次，编译器也不例外——安全补丁和停服日期，不跟项目排期商量。

新载体的回答已经在快照里放好了：环境是第六维，升级就是一次变更事件。

下面这段记法在说什么：把工具链升级登记成一次环境维的变更。

```
# 环境也是一维：工具链升级作为变更事件（示意记法）
chg:Env_E8 a chg:BuildEnv ;
    chg:supersedes chg:Env_E7 .          # 升级后的工具链组合
chg:Change_0104 a chg:ChangeEvent ;
    chg:changedDimension chg:buildEnv ; # 改的是环境维
    chg:changedFrom      chg:Env_E7 ;
    chg:changedTo        chg:Env_E8 .
```

事件登记的那一刻，17.4 那条重开查询换个维度参数照跑，返回绑定旧环境的每一张“通过”——四百多项用例的软件回归卡赫然在列，虽然代码一行没改。小林看着名单沉默了几秒：“当年我要这张名单，用了十一天。”注意状态的措辞：不是“作废”，是“待重判”。标疑是查询的义务，判决仍是差异分析的义务——逐项纪律原样适用。时序敏感的用例大概率要重跑：新工具把同一份源码翻成另一副机器码，优化的手法变了，指令的快慢、代码在内存里的位置、任务之间的相对节拍都可能跟着挪——当年那个“迟到几拍”，走的就是这条路；纯逻辑比对的用例也许凭理由保留：它们判的是输入输出，

不押节拍。但每一行都得有人写下判词。机器给的从来不是结论，是一份不会漏名字的点名册。

漂移也是同一件事的慢版本，这正是白板上第三句能力问题的着落。夹具改版、台架软件更新，从不发通知——但它们在环境对象里都有名字，任何一个换代登记进来，绑定旧值的“通过”就会在下一次例行核对里自己浮出水面；反过来，一张“通过”绑定的环境如果已经不是现行世界里的任何一个成员，它就站在一个不复存在的世界上，核对会把它标出来——这正是当年 ENV-01 档案柜里那份报告的处境，只是这一次不用等到出事才发现。发现漂移不再靠小林休假归来的不安，而是把“现行世界”与“各张通过绑定的世界”定期对表。十一天与一分钟之间，隔着的是算力，是一条登记纪律。

**在纸面世界，环境升级不算变更，因为没有一张单据为它而设；在这个本体里，环境是快照的一维——它一动，绑定它的每一张“通过”当场知道。**

登记、查询、起草判决，这些活助手都干得又快又好。那么人还剩下什么？

## 17.7 助手圈得出集合，签不了字

差异分析定稿那天，值得把完整的一圈记下来。提议：助手依查询结果起草逐项判决，把每张卡的绑定、被触碰的维度、建议判决和理由草稿排成一列，末尾还附了一条自作聪明的建议——台架卡如今装着 1.8.4 的前身，“与新快照高度接近，建议收编为支持证据”。校验：门禁拒绝，“接近”不是任何一个绑定谓词，台架卡的三个维度值与新快照逐维对表，零匹配；理由写得再流畅，也改不动对表的结果——当年要靠小林的判词才能挡住的诱惑，如今在结构上就进不了门。小唐在旁边看得心痒：“草稿都这么齐了，让它顺手把判决也定了呗，反正门禁把着关。”小林摇头——门禁把的是“判决不缺件”，从来没把过“判决是对的”；这两件事之间的距离，恰好是人的位置。执行：被人改定的判决作为受控活动写入记录，签发人是小林，不是助手。审计：助手的原始提议连同那次被拒一起留在档案里——下次再有人起“顺手收编”的念头，会先撞见这条前科。

这一圈里，三种权威各归各位。软件维从什么值变到什么值，是构建系统与变更记录给出的事实，助手复述它们，不制造它们；什么叫过期、保留缺什么部件，是本体给出的语义；而“这几张卡本周重测、预算从哪里出、重测前产线暂停几天”，是小林提、陈工批的决定——门禁报告上，没有留给机器的签名栏。

差异分析本身也不能直接改写行业记忆。一次任务结束时，系统先分别报告工程处置、Semantica 的 execution/regression/receipt/release，以及学习判定。只有多个受控实践反复暴露同一缺口，且现有 package 的 CQ、Shape 或规则确实辨认不出它，才形成完整 candidate。候选要绑定本次快照与证据哈希，并带着旧 CQ、单因反例、歧义例和前版例进入治理外环；未登记的变化不得被解释成“无影响”，没有新知识也不得靠空 commit 制造版本活动。

也要诚实地数一数本体没有做到的事。它不知道 1.8.4 里改了哪两个模块——那是变更记录的事实；它读不出一条保留理由的好坏——“冲振卡与软件维无涉”是否成立，

靠小林和评审人的工程判断，本体只保证这句话必须存在、必须落在指定的部件里；它更保证不了登记是全的——它拦得住缺件的登记，拦不住没有发生的登记。变化的形状让机器接住了两场旧事故，但形状不制造事实，也不拥有决定。

## 17.8 查得动的绑定，查不动的缝

散会后，小蔡走回白板，看着角上 17.4 记下的那笔账：这条查询没有返回硬件卡。当年会议室凭什么圈到它？凭有人知道“硬件分析的绿建立在诊断假设上，而诊断假设依赖软件的行为”。生产写入卡能被机器找回来，是因为登记那天写下了“镜像源于软件维”这条绑定；而“假设依赖软件行为”这条边，从来没有人写下——它不在六个维度里，它是两个专业之间的一条缝。

小林在门口停了停，像是想起了什么：“当年编译器那次，真正出事的也不是编译器——是两个任务的相对节拍。共享的芯片、共用的电源、同一个时钟，变化就是顺着这些缝流过去的。你的查询查得动写下来的绑定，查不动没画出来的缝。”他顿了顿，补了一句当年吴工在门口问过的话的回声：“各自合格，合在一起就安全吗——现在这个问题轮到机器来接了。”

小蔡在白板角上，把那笔账正式写成了一句话，作为交给下一段路的题目：查询的射程，止于登记过的绑定。这不是本章的失败——生产写入卡与四百多项用例证明了登记过的世界有多可靠——而是它诚实的边界：六个维度描述的是一个配置由什么组成，描述不了各个部分之间怎样互相牵动。

这就是本章交出的账：变化有了形状，重开集合有了查询，两种痛快被挡在了门外——但变化沿依赖传播，而依赖得先看得见。第 18 章从这里开始：把方框之间的缝，画成机器能查的边。

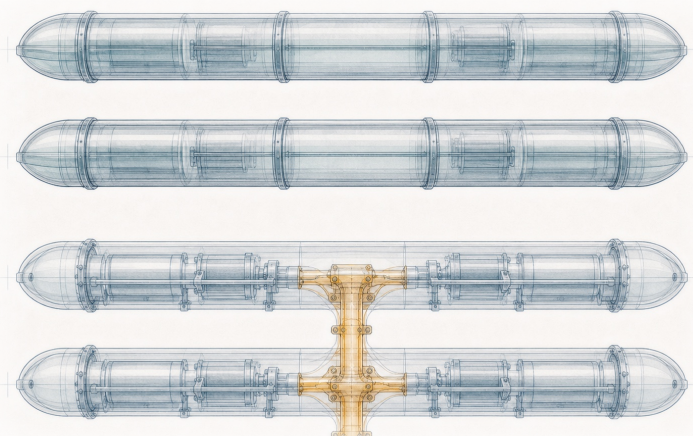
导读的承诺现在兑现。合上书之前，做三件事：找一份你手头最近的“通过”报告，试着为它补出六维快照——补不出来的那几维，就是你的 ENV-01 时刻；下一次变更单到达时，先别开会，写下“若绑定齐全，查询应返回哪些结论”，再与会议结果对照，看谁漏了谁；回想你们上一次“全部作废”或“照单全收”，数一数哪张作废的卡说不出波及面、哪张保留的卡说不出理由——那些说不出的地方，就是本章那道门要拦的地方。

**本章注记：**本章的人物、会议、事故经过与全部产品数字（候选代号、软硬件版本、标定编号、环境与工具链代号、用例数、排查天数等）均为合成教学材料，不对应任何真实企业、真实产品或真实个人；文中对变更管理与配置管理做法的转述为自然语言概括，不构成对任何实际系统的判定或放行结论。各段记法只是教学投影；唯一可执行版本是 `semantica.chapter_packages.vol2.ch17`，当前为 `partial`、`release blocked`。



## 第十八章 看见方框之间的缝： 依赖与独立性本体

Semantica 绑定：本章保存依赖、共同原因与独立性主张的概念化；唯一机器语义是 `semantica.chapter_packages.vol2.ch18`，主场景为 `semantica.vol2.ch18.scenario.primary`。本体、CQ、查询、Shape、依赖案例、规则、oracle、manifest 与 receipt 只在 Semantica。当前为 `partial`、release `blocked`；超出受支持的单调规则或已声明事实范围时必须失败关闭。



**【导读】** 上一章结束时，变化有了形状：哪一次变更、动了哪个维度、该向谁提问，机器都答得出。可每一个“波及谁”的答案，都踩在同一块看不见的地基上——依赖。变化沿依赖传播，依赖若看不见，传播就算不出。本章把第 8 章冻结的三个问题接过来：让供电、时钟、输入、代码来源、刷写源这些“方框之间的缝”成为图里的一等对象；让两条通道向上一追就相遇的共享点由一条查询点名；让“独立”不再是一句口号，而是一条押着事实的主张——查过的缝是它的分母，排除的理由是它的债务，事实换了版本，债主自动上门。第 8 章末尾曾留给你一个位置：“这个事实要是变了，今天谁会知道？”如果那时你的答案是“没有人”，本章就是为这个位置写



的。读完本章，你应当能把任何一句“这两路独立”改写成机器能够反驳的形状，并且答出那个问题：事实变了，谁会知道。

## 18.1 变化会传播，传播踩着依赖

试点推进到这一站的时候，梁工又来了。他带来的不是新方案，是一个日期：双通道监控链路的新一版原理图，两周后冻结投板。

例会上，小蔡先把上一站的成果摆在桌上：版本与变化的世界已经建起来了，一次变更动了哪个维度、按声明不变的又是哪些，机器答得清清楚楚。“但每次有人问‘这个变更波及谁’，”小蔡说，“机器往下走的每一步，踩的都是一条依赖——软件踩着编译它的工具，通道踩着喂它的电。这些依赖今天在哪儿？在原理图的走线里，在代码仓库的配置里，在梁工那张用两百多块报废样板换来的清单里。变化那边已经看得见了，依赖这边还是一片黑。”

梁工把他那张清单推到桌子中间。纸的边角已经卷了，上面是一行行手写的追问：供电向上追到哪里，时间基共享吗，两边有没有同一份库的影子。“清单可以给你们，”他说，“但丑话得再说一遍：它是学费换的，不是推导出来的，上面每一行背后都是一次‘没想到’。谁能担保它对你们这一版架构是全的？我不能。”这句话当时没有人接得住——它要到本章过半，才会得到回答。

小蔡先接能接的部分。这正是当年那场走查散会时冻结的三个问题。翻成机器要回答的能力问题，是三句话：给定两个元素，列出它们之间现存的全部耦合路径——沿哪些类型的缝、经过哪些共享点；给定一条排除理由，列出它押着的事实和各自的版本，指出哪一个已经变了；给定一份独立性结论，随时答出哪些缝立了案还开着、谁在看守。

小蔡把这三句话抄在白板最上方，作为这一站的验收边界：回答不了它们，建再多的边、画再漂亮的图，都不算数。

陈工在批准这一站的范围时只加了一句：“投板之前，我要看到供电那条缝的答案。上次它是在投板之后才被人问起的。”梁工没说话，点了点头。

要让机器回答这三个问题，先得回答一个更基本的：依赖长什么样，机器才看得见？

## 18.2 把缝画成边：依赖成为一等对象

最顺手的做法有两个，先替所有人试一遍。

第一个：在原理图上加一个“共因图层”，把共享的地方标红。这个做法不外行，很多团队真的这么干过。但图层还是几何——“同一份代码库”“同一台刷写设备”在原理图上没有位置，画不进去；而且一道红色标注没有出处，改版三次之后，没人说得清那道

红线是谁标的、还作不作数。第 8 章已经看透了这一点：最危险的耦合恰恰都画不出来。

第二个：把那张走查缝清单做成登记表，每条缝一行，填“查了”或“没查”。也不外行，这是纸面时代最好的纪律。但表格的行是散文，机器读不出一行字的两端连着哪两个对象；更糟的是，“填过”和“查过”在表里长得一模一样——清单可以印成模板，底账还是在人的记忆里。

两条路的共同缺陷是一个：缝在这些做法里都是**注释**，不是**对象**。注释可以被看，不能被查询、被反驳、被跟着版本活。出路只有一条：让每一条依赖自己成为图里的东西——有类型，有两端，有出处，有状态。

这段记法在说什么：下面是一条供电依赖边。注意它不是两个框之间的一根线，而是一个有身份的对象——它声明自己属于哪一类缝、连着谁和谁、

是哪次受控活动把它断言进图的。

```
# 一条供电依赖边（示意记法）
dep:Edge_Pwr_017 a dep:PowerDependency ;      # 类型：供电缝
    dep:from      dep:MonChainMcu ;              # 依赖方：监控通道处理器
    dep:to        dep:Reg5V_U3 ;                 # 被依赖方：上游稳压器
    dep:assertedIn dep:NetlistImport_B2 ;         # 出处：某次网表导入活动
    dep:status     dep:Active .                  # 现行有效
```

最要紧的是出处那一行。这条边不是谁“觉得”有，是某一次网表导入活动从图纸的连接关系里长出来的——将来任何人质疑它，顺着出处就能找到那次活动、那版图纸和批准入图的人。同样的骨架适用于每一类缝：时钟边连着时间基，内存边连着共享的存储区，通信边连着总线，代码来源边连着共享的库，刷写源边连着写入它的配置源。状态那一行则是留给改版的：一条边可以随新一次导入活动退役，退役不是删除——旧边留在历史里，谁曾经依赖过谁，永远可审计。类型清单正是从梁工那张学费清单里长出来的——他的伤疤第一次变成了机器的词汇。

**依赖要成为一等对象：一条边有类型、有两端、有出处、有状态，才谈得上被查询、被反驳、被跟着版本活——注释看得见，对象才查得动。**

边一条条建起来之后，第一个用它的人会看见什么？

### 18.3 同一张电源树，第二次长出来

新一版原理图冻结前的评审，排在一个周四下午——和当年那场架构评审同一个时段，小蔡后来才意识到这个巧合。

会前三天，AI 助手做了它的活：从新版网表和软件构件清单里提取依赖边，一共提出两百多条候选，每条都带着类型、两端和出处。这是**提议**。门禁逐条**校验**：形状不齐、出处指不指得到一次受控活动。其中一条“两条通道共用同一编译器”被拒收了

——助手是从一页说明文档里读来的，没有任何受控活动可押；这条被转成待核对项，交给小林去把构建记录补成正式出处。校验通过的边**执行**入图，每一条挂着导入活动和批准人，**审计**随时可查。助手在这一圈里自始至终没有制造过一条事实：它提议，图押的是活动。

评审开场，小蔡跑了那条陈工点名要的查询。

问题：主通道与监控通道沿供电边向上追，会不会在某个节点相遇？“向上追”追的不是一步：沿供电边逐级上溯，直到再无上游，两条通道各自得到一份完整的上游名单；所谓相遇，就是两份名单里出现同一个名字。

只追一步不算数——吴工当年立的那桩案子，就是向上追了两级才追到汇合点的。

```
SELECT ?node WHERE {
  dep:MainChain dep:feedsFromPlus ?node .    # 主通道供电向上闭包
  dep:MonChain  dep:feedsFromPlus ?node .    # 监控通道供电向上闭包
}
```

答案表：

| ?node        | 说明              |
|--------------|-----------------|
| dep:Reg5V_U3 | 两条通道的供电在这颗稳压器汇合 |

会议室安静了几秒——和很多年前另一间会议室里一样。原来这一版降成本变更包里，有一条把两路各自的稳压合并成同一颗上游稳压器的改动，理由是省一颗料和一块布板面积；提变更的工程师没读过当年那份走查纪要。变更本身登记得规规矩矩——在版本与变化的世界里，它是一条清清楚楚的改动记录。登记看得见“改了什么”，看不见“波及谁”；今天这条查询补上的，正是那半句。顺着图再看一步，供电缝的第二问——谁的失效能拉掉谁的电——答案也已经在图里：那颗稳压器的使能脚，接在主通道处理器的一个输出上——主通道的某几种失效，会顺手拉掉监控通道的电。图连这个拐弯都没放过，因为使能脚在网表里也只是一条连接关系，导入活动一视同仁地把它长成了边。

梁工盯着投影看了很久，说：“这张树我认识。上次我看见它，是在两百多块报废样板上。”他顿了一下，“上次把它问出来的人，是恰好想起要看电源树。这次把它问出来的，是一条每次投板前都会跑的查询。恰好，是不能列入计划的东西。”

处置仍然是人的事。评审当场把合并稳压的改动退了回去，布板面积另想办法；供电这条缝照旧立着案，看守人是吴工，结案押着台架排期里的一项供电跌落试验——查询负责把嫌疑放上桌，嫌疑之后的立案、措施与验证，走的还是当年走查立下的规矩。

同一张电源树，第二次长出来。第一次，它是投板之后由客户的联合走查揪出来的，代价是重新布板、认证重排、项目推迟两个多月；这一次，它是投板之前被一条查

询点名的，代价是改一版原理图，半天。有人会说，靠梁工在场走查也能发现——可当年梁工也在场，靠的是客户那位安全负责人恰好问了电源树；伤疤数据库不可移交，第8章已经把这条路判过了。也有人会说，写条设计规则“两路不得共用稳压器”进检查单就行——可提变更的人没读过纪要，规则也只认识见过的形态：使能脚这种拐了个弯的级联，规则的措辞罩不住，闭包查询罩得住。

同一张电源树，第一次长出来靠学费认领，第二次长出来被查询点名：缝一旦成为图里的对象，共因就从走查的运气，变成投板前的必然。

散会前，小唐说了一句让小蔡记了很久的话：“那以后独立性评审就简单了——跑一遍查询，图上不相连，就是独立。机器背书，比当年的走查还硬。”

这句话对了一半。错的那一半，要从查询的“没有”说起。

**读图任务：**沿两列模块之间的不同连线逐类检查，再对照右侧六格审计清单，最后跟随已盖章格子向下找到它依据的事实基石。

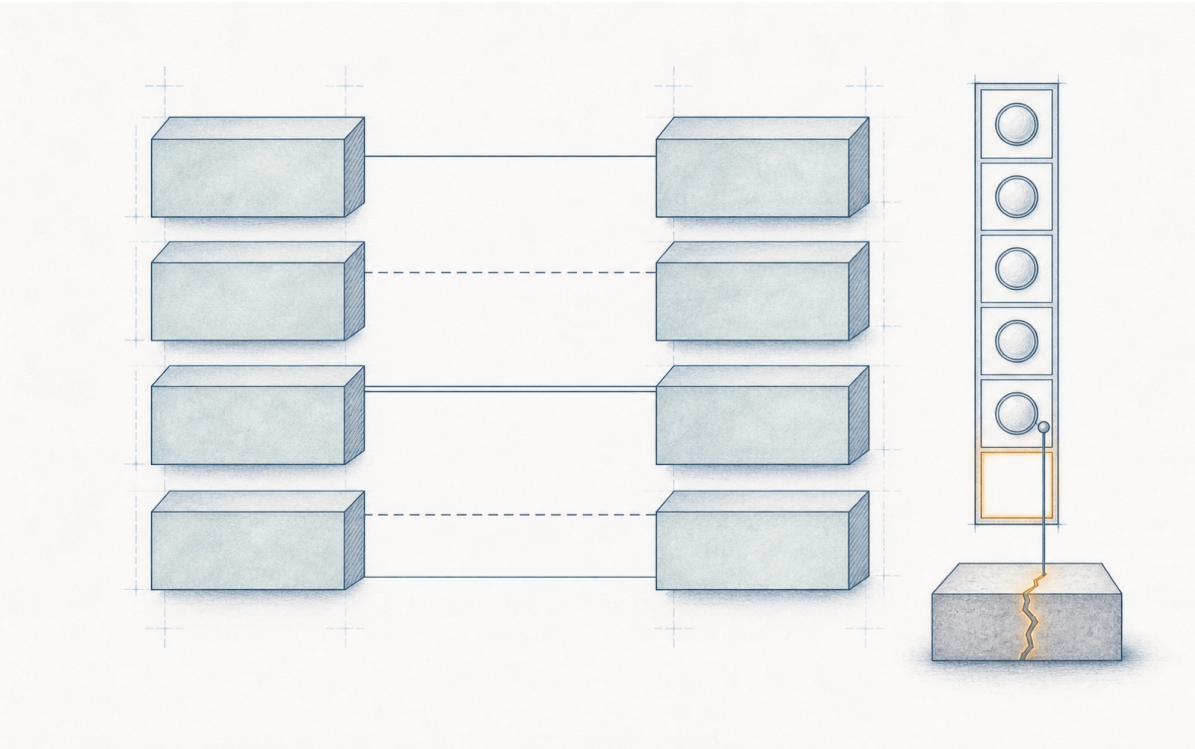


图 18-1 依赖分析的分母是真正检查过的接缝集合，而每一个排除结论又必须押在可追溯事实上。缺一格会破坏完整性，基石失效会迫使结论重开。

**图后判断：**图上印章只表示已记录查看，不证明排除永久有效。本图不评价任何具体架构的独立性，也不替代 DFA 的实际证据。



## 18.4 “图上不相连”的分母

先说小唐对的那一半：他已经在向机器要分母了，这比“两路了就独立了”进步了整整一章。查询答“有”，确实是硬的——共享点就在那里，带着出处，无可抵赖。

小蔡当场做了个试验：把同一个问题换到刷写源这条缝上再跑一遍。答案表是空的。

空表有两种读法。一种是小唐的读法：两条通道不共享刷写源，排除。另一种读法是：图里压根一条刷写边都没有——不是因为两路分开刷写，而是因为刷写关系住在产线的工艺文件里，网表导入活动根本够不着它。事实上后者才是真相：按现在的产线设想，两条通道仍由同一台设备、同一份配置源写入——当年走查立的第三桩案子原封未动，图对它保持沉默。查询的“有”是发现，“没有”只是沉默；把沉默读成排除，就是把“没建模”当成了“不存在”。小唐盯着那张空表看了一会儿，自己把话收了回去：“当年是图上画了两条路就信独立，今天是图里没查到共享就信独立——我这是把同一个错误，搬进了一个更体面的载体。”没有人笑话他：这个错误在新载体里更隐蔽，因为空表看起来比框图更像证据。

那是不是把所有想得到的边都建全，再信查询？这是另一个要排除的对策——建到哪里算全？边的类型清单本身没有底，这正是“把十个本体合成一张大图”那个诱惑的近亲：图越大越像世界，却永远差一条没人想到的边，而你不知道差的是哪条。

出路不在建全，在**声明**。每一类边配一条局部完整性声明：“供电边已由某次导入活动、在声明的范围内录全。”查询答“没有”时，必须引用这条声明才算数；引用不到，答案就不是“没有”，是“未知”——开着，待录。这条纪律试点开张时就立过，此刻第一次派上硬用场：刷写源那条缝的正确答案是“未知”，于是它立了案，案上写着待建的边、该去录它的活动，和一个看守人——而这个名字，不在设计部门的名册上。梁工那句“谁能担保清单是全的”，答案也在这里落定：担保不了，也不需要担保——声明记全的部分机器看守，声明不到的部分明着开案，最怕的从来不是“不全”，是“不全却看起来全”。

**图是声明的产物：查询答“有”是发现，答“没有”只在声明记全的范围内成立——缝要先被建模，才存在于机器的世界里；沉默不是排除。**

为什么“没建模”总被读成“不存在”？因为图不是世界自己长出来的，是各个专业为了各自的交付画的：网表为投板而画，构建记录为发布而留，采购台账为付款而记——每一类边背后，都站着一个为别的目的画它的人。而失效不认交付目录，它沿着电、节拍和数据的实际路径走，专业分工的边界拦不住它。所以图上密的地方，只说明有人天天在那里交付；图上空的地方，未必是世界空了，多半只是没有哪份交付恰好路过。缝之所以是缝，恰恰因为它落在每一份交付的边界之外——把空白读成排除，等于赌世界的形状和组织的分工长得一样。

有了边，有了共享点查询，有了“未知”的纪律，还差最后一块：“独立”这句话本身，在图里应该长成什么形状？



## 18.5 把“独立”写成押着事实的主张

纸面时代对这个问题的最好回答，是那份可以被反驳的走查记录。它的极限也清楚：把纪要扫描归档、做全文检索，字是查得到的，押注查不出——哪句结论押着哪条事实，检索不回答；给它配一个季度复审的日历，节拍又和变化对不上——改版发生在两次复审之间，多数复审空转，该响的时候不响。

图里的做法，是让“独立”成为一个有骨架的对象：它绑定两端元素；它的分母是一张缝清单——查过哪些类型的缝，逐缝给出三种回答之一：立案且开着（配看守人），排除（配理由，理由押着带版本的事实），或者未知（待录）。

这段记法在说什么：下面是那份独立性主张的骨架，取了三条缝做代表——供电缝立了案，输入缝给了排除，排除的理由押在一条带版本的事实上。

```
# 一份独立性主张的骨架（示意记法）
dep:Claim_TwoChains a dep:IndependenceClaim ;
    dep:between      dep:MainChain , dep:MonChain ;
    dep:seamChecked dep:PowerSeam , dep:InputSeam , dep:FlashSeam . 分母
dep:Case_Power a dep:OpenCase ;                # 供电缝：立案，开着
    dep:underClaim  dep:Claim_TwoChains ;
    dep:keeper      dep:HwLeadRole .            # 看守：硬件负责人
dep:Excl_Input a dep:Exclusion ;                 # 输入缝：排除
    dep:underClaim  dep:Claim_TwoChains ;
    dep:reason      " 两路使用不同型号的转矩传感器" ;
    dep:restsOn     dep:Fact_SensorPair_v3 .    # 押着的事实（带版本）
```

门禁对这个骨架有三条硬规矩：没有分母的主张不收——缝清单缺档，整份不进门；不押事实的排除不收——“我们认为不共享”这种理由没有地方落脚；押着过期事实的排除，自动标疑。前两条是收件的形状检查，第三条是活的。

活在哪里，两周后就见到了。降成本包里还有一条改动：两路合用同一颗转矩传感器。这条变更是在版本与变化的世界里登记的——那边的事经由显式的对齐合同流进来，本章不展开——流进来的结果，是传感器配对这条事实升了版。

这段示意的是门禁怎么拒绝：排除理由必须押在现行版本的事实上，事实一换版，

理由自动举手。

# 门禁（示意）：事实换版，理由标疑

对每条 `dep:Exclusion` 检查：

其 `dep:restsOn` 指向的事实是否仍为现行版本？

否 → 该排除记为 `dep:Stale`（理由自动举手）；

其所属 `dep:IndependenceClaim` 记为 `dep:NeedsReopen`。

当年那句“纸不会自己举手”，解就在这四行里：现在举手的不是纸，是押注关系。“同一刻”也不是修辞：这道检查不挂在任何人的日历上，事实换版本本身就是一次入图活动，检查跟着每一次入图跑——季度复审的毛病在节拍对不上变化，而这里根本没有自己的节拍，检查的频率就是变化的频率。“两路使用不同传感器”那行排除，在传感器合并的事实入图的同一刻变成了标疑——不靠任何人恰好记得当年那行字。标疑通知发出的那个下午，小蔡正在别的会上；等她回到座位，待办里已经躺着那条标红的排除、它押着的新旧两个版本，和被连坐成“待重开”的整份主张。没有人翻过纪要，没有人恰好想起——通知比记忆先到。她后来在试点周报里只写了一句话：“这是第一次，一份三个月前的结论在失效的当天就知道自己失效了。”

要紧的是把三件事分干净，这一站在这里分得最清楚。什么叫供电缝、什么叫押注、什么叫标疑，是本体给的语义；边和事实从哪里来，只认网表导入、构建记录、采购台账这些受控活动，助手的话和任何人的印象都不算数；而标疑之后重开走查、关掉案子、在重开后的结论上签字，属于有权的人。陈工在标疑通知发出的当天说了一句话：“机器负责举手，签字的还是人。”

“独立”是一条押了事实的主张：查过的缝是它的分母，排除的理由是它的债务，事实的版本一动，债主自动上门——重开不再靠有人恰好记得。

主张立住了。那么当年分解下的那笔信用，现在能不能入账？

## 18.6 分解的信用账，记在结构里

第 8 章把分解合同读成两页：功能的页——拆出去的每条要求独自扛得起原来那句话；信用的页——原等级的风险降低由组合加上被证明的独立共同赎回。纸面时代记这笔账的办法，是括号记号：拆出来的低等级要求写着括号里的来源等级，提醒后来的人它在一条高等级承诺链上服役。可记号活在文档的命名约定里，机器不读命名，换个模板就丢；在需求库里打个“来自分解”的标签也不够——标签指得出身世，指不回它赖以成立的那份独立性证明，证明失效了标签照绿。

图里的记法，是让分解合同自己成为对象，把赎回押在那份活着的主张上。

这段记法在说什么：一份分解合同，记着原等级、两条拆出的要求，

和它用来赎回信用的独立性主张。

```
# 分解的信用账（示意记法）
dep:Decomp_SG3 a dep:DecompositionContract ;
  dep:originalLevel "D" ;                # 原承诺的等级
  dep:partA      dep:Req_Main_C ;        # 拆出：主通道要求 C(D)
  dep:partB      dep:Req_Mon_A ;         # 拆出：监控通道要求 A(D)
  dep:redeemedBy  dep:Claim_TwoChains . # 赎回：押在这份主张上
# 门禁：redeemedBy 缺席，或其主张处于待重开 → 分解在赊账，不得担保原等级
```

最后一行注释是这份账的牙齿。传感器合并让独立性主张标疑为待重开的那一刻，这份分解合同在图里同步显形为“赊账中”——两条拆出的要求还在各自的等级上开发，但没有人可以再引用“已分解”去担保原来的承诺，直到主张重开、重结。当年“省下的钱要拿去买独立的证明”是一句提醒；现在它是一条查询就能对出来的账。

重开本身也变了性质。当年重开一次走查，等于把整张清单再走一遍，因为没人说得清哪些结论还立得住；这一次，评审只用了一个上午——需要重答的只有被标疑的那一条缝，其余的排除各自押着未动的事实，稳稳地立在原地。分母不只让“独立”可以被反驳，也让重开有了边界：事实动了多少，就重查多少。重开的结论是人签的：合用传感器的降成本提案被退回，输入缝的排除换押到新一版的配对事实上，重新入账。

**分解的信用账要记在结构里：拆开的承诺各自兑现，原等级的赎回押在一份活着的独立性主张上——主张一标疑，赊账当场现形，不用等下一次有人翻出括号。**

账都入了图。可这张图看得见的到底是什么，看不见的又是什么？

## 18.7 机器看得见的，和它看不见的

先把边界说诚实。

这张图不制造事实。那颗被点名的稳压器，本来就在网表里——查询没有发现新世界，只是让旧世界无处可藏。反过来，产线明天悄悄换一台刷写设备，图不会自己知道；图的新鲜靠受控活动喂养，喂养的纪律是人的纪律。缝的类型清单也一样：梁工的学费没有作废，它从走查时的一段记忆变成了缝目录里的类型，从此不会退休——可下一种没人见过的缝，仍要先由某个人在某次事故或某次走查里认出来、命名，机器才接得住看守。本体接住的是已经被命名的担心；命名，还是人的活。

这张图也不拥有决定。标疑不是判决，赊账显形不是否决——重开后的结论、案子的关与不关、投板的放与不放，签字的仍是有权的人。助手全程只做过两件事：提议和执行，两件都在门禁之内，两件都留了审计的痕。

问法倒是可以送人。那家做工业环境监测单元的团队听说了这一站的做法，回去把“两只探头对着同一台参考仪器校准”建成了他们自己图里的一条押注事实——从此

那句“互相校验”的结论，也有了会自动上门的债主。迁走的仍然只是问法：他们的缝要在他们的架构里命名，他们的边要押在他们自己的活动上。

评审收尾时，梁工把他那张卷了角的清单从桌上收了回去，想了想，又放下了。“留给你们吧，”他对小蔡说，“当年它是我一个人的记性，现在它是图里的一系列类型——这大概是学费能有的最好下场。”

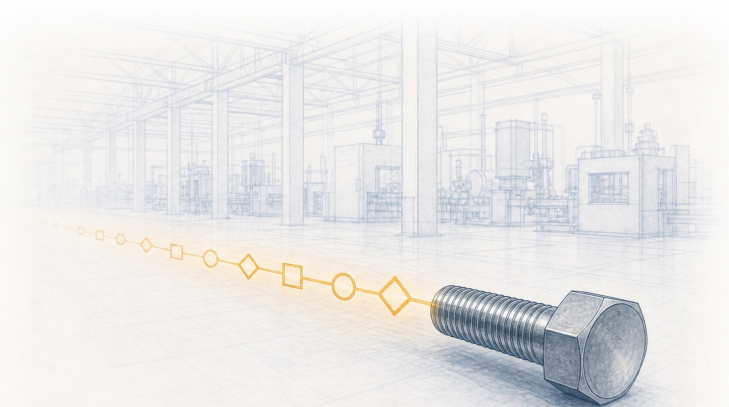
然后是那桩立了案却还没结的事。刷写源那条缝，如今在案卷上写着“未知，待录”，而它待录的边，两端都不在设计数据里：一台真实的刷写设备，一份配置源，一个器件批次——这些对象住在工厂与现场的物理世界里，住在工位、台账、批次和单件之间。独立性在分析里挣来的每一分，产线的一次代换料、维修站的一次重刷，都有机会在无意间还回去；要看住它们，依赖的图就得跟着设计一起走出分析室，延伸进制造与现场——每一颗件的来路，每一次写入的源头，都得有自己的账。下一章就从这里开始：让每一颗件都有自己的历史。

导读的承诺可以兑现了。第 8 章末尾那个位置——“这个事实要是变了，今天谁会知道？”——现在有了新的答案：押着它的每一条理由都会举手，债主会自动上门。而“没有人”这个旧答案，只在一种情况下卷土重来：缝还没被建模，图还在沉默。看见方框之间的缝，是这一站买到的全部东西——也是它对下一站唯一的嘱托。

**本章注记：**本章的试点情节、评审场景、人物与全部产品细节（双通道方案、降成本变更包、稳压器合并与传感器合并、当年供应商事故的样板数量与延期时长等）均为合成教学材料，不对应任何真实企业、真实产品或真实个人。文中对功能安全标准中 ASIL 分解、相关失效分析等要求的转述为自然语言概括，精确要求以标准原文为准。各记法片段只是教学投影；唯一可执行版本是 `semantica.chapter_packages.vol2.ch18`，当前为 `partial`、`release blocked`。

## 第十九章 每一颗件都有自己的历史： 制造与现场本体

Semantica 绑定：本章保存制造身份链、事件历史与替代影响的概念化；唯一机器语义是 `semantica.chapter_packages.vol2.ch19`，主场景为 `semantica.vol2.ch19.scenario.primary`。本体、CQ、查询、Shape、ledger/替代正反案例、规则、oracle、PROV 与 receipt 全部在 Semantica。当前为 `partial`、release `blocked`；教学查询不能声明真实单件、批次、装车或现场范围。



**【导读】** 依赖与共因在图里看得见之后，试点例会上老何只问了一句话：“你们图里的方框，到我车间里会变成一箱一箱的料。料，你们的图认识吗？”本章把第 9 章末尾冻结的三个问题接过来，在机器可查的世界里重建那条从图纸通到每一颗件的链：设计定义、批次、单件各自成为对象；写入、检验、发运、维修、换件各自成为带时间的事件；设计押下的前提从专家的头脑里请出来，挂到产线的控制点上。然后，让几年前那场两千一百台的追回在新载体下重演一遍——这一次算得出差价。读完本章，你应当能对任何一颗现场的件问出三连问——它是谁、它经历过什么、它还站在哪些设计前提上——并说出机器凭什么答得出来。



## 19.1 数据都上系统了，为什么还答不出“哪些件”

试点推进到依赖那一站之后，“两条通道共享哪一路供电”第一次不是被专家走查揪出来，而是被一条查询答出来。庆功的话还没说完，被请来做领域评审的老何把他那个起了毛边的活页夹放在桌上：“你们图里的方框，都是设计世界的东西。到我车间里，方框会变成一箱一箱的料、一颗一颗的件。料和件，你们的图认识吗？”

陈工没有替图辩护。他把第9章末尾冻结的三个问题原样写上白板：给定现场的一颗件，机器能不能答出它从设计定义、料源批次、单件记录到服务历史的完整身份；给定一次代换或偏差，机器能不能答出它击穿哪些设计前提、波及哪个范围；给定一批工单，机器能不能把它们自动关联回批次与配置。“这三问就是这一站的验收边界。答不出来，前面建的图再漂亮，到厂门口就断了。”小蔡带着助手驻厂，第二次去跟老何的线。

动工之前，小唐先提出了一个省力的判断：“这三问咱们厂不是早解决了吗？前年就上了制造执行系统，来料、工位、检验、发运，每一步的数据都进了库。数据都上系统了，追溯还能是问题吗？”

这句话一点都不外行。上系统正是这个行业过去十年做对的事：纸质台账进了数据库，手抄记录变成了扫码，数据量翻了几个数量级。如果这个判断不成立，问题一定藏在“数据”和“追溯”之间的某个缝里。

小蔡没有争论，拉着小唐当场试。第一问：追一颗在役控制器的完整身份。两人打开了四个系统、打了两个电话、翻了一份装箱单的影印件，用了一下午——数据都在，但图号在设计系统里，批次在仓储系统里，序列号在产线库里，工单在售后系统里，每一次跨库都靠人认字段、猜口径。第二问更难看：“哪些件装了某一批料”——装配工位的记录里有料箱扫码，可它记的是“这个班次消耗了这箱料”，没有一条边通向“哪几颗件”。数据一直都在产出，链从来没有被造出来。

两条自然的补救很快被摆出来，又很快被放下。再上一套更大的系统、建个数据中台？第9章已经说过一次的话在这里应验：系统负责存储，不负责关联——把四个库并成一个库，跨不过去的还是跨不过去。那就规范字段、强制填写？字段规范了仍然是字符串：两个库里同样写着“批次号”的两列，没有任何判据保证它们说的是同一种批次。填得再全的表格，也不会自己长出边来。

**追溯缺的从来不是数据，是身份链；没有身份链的系统，只是一个更大的堆。**

那么“身份”到底要几层，链才够用？

## 19.2 三层身份：一张图、一箱料、一颗件

老何当初带小蔡走的三站——来料区、拧紧工位、返修区——其实早把答案摆出来了：图纸只有一张，料是一批一批来的，装到车上的是这一颗。设计定义、批次、单

件，是三种各自能被提问的对象：设计定义回答“这一类应该是什么”；批次回答“这一批从哪来、什么脾气、哪个时间窗做的”；单件回答“这一颗是谁、经历过什么”。

先排除两条更省事的路。其一，统一编码、一物一码：码当然要打，但码是名字，名字不带判据——外壳换过、码重贴过的那颗件，码还是它吗？光有码，同名是否同物依然靠人赌。其二，把批次信息作为单件记录里的一个字段：字段能存“这颗件来自哪批”，却让批次永远当不了提问的立足点——“这一批消耗了哪箱料”“这一批的时间窗是什么”在字段世界里无处安放。名字和字段都不够，三层必须各自成为对象。

这段记法在说什么：三层身份各自立为对象，用两条边串起来——批次实现设计，单件隶属批次；批次还带着自己的料源和时间窗。

```
# 三层身份：一张图、一箱料、一颗件（示意记法）
fld:EPS_Controller_H32 a fld:DesignDefinition .      # 设计定义：一类产品
fld:Batch_A45 a fld:Batch ;
    fld:realizes    fld:EPS_Controller_H32 ;          # 这一批实现哪张设计
    fld:consumedLot fld:SupplierLot_B ;                # 料源：消耗了哪箱料
    fld:madeIn      fld:Window_W45 .                  # 生产时间窗
fld:Unit_SN000417 a fld:Unit ;
    fld:memberOf    fld:Batch_A45 .                  # 这一颗属于这一批
```

注意其中最不起眼的一条边：批次消耗了哪箱料。它在旧世界里是装箱单上的一次扫码，扫完就沉进库里；在图里它是一条随时可以反向行走的边——本章结尾那次“算钱”，全部押在这条边上。这条边不要求产线多干一件事，两头的记录本来就有人在留：料箱进厂，来料登记给了它一个名字——哪家供应商、哪个料批、哪天到的；料箱上线，工位消耗记录说了一句——哪个班次拆开了这箱料。旧世界里这两笔各躺各的库，谁也不认识谁；图做的只是让它们握手：班次落在哪个批次的时间窗里，这箱料就记进哪个批次的账。至于“同名是否同一颗”的裁决判据和谁有权写下“同一”，那是对象与同一本体的地界，本章经由显式对齐合同与第 12 章协作，直接使用其结论，不再重建。

**设计定义、批次、单件是三种对象；名字相同，不使它们成为彼此。**

可单件不只是一个躺在批次里的名字。返修区那颗控制器和线上直通下来的那颗，最终检验结果一模一样——差的是经历。经历，怎么进图？

### 19.3 历史长在单件身上：事件账

旧世界给“经历”留过两个位置，都不够用。一个是备注栏：自由文本，机器读不动，改一次覆盖一次，历史越攒越像传说。另一个是“当前状态”字段：最新软件版本、最新检验结论——状态是历史在此刻的投影，投影丢掉路径；返修那颗与直通那颗状态完全相同，经历完全不同，靠状态字段永远分不开。

出路是把每一次“发生”本身立为对象：写入是一个事件，检验是一个事件，发运、维修、换件各是一个事件，每个事件带时间、带对象、带记录它的受控活动。单件的历史不再是谁记性好，而是一次查询：把落在这颗件上的事件按时间排开。

这段记法在说什么：一颗件的三条历史——写入、检验、发运——各自成为带时间的事件对象，并且每条都写明由哪个受控工位记录。

```
# 一颗件的事件账：历史不是备注栏，是对象（示意记法）
fld:Write_0417_1 a fld:WriteEvent ;
    fld:onUnit fld:Unit_SN000417 ;
    fld:wrote fld:SW183_C41 ;           # 写入了哪套软件与标定
    fld:at    " 第 45 周三 10:12" ;     # 什么时候
    fld:recordedBy fld:FlashStation_7 . # 由哪个受控工位记录
fld:Inspect_0417_1 a fld:InspectionEvent ;
    fld:onUnit fld:Unit_SN000417 ;
    fld:verdict fld:Pass ; fld:at " 第 45 周三 10:15" .
fld:Ship_0417 a fld:ShipmentEvent ;
    fld:onUnit fld:Unit_SN000417 ; fld:at " 第 46 周一" .
```

“由哪个受控工位记录”这一条不是装饰。它是这本账的事实来源纪律：只有受控活动产出的记录才能进事件账——扫码枪扫过的、拧紧机存过的、读回比对留过的，才算发生；口头说“刷过了”不算。第 9 章那张控制卡上“留下什么”一栏的每一行，在这里都变成了事件对象的一个字段。还有一条配套纪律：事件账敢回答“这颗件没返修过”，前提是有人声明过“这颗件的返修记录已记全”——没有这句局部完整性声明，账上的空白只是空白，机器必须答“未知”，不许替人答“清白”。

**历史长在单件身上：每一次写入、检验、发运、维修、换件都是一个带时间的事件对象；单件的历史是查询的结果，不是备注栏的传说。**

件有了身份，也有了历史。可老何事故里真正断掉的那一环还没接上——设计押在这颗料上的前提，还锁在做过分析的人的头脑里。前提怎么成为对象？

## 19.4 前提请出头脑：设计前提对象与代换合同

第 9 章末尾说得很直白：纸面能记下结论，记不下结论押在哪里。事故之后加的那栏“设计评估意见”，本质是把问题转给另一群人的记忆；对照表加长的路也早已判死——不知道设计押了哪些行为，表就永远缺一行。两条路的共同缺陷是一样的：前提没有身份，就没有东西可以被点名、被看守、被追问。

新载体的做法，是把移交包里那份“点名清单”上的每一项立为对象：一条设计前提，写明押在哪颗料上、主张什么、撑着哪条设计结论、由产线上哪个控制点看守。第

9 章的控制卡由此获得了它的另一半——控制点不再只是“查什么”，而是明确地看守着某几条前提。

这段记法在说什么：当年那颗电流采样电阻上押着的前提成为对象，挂上它撑着的结论和看守它的控制点；一张代换申请也成为对象。

```
# 设计前提成为对象，挂在产线控制点上（示意记法）
fld:Premise_TempCo a fld:DesignPremise ;
    fld:aboutPart fld:SenseResistor_R12 ;    # 押在哪颗料上
    fld:claims    " 低温端温度系数走原厂实测曲线" ;
    fld:supports  fld:DiagWindowResult ;    # 撑着哪条设计结论
    fld:guardedAt fld:IncomingCheck_3 .    # 由哪个控制点看守
fld:SubReq_47 a fld:SubstitutionRequest ;
    fld:replaces fld:SenseResistor_R12 ;
    fld:withPart fld:SenseResistor_R12B .    # 拟代换的新料
```

代换在这张图里是一种关系，而关系有闭合条件：任何一张代换申请，必须经过一个设计评估对象——评估要覆盖新料触及的每一条前提——这条关系才算成立。评估结论怎么下，是设计的专业与责任；图只管一件事：这条边缺不缺。

这段记法在说什么：机器怎么拒绝一张没有评估的代换单。

```
# 门禁：缺设计评估的代换，机器拒绝（示意）
拒绝 fld:SubReq_47 ，因为：
    要求 每个 fld:SubstitutionRequest
        必须有 fld:assessedBy → 一个 fld:DesignAssessment ，
        且该评估覆盖新料触及的每条 fld:DesignPremise ；
    实查 fld:SubReq_47 没有 fld:assessedBy 这条边。    # 缺评估即拒
```

**“等效”在图里重新成为设计结论：代换关系必须经过设计评估对象才能闭合——缺了评估，机器连这条边都不让你画。**这不是又一条要求人记得的规定，而是一处结构：规定可以被忙碌绕过，结构绕不过去。

老何盯着那六行看了很久，说：“当年没有这道门。”陈工接的话是一个决定：“那就演一遍。把当年那六个月的台账搬进图里，让它第二次发生。”

**读图任务：**从左侧被聚光的料箱开始，逐跳跟随批次、其中单件、装车 and 发运路径，找出哪一跳由实线变成了断开的琥珀色虚线。



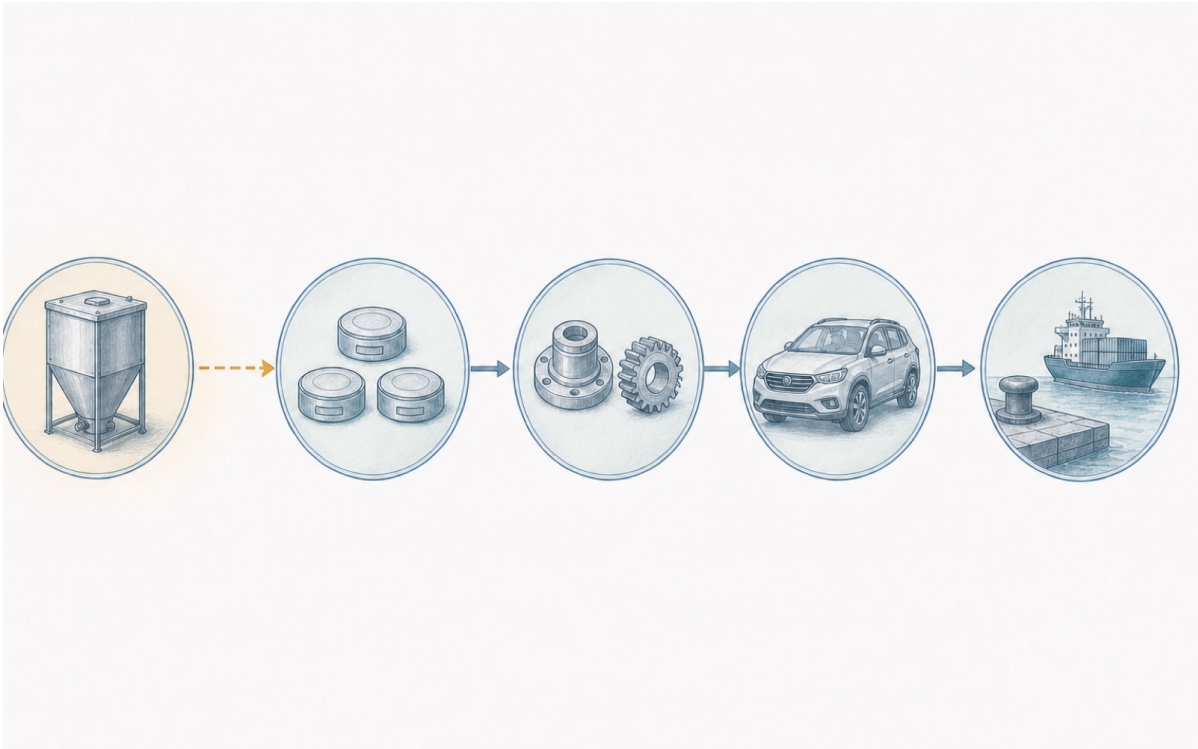


图 19-1 现场追查是一条可执行的身份路径，而不是某个系统里有一张表就算完成。任一跳未知，都应显式限制搜索结果和影响范围的声称。

**图后判断：**图中路径是教学模型，不是供应链事实记录。聚光和断链不证明风险大小，也不替代召回范围、所有权或隐私授权判定。

### 19.5 同一次断供，第二次发生

演习先要把旧台账变成图。这一段是助手的活，也是它的边界最清楚的一段：它从装箱单影印件和旧库表里逐条提出三元组——哪箱料、哪个批次、哪个班次——每一条提议都要过门禁（事实必须挂着受控记录的来源），老何按批抽检，通过才写入，每次写入留审计痕。中途助手发现两个批次的料箱消耗记录缺失，提议“按时间就近推断补上”，被门禁原样退回：推断可以作为标了记号的候选留给人判，不能写成事实。两个候选后来人判掉一个：仓库的纸账翻了出来，补上实据入图；另一个翻遍旧账没有着落，就让它空着。图不嫌账有洞，图只拒绝把洞粉刷成墙。

第一幕，代换单重演。十四周的交期、三周的库存、十二行的对照表，原样提交。门禁把它停在了闸口：没有设计评估对象。当年老何那句“这不叫改设计，这叫换个牌子买同一颗料”，这一次被机器温和地顶了回来——对照表不是评估。补一次评估只用了一个下午：设计工程师调出新料触及的前提清单，低温温度系数那一条赫然在列，结论是“需实测低温曲线后再判”。当年要三周排查加一个冬天的投诉才浮出水面的那行字，现在是评估清单上现成的一行。事故在第一幕就已经不会发生了。



但老何不肯就此收场：“当年就是放进去了。放进去之后呢？”于是第二幕，反事实重演：假设有权者当年顶着缺口签了放行——图拦不住有权的决定，它只把缺口和签名记录在案——三个月混线生产照旧，入冬后北方的工单进来。三十七张“低温助力降级”的工单先自己聚了类：沿着车辆、单件、批次往回走，全部落在同一个料源批次、同一段时间窗上。这是白板上第三问的答案顺手兑现。然后是那次最能算钱的查询。

问题：哪些在役的件，真的装了代换批的料？

```
# 查询：从代换料批反向走到每一颗在役件（示意）
# 沿批次-单件链反查嫌疑单件与发运去向
# （装车此处从简记作一条边；完整建模中应为带时间的事件）
选出 ?unit ?vehicle ?shipDate ，条件是：
    ?batch fld:consumedLot fld:SupplierLot_Sub47 . # 消耗了代换料批
    ?unit fld:memberOf ?batch .
    ?unit fld:installedOn ?vehicle .
    ?ship a fld:ShipmentEvent .
    ?ship fld:onUnit ?unit .
    ?ship fld:at ?shipDate .
```

| 圈定方式            | 范围                       |
|-----------------|--------------------------|
| 旧世界：按三个月时间窗全量追回 | 两千一百台                    |
| 新世界：沿批次-单件链反查   | 六百八十三台，另有一批次消耗记录缺失，标“未知” |

那条第 19.2 节埋下的“消耗了哪箱料”的边，在这里兑付。标“未知”的那个批次，图不说“没装”——老何让人翻出当年的装箱单，把代换料批的箱子逐箱对账：每一箱都对到了别的批次头上，一箱不剩，这才把它排除；排除本身也进了图，挂着依据和经手人的名字——洞不是被悄悄抹平的，补洞的人写在洞的旁边。最终嫌疑六百八十三台，发运去向逐台在案，精准扣留，不必停线。老何在答案表前站了一会儿，说了一句和当年复盘会对仗的话：“当年多追回的一千四百多台，赔的是答不出‘哪些件’的钱。这条八行的查询，就是那笔钱的对面。够把这个试点养上十年。”

值得说破的是这场演习里的三种分工：什么算一次代换、什么算一条前提、边怎样才闭合，是本体定的——这是语义；台账里哪箱料进了哪个批次，只能来自当年受控活动留下的记录——这是事实；评估怎么下结论、缺口能不能顶着走、追回追到哪一台为止，签字的是人——这是决定。三者谁也替不了谁：语义不制造事实，事实不产生授权，授权也改不动语义。

同一个问题，旧世界的价钱是一千四百多台的冤枉追回，新世界的价钱是一条八行的查询——差价买的不是软件，是那条平时没人看见的身份链。

链修到厂门为止吗？车开出去之后，谁接着往这本账上写？

## 19.6 厂门之外：换件、回声与另一台设备

服务站的那一幕也重演了一遍。旧硬件的车进站换件，技师习惯性去点“最新软件包”——这一次拦住他的不是工具碰巧带的告警，而是同一张图：刷写提议先过绑定检查，按这一颗件的硬件修订和这辆车的配置历史取适用包。

这段记法在说什么：换件重刷前，机器按“这一颗”的身份拒绝不兼容的提议。

# 门禁：换件重刷前的绑定检查（示意）

拒绝 本次刷写提议，因为：

目标包 `fld:SW230` 要求硬件修订不低于 `fld:RevD`；

本车件 `fld:Unit_SNP2201` 的修订是 `fld:RevC`；

结论 包与这一颗件不兼容 → 提议退回，  
改按该件配置取历史适用包。

远程复核的正是小林。他看完审计记录只说了一句：“上次是工具替我们碰巧拦了一回，这次是图里本来就查得出。”换件本身也进了账：一个换件事件，旧件退出、新件装入、各带时间——旧件的历史不因退出而终止，它带着完整的事件账进回收线，一路连续到拆解。第 9 章担心的“身份链在报废那一站被剪断、在看不见的地方接错”，在图里没有那一站：链不终止，只改变去向。现场的回声从此有了回程票：每一张工单沿着车辆、单件、批次、写入事件走向“哪一颗、哪一版、哪个时间窗”——第 9 章说的“回不去的数据只是繁忙的噪声”，回得去，就是证据。

同样的问法，也被 ENV-01 的团队借走了。那台环境监测设备两年前的旧账——换过传感器模块之后，绑在整机序列号上的校准证据还属不属于它——在他们自己的世界里有了新的问法：整机序列号只是名字，校准证据应当绑在模块单件的事件账上，换件是一个带时间的事件，换件之后哪些证据随旧件退役、哪些仍对整机有效，是查询能答的问题。但模块划到什么粒度、校准周期怎么绑、服务商的记录算不算受控活动，每一条都得在他们自己的世界里重新挣——问法可以借，答案不能抄。

图答得越来越多了。也正因为如此，得趁热说清楚：它不是什么。

## 19.7 图不制造事实：诚实边界与放行桌

演习收尾的会上，小唐盯着那张六百八十三台的答案表，提了一个顺理成章的建议：“范围是它算的，比谁都准。追回指令也让助手直接发了吧，还等什么？”

这话的诱惑在于它几乎是对的。但把它拆开，是两个必须分开的东西：圈定范围是回答问题，发追回指令是行使权力。助手在这个试点里的地位从第一天起就没变过：提议者与执行者——它可以把追回方案连同证据链起草到可以直接签发的程度，签发那一下，必须落在有权者的名字上。机器给的是可核对的射程，不是决定。当年老何签

错的那张代换单，错不在签字这个动作，而在签字的人手里没有前提清单；今天的图把前提递到了签字的人手里，签字仍然是他的。

另一半边界同样要紧：本体不制造事实。产线上没扫的码、服务站里没记的换件、供应商没留的批次，图里就是没有——它们不会因为图漂亮就长出来。演习里那个标“未知”的批次是这条边界的活样本：图的全部诚实，就在于它把洞标成洞，而把“补洞”留给受控活动和有权的人。这本事件账的价值不是全知，是有洞也知道洞在哪。

现在可以把这一站的账数一遍：件有三层身份，历史是带时间的事件账，设计前提成为被看守的对象，代换必须经过设计评估才能闭合，换件绑定“这一颗”，现场回声沿链走向批次与配置。白板上那三个冻结的问题，逐一有了机器的答案。从概念、需求、度量、版本、依赖到制造与现场，试点把链修到了最后一环——每一环的证据都是活的，随时可查，过期自知。

下一轮候选的放行窗口已经排上了日程。把这条全链证据搬回第 1 章那张放行桌的时候，桌上摞着的将第一次不是纸，而是能被查询的链。可放行桌上还坐着一份最重要的文档：安全案例——主张、论证、证据组成的那份总账。证据都活了，它还是死的吗？产品变一版，它能不能自己报出哪些主张过期、哪些论证要重开？第 20 章回到那间会议室，做全书最后一件事：让安全案例活起来。

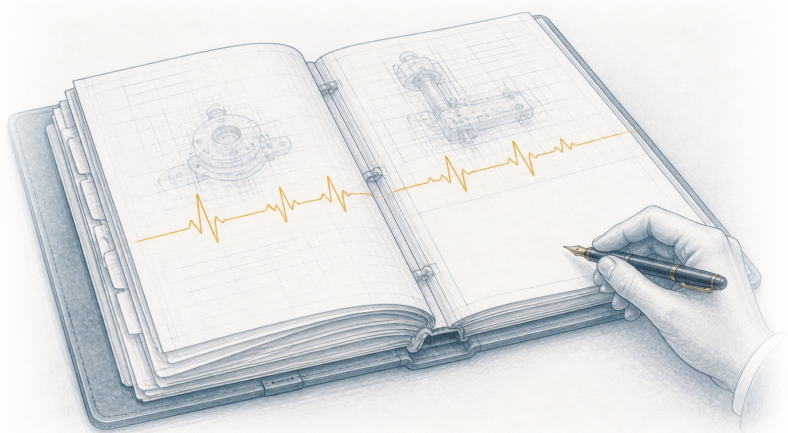
导读的承诺可以兑现了。合上书之前，做两件事：回到第 9 章末尾你卡住的那一环，判断卡住的原因是缺数据还是缺身份链——十有八九是后者；再挑一颗你产品里的关键料，试着把“设计押了它什么”写成一条有对象、有主张、有看守点的前提，看看第一稿能写出几行。写不出来的那几行，就是你的老何正在用记忆替系统扛着的地方。

**本章注记：**本章的人物、试点场景、演习过程与全部产品数字（批次数、台数、时间跨度等）均为合成教学材料，不对应任何真实企业、真实产品或真实个人；文中对生产、服务与追溯相关标准要求的转述为自然语言概括，精确表述以标准原文为准。各节记法只是教学投影；唯一可执行版本是 `semantica.chapter_packages.v012.ch19`，当前为 `partial`、`release blocked`。



## 第二十章 活的安全案例： 发布与保证本体

Semantica 绑定：本章保存持续保证、发布绑定和人机决定边界；唯一机器语义是 `semantica.chapter_packages.vol2.ch20`，主场景为 `semantica.vol2.ch20.scenario.primary`。本体、CQ、查询、Shape、正反案例、工程规则、legacy capstone 迁移证据、oracle、snapshot、PROV、receipt 与 release verdict 均在 Semantica。当前为 `partial`、release `blocked`；机器把候选算到可审查，不能替有权人签字，也不能把被阻断的发布说成完成。



**【导读】** 试点走到最后一站，要换载体的轮到放行桌自己。本章先把前半部收口时冻结的三个“纸面到不了的地方”翻译成机器必须能回答的问题；再用一个最小的发布保证本体，把发布快照、证据包、主张—论证—证据三层、冲突、重开条件与授权决定——变成对象；然后让那个深夜的旧事故第二次发生——这一次在装包的当口就被拦下；让一场从天而降的模型升级考验这套结构里什么会死、什么会活；最后陪 RC18 走完第一场“活的”放行评审，把全书悬着的三个问题收回来。读完本章，你应当能说出一份安全案例怎样才算活着——它靠什么自己对表、自己现身、自己举手，又在哪一步必须把笔交还给人。



## 20.1 十套语言，回到一张桌子

试点进行到第十一个月，放行桌上最先消失的是推车。从设计到现场，每一环的证据不再装箱运来，而是各自以对象的身份住在自己的世界里：担心与目标住在危害那边，需求与派生理由住在追溯那边，数字带着来源住在测量那边，版本与变更住在变化那边，依赖与共因住在依赖那边，批次与单件住在制造与现场那边。老何上个月刚用单件履历把一次嫌疑圈定从两千一百台压到七百台以内；此刻他把那条批次—单件链的出口，接到了放行桌上。全链证据都回得来了。问题只剩一个，也是当初冻结时的最后一个：案例能不能活起来？

牵头收这一站的还是小蔡。一年前她还是那个把三百多份文件装进三个纸箱、以为“装订好就齐了”的新人；如今九个本体的对齐合同上，一半有她的名字。试点作战室的墙上贴着三句话，是她从那次收口评审抄回来的，也是前半部留下的总账：证据包没法自己核对同一基线；变更影响靠人工圈定；安全案例是死文档，产品是活的。试点的最后一站，就是把这三句话逐句销账。而放行这一站还有个别处没有的难处：前面九个本体在这张桌上全要露面。RC17 当年写下的三条重开条件，先后响了两条——升版获准落地，决定窗口到期。于是有了新一轮候选 RC18：H3.2 硬件、转为正式版的 SW1.8.4、经差异分析确认沿用的 C41，D7 与 V12 依旧。它的放行，就是这场试点的毕业考试。

动手之前，先冒出来一个所有人都动过的念头。这次把它说出口的是小唐：“咱们手里已经九套词汇了，加上这套是十套。放行桌上哪一套都躲不开——那不如趁收尾把十个本体合成一张总图，以后维护一处、查询一处，一劳永逸。”

这个提议一点都不外行。把分散的模型统一成一个大模型，是数据库年代真实成功过的路线；十套词汇十套账，任谁看着都觉得冗余。小蔡没有争论，她请助手做了一下午试验：先挑两家试着并——把测量那边的“受测对象”与制造那边的“物理单件”合成一个类。下午还没过完，试验自己停了。两家对“还是不是同一个”的判据本来就不同：一边按配置与构建认同一，一边按序列号与履历认同一；硬并成一个类，等于逼着两家共用一套身份判据——那正是当年花了一整章才拆开的东西。更要命的是看守：每个本体都有自己的看守人、自己的完备性声明、自己的重开纪律，一张大图归谁看守？改一条边，十个领域一起震动——制造那边想给单件补一条报废去向的边，得先问过测量、变更、依赖各家答不答应——结局不是一劳永逸，是谁也不敢动。

那就换个方向偷懒：十个本体不动，放行桌自己再建一套“什么都说一遍”的超级本体，把各家内容复述进来。这条路更隐蔽也更糟——复述的那一刻就开始漂移，从此世上有两份真相，每次不一致都要开会裁决谁算数。同名异义的老病，换了载体又要复发。

小唐听完试验汇报，自己把提议收了回去。陈工只补了一句裁决：“十个院子十把锁。这张桌上，只摆各家愿意交出来的钥匙牌。”

两条路都堵住，放行本体该长什么样才显出来。放行桌需要知道的其实很薄：每份证据是谁、绑定在哪个快照上、作证到哪里、此刻什么状态、归谁看守——至于它内部更深的结构，留在自家本体里，经显式对齐合同按需去查。**放行需要的不是一张吞下一切的大图，而是每个本体守好自家边界之后，愿意向桌面交出的那层薄接口。**十个本体还是十个，桌面上多出来的只是一层。

薄归薄，这层接口要接住的却是三句重话。把墙上的账翻译成机器必须能回答的问题，就是这个本体的验收边界。其一，任何时刻问“这个证据包的成员是否全部绑定同一个发布快照”，机器要能直接回答，并指出出不合群的那一份——不再依赖谁深夜逐行抄表。其二，任何一处获准的变更落地之后，问“案例里哪些主张、论证与证据关系因此过期、哪些决定需要重开”，答案要从结构里查出来，不再出自某个人的私人记忆。其三，任何时刻问“这份案例现在的状态”——哪些在案、哪些过期、哪些未知、哪些缺口、谁看守、什么条件下重开——案例要能自己举手，而不是躺在库里等人想起。

三个问题都要机器作答，案例就不能再是一摞文档。它得变成东西——变成什么样的东西？

## 20.2 把案例变成对象：最小发布保证本体

第一步还是老老实实把话说白。

发布快照，是那句大主张的主语：一组配置的共同身份。第 1 章补主语、补配置补出来的东西，在这里第一次成为可以被指着说的对象，而不是封面上的一行字。证据包，是一次放行所引用证据的显式集合。关键动作有两个：包自己声明“我属于哪个快照”，每个成员各自声明“我绑定在哪个快照”——两处声明必须相等。“同一基线”从此不是核对出来的结论，而是结构里一条可查的边。

这段记法在说什么：下面立出 RC18 这个快照和它的证据包。注意三处细节——五项配置

各是对象；包上那行“清单已记全”；以及成员自带的两条边。

```
# 发布快照与证据包（示意记法）
rel:RC18 a rel:ReleaseSnapshot ;
    rel:configItem rel:HW_H32 , rel:SW_184 ,
        rel:CAL_C41 , rel:DRV_D7 , rel:BASE_V12 .
rel:PKG_RC18 a rel:EvidencePackage ;
    rel:declaredSnapshot rel:RC18 ;           # 包声明自己属于谁
    rel:hasMember        rel:SWRegReport_r11 ;
    rel:memberListClosed true .               # 清单已记全，缺席可判
rel:SWRegReport_r11
    rel:boundTo    rel:RC18 ;                 # 成员自带绑定
```

```
rel:producedBy rel:SWRegRun_r11 .          # 事实出自受控活动
```

“清单已记全”那一行最容易被看轻，它是机器有资格喊“缺”的前提：只有在被告知成员清单已经封口之后，论证里引用了、包里却没有的那张卡，才成立为缺口，而不是永远的沉默。最后一行也不是装饰：每份证据必须指回产出它的受控活动——本体负责说清这份证据是什么，而它作证的内容真不真，只能由台架、产线与试验场的活动记录负责。语义与事实，从进门第一行就分账。

第二步，把当年那场“写在纸上的辩护”逐层对象化。主张是对象，论证是对象，证据是对象；论证引用证据的每一次引用是一条边；而方工说过“论证是唯一不能由单个专业交出的东西”，那句“为何这些证据合起来够了”的理由，现在也写成对象的属性，不再只活在会议室的空气里。

这段记法在说什么：三层结构挂好之后，把当年写在纸上的重开条件也变成对象

注意它的看守人是一个角色，不是一个人名。

```
# 主张—论证—证据三层，外加重开条件（示意记法）
rel:Case_RC18 a rel:AssuranceCase ;
    rel:topClaim rel:Claim_ReleaseRC18 .
rel:Claim_ReleaseRC18 rel:supportedBy rel:Arg_SafetyGoals .
rel:Arg_SafetyGoals
    rel:cites    rel:SWRegReport_r11 ;          # 论证引用证据
    rel:warrant " 这些射程有限的证据为何合起来足够" .
rel:Reopen_AnomalyRecur a rel:ReopenCondition ;
    rel:trigger  " 台架异常读数复现" ;
    rel:reopens  rel:Arg_SafetyGoals ;          # 触发即重开该分支
    rel:guardian rel:Role_RigOwner .            # 看守的是角色，占位的是人
```

重开条件从“写得再好也得有人来读”的三行字，变成了长着触发器的对象：它声明触发条件、声明重开范围、声明看守角色。看守写成角色而不是人名，这一笔到本章结尾会显出分量——人会调岗、会退休，角色不会跟着走。图里还有两类对象在待命。一类是冲突对象，用来接住“一份证据与案例的声明顶了牛”这件事本身：机器既不悄悄丢弃它，也不悄悄收留它，而是立案、通知看守、等人处置，处置记录永远在案。另一类是授权决定对象，把签字变成结构：谁（角色）、基于签署那一刻的案例状态、授权范围、决定窗口、附带哪些重开条件。它旁边站着一个刻意分开的类——提议对象。助手能生成的只有后者。这条界线，要到全章最后一幕才真正兑现。

结构立起来了。可结构的第一场大考不是评审会，而是所有人都记得的那个深夜——它还会再来吗？

### 20.3  同一个深夜，第二次到来

装包那天是个周三。下午四点，助手按论证层的引用清单提议了装包草案：逐条列出应到成员，从各家世界取来绑定 RC18 的修订，生成清单交门禁校验——提议、校验、执行、审计，一圈都在明处。四点零六分，门禁停在了一个成员上。

还是软件回归报告。小林组的习惯没改：为下一轮升版做预演，预演工作区照常生成了一份更新的修订。预演本身没有错——当年错的从来不是预演，是打包脚本那条“取最新修订”的规矩。今天的取件规矩换了：按绑定取件。而那份新修订绑定的是下一版软件的预览快照，不是 RC18。

问题：装包草案的成员，是否全部绑定在 RC18 上？

```
SELECT ?member ?binding WHERE {
  rel:PKG_RC18_draft rel:hasMember ?member .
  ?member rel:boundTo ?binding .
  FILTER ( ?binding != rel:RC18 )      # 找出不合群的那一份
}
```

| 成员                  | 绑定                          |
|---------------------|-----------------------------|
| rel:SWRegReport_r12 | rel:Snapshot_NextSW_preview |

答案表只有一行，但这一行在一年前值一个通宵。这段记法在说什么：门禁的拒绝不是  
一条报错，而是一份留痕的处置开单——注意最后两行。

```
# 装包门禁：成员绑定必须等于包声明的快照——机器这样拒绝
不通过  rel:PKG_RC18_draft
违例成员  rel:SWRegReport_r12
成员绑定  rel:Snapshot_NextSW_preview  # 下一轮升版的预演工作区
包内声明  rel:RC18
处置      拒收该成员；生成 rel:Conflict_PKG18_01 ；
          通知软件侧看守角色，处置留痕
```

四点十一分，小林收到通知；四点二十，他把正确的修订确认回包，那个冲突对象记下处置理由，关卷归档——不删除，就像当年那条共享供电的反证一样留在案里。上一回，整个组织为同一件事付出的是一次延期、一个周末和一场向全体与会人的追回；这一回，它的全部成本是走廊里来回的十四分钟。

值得停一秒看清楚，是机器在这一幕里做对的究竟是什么。它没有生成任何东西——拦截那一下，靠的全是别人交来的旧材料：包的声明是装包纪律给的，成员的绑定是各家本体给的，判据是门禁写死的。助手在整个回合里只做了提议和跑腿，连

“拒绝”都不是它的决定，是门禁的。一年的试点里团队慢慢懂了一件反直觉的事：这套结构最值钱的能力从来不是替人写出什么，而是替人拒绝什么——拒绝得有据、有名、有痕。

当年会后不是没想过办法。双人复核提过，总检查单也提过，收口评审的桌上就摆着这两个方案，也当场承认了它们的天花板：前者给“人会疲劳”增加更多会疲劳的人，后者查得出“字段在”、查不出“两份报告说的是不是同一个东西”。两条路都是往纸上加纸。真正的第三条路今天才走通：“**同一基线**”不再是一条值得表扬的纪律，而是**每份证据自带、门禁可核对、装包那一刻就能拒绝的属性——对表不再依赖谁恰好没睡**。墙上第一句话，销账。

那天夜里十一点四十，小蔡还是醒着——不为对表，是老习惯让她翻了一眼记录。冲突在下午就已拦下、处置、归档，审计页上每一步都有名字：谁提议的，哪道门禁拒的，谁确认的，理由是什么。她把手机扣回床头。一年前的那个周三，同样的时刻，她正在打第三个电话。这一次，她在放行前夜睡了个整觉。

旧事故被接住了。可试点里最凶险的一场考试，考的不是旧事故，而是这套结构自己的命——模型一换，这一年搭起来的东西还能剩下什么？

## 20.4 模型换了，什么还活着

通告只有两行：助手的底层模型下周升级；旧版本三十天后停止服务。没有商量的余地，升不升、何时升，都不由这支团队决定。

演示会上，新模型确实惊艳。吴工看完，说了句实在话：“要我说，它现在这么强，直接让它把放行包三百份文件通读一遍、当场问答，比我们养这一堆结构省事。搭了一年的这些类啊边啊，会不会本来就是给旧模型当的拐杖？”

这话不外行。新模型当场把一份补测报告的射程复述得八九不离十，放在半年前想都不敢想。可毛病恰恰在“八九”：没人知道那“一”丢在哪里。散会前小蔡把三个月前问过旧模型的同一个问题重新问了一遍——同一份报告，两代模型给出两个各自流畅、彼此并不重合的“八九”；而同一时刻，门禁对这份报告的回答，前后一字不差。流畅是模型的属性，可查是结构的属性；把放行押在一个版本之间自己会变的属性上，等于把基线交还给运气。

小林提的是另一头：“那就锁版本。旧模型冻结在案，谁也别升。”这也不外行——把工作环境钉死在受控版本上，正是他被“取最新”教育过之后长出的配置管理本能。但模型不是自家的受控工件：它是租来的河水，停用日期写在别人的日历上；何况准入评测明明白白显示新模型翻译得更好。问题从来不是升不升，而是升级那一天，什么必须重调、什么必须一动不动。

升级周的账，两天就摊清了。作废的一侧：围着旧模型打磨的提示词成片失效——旧模型要哄着才肯分条列点，新模型不哄也列，可原来的哄法反而让它啰嗦；几段封装接线要重写；新模型还热情过头，把一份装包提议的措辞写成了“已完成”——审计一圈



对不上执行记录，当场打回。小唐和小蔡把提示词逐条重裁，裁到第二天傍晚，准入评测揪出最后一处暗伤：新模型在转述“未知”时爱补一句自己的猜测，评测按三值纪律判它越界。改完这一处，新模型和任何新上岗的人一样，把整套考卷从头考了一遍，才重新领到工作边界。存活的一侧：本体的每一个类与边，一字未改；全部事实——快照、证据、案例、冲突、决定——原样在账；门禁与查询照常运转，试点一天都没停，因为它们从头到尾不问模型任何问题；连那套考卷本身，也原样考住了新模型。

两侧为什么这么分，说穿了是押的东西不同。提示词押的是模型的习惯——哪种问法能让它分条列点，哪种口吻能让它不抢答；习惯长在那一版模型身上，版本一换，习惯换一套，押注跟着作废。本体押的是世界——批次就是批次，绑定就是绑定，证据包该有哪几个成员，不因为读它的模型聪明了一代就多一份少一份。押在习惯上的，寿命跟着版本走；押在世界上的，寿命跟着世界走。

做应用的人都听过那句丧气话：模型一更新，什么都被吃掉。这个星期用两天的代价把这句话切开了——被吃掉的和留下来的，第一次分得清清楚楚。**提示词与封装是围着模型裁的衣服，模型一换就得重裁；本体、事实、门禁与评测是自己家的骨头——模型更新吃掉的只有衣服，吃不掉骨头。**吴工后来自己补了结案陈词：“拐杖是给旧模型当的，没错。骨头不是。”

骨头经住了换水。正戏还等在前面：RC18 的放行，在新载体上究竟怎么走？

**读图任务：**从顶部主张经中层论证跟随到底部证据卡，找到两张主动举起过期旗的卡，然后沿重开丝带看向右下角留给人的签名框。

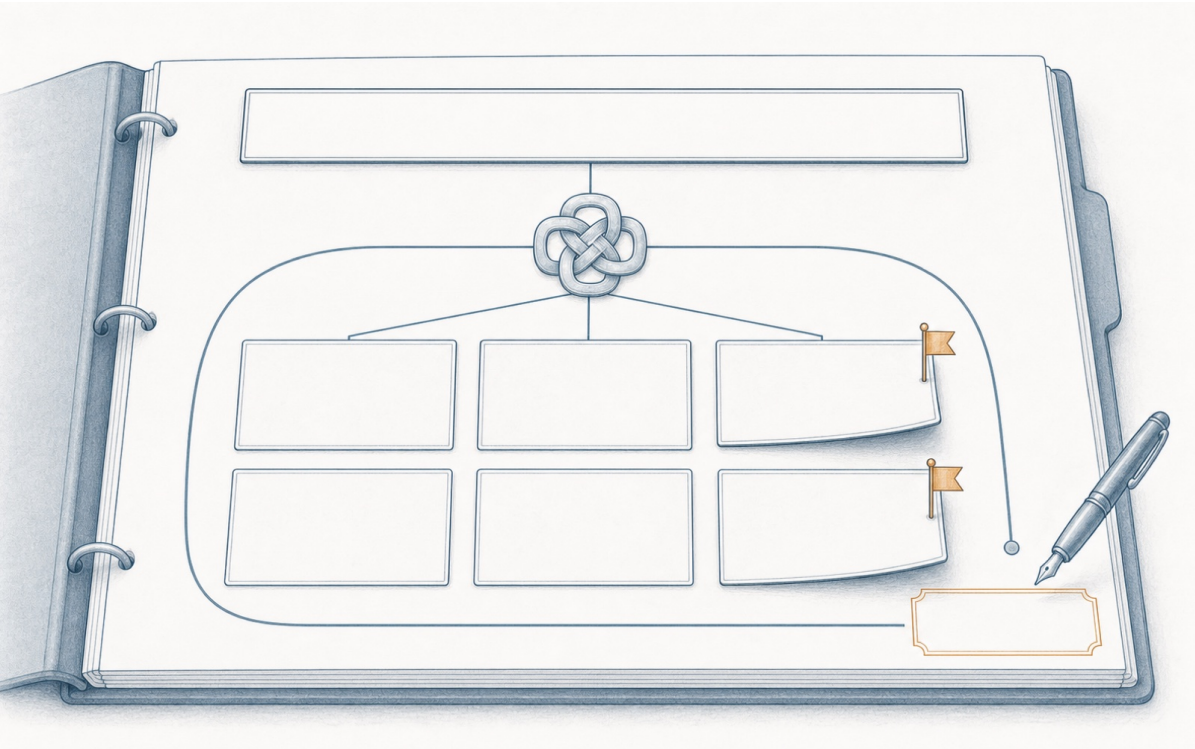


图 20-1  活的保证案例会在对象、配置、证据或假设改变时主动暴露过期和重开条件。自动检查负责举旗，最终承担风险的决定和签名仍然属于人。

**图后判断：**图中过期标记不等于证据错误，签名框也不表示放行已获授权。本图不替代真实案例的反驳、缺口处置和责任人核验。

## 20.5  RC18：活的安全案例第一次开庭

正戏其实在开会之前，就已经演完了一半。

升版落地那天，变更获准的消息经由与第 17 章的显式对齐合同送进案例，案例把它翻译成自己的一条事件，当场举手：三条主张标为过期，论证层七处引用重开，两处经声明理由保留——保留理由本身也成了对象，第 1 章那句“保留是一个判断，不是一个默认”，第一次长进了结构里。哪些重开、哪些保留、缺哪张卡，是查询返回的，不是小林熬夜圈出来的。墙上第二句话——变更影响靠人工圈定——销账。

会前一天，小蔡把案例的状态表调了出来。这一次，六张绿卡的后代第一次自己开口。

问题：此刻 RC18 的案例里，每一路证据各自作证到哪里、状态如何？

```
SELECT ?evidence ?testifiesTo ?scope ?status WHERE {
  rel:Case_RC18 rel:citesEvidence ?evidence .    # 案例到证据的汇总边
  ?evidence rel:testifiesTo ?testifiesTo ;
           rel:scopeNote    ?scope ;
           rel:status       ?status .
}
```

| 证据         | 作证对象          | 射程要点          | 状态 |
|------------|---------------|---------------|----|
| 概念边界分析·修订版 | RC18 相关项边界    | 选定场景内的危害与目标点名 | 在案 |
| 硬件失效率评估    | H3.2 控制器      | 声明剖面与诊断假设之内   | 在案 |
| 软件回归·正式构建  | SW1.8.4 加 C41 | 既定测试集与判据之内    | 在案 |
| 台架定向补测     | RC18 目标配置     | 边界工况重跑，含低温冷起动 | 在案 |
| 冲击振动试验     | 局部机械总成        | 升版不波及，保留理由在案  | 保留 |
| 生产写入检查     | RC18 镜像与批次    | 声明工位与批次内写入正确  | 在案 |

第 1 章里，这张表是陈工一句一句问出来的；收口评审上，是四个专业、两家单位两个下午挂出来的；今天它是一条查询。郑工坐在老位置上，第一次不用翻记录念配置——他的台架卡自己报了家门。开放项也在表外分栏立着，一条不藏：台架异常读数，至今未复现，状态未知，看守在案；首批现场回传未满一个完整高温季，声明为缺口，窗口另计。缺口与未知不再靠自觉罗列，它们是案例的常驻对象。

会好像没什么可开的了。老何先把这层意思说了出来：“都绿了，门禁也都过了，那这会就走个过场呗——签字画押，散会吃饭。”这话在产线上是常识：放行流程电子化这么多年，绿灯放行天经地义。可它恰好是当年那句“都通过了还等什么”的孙子辈——门禁通过是机器建立的第三层绿，授权是只有人才能建立的第四层；让绿灯自动生成签字，等于把残余风险的承担委托给一段没有肩膀的逻辑。反过来也有人劝陈工按老规矩把三百份文件重读一遍再签——也不必：人重读机器已经核对过的东西，是用勤奋冒充责任；人该读的是机器压不掉的那几页——未知要不要继续背着走，缺口的窗口给多长，取舍认不认，代价谁来担。机器把案例算到“可签”，人把名字签成“担责”——这两步，谁也替不了谁。

方工宣读独立评估：建议接受，附条件。他的评估这一次也换了做法——挑战不再是通读之后的观感，而是一份反向查询清单：他专挑案例声称“已闭合”的分支，逐条查

论证的理由链断不断、引用的证据在不在射程内、冲突处置有没有绕过看守。评估的人和拿主意的人依旧分开坐，只是挑战者手里第一次有了撬棍。附的条件写成了三条重开：台架异常读数复现即重开；决定窗口到期即重开；下一轮升版获准即重开——第三条与当年那条不同：当年钉的是点名的一版，这一次钉住任何下一轮，是变更那次演习教会所有人的。然后，是助手的最后一个动作。

这段记法在说什么：助手把它能做的最后一件事做成了一个对象——一份提议；决定对象

另立，属于有权角色。两个对象之间那条边，是整个试点最重要的一条边。

```
# 最后一步：机器提议，人决定（示意记法）
rel:Proposal_RC18 a rel:AgentProposal ;
    rel:proposes          rel:Decision_RC18 ;
    rel:basedOnCaseState rel:CaseState_AtSigning ; # 签署时刻的案例状态
    rel:status            rel:ProposedOnly .        # 提议，不是决定
rel:Decision_RC18 a rel:AuthorizationDecision ;
    rel:acceptedBy rel:Role_ReleaseAuthority ;      # 有权角色，今天是陈工
    rel:withReopen rel:Reopen_AnomalyRecur ,
                    rel:Reopen_WindowExpiry ,
                    rel:Reopen_NextUpgrade ;
    rel:scopeNote " 仅授权本轮放行，不授予永久可信" .
```

陈工签字之前，把桌上的三样东西各按了一下，像老师傅清点工具：案例——大家在谈什么，一查便知；活动记录——世界上真发生了什么，台架和产线说了算；这支笔——接下来由谁承担什么，只有他能落。然后他签了。决定对象引用的是签署那一刻的案例状态：将来任何人回看这个决定，看见的都是陈工当时看见的那份账，不多一页，不少一页。墙上第三句话——安全案例是死文档——销账：从签字的下一刻起，产线在写入新的批次，现场在回传新的工况，下一轮升版排上日程，而案例会跟着标注、跟着举手、跟着把该重开的分支交回人的手上。它活着。

散会时，陈工在白板前站了一会儿。试点开始前写下的那行“RC17，已接受，附条件”还在。他拿起板擦，把它擦掉了：“记得这件事的，如今不止一块白板。”小唐望着投影上还亮着的状态表，忽然说：“我现在知道当年那句‘还在等什么’在等什么了——等的就是这张表，和肯在它下面签名的人。”

会散了，试点结项。可这本书，还欠着三个问题。

## 20.6 谁在指挥，谁在记得，谁有权改写

先把边界说老实，免得把结项说成神话。这一年建起来的十个本体，没有让 RC18 更安全一分——安全是台架的补测、产线的校验、设计的余量挣来的，三元组挣不来；本体做到的，是让“它是否可信”从一句没人能查的口号，变成随时可查、可反驳、也可推翻的问题。门禁看得住声明过的维度，看不住没人声明的维度：真正的意外，向来从没人想到要建模的方向进来，所以完备性声明与人的看守永不下岗。十个本体也仍然是十个——隔壁 ENV-01 的团队来旁听了 RC18 的评审，带走的是问法、结构和纪律，一条业务事实也没带走，也不该带走；他们的世界，要用他们自己的证据重新回答。

然后，三个问题可以收账了。

谁在指挥？是人。一整年里，助手提议过几千次、执行过几百回，每一次都在门禁与审计的圈里；它的最后一个动作，是一份状态写着“仅为提议”的对象。指挥从没有换过手。

谁在记得？从前的答案是：加班的人、抽屉，和某位工程师的私人记忆——郑工的差异说明靠他记得，小林的影响清单靠他记得，深夜的对表靠小蔡恰好没睡。现在的答案是：一份随产品一起活着的结构在记得，人从记忆的载体，退回记忆的看守与主人。这是试点真正交付的东西。

谁有权改写这份记忆？还是有权的人——本体没有没收任何人的权力。但从今往后，改写要留下名字：谁改的、依据什么、经过哪道门禁、谁授权的，全部在案。有权的人仍然可以改写记忆，却再也不能无痕地改写记忆；而写下每一条的人，会一直留在他写下的那一行上。

在 Semantica 的炼化状态里，这种区别被刻意拆开。**committed** 说明一份候选记忆已经不可变地留下；**regression\_passed** 说明旧问题与新反例没有被静默遗忘；**release\_complete** 说明 package、能力、receipt、来源和权利的技术条件闭合；只有另一次有权授权才能把它 **promoted** 到行业 registry。即使如此，产品放行、书稿修改、代码 push 和对外 **published** 仍是各自独立的人的决定。六个生命周期状态不是六盏绿灯，更不是一支自动签字的笔；它们只是让签字者清楚自己正依据哪一版记忆、哪些证据和哪些未决项。

还有一层容易被误当作“流程配置”的保证边界：候选第一次登记时使用的 binding，是它以后不可扩大的生命周期能力上限。若最初只允许走到 **proposed**，后来即使换上一份更宽的 binding，也不能把同一条候选记录救成 **committed**；否则控制面就能在评审之后悄悄改写当时的权限条件。正确做法是从首次 proposal 起声明完整的预期上限，再让每一次新的 task 只请求眼前动作，让 commit 与 promotion 分别引用新的有权决定。能力上限、当前请求和人的授权三者分离，既不会让“有能力”自动变成“已获准”，也不会让一次短任务永久截断一条本来准备接受完整保证审查的候选。

同样，来源锚点的哈希相等只证明“当时引用的字节仍是那些字节”，不能证明 successor candidate 的新结论已经被本书吸收。每当闭包、receipt 或授权语义改变，影响



记录都要重新区分：正文原已覆盖的原则、新增的通用保证方法、只属于项目案例的数字，以及 ISO 推演是否真的改变。没有这份新的内容绑定说明，旧 book-impact 记录只能作为历史来源证据，不能充当“书稿已同步”的绿灯。

导读的期票，最后兑现一次。合上全书之前，做三件事：翻出你手头最近的安全案例，找到主张—论证—证据里“论证”那一层，问它上一次产品变更之后动过没有——没动过，它多半已经死了；抽出放行包里任意两份报告，看有没有“绑定在哪个快照”这一行——没有这一行，下一个深夜对表的人就还得是你；再找到上一次授权决定，看它引用的是签署时刻的案例状态，还有一句“以当时材料为准”。三件事做完，你就知道自己离那份活的案例还有几步。

结项一周后，郑工来找小蔡。他递了退休报告，下个月交台架。两个人一起做的最后一件事，是移交那条重开条件的看守：屏幕上，看守角色的占位从他换成他带了三年的年轻人。系统请郑工本人确认移交——哪怕是离开，放下看守也是一个只有他有权作的决定。他按了确认，然后盯着那个条目看了很久。看守人一栏换了名字；第一行没有换——那里仍然记着，当年是谁把那页读数从抽屉里拿出来，放到了所有人看得见的地方。

“我以前最怕的，”郑工说，“是我走了，这一页跟着我走。现在它留下了，名字也留下了。够了。”

小蔡送他到楼下。电梯门合上之前，郑工朝她挥了挥手。她回到工位，那条重开条件安安静静地亮在屏幕上：看守换了人，写下它的人继续被看见。

一件产品值得被相信，从来不是因为谁说了一句“放心”——而是因为有一群人，肯把自己的名字一行一行留在它活着的记忆里。

**本章注记：**本章的试点情节、人物、事故经过与全部产品数字（候选代号、软硬件与标定版本、时间跨度、台数等）均为合成教学材料，不对应任何真实企业、真实产品或真实个人。正文中的片段为教学示意，不构成完整形式化定义；唯一可执行版本是 `semantica.chapter_packages.vol2.ch20`，当前为 `partial`、`release blocked`。本章对安全案例与发布保证活动的转述为自然语言概括，精确要求以标准原文为准；本章不构成对任何实际系统的判定或放行结论。

# 附录 A 半导体应用指南：把 Part 11 读成一条证据链

Semantica 边界：本附录保留 Part 11 的可读解释、来源坐标叙述、算例与建模蓝图；它不在书内发布本体、查询、Shape 或 runner。现有 `semantica.chapter_packages.vol2.ch06`、`vol2.ch16`、`vol2.ch18` 分别承接度量、测量证据和依赖主题的章级机器合同，但当前 Semantica registry 尚无“Part 11 全附录”的独立 released package。本附录中的形式化片段因此只是书面蓝图；若要执行，必须先作为显式 Semantica package delta 注册、补齐 CQ/正反例/oracle/来源与 release 证据，未注册前一律视为 unsupported/blocked，不能回退到旧本地文件。

**导读。**正文各章把 ISO 26262 的生命周期走了一遍，芯片在其中大多以“某个元件、某个失效率”的面目一闪而过。本附录面向想深入半导体功能安全的读者，把 ISO 26262-11:2018（下称 Part 11）组织成一条“对象划分 → 基础失效率 → 相关失效分析 → 故障注入 → 具体技术案例”的证据链。读完本附录，你应当能：1. 拿到一颗芯片或一个 IP，说清它在 component/part/subpart 层次里的位置、失效模式与故障模型如何挂接，以及 IP 供需双方各自欠对方什么工作产物；2. 面对一个“XX FIT”的基础失效率，追问并核对它的来源、任务剖面、置信水平与适用边界，手算复核 Part 11 自带的示例；3. 用 DFI 七分类和 B1-B12 工作流组织一次半导体相关失效分析，并说清哪些结论可以量化、哪些只能定性留证。

**来源性质声明（先读）：**Part 11 全卷为资料性 (informative)。其 Scope 明说“This document has an informative character only. It contains possible interpretations of other parts of ISO 26262 with respect to semiconductor development” (Part 11 物理页 p9)。也就是说，本附录中的 Part 11 内容是对 ISO 26262 其他分册在半导体开发中的可能解释，不构成新的规范性要求，也不能替代任何适用的 shall。凡本附录说“可以这样算/这样分类”，Part 11 锚点只支撑这一解释的来源归属；需要形成项目义务时，须逐项回到适用的规范性条款（主要位于 Parts 2-9，术语还需回看 Part 1），或显式标成 BookHousePolicy/ProjectSpecificStrategy，不能靠 Part 11 单独升级。

**标签边界：**带 ISO 条款/表格锚点的陈述才可作为“标准事实”候选，且还要区分 shall、NOTE 与 EXAMPLE；“本书归纳/本书建议”是**作者综合**；TeachingExample 是合成教学数据；BookHousePolicy 是本书的证据接收或知识治理规则；ProjectSpecificStrategy 是具体项目的选择。五者不得互相代替。

**案例边界：**本附录引用的数值多数是 Part 11 自带的计算示例；关键数值与 A.8 的来源坐标曾用两种同源提取视图定位，并回看原 PDF。后文简写的“两源核对”均只表示这两种提取视图一致，不表示存在两个独立来源。本附录自行补充的算例一律标注“教学示例”(TeachingExample)，不代表任何真实器件的失效率或量产结论。页码为 1-based 物理页。

**与全书二十章的衔接：**本书前十章现为叙事重写，方法表语义、独立性等级记号、FMEDA 与失效分类等器具性内容已集中由附录承载；正文各章只提供活动与主题的工程语境。本附录因此按自足深讲编写——凡指向“第 X 章”处均指主题语境，不意味着正文有逐表逐条的展开。

## A.1 导读：芯片不是一只黑盒

先讲一个会真实发生的场景。某整车项目的系统安全团队按第 6 章讲过的流程做硬件度量：技术安全需求分配到了电子控制单元，FMEDA——第 6 章展开过的失效模式、影响及诊断分析——排到了主控芯片这一行。芯片是外购的：一颗集成了 CPU、存储器、模数转换器和若干通信外设的车规微控制器，其中还嵌着一块第三方授权的处理器核。团队手里只有一份功能数据手册：引脚定义、电气参数、外设清单，一应俱全；但要填进 FMEDA 的三样东西——失效率怎么分到内部模块、每个模块坏了算哪类故障、片内自检覆盖到什么程度——手册上一个字都没有。芯片对他们来说是一只黑盒，而定量分析恰恰要求把盒子打开。

这个困境不是哪家公司管理不善，而是半导体供应链的结构性事实。整车厂不设计芯片，芯片厂不了解整车安全目标，芯片里还嵌着第三方的设计模块；分析所需的信息分散在链条各处，谁也无法独自完成论证。Part 11 就是为这个断层写的说明书。它把断层拆成四个可以逐个处理的问题：

**第一，划分粒度。**芯片内部要拆到多细才能做分析？拆浅了失效模式挂不上诊断，拆深了几百万门根本算不动。Part 11 §4.2 给出 component→part→subpart→elementary subpart 的层次词汇，并明说粒度取决于安全概念与所用的安全机制（p10）——这是 A.2 节的主题。

**第二，使用假设。**芯片厂做分析时并不知道你的系统长什么样，只能假设“用户会这样用我”。这类假设有个专名：先记白话版——“我按你会给我配软件驱动、会把我的报错引脚接到监控器来设计”这类前提；正式名是**使用假设**（Assumptions of Use, AoU）。Part 11 §4.1.1 与 §4.4 资料性地描述了把 AoU 写出并在更高集成层级核对的做法（p10、

p13)；当元素按 SEooC 裁剪时，真正的 shall 在 Part 2 §6.4.5.7：开发须基于预期使用与上下文假设，集成进目标应用时须验证这些假设。假设不核对，芯片级分析的漂亮数字到了系统级就是空中楼阁——A.2 节的 ECC 软件驱动反例会演示这一点。

**第三，证据可见性。**IP 供应商未必愿意、也未必被允许交出设计细节；极端情况是加密网表或混淆源码的“黑盒 IP”。看不见内部时，谁来估失效率、谁来定义外部安全机制？Part 11 §4.5.5 的资料性回答是：用**开发接口协议**——第 10 章深讲过的 DIA (Development Interface Agreement)，明确责任、可交付证据与接受边界 (p22–23)。DIA 是 ISO 26262 的分布式开发接口工件；它在某个司法辖区是否同时构成法律合同，不由 Part 11 这几个示例决定。

**第四，责任划分。**半导体开发者在供应链里既是供应商又是客户；确认措施、生产控制、现场监控这些正文 ch03/ch09 讲过的义务，落到芯片上要怎么裁剪？A.6 节沿 §4.9–§4.12 走一遍。

本附录的路线与正文的回指关系如下表：

| 节                    | Part 11 素材 | 物理页     | 回指正文                         |
|----------------------|------------|---------|------------------------------|
| A.2 部件划分、故障模型与 IP 边界 | §4.1–§4.5  | p10–23  | 第 2 章概念层次、第 10 章 SEooC/DIA   |
| A.3 基础失效率            | §4.6       | p23–49  | 第 6 章 λ 分类与 FMEDA 输入链        |
| A.4 半导体 DFA          | §4.7       | p49–63  | 第 8 章 DFA 与独立性               |
| A.5 故障注入             | §4.8       | p63–65  | 第 5/6 章验证计划与覆盖率主张            |
| A.6 生产、分布式开发与确认      | §4.9–§4.12 | p65–68  | 第 9 章生产、第 10 章 DIA、第 3 章确认措施 |
| A.7 五类技术案例           | §5.1–§5.5  | p68–140 | 各技术章                         |
| A.8 本体化实践            | 全部锚点       | —       | 第 6 章数值诚实、第 8 章依赖链           |

怎么读一卷“资料性”标准，也值得先说破。informative 不等于没有工程价值：Part 11 集中给出了诸多把 Parts 2–9 应用到半导体开发的解释与示例，在相关评审、审计或供应链沟通中可作为参考；但它也不等于照抄即合规。原文常用“can be used”“is recommended”“a rationale is provided”这类表达，选择与论证仍要回到具体上下文。本书的阅读策略（作者综合）是：把方法当作**候选**，把 NOTE 当作**解释与边界信息**，把数值示例当作**练习题的参考答案**而不是项目数据。本附录因而说“Part 11 建议/示例”，不说“Part 11 要求”。

一句话立骨：**正文教你要什么证据，本附录教你这些证据在硅片上长什么样。**



## A.2 半导体部件划分、故障模型与 IP 边界

### A.2.1 四层划分：分析粒度是安全概念的函数

第 2 章讲过的 element 层次词汇——item/system/component/part——在芯片内部继续向下延伸。Part 11 §4.2 按 Part 1 §3.21 的定义给出示范：整颗半导体是 **component**；第二层级（例如一个 CPU）是 **part**；再往下（例如 CPU 的寄存器组）是 **subpart**；直到**基本子部件**（elementary subpart，例如寄存器组里的单个寄存器及其相关逻辑，或一个触发器、一只模拟晶体管）（p10，Figure 2 见 p11）。

关键不在名词，而在紧随其后的那条 NOTE：拆到哪一层、“基本子部件”取什么颗粒，取决于安全概念、分析所处阶段与所用的安全机制（p10）。这句话是全附录反复回响的主旋律。两个对照例子（§4.3.2，p12）：同一个 CPU，若安全机制是硬件锁步——两个核逐拍比对输出——对该机制所覆盖的 CPU 内部故障，失效模式可以在“CPU 功能整体”这一层定义；这不表示锁步能发现所有内部或共因故障。若安全机制换成软件自检程序，为得到可辩护的诊断覆盖率通常需要拆细，因为自检对不同内部结构的覆盖率不同。机制越“包圆”，划分可以越粗；机制越“挑食”，可信分析所需的划分通常越细。这是 Part 11 的分析指南，不是独立的粒度 shall。

### A.2.2 故障模型、失效模式与“无缝隙”检查

第 2 章讲过的 fault→error→failure 因果链在芯片语境里落地为两个衔接的概念。**故障模型**（fault model）是物理故障的抽象表示（§4.3.1，p11）：晶体管栅氧击穿、金属互连断裂这些物理事件数不胜数，工程上把它们归并为“节点恒为 0/1（stuck-at）”“开路”“桥接”这类可枚举、可注入、可统计的模型。**失效模式**（failure mode）则描述功能层面“怎么坏”：Part 11 建议用关键词法——错误的程序流、数据损坏、访问了不该访问的位置、死锁、活锁、错误指令执行（§4.3.2 EXAMPLE 4，p12）。

两者的挂接关系直接决定诊断覆盖率怎么算。§4.3.1 的示例值得手算一遍（p12）：若某失效模式由 stuck-at 故障贡献 X%、由短路贡献 Y%，而安全机制只覆盖 stuck-at 且覆盖率为 Z%，则可主张的诊断覆盖率是  $X\% \times Z\%$ 。代入具体数（教学示例）：stuck-at 占 70%、短路占 30%、机制对 stuck-at 覆盖 90%，则总覆盖率是  $0.7 \times 0.9 = 63\%$ ，而不是 90%——短路那 30% 完全在机制视野之外。**当局部覆盖率只针对某个故障模型子集时，总体主张必须按该子集对相关失效模式的贡献加权；不能把局部 Z% 直接写成总体 Z%。**这是本书审阅半导体安全手册时采用的一把尺。

§4.3.2 还给出了一项“无缝隙”完整性指南（p12）：用证据支撑失效模式与电路实现的关联，检查**每个失效模式都分配到某个 part/subpart，每个相关的 part/subpart 至少有一个失效模式**。目的写得很直白：确保电路实现与失效模式清单之间没有缝隙。这与第 6 章 FMEDA 的完整性纪律同构——那里防“漏元件”，这里防“漏模块”。本书把这



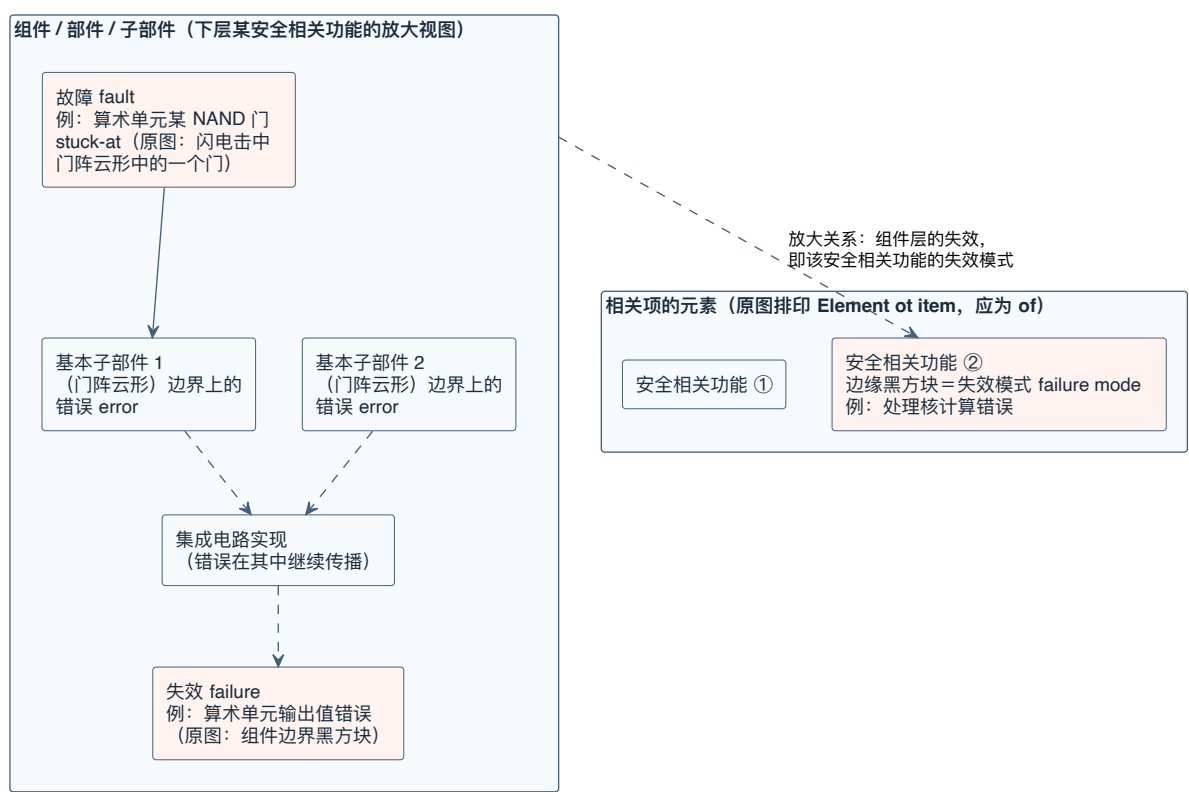


图 A-1 硬件故障与失效模式的关系（重绘自 Part 11 Figure 3）——基本子部件内的物理故障在子部件边界成为错误、经集成电路实现传播为组件层失效；同一失效在相关项元素层即某安全相关功能的失效模式，两层由放大关系相连。

项资料性指南采纳为审稿/接收策略时，规则依据应标成 **BookHousePolicy**，不能标成 **DirectISORequirement**。

失效率往失效模式上摊时 (§4.3.3, p12)，精度同样跟着安全机制走：锁步 CPU 不需要精细分布，软件自检 CPU 需要。没有数据支撑分布时，Part 11 给出两条资料性做法——均匀分布，或给出带论证的专家判断——并在 NOTE 中建议做**敏感性分析**，看分布假设对诊断覆盖率与定量结果的影响有多大。退路不是免费的：均匀分布省了测量，就得用敏感性分析补充对结论稳健性的认识；若项目把它设为放行条件，应另标 **BookHousePolicy**。

### A.2.3 从芯片级到系统级：自底向上与假设核销

§4.4 回答“芯片级细粒度分析如何对接系统级粗粒度分析”(p13)。方向一：把芯片的详细失效模式**自底向上**归并成系统分析需要的高层失效模式 (Figure 4)，自顶向下的 FTA 与自底向上的 FMEA 在中间会师；从低抽象层出发的好处是失效分布可以定量而非拍脑袋 (NOTE 2)。方向二：诊断覆盖率可以在更高层级**被改善**——芯片里一个没有任何硬件安全机制的模数转换器，孤立看覆盖率为零；到了系统级它被纳入闭环，软件一致性检查能发现它的故障，覆盖率随之上升 (EXAMPLE 1)。

方向三是最容易翻车的：芯片级覆盖率可能建立在特定使用假设之上。§4.4 的 EXAMPLE 2 (p13) 是个教科书级反例：某芯片的存储器带 ECC，单比特错误可纠正并向 CPU 报告；芯片级分析**假设**用户会实现一个软件驱动来处理这个报告事件。系统集成时，出于性能考虑这个驱动没做——假设失效，芯片被改配置为把纠错标志直接甩到片外。此时芯片安全手册里那一行漂亮的覆盖率数字已经不再成立。**AoU 不是参考资料，是待核销的债务清单**：在 SEooC 场景，逐条验证的规范依据是 Part 2 §6.4.5.7 b)；本书再用 **BookHousePolicy** 把核销结果固定成“满足/不满足/替代措施”三态，避免只继承结论不继承前提。

### A.2.4 IP：四条路径、两种类别、一套工作产物

**IP** 在 Part 11 里指打算作为 part 或 component 集成进设计的可复用逻辑或物理设计单元；提供方叫 IP 供应商，集成方叫 IP 集成者——两者可以是不同公司，也可以是同一公司的两个部门 (§4.5.1.1, p14)。按交付形态分两类 (Table 1, p15)：**硬 IP** (物理表示：完整版图，如 ADC 宏、PLL 宏) 与**软 IP** (模型表示：Verilog/VHDL 或晶体管级原理图，还要经过综合、布局布线才成为硅上实现)。这个区分有安全后果，A.2.5 会用到：对尚未确定最终工艺与物理实现的软 IP，失效率与分布只能先作为基于实现假设的参考估计，并由集成者对具体实现重新核对 (§4.5.4.5 NOTE 1, p20)。

基于 ISO 26262 的既有机制，Part 11 识别出 IP 进入安全设计的**四条路径** (§4.5.3, Figure 5, p14、p17–19)：

| 路径                   | 依据                | 适用场景               | 集成者的核对重点          |
|----------------------|-------------------|--------------------|-------------------|
| SEooC 开发             | Part 10 (ch10 深讲) | IP 面向一类假想上下文开发     | 逐条验证 AoU 与实际上下文一致 |
| 在上下文中开发              | Part 2 §6.4.5.1   | 供需双方同项目协作，安全需求已知   | 常规分布式开发管理         |
| 硬件元素评估               | Part 8 Clause 13  | 无 SEooC/在场信息的既有 IP | 按评估程序补置信          |
| 在用验证 (proven in use) | Part 8 Clause 14  | 有大规模现场历史           | 现场监控与配置一致性论证      |

第四条路的门槛被 Part 11 特意泼了冷水 (§4.5.3.5, p19)：IP 的现场反馈通常很有限、配置又千差万别，Part 8 §14.4.5.3 要求的有效现场监控计划很难满足——“用了很多年”不等于“在用验证成立”。

按安全需求分配情况，Part 11 §4.5.2 资料性地区分两种 IP (p15)：**未分配安全需求**的 IP 通常不因 Part 11 自身增加事项；若安全分析识别出它会影响安全相关元素，适用的 Part 9 Clause 6/7 要求仍需处理。**分配了安全需求**的 IP 则根据具体上下文裁剪并适用 Parts 2/4/5/8/9 的相应要求，再按“是否内建安全机制”细分 (Figure 6, p16)。内建机制的 IP（例如带内建总线监督器的总线 fabric、带过压/欠压监控和自诊断的电压调节器）可以主张对特定失效模式的覆盖；无机制的 IP（同样的 fabric/调节器裸奔版）通常只能给出失效模式清单与分布，把覆盖论证留给集成层。类别划分来自 Part 11 指南，具体义务来自被判定适用的 Parts 2–9 条款。

还有两类“生成物 IP”值得单独说。**可配置 IP**：逻辑设计形态的 IP 常带配置选项——总线位宽、存储容量、中断数量、是否例化故障检测机制——配置由集成者指定 (§4.5.1.2 NOTE 1, p15)。配置不是无害的开关：§4.5.4.5 资料性地说明，可配置 IP 的安全分析可描述配置选项对失效模式分布的影响，其 NOTE 3 还提到对安全机制实现与诊断覆盖率的影响分析 (p20)；§4.5.3.2 又说明 SEooC IP 供应商提供**哪些配置已被测试与验证活动覆盖**的信息 (p19)。若所选组合不在该信息范围内，本书的证据接收策略 (BookHousePolicy) 把覆盖率主张视为待重新论证，而不是自动继承。顺带一条 NOTE：IP 配置不同于软件的配置数据，Part 6 Annex C 不直接适用于 IP (§4.5.3.2 NOTE 1, p19)。**工具生成的 IP**（存储编译器、C 转 HDL 编译器、片上网络生成器）：对生成工具的信任可按 Part 8 Clause 11 的软件工具置信度框架处理，并结合对生成 IP 的验证程度裁剪**工具鉴定**活动；生成结果本身的正确性验证则由适用的供需方分工，例如写入 DIA (§4.5.1.2 NOTE 2, p15)。

SEooC 路径还有一个“能力不匹配”的局面 (§4.5.3.2, p18)：IP 的 SEooC ASIL 能力（按 Part 1 §3.2）低于集成者的 ASIL 要求。Part 11 资料性地给出两条候选路径：集成者在 IP 外部追加安全机制、并补充系统性失效避免措施；或者按 Part 9 Clause 5 做 ASIL 分解——但要论证冗余安全需求及充分独立性，并把集成 IP 的系统性失效避

免与控制方法一并纳入考虑。这与第 8 章的边界一致：缺少独立性证据时，不能据此主张 ASIL 分解成立。

对应地，§4.5.4.10 给出按类别裁剪工作产物的资料性解释（p22）：对无内建机制的 IP，安全分析信息可聚焦失效模式分布——没有机制就谈不上由该 IP 自身提供硬件度量估算；集成文档集可缩减为集成环境假设与接口说明；相关失效分析是否需要则取决于共存与依赖。**类别影响证据范围，但不由 Part 11 单独创造或免除义务**——审阅 IP 交付包时先判类别，再回到适用的 Parts 2-9 条款与 DIA 对清单。

Part 11 给出的 IP 工作产物指南（§4.5.4，p19-22）包括：安全计划、分配到 IP 的硬件安全需求、设计验证与验证评审、安全分析报告（定性给失效模式，定量给失效率与分布估计，配置类 IP 还可分析配置选项对分布与机制覆盖的影响）、相关失效分析、确认措施结果（安全计划/安全分析/安全档案完整性的确认评审、审核与评估报告——回指第 3 章的确认措施框架）、DIA，以及**集成文档集**。§4.5.4.9 说明这些信息可合并为一册**安全手册**（Safety Manual）或安全应用笔记，内容包括：生命周期裁剪说明、AoU（假定安全状态、最大故障处理时间与多点故障检测间隔假设、集成环境与接口假设、推荐配置）、安全架构描述（故障检测/控制/报告/自检/恢复机制及配置参数影响）、支撑安全机制所需的硬件-软件接口、片上故障软件测试例程规格、安全分析结果与确认措施说明（p21-22）。两条资料性 NOTE 值得画线：基础失效率**只能作为参考值**提供，因为它取决于 IP 在集成电路中的实际实现工艺与使用条件，NOTE 4 将按实际用例重算的责任指向 IP 集成者（§4.5.2，p17）；NOTE 1 则说明 IP 安全机制需求应以可追溯到集成者需求的方式表达（§4.5.4.9，p22）。这两句是 Part 11 的指南，不是新增 shall。

### A.2.5 黑盒 IP：看不见内部时，责任写进 DIA

有时集成者被要求集成一个内容不披露的 IP：客户的专有通信接口逻辑、竞争对手的模块（为了多源供货协议）。交付形态可能是预硬化版图、加密网表或混淆 RTL（§4.5.5，p22）。此时，基于内部结构的分析会受限，但外部测试、集成指南与外部安全机制并未因此失效。Part 11 的资料性对策是通过 DIA **明确 IP 供应商、IP 集成者与集成者客户之间的责任划分与证据交换**（p23）。DIA 可以约定：由客户负责评估并接受黑盒 IP 在安全上下文中的适用性（EXAMPLE 3）；由供应商提供含验证测试集的集成指南（EXAMPLE 4）；当 IP 不是按 ISO 26262 开发（例如按 IEC 61508）时，由谁负责接受可用证据（EXAMPLE 5 与 NOTE 2 的差距分析）。基础失效率同理：集成者不一定拥有足够数据估算黑盒 IP 的失效率，Part 11 建议在 DIA 中识别估算责任（p23）；若需要外部安全机制而集成者信息不足，也可在 DIA 中约定由谁给出相应需求。DIA 相关项目义务的规范性来源是适用的 Part 8 Clause 5，而不是 Part 11 的示例措辞。

一句收口（作者综合）：**看得见的 IP 用内部证据集成，看不见的 IP 则把证据缺口与责任边界写清**。本书的 BookHousePolicy 不允许黑盒状态把未分配责任默认成“无人处理”。

## A.3 基础失效率：来源、假设与计算边界

### A.3.1 基础失效率是什么，不是什么

第 6 章的 FMEDA 输入链里，每一行的起点都是一个  $\lambda$ 。那个  $\lambda$  的学名在 Part 11 里叫**基础失效率**（base failure rate，也叫 raw failure rate）：元件在**没有任何安全机制干预之前**的固有随机硬件失效率，是 Part 5 定量分析与度量计算的首要输入（§4.6.1.1, p23）。单位沿用第 6 章讲过的 FIT：1 FIT =  $10^{-9}$  次/小时。

三条边界先划清楚。**其一，只算随机，不算系统性**（§4.6.1.3, p24）。多数可靠性预计技术不区分两者，结果可能因掺入系统性因素而过度保守；有些模型甚至内置“过程质量因子”（如 FIDES 的  $\pi_{pm}$ 、 $\pi_{process}$ ），这些因子对应系统能力，在 ISO 26262 语境下要剥离——A.3.4 的 FIDES 示例正是把它们置 1。**其二，不与诊断混算**（§4.6.1.4, p24）。一个反直觉的例子：带单纠双检 ECC 的 SRAM，厂商报出的平均无故障时间（MTTF）通常不把“可纠正错误”算作失效——这等于把基础失效率与诊断效果搅在了一起，而 Part 5 的度量计算要求两者分开： $\lambda$  是  $\lambda$ ，覆盖率是覆盖率，先分后乘。**其三，PMHF 目标值不是可靠性预计输入**（§4.6.1.2, p24）。第 6 章讲过的 PMHF 随机硬件失效概率度量，其量化目标值按 Part 5 §9.4.2.2 NOTE 1 只有相对意义——用于新旧设计对比与设计引导——不能拿去“倒推”元件该有多可靠。

还有一条埋得深但杀伤力大的假设：**常数失效率**（§4.6.1.5, p25）。标准化模型普遍采用“浴盆曲线”简化——早夭段已被供应商筛选剪除、磨损段（电迁移、时变介质击穿、热载流子、负偏压温度不稳定性）在任务寿命内可忽略，只剩平底段的常数  $\lambda$ 。若可靠性模型给出的分布不满足常数假设，Part 5 的架构度量算法就不兼容；出路是取分布的最大值做保守常数近似，或限制产品运行寿命使常数近似成立（EXAMPLE 1/2）。NOTE 1 提醒：如果指数模型下产品寿命内会走到浴盆尽头并超出失效率目标，那是一个**系统性问题**，其可接受性不在 Part 5 Clause 8/9 的硬件度量中评价，而要单独评估；原文举的一种证据是按 AEC-Q100 完成的集成电路**鉴定（qualification）**结果（p25）。更准确地说，Clause 8/9 度量使用常数失效率近似；早期失效与磨损机理还需在度量之外另行评估，不能笼统地说成“交给认证就结束”。

### A.3.2 四类来源与配套假设

估基础失效率的技术分四族（§4.6.1.6, p26）：



1. **实验测试**：高温工作寿命试验（HTOL/TBOL/ELT）、工艺可靠性测试芯片与片上测试结构、辐射源暴露的软错误测试（JESD89 给方法）、筛选加速试验的收敛特性；
2. **现场数据**：退货失效品分析折算，A.3.5 详述；
3. **工业数据手册**：IEC 61709、SN 29500、FIDES Guide、以及“电子元件可靠性预计模型”（前 IEC TR 62380）——注意 NOTE 3 的方向感：实际达到的失效率**预期**低于手册推出的值，手册天生偏保守；
4. **路线图投影**：IRDS/ITRS 对各工艺代软错误率的投影值，可作首次估计，待工艺实测数据到位后再精化。

不同技术对失效机理的假设不同，**未经机理与应力口径对齐的两个失效率不能直接比较** (§4.6.1.1, p23–24)——同一颗芯片用两种手册算出不同结果，可能是各自计入的机理集合与应力假设不同，不能先验地判定某一方错误。调和 (harmonization) 的做法原文给了方向：考虑同一组失效机理与同一组应力来源 (EXAMPLE 1, p24)。本书将它操作化为 **BookHousePolicy**：先列出各来源计入的机理清单（氧化层击穿？电迁移？焊点疲劳？软错误？），再对齐应力假设（结温、循环幅度、湿度）；只比较口径可对齐的部分，其余差异要么补测，要么显式声明为暂不可比。Part 11 还建议参考 JEDEC、IRDS、SEMATECH/ISMI 等行业组织的出版物获取机理级认识 (p24)。

正因来源五花八门，§4.6.1.7 给出一份**假设文档化指南** (p26–27)：随失效率说明选用的方法（手册还是现场数据）、假定的任务剖面、数据的置信水平、施加过的缩放或降额、非工作时间与焊点如何计入、现场数据用的是威布尔还是指数模型。这份清单是集成者在元素或相关项层面评价、比较乃至**调和**不同供应商失效率的重要抓手。本书把“缺少这份清单就不接受 FIT 值”设为 **BookHousePolicy**；这是证据接收策略，不是 Part 11 产生的 shall。把它类比成第 6 章的说法：**没有假设清单的 FIT 值，等于没有量纲的数字**。

瞬态故障的量化有专门指南 (§4.6.1.8, p27–28)。由内外部  $\alpha$ 、 $\beta$ 、中子、 $\gamma$  辐射引发的软错误属于随机硬件失效，可用测量数据支撑的概率方法量化；对电磁干扰或串扰引发的瞬态，Part 11 建议不走随机硬件量化，而按主要为系统性成因处理。它也建议把瞬态与永久故障分开分析，并逐类基本子部件（触发器、锁存器、存储单元、模拟器件）排查软错误敏感性；封装材料也参与其中——低  $\alpha$  (LA) /超低  $\alpha$  (ULA) 封装材料能改变  $\alpha$  粒子致错率 (EXAMPLE 1)。三条防“注水”提醒同样属于资料性解释：不要只按整车运行时间摊薄软错误基础失效率 (EXAMPLE 2)；不要预先用架构脆弱性因子 (AVF) 或 ECC 效果降额——脆弱性因子属于安全故障占比的账，在 5.1.7.2 里算；若交付的是降额后软错误率，应在安全手册中披露降额因子 (NOTE 4–6)。本书将这三项设为数据接收 **BookHousePolicy**，才使用“不得混账/必须披露”的门禁语气。

封装的账目归属要小心 (§4.6.1.9, p28)：芯片厂的失效率覆盖裸片、封装外壳与连接点（引脚），而**引脚到电路板的焊点**通常算板级失效、由系统集成者在系统或元素层分析。三种手册的口径不一样：前 IEC TR 62380 模型的  $\lambda_{\text{package}}$  把焊点也包了进去；SN 29500-2 的元件失效率含裸片与封装但不含焊点（焊点另见 SN 29500-5）；FIDES 则把外壳与焊点分开给。**跨手册比较前，先对齐“封装”一词的边界。**

### A.3.3 手册路线之一：前 IEC TR 62380 模型的完整走查

Part 11 §4.6.2.1.1 以前 IEC TR 62380 为底本给出一套完整模型 (p28-38)： $\lambda =$  裸片项 + 封装项 + 过应力项。裸片项的参数：每晶体管失效率  $\lambda_1$ （按电路族查表）、工艺掌握度失效率  $\lambda_2$ （整片一个值）、晶体管数  $N$ 、工艺年代降额指数  $\alpha$ （与参考年 1998 的年差）、工作/储存时间比  $\tau_i/\tau_{\text{on}}/\tau_{\text{off}}$ 、结温应力因子  $(\pi_t)_i$ ；封装项的参数：热膨胀失配因子  $\pi_\alpha$ （板材  $\alpha_s$  与封装  $\alpha_c$  之差）、年热循环数与幅度  $(n_i, \Delta T_i)$ 、封装基础失效率  $\lambda_3$ （按引脚数  $S$  或封装对角线  $D$  查表）；过应力项：接口暴露因子  $\pi_I$  与环境过应力率  $\lambda_{\text{EOS}}$  (p29)。

**裸片示例手算** (Table 2, p34；数值两源核对一致)。一颗 CMOS 微控制器：5 万门 CPU（每门 4 管， $N = 200\,000$ ）与 16 kB SRAM（低功耗 SRAM 每位 6 管， $N = 786\,432$ ），功耗 0.5 W，144 脚四方扁平封装自然对流散热，“电机控制”温度剖面下结温升  $\Delta T_j = 26.27\,^\circ\text{C}$ ，激活能 0.3 eV，温度降额因子算得 0.17。CPU 行： $\lambda_1 = 3.4 \times 10^{-6}$  FIT/管， $\lambda_1 \cdot N = 0.68$  FIT；工艺年代项  $e^{(-0.35 \times 10)} \approx 0.030$ ，得 0.021 FIT；加上按晶体管数加权的  $\lambda_2$  份额  $(200\,000/986\,432) \times 1.7 \approx 0.345$  FIT，乘降额 0.17  $\approx$  **0.06 FIT**。SRAM 行： $1.7 \times 10^{-7} \times 786\,432 \approx 0.134$ ， $\times 0.030 \approx 0.004$ ；加权  $\lambda_2$  份额  $(786\,432/986\,432) \times 8.8 \approx 7.02$ ，乘 0.17  $\approx$  **1.18 FIT**。合计裸片失效率 **1.25 FIT**——与表中一致。读表提醒：Table 2 的“Base failure rate”列印的是  $\lambda_1$  项加**未加权**的整值  $\lambda_2$  (1.73/8.80)，而“Effective”列按加权公式算，两列口径不同，抄表时容易踩坑。若嫌加权麻烦，Table 5 (p36) 给了粗粒度算法：整片按同一电路族计算， $\lambda_1 \cdot N \cdot e^{(-3.5)} \approx 0.10$ ，加  $\lambda_2 = 1.7$  得 1.80，乘 0.17 得 **0.31 FIT**——在该示例中比逐元素算保守。混合信号的对应处理见 Table 3 (p34)： $\lambda_2$  取各族**最大值**做保守合并（示例结果 3.44 FIT）；BiCMOS 三元素的完整逐步计算（式 (3)–(17)，p35–36）最终  $\lambda_{\text{die}} \approx 2.01$  FIT，其中  $\lambda_2$  相关项 2.01 FIT 远大于  $\lambda_1$  相关项  $6.05 \times 10^{-4}$  FIT——**在这个 BiCMOS 示例中，工艺掌握度项是主导项**；不应把这一示例直接外推到所有模拟工艺。

走完这个示例，把模型的温度降额参数摊开看一眼 (§4.6.2.1.1.2, p36)： $(\pi_t)_i$  是任务剖面第  $i$  个相位的结温应力因子， $\tau_i$  是该相位的工作时间比， $\tau_{\text{on}} = \sum \tau_i$  是总工作时间比， $\tau_{\text{off}}$  是储存（或休眠）时间比，两者之和恒为 1。降额因子  $= \sum (\pi_t)_i \cdot \tau_i / (\tau_{\text{on}} + \tau_{\text{off}})$ 。BiCMOS 示例的中间步骤展示了它怎么算（式 (4)–(7)，p35）：三个温度相位 (32/60/85  $^\circ\text{C}$ ) 各自按 Arrhenius 式算出  $\pi_t$  再乘时间比，加和得  $8.25 \times 10^{-2}$ ——

任务剖面就是这样一相位一相位地进入失效率的。也因此，本书的 BookHousePolicy 把剖面里每一行“多少小时、什么温度”的出处列为 §4.6.1.7 假设清单的必填项。

**降额因子的方向感** (§4.6.2.1.1.2, p36-37):  $\tau_{\text{off}}$  是储存时间比,  $\tau_{\text{on}} + \tau_{\text{off}} = 1$ 。保守做法是令  $\tau_{\text{off}} = 0$  (只按工作相位算), 示例里降额因子从 0.17 变 2.91, 有效失效率从 0.31 FIT 涨到 **5.24 FIT**——差 17 倍。这个示例显示, 任务剖面里“车停着不通电”的时间占比可以改变结论量级, 这正是假设文档化纪律有工程价值的理由。

**封装示例** (Table 6, p37): PQFP 144, FR4 基板  $\alpha_s = 16$  与塑封  $\alpha_c = 21.5$  得  $\pi_\alpha = 1.05$ , 按 0.5 mm 间距、20 mm 边宽查  $\lambda_3 = 11.87$  FIT, 热循环项中间量 6 009 (模型内部的无量纲量, 随年循环数与温幅进入公式), 含焊点的封装失效率 206 FIT; 按“焊点约占 20%”的经验扣除后  $\lambda_{\text{package}} = \mathbf{166 \text{ FIT}}$  (20% 粗算为 164.8, 表值 166 的差来自查表取整)。按引脚均摊:  $166/144 \approx \mathbf{1.15 \text{ FIT/脚}}$  (手算核对:  $166 \div 144 = 1.153$ )。NOTE 3 提醒均摊不总是成立——BGA 某些位置的分布更高 (p38)。

**过应力项** (§4.6.2.1.1.4, p38): 模型没有汽车电气环境的  $\lambda_{\text{EOS}}$ , 示例借“民用航电 (机载计算机)”的 20 FIT; 器件直连外部环境 (是接口) 则  $\pi_I = 1$ 、 $\lambda_{\text{overstress}} = 20$  FIT, 否则为 0。NOTE 又把门打开了一半: 电气过应力可视为系统性失效模式, 在随机硬件度量计算中降为 0 FIT——前提是你论证并缓解了它的系统性成因, 而不是当它不存在。

### A.3.4 手册路线之二与三: SN 29500 与 FIDES

**SN 29500** 走查表路线 (§4.6.2.1.2, p38-41): 按产品类型、工艺、晶体管数查参考条件下的  $\lambda_{\text{ref}}$ , 再用修正的 Arrhenius 方程折算到实际工作条件 (温度、电压、漂移)。表格没有你的规格时允许插值: 示例中 50 万门微控制器落在 CMOS 表 10 万门 (76 FIT@80 °C 虚拟结温) 与 100 万门 (80 FIT@90 °C) 之间, 先把各列折算到同一虚拟参考温度 90 °C, 线性插值得  $\lambda_{\text{ref}}(500\text{K}@90 \text{ °C}) = 78.2$  FIT, 虚拟结温插值得 84.4 °C, 再回折得 **62 FIT** (p39)。套上“电机控制”任务剖面 (年工作 500 小时、三档环境温度加权) 得总温度因子  $\pi_T = 1.31$ , 整片有效失效率  $\approx \mathbf{105 \text{ FIT}}$  (Table 8, p40)。与 62380 模型的口径差异: SN 29500 默认**只算工作小时**, 非工作相位要额外乘应力因子  $\pi_w$ ——示例中按全球昼夜均温 10.5 °C 算得  $\pi_w = 0.06$ , 104.65 FIT 折到含非工作相位的 **6.3 FIT** (Table 9, p41; 手算核对  $104.65 \times 0.06 = 6.28 \approx 6.3$ )。同一颗芯片, “每工作小时”与“每日历小时”两种单位差 17 倍——**单位口径是 §4.6.1.7 建议披露的事项, 也是本书数据接收 BookHousePolicy 的必填项**。SN 29500 整值不拆裸片/封装, 需要拆分时按专家判断, 例如借用 62380 的比例 (§4.6.2.1.2.4, p41)。

**FIDES Guide** 路线 (§4.6.2.1.3, p41-44): 失效率 = 物理贡献  $\lambda_{\text{Physical}} \times$  过程贡献  $\pi_{\text{Process}} \times$  制造贡献  $\pi_{\text{PM}}$ 。后两个因子对应系统性问题, Part 11 的 ISO 26262 计算示例将它们置 1。物理贡献覆盖热、温度循环、机械、湿度四族应力 (示例为简化只算前两族)。示例数值: 裸片  $\lambda_{\text{0TH}}$  合计 0.13 FIT (CPU 0.08 + SRAM 0.06), 简

化任务剖面下温度降额 5.79，裸片有效 **0.75 FIT**；细化任务剖面（增加满载、高速公路等相位）降额升到 12.44，裸片有效 **1.61 FIT** (Table 13/16, p43-44)。在这个示例中，剖面细化后结果约翻一倍，说明任务剖面可能是高敏感输入，但不证明它在所有模型和项目中都排第一。封装（含循环降额）0.25 FIT；焊点 0.12 FIT 仅供参考、不计入封装。

三条手册路线并排看的教学结论：62380 模型、SN 29500、FIDES 对同一颗教学芯片给出的失效率分别在 1.25（裸片）、105（整片，每工作小时口径）/6.3（整片，含非工作）、0.75-1.61（裸片）FIT 量级——注意 SN 29500 的整值不拆裸片/封装，这三组数连口径都不同层。这些差异主要提醒读者核对模型边界、失效机理集、任务剖面和小口径；不能只用“假设不同”解释一切差异，也不能从三个示例中选一个当作真值。本书建议选定有适用性理由的路线、写全假设、在整个项目里保持口径一致，并在与供应商数据对表时先做机理与口径调和。

### A.3.5 现场数据路线与实验路线

现场数据看似最“真”，实则最需要戒心 (§4.6.2.2, p44-45)。先审流程三问：已知质量问题在退货流程中怎么处理的？真实任务剖面知道多少？现场监控的有效性如何？Part 2 §7.4.2.3 的 shall 是组织建立、执行并维护生产/运行/服务/退役阶段的功能安全过程，其 NOTE 说明这包括现场监控；Part 11 再资料性地建议半导体供应商关注数据收集、上置信限、系统性故障剔除条件、**修正因子** (CF) 理由，以及经验证模型给出的**加速因子** (AF)。本书把这些建议转成现场数据接收清单时标记为 **BookHousePolicy**，不把整张清单冒充 Part 2 原句。

计算用指数模型加卡方统计 (§4.6.2.2.1, p45)：
$$FIT = \chi^2(CL; 2n+2) \times 10^9 / (2 \times \text{累计运行小时} \times AF)$$
，其中  $n$  为失效数乘修正因子；Part 11 建议使用至少 70% 置信水平的单侧上置信限。原文把它口语化为“真实失效率有 70% 概率低于该值”，但频率学上更严谨的解释是：**若在固定真值下反复按同一抽样与构造程序求上限，至少约 70% 的上限会覆盖该真值**；不能把固定真值说成在本次区间内具有 70% 的后验概率。示例 (§4.6.2.2.2, p45-47)：三款在役芯片的任务剖面折算出等效结温 (55.1/51.4/67.4 °C)，用 0.3 eV 激活能的 Arrhenius 折到参考温度 55 °C，合计裸片面积小时 805 317 百万  $\text{mm}^2 \cdot \text{小时}$ ；保修期失效 4 起，乘  $CF = 5$  得 20 起，卡方 70% 上界折算得 **0.029 FIT/ $\text{mm}^2$**  (Table 19, p46)。新设计芯片 23  $\text{mm}^2$ 、等效结温 75 °C、加速因子 1.84： $0.029 \times 1.84 \approx 0.053 \text{ FIT}/\text{mm}^2$ ， $\times 23 \approx 1.23$ ，与表值 **1.22 FIT** 的微小出入来自表内中间值修约 (Table 20, p47；表列  $\text{FIT}/\text{mm}^2$  修约为 0.05)。封装失效率同法，但加速因子换 Coffin-Manson 或 Norris-Landzberg 模型 (NOTE 4)。

加速寿命试验路线 (§4.6.2.3, p47-48)：从试验温度降额到最高工作温度用 0.7 eV 激活能的 Arrhenius（原文建议对目标失效机理估计并验证激活能）；样本失效数经卡方分布外推到总体；CMOS 还要考虑电压加速。此处有一个值得保留的源文疑点：Part



11 式 (25) 在原 PDF 与提取视图中均排印为  $AF_v = \exp(\beta) \times [V_t - V_o]$ ，同页又把  $\beta$  的单位写成  $1/V$ ，因而按量纲检查存在疑点。本附录忠实记录该排印，但不把它当作可直接执行的计算式；项目应回到所选可靠性模型的受控公式与参数定义核对。

A.3.6 失效率的再分配与多芯片模块

算出整片失效率后，还要摊到组成元素的失效模式上 (§4.6.2.4, p48)。裸片部分两法：**按面积**——失效率密度 (FIT/mm<sup>2</sup>) 乘以该 part/subpart 的占用面积 (Part 11 §5.1.7.1 的例子：CPU 占裸片面积 3%，就摊总失效率的 3%)；**按门数**——基本子部件 (门/晶体管) 失效率乘以数量。封装部分：有封装失效率时按“总失效率 ÷ 总引脚数”得每脚失效率，再按安全相关引脚分摊。选哪种方法看版图信息可得性与失效模式在硬件元素间的共享方式 (NOTE)。多芯片模块 (MCM) 单列一条 (§4.6.2.5, p49)：Part 11 建议套用工业手册时给出适用性或定制理由——手册的统计基础通常是单芯片封装，多裸片互连的失效机理未必被覆盖；本书将该理由设为接收条件时标记为 BookHousePolicy。

A.3.7 选路线的决策清单

四条路线不是并列的自助餐，各有前置条件。给一张教学用的决策清单 (本表为本书归纳，非 Part 11 原文表格；最后一列是 BookHousePolicy，不是 ISO shall)：

| 你手里有什么      | 优先路线                               | 主要风险                                    | 本书接收策略要求的补充说明          |
|-------------|------------------------------------|-----------------------------------------|------------------------|
| 只有设计规格与工艺信息 | 工业手册 (62380 模型 / SN 29500 / FIDES) | 模型结果往往偏保守，手册版本、参数选择理<br>但仍需验证机理集与应力的适用性 | 由、口径 (工作/日历小时)         |
| 同工艺同封装的量产历史 | 现场数据 + $\chi^2$ 上界                 | 退货流程失真、任务剖面未知                           | CF 修正因子理由、监控有效性、剖面文档   |
| 新工艺、无历史     | 加速寿命试验 / 可靠性测试芯片                   | 激活能假设与目标机理不符会显著扭曲外推                     | 目标机理的激活能实测验证           |
| 深亚微米软错误     | JESD89 测量 + IRDS 投影过渡              | 投影值只是首次估计                               | 拿到工艺实测后更新，标注中子通量/海拔等条件 |

本书采用三条通用 BookHousePolicy：**先调和，后组合或比较**——供应商 A 用现场数据、供应商 B 用 SN 29500 时，集成者按 §4.6.1.7 的假设清单对齐机理与口径，不直接把两个未调和的 FIT 值相加或比较；**声称保守就要验证方向**——对常数失效率、 $\lambda_2$  取最大、 $\tau_{off}$  置零、70% 上界等具体近似，逐一说明其为何不低估目标量，而不是



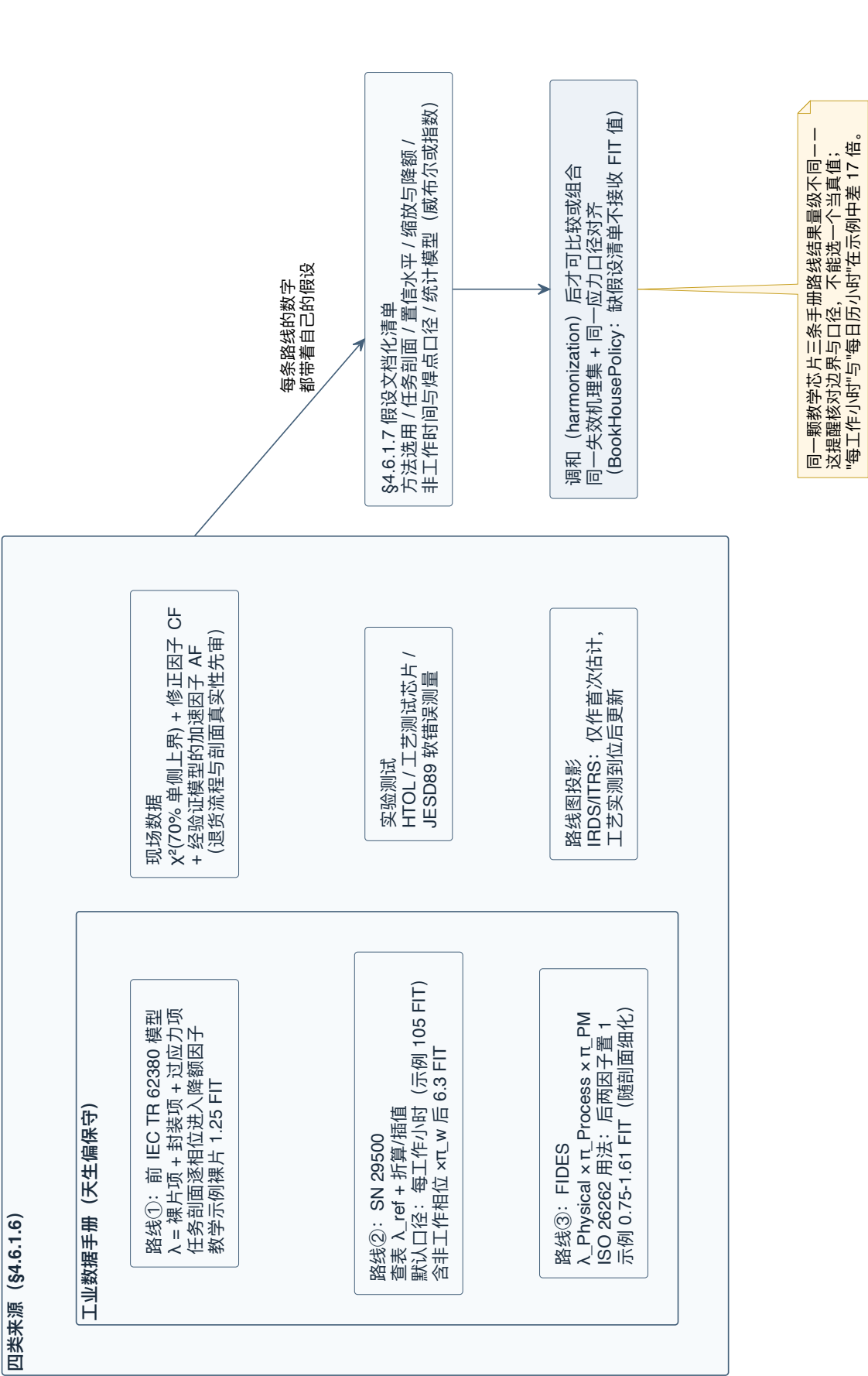


图 A-2 基础失效率来源路线图——工业手册三条路线（前 IEC TR 62380 模型、SN 29500、FIDES）与现场数据、实验测试、实验测试、路线图投影四类来源各带前置条件，全部汇入 §4.6.1.7 假设文档化清单；未经机理与口径调和的数字不进入比较与组合。

笼统地声称“所有近似都保守”；**更新有触发器**——工艺、封装或任务剖面变更，以及获得足以改变估计的新现场数据时，启动失效率复核并保留版本链。

收口对仗：**来源给数字，假设给资格；口径不对齐，数字无意义**。这些纪律与第 6 章“数值诚实”的映射，在 A.8 节落成三元组。

## A.4 半导体 DFA：DFI 七分类与 B1-B12 workflow

### A.4.1 为什么芯片内的相关失效要单独讲

第 8 章首讲过相关失效分析——DFA (Dependent Failure Analysis)，以及它服务的两大论证：ASIL 分解要求的**独立性** (Part 9 Clause 5) 与共存分析要求的**免于干扰** (Part 9 Clause 6)。Part 11 §4.7 把镜头推进到**同一块硅片内部**：所有“独立”通道共享同一片衬底、同一套电源网络、同一棵时钟树、同一个封装——在系统层看似天各一方的元素，在芯片里可能只隔几微米。Part 11 §4.7 的范围声明就是这么写的：单一裸片内硬件元素之间、以及硬件与软件元素之间的 DFA (p49)。

核心术语沿用 Part 1 §3.30 的定义：**相关失效引发源** (Dependent Failure Initiator, DFI) ——经由耦合因子导致多个元素失效的单一根因 (p49)。这里要还一笔第 8 章挂过的账：Part 9 正文只说 root cause 与 coupling factor，把 **DFI 做成成体系分类的是 Part 11 §4.7.5.1**；Part 9 Annex C 表 C.1 最后一列的映射对象正是 Part 11 的这些 DFI 名。第 8 章的 DFA 九主题与七耦合因子是“分析什么”的清单，Part 11 的 DFI 七分类是“根因长什么样”的清单，两者是同一件事的两个切面。

DFA 与常规安全分析的分工 (§4.7.1–§4.7.2, p49–50)：安全分析 (Part 5 §7.4.3、Part 9 Clause 8) 负责找单点/多点故障、算度量、定机制；DFA 补的是“机制的有效性会不会被相关失效掏空”。哪些元素需要 DFA，常常能从安全分析结果里直接读出来：**双点失效场景**——功能与其安全机制 (含故障反应路径上的整条元素/任务链)、功能冗余 (两路电流驱动器、两路 ADC)；**共享基础设施的单点 (残余) 场景**——时钟生成、嵌入式电压调节器、以及上述元素共用的任何硬件资源。Part 11 指南说明：已识别依赖关系且可按残余/单点/多点故障归类的随机硬件 DFI，可纳入 Part 5 Clause 8/9 的定量分析；不能可靠量化的系统性、环境性根因则继续定性处理 (§4.7.3 NOTE 1, p51)。纳入度量不等于删除 DFA 追溯或自动闭环 Part 9 义务；本书的 **BookHousePolicy** 仍要求保存依赖识别、分类与措施理由。

### A.4.2 相关失效场景的解剖学

§4.7.3 (p50–52) 给相关失效场景做了一次解剖，四个观察维度日后都会变成缓解措施的设计输入：

- **耦合机制**：根因如何把扰动送到受害元素——传导耦合（电或热，经导体直接接触）、近场耦合（容性 = 电场感应、感性 = 磁场感应，距离小于一个波长）、机械耦合（应力经物理介质传递——MEMS 的梳齿结构可能因特定共振冲击而粘连，专名 stiction，见 5.5.3.2）、辐射耦合（距离大于波长，源与受害者互为天线）；
- **传播介质**：信号线、时钟网络、电源网络、衬底、封装、空气；
- **局部性**：扰动是同时打击多个元素，还是只打击一个、再由其错误输出**级联**到下游；
- **时间特征**：传播延迟（如温度梯度）、周期性（如电源纹波）。

这四个维度不只是分类，还可成为措施设计的输入表。本书的操作化示意是：传导耦合可考虑电气隔离与独立供电域，容性/感性近场耦合可考虑间距与屏蔽，机械耦合可考虑阻尼与结构设计，辐射耦合可考虑屏蔽与滤波；走电源、衬底或时钟网络的扰动，分别提示在电源域边界、物理隔离结构或时钟监督处寻找候选防线；局部性用于区分共因与级联传播；时间特征用于设计监控带宽和采样策略。这些都只是**候选手段**，不是某类耦合的唯一处方，也不自动证明措施足够。§4.7.5.2 资料性地建议论证“措施与目标 DFI 相称”；本书把四维逐项对表作为 **BookHousePolicy** 的一种操作化，而不是 Part 11 的独立要求。

两张示意图值得记住 (Figure 21/22, p52–53)。图 21：元素 A1 与 A2 冗余，比较器 B 检查输出——若同一根因让 A1、A2 产生**相同**的错误输出，B 在比对时刻无法分辨；只要两路错误存在时间或空间上的差异（扰动传播路径不同、时序违例效应不同），依赖场景仍可控 (NOTE 2) ——这正是后文“影响多样化”类措施的原理。图 22：共享元素 X 出错，错误输出同时馈入 A1 与 A2——“共享资源”类 DFI 的标准像。

级联失效与共因失效的区分，Part 11 §4.7.4 给出一个务实的资料性解释 (p53)：对某些半导体分析，精确区分可能不可行或对分析目的没有增益，此时指南认为可不再细分。这不是对 Part 9 Clause 7 的普遍豁免；是否合并取决于项目能否仍满足所需独立性或免于干扰论证。若分析目的聚焦两个元素间的免于干扰，指南示例的四步是：找出元素 A 会影响 B 的失效模式 → 判断这些模式是否经 B 的失效导致安全目标违背 → 必要时定义缓解措施（例如给 DMA 配一个地址监控安全机制）→ **换位重做一遍**。

### A.4.3 DFI 七分类与缓解措施两分法

§4.7.5.1 (p53–58) 给出 DFI 的七个类别——这是本书第 8 章预告过的“DFI 七分类”的原文出处：

1. **共享资源失效**（随机硬件，Table 21, p55）；
2. **单一物理根因**（随机硬件，Table 22, p56）；

3. 环境条件（系统性，Table 23，p56）；
4. 开发故障（系统性，Table 24，p57）；
5. 制造故障（系统性，Table 25，p57）；
6. 安装故障（系统性，Table 26，p58）；
7. 服务故障（系统性；对半导体元件通常为空集，理由见下段 NOTE 4）。

NOTE 1 保留了开放性：其他分类法也可能适用（p54）。NOTE 4 解释了为什么第七类在半导体上通常是空集：汽车维修一般整体换 ECU 或传感器模块，半导体元件本身不被服务，所以服务故障通常不构成半导体的 DFI（p54）。NOTE 3 划了软件的界：软件引起的 DFI 不在此清单——正确的软件开发归 Part 6 管，但 DFA 的结果可以影响软件元素的 ASIL 分配（p54）。这些 NOTE 都是指南边界，不是封闭分类规则。

每个类别的措施分**两族**（p54）：**防违背措施**——不阻止相关失效发生，但阻止它违背安全目标（运行时检测与反应）；**防发生措施**——在运行前或运行外消灭它（设计、生产、安装环节）。这个两分法与第 6 章“覆盖率是掐断因果链，不是让元件不坏”一脉相承。把六张表压缩成一张速览（详表请回原文对应页）：

| DFI 类别 | 典型根因示例                                                                                                           | 防违背措施示例                                                                                                                  | 防发生措施示例                                                                                  |
|--------|------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| 共享资源失效 | 时钟（PLL/时钟树/使能）、测试与调试逻辑（DFT 扫描链、调试路由、trace）、电源（配电网络/共享调节器/带隙基准）、非同上电（门锁/浪涌）、复位逻辑、共享模块（RAM/Flash/ADC/DMA/中断控制器/总线） | 共享资源专用独立监控（时钟监控、电压监控、存储 ECC、配置寄存器 CRC、测试/调试模式信号化）；选择性抗软错误加固；启动/运行自检；影响多主查核、不同关键路径；间接检测（会因共享资源失效而失败的周期自检）；专用传感器（延迟线共因传感器） | 保守规格等避错措施；共享资源内部功能冗余（多过孔/触点）；故障诊断与隔离/重构；专用生产测试（能发现复杂故障的 SRAM 全检）；拆分资源以缩小共享面；自适应降敏（降压/降频） |

| DFI 类别 | 典型根因示例                                                                                | 防违背措施示例                         | 防发生措施示例                                                    |
|--------|---------------------------------------------------------------------------------------|---------------------------------|------------------------------------------------------------|
| 单一物理根因 | 短路(局部缺陷、电/过孔/触点迁移、氧化层击穿)、门锁、串扰(衬底电流、容性耦合)、局部发热(失效的调节器或输出驱动)                           | 影响多样化；间接检测或专用传感器                | 专用生产测试；物理隔离/间距设计规则；单芯片上的物理分离                               |
| 环境条件   | 温度、振动、压力、湿度/凝露、腐蚀、EMI、外部过压、机械应力、磨损、老化、进水进液                                            | 影响多样化；环境条件直接监控(温度传感器)或间接监控(延迟线) | 保守规格/稳健设计；物理分离(远离热源)；自适应降敏；限制访问频次或操作循环数(EEPROM 写次数)；封装稳健设计 |
| 开发故障   | 需求/规格错误、实现错误、串扰或门锁预防设计不足、错误配置、版图错误(冗余块上布线、绝缘/隔离/屏蔽不足)、功耗热梯度导致敏感电路失配                   | 监控器(协议检查器)                      | 符合 ISO 26262 的设计流程；多样性(实现/功能/架构多样性或开发多样性)                  |
| 制造故障   | 过程规程与培训问题、控制计划与特殊特性监控缺陷、烧录与产线末端编程错误(版本/条件/协议/时序)、掩模错位、末端修调或熔丝错误(激光修调、OTP/EEPROM 校准系数) | 无                               | 专用生产测试；符合 ISO 26262 (见 §4.9)；多样性                           |
| 安装故障   | 线束布线、部件互换性、相邻件失效(接口配置错误、负载错误)、PCB 连接错误、备用存储配置错误                                       | 无                               | 专用安装测试；符合 ISO 26262；多样性                                    |

注意 Part 11 Table 25/26 在制造与安装两类的“防违背措施”栏列为 **none**，把重点放在生产/安装前的预防与测试。应把它读成该指南没有列出可依赖的运行时措施，而不是证明所有设计中绝无任何缓解可能；具体项目仍按根因、传播路径和适用的 Part



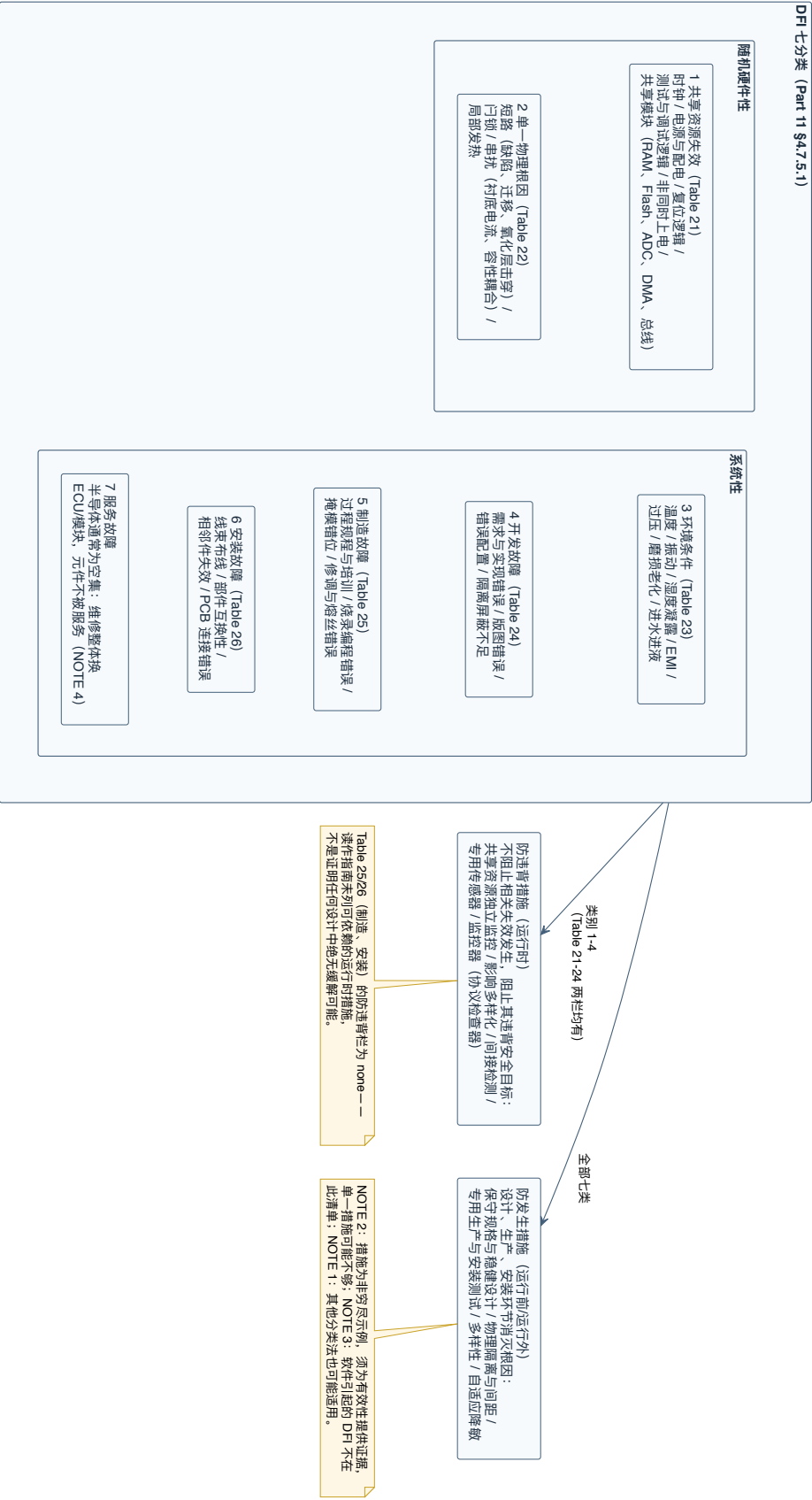


图 A-3 DFI 七分类与措施两分法——两类随机硬件性原因与五类系统性原因（服务故障对半导体通常空集），各配防违背（运行时）与防发生（运行前）两族措施；制造与安装两类在指南表中防违背栏为 none。

7/9 要求分析。这为 A.6 节“在该资料性解释下全线按安全相关处理”的论证提供了一项半导体侧理由，但不证明这是唯一的生产策略。

NOTE 2 的免责声明值得带进项目模板 (p54)：所列措施是**非穷尽的示例**，有效性取决于电路类型与工艺，Part 11 建议为所声称的有效性提供证据；单一措施可能不够，需组合。验证有效性的方法族见 §4.7.5.2 (p58–59)：已知原理的分析法、按文档化试验协议做的**流片前仿真**（时钟/电源扰动仿真、EMI 仿真，配合故障注入产生目标扰动）、**流片后稳健性测试**（EMI、老化、加速老化、电应力）、以及有文档理由支撑的专家判断。两条边界要分层：Part 9 §7.4.2 NOTE 资料性地指出 IEC 61508 式的  $\beta$  因子量化**一般不是足够可靠的方法**；Part 11 则建议用**多样性或隔离/分离**做措施时，论证其程度与目标 DFI 相称。项目若把这两项设为放行条件，应标 BookHousePolicy；不能称为 Part 11 shall。

#### A.4.4 B1–B12 workflows：一条有回路的流水线

§4.7.6 用一张流程图 (Figure 23, p59) 把 DFA 组织成 B1–B12 十二个框，注意它不是直线而是带两处“信息不够就回炉”的回路：

- **B1 启动判定与元素识别** (p60)：六种情形触发 DFA——诊断功能分配到硬件/软件元素；相似或相异冗余；元件/part 上的共享资源（时钟、复位、电源、存储、ADC、I/O、测试逻辑）；共享硬件上跑多个软件任务；共享软件功能（I/O 例程、中断处理、数学库）；系统或元素级 ASIL 分解派生的独立性要求落到同一 IC 的不同元素上。输入是技术安全需求（尤其独立性/免于干扰要求）、架构描述（框图、流程图、故障树、状态图、软硬件分区）与安全措施；产出是“可能被相关失效影响的元素对/组清单”及其独立性要求。NOTE 1 顺手划了个常被忽略的边界：跑在可编程硬件上的固件与微码，无论载体叫不叫 CPU，都可视为**软件元素** (p60)。
- **B2 DFI 识别** (p60)：以 §4.7.5.1 的典型 DFI 清单为查验表，补全架构分析没想到的已知依赖，并与定量分析中发现的依赖机制交叉核对。
- **B3/B4 效应信息充分性判定** (p60–61)：逐个 DFI 检查现有文档是否足以判断其对目标架构的有效性；不够就补充资料、回到 B2。NOTE 推荐分层法：顶层视图看清共享资源，再下钻到“硬件 subpart+ 其安全措施”的分解视图找设计级依赖。
- **B5 相关 DFI 清单固化** (p61)：经评审固化清单；随机硬件类 DFI 中能**可靠量化的部分**纳入 Part 5 Clause 8/9 的定量分析（共享振荡器输出过快时钟、调节器故障导致内部过压，见 NOTE 2 列举）；不能可靠量化的（时钟树故障的时序效应、元素与其机制间的热耦合、衬底电流）继续走定性通道。

- **B6 措施识别** (p61)：为目标架构相关的相关失效补充必要的安全措施，§4.7.5.1 的措施列做起点，最终措施需求要文档化。
- **B7/B8 措施信息充分性判定** (p61)：文档不足以评估措施有效性时，补充细节再回 B6。
- **B9 措施清单固化** (p61)：经评审固化。NOTE 2 藏着一个容易漏的闭环：新增措施改变芯片面积，从而改变各部分的失效率分布——**定量分析通常要跟着更新**。
- **B10 有效性评估** (p62)：验证方法与 Part 5 Clause 10 的安全措施验证同族——FTA/ETA/FMEA、故障注入仿真、基于工艺资格测试的设计规则、电压等级或间距的过设计、温度/过压应力测试、EMC 与 ESD 测试、专家判断。两条 NOTE：实现措施的元素本身也要计入 Clause 8/9 的定量分析；**新引入的措施自己也可能成为相关失效的宿主**，此时对新依赖重新走一遍 DFA 程序。
- **B11/B12 风险充分性评估与措施改进** (p62)：评估剩余风险；不充分则改进措施 (B12) 并重评。量化的残余风险计入定量分析——例如功能与其安全机制被同一相关失效波及时，把机制的失效模式覆盖率**按未缓解的依赖折减**。结尾的 NOTE 把定量与定性的分工再说了一遍：若定量度量目标已达成，随机硬件视角的风险即视为足够低，即使受影响元素没有专门措施；同一元素上的系统性 DFI 仍在 DFA 里定性处理，并可能独立于定量结果引出措施。

硬件与软件之间的相关失效通常分开考虑，只有当“针对硬件的安全机制用软件实现”时才合并分析 (§4.7.8, p63)。两个原文例子：软件 CPU 自检与独立硬件看门狗互为兜底——CPU 坏了，自检抓不到就由看门狗抓；E-GAS 三层监控概念中 L2 软件监控 L1 软件，两层本身相异，为进一步压低硬件相关失效导致安全目标违背的概率，再加程序流监控、CPU 自检、L2 关键变量在 RAM 中反码冗余存储、独立的质询-应答看门狗。

Annex B (p144-157) 给了两个完整判例，值得多花一段拆它的论证结构。**微控制器例** (B.1, p144-148)：分析对象是一颗带冗余 CPU 的微控制器，安全需求被刻意精简到两条便于聚焦——MCU-REQ-1 要求“硬件元素 1.1 处理信号 1 的故障须在 20 ms 内检出”，其子需求规定由硬件元素 1.2 冗余处理、软件比对结果、不一致时经看门狗接口报错；MCU-REQ-2 要求“导致 CPU 错误输出的随机硬件故障须在 20 ms 内检出”，子需求规定冗余 CPU 逐拍硬件比对、不一致即产生错误事件 (p146)。这些都是 Annex B 判例内部的需求，不是可复用到其他 MCU 的 20 ms 标准值。DFA 只聚焦有潜力违背 MCU-REQ-2 的 DFI，沿工作流走：B1/B2 阶段用一棵**定性故障树** (Figure B.2) 分析架构，把共享资源（时钟、电源、复位）与冗余元素摆上台面；然后逐行落进一张五列账表 (Table B.1, p147)：元素—冗余元素—DFI（分“共享资源”与“单一物理根因”两列）—故障的避免 (A) 或控制 (C) 措施—验证方法。表中每条措施都标了 A/C 属性

(对应 §4.7.5.1 的两分法)，并留出验证方法列。本书由此采用一条 **BookHousePolicy**：未给出有效性验证方法的措施仍是候选措施，不标记为已闭环。**模拟例** (B.2, p149–157) 走查两类场景：共享电压调节器 (B.2.2) 与耦合机制 (B.2.3, 衬底耦合、热耦合这类“看不见的线”)，示范了当 DFI 无法量化时定性论证怎么写。判例的正确用法与第 4 章 HARA 判例目录相同：**学它的论证结构，不复制它的结论**——你的芯片的共享资源清单、措施与覆盖率，都要从你自己的架构分析里长出来。

最后是本书的状态纪律 (**BookHousePolicy**, 回指第 8 章的放行语义)：把 B1–B12 建成流程模型、把七类 DFI 建成分类表，都只是**结构就绪**；每一条具体依赖的可信度判定、措施有效性证据、评审结论必须落在各自状态词表，不能由流程图存在推断完成。 **workflow画完不等于 DFA 做完，DFA 做完不等于独立性成立**——中间隔着证据。

## A.5 故障注入与验证证据

### A.5.1 故障注入能回答什么问题

故障注入——人为把故障塞进设计或实物、观察安全机制接不接得住——在附录 D 的集成测试与软件测试方法表里都露过面。Part 11 §4.8 讲它在半导体层的专门用法 (p63)：支撑硬件架构度量的评估；评估安全机制的诊断覆盖率 (软件自检、硬件内建自检)；评估诊断时间间隔与故障反应时间间隔——回指第 2 章的时间间隔家族；确认故障效应 (例如计算瞬态故障的架构脆弱性因子：一个故障在特定输入下产生可观察错误的概率)；以及支撑安全机制的流片前验证 (触发机制、验证软件级反应、验证边角用例、验证互连)。

规模问题有官方台阶 (NOTE 1, p63)：若以合理资源在合理时间内做不完精确注入，可以缩小战役范围 (只在 IP 块级注入)、只用分析法、或分析与注入组合。台阶不是滑梯——缩了范围，结论的适用范围也要跟着缩。

### A.5.2 把一次注入战役写成一个实验对象

§4.8.2 (p63–65) 资料性地给出一份可帮助验证策划、并建议在策划中描述和论证的信息清单。本书把它操作化为**七元组实验对象**——目标、故障模型、注入位置、注入时间、观测量、判定准则、结果。七元组是 **BookHousePolicy**，不是 Part 11 原生数据模型。逐项拆开：

- **故障模型与抽象层级**：stuck-at 注在门级网表，位翻转在 RTL 级就够 (EXAMPLE 1)；层级选择还取决于所需精度——用门级网表对 CPU 软件自检做端口/网络故障注入，置信度高 (EXAMPLE 2)。Part 11 建议给出层级选择理由；本书将其设为接收条件。

- **观测点与诊断点：**在哪一层看故障效应（观测点）、在哪一层看机制反应（诊断点）。奇偶校验电路的覆盖率验证可设在 part/subpart 级；跨 IO 回环验证设在顶层（EXAMPLE 3/4）。顶层注入不可行时，Part 11 给出用**功能等价模型**替代安全机制的资料性做法（例如把看门狗换成等价模型），并建议提供“模型充分反映机制安全特性”的证据（NOTE 3、EXAMPLE 5）；本书将这份证据设为接收条件。
- **注入方法：**已知故障位置的直接注入、形式化方法注入、仿真加速器（emulation）注入、辐照注入（EXAMPLE 6）。
- **故障位置与故障清单：**与被验证的失效模式挂钩。Part 11 允许在有理由时抽样，并在 NOTE 5 列出样本量  $n$ 、样本分析结果  $p$ （如机制检出 stuck-at 的比例）、期望置信度、误差边界与统计独立性；原文用  $\alpha$  表示“desired confidence”，统计计划应明确  $\alpha$  究竟指置信水平还是通常意义上的显著性水平。也可用故障折叠（fault collapsing）把故障群压缩到质故障。范围裁剪要有机理依据：验证双核锁步的覆盖率，相关故障群可以限于被比较的 CPU 输出及相关位置；验证软件自检，相关的是 CPU 内部故障（EXAMPLE 7/8）。本书要求预先冻结这些选择，规则依据为 BookHousePolicy。
- **注入控制：**注什么类型、瞬态持续多久、一次仿真注几个、何时何地注、观察窗开多大（EXAMPLE 9）。

抽样的统计学值得停一拍做算术直觉（教学示例，非 Part 11 原文数值）。假设某安全机制的目标覆盖率主张是 99%，从百万级故障空间抽取  $n = 1\,000$  个故障，机制检出 995 个，点估计  $\hat{p} = 99.5\%$ 。用预先选定的**双侧 95% Wilson score 区间**计算，区间约为 [98.835%, 99.786%]；下界低于 99%，所以这批数据不能按该判据背书“覆盖率至少 99%”。若只抽  $n = 100$  并检出 99 个，Wilson 95% 区间更宽，约为 [94.551%, 99.823%]。若验收问题只关心下界，可预先规定单侧下置信限与构造方法，但不能看到结果后再换口径；即使用单侧 95% Wilson 下限，995/1 000 也只有约 98.976%，仍低于 99%。

这里的“95%”也不是“真实覆盖率有 95% 概率落在本次区间内”：在频率学解释中，真实覆盖率是固定参数；若反复采用同一抽样机制和 Wilson 构造方法，长期约 95% 的区间会覆盖它。这个解释还依赖二项模型、代表性抽样与独立性；故障折叠、分层抽样或相关注入会改变有效样本量，不能把公式机械套上。Part 11 NOTE 5 只是资料性地列出  $n$ 、 $p$ 、置信度、区间与统计独立性；本书要求预注册统计方案并以区间下界放行，属于 BookHousePolicy。

- **工作负载：**验证锁步比较器的完整性，基础负载即可（只需激励 CPU 输出）；验证非对称冗余的覆盖率，用功能测试集派生的激励；验证瞬态故障的安全故



障占比，用接近实际用例的负载；验证软件自检——**负载主要就是自检程序本身** (EXAMPLE 10-13)。不同负载会激活不同故障与传播路径，选择不匹配可能使覆盖率产生方向性偏差；因而负载是需要单列理由的偏差源，不应用“最隐蔽”这类无数据排名代替论证。

A.5.3 结果的用途与两条诚实条款

注入结果可用于验证安全概念与其底层假设：机制有效性、诊断覆盖率、安全故障数量 (§4.8.3, p65)。两条 NOTE 都是资料性证据指南：NOTE 1 建议为功能安全审核保留注入证据；NOTE 2 说明仿真的故障与安全分析里识别的故障**不总是一一对应**（例如开路故障难以直接仿真），可用其他代表性故障的结果精化分析（如 5.1.10.2 的 N-detect 法），同时为替代关系提供理由。本书进一步要求“做过注入”必须对应可复核的七元组战役记录；这是 **BookHousePolicy**，不是 NOTE 被提升成了 shall。

把七元组落成一张随身检查单（本表为本书归纳）：

| 元组项            | 要写清的问题                          | 常见缺口                            |
|----------------|---------------------------------|---------------------------------|
| 目标             | 这次战役支撑哪条覆盖率/时间量主张               | 一次注入背书多条主张，范围含混                 |
| 故障模型 + 抽象层     | 注的是什么故障、在哪一层注、为何该层足够            | RTL 层注 stuck-at 却主张门级覆盖         |
| 注入位置 + 清单      | 故障群怎么选的、抽样统计要素齐不齐               | 只报点估计 p，不报 n 与置信区间              |
| 注入时间/控制观测点/诊断点 | 何时注、持续多久、观察窗多大<br>在哪看效应、在哪看机制反应 | 观察窗短于故障反应时间<br>用等价模型替代机制却无充分性证据 |
| 判定准则           | 什么算检出、什么算安全故障                   | “无输出差异”与“机制报警”混判                |
| 结果工件           | 记录存在哪、审计时拿什么复现                  | 只留结论表，不留战役配置                    |

对仗收口：注入回答的是“机制接得住吗”，七元组回答的是“你凭什么这么说”。把七元组对象化成知识模型，是 A.8 节的练习题之一。

A.6 生产、分布式开发与确认接口

### A.6.1 生产：标准流程、全线安全相关

第 9 章讲过 Part 7 的两大目标：开发并维持安全相关产品的生产过程。落到半导体 (§4.9.1, p65–66)，Part 11 给出两个行业解释。其一，半导体用的是**标准化生产流程**——晶圆加工、装配各工序（扩散、氧化沉积、离子注入、贴装）为特定产品单独开流程的情况很少；其二，指南认为通常无法把流程步骤清楚分成安全相关与非安全相关，因此在该解释下**全部按安全相关处理**。风险缓解可复用过程 FMEA 与控制计划等质量工具：制造测试对照电气规格验证产品、过程控制规格对照控制计划验证过程，两头合起来支撑“制造出的元件符合含硬件安全需求在内的要求”。“全线按安全相关处理”是 Part 11 的资料性解释；具体生产义务仍来自适用的 Part 7 Clause 5/6。

Part 11 §4.9.2 资料性地说明工作产物可复用 (p66)：满足 IATF 16949 的质量管理体系文档可能部分或全部支撑 Part 7 Clause 5/6——生产控制计划的安全相关内容可复用质量体系控制计划，控制措施报告同理。**复用的是证据，不是自动合规或豁免**：是否满足要求仍回到 Part 7 逐项判断。这里与 A.4.3 的 DFI 速览表首尾呼应：制造类 DFI 的“防违背措施”一栏列为 none，提醒读者不要把运行时措施当作一定存在的后手；但这一表项本身不证明掩模错位、末端修调错误在所有设计中都绝无运行时缓解。本书的工程归纳是：对这类根因，生产预防与检出通常是主要防线，这也是“在该解释下全线安全相关”的工程理由之一。

Part 11 §4.9.3 资料性地观察到：半导体元件通常不单独维护或维修，相关服务/退役活动往往可在安全计划中裁剪，供需双方也可在 DIA 中对齐预期 (p66)。实际裁剪须按 Part 2 §6.4.5 给出上下文理由，不能仅凭 Part 11 的通常情形自动省略工作产物。

### A.6.2 分布式开发：一颗芯片的上下游

§4.10 (p66–67) 资料性地说明半导体开发者常具有双重角色：向 Tier 1 或半导体集成方供货时，它可处于 Part 8 Clause 5 的**供应商**角色；采购 IP、封测服务时又处于**客户**角色。DIA、供应商安全计划、供应商选择等具体义务须按 Part 8 Clause 5 的适用条件判定，不能从 Part 11 的角色示例直接推出。指南还把安全相关分布式开发的最低层级解释为安全责任终止的层级；再往下的供应商（如生产材料供应商）可由质量管理体系等其他要求约束。它与第 10 章 DIA 的 a)–k) 条目框架兼容，补充的是“同一主体在链条中双重角色”的半导体常态。

### A.6.3 确认措施与集成验证的半导体裁剪

确认措施——第 3 章首讲的确认评审、功能安全审核与功能安全评估——在半导体开发中需按上下文裁剪。Part 11 §4.11 资料性地给出 SEooC/IP 场景的裁剪示例，并指出相关项层面的确认评审通常超出半导体供应商范围 (p67)；实际裁剪仍须按 Part 2 §6.4.5 等适用条款给出理由。指南还示例用**过程安全审核** (Process Safety Audit, PSA)

落地审核；一旦选定某项 Part 2 Table 1 确认措施，其独立性等级仍由该规范表决定——资料性示例不能降低等级。

硬件集成与验证的对接见 §4.12 (p67–68)：Table 27/28 逐行解释 Part 5 Table 10/11 的方法在半导体层怎么读——“需求分析”对应流片前（仿真级）与流片后（硅片级）验证；“内外接口分析”对应 IC 集成与 IO 相关的验证活动；“共因限制条件分析”对应时钟/电源/温度/EMI 测试；“环境与使用条件分析”对应温度循环、软错误率实验、高温工作寿命测试；“标准”对应 CAN/I2C/UART/SPI 等协议标准；“显著变体分析”对应工艺偏差/电压/温度（PVT）特性化测试。Part 11 把表格称为“起点”并建议按用例修改时给出理由（NOTE 1）；方法的规范性推荐度仍须回到 Part 5 Table 10/11。还有一处术语校准（p68）：“test case”在系统与半导体两界含义不同：流片后验证针对少量样品、聚焦正确集成与系统性故障；**量产测试**针对全部器件、聚焦生产引入的故障，采用结构化测试——后者属于“生产”语境，不等同于硬件集成验证。等价类与错误猜测两法在半导体测试中通常不太适用（p68）。

## A.7 五类技术案例：同一问题框走五遍

Part 11 §5 (p68–140) 按技术类型给了五份“应用说明”。本节用同一个问题框走五遍：**这类技术的故障模型是什么？安全机制改变了什么？定量分析需要哪些假设？还有哪些系统性故障需要人类审查？**每小节末尾给出安全文档（安全手册）的落点。用同一个框有两个目的：一是让你在换技术领域时带着熟悉的问题清单上手，而不是从零学一遍“这个领域的规矩”；二是暴露各领域真正的差异所在——问题相同、答案不同的地方，才是该领域的知识增量。

### A.7.1 数字元件与存储器 (§5.1, p68–88)

**故障模型。**非存储数字逻辑的永久故障：stuck-at（节点恒 0/恒 1）、开路（节点或互连断裂）、桥接（相邻信号意外连通，因电路结构呈线或/线与）、单粒子硬错误 SHE（单次辐射事件造成永久损伤，如栅氧破裂）；瞬态故障：单粒子瞬态 SET（节点上的瞬时电压尖峰）、单粒子翻转 SEU、单比特翻转 SBU、多单元翻转 MCU（一次事件翻多个通常物理相邻的位）、多比特翻转 MBU（同一字节/字内多位错——简单单纠 ECC 纠不了） (§5.1.2, p68–69)。NOTE 3 给出“等效覆盖”论证思路：针对 stuck-at 设计的机制可在有对应性理由时声称覆盖最终表现为 stuck-at 的桥接/开路故障。存储器故障模型另有一表（Table 29, p70）：Flash/ROM/OTP/eFUSE/EEPROM/RAM/DRAM 各配 stuck-at 加一族附加模型（stuck-open、耦合故障、寻址故障、过渡故障、邻域图形敏感故障、擦写扰动等）。Part 11 NOTE 1 说明，典型应力条件通常只能激活其中一部分，其他模型可在产线末端测试设备上激活，并要求用证据说明测试对应条件下的有效性；本书把该说明设为覆盖率主张的接收条件。

**失效模式的定义术。**§5.1.4 (p70) 给出四缺省键：功能缺失 (FM1, 该给不给)、功能误动 (FM2, 不该给却给)、时序错误 (FM3, 给早给晚)、数值错误 (FM4, 给错内容)。好的失效模式集有三判据：能把工艺故障映上来、能支撑诊断覆盖率评估、理想情况下各模式不相交。Table 30 (p71–74) 按此框架给出 CPU、中断处理、MMU、中断控制器、DMA、总线、SDRAM 控制器（并演示第二层抽象：行地址/列地址/命令/数据通道分列）、外部与嵌入式 Flash、SRAM、数据一致性、通信外设 (CAN/FlexRay/以太网/SPI)、信号处理加速器的示范定义——FMEDA 排到哪个模块，先来这张表对号。

**定量分析的假设。**永久故障：分层划分（工具可辅助检查跨模块边界的一致性与完整性，但不能单独保证语义无遗漏）→ 按面积或按门数摊失效率（CPU 占裸片 3% 面积即摊 3% 失效率）→ 分类为安全/残余/检出双点/潜伏双点故障 → 定失效模式覆盖率（可把寄存器组拆到单个寄存器逐个算再合并） (§5.1.7.1, p75–77)。瞬态故障单独立账 (§5.1.7.2, p77)：RAM 密度高，瞬态失效率常显著高于逻辑，Part 5 §8.4.7 NOTE 1 建议为 RAM 与其余部分**分开算度量**；安全故障占比是瞬态分析的主角——原文例子值得手算：一个每 10 ms 写一次、写后 1 ms 用一次的寄存器，瞬态故障只有落在那 1 ms 窗口内才可能违背安全目标，其余 9 ms 都是安全故障，安全占比 90%。瞬态不算潜伏故障覆盖——根因转瞬即逝，错误状态大概率在下一个下电周期被清除 (NOTE 2)。**需特别核对的限定：**Part 5 Annex D 的覆盖率表部分数值**只对永久故障有效**——RAM March 测试对永久故障覆盖“高”，对软错误覆盖为**零** (§5.1.7.2.2 EXAMPLE 2, p77)；因此引用 Annex D 数值时要同时标明故障类型与适用条件。

**安全机制清单** (§5.1.13, p85–88)：存储 ECC（检纠能力随冗余位数）、修正校验和、存储签名、块复制（双存储硬件或软件比对）、RAM 图形测试、奇偶位、RAM March 测试（通常只能在初始化或停机时运行，NOTE 2）、运行校验和/CRC（漏检概率  $1/2^n$ ,  $n$  为校验和位数；随存储增大覆盖率下降）。每条机制在原文都有“目标—描述—适用限制”三段，选型时连限制一起抄。

**系统性故障与文档。**§5.1.9 把 RTL 设计阶段的避错技术与 Part 5 表格挂接 (p78–82)，供数字 IP 的开发过程审查用；黑盒软 IP 的集成者还可参照 Part 6 相应表格类比处理编译产物 (§4.5.5 NOTE 3, p23)。安全文档形态见 §5.1.11 (p83)。

**判例走查：DMA 的覆盖率是怎么估出来的** (Annex A, p140–143)。这是 Part 11 里最适合手算的一个完整判例，把“失效模式定义 → 机制映射 → 覆盖率估算”三步走通。用例设定：通信外设每  $X$  ms 收一条消息，触发 DMA 请求；DMA 把消息从外设接收缓冲搬到固定 RAM 区，完成后触发 CPU 中断；CPU 再按消息 ID 把消息拷到另一块 DMA 无权访问的 RAM 区。四个安全机制站好位：专用存储保护单元（限制 DMA 只能写目的地址、读源地址）、端到端保护（8 位 CRC + 4 位消息 ID + 4 位消息计数器，16 个 ID 里只有 12 个合法）、超时监控、中断源合法性监控。

然后按 Table 30 的四键失效模式逐个对号。**DMA\_FM1 (该传不传)**：超时监控在时限内等不到传输完成信号，判例覆盖率估 100%。**DMA\_FM2 (不该传却传)** 拆成两个子模式，覆盖率的估法完全不同——DMA\_FM2.1 搬的是**上一条消息**：内容合



法但计数器/ID 对不上，端到端保护抓个正着，判例估 100%；DMA\_FM2.2 搬的是**随机值**（按“白噪声”建模，每个错误状态等概率）：只有当随机数据同时碰对合法 CRC、合法 ID、正确计数器，错误才会漏网。三个概率相乘： $p = (1/2^8) \times (12/16) \times (1/2^4) = (1/256) \times 0.75 \times (1/16) \approx 0.000\ 183$ ，覆盖率 =  $1 - p \approx 99.98\%$ （p141；数值两源核对一致）。手算复核： $0.75/4096 = 1.831 \times 10^{-4}$  ✓。DMA\_FM3（**时序错**）的子模式里，“取数期间被下一条消息部分覆盖”的最坏情形假设 ID 必然合法（ $p=1$ ）、计数器最坏也合法（ $p=1$ ），只剩 CRC 一道网：覆盖率 =  $1 - 1/2^8 \approx 99.6\%$ （p142）——保守估算取两个子模式中较低者。DMA\_FM4（**值错**）再细分：读写方向颠倒由端到端保护和存储保护单元检出，判例估 100%；访问宽度错按端到端校验估 99.6%；访问地址错由存储保护单元检出，估 100%；数据输出错按白噪声归到 99.98%；错误中断请求在该用例的单一中断源假设下由中断源监控检出，估 100%（p143）。这些 100%/99.98%/99.6% 均是 Annex A 特定架构、失效模式细分与概率假设下的估计，不是元件认证值，也不能直接迁移到其他设计。

这个判例教的不是那几个百分数，而是三条方法论：**覆盖率是逐失效模式、逐机制组合估出来的**，不存在“这个模块覆盖率 99%”的整体拍板；本书的接收策略是**只在明确写出随机模型时接受概率论证**（白噪声假设要写明），确定性检测（超时、地址保护）直接给 100% 时也注明机理；**子模式分布未知时取保守下界**。更精确的总覆盖率还要乘上 FM2.1/FM2.2 之间的失效分布——分布不知道，可参照 §4.3.3 的均匀假设并做敏感性分析。这三条是从 Part 11 判例提炼的 BookHousePolicy，不是判例自身产生的 shall。

### A.7.2 模拟与混合信号元件 (§5.2, p88–109)

**划分**。信号不限于数字状态的元素即模拟元素；混合信号元件至少含一个模拟与一个数字元素，建议明确分界，因为两者的设计、版图、验证、测试方法论与工具可能存在显著差异 (§5.2.1, p88)。切分子元素的判据：信号流（反馈 ADC—数字调节器—输出驱动构成的控制环）、连通性（基准与偏置电路服务多个模拟块）、工艺差异（HV 开关是 DMOS，栅驱动是常规 MOS——失效率可能差数量级、故障模型也不同）、电源域差异、频率分区（p89）。一条反内卷提示：**粒度更细未必让安全分析更好**——细到超过安全需求、安全机制与独立性证据的需要，只是徒增工作量（p89）。

**失效模式是语境的函数**。这是模拟节最重要的一课 (§5.2.2.1, p90)。同一个电压调节器：常识失效模式是过压/欠压，配 OV/UV 监控即可；但当它作为**传感器供电或 ADC 基准**，阈值之内的精度漂移与振荡同样致命——要靠独立 ADC 周期回测；当它给**射频模块**供电，纹波抑制比（PSRR）成了安全属性——阈值内的纹波与尖峰要低滤波兜底；当它给 MCU 内核供电且启动过快导致浪涌压降，软启动功能就是缓解措施。四个例子同一器件、四种失效模式清单——**先问“它在系统里当什么”，再开失效模式清单**。把失效模式判为非安全相关时要在安全分析中留理由；失效模式分布由半



导体供应商给出定量数据，且各模式分布之和按 100% 处理（NOTE 2）——这是定量分析能做加法的前提。Table 36（p91–98）给出各类模拟 part/subpart 的失效模式参考清单，可用来扩充 Part 5 Annex D。瞬态方面（§5.2.2.2，p98）模拟电路对软错误相对钝感，但混合信号里的数字辅助逻辑照旧要考虑。

**粒度的两难有原文判例**（§5.2.3.2，p99）。一个线性稳压器配窗式电压监控（ $1.2\text{ V} \pm 0.12\text{ V}$  之外算故障）：分析停在稳压器**输出端**，容易区分安全/危险失效并量化监控的保护效果，但难以量化每类失效的可能性；分析下钻到内部**带隙基准**，传播路径与可能性好分析了，监控对带隙本身的保护又难量化了。取舍规则：安全机制在系统/元素级时，往下钻只会徒增复杂度；要量化失效分布时，适度下钻反而更准——对稳压器五个失效模式做均匀分布，不如对带隙、缓冲、驱动这些 subpart 做均匀分布再合并（按 §4.2 术语：稳压器是 part，带隙/缓冲/驱动是 subpart）。分布来源两档（§5.2.3.3，p99–100）：初始分析用均匀分布（五个模式各 20%），细化时按根因电路的面积占比给。

**模拟元件的安全故障有天然来源**（§5.2.3.4，p100）。模拟信号是连续的，分配给模拟元件的安全需求容差往往比元件自身实际容差**松**——参数漂移落在需求容差之内的那部分失效就是安全故障。三个原文例子：限流电阻阻值漂移 50% 但仍能限流，安全；输出驱动的摆率控制只为 EMI 服务、与安全目标无关，其失效安全；稳压器只给数字电路供电时，阈值内的精度与稳定性失效安全——与本小节开头“给 ADC 做基准”的场景对照：**同一失效模式，换个下游就换一次分类**。

**安全机制**（§5.2.4，p102–105）：上拉/下拉电阻、过压/欠压监控、电压钳位、过流监控、限流、上电复位、模拟看门狗、滤波器、热监控、模拟内建自检、ADC 监控、ADC 衰减检测、ADC 通道卡死检测。**定量核查**：架构度量计算的验证单列一节（§5.2.3.7，p101）；混合信号的软错误只在数字部分按 5.1.7.2 处理——模拟电路的辐照软错误率测量困难，主要靠更细的分析替代（p99 NOTE）。**系统性故障**：开发阶段避错技术表（§5.2.5，p105–108）覆盖稳健设计、原理图检查等人工与工具手段。安全文档：与数字侧同构的“安全手册/安全应用笔记”，外加失效率计算信息（如晶体管数）与 AoU 描述（§5.2.6，p108–109）。

### A.7.3 可编程逻辑器件（§5.3，p109–124）

**对象**。PLD = 可配置 I/O + 非固定功能（逻辑块 + 用户存储 + 配置技术）+ 布线资源 + 固定功能（时钟/电源/复位，复杂器件还有硬核 CPU、存储控制器）（§5.3.1.1，p109–110）。类型从一次可编程的 PAL、可重编程的 GAL、非易失的 CPLD 到以易失 SRAM 配置为主的 FPGA（Table 42，p111）。配置技术是安全链的正式一环——易失与非易失工艺的安全能力不同（NOTE，p110）。

**生命周期的双主体裁剪**。PLD 的特殊性在于**制造者与使用者分担生命周期**（§5.3.1.3，p111–113）：PLD 制造者负责器件本身，PLD 用户在器件上实现自己的电路。Part 11 逐部给出适用性解释：Part 2 的活动可按双方贡献裁剪，相关项级 HARA

等活动通常不落在仅作为器件供应商的制造者；Part 3 通常不适用于制造者，除非其兼任相关项集成角色；Part 4 可在 SEooC 场景按上下文应用；Part 5 的相关活动由双方按各自贡献考虑；Part 6 主要涉及高层次综合流程（OpenCL、C 转 HDL、基于模型），传统 HDL 流程更多回到 Part 5；Part 7 对应配置烧录的生产控制。无内建机制时，指南建议制造者提供基础失效率、失效模式与分布，特定设计的度量由掌握实现上下文的用户计算；Table 52 又把配置正确性检查（如校验和）列为集成验证方法。以上是 Part 11 的裁剪指南；实际项目仍需按适用性逐项回到相关 Parts 的条款，不能把“Part 5 全部条款对双方适用”当成未经裁剪的总括 shall。

**故障模型与失效模式。**PLD 的失效模式表（Table 43, p114）按 Figure 25 的结构分块给：固定功能 IP 与数字/模拟 I/O 直接回指数字节的 Table 30、Part 5 Table D.1 与模拟节的 Table 36——固定功能通常独立成区，可参照普通数字/模拟元件的相应方法处理；**逻辑块**的失效模式是“所实现功能的永久/瞬态损坏”；**配置技术**的失效模式是“逻辑块配置被永久/瞬态意外改变”——瞬态项的相关性取决于 PLD 工艺。SRAM 配置器件与 Flash/反熔丝等非易失工艺的安全能力可能不同；反熔丝工艺是否对某类瞬态故障不敏感，仍须按具体器件工艺与供应商证据判断。**布线资源**的失效模式是“一组逻辑块共同实现的功能损坏（含时延变化）”，配置技术自身的连线也归入布线资源。这张表示范了一个建模要点：**同一物理故障（配置位翻转）在 PLD 里被拆成“功能损坏”与“配置改变”两个语义层**，因为两者的检测机制不同——前者靠功能级冗余或自检，后者靠配置校验和/回读。

**定量与 DFA。**PLD 裸片失效率可按 §4.6.2.1.1 的模型算，配置技术的晶体管既可单列一项、也可摊进逻辑块与用户存储（§5.3.3.1.1, p114–116）；瞬态失效率示例见 §5.3.3.1.2。失效模式分布的三种算法（Table 47, p118）值得对照：a) 均匀分布（四个失效模式各 25%）；b) 按各失效模式牵涉子部件**估计**消耗的逻辑块数加权（10/110 = 9.09% 起）；c) 按**实测**的资源贡献计算（细到“总线接口对两个数据类失效模式各贡献 50%、对配置失效模式贡献 100%”）——精度逐级上升，工作量也是。用户侧故障注入有个现实困难：不知道自己的设计被映射到哪些逻辑块。§5.3.3.1.4 的出路是在映射前的逻辑设计上注入（例如把用户 RTL 综合成参考网表再注入），但要论证“注入的故障相对假定实现是有意义的”（p118）。DFA 按 §4.7 的流程走，Table 48（p119）逐步列出制造者与用户各自的特殊性——布线资源与配置存储是 PLD 特有的共享资源。安全机制示例（§5.3.4, p120）与系统性避错（§5.3.5, p121–123）分制造者侧（工艺、基础架构）与用户侧（设计流程、支撑工具——工具置信度回指第 10 章 TCL 框架）。生产侧同样双主体：制造者按 Part 7 对 PLD 建产线监控与现场监控流程，用户参与相关项生产时同样适用；退役指令通常不适用（§5.3.1.3.7, p113）。完整定量判例在 Annex E（p177–188）。

### A.7.4 多核元件 (§5.4, p124–127)

**对象。**同构多核（相同处理单元）与异构多核（不同指令集架构的处理单元） (§5.4.1, p124–125)。Part 11 关心的场景：原本分配给多个元件的安全需求，现在挤进一颗多核。

**FFI 是主战场。**多 ASIL 软件共存于多核时，按 Part 9 Clause 6 做免于干扰分析——第 7 章提名、第 8 章深讲的 FFI 在此落地；故障清单以 Part 6 Annex D 为起点 (§5.4.2.2, p125)。三类干扰逐个给了机理与对策：**存储**——私有变量被其他核端翻（访问监督/独占机制）、非易失程序区被误编程（限高 ASIL 元素编程）、共享外设被搅（端到端保护，Part 5 Table D.6 一族）；**时间与执行**——低 ASIL 核连续占用 CAN 外设把高 ASIL 核饿死（时间监控，Table D.8 一族）；**信息交换**——非安全核的消息被当成安全消息（伪装故障，端到端保护）。Part 11 EXAMPLE 4 (p126) 资料性地示范：ASIL X 需求分解为两个软件监控元素 Y(X) 与 Z(X)，DFA 发现共享核、共享 RAM、共享驱动三处威胁独立性；示例用不同处理单元、MPU 和按初始 ASIL 开发的共享驱动处理这些依赖。其规范性内核来自 Part 9 §5.4.3：若用某措施控制已识别的相关失效原因，该措施须按初始安全需求的 ASIL 充分开发。**这不等于所有共享资源自动继承初始 ASIL**；先识别原因、再明确哪个元素承担控制措施，才由 §5.4.3 决定该措施的 ASIL。

**虚拟化的两面** (p126–127)：hypervisor/微内核可支撑软件分区论证（配合 Part 6 §7.4.9），但两条 NOTE 泼冷水——hypervisor 自身的系统性故障会瓦解隔离主张，其依赖的硬件资源（MMU）与共享资源的硬件故障照样要按 Clause 8 分析；**虚拟化通常不能充分预防或检测多核的永久/瞬态硬件故障**，只有当故障恰好表现为分区违例时才可能被逮到，且要逐例分析论证。**时序** (§5.4.2.3, p127)：多核天然易发时序故障，Part 6 的时序条款（软件安全需求含时序约束、资源上限估计、含执行时间）要配专门分析（最坏执行时间上界估计）与对策（看门狗、时序监督单元、软件对策）。

### A.7.5 传感器与换能器 (§5.5, p127–140)

**对象与故障模型。**换能器按 Part 1 §3.172 定义：把能量从一种形式转换为另一种形式的硬件部件；传感器 = 换能器 + 支撑/调理/处理其输出的硬件元素 (§5.5.1, p127)。传感器的失效模式清单 (§5.5.2, p128–130) 涵盖输出卡死、灵敏度漂移、偏置、动态范围异常等。与前四类技术最大的不同：**生产工序本身是一等故障源** (§5.5.2.1, p130)。晶圆减薄、划片、贴装、键合、堆叠、封装的机械应力会改变迁移率等材料特性，直接冲击偏置、灵敏度等技术规格；供应商产线校准过的传感器，到了下游客户的贴片、夹持、回流焊、三防漆工序又可能被重新应力化——**装配前没有的失效模式，装配后不保证没有**。Part 11 建议在可行时于各级供应商产线末端复验规格（Table 54 列出灵敏度漂移/丧失、偏置三类产线诱发异常及其成因）；是否设为项目放行条件需在策略中另行决定。

**MEMS 的机理案例** (§5.5.2.2, p130–133)：微机电传感器靠可动结构（质量块、弹簧、锚点、梳齿电容、密封腔）做机械-电转换，结构缺陷直接变成电学失效——弹簧断裂 → 非参数化灵敏度（部分行程正常、断裂侧失控）；梳齿断裂 → 总电容下降 → 灵敏度与偏置漂移；密封腔泄漏 → 阻尼气体压力下降 → 灵敏度上升乃至截止频率改变；膜片破裂 → 偏置或完全失敏（表现为对地卡死）；锚点断裂 → 质量块越界接触腔壁 → 卡死；导电/非导电颗粒 → 多种模式 (Table 55, p132)。A.4.2 讲过的**机械耦合 DFI** 在此有了具体面孔：特定共振冲击让加速度计梳齿粘连 (stiction)。

**分析与措施**。基础失效率的确定与分摊要特别小心 (§5.5.3.1, p133–134)：工业手册对 MEMS 机械结构的覆盖薄弱，常需依赖实验与现场数据路线；DFA 专节 §5.5.3.2 与定量分析 §5.5.3.3 相应展开。安全措施带强烈的领域色彩 (§5.5.4, p134–137)，三族各举其要：**结构性防御**——密封质量块滤波器（把颗粒挡在可动结构之外）、冗余膜片（一片破裂另一片维持功能）；**参数性防御**——偏置消除、自动增益控制、灵敏度调整（把装配应力造成的漂移在线拉回）；**检测性防御**——换能器专用自检（对可动结构施加电激励、验证机械响应，等于给“弹簧还在不在”做体检）、信号通路完整性测试、评估 MEMS 物理属性的测量手段。还有一类在前四种技术里没出现过的角色：**非 E/E 安全机制**——MEMS 特有的机械或光学机制可直接检测或控制换能器内部失效模式。不能由此推出它们“不消耗失效率预算”；特定用例下的诊断覆盖率需由领域专家进行有工程依据的评估，并将取值理由与验证活动完整记入安全论据。由于故障可能无法在换能后电子信号中直接观察，有效性也不应只靠电学注入主张，还要有与故障机理相配的领域分析与试验。系统性避错见 §5.5.5 (p138)，安全文档见 §5.5.6 (p139–140)。

五遍走完，问题框的答案汇总成一张速查表：

| 技术       | 故障模型的重心             | 机制改变了什么             | 定量分析的关键假设                         | 留给人类审查的系统性问题     |
|----------|---------------------|---------------------|-----------------------------------|------------------|
| 数字/存储    | stuck-at 家族 + 软错误家族 | 局部覆盖率在故障模型子集内按贡献加权  | 面积/门数分摊；RTL 瞬态与永久分账；March 对瞬态覆盖为零 | 设计避错、黑盒 IP 过程证据  |
| 模拟/混合    | 语境依赖的功能失效模式         | OV/UV 之外还要管精度/纹波/启动 | 分布之和 =100%；区域摊失效率                 | 原理图级稳健设计、人工检查    |
| PLD      | 配置技术 + 布线资源         | 配置校验进入安全链           | 制造者给分布，用户算度量                      | 支撑工具置信度 (TCL)    |
| 多核       | 干扰三类（存储/时序/信息）      | MPU/OS 可成为安全机制      | 控制相关失效原因的措施按初始 ASIL               | hypervisor 系统性故障 |
| 传感器/MEMS | 机械结构缺陷 → 电学失效       | 机械自检与非电子机制          | 手册不覆盖机械结构                         | 全链装配应力复验         |



表格之外还有一条纵向观察：五类技术的“定量分析关键假设”一列，本质上都是 A.3 节某条纪律的领域化身——数字的面积/门数分摊对应 §4.6.2.4 的分布方法，模拟的“分布之和 = 100%”对应定量加法的前提，PLD 的“制造者给分布、用户算度量”对应 IP 责任分工，多核的“共享资源先经 DFA 判断、承担控制措施者再按 Part 9 §5.4.3 确定 ASIL”对应依赖分析与规范义务的衔接，传感器的“手册不覆盖机械结构”对应来源适用性论证。**§4 是纲，§5 是目**；本书建议先读透 §4 的四大块，再把 §5 视为这四大块在五个领域的投影。这是作者组织阅读的建议，不是 Part 11 规定的学习顺序。

## A.8 本体化实践：数值诚实与依赖链的半导体延伸

本节把附录 A 的两条主线——失效率数字与相关失效依赖——接到全书两条对应线索上：数字要能作证（第 6、16 章）与依赖要看得见（第 8、18 章）。按唯一执行语义红线，以下三元组都是**未注册的 Semantica package delta 蓝图**，不引用书内共享 TBox，也不构成可运行资产。未来 package 必须把标准来源事实与 EPS 合成项目事实放在不同的命名图/资产角色中，并分别绑定来源与 TeachingExample 身份。Part 11 锚点只能支持“共享时钟被指南列为 DFI 示例”，不能证明“EPS MCU 实际共享某棵时钟树”。

**第一件事：把 Part 11 的示例数值按 ch06 范式对象化。**第 6 章的教训是“没有来源坐标与校订留痕的数字不可信”。下面是本书拟采用的 BookHousePolicy 表达，不是 Part 11 的数据格式要求：

```
# 建模蓝图（未落库）：Part 11 Table 2 裸片失效率示例
iso262:Clause_11_4_6_2_1_1 a iso262:Clause ;
    iso262:clauseId "11-4.6.2.1.1" ;
    iso262:sourcePdfPage 28 ;    # 1-based 物理页
    iso262:hasEvidenceFragment iso262:Fragment_11_Table2_DieFIT ;
    iso262:sourceArtifact "structured/mineru/ISO-26262-2018/part-11-semicon]
    ductor-guidelines/native-full/ISO 26262-11-2018/auto/ISO
    26262-11-2018_content_list_v2.json" .

iso262:Fragment_11_Table2_DieFIT a iso262:SourceFragment,
iso262:InformativeStatement ;
    iso262:fragmentId "11-table2-die-failure-rate" ;
    iso262:hasRuleBasis iso262:ISOInformativeGuidance ;    # 全卷 informative
    iso262:sourcePdfPage 34 ; iso262:sourceBlock 6 ;
    iso262:sourceBbox "58,426,880,580" ;
```



```
iso262:sourceArtifact "structured/mineru/ISO-26262-2018/part-11-semicon_
ductor-guidelines/native-full/ISO 26262-11-2018/auto/ISO
26262-11-2018_content_list_v2.json" ;
rdfs:comment "示例 MCU 裸片失效率 1,25 FIT; pdftotext 与 MinerU HTML
是同一 PDF 的两种提取视图, 回看原表后将欧式小数逗号按 ocrCorrected
规范化为 1.25。"@zh ;
iso262:ocrCorrected true .
```

三处纪律有意为之：页码、块号、bbox 与工件路径实提自 MinerU JSON（本附录写作时已核对）；hasRuleBasis iso262:ISOInformativeGuidance 让下游查询看见这个数字**不是规范性要求**；原文的欧式小数逗号（1,25）属于会被下游脚本误读的表示形式，按第 6 章的 ocrCorrected 范式存规范化值并留痕。本书计划让 A.3 节每个关键数各配一条 SourceFragment，执行“一个数字一个锚点”；这是来源治理 BookHousePolicy，不是 Part 11 shall。

**第二件事：把 DFI 指南与 ch08 的项目证据链分桶。**标准来源桶先登记 Part 11 Table 21 的资料性分类事实；它只支持“共享时钟是一类 DFI 示例”：

```
# 来源注册表蓝图（未落库）：标准事实，不含 EPS 架构事实
iso262:Clause_11_4_7_5_1 a iso262:Clause ;
  iso262:clauseId "11-4.7.5.1" ;
  iso262:sourcePdfPage 53 ; iso262:sourceBlock 10 ;
  iso262:hasEvidenceFragment iso262:Fragment_11_Table21_SharedClockDFI ;
  iso262:sourceArtifact "structured/mineru/ISO-26262-2018/part-11-semicon_
ductor-guidelines/native-full/ISO 26262-11-2018/auto/ISO
26262-11-2018_content_list_v2.json" .

iso262:Fragment_11_Table21_SharedClockDFI
  a iso262:SourceFragment, iso262:InformativeStatement ;
  iso262:fragmentId "11-table21-shared-clock-dfi" ;
  iso262:hasRuleBasis iso262:ISOInformativeGuidance ;
  iso262:sourcePdfPage 55 ; iso262:sourceBlock 0 ;
  iso262:sourceBbox "115,120,937,683" ;
  iso262:sourceArtifact "structured/mineru/ISO-26262-2018/part-11-semicon_
ductor-guidelines/native-full/ISO 26262-11-2018/auto/ISO
26262-11-2018_content_list_v2.json" ;
  rdfs:comment "Table 21 将公共 PLL、时钟树、时钟使能等失效列为共享资源 DFI
示例；不证明任何具体项目存在该拓扑。"@zh .
```

按本书建模蓝图，项目教学桶从 DFA 活动一路建到验证活动，并避免把执行或评审状态塞在原因对象上：

```
# EPS 教学 ABox 蓝图（未落库）：项目事实全部显式标 TeachingExample
eps:DFA_EPS_MCU_SharedClock_Activity
  a iso262:DependentFailureAnalysisActivity, iso262:TeachingExample ;
  iso262:activityStatus iso262:Draft ;
  iso262:dfaExecutionStatus iso262:AnalysisPlanned ;
  iso262:produces eps:DFA_EPS_MCU_SharedClock ;
  iso262:exampleStatus " 教学计划；DFA 尚未执行。"@zh .

eps:DFA_EPS_MCU_SharedClock
  a iso262:DependentFailureAnalysis, iso262:TeachingExample ;
  iso262:reviewStatus iso262:Draft ;
  iso262:evidenceStatus iso262:EvidenceCandidate ;
  iso262:hasDFATopicAssessment eps:DFAAssessment_EPS_RandomHardware ;
  iso262:hasDFAFinding eps:DFAFinding_EPS_SharedClock ;
  iso262:hasDFAConclusion iso262:DFAConclusionUndetermined ;
  iso262:exampleStatus " 教学工作产物；不存在已完成分析或独立性结论。"@zh .

eps:DFAAssessment_EPS_RandomHardware
  a iso262:DFATopicAssessment, iso262:TeachingExample ;
  iso262:assessesDFATopic iso262:DFATopicRandomHardwareFailures ;
  iso262:dfaTopicDisposition iso262:DFATopicPlausibleCauseIdentified ;
  iso262:topicPlausibilityRationale
    "教学假定存在由双核共享时钟导致的共同影响路径；待真实架构证据确认。"@zh ;
  iso262:topicImpactRationale "教学假定该路径可能同时破坏主核与检查核；
    尚未完成影响分析。"@zh .

eps:DFAFinding_EPS_SharedClock
  a iso262:DFAFinding, iso262:TeachingExample ;
  iso262:arisesFromTopicAssessment eps:DFAAssessment_EPS_RandomHardware ;
  iso262:findingPlausibility iso262:DFAFindingPlausible ;
  iso262:findingPlausibilityRationale "预登记为教学候选原因；
    真实项目仍须用架构与故障传播证据评价。"@zh ;
  iso262:findingImpactRationale "教学假定可能同时影响锁步双核；
    影响范围与安全目标后果尚未验证。"@zh ;
  iso262:findingClosureStatus iso262:DFAFindingOpen ;
  iso262:identifiesDependentFailureCause eps:DFI_EPS_MCU_SharedClock ;
```

```

iso262:resolvedByMeasure eps:Measure_EPS_IndependentClockMonitor .

eps:DFI_EPS_MCU_SharedClock
  a iso262:DependentFailureCause, iso262:TeachingExample ;
  rdfs:label "EPS 教学 MCU 锁步双核的假定共享时钟树"@zh ;
  iso262:causeStatement " 共享时钟故障可能同时影响主核与检查核。"@zh ;
  iso262:hasSourceProvision iso262:Fragment_11_Table21_SharedClockDFI ;
  iso262:exampleStatus "共享时钟拓扑是合成教学假设；Part 11 片段只支持 DFI
  分类。"@zh .

eps:Measure_EPS_IndependentClockMonitor
  a iso262:DependentFailureResolutionMeasure, iso262:TeachingExample ;
  iso262:hasRuleBasis iso262:ProjectSpecificStrategy ;
  iso262:hasMeasureVerificationActivity eps:Verify_EPS_ClockMonitor ;
  iso262:exampleStatus "候选措施；尚无有效性证据，不表示 finding 已关闭。
  "@zh .

eps:Verify_EPS_ClockMonitor
  a iso262:DependentFailureMeasureVerificationActivity,
  iso262:TeachingExample ;
  iso262:activityStatus iso262:Draft ;
  iso262:verificationExecutionStatus iso262:VerificationPlanned ;
  iso262:verifies eps:Measure_EPS_IndependentClockMonitor ;
  iso262:produces eps:ClockMonitorVerificationReport_Template ;
  iso262:exampleStatus " 验证尚未执行。"@zh .

eps:ClockMonitorVerificationReport_Template
  a iso262:DependentFailureMeasureVerificationReport,
  iso262:TeachingExample ;
  iso262:reviewStatus iso262:Draft ;
  iso262:exampleStatus " 报告模板；没有测试记录或结论。"@zh .

```

候选 conceptualization 用 `rdfs:domain/range` 表达属性两端的预期语义：`arises FromTopicAssessment` 对应 `DFAFinding→DFATopicAssessment`，不是 `Cause→Topic`；`identifiesDependentFailureCause` 对应 `Finding→Cause`，措施通过 `Finding` 的 `resolvedByMeasure` 接入；活动、DFA 执行和措施验证状态必须使用不同属性。`domain/range` 只会推导类型，不会拒绝误用；若未来注册 `Semantica package`，禁用、枚举、基数与闭环条件必须进入包内 `SHACL`，且补齐正反例与 `exact oracle`。下面的 `SELECT` 只是书面投影，

当前不能作为可执行门禁调用：

```
SELECT ?dfa ?finding ?dfi ?guidance ?measure ?dfaExec ?closure
?verificationExec WHERE {
  ?dfa a iso262:DependentFailureAnalysis ;
      iso262:hasDFAFinding ?finding .
  ?finding iso262:arisesFromTopicAssessment ?assessment ;
      iso262:identifiesDependentFailureCause ?dfi ;
      iso262:findingClosureStatus ?closure .
  ?assessment iso262:assessesDFATopic ?topic .

  OPTIONAL {
    ?dfi iso262:hasSourceProvision ?guidance .
    ?guidance a iso262:InformativeStatement ;
        iso262:hasRuleBasis iso262:ISOInformativeGuidance .
  }
  OPTIONAL {
    ?dfaActivity a iso262:DependentFailureAnalysisActivity ;
        iso262:produces ?dfa ;
        iso262:dfaExecutionStatus ?dfaExec .
  }
  OPTIONAL {
    ?finding iso262:resolvedByMeasure ?measure .
    OPTIONAL {
      ?measure iso262:hasMeasureVerificationActivity ?verification .
      ?verification iso262:verificationExecutionStatus ?verificationExec .
    }
  }
}
```

这条查询只能回答三类“图中声明了什么”的问题：某个 DFI 是否链接 Part 11 informative provision？Finding 声明了什么闭环状态、链接了什么措施？DFA 与措施验证分别声明了哪个执行状态？它不检查措施的 ASIL、验证报告的批准状态或证据充分性，因此不能回答“真的闭环了吗”。OPTIONAL 未绑定只表示当前图中缺失或未知，不自动构成违规；只有在某个明示的 **BookHousePolicy** SHACL Shape 将该字段或证据链设为特定放行场景的条件时，未绑定才会成为待补项或校验结果。

注册 Semantica package 之前，给未来集成者留一份本书**交接检查单**（全部为 **BookHousePolicy**）：① Part 11 来源事实与 EPS TeachingExample 进入不同资产角色/命名图；② 每个来源片段记录页、块、bbox、受控来源哈希并回看原

文；③ fragment 标 InformativeStatement + ISOInformativeGuidance，合成实例标 ‘TeachingExample + exampleStatus’；④ 保持 Assessment→Finding→Cause 与 Finding→Measure→Verification；⑤ 分开 AnalysisPlanned、VerificationPlanned、Draft、EvidenceCandidate 与 DFAFindingOpen；⑥ CQ、Shape、反例、oracle、capability 声明和 source/wheel lock 必须作为同一个 Semantica package revision 评审。本附录文本本身不授权修改机器语义。

**边界声明收口：**本节全部实例为未注册教学蓝图；AnalysisPlanned/ VerificationOnPlanned/EvidenceCandidate 不是完成，informative provision 不是项目事实或规范性依据，未来知识门禁即使通过也不是 ISO 验证——三个“不是”，一个都不能少。

## A.9 要点回顾与思考题

### 要点回顾：

1. Part 11 全卷资料性，不新增 shall；具体义务回到 Parts 2-9 的适用规范性条款判定，术语还需回到 Part 1。
2. 划分粒度是安全概念的函数：机制越“包圆”划分可越粗，机制越“挑食”可信分析所需粒度通常越细；“无缝隙”是 Part 11 指南、本书接收策略，不是新增 shall。
3. IP 有四条资料性集成路径、两种类别；AoU 是待核销的债务清单；黑盒 IP 可借 DIA 分配责任，具体义务回到适用的 Parts 2-9。
4. 基础失效率只算随机、不掺诊断；四类来源假设各异，跨源比较先对齐机理与口径；任务剖面与工作/日历小时口径可能是高敏感输入。
5. DFI 七分类中，Table 25/26 未为制造与安装两类列出防违背措施，这不等于所有设计都不存在运行时缓解；B1-B12 是带回路的流水线，能够可靠量化的随机硬件 DFI 可进入定量分析，其余依赖仍需定性处理。
6. 本书把故障注入操作化为七元组实验对象；这是 BookHousePolicy，用于守住复现链。
7. 五类技术共用一个问题框：故障模型—机制—定量假设—人类审查项。

### 思考题（依学习目标设置）：

1. 【概念辨析】某 IP 供应商的安全手册声称“本 IP 的 SPFM 为 99.2%”。结合 A.2.4 与 A.3.1，列出按本书接收策略在接受这个数字之前应核对的至少四项前提，并说明哪几项属于 AoU 核销、哪几项属于失效率口径。



2. 【工程推导】教学场景：一颗 MCU 的某寄存器每 20 ms 刷新一次，刷新后 2 ms 内被安全相关计算读取一次。仿照 §5.1.7.2.2 的原文示例，估算该寄存器瞬态故障的安全故障占比，并说明这个估算依赖的两个假设。
3. 【工程推导】用 A.4.3 的速览表分析：一个双核锁步系统的两个核共享同一片上电压调节器。该依赖属于哪类 DFI？给出一条防违背措施与一条防发生措施，并说明各自的验证方法应从 §4.7.5.2 的哪一族里选。
4. 【工程推导】教学场景：一个 DMA 通道的端到端保护改为 16 位 CRC、8 位消息 ID（其中 200 个合法）、8 位计数器。仿照 A.7.1 判例的白噪声模型重算“数据传输无请求（随机值）”失效模式的覆盖率，并与原文 99.98% 比较，说明哪个参数的改动贡献最大。
5. 【动手题】按 A.8 的建模蓝图，为 A.3.4 中 SN 29500 示例的“105 FIT→6.3 FIT”这对数值写出两条 SourceFragment 三元组（含物理页码；仅在发生表示校订时标 ocrCorrected），并写一条 SPARQL 查询，找出所有“声明发生 OCR/表示校订却没有校订留痕”的 Part 11 数值片段。产物可留作后续综合案例与本体化配套材料的输入。

A.10 回正文索引

| 本附录内容                               | 回指章节        | 衔接点                 |
|-------------------------------------|-------------|---------------------|
| component/part/subpart 层次 (A.2.1)   | ch02        | element 概念家族的芯片内延伸  |
| fault→failure mode 挂接与覆盖率乘法 (A.2.2) | ch02 / ch06 | 因果链；诊断覆盖率主张         |
| AoU 核销、SEooC (A.2.3/A.2.4)          | 路径 ch10     | SEooC 首讲与跨阶段复用边界    |
| IP 工作产物、DIA、黑盒 IP (A.2.4/A.2.5)     | ch10 / ch03 | DIA 深讲；确认措施框架       |
| 基础失效率与 λ 分类、FMEDA 输入链 (A.3)         | ch06        | λ 五分类、数值诚实、PMHF 输入链 |
| PMHF 目标值的相对意义 (A.3.1)               | ch06        | PMHF/EEC 两条评估路线     |
| DFI 七分类、B1–B12 工作流 (A.4)            | ch08        | DFA、独立性证据、共存分析      |
| 相关失效控制措施按初始 ASIL (A.7.4)            | ch08        | ASIL 分解义务三分法        |

| 本附录内容                       | 回指章节        | 衔接点                  |
|-----------------------------|-------------|----------------------|
| 故障注入与验证计划 (A.5)             | ch05 / ch06 | 集成测试方法表；覆盖率证据        |
| 生产全线安全相关、控制计划<br>复用 (A.6.1) | ch09        | 特殊特性、控制计划、现场闭环       |
| 分布式开发双重角色 (A.6.2)           | ch10        | DIA a)-k) 框架         |
| PSA 与确认措施裁剪 (A.6.3)         | ch03        | 确认措施 I0-I3 与 Table 1 |
| PLD 工具链置信度 (A.7.3)          | ch10        | TI/TD/TCL 判定         |
| 多核 FFI 三类干扰 (A.7.4)         | ch07 / ch08 | FFI 提名与深讲            |
| 数值诚实与依赖链建模 (A.8)            | ch16 / ch18 | 测量证据本体与依赖本体          |

\* 本附录素材：ISO 26262-11:2018 物理页 p9-188，关键数值与 A.8 中的来源坐标已用 `pdftotext -layout` 与 `MinerU content_list_v2.json` 交叉核对；Part 11 为资料性指南，本附录不以其替代 Parts 2-9 的适用要求，术语依据还需回到 Part 1。\*



## 附录 B 摩托车适配与 T&B 边界索引：

### Part 12 改了什么、为什么

Semantica 边界：本附录保存 Part 12 的可读适配说明、两张表的人工转述与教学走查；书内不再保存或运行 TBox、表实例、SPARQL、SHACL、fixture 或测试脚本。`semantica.chapter_packages.vol2.ch04` 只承接本书第 4 章声明的 HARA 场景；当前 `semantica.chapter_packages.vol2.normative` 只登记已经迁移的 Part 1/Part 3 工程释义资产，并不声称覆盖 Part 12 全表。因此本附录的 36 格 MSIL 表、5 行映射表及专项门禁目前没有 released executable successor：任何机器化声称必须先进入新的 Semantica package revision，未完成前保持 unsupported/blocked，绝不调用书内旧路径。

**导读。**正文第 4 章的 HARA、S/E/C 与 ASIL 判定都以乘用车为默认舞台。ISO 26262-12:2018（下称 Part 12）回答的问题是：同一套标准搬到摩托车上，哪些条款要换、换成什么、为什么非换不可。读完本附录，你应当能：1. 说清 Part 12 的适用边界与“替代关系”——它改写了 Parts 2-4 的哪几处，又如何与其余适用要求和工作产物衔接；2. 独立走一遍摩托车 HARA：用与第 4 章相同的 S/E/C 骨架得出 MSIL，再经映射表落到 ASIL，并解释 MSIL 与 ASIL 为什么不是同义词；3. 说明卡客车（T&B）在 Part 12 里的真实地位——只有标记约定，没有适配体系——以及这对想做 T&B 项目的团队意味着什么。

**来源性质声明：**与全卷资料性的 Part 11 不同，**Part 12 的正文条款（Clause 4-10）是规范性的**，含 shall 要求，对适用的摩托车相关项**取代**其他部的对应要求（§4.5，物理页 p12）；其 Annex A/B/C 为资料性判例与说明（p32、p38、p46）。本附录引用的页码为 1-based 物理页，锚点曾用两种独立生成的提取视图交叉定位；两种视图来自同一份 PDF，不构成两个独立来源。

**案例边界：**本附录自行构造的判定走查均标注“教学示例”（TeachingExample）；Annex B/C 的示例是资料性判例，不得直接复制为任何项目的 HARA 结论。

**与全书二十章的衔接：**本书前十章现为叙事重写，方法表语义、独立性等级记号、FMEDA 与失效分类等器具性内容已集中由附录承载；正文各章只提供活动与主

题的工程语境。本附录因此按自足深讲编写——凡指向“第 X 章”处均指主题语境，不意味着正文有逐表逐条的展开。

## B.1 导读：同一套标准为何需要车型适配

先看一个会把乘用车直觉带进沟里的场景（教学示例）。一份合成的乘用车电子油门 HARA 记录，先把“非预期减速（相当于发动机制动）”在拥堵市区的可控性写成 C1；这个等级只是叙事输入，不代表任何车型或数据库结论。现在同一个功能装上摩托车，团队顺手沿用了那行判定。评审——把工作摊在桌上让同行挑毛病的会议——上被一位骑手出身的工程师拦下：摩托车过弯时车身是**倾斜**的，轮胎侧向力与纵向力共享附着着力预算；弯道中段突然出现的发动机制动会改变纵向载荷与侧向平衡，骑手需要在有限反应窗口内完成判断和姿态修正。这个反问只说明两轮与四轮不能机械复用同一可控性等级；是否改级、改多少，必须由具体场景、车辆与骑手证据支撑。

这就是 Part 12 存在的理由，浓缩成三条摩托车事实：**动力学不同**——两轮车靠骑手参与才能维持稳定，车辆响应与骑手动作构成一个耦合系统，Part 12 §8.2 明说摩托车的动态行为与标准范围内的其他车辆差异巨大（p20）；**暴露的人不同**——骑手没有座舱、安全带和气囊，标准假设的防护只有头盔、护具这类穿戴装备（§8.4.3.2 NOTE 5, p22）；**行业基线不同**——§8.2 的 NOTE 直言：摩托车行业的产品开发过程与技术方案不同于汽车行业，全球已确立的技术水平（state-of-the-art）表明**直接套 ASIL 分级并不合适**，因此 HARA 的输出改为 MSIL，再经对齐关系映射回 ASIL，以便继续使用其他部的要求（p20）。

注意 Part 12 的克制：它**没有**重写生命周期。Parts 2-9 构成完整基线，Part 12 在五个技术域提供摩托车专用替代要求：安全文化（Clause 6）、确认措施（Clause 7）、HARA（Clause 8）、整车集成与测试（Clause 9）、安全确认（Clause 10）。这不表示其他要求机械地“原样全做”：仍可按 §4.2 通过安全生命周期裁剪证明某要求不适用，或对可接受的不符合给出并评价理由。本附录按“改了什么、为什么”逐处走查，最后交代 T&B 的边界。

## B.2 适配总则与适用边界

**替代关系**（§4.5, p12）：对适用 Part 12 要求的摩托车相关项或元素，**Part 12 的要求取代其他部的对应要求**。这是一条精确的外科手术式规则——取代的只是“对应”的那几条，不是整部标准。§5.2（p12）把对应关系点了名：Part 12 的摩托车专用要求对应 Part 2 的 5.4.2（安全文化）、Part 2 的 6.4.9（确认措施）、Part 3 的 Clause 6 与 Annex B（HARA）、Part 4 的 7.4.4（整车集成测试）与 Part 4 的 Clause 8（安全确认）。除此之外，Parts 2-9 仍是基线，并继续受 §4.2 的裁剪与可接受不符合规则约束



——这才是“完整基线 + 五处技术域差分”的准确结构;“五处”不表示 Part 12 的 Clause 4/5 没有规范作用。

**适用范围** (Clause 1, p9): 对象是量产道路车辆上含一个或多个 E/E 系统的安全相关系统, **不含轻便摩托车 (moped)**, 也不处理为残障驾驶者等特殊车辆设计的独有 E/E 系统。已量产或在本文件发布前已经开发的系统及组件不纳入本版范围, 但既有系统的后续变更、以及既有系统与按本文件开发系统的集成, 仍要按变更或集成情形裁剪安全生命周期。范围只覆盖安全相关 E/E 系统失效行为直接造成的危害; 电击、火灾、烟、热、辐射、毒性等危害若并非由这类失效直接导致, 则不在本文件范围内; 名义性能也不在范围内。四个摩托车专用术语在 Part 1 里定义、在 Part 12 里使用 (§5.2 NOTE, p12): **expert rider** (专家骑手)、**motorcycle** (摩托车)、**MSIL** (Motorcycle Safety Integrity Level, 摩托车安全完整性等级)、**CCP** (Controllability Classification Panel, 可控性分类小组) ——后三个是本附录的主角。

**T&B 的真实地位** (§4.6, p12): 原文只有一句话——“Content that is intended to be unique for trucks, buses, trailers and semi-trailers (T&B) is indicated as such.”即: 卡车、客车、挂车与半挂车的**专属内容会被显式标记**。这是一条标记约定, 不是一套适配体系。ISO 26262:2018 第二版把商用车纳入范围的方式, 是在 Parts 1-9 的正文里就地写入 T&B 专属条款并加标记, 而**不是**像摩托车那样单独立一部适配标准。因此: “Part 12 = 摩托车适配 + T&B 一句标记约定”; 想做 T&B 项目的读者, 需要的是跨 Parts 1-9 的 T&B 条款清单, 本附录 B.7 交代这份清单的建法与现状, 但不伪造一部不存在的“卡客车指南”。

迷路时有一张官方地图: **Annex A** (资料性, p32-37) 按“条款—目标—前提—工作产物”四列给出摩托车实施 Parts 2/3/4 的总览与工作流。Table A.1 (p32-34) 摆安全管理: Part 2 Clause 5 的总体安全管理在列, 其下“本文件 Clause 6 安全文化”占据对应位置; 项目级安全管理一栏里“本文件 Clause 7 确认措施”对应 Part 2 Clause 6 的相关输出 (影响分析、安全计划、安全档案、确认措施报告、量产放行报告)。Table A.2 (p34-35) 摆概念阶段: 相关项定义仍走 Part 3 Clause 5, “本文件 Clause 8”替代 Part 3 Clause 6 的 HARA 位置, 功能安全概念仍走 Part 3 Clause 7。Table A.3 (p36-37) 摆系统级开发: 技术安全概念走 Part 4 Clause 6, “本文件 Clause 9”对应整车集成、“本文件 Clause 10”对应安全确认。这三张表把“替代关系”画成了拼图: **Part 12 的五个专用领域放在哪个位置、与哪些前提和工作产物衔接, 可按表格查询**。Annex A 由此支持的结论是对应接口和工作产物继续适用; 具体项目的产物是否齐全、前提是否满足, 仍须逐项评审。

还有一条读表规则要带上 (§4.3-§4.4, p11): Part 12 沿用全系列的方法表语义——第 7 章讲过的 ++/+/o 推荐度、连续条目与备选条目; 括号 ASIL 的含义也一致: **写成 ASIL (A) 的条款对该 ASIL 是建议而非要求**, 与 ASIL 分解的括号记号无关。B.5 节的四张测试方法表会用到这条。

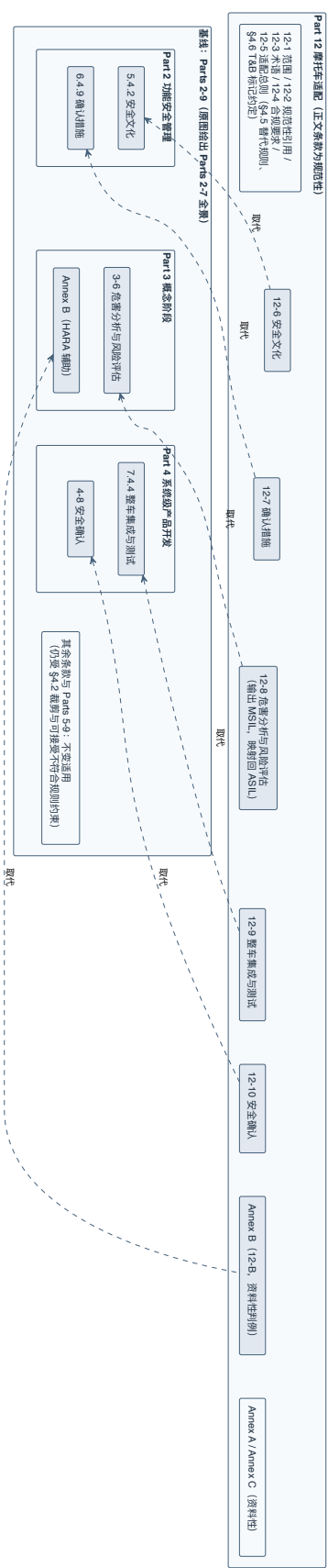


图 B-1 Part 12 与其他部的替代关系（重绘自 Part 12 Figure 2）——五个技术条款加 Part 3 Annex B 由 12-6 至 12-10 与 12-B 六条对应逐一取代，其余条款按基线不变适用；重绘压缩了原图的分部全景，对应关系数量与方向保持原图。

## B.3 安全文化与确认措施的摩托车适配

### B.3.1 安全文化：义务不打折

Clause 6 是对 Part 2 §5.4.2 的裁剪 (§6.1, p13)，但读完会发现“裁剪”几乎没有减项：组织应创建、培育并维持支持摩托车功能安全有效达成的安全文化 (6.2.1)；应建立并维持组织级规则与流程 (6.2.2)；**适用时**，应在功能安全、网络安全及其他可能相互作用、且与达成功能安全相关的学科之间建立有效沟通渠道 (6.2.3)；应按 Part 8 Clause 10 执行文档管理 (6.2.4)；应提供所需资源 (6.2.5)；应建立持续改进机制 (6.2.6)；应赋予安全责任人足够权限 (6.2.7) (p13-14)。对照第 3 章的安全文化 18 判据：两轮车企业不因产品小而自动免除文化义务，但每一条仍须保留原文适用条件。真正的差分在下一小节。

### B.3.2 确认措施：同一张表，少一列 D

Clause 7 定义与 ASIL 挂钩的确认措施独立性要求 (§7.1, p14)。第 3 章首讲过确认措施三件套——确认评审、功能安全审核、功能安全评估——与独立性等级 I0-I3。Part 12 的关键动作是：**对摩托车，Part 12 的 Table 1 取代 Part 2 的 Table 1** (7.2.1 a) NOTE 1, p14)。

两张表并排看，主要活动与独立性等级的组织骨架相似，但不能直接视为同一矩阵；至少要核对三类差别。**其一，列头变了**：Part 2 Table 1 的独立性列是 QM/A/B/C/D 五列；Part 12 Table 1 只有 **QM/ASIL A/ASIL B/ASIL C 四列** (p16-18)——没有 D 列。结合 B.4 的 Table 6 可作一个教材解释：MSIL 最高映射到 ASIL C，因此摩托车 HARA 路线不会给安全目标分配 ASIL D；但“因此少 D 列”是跨条款推论，不是 Table 1 旁的标准原句。**其二，部分阶梯在 C 列截顶，而不是所有行整体下移**：影响分析与 HARA 确认评审仍然在 QM/A/B/C 全列 I3；其他行则没有 Part 2 的 D 列。以代表行对照（教学归纳；Part 2 侧数值见第 3 章 Table 1 矩阵）——

| 确认措施                            | Part 12 (QM/A/B/C) | Part 2 差分 (QM/A/B/C/D)               |
|---------------------------------|--------------------|--------------------------------------|
| 影响分析确认评审                        | I3/I3/I3/I3        | I3 全列<br>不变——判“新旧相关项”错了会波及一切，独立性不让步  |
| HARA 确认评审                       | I3/I3/I3/I3        | I3 全列<br>不变——HARA 是一切下游的地基           |
| 安全计划/FSC/TSC/安全分析与 DFA/安全档案确认评审 | —/I1/I1/I2         | 对应行大体—<br>少了 D 列的 I3 顶格 /I1/I1/I2/I3 |
| 集成测试策略、安全确认规格确认评审               | —/I0/I1/I2         | 类似梯度加 D 列<br>同上                      |
| 功能安全审核、功能安全评估                   | —/—/I0/I2          | 梯度加 D 列<br>ASIL A 免做、B 档 I0 (做则须他人做) |

其三，范围列写进了 MSIL 语汇：HARA 确认评审的范围包括判断“确定的 ASIL、QM 评级以及导致无 ASIL 的参数（如 C0/S0/E0）是否正确”（p16）——评审对象是“从 MSIL 映射来的 ASIL”链条整体，映射本身也在受审之列。

独立性等级的定义原文重述了一遍（Table 1 脚注 a，p18），与第 3 章一致：一无要求无建议；I0 应当（should）执行，若执行则须由工作产物创建者以外的人执行；I1 必须由他人执行；I2 必须由不同团队的人（不向同一直接上级汇报）执行；I3 必须由不同部门或组织的人执行。Figure 3（p19）给了公司组织结构的示意。还有两条实操 NOTE 值得抄：确认措施报告要含所分析工作产物的名称与版本号（NOTE 6，p15）；相关项在确认措施完成后发生变更的，相应措施要重做或补充（NOTE 7，p15）；确认评审、审核可与评估合并执行以支持相似变体（NOTE 8，p15）——变体管理正是摩托车行业车型繁多的现实。

## B.4 摩托车 HARA 与 MSIL 判定

### B.4.1 骨架不变：还是第 4 章那条流水线

Clause 8 是 Part 12 的技术核心，对应 Part 3 Clause 6。先说继续适用的主要步骤，免得读者以为要重学一遍 HARA：HARA 基于相关项定义（8.4.1.1）；**评估时不考虑相关项内部的安全机制**——已实现或将实现的安全机制属于功能安全概念，不进 HARA（8.4.1.2，p20）；运行场景与运行模式要覆盖正确使用与可合理预见的误用（8.4.2.1）；危害用 FMEA/HAZOP 等方法系统性识别、在**整车层面**定义（8.4.2.2/8.4.2.3）；后果要合并评估多功能同时丧失的情形——供电失效可能同时丢“发动机扭矩”与“前照

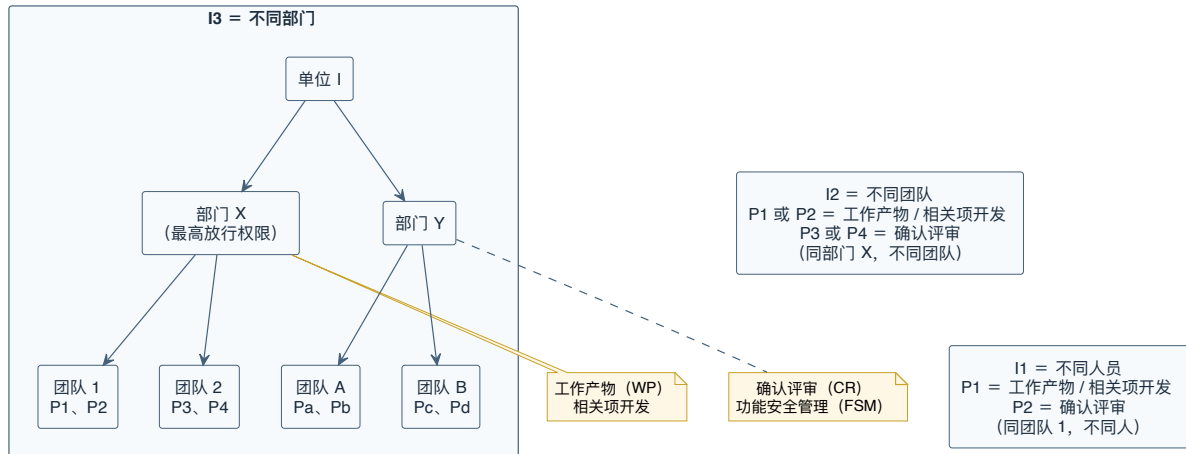


图 B-2 确认评审独立性等级 (重绘自 Part 12 Figure 3) ——I1 不同人员、I2 不同团队、I3 不同部门：工作产物创建方与确认评审方在组织树上的距离即独立性等级，最高放行权限所在部门做开发、另一部门做评审即 I3。

明” (8.4.2.6 EXAMPLE, p21)；运行场景切分过细会不当压低完整性等级，要聚合相似场景 (8.4.2.7, p21)；分类有疑义时保守取高 (8.4.3.1 NOTE, p21)。这些共通步骤之后，仍须按 Part 12 核对摩托车专用的分类语境、MSIL 输出和映射要求。

预期功能之外的危害同样出界：仅由相关项行为 (无失效) 引发的危害不在范围内——正常摩托车不被期望高速穿越无铺装路面、不被期望参加公路赛或越野赛 (8.4.2.1 NOTE 2 与 EXAMPLE, p21)；超出 ISO 26262 范围的危害按组织自有程序处理 (8.4.2.4)。

#### B.4.2 S/E/C：同一副度量衡，换了持秤人

三个分类参数采用与 Part 3 相同的类目集合：严重度 S0-S3 (Table 2, p22)、暴露概率 E0-E4 (Table 3, p23)、可控性 C0-C3 (Table 4, p24)。共同类目让 MSIL 与 ASIL 的表格关系可以逐项表达；摩托车适配的差异则落在判定语境、证据和后续映射：

**严重度**：风险评估聚焦每个暴露于风险的人——包括肇事车辆的骑手与乘员，以及骑行者、行人、其他车辆乘员 (8.4.3.2 NOTE 1, p22)；伤害刻画可用 AIS (Abbreviated Injury Scale, 简明损伤定级) ——B.6 展开；**标准穿戴装备 (头盔、护甲、手套、靴子——按用户手册规定) 假定在用** (NOTE 5, p22) ——这条乘用车没有对应物：四轮的被动安全在车上，两轮的被动安全穿在人身上。三条通用规则保留：多处伤害可合并抬级 (NOTE 2)；“差值定级”规则照搬 Part 3——若危害事件只是让本已发生的事故雪上加霜，严重度可取“有失效”与“无失效”两种结局之差 (8.4.3.3 的气囊例：不弹出的气囊对应 S3，正常弹出对应 S2，差一级，故该失效可判 S1, p22)；纯财产损失判 S0、免除 MSIL 定级 (8.4.3.4)。



**暴露概率：**装车率不得计入——评估假定每辆车都装了该相关项，“不是每辆车都有这个功能”不构成降 E 的理由（8.4.3.6，p23）；E0 留给“难以置信”的情形（不可抗力如地震、高速路上迫降的飞机），须记录排除理由，判 E0 免除 MSIL 定级（8.4.3.7，p23）。

**可控性：**这是摩托车 delta 最深的一处。评估对象是骑手**或其他卷入场景的人**获得足够控制、避免特定伤害的概率（8.4.3.8，p23）；骑手画像有明确假设——状态适宜（不疲劳）、持照且受过训练、了解所骑车辆的操作特性、遵守法规（NOTE 2，p23——对照第 4 章乘用车的“有驾照的普通驾驶员”假设，这里多了“了解车辆特性”一条，因为摩托车之间的动力学差异远大于乘用车之间）；与车辆运动方向/速度无关的危害（如肢体卷入运动部件），可控性改为评估当事人自行脱困或被他人解救的概率（NOTE 3）；涉及多个交通参与者时可综合故障车的可控性与其他人的假定动作（NOTE 4）；**摩托车危害事件的可控性评估方法单列 Annex C**（NOTE 5）——专家骑手与 CCP 在那里登场，B.6 展开；有官方最低性能法规且有实际使用证据支撑时，法规可作为选级理由的一部分（NOTE 6/7，p23）。C0 的用法与乘用车一致：不影响车辆安全运行的功能丧失（部分骑手辅助系统）或常规骑手动作即可避免事故的情形，判 C0 免除 MSIL 定级（8.4.3.9，p24）。

B.4.3 MSIL 判定表与映射表：两张表、两种关系

**MSIL 判定**（8.4.3.10，Table 5，p24）：按 S×E×C 查表；当前转录中的 36 个 S1-S3×E1-E4×C1-C3 单元与 Part 3 Table 4 具有相同索引结构，结果词表则使用 QM/MSIL。最低组合给 QM，S3+E4+C3 顶格给 MSIL D。下表是当前转录；转录所据的两种提取视图同出一份 PDF，关键使用仍应回看原表：

| 严重度 | 暴露 | C1 | C2 | C3 |
|-----|----|----|----|----|
| S1  | E1 | QM | QM | QM |
| S1  | E2 | QM | QM | QM |
| S1  | E3 | QM | QM | A  |
| S1  | E4 | QM | A  | B  |
| S2  | E1 | QM | QM | QM |
| S2  | E2 | QM | QM | A  |
| S2  | E3 | QM | A  | B  |
| S2  | E4 | A  | B  | C  |
| S3  | E1 | QM | QM | A  |
| S3  | E2 | QM | A  | B  |
| S3  | E3 | A  | B  | C  |
| S3  | E4 | B  | C  | D  |

单看这张表，你会以为 MSIL 就是 ASIL 换了个字母。真正的 delta 在下一步。**映射** (8.4.3.11, Table 6, p25)：在定义安全目标**之前**，MSIL 须映射为 ASIL，以便采用 ISO 26262 其他部的要求——

| MSIL | ASIL |
|------|------|
| QM   | QM   |
| A    | QM   |
| B    | A    |
| C    | B    |
| D    | C    |

不要用“整体降一档”代替查表。Table 6 给出的五行事实是：**QM→QM、MSIL A→QM、MSIL B→ASIL A、MSIL C→ASIL B、MSIL D→ASIL C**。把后四行口头概括成“字母向下一格”只能算作者帮助记忆的说明，它会漏掉 QM→QM，并容易把两个分级体系误说成同一把标尺。三条 NOTE 是这张小表的全部注释，逐条读：映射出的 QM 不等于没事——对应危害事件仍可能有安全后果，仍可为其制定安全需求，QM 只表示质量流程足以管理该风险 (NOTE 1)；映射的目的是让摩托车应用以**最恰当的严格程度**避免不合理残余风险 (NOTE 2) ——背后是 §8.2 NOTE 的行业基线论证；映射出的 ASIL 是**最低要求** (NOTE 3, p24-25) ——想按更高等级开发没人拦着。

为什么设置这张对齐表？标准给出的依据要克制地复述。§8.2 正文指出摩托车动态行为与其他车辆差异很大，摩托车危害事件的可控性更强调骑手，因此风险评估方法需要适配；紧随其后的 NOTE 又说明，摩托车行业的开发过程与技术方案不同，全球既有技术水平表明直接使用 ASIL 分类并不合适，于是以 MSIL 作为 HARA 输出，再建立 MSIL-ASIL 对齐以使用其他各部要求并容纳行业能力。标准没有把年行驶里程、季节性、恶劣天气回避或“已吸收高风险所以再用 ASIL 会过度要求”列为这张表的正式因果链；这些可以成为研究假设，不能替标准发言。项目能做的是逐行采用 Table 6 并把映射结果作为最低要求，不能以“本行业不同”为由自创另一张映射表。

**MSIL 与 ASIL 不是同义词**，教材必须把这句话钉死。它们类目同构、判定表同构，但：MSIL 是摩托车 HARA 的原生输出，ASIL 是全系列其他部的通用语言；两者之间隔着一张**不可逆**的映射表（从 ASIL 反推 MSIL 是多值的：ASIL QM 可能来自 MSIL QM 或 MSIL A）；确认措施的独立性查 Part 12 Table 1 时用的是映射后的 ASIL。建模时 (B.7) 也因此不能把 MSIL 写成 ASIL 的别名。

**B.4.4 安全目标与验证：出口对齐第 4 章**

安全目标的确定沿用第 4 章的主要结构，但增加了摩托车映射与确认语境 (8.4.4, p25)：每个带 ASIL（从 MSIL 映射）的危害事件确定一个安全目标，相似安全目标可合并、取最高 ASIL；安全目标按 Part 8 Clause 6 规格化，可含容错时间间隔或物理

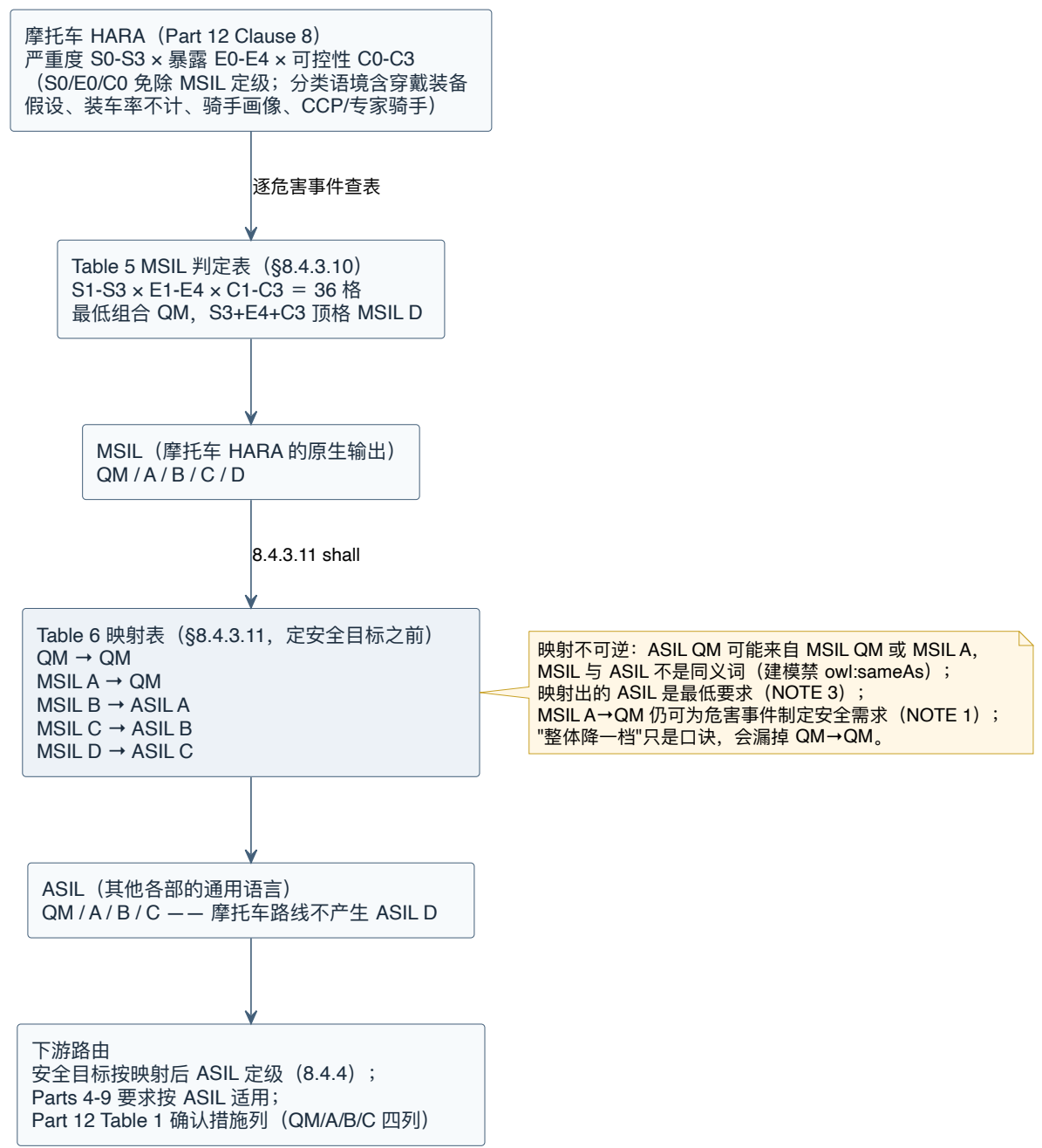


图 B-3 MSIL→ASIL 两表两种关系——S×E×C 经 Table 5 三十六格判定 MSIL，定安全目标之前经 Table 6 五行（QM→QM、A→QM、B→A、C→B、D→C）映射为 ASIL 再路由下游；映射不可逆、结果为最低要求，摩托车路线不产生 ASIL D。

特征（如非预期加速度上限）；HARA 中使用或产生的假设（骑手动作、外部措施）要识别并在 **Clause 10 的安全确认中对集成后的相关项验证**（8.4.4.4, p25）。验证要求（8.4.5.1, p25）列了六项证据，前五项与 Part 3 对应，第六项是摩托车特有的 **MSIL-ASIL 映射一致性**。工作产物为 HARA 报告与其验证报告（8.5, p26）。

**教学走查**（教学示例，非任何真实项目结论）：摩托车牵引力控制系统，危害事件“正常过弯时非预期扭矩削减”。为演示查表，先显式冻结三项合成输入：E4 的依据只采用 Annex B 明列的“normal cornering”或“slight lean angle or less”；S2 假设来自指定车速、碰撞对象与 AIS 伤情谱分析；C2 假设来自按 CCP 程序形成的代表性骑手可控概率证据。后两项在本附录没有真实事故数据库、速度包络或 CCP 记录，因此只是待证输入，不能说由 Annex B 自动推出。在**给定这组教学输入**的前提下，查 Table 5 得 **MSIL B**，再查 Table 6 得 **ASIL A**，随后可按 ASIL A 查 Parts 4-9 的适用要求和 Part 12 Table 1 的确认措施列。这个算例演示的是特定输入下的查表、映射和后续路由步骤，不确认教学输入在真实项目中成立。

## B.5 整车集成、测试与安全确认

### B.5.1 整车集成与测试：四个测试目标，两条摩托车脚注

Clause 9 裁剪 Part 4 §7.4.4 (§9.1, p26)，只动整车集成这一层——第 5 章讲过的集成三层（软硬件、系统、整车）中，前两层照旧走 Part 4。要求主干：相关项集成入整车并执行整车集成测试（9.2.1.1）；相关项与车内通信网络、供电网络的接口规格要验证（9.2.1.2, p26）；测试目标由四张方法表承载（9.2.2, p26-28），列头都是 ASIL A/B/C——又一次，没有 D 列：

- **Table 7 功能安全需求的正确实现**（p27）：基于需求的测试、故障注入测试、长期测试、真实条件用户测试，四法对 A/B/C 全部 ++；
- **Table 8 安全机制的功能表现、准确性与时序**（9.2.2.3，适用于 ASIL (A)(B) 和 C——括号即“对 A、B 是建议”）：性能测试、长期测试、用户测试 +/+ /++，故障注入、错误猜测、现场经验派生测试 o /+ /++（p27）；
- **Table 9 内外接口实现的一致与正确**（9.2.2.4）：内部接口、外部接口、交互/通信测试，均 +/+ /++（p28）；
- **Table 10 整车级稳健性**（9.2.2.5）：资源占用测试、应力测试、环境条件抗扰与稳健性测试（含 EMC/ESD）、长期测试，均 +/+ /++（p28）。

四张表的脚注还顺带给了方法定义，几条值得抄进测试计划模板（p27-28）：**基于需求的测试**针对功能与非功能需求；**故障注入测试**经专用测试接口或特制元件把故障

引入相关项；**长期测试与真实条件用户测试**类似现场经验测试，但样本更大、由普通用户执行、不受预定测试场景约束；**错误猜测测试**用专家经验与既往教训定向设计测试；**性能测试**验证整车层面的容错时间间隔与故障存在时的车辆可控性。摩托车味道正在脚注里：长期测试或用户测试可能不可行；9.2.2.1 NOTE 3 也说明存在骑手安全顾虑时，可以改选测试方法或把活动移到其他子阶段。这里不能把 §4.3 缩成“说明原方法不可行即可”：替换连续条目的方法时，理由必须说明替代方法为何满足对应 requirement；选取备选条目组合时，也必须说明组合或单一方法为何满足对应 requirement。骑手风险解释了为什么换，合规理由还必须解释换完后为什么仍够。

## B.5.2 安全确认：把 HARA 的假设押回整车上验

Clause 10 裁剪 Part 4 Clause 8 (§10.1, p28)。目标两条：证明安全目标在相关项集成进车辆后达成；证明功能安全概念与技术安全概念对该相关项是恰当的。与验证的分工 (§10.2, p29) 沿用第 5 章讲过的口径：验证证明“结果符合规格”，确认证明“适合预期用途”——面向一类或一组代表性车辆确认安全措施充分性。

确认环境 (10.4.1, p29)：安全目标在**整车层面的代表性语境**中确认；代表性车辆按车型与配置选取，HARA 报告可作为选取依据；还要考虑影响技术特征的使用变化。确认规格 (10.4.2, p29–30) 三要素：①受确认的相关项配置（含标定数据，按 Part 6 Annex C；全配置确认不可行时选合理子集）；②**确认程序、测试用例、骑行机动动作与接受准则**；③设备与环境条件——“骑行机动动作” (riding manoeuvres) 写进规格是摩托车特色：确认场景以机动动作为单位定义。执行 (10.4.3, p30) 要评估四件事：**可控性**（用含预期使用与可预见误用的运行场景确认；接受准则可以是“安全状态下的可控性充分”，且单一准则未必够，NOTE 1–3）、外部措施的有效性、其他技术要素的有效性、以及 **HARA 中影响 ASIL (从 MSIL 映射) 的假设**——8.4.4.4 埋的那条线在此收口：机械部件被假定能缓解某 E/E 失效的，其有效性要在整车层面验证 (EXAMPLE, p30)。

方法集 (10.4.3.4, p30–31)：可重复测试（正向测试、黑盒测试、仿真、边界条件测试、故障注入、耐久、应力、高加速寿命试验 HALT、外部影响仿真）、分析 (FMEA/FTA/ETA/仿真)、长期测试（车辆行驶计划、捕获车队——**对目标用户的长期测试可能不可行**）、真实条件用户测试/盲测/专家小组（**用户测试可能不可行，真实条件可用仿真条件替代**，NOTE 2, p31）、评审。评价 (10.4.4, p31)：确认结果要给出“实现的安全目标为相关项达成功能安全”的证据。工作产物：确认规格（含环境描述）与确认报告 (10.5)。

一句对仗收束 B.5：测试核对车是否按规格造对，确认评价规格和集成结果是否适合骑手的预期使用。



## B.6 附录判例: 严重度、暴露度与可控性怎么给证据

Part 12 的三个资料性附录是判定证据的“教具库”。用法纪律先立: Annex B 开门就声明其示例 for information only 且不穷尽 (B.1, p38); 示例代表既往事故分析的期望值, 不能从个别描述导出普遍结论 (B.2.1, p39)。判例的价值在“证据如何支撑分类”的示范, 不在数字本身。

**风险的解剖** (B.1, p38):  $R = F(f, C, S)$ ——风险是危害事件频度、可控性与严重度的函数; 频度  $f$  又拆成“处于可能发生该事件的场景中的概率”(即  $E$ ) 与“相关项故障率”, 后者**不进 HARA**——完整性等级恰恰是用来事后约束故障率的, 这与第 4 章的乘用车逻辑完全一致。

**严重度判例** (B.2, p39–40): AIS 把单一伤害分成七级——AIS 0 无伤到 AIS 6 极危重/致命 (p39 逐级列出: AIS 1 皮外伤/肌肉痛, AIS 2 深部伤/短暂昏迷/单纯骨折, AIS 3 重伤不危及生命, AIS 4 危及生命但可能存活, AIS 5 存活不确定, AIS 6 极危重如高位颈椎伤及脊髓); 也可换用 MAIS、ISS、NISS 等量表, 取决于当时的医学研究水平。Table B.1 (p40) 把 S 类目翻译成**概率化的伤情谱**:  $S0 = \text{AIS 0}$  且 AIS 1–6 概率小于 10% (或纯财产损失);  $S1 = \text{AIS 1–6}$  概率大于 10% (且不到  $S2/S3$ );  $S2 = \text{AIS 3–6}$  概率大于 10% (且不到  $S3$ );  $S3 = \text{AIS 5–6}$  概率大于 10%。配套的情景例: 步行速度撞路侧设施属  $S0$  档; 市区车速撞路侧设施、步行速度撞行人属  $S1$  档; 主干道车速撞路侧设施、市区车速撞行人、无二次撞击的高速低侧滑属  $S2$  档; 高速公路车速的碰撞、主干道车速被侧面撞击属  $S3$  档。读法示范: 给某危害事件判  $S2$ , 理由不该写“感觉挺严重”, 而应写“该场景的典型后果谱系对应‘主干道车速与固定物碰撞’, 参照事故数据库该类事故 AIS 3–6 概率超过 10%”。

**暴露判例** (B.3, p40–43):  $E$  的两种估法——按**持续时间** (该场景时长占总运行时间比例:  $E2 < 1\%$ ,  $E3$  为  $1\%–10\%$ ,  $E4 > 10\%$ , Table B.2, p42) 或按**频度** ( $E1$  大多数骑手一年不到一次,  $E2$  一年几次,  $E3$  平均骑手一月一次以上,  $E4$  几乎每次骑行, Table B.3, p42–43)。同一场景两种口径可能给出不同档位 (如在停车场骑行), 取对所分析危害**更合适**的口径。判例的颗粒度值得学: 倾角本身就是分档变量——大倾角  $E1$ 、中倾角  $E2$ 、小倾角  $E3$ 、微倾角  $E4$ ; 路面条件——冰雪  $E1$ 、湿滑树叶/砂石/油污  $E2$ 、雨雾湿路  $E3$ ; 机动动作——故意翘头  $E1$ 、紧急制动  $E2$  档、超车  $E3$  档、正常过弯  $E4$  档。还有一条针对“按需动作”设备的公式 (p41): 故障可能长期潜伏的 (如安全气囊类按需设备), 暴露概率按  $\sigma \times T$  估—— $\sigma$  是场景发生率,  $T$  是故障未被察觉的时长 (可长达车辆寿命), 该近似在乘积很小时有效; 且 HARA 不考虑相关项自带的安全机制来缩短  $T$  (NOTE 2, p41)。

**可控性判例** (B.4, p43–45): Table B.4 把  $C1/C2/C3$  钉在两个分位点上—— **$C1$ : 严格多于 99%** 的代表性骑手或其他交通参与者能避免伤害;  **$C2$ : 90%–99%**;  **$C3$ : 不足 90%** (p44)。示例矩阵按“危害  $\times$  运行场景 (括号内为避害动作)”组织, 几行典型: 失去牵引力——起步加速时 (收油、拉离合) 判  $C0$  档, **压弯加速时** (收油、反打方向、

制动)判 C3; 非预期减速(相当于发动机制动)——拥堵市区(骑手拉离合、他车制动)判 C0, 过弯中(拉离合)判 C1; 非预期最大制动——拥堵市区(他车可制动)判 C1, 弯道中(反打方向、重心转移)判 C3; 前轮非预期抱死——接近停车线(重心转移)与弯道中(转向、重心转移)都判 C3; 失去前照明——夜间无照明乡道(减速停车、开远光)判 C1。对照 B.1 的开场故事: **同一危害换一个机动动作, 可控性可以从 C0 跳到 C3**——这就是“运行场景与危害配对分类”而非“给功能整体打分”的原因。NOTE 2 又把边界收紧一圈: 这些示例假定中型公路摩托车, 评估时要按所考虑车型与性能复核 (p44)。

**可控性评估技术** (Annex C, p46-49): 核心概念是**可控性分类小组 (CCP)**——一个可按项目裁剪人数与构成的评估小组, 专长覆盖摩托车可控性评估(由专家骑手执行)、车辆动力学、E/E 系统、功能安全、骑手行为五个方向 (C.2, p46); 多组织可共担 CCP, 概念阶段即可由 CCP 评估可控性分类, 需有理由支撑。为什么不能照搬汽车的“代表性驾驶员群体实测”? C.3 说得直白 (p47): 摩托车动力学对人的依赖远超乘用车——稳定性、轨迹与姿态都靠骑手参与维持, 骑手的控制行为也与汽车驾驶员截然不同, 所以业界通行做法是**由专家骑手试乘并判断“代表性骑手能否应对”**——专家骑手有经验与技巧处理相当极端的危害事件, 但其安全要有充分的风险控制 (C.3/C.5, p47-48)。专家骑手不设资格认证标准, 由厂商/测试机构/供应商按内部程序选拔, 选拔参考项: 多年全场景骑行经验、掌握公司标准化可控性分级、有评估经验、能把测试结果**折算到代表性骑手**、能用技术语言讨论结果、受过公司骑手培训或持公司专家骑手认定 (C.4, p47)。四种评估技术 (C.5, p47-48): 代表性骑手群体评估(仅适用于不影响稳定性/轨迹/姿态的低风险事件, 如电加热把手); 专家骑手评估 (Annex C 将其说明为使用频度较高的方法, 建议多名专家骑手交叉); 骑行模拟器(真人骑手 + 物理机电动力学台架); 数学建模与仿真(车辆动力学 + 骑手/控制器模型全软件)。Annex C 还说明, 对专家骑手也不能以可接受安全水平执行的机动, 应分类为 C3 (C.5, p48)。评估维度三张小表 (C.6, Table C.1-C.3, p48-49): 车辆响应与性能变化(无/轻微/中等/重大)、骑手感知与警觉(不可察觉/可察觉不惊扰/可察觉且惊扰/强烈惊扰, 及骑手动作时机从无关紧要到性命攸关)、控制行为(无需改变/正常补偿动作足够/需超出正常补偿/需非凡技巧或超常控制力)——给 C 分级写理由时, 可从这三个维度组织证据; 具体分级仍要按项目对象、场景和评估记录确认。

## B.7 本体化实践: MSIL 表的 Semantica 候选合同与未完成边界

本节把 Part 12 的两张分级表接到第 4 章的判定表模式上, 但 \*\* 不复用 Part 3 专属的 `iso262:ASILMapping` 契约 \*\*。旧本体仓库曾有一套表模型、来源锚、Shape、查询和反例; 它们不在现行书稿中, 也没有登记为当前 Semantica built-in package。所以

下文只保存候选 conceptualization 和验收分母, 不能描述为“已经落库”或“机器门禁已通过”。

**先分开分类值与关系类型。** Table 5 继续使用既有的 iso262:Severity、iso262:Exposure 与 iso262:Controllability 受控值, 但输出另建为 iso262:MSIL\_A 至 iso262:MSIL\_D; iso262:QM 可作为两张表的输入或输出, 却既不是 MSIL, 也不是 ASIL。iso262:MSIL 与 iso262:ASIL、iso262:QMClassification 保持类型不相交。两张表也各有自己的关系类: Table 5 的单元属于 iso262:MSILDeterminationMapping, Table 6 的行属于 iso262:MSILToASILMapping。因此不得把任一单元写成 iso262:ASILMapping, 也不得用 owl:sameAs 把 MSIL 与 ASIL 合并。

**若进入 Semantica, 应把两张表登记成两种关系。** 候选 iso262:MSILTableRegistry\_2018 分别指向 Table 5 与 Table 6。验收分母必须明确: Table 5 是 S1-S3 × E1-E4 × C1-C3 的 **36 格**; Table 6 是 **5 行**: QM→QM、MSIL A→QM、MSIL B→ASIL A、MSIL C→ASIL B、MSIL D→ASIL C。以下 Turtle 只是

书面建模蓝图, 不是当前 registry 中可加载的资产:

```
iso262:MSIL_T5_S2_E4_C2 a iso262:MSILDeterminationMapping ;
    iso262:inMSILDeterminationTable iso262:MSILDeterminationTable_12_5 ;
    iso262:msilMappingSeverity iso262:S2 ;
    iso262:msilMappingExposure iso262:E4 ;
    iso262:msilMappingControllability iso262:C2 ;
    iso262:msilDeterminationResult iso262:MSIL_B ;
    iso262:hasSourceAnchor iso262:Table_12_5 .

iso262:MSIL_T6_B_A a iso262:MSILToASILMapping ;
    iso262:inMSILToASILTable iso262:MSILToASILTable_12_6 ;
    iso262:mappingMSILSource iso262:MSIL_B ;
    iso262:mappingASILResult iso262:ASIL_A ;
    iso262:hasSourceAnchor iso262:Table_12_6 .
```

正式迁入时, 每张表还必须绑定用户合法持有来源中的物理页、块号与坐标。当前公开书稿只陈述人工核对过的来源位置, 不声称存在 released 机器来源账。即使未来锚点与表单元同时通过, 也仍不会产生项目合规结论。

**候选验收必须用查询、全量预期集和单因反例守住转录。** 一个合格的未来 Semantica package 至少要检查唯一 registry、36/5 的精确数量、表归属、来源锚点、属性单值性、S/E/C 组合唯一性、五行映射和 QM/MSIL/ASIL 类型边界, 并拒绝 **MSIL\_\*** owl:sameAs **ASIL\_\***。CQ 仍可分三层: 单链 **S3/E4/C3→MSIL D→ASIL C**、36 行 exact bindings、5 行 exact bindings; 反例至少覆盖缺格、重复格、错误结果、缺锚点、

缺行和类型偷换。这些是待注册合同，不是已经执行的 oracle；紧凑分值计算也只能是本书实现选择，不能冒充 ISO 的通用公式。

这里必须把能力边界钉死：**\*\* 当前没有 released Part 12 package，也没有摩托车项目危害事件 ABox、HARA 报告、验证报告或 §8.4.5.1 f) 的执行状态实例。\*\*** 因而现阶段连“给定 S/E/C 时两张机器表返回什么”都不能作为 Semantica 执行结果报告，只能回到正版标准人工核对；更不能回答某项目映射是否已验证。

其余两块仍保持规划态，不能借 Table 5/6 的完成度代替：

| 模块                          | 当前状态              | 边界                                                                   |
|-----------------------------|-------------------|----------------------------------------------------------------------|
| Part 12 Table 5/6 MSIL 表模型  | planned / blocked | 36+5 验收分母保留在书中；尚无 released Semantica package                         |
| “基线条款 → 适配 delta→ 摩托车域适用性”链 | planned           | Clause 4.5 – 10 目前主要是 anchor_only 来源坐标；其余 provision、CQ 与 SHACL 尚未对象化 |
| 跨 Parts 1-9 的 T&B 专属条款账本    | planned           | Part 12 §4.6 只提供标记边界；完整条款盘点、覆盖状态与门禁尚未建立                              |

这一区分决定了当前能发布什么：可以发布本附录的人工阅读快照；不能发布“两张表的受控机器转录已就绪”“Part 12 全部适配 delta 已可查询”或“T&B 适配指南已经完成”。

## B.8 要点回顾与思考题

### 要点回顾：

- Part 12 是规范性适配：完整基线（Parts 2 – 9）+ 五处差分（文化/确认措施/HARA/整车集成/安全确认），替代关系精确到条款。
- Part 12 Table 1 的列是 QM/A/B/C，没有 D；各确认措施的 I0–I3 要逐行读取。“这与 Table 6 最高映射到 ASIL C 相呼应”是教材解释，不是 Table 1 自述的因果结论。
- 摩托车 HARA 骨架与第 4 章一致；S/E/C 类目不变，判定语境变：穿戴装备假设、装车率不计、骑手画像、专家骑手评估。
- Table 6 的五行是 QM→QM、MSIL A→QM、B→ASIL A、C→ASIL B、D→ASIL C；这是摩托车域的逐行映射事实，不是可向其他车辆域泛化的“降级算法”，MSIL 与 ASIL 也不是同义词。



5. 整车测试与确认的方法表没有 D 列, 长期/用户测试对摩托车可能不可行; HARA 假设应在安全确认中闭环验证, 但本书的表模型并不证明某项目已经完成该闭环。
6. Annex B/C 是判例教具: AIS 概率化伤情谱、双口径暴露估计、90%/99% 可控性分位点、CCP 与专家骑手。
7. Table 5 的 36 格与 Table 6 的 5 行是未来 Semantica package 的精确验收分母; 当前尚未 released, 适配 delta 语义链也仍为 **planned/blocked**。
8. T&B 在 Part 12 里只有标记约定; 跨 Parts 1-9 的 T&B 条款覆盖账本仍为 **planned**, 建成前不能输出完整指南。

#### 思考题 (依学习目标设置):

1. **【概念辨析】**“MSIL B 的系统按 ASIL A 开发, 等于摩托车行业给自己放水。”用 §8.2 NOTE、Table 6 的三条 NOTE 与 B.4.3 的论证结构反驳或限定这句话, 并指出哪一步论证只有标准委员会有资格做。
2. **【工程推导】**教学场景: 摩托车电子悬架的危害事件“高速直线行驶中阻尼突变为最硬”。参照 Annex B 的判例组织方式, 为 S、E、C 各写一条带证据类型的判定理由 (不必给出最终等级), 并说明其中哪一条最需要 CCP 介入。
3. **【工程推导】**某确认测试计划把 Table 8 中 ++ 的故障注入测试替换为台架仿真。按 §4.3 的读表规则与 9.2.2.1 NOTE 3, 写出这个替换要补的两段论证。
4. **【合同设计题】**把下面的 SPARQL 当作未来 Semantica Part 12 package 的**候选查询蓝图**, 人工推演  $S2+E4+C2 \rightarrow MSIL\ B \rightarrow ASIL\ A$ : Table 5 侧使用 `inMSILDeterminationTable`、`msilMappingSeverity`、`msilMappingExposure`、`msilMappingControllability`、`msilDeterminationResult`, Table 6 侧使用 `inMSILToASILTable`、`mappingMSILSource`、`mappingASILResult`, 并返回两侧 `hasSourceAnchor`。说明 package 尚未 released 时为什么不能把人工推演写成查询命中; 未来即使 exact oracle 通过, 也只证明登记图与冻结预期在已编码字段及联接上相符, 转录忠实仍须对照受控来源, 且结果不能证明 S/E/C 项目输入成立或 §8.4.5.1 f) 已执行。

```
PREFIX iso262: <https://w3id.org/iso26262#>
SELECT ?determination ?msil ?conversion ?asil ?anchor5 ?anchor6 WHERE {
  iso262:MSILTableRegistry_2018
    iso262:hasMSILDeterminationTable ?table5 ;
    iso262:hasMSILToASILTable ?table6 .
  ?determination a iso262:MSILDeterminationMapping ;
```



```
iso262:inMSILDeterminationTable ?table5 ;
iso262:msilMappingSeverity iso262:S2 ;
iso262:msilMappingExposure iso262:E4 ;
iso262:msilMappingControllability iso262:C2 ;
iso262:msilDeterminationResult ?msil ;
iso262:hasSourceAnchor ?anchor5 .

?conversion a iso262:MSILToASILMapping ;
iso262:inMSILToASILTable ?table6 ;
iso262:mappingMSILSource ?msil ;
iso262:mappingASILResult ?asil ;
iso262:hasSourceAnchor ?anchor6 .

}
```

B.9 回正文索引

| 本附录内容                                     | 回指章节           | 衔接点                        |
|-------------------------------------------|----------------|----------------------------|
| 完整基线 + 五处差分的结构 (B.2)                      | ch01/ch03–ch05 | 生命周期总览；被替代条款的宿主章           |
| 安全文化七条义务 (B.3.1)                          | ch03           | 安全文化 18 判据                 |
| Part 12 Table 1 取代 Part 2 Table 1 (B.3.2) | ch03           | 确认措施 I0–I3 与 55 格矩阵        |
| HARA 骨架、S/E/C、切香肠 反模式 (B.4.1/B.4.2)       | ch04           | HARA 首讲、判定表模式              |
| MSIL 判定表与映射表 (B.4.3)                      | ch04           | ASIL 判定表；分级方案变体            |
| 安全目标合并与规格化 (B.4.4)                        | ch04/ch05      | SG 出口；Part 8 Clause 6 需求规格 |
| 整车集成测试四表 (B.5.1)                          | ch05           | 集成三层与 Part 4 方法表读法         |
| 方法表 ++/+/o 与括号 (B.2/B.5)                  | ASIL ch07      | 方法表语义首讲                    |
| 安全确认与 HARA 假设闭环 (B.5.2)                   | ch04/ch05      | 8.4.4.4 假设 →10.4.3.2 验证    |
| AIS/暴露口径/可控性分位点 (B.6)                     | ch04           | S/E/C 证据化判定                |
| MSIL 两关系表模型、CQ 与门 禁 (B.7)                 | ch04/ch08      | 判定表形态；来源、类型边界与反例回归         |

| 本附录内容                          | 回指章节 | 衔接点                                     |
|--------------------------------|------|-----------------------------------------|
| Part 12 delta 与 T&B 规划边界 (B.7) | ch20 | 两项仍为 <b>planned</b> ; 待补证据清单的验收归属见发布保证章 |

\* 本附录素材: ISO 26262-12:2018 物理页 p9-52; 锚点与表格数值经两种同源提取视图交叉定位, 仍以原页复核为准。Part 12 正文为规范性适配条款, Annex A/B/C 为资料性判例。\*



# 附录 C 术语表：一份受控词汇的五条概念路径

Semantica 绑定：本附录是 185 个词条及五条概念路径的人可读解释；机器投影的唯一位置是 `semantica.chapter_packages.vol2.normative`，其中 Part 1 资产登记为 `part1-vocabulary` 与 `part1-cards`。该 package 当前为 `partial`、`release blocked`；它保存的是本书发布的工程释义、来源坐标和教学映射，不是 ISO 原文、授权译文、官方解释或合规证据。本目录没有 RDF 锚点或查询副本。

本附录以 ISO 26262-1:2018 第 3 章 (§3.1–§3.185, 1-based 物理页 9–36) 为唯一受控词条集，按本附录建立的五个概念群组织正文（与第 2 章的术语铺垫相衔接），字母序只作为末尾的入口索引。附录不建立第二套术语体系：每个条目的“一句白话”是教学重述，不是授权译文，更不能替代原文词条在合同、审核或评估场合的效力。

**与全书二十章的衔接：**本书前十章现为叙事重写，方法表语义、独立性等级记号、FMEDA 与失效分类等器具性内容已集中由附录承载；正文各章只提供活动与主题的工程语境。本附录因此按自足深讲编写——凡指向“第 X 章”处均指主题语境，不意味着正文有逐表逐条的展开。

## C.0 怎么使用这份术语表（条目模板与受控来源）

翻开标准的词汇卷，一种常见读法是“用到哪个查哪个”——按字母序找到词条，读完定义就合上。这种读法有一个隐蔽的代价：字母序把概念网剪成了 185 个孤立的点，“fault”排在 f，“failure”也排在 f，两者只隔几行，但它们之间那条“故障显现为失效”的因果箭头在字母序里是看不见的。第 2 章花了整整一章把这张网重新连起来；本附录延续同一套连法，把词条按**五个概念群**分节，让读者查一个词的时候顺便看见它的邻居。

每个条目固定使用同一个模板：

- **中文名 (English name, 缩写)** (词条号 | 首讲章) ——一句白话，说清它是什么、和邻居差在哪。

四个字段各有严格口径：

1. **词条号**：形如 1-3.84，表示 ISO 26262-1:2018 第 3 章第 84 个词条。这是每个条目的受控身份，也是与原文核对的唯一入口。本附录定稿前曾用两种独立生成的提取视图交叉定位词条；两种视图来自同一份 PDF，不构成两个独立来源。其中四个词条 (3.60、3.153、3.154、3.175) 的标题行存在 OCR 噪声，已回看原 PDF 校订。
2. **首讲章**：指向本书中负责首次完整讲解该术语的章；本附录 C.7.1 给出五群速查，各条目再标明具体去向。术语表只给一句白话；想看完整的直觉铺垫、ISO 定义精读和工程案例，请回到首讲章。个别术语另有概览章与深讲章（如独立性的组织含义在 ch03、技术独立与相关失效在 ch08），条目里会一并标出。
3. **一句白话**：以原文定义为底、面向工程背景大学生的重述。白话追求“读一遍就懂”，代价是省略了定义中的限定条件——凡是要做合规判断的场合，必须回到原文词条。
4. **概念群**：一个词条只有一份，但可以出现在多条概念路径上（如 hazard 既是风险群主角、又是时间群时间轴的触发端）。正文按其主路径归群，交叉关系在白话里点出。

机器侧唯一对应物是 `semantica.chapter_packages.vol2.normative` 中的 `part1-vocabulary` 与 `part1-cards` 资产；它们为已迁移词条保存工程释义与来源坐标，供该包声明的查询和门禁使用。**注意分母账**：Part 1 的受控词条集共 185 条；package 当前是 `partial`、release `blocked`，必须从当次 manifest/receipt 读取实际覆盖，不能用局部实例数冒充 185 条全量，也不能把释义资产冒充 ISO 原文。

关于缩写还有一条纪律。本附录在条目里给出的缩写 (ASIL、HARA、FTTI、SEooC 等) 以原文词条标题或全书首讲章的展开为准；正文遵循“首现三步走”，缩写只在全称出现过之后使用。工程文档需要特别防范**裸抛缩写**——一份写满 FDTI、MPFDTI、EOTTI 的报告，对没有背景的读者是密文。建议的做法与本书相同：每份文档第一次使用缩写时给出全称与词条号，之后再使用缩写；词条号让读者能沿着本附录回到定义。另外注意有些常用简称（如 FSR、TSR）虽在工程口语中通行，其词条标题是完整短语，缩写地位来自行业惯例——本附录如实标注，不为惯例升格。

还有一条使用纪律值得写在最前面：**白话与原文的分工是“引路”与“裁决”**。白话负责让你在三秒内想起这个词在概念网里站在哪个位置、和哪个近邻容易混；原文词条负责在争议出现时给出裁决依据。一种需要警惕的错用是拿着白话去做裁决——例如仅凭“安全故障就是无关大局的故障”这句引路语，就把某个未分析的故障归入安全



故障，而跳过了原文定义里“不显著提高违背安全目标概率”这个需要论证的判据。每当一个词条要进入 HARA 表格、安全分析报告或供应商协议，请以词条号为键回到正版原文；本附录的词条号可作为检索键，关键使用仍应核对适用版本和原词条。

## C.1 结构群：从相关项到软件单元

首讲 ch02 §2.2。这一群回答“我们给什么东西做安全”：从整车层面的记账单位一路分解到可以单独测试的最小软件颗粒。读这一群的窍门是记住三条轴：**部分-整体轴**（item→system→component→part/unit）、**实现轴**（架构→载体技术）、**边界轴**（谁在相关项里、谁在外面）。

### C.1.1 分解主干：从记账单位到最小颗粒

- **相关项 (item)** (1-3.84 | 首讲 ch02) ——在**整车层面**实现某个功能（或功能的一部分）、被 ISO 26262 当作记账单位的那个系统或系统组合。边界划到哪里，安全责任就算到哪里；HARA、安全目标、安全档案全都以它为单位展开。
- **系统 (system)** (1-3.163 | 首讲 ch02) ——至少把一个传感器、一个控制器和一个执行器关联起来的组件集合。“最小三件套”是它区别于普通组件的判据：少一件，就还不配叫 system。
- **元素 (element)** (1-3.41 | 首讲 ch02) ——系统、组件（硬件或软件）、硬件部件或软件单元的**统称**。当一句话想同时覆盖各分解层级时就用它；说“软件元素”时指软件组件或软件单元。
- **组件 (component)** (1-3.21 | 首讲 ch02) ——非系统级、逻辑上或技术上可分离的元素，由**多个硬件部件**，或由一个**及以上软件单元**组成。硬件分支与软件分支的数量门槛不同，不要误读成两边都必须有多个。
- **硬件部件 (hardware part)** (1-3.71 | 首讲 ch02) ——硬件组件第一层分解出来的部分，如微控制器里的 CPU、一只电阻、一片闪存阵列。
- **硬件子部件 (hardware subpart)** (1-3.73 | 首讲 ch06) ——硬件部件再往下逻辑划分的第二层及更深层，如微控制器 CPU 里的 ALU。
- **硬件基本子部件 (hardware elementary subpart)** (1-3.72 | 首讲 ch06) ——安全分析中考虑的最小硬件颗粒，如带逻辑锥的触发器、一个寄存器。硬件度量的分母就数到这一层为止。

- **软件组件 (software component)** (1-3.157 | 首讲 ch02) ——一个或多个软件单元的集合。定义只有一句话，但它是软件架构分层的基本砖块。
- **软件单元 (software unit)** (1-3.159 | 首讲 ch02, 操作深讲 ch07) ——软件架构中**可以单独测试**的原子级软件组件。“能不能单独测”是判据；Part 6 的单元设计与单元验证都以它为对象。
- **嵌入式软件 (embedded software)** (1-3.42 | 首讲 ch07) ——完成集成、将在处理单元上执行的软件整体。Part 6 末段的“嵌入式软件测试”测的就是这个集成后的整体，而非某个单元。

### C.1.2 架构与实现载体

- **架构 (architecture)** (1-3.1 | 首讲 ch05) ——相关项或元素的结构表示，能从中识别构造块、块的边界与接口，并包含需求到这些构造块的分配。注意它是“表示”(representation)，不是实物本身；原词条说的是需求分配，不是“运行所需过程”。
- **安全架构 (safety architecture)** (1-3.135 | 首讲 ch05) ——为满足安全要求而组织起来的元素集合及其交互。同一个系统的安全架构是其整体架构中承担安全的那张投影。
- **电气/电子系统 (electrical and/or electronic system, E/E 系统)** (1-3.40 | 首讲 ch02) ——由电气或电子元素构成的系统，包括可编程电子元素。ISO 26262 的适用范围就圈在 E/E 系统的功能异常上。
- **其他技术 (other technology)** (1-3.105 | 首讲 ch02) ——E/E 之外的技术，如机械、液压。标准不管它们的开发，但安全机制可以借助它们实现。
- **处理单元 (processing element)** (1-3.113 | 首讲 appA) ——提供数据处理功能的硬件部件，通常含寄存器组、执行单元和控制单元。多核、半导体讨论的基本单位。
- **多核 (multi-core)** (1-3.95 | 首讲 appA) ——含两个及以上可相互独立运行的处理单元的硬件组件。“可独立运行”是它进入 DFA 议题的原因。
- **可编程逻辑器件 (programmable logic device, PLD)** (1-3.114 | 首讲 appA) ——出厂时电路功能未定义、在集成阶段才配置的硬件组件或部件，如 FPGA。
- **传感转换器 (transducer)** (1-3.172 | 首讲 appA) ——把一种能量形式转换为另一种形式的硬件部件，其灵敏度决定输出与被测输入的关系。

- **分区 (partitioning)** (1-3.106 | 首讲 ch07, 深讲 ch08) ——为实现某种设计而对功能或元素做的分隔；可用于故障遏制，避免级联失效，是免于干扰的常用手段。
- **目标环境 (target environment)** (1-3.166 | 首讲 ch07) ——特定软件预期运行的环境；对应用软件而言就是那块微控制器加上系统软件。

### C.1.3 边界物种与功能视角

- **候选者 (candidate)** (1-3.16 | 首讲 ch10) ——定义和使用条件与已发布产品相同或高度一致的相关项/元素——它是“在用证明”论证的主角：想免做部分开发，先证明你是合格的候选者。
- **独立于环境的安全要素 (safety element out of context, SEooC)** (1-3.138 | 首讲 ch02) ——不针对某个具体相关项开发的安全相关元素，如通用微控制器、AUTOSAR 基础软件。它靠“假设的使用条件”参与安全论证，装车时必须逐条核对假设是否成立。
- **安全相关元素 (safety-related element)** (1-3.144 | 首讲 ch02) ——有潜力违背或达成某个安全目标的元素。注意双向措辞：不只是“会闯祸的”，还包括“负责兜底的”。
- **安全相关功能 (safety-related function)** (1-3.145 | 首讲 ch02) ——同上的功能视角版本：有潜力违背或达成安全目标的功能。
- **车辆功能 (vehicle function)** (1-3.178 | 首讲 ch04) ——由一个或多个相关项实现、顾客可观察到的车辆行为，如自适应巡航。功能是顾客视角，相关项是工程记账视角。
- **预期功能 (intended functionality)** (1-3.83 | 首讲 ch04) ——为相关项规定的行为，**不含安全机制**；规定的行为在整车层面描述。安全机制是为守护预期功能加上的，不算进预期功能本身。
- **功能概念 (functional concept)** (1-3.66 | 首讲 ch04) ——为达成期望行为而对预期功能及其交互的规格说明。它是 HARA 的输入：先说清车要干什么，才能问会怎么出事。
- **标定数据 (calibration data)** (1-3.15 | 首讲 ch07) ——软件构建完成后作为参数值写入的数据，如低怠速目标值、发动机特性曲线。改标定不改代码，但可能改安全行为。

- **配置数据 (configuration data)** (1-3.22 | 首讲 ch07) ——在元素构建阶段赋值、控制构建过程本身的数据，如预处理器变量。与标定数据的分界在于作用时机：一个管“怎么构建”，一个管“构建完怎么调”。
- **基线 (baseline)** (1-3.10 | 首讲 ch10) ——经批准的一组工作产物、相关项或元素的版本，作为后续变更的基准。没有基线，变更管理就没有“变更前”可言。

**路径走查：**拿全书贯穿的 EPS-RC17 (电动助力转向候选) 把这条路径走一遍。EPS 作为**相关项**记账——它在整车层面实现“转向助力”这个**车辆功能**；相关项里的 ECU 凑齐了传感器（扭矩传感器）、控制器（微控制器）、执行器（电机驱动）三件套，够格叫**系统**；微控制器是一个**组件**，它内部的 CPU、闪存是**硬件部件**，CPU 里的 ALU 是**硬件子部件**；软件侧，扭矩合理性检查是一个可单独测试的**软件单元**，若干单元组成**监控软件组件**，全部集成后烧进片子的是**嵌入式软件**。当供应商拿来一颗按 ASIL D 流程开发、但没针对这台车的通用微控制器时，它是 **SEooC**——装进相关项边界前，必须逐条核对它的使用假设。一句话可以覆盖上述任何一层的说法，就是**元素**。分解到哪一层就用哪一层的词，是结构群最基本的语言纪律：把“软件单元”说成“模块”、把“相关项”说成“系统”，在评审桌上都要付出返工的代价。

## C.2 因果群：故障、错误、失效与依赖失效

首讲 ch02 §2.3；λ 五分类深讲 ch06；依赖失效家族深讲 ch08。这一群回答“东西是怎么坏的”：主链 fault→error→failure 每个箭头都写着 can（可能而非必然），旁支按来源、按时长、按后果各切一刀，最后延伸到“多个元素一起坏”的依赖失效家族。

### C.2.1 主链三词与显现

- **故障 (fault)** (1-3.54 | 首讲 ch02) ——**可能**导致元素或相关项失效的异常状态。它是因果链的起点：故障可以潜伏很久而不发作。
- **错误 (error)** (1-3.46 | 首讲 ch02) ——计算、观测或测量出的值/状态，与真实、规定或理论正确的值/状态之间的**偏差**。它是故障被激活后的中间形态。
- **失效 (failure)** (1-3.50 | 首讲 ch02) ——由于故障显现，元素或相关项的预期行为**终止**。三个词按“状态 → 偏差 → 行为终止”排队，不可混用。
- **功能异常表现 (malfunctioning behaviour)** (1-3.88 | 首讲 ch02) ——相关项相对设计意图的失效或**非预期行为**。它比失效更宽：行为没终止但偏离了意图，也算。功能安全定义里点名的正是它。

- **失效模式 (failure mode)** (1-3.51 | 首讲 ch06) ——元素或相关项未能提供预期行为的**方式**。同一个元件可以有多种失效模式（断路、短路、漂移），安全分析按模式逐个记账。
- **失效率 (failure rate,  $\lambda$ )** (1-3.53 | 首讲 ch06) ——失效概率密度除以存活概率，硬件可靠性记账的基本量，通常在使用寿命内假设为常数。
- **基础失效率 (base failure rate)** (1-3.8 | 首讲 ch06) ——给定应用工况下硬件元素的失效率，作为安全分析的**输入**。它是后续 SPFM/LFM/PMHF 计算链的最上游数字。
- **故障模型 (fault model)** (1-3.58 | 首讲 appA) ——由故障导出失效模式的表示方法，用来评估特定故障的后果，如 stuck-at 模型。

### C.2.2 按来源与时长分

- **随机硬件故障/失效 (random hardware fault / random hardware failure)** (1-3.119 / 1-3.118 | 首讲 ch06) ——服从概率分布的硬件故障，及其在寿命期内不可预测地发生、但整体遵循概率分布的失效。对付它靠**度量与机制**（算  $\lambda$ 、加诊断），不靠改流程。
- **系统性故障/失效 (systematic fault / systematic failure)** (1-3.165 / 1-3.164 | 首讲 ch06，软件治理见 ch07) ——以确定性方式与某个原因关联、只能靠**更改设计或流程**才能消除的故障/失效。软件缺陷全是系统性的；对付它靠开发过程，算不出  $\lambda$ 。
- **永久故障 (permanent fault)** (1-3.109 | 首讲 ch06) ——发生后一直存在、直到移除或修复的故障，如 stuck-at、桥接。
- **瞬态故障 (transient fault)** (1-3.173 | 首讲 ch06) ——发生一次随后消失的故障，如电磁干扰或粒子翻转引起的位翻转。诊断策略对永久/瞬态故障的假设完全不同。

### C.2.3 按后果分： $\lambda$ 五分类

- **单点故障/单点失效 (single-point fault / single-point failure)** (1-3.156 / 1-3.155 | 首讲 ch06) ——元素中**没有任何安全机制覆盖**、自身直接导致违背安全目标的硬件故障，及其导致的失效。注意判据有两半：直接违背 + 无机制覆盖。



- **残余故障 (residual fault)** (1-3.125 | 首讲 ch06) ——元素有安全机制覆盖、但机制没盖住的那一部分随机硬件故障，其自身仍导致违背安全目标。单点与残余的分界只在“有没有机制”，这也是 SPFM 分子里两项并列的原因。
- **双点故障/双点失效 (dual-point fault / dual-point failure)** (1-3.39 / 1-3.38 | 首讲 ch06) ——需要与另一个独立故障组合才导致违背安全目标的单个故障，及两个独立故障组合直接导致的失效。“被监控功能坏了 + 监控它的机制也坏了”是一个用来解释该概念的典型组合，不代表频率排名。
- **多点故障/多点失效 (multiple-point fault / multiple-point failure)** (1-3.97 / 1-3.96 | 首讲 ch06) ——推广到  $n$  个独立故障组合的版本。若未被检测、未被感知，多点故障就在系统里潜伏着等队友。
- **潜伏故障 (latent fault)** (1-3.85 | 首讲 ch06) ——在多点故障检测时间间隔内，既没被安全机制检出、也没被驾驶员感知的多点故障。它是 LFM 的分子对象：机制自己坏了而未被发现，是一个典型的教学例。
- **被检出故障 (detected fault)** (1-3.31 | 首讲 ch06) ——在规定时间内被安全机制检出的故障。“规定时间”通常就是时间群里的故障检测时间间隔。
- **被感知故障 (perceived fault)** (1-3.108 | 首讲 ch06) ——通过整车层面的行为偏差被间接察觉的故障，如方向盘变沉。项目若要用驾驶员感知支持故障分类，应说明在所主张的场景中行为偏差可被感知、代表性人员能识别并作出响应，且感知和响应时间满足相关 MPFDTI、FTTI 或其他已规定时间窗口。它也可影响诊断、HMI、警告与服务策略的假设；词条本身不使这些假设自动成立。
- **安全故障 (safe fault)** (1-3.130 | 首讲 ch06) ——发生后不会显著提高违背安全目标概率的故障。名字容易误导：它不是“安全机制的故障”，而是“无关大局的故障”。
- **故障容忍 (fault tolerance)** (1-3.60 | 首讲 ch05) ——在一个或多个规定故障存在的情况下，仍能交付规定功能的能力。冗余架构买的就是这个能力。

#### C.2.4 依赖失效家族

- **相关失效 (dependent failures)** (1-3.29 | 首讲 ch08) ——统计上不独立的多个失效：组合发生的概率不等于各自概率的简单乘积。冗余的算术全建立在独立假设上，相关失效就是拆这个台的。
- **独立失效 (independent failures)** (1-3.79 | 首讲 ch08) ——同时或相继发生的概率恰好等于各自无条件概率乘积的失效。它是相关失效的对照组，也是概率计算的前提假设。

- **级联失效 (cascading failure)** (1-3.17 | 首讲 ch08) ——某个相关项中的元素因内外部根因失效，进而**引起**同一相关项或不同相关项中的另一个或多个元素失效。A 推倒 B，关键是有传播方向，不限于同一相关项内部。
- **共因失效 (common cause failure, CCF)** (1-3.18 | 首讲 ch08) ——两个及以上元素**直接**由同一个特定事件或根因导致的失效。同一电源掉电导致主备双双失效是标准例子；与级联的区别在于“同一根因直接打到多个元素”而非逐个传导。
- **共模失效 (common mode failure)** (1-3.19 | 首讲 ch08) ——CCF 的特例：多个元素以**相同方式**失效；何种相似程度足以归为共模，要结合语境判断，并不要求失效表现完全相同。若两个传感器因同一批次缺陷这一共同根因而以相近方式漂移，可构成共模失效。
- **耦合因素 (coupling factors)** (1-3.26 | 首讲 ch08) ——导致多个元素失效相关联的共同特征或关系，如同一供电、同一位置、同一开发团队。DFA 可以按已声明的分析边界和耦合因素分类，系统搜索可信的相关失效路径；清单不自动代表搜索穷尽。
- **相关失效引发点 (dependent failure initiator, DFI)** (1-3.30 | 首讲 ch08) ——通过耦合因素让多个元素失效的**单一根因**。DFA 应在声明边界内识别可信的 DFI 候选及所影响元素，并记录消除、控制或接受理由；不能从“已列清单”推出所有发起者已被找到或挡住。
- **独立性 (independence)** (1-3.78 | 组织含义首讲 ch03, 技术含义首讲 ch08) ——两重含义：元素之间不存在会违背安全要求的相关失效；或组织之间不存在利益冲突。ASIL 分解的前提、确认措施的资格，用的都是这个词。
- **免于干扰 (freedom from interference, FFI)** (1-3.65 | 首讲 ch08) ——两个及以上元素之间不存在会导致违背安全要求的**级联失效**。软件分区要证明的就是它：QM 分区不得拖垮 ASIL 分区。正文以“互不干扰”的叙事语汇出现，合规判据以本词条为准。
- **安全异常 (safety anomaly)** (1-3.134 | 首讲 ch09) ——偏离预期且可能导致伤害的状况。评审、测试、现场运行中发现的“不对劲”都先记为安全异常，再决定处置。

### C.2.5 覆盖率与观测

- **诊断覆盖率 (diagnostic coverage, DC)** (1-3.33 | 首讲 ch06) ——硬件元素失效率中被安全机制**检出或控制**的百分比。它是把“装了诊断”翻译成数字的桥梁。
- **失效模式覆盖率 (failure mode coverage)** (1-3.52 | 首讲 ch06) ——某失效模式的失效率中被安全机制检出或控制的比例。DC 按元素记总账，它按失效模式记明细账。
- **硬件架构度量 (hardware architectural metrics)** (1-3.70 | 首讲 ch06) ——评价硬件架构对安全的有效性的度量，即 SPFM 与 LFM 的统称。
- **诊断点 (diagnostic points)** (1-3.34 | 首讲 appA) ——观察到故障被**检测或纠正**的元素输出信号。
- **观测点 (observation points)** (1-3.101 | 首讲 appA) ——观察到故障**潜在影响**的元素输出信号，如存储器的输出。诊断点看“抓没抓住”，观测点看“有没有影响”。

**路径走查：**EPS 扭矩传感器的信号线发生桥接短路——这是一个**永久的随机硬件故障**；它让 ADC 读出的扭矩值偏离真实值，这是**错误**；错误穿过滤波与控制律，最终让助力电机输出非预期扭矩、“按需助力”这一预期行为终止，这是**失效**，其**失效模式**是“助力扭矩超出请求值”。现在按后果分类记账：如果这条信号链没有任何安全机制覆盖，它就是**单点故障**；如果装了合理性检查但覆盖率只有 90%，漏网的 10% 是**残余故障**；合理性检查电路自己坏了而无人察觉，则是一个正在潜伏的**多点故障**——若在**多点故障检测时间间隔**内既没被检出也没被驾驶员感知，就归入**潜伏故障**。是否已被感知不能只凭故障名称假定，要用所主张场景中的可感知性、驾驶员响应和时间窗口证据支持。再横向看一眼：主备两路传感器共用同一 5 V 供电，供电跌落让两路同时漂移——这不是两个独立失效的巧合，而是以供电为**耦合因素**、由同一个**相关失效引发点**触发的**共因失效**。因果群的词汇帮助把这段叙述拆成可分析、可追溯的对象，具体归类仍取决于项目边界、故障模型与证据。

## C.3 风险群：危害事件、S/E/C 与完整性等级

首讲 ch02 §2.5；操作深讲 ch04。这一群回答“坏了**有多危险**”：从伤害一路走到 ASIL。路线上最关键的一步是“危害 × 运行场景 = 危害事件”——分级的对象从来不是部件，而是整车层面的危害事件。

### C.3.1 从危害到危害事件

- **安全 (safety)** (1-3.132 | 首讲 ch02) ——不存在不合理的风险。整本标准的地基只有这一句：安全不是零风险。
- **功能安全 (functional safety)** (1-3.67 | 首讲 ch02) ——不存在由 E/E 系统功能异常表现引起的危害所导致的不合理风险。三个限定词划定了整本书的边界：对象是 E/E 系统的功能异常及其危害，目标是不存在不合理风险，不是宣称把风险消除为零。
- **伤害 (harm)** (1-3.74 | 首讲 ch04) ——对人的身体损伤或健康损害。注意对象只有人：撞坏护栏本身不算 harm。
- **危害 (hazard)** (1-3.75 | 首讲 ch04) ——由相关项功能异常表现引起的潜在伤害来源。它是“可能性”而非“事件”：非预期的转向助力是危害，撞车才是事故。
- **非功能性危害 (non-functional hazard)** (1-3.100 | 首讲 ch04) ——由 E/E 功能异常之外的因素（如电击、火灾、毒性）引起的危害。ISO 26262 明确不管它们，划界时要说得出去哪儿管。
- **运行场景 (operational situation)** (1-3.104 | 首讲 ch04) ——车辆生命期内可能出现的场景，如高速行驶、坡道停车、维修。它是 E 参数的评估对象。
- **运行模式 (operating mode)** (1-3.102 | 首讲 ch04) ——相关项或元素因使用而处的功能状态，如系统关闭、正常运行、降级运行。
- **车辆运行状态 (vehicle operating state)** (1-3.179 | 首讲 ch04) ——运行模式与运行场景的组合。场景在车外、模式在车内，两者合起来才描述“车此刻的处境”。
- **危害事件 (hazardous event)** (1-3.77 | 首讲 ch04) ——危害与运行场景的组合。同一个危害配上不同场景，严重度和可控性都可能不同——这是 HARA 按事件而非按危害逐行打分的原因。
- **危害分析与风险评估 (hazard analysis and risk assessment, HARA)** (1-3.76 | 首讲 ch04) ——识别并分级相关项的危害事件、进而制定安全目标及其 ASIL 的方法。它是概念阶段的发动机。

### C.3.2 三参数与风险

- **严重度 (severity, S)** (1-3.154 | 首讲 ch04) ——潜在危害事件中一人或多人可能受到伤害程度的估计。HARA 中取 S0-S3。
- **暴露 (exposure, E)** (1-3.48 | 首讲 ch04) ——处于某个运行场景中的状态，该场景若与被分析的失效模式同时出现则可能有危害。E 参数评估的是**场景出现的可能性**（正文因此把这项判断称作“暴露概率”，不是失效的可能性——这是本附录要重点辨析的区别。
- **可控性 (controllability, C)** (1-3.25 | 首讲 ch04) ——相关人员通过及时反应（可能借助外部措施）避免特定伤害或损失的能力。相关人员可包括驾驶员、乘员或车辆外部附近人员；在具体 HARA 场景中应识别真正涉事的人，不能把定义直接缩成“典型驾驶员”。
- **专家骑手 (expert rider)** (1-3.47 | 首讲 appB) ——摩托车领域中有能力基于实车操作评估可控性分级的角色。Part 12 用它校准摩托车的 C 参数。
- **风险 (risk)** (1-3.128 | 首讲 ch04) ——伤害发生**概率**与该伤害**严重度**的组合。两个因子缺一不可。
- **不合理风险 (unreasonable risk)** (1-3.176 | 首讲 ch02) ——依据当时有效的社会道德观念，在特定情境下被判定为不可接受的风险。注意判据是社会共识而非工程计算——它解释了为什么“安全”没有全球统一的数值阈值。
- **残余风险 (residual risk)** (1-3.126 | 首讲 ch04) ——部署安全措施**之后**剩下的风险。安全工程的目标是把残余风险压到不合理线以下，而不是压到零。
- **合理可预见 (reasonably foreseeable)** (1-3.120 | 首讲 ch04) ——技术上可能、且发生率可信或可测的情形。可预见的误用要管，科幻式的滥用不用管，界线就画在这个词上。
- **外部措施 (external measure)** (1-3.49 | 首讲 ch04) ——独立于相关项之外、能降低或缓解相关项风险的措施，如道路护栏。HARA 打分时它能救 C 参数一命，但不算相关项自己的安全机制。

### C.3.3 完整性等级

- **汽车安全完整性等级 (automotive safety integrity level, ASIL)** (1-3.6 | 首讲 ch02, 操作深讲 ch04) ——四个等级 (A 最低、D 最高) 之一，用来规定相关项或元素为避免不合理风险而需适用的 ISO 26262 要求与安全措施。它把危害事



件的风险分级连接到工程严谨度；原词条说的是 unreasonable risk，不是另造一个“不合理残余风险”术语。

- **ASIL 能力 (ASIL capability)** (1-3.2 | 首讲 ch10) ——相关项或元素满足**假设的**、按给定 ASIL 分配的安全要求的能力。SEooC 的说法：“我按 D 的流程开发过”说的是能力，不是这次装车就自动合格。
- **ASIL 分解 (ASIL decomposition)** (1-3.3 | 首讲 ch08) ——把冗余的安全要求分配到**充分独立**的多个元素上、服务于同一安全目标，从而降低各元素 ASIL 的做法。注意三个不可省略的限定：冗余、充分独立、同一安全目标——独立性证据不到位，分解就是自欺。
- **摩托车安全完整性等级 (motorcycle safety integrity level, MSIL)** (1-3.94 | 首讲 appB) ——摩托车领域的四级风险降低等级，需按 Part 12 规则**换算**为 ASIL 后再套用各部要求。同名不同域，不可直接混用。
- **质量管理 (quality management, QM)** (1-3.117 | 首讲 ch04) ——组织为质量而进行的协调指挥与控制活动。Part 1 的注记只明确两点：**QM 不是 ASIL**，但可以在 HARA 中指定；至于某个 HARA 结果后续适用哪些活动，应回到 Part 3 及项目适用性判断，不能从词条本身推出“无需 ISO 26262 附加措施”这一无条件结论。

**路径走查**（教学假设，非任何项目结论）：继续 EPS 的故事。“非预期的转向助力”是一个**危害**——潜在的伤害来源，还不是危害事件。把它分别与“高速过弯”和“停车场低速挪车”等**运行场景**组合，才得到需要在 **HARA** 表中分行评估的**危害事件**。为演示查表，假设项目已用指定车速、道路曲率、碰撞对象与伤情谱支持 **S3**，用目标车队的运行场景数据支持 **E4**，并用代表性驾驶员、故障施加时刻、可用纠正动作与反应时间窗口的评估记录支持 **C3**。在这组给定教学输入下，S3+E4+C3 查表得 **ASIL D**；本附录不提供上述项目证据，因而不为现实 EPS 场景作等级断言。这段走查要区分的是：E 评场景出现的可能性而不是故障概率，C 评场景中相关人员避免特定伤害的能力，S 估计伤害程度而不是维修费。相似安全目标合并时，取其所关联危害事件中的最高 ASIL。若某行在 HARA 中被指定为 **QM**，表示该结果不是 ASIL；后续活动应按 Part 3、项目适用性和组织质量管理体系判断，不能把 QM 简化成“没有其他义务”。

## C.4 需求群：安全目标、需求与工作产物

概览 ch02；操作深讲 ch04/ch05，档案与确认深讲 ch03。这一群回答“我们答应做到什么、拿什么证明”：需求沿“安全目标 → 功能安全要求 → 技术安全要求 → 硬件/软件实现”逐层落地；措施与机制负责兑现承诺；工作产物与各种评审、审核构成证据链。词汇表里没有 HSR 和 SSR——这条诚实边界由本附录维持，此处不再造词。

### C.4.1 需求层级

- **安全目标 (safety goal, SG)** (1-3.139 | 首讲 ch04) ——HARA 得出的**整车层面最高层**安全要求。一个安全目标可以对应多个危害事件，其 ASIL 取所涉事件中最高者。
- **功能安全概念 (functional safety concept, FSC)** (1-3.68 | 首讲 ch04) ——功能安全要求的规格说明及其到相关项架构元素的分配。它回答“用什么**策略**兑现安全目标”。
- **功能安全要求 (functional safety requirement, FSR)** (1-3.69 | 首讲 ch04) ——与实现无关的安全行为或安全措施的规格说明。判据就在“implementation-independent”这一个词上：说了“用哪种传感器”就已经不是 FSR。
- **技术安全概念 (technical safety concept, TSC)** (1-3.167 | 首讲 ch05) ——技术安全要求的规格说明及其到系统元素的分配，附带实现所需的信息。
- **技术安全要求 (technical safety requirement, TSR)** (1-3.168 | 首讲 ch05) ——为**实现**关联的功能安全要求而导出的要求。FSR 说“要防非预期助力”，TSR 说“扭矩指令超阈值 x ms 内切断”——从策略到技术指标的翻译。
- **继承 (inheritance)** (1-3.81 | 首讲 ch05) ——开发过程中把需求属性**原封不动**传递到下一细化层级。ASIL 沿需求链往下传，默认走的就是继承，除非启动 ASIL 分解。

### C.4.2 措施与机制

- **安全措施 (safety measure)** (1-3.141 | 首讲 ch02) ——避免或控制系统性失效、检测或控制随机硬件失效或缓解其危害影响的**活动或技术方案**。范围很宽：编码规范、评审、看门狗都算。

- **安全机制 (safety mechanism)** (1-3.142 | 首讲 ch02) ——由 E/E 功能/元素或其他技术实现的**技术方案**，用于检测并缓解或容忍故障、控制或避免失效，以保持预期功能或达到安全状态。机制是措施的真子集：只有做进产品里的技术方案才叫机制，流程活动不算。
- **专用措施 (dedicated measure)** (1-3.27 | 首讲 ch06) ——确保安全目标违背概率评估中所声称失效率成立的措施，如设计特性、专项样本测试、老化筛选。声称  $\lambda$  有多低，就要有对应的专用措施撑住。
- **冗余 (redundancy)** (1-3.122 | 首讲 ch08) ——在足以完成功能的手段**之外**再增加的手段，如双通道采集。冗余是分解的物质基础，但冗余本身不自动等于独立。
- **多样性 (diversity)** (1-3.37 | 首讲 ch08) ——用**不同的**方案满足同一需求、以谋求独立性的做法。词条注记提醒：多样性不保证不发生共因失效，只是降低其可能。
- **稳健设计 (robust design)** (1-3.129 | 首讲 ch09) ——在无效输入或恶劣环境条件下仍能正确工作的设计；对软件即对无效输入的耐受，对硬件即对环境应力的耐受。
- **故障注入 (fault injection)** (1-3.57 | 首讲 ch07) ——向元素中插入故障、错误或失效，观察其反应以评估故障影响的**方法**。它是方法表里的常客：单元、集成、整车各层都有它的行。
- **基于模型的开发 (model-based development)** (1-3.90 | 首讲 ch07) ——用模型描述待开发元素行为或属性的开发方式。模型可以生成代码，也因此把工具链的置信度问题 (Part 8) 带进了软件章。

### C.4.3 工作产物与安全档案

- **工作产物 (work product)** (1-3.185 | 首讲 ch03) ——由 ISO 26262 一个或多个相关要求产生的**文档化结果**；它可以是一份完整文档，也可以由多份文档共同承载。是否充分证明活动已执行，要结合相应要求、过程记录和评审判断，不能把“没文档等于没做”当成词条定义的一部分。
- **安全档案 (safety case)** (1-3.136 | 首讲 ch03) ——论证相关项或元素已达成功能安全的**论证** (argument)，由工作产物汇编的证据支撑。注意词性：它是论证结构，不是文件夹——把工作产物堆在一起不构成安全档案。

- **安全计划 (safety plan)** (1-3.143 | 首讲 ch03) ——管理和引导项目安全活动执行的计划，含日期、里程碑、任务、交付物与责任人。
- **安全活动 (safety activity)** (1-3.133 | 首讲 ch03) ——在安全生命周期一个或多个阶段/子阶段中执行的活动。
- **安全生命周期 (lifecycle)** (1-3.86 | 首讲 ch03) ——相关项从概念到退役的全部阶段。
- **阶段 (phase)** (1-3.110 | 首讲 ch03) ——安全生命周期中由 Part 3-7 规定的阶段，如概念阶段、系统层开发。
- **子阶段 (sub-phase)** (1-3.161 | 首讲 ch03) ——阶段在某个条款中规定的细分，如 HARA 是概念阶段的一个子阶段。
- **安全经理 (safety manager)** (1-3.140 | 首讲 ch03) ——负责监督并确保达成功能安全所需活动得以执行的人或组织。
- **安全文化 (safety culture)** (1-3.137 | 首讲 ch03) ——组织中持久的价值观、态度、动机与知识，使安全在决策和行为中**优先于竞争性目标**。它决定了流程条文在压力下是否还算数。
- **管理体系 (management system)** (1-3.87 | 首讲 ch03) ——组织为达成目标所用的方针、程序与过程的总称。

#### C.4.4 验证、确认与各种“看一遍”

- **验证 (verification)** (1-3.180 | 首讲 ch05, 框架深讲 ch10) ——判定被检对象是否满足其规定要求。问的是“做得对不对”(against spec)。
- **验证评审 (verification review)** (1-3.181 | 首讲 ch10) ——确保开发活动结果满足项目要求或技术要求的验证活动。
- **安全确认 (safety validation)** (1-3.148 | 首讲 ch05) ——基于检查和试验，对安全目标**是否充分，以及是否以足够完整性得到实现**给出确信。它与验证的对象和判据不同：这里问的是安全目标是否适当、是否已经达成，不能简单说成“比验证高一层”。
- **测试 (testing)** (1-3.169 | 首讲 ch07) ——为验证满足要求、检测异常、建立信心而对相关项或元素进行规划、准备、运行或施加激励的过程。
- **评审 (review)** (1-3.127 | 首讲 ch03) ——按评审目的，对工作产物是否达成其预期目标进行的检查。

- **审查 (inspection)** (1-3.82 | 方法表见附录 D) ——遵循正式规程检查工作产物以检出安全异常。与走查同为验证手段，区别在正式程度：审查有规程、角色和进出条件。
- **走查 (walk-through)** (1-3.182 | 首讲 ch08) ——系统地过一遍工作产物以检出安全异常，形式比审查轻。方法表里两者是备选关系 (Part 6 Table 7 的 1a/1c)。
- **确认措施 (confirmation measure)** (1-3.23 | 首讲 ch03) ——针对功能安全的确认评审、审核或评估的统称。执行人的独立性等级 (I0-I3) 随 ASIL 升高，首讲章有完整矩阵。
- **确认评审 (confirmation review)** (1-3.24 | 首讲 ch03) ——确认某工作产物为达成功能安全提供了充分且有说服力的证据。对象是工作产物。
- **功能安全审核 (audit)** (1-3.5 | 首讲 ch03) ——对已实施过程是否符合过程目标的检查。对象是过程。
- **功能安全评估 (assessment)** (1-3.4 | 首讲 ch03) ——对相关项或元素的特性是否达成 ISO 26262 目标的检查。对象是达成的功能安全本身；评审、审核、评估三者对象不同，不可互相顶替。
- **回归策略 (regression strategy)** (1-3.123 | 首讲 ch10) ——验证变更没有影响未变更且已验证部分的策略。改一行代码要不要重测全部，答案写在这里。

#### C.4.5 表示法与结构覆盖率

- **形式化/半形式化/非形式化表示法 (formal / semi-formal / informal notation)** (1-3.63 / 1-3.149 / 1-3.80 | 首讲 ch07) ——按语法和语义的定义完备程度递减的三档描述技术：形式化两者皆完备（如模型检查器输入语言）；半形式化语法完备、语义可不完备（如 UML 子集）；非形式化连语法都不完备（如自然语言配图）。方法表按 ASIL 调节三档的推荐度。
- **形式化/半形式化验证 (formal / semi-formal verification)** (1-3.64 / 1-3.150 | 首讲 ch07) ——形式化验证使用形式化表示法，证明相关项或元素相对于其功能或属性规格的正确性；半形式化验证则基于半形式化表示法给出的描述实施验证，例如用半形式化模型生成测试向量并检查系统行为。两者首先区分的是表示法与方法依据，不能脱离具体方法和验证对象，把差异一概简化成“证明”与“检查”的证据等级。
- **语句覆盖率 (statement coverage)** (1-3.160 | 首讲 ch07) ——测试中被执行的软件语句百分比。三种结构覆盖率中最弱的一档。



- **分支覆盖率 (branch coverage)** (1-3.13 | 首讲 ch07) ——测试中被执行的控制流分支百分比；100% 分支覆盖蕴含 100% 语句覆盖，反之不然。
- **修正条件/判定覆盖率 (modified condition/decision coverage, MC/DC)** (1-3.92 | 首讲 ch07) ——独立影响判定结果的单个条件取值被演练到的百分比，三档中最强，Part 6 Table 9 只对 ASIL D 给 ++。

#### C.4.6 供应链、工具与既有信任

- **开发接口协议 (development interface agreement, DIA)** (1-3.32 | 首讲 ch10) ——客户与供应商之间的协议，规定分布式开发中双方各自负责执行的活动、评审的证据与交换的工作产物。安全责任在供应链上的“分界合同”。
- **供货协议 (supply agreement)** (1-3.162 | 首讲 ch10) ——客户与供应商间关于活动、证据或工作产物的责任约定；量产供货语境下与 DIA 相邻但不相同。
- **分布式开发 (distributed development)** (1-3.36 | 首讲 ch10) ——相关项或元素的开发责任在客户与供应商之间划分的开发形态。DIA 因它而生。
- **软件工具 (software tool)** (1-3.158 | 首讲 ch10) ——用于相关项或元素开发过程的计算机程序。编译器、代码生成器、静态分析器都是；它们不装车，但可能把错误注入装车的东西——这是工具置信度 (TCL) 议题的起点。
- **在用证明论证/在用证明信用 (proven in use argument / proven in use credit, PiU)** (1-3.115 / 1-3.116 | 提名 ch09, 深讲 ch10) ——前者是基于候选者现场数据分析所得的证据，用来说明任何可能损害安全目标的候选者失效概率满足适用 ASIL 的要求；后者是在此论证成立后，用它及相应工作产物替代给定生命周期子阶段。不能把适用 ASIL 的整套要求缩成一个脱离语境的“低于某目标值”。
- **现场数据 (field data)** (1-3.62 | 首讲 ch09) ——相关项或元素使用中获得的数据，含累计运行小时、全部失效与在役安全异常。在用证明的原料。
- **久经考验 (well-trusted)** (1-3.184 | 首讲 ch10) ——曾在可比应用中使用且无已知安全异常，如久经考验的设计原则、工具、软件模块。口语的“用过很多年”必须落到“可比应用 + 无已知异常”两个判据上。
- **安全相关特殊特性 (safety-related special characteristic)** (1-3.147 | 首讲 ch09) ——相关项、元素或其生产过程的特性，其合理可预见的偏差可能影响、促成或引起功能安全的潜在降低。它不只覆盖会直接违背安全目标的偏差，也是安全要求从开发端伸进工厂的重要接口。

**路径走查：**需求群是一条自上而下的翻译链，配一条自下而上的证据链。往下走：HARA 给出**安全目标**“防止非预期助力（ASIL D）”；**功能安全概念**给出策略——“监控扭矩指令，超限即切断”，写成与实现无关的**功能安全要求**并分配到架构元素；系统层把策略翻译成**技术安全要求**——“指令超阈值持续 x ms 内切断电机供电”，汇入**技术安全概念**；ASIL 沿链**继承**，除非启动分解。兑现承诺的是**安全机制**（扭矩合理性检查这个做进产品的技术方案）和更广义的**安全措施**（包括编码规范、评审这些流程活动）。往上走：每一步活动留下**工作产物**；**验证**逐层确认“做得对不对”，**安全确认**在整车上确认“目标本身对不对、真达到没有”；**确认评审**、**审核**、**评估**三种**确认措施**由具备独立性的人分别检查工作产物、过程与达成的功能安全；相关证据最终编入**安全档案**——注意它是论证不是文件夹。链条两端接上，“承诺—兑现—证明”才闭环。

## C.5 时间群：容忍、检测、处理与运行时间

首讲 ch02 §2.4；预算与分配深讲 ch05；现场时序深讲 ch09。这一群回答“来得及吗”。所有词条挂在同一条事件时间轴上：**故障发生** → **检测** → **反应** → **安全状态**（或**应急运行** → **安全状态**）。读这一群务必分清两类归属：FTTI 是**相关项**的属性（留给你多少时间），FDTI/FRTI/FHTI 是**安全机制**的属性（你实际用掉多少时间）。Part 1 的词条注记用二者的比较解释何谓及时覆盖；具体项目仍应在安全要求和时序论证中规定并验证预算，不能把术语注记冒充独立的 shall。

- **故障容忍时间间隔**（fault tolerant time interval, FTTI）（1-3.61 | 首讲 ch02, 预算深讲 ch05）——在安全机制未激活的情况下，从故障在相关项中发生到**可能**出现危害事件的最短时间。它是留给所有防线的总预算，属于相关项。
- **故障检测时间间隔**（fault detection time interval, FDTI）（1-3.55 | 首讲 ch02）——从故障发生到被检出的时间。
- **故障反应时间间隔**（fault reaction time interval, FRTI）（1-3.59 | 首讲 ch02）——从故障被检出到达到安全状态或进入应急运行的时间。
- **故障处理时间间隔**（fault handling time interval, FHTI）（1-3.56 | 概念首讲 ch02, 预算深讲 ch05）——FDTI 与 FRTI 之**和**，即安全机制从故障发生到完成反应的用时。Part 1 对 FDTI 的 Note 4 说明：若 FDTI + FRTI 小于相关 FTTI，则相应安全机制及时覆盖该故障；这是资料性的术语说明，项目义务仍须追到适用的规范性条款和安全要求。
- **诊断测试时间间隔**（diagnostic test time interval, DTTI）（1-3.35 | 首讲 ch06）——安全机制两次在线诊断测试执行之间的时间，且包含一次在线诊断

测试的执行时长。它描述诊断执行间隔；允许的周期或上限要由具体诊断设计、FDTI/FTTI 分配和安全要求论证，不能从 DTTI 词条单独推出。

- **多点故障检测时间间隔 (multiple-point fault detection time interval, MPFDTI)** (1-3.98 | 首讲 ch06) ——在多点故障可能凑成多点失效之前将其检出的时间窗。潜伏故障的定义就以它为限。
- **应急运行 (emergency operation)** (1-3.43 | 首讲 ch05) ——故障反应之后、过渡到安全状态之前，为维持安全而设的相关项运行模式。直接断电不安全的系统（如行驶中的转向助力）需要这段“降级续命”。
- **应急运行时间间隔 (emergency operation time interval, EOTI)** (1-3.44 | 概览 ch02, 预算深讲 ch05) ——应急运行实际维持的时间段。
- **应急运行容忍时间间隔 (emergency operation tolerance time interval, EOTTI)** (1-3.45 | 概览 ch02, 预算深讲 ch05) ——应急运行在不产生不合理风险的前提下可以维持的规定时长。EOTI 必须收敛在 EOTTI 之内；一对“实际用时 vs 容忍上限”的关系，与 FHTI/FTTI 同构。
- **最大修复时间间隔 (maximum time to repair time interval)** (1-3.89 | 首讲 ch09) ——安全状态可以维持的规定时长；当安全状态需要靠维修才能退出时，这个窗口决定了“多久内必须修好”。
- **安全状态 (safe state)** (1-3.131 | 概览 ch02, 系统策略深讲 ch05) ——发生失效时相关项不存在不合理风险的运行模式。时间轴的终点站。注意它是运行模式而非“关机”：对某些系统，关机恰恰不安全。
- **降级 (degradation)** (1-3.28 | 首讲 ch09) ——相关项或元素功能或性能降低的状态或向该状态的过渡。限速跛行回家是典型降级。
- **警告与降级策略 (warning and degradation strategy)** (1-3.183 | 首讲 ch09) ——规定如何提醒驾驶员功能可能降低、以及如何提供降级功能以到达安全状态的规格说明。时间轴上“人”这条支线的剧本。
- **运行时间 (operating time)** (1-3.103 | 首讲 ch06) ——相关项或元素处于工作状态的累计时间，含降级模式。PMHF 按每小时记账，分母就是它。

**路径走查：**给时间轴配一次算术（数值为教学示例，非任何产品结论）。假设项目在危害分析基础上，为 EPS 相应安全目标规定：非预期助力持续超过 100 ms 才可能酿成危害事件——**FTTI = 100 ms**，这是相关项给所有防线的总预算。安全机制这边：扭矩合理性检查每 10 ms 跑一轮（**诊断测试时间间隔** 10 ms），最坏情况下故障发生后 20 ms 被检出（**FDTI = 20 ms**），再用 30 ms 切断电机供电、进入无助力状态（**FRTI**

= 30 ms)，合计 **FHTI** = 50 ms < 100 ms，不等式成立，且留了 50 ms 教学裕量。若这台车行驶中直接失去助力并不安全，就不能一步跳到安全状态：先进入限制助力的**应急运行**，实际维持的 **EOTI** 必须收敛在 **EOTTI**——“降级续命不产生不合理风险”的容忍上限——之内，最终停靠**安全状态**。这次走查要重点辨析三件事：FTTI 属于相关项，而非某个机制的独立指标；FHTI 由从故障发生起算的 FDTI 与从检出起算的 FRTI 组成；安全状态是具体系统在特定场景下不存在不合理风险的运行模式，不能被概括成“关机”。

## C.6 本体化实践：一份词表、多条概念路径

本附录由两种互补载体共同维护：Markdown 正文承担五群讲解、字母索引与首讲回指；Semantica normative package 的 **part1-vocabulary** 资产为**已迁移的术语子集**建立可查询锚点，保存词条号、标题、证据坐标和受控来源标识。以下 item 片段是该数据形态的

书面投影；不得从本书代码块复制出第二份运行资产：

```
iso262:Term_1_item a iso262:Clause ;
    iso262:clauseId "1-3.84" ;
    dcterms:title "item"@en ;
    iso262:sourcePdfPage 24 ; iso262:sourceBlock 5 ;
    iso262:sourceBbox "58,193,102,206" .
iso262:Item iso262:hasSourceAnchor iso262:Term_1_item .
```

第二条三元组把 **Item** 类挂到词条锚点上。Semantica 场景可以检查这条绑定及已编码坐标合同；至于 **Item** 的建模是否完整忠实于 1-3.84，仍须回到合法持有的原文和建模语境人工审查。坐标可解析也不表示 OCR、中文转述或原 PDF 语义已经被机器核对。

下面的 SPARQL 只解释 package 查询想回答的问题；实际查询文本与 oracle 的唯一正本在

Semantica normative package:

```
SELECT ?term ?clauseId ?title WHERE {
    ?cls iso262:hasSourceAnchor ?term .
    ?term iso262:clauseId ?clauseId ; dcterms:title ?title .
} ORDER BY ?clauseId
```

这条查询不会返回本附录的五个概念群，也不会给出首讲章，更不能再生 C.7.2 的字母索引：当前 normative package 的已迁移 RDF 资产没有保存这些编辑属性。C.1-

C.5 的分群、C.7.1 的首讲速查、C.7.2 的字母索引以及各章正文回指，仍是人工维护的阅读视图。锚点给它们提供统一的受控标识和抽查入口，可以降低漂移风险，但尚未做到“改一处即全网一致”。修订时必须同时审查 Semantica 绑定与这些 Markdown 视图。

当前自动检查的能力边界如下。`semantica.vol2.normative.scenario.modality-fidelity` 只执行其 CQ registry 明示的机器合同；坐标或绑定通过不表示 OCR、词条转述或原 PDF 已经完成语义核对，它也不读取本附录 Markdown，因而发现不了正文写错词条号。`severity` 等 OCR 噪声仍须回看原 PDF 并以校订留痕处理，不能归功于绑定查询。

诚实边界也要说清：Part 1 的出版覆盖分母始终是 185，实际机器锚定数只能从源锁定 Semantica manifest/receipt 查询，不能硬编码成永久进度。未锚定词条可以在回看原文后用于正文，但不能假装已有机器门禁保护。若未来让分群、索引和首讲回指由查询生成，必须先把编辑属性注册进 Semantica package，并为生成结果与 Markdown 建立独立校验；在那之前仍需人工编辑审查。

## C.7 字母索引、首讲章与交叉引用

### C.7.1 首讲章速查

| 概念群                                            | 首讲章     | 深讲/操作章                                |
|------------------------------------------------|---------|---------------------------------------|
| 结构群 (item/system/element/component/unit、SEooC) | ch02    | 各章；软件颗粒 ch07；半导体载体 appA               |
| 因果群 (fault/error/failure 主链)                   | ch02    | λ 五分类 ch06；依赖失效与 FFI ch08             |
| 风险群 (HARA、S/E/C、ASIL、SG)                       | ch02 概览 | 操作 ch04；MSIL 换算 appB                  |
| 需求群 (SG→FSR→TSR、工作产物、确认措施)                     | ch02 概览 | ch04/ch05；档案与文化 ch03；工具与 PiU/DIA ch10 |
| 时间群 (FTTI 家族、安全状态)                             | ch02 概览 | 预算分配 ch05；现场时序 ch09                   |

### C.7.2 字母序全量入口 (185 条)

以下按英文首字母列出全部 185 个受控词条的“词条号 · 本附录位置”。**粗体**为正文已展开条目（含合并讲解的成对条目），未加粗者见 C.7.3 速览。

- **A**: architecture **3.1**(C.1.2); ASIL capability **3.2**(C.3.3); ASIL decomposition **3.3**(C.3.3); assessment **3.4**(C.4.4); audit **3.5**(C.4.4); automotive safety integrity level **3.6**(C.3.3); availability 3.7(C.7.3)



- **B**: base failure rate **3.8**(C.2.1); base vehicle 3.9; baseline **3.10**(C.1.3); body builder 3.11; body builder equipment 3.12; branch coverage **3.13**(C.4.5); bus 3.14
- **C**: calibration data **3.15**(C.1.3); candidate **3.16**(C.1.3); cascading failure **3.17**(C.2.4); common cause failure **3.18**(C.2.4); common mode failure **3.19**(C.2.4); complete vehicle 3.20; component **3.21**(C.1.1); configuration data **3.22**(C.1.3); confirmation measure **3.23**(C.4.4); confirmation review **3.24**(C.4.4); controllability **3.25**(C.3.2); coupling factors **3.26**(C.2.4)
- **D**: dedicated measure **3.27**(C.4.2); degradation **3.28**(C.5); dependent failures **3.29**(C.2.4); dependent failure initiator **3.30**(C.2.4); detected fault **3.31**(C.2.3); development interface agreement **3.32**(C.4.6); diagnostic coverage **3.33**(C.2.5); diagnostic points **3.34**(C.2.5); diagnostic test time interval **3.35**(C.5); distributed development **3.36**(C.4.6); diversity **3.37**(C.4.2); dual-point failure **3.38**(C.2.3); dual-point fault **3.39**(C.2.3)
- **E**: electrical and/or electronic system **3.40**(C.1.2); element **3.41**(C.1.1); embedded software **3.42**(C.1.1); emergency operation **3.43**(C.5); emergency operation time interval **3.44**(C.5); emergency operation tolerance time interval **3.45**(C.5); error **3.46**(C.2.1); expert rider **3.47**(C.3.2); exposure **3.48**(C.3.2); external measure **3.49**(C.3.2)
- **F**: failure **3.50**(C.2.1); failure mode **3.51**(C.2.1); failure mode coverage **3.52**(C.2.5); failure rate **3.53**(C.2.1); fault **3.54**(C.2.1); fault detection time interval **3.55**(C.5); fault handling time interval **3.56**(C.5); fault injection **3.57**(C.4.2); fault model **3.58**(C.2.1); fault reaction time interval **3.59**(C.5); fault tolerance **3.60**(C.2.3); fault tolerant time interval **3.61**(C.5); field data **3.62**(C.4.6); formal notation **3.63**(C.4.5); formal verification **3.64**(C.4.5); freedom from interference **3.65**(C.2.4); functional concept **3.66**(C.1.3); functional safety **3.67**(C.3.1); functional safety concept **3.68**(C.4.1); functional safety requirement **3.69**(C.4.1)
- **H**: hardware architectural metrics **3.70**(C.2.5); hardware part **3.71**(C.1.1); hardware elementary subpart **3.72**(C.1.1); hardware subpart **3.73**(C.1.1); harm **3.74**(C.3.1); hazard **3.75**(C.3.1); hazard analysis and risk assessment **3.76**(C.3.1); hazardous event **3.77**(C.3.1)
- **I**: independence **3.78**(C.2.4); independent failures **3.79**(C.2.4); informal notation **3.80**(C.4.5); inheritance **3.81**(C.4.1); inspection **3.82**(C.4.4); intended functionality **3.83**(C.1.3); item **3.84**(C.1.1)

- **L**: latent fault **3.85**(C.2.3); lifecycle **3.86**(C.4.3)
- **M**: management system **3.87**(C.4.3); malfunctioning behaviour **3.88**(C.2.1); maximum time to repair time interval **3.89**(C.5); model-based development **3.90**(C.4.2); modification 3.91; modified condition/decision coverage **3.92**(C.4.5); motorcycle 3.93; motorcycle safety integrity level **3.94**(C.3.3); multi-core **3.95**(C.1.2); multiple-point failure **3.96**(C.2.3); multiple-point fault **3.97**(C.2.3); multiple-point fault detection time interval **3.98**(C.5)
- **N**: new development 3.99; non-functional hazard **3.100**(C.3.1)
- **O**: observation points **3.101**(C.2.5); operating mode **3.102**(C.3.1); operating time **3.103**(C.5); operational situation **3.104**(C.3.1); other technology **3.105**(C.1.2)
- **P**: partitioning **3.106**(C.1.2); passenger car 3.107; perceived fault **3.108**(C.2.3); permanent fault **3.109**(C.2.2); phase **3.110**(C.4.3); physics of failure 3.111; power take-off 3.112; processing element **3.113**(C.1.2); programmable logic device **3.114**(C.1.2); proven in use argument **3.115**(C.4.6); proven in use credit **3.116**(C.4.6)
- **Q**: quality management **3.117**(C.3.3)
- **R**: random hardware failure **3.118**(C.2.2); random hardware fault **3.119**(C.2.2); reasonably foreseeable **3.120**(C.3.2); rebuilding 3.121; redundancy **3.122**(C.4.2); regression strategy **3.123**(C.4.4); remanufacturing 3.124; residual fault **3.125**(C.2.3); residual risk **3.126**(C.3.2); review **3.127**(C.4.4); risk **3.128**(C.3.2); robust design **3.129**(C.4.2)
- **S**: safe fault **3.130**(C.2.3); safe state **3.131**(C.5); safety **3.132**(C.3.1); safety activity **3.133**(C.4.3); safety anomaly **3.134**(C.2.4); safety architecture **3.135**(C.1.2); safety case **3.136**(C.4.3); safety culture **3.137**(C.4.3); safety element out of context **3.138**(C.1.3); safety goal **3.139**(C.4.1); safety manager **3.140**(C.4.3); safety measure **3.141**(C.4.2); safety mechanism **3.142**(C.4.2); safety plan **3.143**(C.4.3); safety-related element **3.144**(C.1.3); safety-related function **3.145**(C.1.3); safety-related incident 3.146; safety-related special characteristic **3.147**(C.4.6); safety validation **3.148**(C.4.4); semi-formal notation **3.149**(C.4.5); semi-formal verification **3.150**(C.4.5); semi-trailer 3.151; series production road vehicle 3.152; service note 3.153; severity **3.154**(C.3.2); single-point failure **3.155**(C.2.3); single-point fault **3.156**(C.2.3); software component **3.157**(C.1.1); software tool **3.158**(C.4.6); software unit **3.159**(C.1.1); statement coverage

- 3.160**(C.4.5); sub-phase **3.161**(C.4.3); supply agreement **3.162**(C.4.6); system **3.163**(C.1.1); systematic failure **3.164**(C.2.2); systematic fault **3.165**(C.2.2)
- **T**: target environment **3.166**(C.1.2); technical safety concept **3.167**(C.4.1); technical safety requirement **3.168**(C.4.1); testing **3.169**(C.4.4); tractor 3.170; trailer 3.171; transducer **3.172**(C.1.2); transient fault **3.173**(C.2.2); truck 3.174; T&B vehicle configuration 3.175
  - **U**: unreasonable risk **3.176**(C.3.2)
  - **V**: variance in T&B vehicle operation 3.177; vehicle function **3.178**(C.1.3); vehicle operating state **3.179**(C.3.1); verification **3.180**(C.4.4); verification review **3.181**(C.4.4)
  - **W**: walk-through **3.182**(C.4.4); warning and degradation strategy **3.183**(C.5); well-trusted **3.184**(C.4.6); work product **3.185**(C.4.3)

### C.7.3 未展开词条速览（车辆类别与 T&B 领域等）

以下 23 条未在正文展开，多为车辆类别与卡客车（T&B）领域词汇，工程语境见附录 B；此处各给一句定位，词条号仍以 Part 1 §3 为准。

- **可用性 (availability, 3.7)** ——产品在给定条件下按需提供规定功能的能力；与安全相关但不是安全本身。
- **基础车辆 (base vehicle, 3.9)** ——加装上装设备之前的 OEM 卡客车配置。
- **上装企业 (body builder, 3.11)** ——在基础车辆上加装车身、货箱或设备的组织。
- **上装设备 (body builder equipment, 3.12)** ——装到 T&B 基础车辆上的机具、车身或货箱。
- **客车 (bus, 3.14)** ——设计用于载人及行李、座位数九座以上的机动车。注意：此处是“公共汽车”，**不是数据总线**——这是本词表最容易望文生义的一条。
- **完整车辆 (complete vehicle, 3.20)** ——装好上装设备的 T&B 基础车辆整车，如垃圾收集车、自卸车。
- **改型 (modification, 3.91)** ——从既有相关项创建新相关项；变更管理与在用证明都会引用它（深讲 ch10）。

- **摩托车 (motorcycle, 3.93)** ——两轮或整备质量不超 800 kg 的三轮机动车（不含轻便摩托车），Part 12 的主角（见 appB）。
- **全新开发 (new development, 3.99)** ——创建具有先前未规定功能的相关项/元素，或对既有功能的全新实现（对照 modification，深讲 ch10）。
- **乘用车 (passenger car, 3.107)** ——以载人为主、含驾驶员不超九座的车辆。
- **失效物理 (physics of failure, 3.111)** ——基于失效机理研究的可靠性科学方法（半导体语境见 appA）。
- **取力器 (power take-off, 3.112)** ——让卡车/牵引车动力源驱动外接设备的接口。
- **改装 (rebuilding, 3.121)** ——改变 T&B 原始配置以执行不同任务。
- **再制造 (remanufacturing, 3.124)** ——T&B 服役一段时间后按原规格拆解并换新/翻新部件。
- **安全相关事件 (safety-related incident, 3.146)** ——安全相关失效的实际发生；现场监控的记账对象（见 ch09）。
- **半挂车 (semi-trailer, 3.151) / 挂车 (trailer, 3.171) / 牵引车 (tractor, 3.170) / 卡车 (truck, 3.174)** ——T&B 车辆类别四件套：卡车运货，牵引车拖半挂，半挂以鞍座连接并把垂直载荷压在牵引车上，挂车则自承全部重量。
- **量产道路车辆 (series production road vehicle, 3.152)** ——预期用于公共道路、非原型的车辆。
- **维修提示 (service note, 3.153)** ——执行维修程序时须考虑的安全信息文档。
- **T&B 车辆配置 (T&B vehicle configuration, 3.175)** ——运行中不变的基础车辆与上装设备技术特征，如轴距、轴荷分配。
- **T&B 运行差异 (variance in T&B vehicle operation, 3.177)** ——服役期内因载货或牵引而动态特性不同的使用形态。

#### C.7.4 最易混的八对近邻

按五群走一遍之后，把全表最容易在评审桌上引起争论的近邻对收拢在一处，作为自测清单：

1. **fault / failure**——异常状态 vs 行为终止；故障可以潜伏，失效是显现的结果（C.2.1）。

2. **hazard / hazardous event**——潜在伤害来源 vs 它与运行场景的组合；HARA 按后者逐行打分 (C.3.1)。
3. **safety measure / safety mechanism**——活动或技术方案的全集 vs 做进产品里的技术方案；评审是措施但不是机制 (C.4.2)。
4. **verification / validation**——对着规格问“做得对不对”vs 对着安全目标问“目标对不对、真达到没有” (C.4.4)。
5. **confirmation review / audit / assessment**——分别检查工作产物、过程、达成的功能安全，对象不同不可互相顶替 (C.4.4)。
6. **FTTI / FHTI**——相关项的容忍时间 vs 机制的处理用时；二者属于不同对象，Part 1 词条注记用  $FDTI + FRTI < \text{相关 FTTI}$  解释及时覆盖，项目义务仍须追到适用条款与安全要求 (C.5)。
7. **single-point fault / residual fault**——都直接违背安全目标，分界只在该元素有没有安全机制覆盖 (C.2.3)。
8. **ASIL / MSIL / QM**——汽车等级、摩托车等级（须换算）、以及根本“不是一个 ASIL”的质量管理档 (C.3.3)。

### C.7.5 使用提醒

查词的读者从 C.7.2 进，按字母找到词条号与所在小节；学概念的读者从 C.1–C.5 进，沿着五条路径把邻居一起带走；做符合性工作的读者请记住三级效力：本附录白话 < 首讲章的三步走精读 < 原文词条。**词条号与标准版次**是三级之间的主要回查键之一；跨版次、勘误或上下文变化时仍须核对定义正文与适用范围。最后重复一遍 C.0 的纪律：白话引路，原文裁决——引路人不上裁判席。





## 附录 D 28 张方法表速查

Semantica 边界：本附录保留 28 张方法表的人工速查、读法和易错提示；书内不再保存 RDF 表模块、来源锚点、转录检查脚本或 runner。当前 20 个卷二章包可表达相关活动的受限场景，但 Semantica registry 尚无覆盖 28 张表、123 行、492 单元的 released executable table package。因此表格机器化目前是明确缺口：必须通过新的 Semantica package revision 登记 exact inventory、来源、CQ、反例与 oracle；在此之前不能声称自动核查已通过，也不能回退旧本体仓库。

本附录集中收录 28 张方法推荐表，作为全书唯一的逐表速查（第 5、7、10 章提供相应活动的工程语境）：Part 4 系统与整车集成 14 张、Part 6 软件生命周期 12 张、Part 8 工具资格认证 2 张，共 123 个方法行、492 个推荐单元。当前矩阵只保留人工校订的 Markdown 快照（详见 D.7）；尚无 released Semantica 全表执行包。“一句话读法”与“易错提示”由人工维护，不是可证明的自动导出物。速查卡是**选法入口**，不是“选了 ++ 就合规”的打勾清单。

**与全书二十章的衔接：**本书前十章现为叙事重写，方法表语义、独立性等级记号、FMEDA 与失效分类等器具性内容已集中由附录承载；正文各章只提供活动与主题的工程语境。本附录因此按自足深讲编写——凡指向“第 X 章”处均指主题语境，不意味着正文有逐表逐条的展开。

### D.1 读表总则：§4.3、§4.4 与三层对象

翻任何一张卡之前，先把三条总则装进脑子。它们分别来自各 Part 的 §4.3（表的解释规则）、§4.4（要求的 ASIL 适用性记法），以及本书反复强调的证据观。

**总则一：++/+/o 是推荐度，不是逐行义务。**矩阵里每个格子取三个值之一：++ 表示该方法对该 ASIL 强烈推荐 (highly recommended)、+ 表示推荐 (recommended)、o 表示标准不支持也不反对 (no recommendation for or against)。符号本身既不是 shall，也不是禁止。项目应先识别表的条目类型，再按 §4.3 形成方法选择和理由；不能看到一个 ++ 就直接推出“这一行必须执行”，也不能看到一个 o 就推出“这一行不得使用”。

**总则二：条目编号决定组合规则。**连续条目（1、2、3.....）下，对相应 ASIL 标为 ++ 或 + 的列出方法原则上都适用；可以用表外方法替代，并说明其为何满足相应要求。若已有一份理由说明不选择全部条目仍能满足要求，就不必再为每个遗漏方法逐项另写理由。**备选条目**（1a、1b、1c.....）下，应按相应 ASIL 选择适当的方法组合，方法可以来自表内或表外；推荐度不同的，宜优先考虑更高者，并说明所选组合、甚至所选单一方法为何满足相应要求。本附录 28 张表均采用字母编号，因此它们都属于后一种：没有“所有 ++ 必须全做”、“每个未选 ++ 都要找等价替代”或“只能任选一项”的默认规则。

**总则三：括号 ASIL 改变子条款的规范力度。**各 Part §4.4 规定：若 ASIL 写在括号内，相应子条款对该 ASIL 应视为 **recommendation**，而不是 **requirement**；这与 ASIL 分解的括号记法无关。它不等于标准授予一个笼统的“可裁剪”标签，也不自动取消其他适用要求。读卡时应把括号信息连同对应子条款、项目安全计划和其他约束一起判断。

最后是贯穿全书的证据观：**推荐度、项目选择、验证完成证据是三层不同的对象。**表格给出的是第一层——标准的推荐度矩阵；项目在方法选择记录里作出的组合与理由是第二层；测试报告、评审记录等工作产物构成第三层。速查卡只能帮助从第一层进入第二层，任何一格 ++ 都不能直接当作第三层的证据。本附录 D.6 的教学记录明确停在 Candidate/Planned；其他案例对象以各自实际状态为准。

### D.1.1 卡片字段说明

每张卡固定五个部分：**标题行**（表号与主题）；**溯源行**（要求条款、原文表号与 1-based 物理页、行数与格数）；**推荐矩阵**（按合法持有的受控来源人工核对录入的方法 × ASIL 全矩阵，含适用前提脚注）；**一句话读法**（这张表的主旋律）；**易错提示**（本书提示需要重点辨析的格子）。溯源行的条款号格式为“Part-条款”，如 4-7.4.2.2.2 即 ISO 26262-4:2018 §7.4.2.2.2。这里的“核对录入”是出版校订事实，不表示当前已有可执行全表包。

### D.1.2 读卡示范：三分钟读一张表

以后文的 6-Table 9（结构覆盖率）为例走一遍标准读法。第一步看**溯源行**：要求条款 6-9.4.2/9.4.3/9.4.4，说明这张表服务于软件单元验证——对象是单元级测试的充分性，不是集成级；物理页 p31 让你随时能翻回原文核对。第二步扫**矩阵的列**：竖着读 ASIL D 列得到 +/++/++——分支覆盖与 MC/DC 的推荐度高于语句覆盖。第三步扫**矩阵的行**：横着读 1a 语句覆盖得到 ++/++/+/+——它随 ASIL 升高反而降档，这类“反直觉行”正是易错提示要抓的。第四步落到**项目决定**：假设项目是 ASIL B，可以把语句覆盖与分支覆盖作为候选组合，但这不是表格自动生成的默认答案；项目还要说明所选组合或单一方法如何满足相应要求。若只选择语句覆盖，不能仅凭“少选了

一个 ++”就宣判不合规，也不能不解释为什么该单一方法足够。三分钟里最重要的动作仍是第一步——先确认表的对象、条目类型和层级，再看符号。

还要提醒一句计数口径：当前登记范围为 28 张表、123 个方法行、492 个推荐单元；这些数字只描述本附录三组人工速查表的当前库存，不表示 ISO 26262 全部方法表已经覆盖。需要决定时，应回查合法持有的原表与人工校订记录；若未来建立 released Semantica 全表包，还应同时核对其执行收据。本附录 Markdown 不是自动跟随某个本体的导出视图。

## D.2 Part 4：系统与整车集成的 14 张卡

Part 4 的方法表覆盖集成与测试阶段的三层四类：测试用例怎么推导 (Table 3，一张表供三层共用)，然后在**硬件-软件层** (Table 4-8)、**系统层** (Table 9-12)、**整车层** (Table 13-16) 各自回答四个验证目标——需求正确实现、性能/精度/时序、接口实现、稳健性 (硬件-软件层多一张安全机制有效性表)。纵向对比是读这组卡的钥匙：同名方法 (故障注入、背靠背、压力测试.....) 随层级上移，推荐度矩阵并不复制，也不存在统一的单调迁移规律——有的保持不变，有的 ++ 起点提前或后移，还有的整行全为 ++。因此必须逐表逐行核对，不能根据层级猜档位。

另一条贯穿本组的线索是 §4.4 括号语义的分布：“性能/精度/时序”与“稳健性”目标的相应子条款普遍对低 ASIL 带括号，部分接口表 (Table 6/15) 也如此，而三张“需求正确实现”表 (Table 4/9/13) 没有这些括号。准确读法是：带括号的子条款在相应 ASIL 下是 recommendation；不能进一步概括成低 ASIL 项目获得了笼统的“裁剪权”。带括号的卡在引注里逐一标明，读卡时先看引注再看矩阵。活动语境见第 5 章。

### D.2.1 测试用例推导 (1 张)

### D.2.2 Table 3 集成测试用例推导

要求条款 4-7.4.1.6 | 原文 4-Table 3 (1-based 物理页 p25) | 9 行 × A/B/C/D = 36 格

| 条目 | 方法                                    | A  | B  | C  | D  |
|----|---------------------------------------|----|----|----|----|
| 1a | 需求分析（集成测试用例）                          | ++ | ++ | ++ | ++ |
| 1b | 内外接口分析                                | +  | ++ | ++ | ++ |
| 1c | 等价类生成与分析（硬件-软件集成）                     | +  | +  | ++ | ++ |
| 1d | 边界值分析（集成）                             | +  | +  | ++ | ++ |
| 1e | 基于知识/经验的错误猜测（集成）                      | +  | +  | ++ | ++ |
| 1f | 功能依赖分析（集成）                            | +  | +  | ++ | ++ |
| 1g | 公共极限条件/时序/相关失效来源分析（见 Part 9 Clause 7） | +  | +  | ++ | ++ |
| 1h | 环境条件与运行使用用例分析                         | +  | ++ | ++ | ++ |
| 1i | 现场经验分析                                | +  | ++ | ++ | ++ |

**一句话读法：**这是 Part 4 唯一一张回答“测试用例从哪来”的表，供后面各集成测试层引用：1a 需求分析在全部 ASIL 都是 ++；其余八种推导来源随 ASIL 升高逐一转为强烈推荐。它描述推荐画像，不预先规定每个项目的组合。

**易错提示：**1b（接口分析）、1h（环境条件与运行用例）、1i（现场经验）在 ASIL B 就已升为 ++，而 1c-1g 要到 ASIL C 才升为 ++。按“到了 C 才需要认真”的直觉勾选，B 级项目会漏掉三行强推方法。

D.2.3 硬件-软件层（5 张）

D.2.4 Table 4 硬件-软件层 TSR 正确实现

要求条款 4-7.4.2.2.2 | 原文 4-Table 4（1-based 物理页 p26） | 3 行 × A/B/C/D = 12 格



| 条目 | 方法               | A  | B  | C  | D  |
|----|------------------|----|----|----|----|
| 1a | 基于需求的测试（硬件-软件层）  | ++ | ++ | ++ | ++ |
| 1b | 故障注入测试（硬件-软件层实现） | +  | ++ | ++ | ++ |
| 1c | 背靠背测试（硬件-软件层实现）  | +  | +  | ++ | ++ |

**一句话读法：**这组三种测试面向技术安全要求在硬件-软件层的正确实现：基于需求的测试全档 ++，故障注入 B 起 ++，背靠背 C 起 ++；实际选择仍按备选条目规则形成组合并说明理由。

**易错提示：**1c 背靠背测试在 A/B 是 +、在 C/D 是 ++，且方法本身需要可比较的参考实现或模型。没有相应资产时，应先记录适用性，再在所选组合的总体理由中说明如何满足验证要求；§4.3 并不要求为每个未选的备选行逐项指定等价替代。

D.2.5 Table 5 硬件-软件层性能/精度/时序

要求条款 4-7.4.2.2.3 | 原文 4-Table 5（1-based 物理页 p26） | 2 行 × A/B/C/D = 8 格

要求条款依 §4.4 括号语义适用于 ASIL (A), B, C, D：括号内 ASIL 为建议而非要求。

| 条目 | 方法                 | A | B  | C  | D  |
|----|--------------------|---|----|----|----|
| 1a | 背靠背测试（硬件-软件层性能/时序） | + | +  | ++ | ++ |
| 1b | 性能测试（硬件-软件层）       | + | ++ | ++ | ++ |

**一句话读法：**性能、精度与时序的专项验证只有两行：背靠背与性能测试；性能测试 B 起 ++，背靠背 C 起 ++。

**易错提示：**相应子条款对 ASIL A 带括号，因此在 A 下应视为 recommendation 而非 requirement。它不是一句笼统的“整表可以不做”；仍要结合其他适用要求和验证计划判断，采用表中方法时也应按 A 列理解推荐度。

D.2.6 Table 6 硬件-软件层接口实现

要求条款 4-7.4.2.2.4 | 原文 4-Table 6 (1-based 物理页 p26) | 3 行 × A/B/C/D = 12 格

要求条款依 §4.4 括号语义适用于 ASIL (A), B, C, D: 括号内 ASIL 为建议而非要求。

| 条目 | 方法                  | A | B  | C  | D  |
|----|---------------------|---|----|----|----|
| 1a | 外部接口测试<br>(硬件-软件层)  | + | ++ | ++ | ++ |
| 1b | 内部接口测试<br>(硬件-软件层)  | + | ++ | ++ | ++ |
| 1c | 接口一致性检查<br>(硬件-软件层) | + | ++ | ++ | ++ |

**一句话读法：**接口实现的三种候选方法（外部接口测试、内部接口测试、一致性检查）在 B 起全部为 ++；这表示三者在该列具有相同的最高推荐度，不表示三者自动同时成为强制活动。

**易错提示：**三行是备选条目。§4.3 允许选择适当组合，甚至在有理由时选择单一方法；表格本身没有规定“全做”或“三选一”。一致性检查与动态接口测试观察的证据不同，若只选其中一种，理由应说明它为何足以满足相应要求；未执行的方法不能被记成已执行。

D.2.7 Table 7 硬件-软件层安全机制有效性

要求条款 4-7.4.2.2.5 | 原文 4-Table 7 (1-based 物理页 p27) | 2 行 × A/B/C/D = 8 格

要求条款依 §4.4 括号语义适用于 ASIL (A), (B), C, D: 括号内 ASIL 为建议而非要求。

| 条目 | 方法                  | A | B | C  | D  |
|----|---------------------|---|---|----|----|
| 1a | 故障注入测试<br>(安全机制有效性) | + | + | ++ | ++ |
| 1b | 错误猜测测试<br>(安全机制有效性) | + | + | ++ | ++ |

**一句话读法：**这张表面向硬件-软件集成层的安全机制有效性，故障注入与错误猜测两行都是 C 起 ++；它给出方法推荐度，不单独证明机制已经有效。

**易错提示：**单元层故障注入与本表的验证对象不同，不能仅凭方法同名就自动计作硬件-软件集成证据。若一份跨层测试确实覆盖本表目标，应明确其对象、注入点、观测点和需求追溯，而不是重复打勾。

D.2.8 Table 8 硬件-软件层稳健性

要求条款 4-7.4.2.2.6 | 原文 4-Table 8 (1-based 物理页 p27) | 2 行 × A/B/C/D = 8 格

要求条款依 §4.4 括号语义适用于 ASIL (A), (B), (C), D: 括号内 ASIL 为建议而非要求。

| 条目 | 方法                 | A | B | C | D  |
|----|--------------------|---|---|---|----|
| 1a | 资源使用测试<br>(硬件-软件层) | + | + | + | ++ |
| 1b | 压力测试 (硬<br>件-软件层)  | + | + | + | ++ |

**一句话读法：**稳健性两件套（资源使用测试、压力测试）在 A–C 都是 +，到 D 升为 ++。矩阵只能直接支持这一推荐度变化，不能单凭单元格反推出标准采用该分布的原因。

**易错提示：**把“只有 D 是 ++”读成“A–C 标准不支持这些方法”是错误的：+ 仍表示推荐。由于本表是备选条目，记录重点是所选组合为何满足相应要求，而不是为每个未选的 + 方法逐项写“放弃理由”。

D.2.9 系统层 (4 张)

D.2.10 Table 9 系统层 FSR/TSR 正确实现

要求条款 4-7.4.3.2.2 | 原文 4-Table 9 (1-based 物理页 p28) | 3 行 × A/B/C/D = 12 格

| 条目 | 方法            | A  | B  | C  | D  |
|----|---------------|----|----|----|----|
| 1a | 基于需求的测试（系统层）  | ++ | ++ | ++ | ++ |
| 1b | 故障注入测试（系统层实现） | +  | +  | ++ | ++ |
| 1c | 背靠背测试（系统层实现）  | o  | +  | +  | ++ |

**一句话读法：**系统层的 FSR/TSR 实现验证与硬件-软件层（Table 4）行型相近：基于需求的测试全档 ++，故障注入 C 起 ++，背靠背 D 才 ++；这些是候选方法的推荐度分布。

**易错提示：**1c 背靠背在 A 是 o——Part 4 矩阵里少见的“无倾向”格。反过来读同样重要：o 不是禁止，A 级项目有现成模型时仍可用它，只是标准不替你背书。

D.2.11 Table 10 系统层性能/精度/时序

要求条款 4-7.4.3.2.3 | 原文 4-Table 10（1-based 物理页 p28） | 5 行 × A/B/C/D = 20 格

要求条款依 §4.4 括号语义适用于 ASIL (A), (B), (C), D: 括号内 ASIL 为建议而非要求。

| 条目 | 方法               | A | B | C  | D  |
|----|------------------|---|---|----|----|
| 1a | 背靠背测试（系统层性能/时序）  | o | + | +  | ++ |
| 1b | 故障注入测试（系统层性能/时序） | + | + | ++ | ++ |
| 1c | 性能测试（系统层）        | o | + | +  | ++ |
| 1d | 错误猜测测试（系统层）      | + | + | ++ | ++ |
| 1e | 源自现场经验的测试（系统层）   | o | + | ++ | ++ |

**一句话读法：**系统层性能/时序验证五行中，故障注入与错误猜测 C 起 ++，其余（背靠背、性能测试、现场经验）D 才 ++。这只是推荐度随 ASIL 的分布，不表示方法必须按该顺序递进或最终“加码到全套”。

**易错提示：**与 Table 5（硬件-软件层）同名的方法在本表推荐度不同——背靠背在那边 C 起 ++、这边 D 才 ++。本体用 recTable 把每个格子锁在所属表上，防的就是同名方法跨表串用推荐度。

D.2.12 Table 11 系统层接口实现

要求条款 4-7.4.3.2.4 | 原文 4-Table 11 (1-based 物理页 p29) | 4 行 × A/B/C/D = 16 格

| 条目 | 方法                | A  | B  | C  | D  |
|----|-------------------|----|----|----|----|
| 1a | 外部接口测试<br>(系统层)   | +  | ++ | ++ | ++ |
| 1b | 内部接口测试<br>(系统层)   | +  | ++ | ++ | ++ |
| 1c | 接口一致性检<br>查 (系统层) | +  | +  | ++ | ++ |
| 1d | 交互/通信测<br>试 (系统层) | ++ | ++ | ++ | ++ |

**一句话读法：**系统层接口验证四行：交互/通信测试全档 ++，外部/内部接口测试 B 起 ++，一致性检查 C 起 ++。

**易错提示：**1d 交互/通信测试是本表唯一全 ++ 行，却容易被简化成“跑一遍通信矩阵”。从工程证据看，它应覆盖与验证目标有关的元素交互场景；“报文收发正常只是起点”是本书的实践提示，不是由 ++ 单元格单独推出的附加要求。

D.2.13 Table 12 系统层稳健性

要求条款 4-7.4.3.2.5 | 原文 4-Table 12 (1-based 物理页 p29) | 3 行 × A/B/C/D = 12 格



| 条目 | 方法                                      | A  | B  | C  | D  |
|----|-----------------------------------------|----|----|----|----|
| 1a | 资源使用测试<br>(系统层)                         | o  | +  | ++ | ++ |
| 1b | 压力测试 (系<br>统层)                          | o  | +  | ++ | ++ |
| 1c | 特定环境条件<br>下抗干扰与稳<br>健性测试 (含<br>EMC/ESD) | ++ | ++ | ++ | ++ |

**一句话读法：**系统层稳健性三行里，抗干扰/EMC/ESD 测试全档 ++；资源使用与压力测试 A 档是 o、C 起 ++。

**易错提示：**同一张表里 1a/1b 在 A 是 o、1c 却全档 ++，说明整表不是同一节奏。对 A 级项目，1c 仍是强烈推荐；这不等于单元格本身把该方法改写成一条独立的 shall。

D.2.14 整车层 (4 张)

D.2.15 Table 13 整车层 FSR 正确实现

要求条款 4-7.4.4.2.2 | 原文 4-Table 13 (1-based 物理页 p30) | 4 行 × A/B/C/D = 16 格

| 条目 | 方法                       | A  | B  | C  | D  |
|----|--------------------------|----|----|----|----|
| 1a | 基于需求的测<br>试 (整车层)        | ++ | ++ | ++ | ++ |
| 1b | 故障注入测试<br>(整车层实现)        | ++ | ++ | ++ | ++ |
| 1c | 长期测试 (整<br>车层实现)         | ++ | ++ | ++ | ++ |
| 1d | 真实条件用户<br>测试 (整车层<br>实现) | ++ | ++ | ++ | ++ |

**一句话读法：**整车层 FSR 实现验证的四行 16 格全部为 ++，是当前 28 张卡中唯一的全 ++ 矩阵；准确含义是四个候选方法在各 ASIL 下都处于最高推荐度。

**易错提示：**全 ++ 不会把备选条目改成连续条目。仍应按 §4.3 选择适当组合、优先考虑高推荐度方法，并说明所选组合或单一方法如何满足要求；标准没有要求为每个未选行逐项提供“同等替代”。

D.2.16 Table 14 整车层性能/精度/时序

要求条款 4-7.4.4.2.3 | 原文 4-Table 14 (1-based 物理页 p30) | 6 行 × A/B/C/D = 24 格

要求条款依 §4.4 括号语义适用于 ASIL (A), (B), C, D: 括号内 ASIL 为建议而非要求。

| 条目 | 方法                 | A | B | C  | D  |
|----|--------------------|---|---|----|----|
| 1a | 性能测试（整车层）          | + | + | ++ | ++ |
| 1b | 长期测试（整车层性能/时序）     | + | + | ++ | ++ |
| 1c | 真实条件用户测试（整车层性能/时序） | + | + | ++ | ++ |
| 1d | 故障注入测试（整车层性能/时序）   | o | + | ++ | ++ |
| 1e | 错误猜测测试（整车层）        | o | + | ++ | ++ |
| 1f | 源自现场经验的测试（整车层）     | o | + | ++ | ++ |

一句话读法：整车层性能/时序六行分两档：性能、长期、真实用户测试 C 起 ++；故障注入、错误猜测、现场经验 A 档为 o、C 起 ++。

易错提示：1b 长期测试与 1c 真实条件用户测试在 C/D 都是 ++。若项目选择这些方法，应尽早核实所需日历周期、车辆资源和退出准则；“周期以月计”取决于项目方案，不是表格给出的固定时长。

D.2.17 Table 15 整车层接口实现

要求条款 4-7.4.4.2.4 | 原文 4-Table 15 (1-based 物理页 p31) | 3 行 × A/B/C/D = 12 格

要求条款依 §4.4 括号语义适用于 ASIL (A), (B), C, D: 括号内 ASIL 为建议而非要求。

| 条目 | 方法               | A | B | C  | D  |
|----|------------------|---|---|----|----|
| 1a | 内部接口测试<br>(整车层)  | + | + | ++ | ++ |
| 1b | 外部接口测试<br>(整车层)  | + | + | ++ | ++ |
| 1c | 交互/通信测试<br>(整车层) | + | + | ++ | ++ |

**一句话读法：**整车层接口三行同构：A/B 为 +，C/D 为 ++；对象是整车上的内外接口与交互/通信。

**易错提示：**对照系统层 Table 11：外部/内部接口测试在系统层从 B 起 ++，在整车层从 C 起 ++；交互/通信测试在系统层全档 ++，在整车层则从 C 起 ++；系统层另有接口一致性检查一行。各行变化并不统一，拿系统层的选择直接复制到整车层，档位和对象都可能错位。

D.2.18 Table 16 整车层稳健性

要求条款 4-7.4.4.2.5 | 原文 4-Table 16 (1-based 物理页 p32) | 4 行 × A/B/C/D = 16 格

要求条款依 §4.4 括号语义适用于 ASIL (A), (B), C, D：括号内 ASIL 为建议而非要求。

| 条目 | 方法                     | A | B | C  | D  |
|----|------------------------|---|---|----|----|
| 1a | 资源使用测试<br>(整车层)        | + | + | ++ | ++ |
| 1b | 压力测试 (整车层)             | + | + | ++ | ++ |
| 1c | 特定环境条件下抗干扰与稳健性测试 (整车层) | + | + | ++ | ++ |
| 1d | 长期测试 (整车层稳健性)          | + | + | ++ | ++ |

**一句话读法：**整车层稳健性四行（资源、压力、抗干扰、长期）全部 A/B 为 +、C/D 为 ++，节奏整齐。

**易错提示：**1d 长期测试同时出现在 Table 13（实现，全档 ++）和本表（稳健性，C 起 ++）。同一次测试可以产生多目标证据，但应分别追溯两个验证目的、判据和结

果；这是防止“一份报告按名称重复计数”的本书工程建议，不是方法表单元格直接规定的规则。

D.3 Part 6：软件生命周期的 12 张卡

Part 6 的 12 张表沿软件 V 模型排开：**架构设计**用什么表示法（Table 2）、怎么验证（Table 4）；**单元设计**用什么表示法（Table 5）；**单元验证**做什么（Table 7）、用例从哪来（Table 8）、测得够不够（Table 9）；**软件集成**同样三连问（Table 10/11/12）；最后**嵌入式软件测试**在哪测（Table 13）、做什么（Table 14）、用例从哪来（Table 15）。三组“方法—推导—覆盖率”三连表结构相似、矩阵各异，因而本书在多张卡中主动提示跨表对比。这些手工注释的用途是帮助读者区分同名或近似方法在不同活动、层级和表中的推荐度；它们是编辑聚焦，不是对项目错误频率或风险分布的统计。活动语境见第 7 章。

D.3.1 架构与单元设计（3 张）

D.3.2 Table 2 架构设计表示法

要求条款 6-7.4.1 | 原文 6-Table 2 (1-based 物理页 p19) | 4 行 × A/B/C/D = 16 格

| 条目 | 方法            | A  | B  | C  | D  |
|----|---------------|----|----|----|----|
| 1a | 自然语言（架构设计表示）  | ++ | ++ | ++ | ++ |
| 1b | 非正式表示法（架构设计）  | ++ | ++ | +  | +  |
| 1c | 半形式化表示法（架构设计） | +  | +  | ++ | ++ |
| 1d | 形式化表示法（架构设计）  | +  | +  | +  | +  |

**一句话读法：**架构设计表示法的推荐画像是：自然语言全档 ++，非正式表示法在 A/B 为 ++，半形式化表示法在 C/D 为 ++。表格没有规定这些表示法必须彼此“伴随”，组合由项目说明。

**易错提示：**1d 形式化表示法在任何 ASIL 都只是 +、从不升 ++——“越形式越好”不是这张表的逻辑；C/D 的强推档位给的是半形式化（如带约束的建模语言），不是定理证明器。

D.3.3 Table 4 架构设计验证方法

要求条款 6-7.4.14 | 原文 6-Table 4 (1-based 物理页 p24) | 8 行 × A/B/C/D = 32 格

| 条目 | 方法              | A  | B  | C  | D  |
|----|-----------------|----|----|----|----|
| 1a | 架构设计走查          | ++ | +  | o  | o  |
| 1b | 架构设计审查          | +  | ++ | ++ | ++ |
| 1c | 设计动态行为<br>仿真    | +  | +  | +  | ++ |
| 1d | 原型生成            | o  | o  | +  | ++ |
| 1e | 形式化验证<br>(架构)   | o  | o  | +  | +  |
| 1f | 控制流分析<br>(架构验证) | +  | +  | ++ | ++ |
| 1g | 数据流分析<br>(架构验证) | +  | +  | ++ | ++ |
| 1h | 调度分析            | +  | +  | ++ | ++ |

**一句话读法：**架构设计的验证：审查 1b 从 B 起 ++ 是主力；控制流/数据流分析与调度分析 C 起 ++；仿真与原型往 D 走才强推。

**易错提示：**1a 架构走查在 C/D 是 o，而 1b 审查是 ++，说明高 ASIL 下标准明显偏好审查。由于它们仍是备选条目，不能仅凭矩阵断言走查绝不能成为所选单一方法；项目若这样选择，须说明为何满足相应要求。原型生成在 D 为 ++，原型证据能否用于产品结论还取决于对象、配置和验证计划。

D.3.4 Table 5 单元设计表示法

要求条款 6-8.4.3 | 原文 6-Table 5 (1-based 物理页 p26) | 4 行 × A/B/C/D = 16 格

| 条目 | 方法                | A  | B  | C  | D  |
|----|-------------------|----|----|----|----|
| 1a | 自然语言（单元设计表示）      | ++ | ++ | ++ | ++ |
| 1b | 非正式表示法<br>(单元设计)  | ++ | ++ | +  | +  |
| 1c | 半形式化表示法<br>(单元设计) | +  | +  | ++ | ++ |
| 1d | 形式化表示法<br>(单元设计)  | +  | +  | +  | +  |



**一句话读法：**单元设计表示法与架构层 (Table 2) 矩阵完全相同：自然语言全 ++、非正式 A/B ++、半形式化 C/D ++、形式化恒为 +。

**易错提示：**矩阵相同不等于证据对象相同：表的对象从架构设计换成了单元设计。项目可以复用同一表示策略，但应能说明它如何覆盖单元设计要求。未来全表包必须用类似 `recTable` 的显式表归属区分两张表；当前没有可执行全表门禁，更不会替项目作出选择。

D.3.5 软件单元验证 (3 张)

D.3.6 Table 7 单元验证方法

要求条款 6-9.4.2, 6-9.4.3, 6-9.4.4 | 原文 6-Table 7 (1-based 物理页 p29) | 14 行 × A/B/C/D = 56 格

| 条目 | 方法                    | A  | B  | C  | D  |
|----|-----------------------|----|----|----|----|
| 1a | 走查 Walk-through       | ++ | +  | o  | o  |
| 1b | 结对编程 Pair-programming | +  | +  | +  | +  |
| 1c | 审查 Inspection         | +  | ++ | ++ | ++ |
| 1d | 半形式化验证                | +  | +  | ++ | ++ |
| 1e | 形式化验证                 | o  | o  | +  | +  |
| 1f | 控制流分析                 | +  | +  | ++ | ++ |
| 1g | 数据流分析                 | +  | +  | ++ | ++ |
| 1h | 静态代码分析                | ++ | ++ | ++ | ++ |
| 1i | 基于抽象解释的静态分析           | +  | +  | +  | +  |
| 1j | 基于需求的测试               | ++ | ++ | ++ | ++ |
| 1k | 接口测试                  | ++ | ++ | ++ | ++ |
| 1l | 故障注入测试                | +  | +  | +  | ++ |
| 1m | 资源使用评估                | +  | +  | +  | ++ |
| 1n | 模型-代码背靠背对比测试 (适用时)    | +  | +  | ++ | ++ |

- 条目 1n 适用前提：仅在存在能够模拟软件单元功能的模型时适用；模型与代码接受相同激励并比较结果。

**一句话读法：**全书最长的备选组有 14 种单元验证方法。推荐度分布中，静态代码分析、基于需求的测试和接口测试全档 ++；控制流/数据流分析、审查与半形式化验证在较高 ASIL 升档。它们是候选画像，不是预设“骨干套餐”。

**易错提示：**1a 走查在 C/D 是 o 而 1c 审查是 ++，与架构层同样的换挡最容易被忽略；1l 故障注入与 1m 资源评估只有 D 是 ++；1n 背靠背带适用前提——没有单元级模型时选它属于用不存在的输入打勾。

D.3.7 Table 8 测试用例推导方法

要求条款 6-9.4.2, 6-9.4.3, 6-9.4.4 | 原文 6-Table 8 (1-based 物理页 p30) | 4 行 × A/B/C/D = 16 格

| 条目 | 方法           | A  | B  | C  | D  |
|----|--------------|----|----|----|----|
| 1a | 需求分析         | ++ | ++ | ++ | ++ |
| 1b | 等价类生成与分析     | +  | ++ | ++ | ++ |
| 1c | 边界值分析        | +  | ++ | ++ | ++ |
| 1d | 基于知识/经验的错误猜测 | +  | +  | +  | +  |

**一句话读法：**单元测试用例从哪来：需求分析全档 ++，等价类与边界值 B 起 ++，错误猜测恒为 +。

**易错提示：**1d 错误猜测在各 ASIL 都是 +，而等价类与边界值从 B 起是 ++，所以标准明显偏好后两者。由于本表是备选条目，矩阵本身不能宣判某个单一方法“必然不达标”；若只选低推荐度方法，所给理由仍须说明该单一方法如何满足相应要求。

D.3.8 Table 9 结构覆盖率度量

要求条款 6-9.4.2, 6-9.4.3, 6-9.4.4 | 原文 6-Table 9 (1-based 物理页 p31) | 3 行 × A/B/C/D = 12 格

| 条目 | 方法       | A  | B  | C  | D  |
|----|----------|----|----|----|----|
| 1a | 语句覆盖     | ++ | ++ | +  | +  |
| 1b | 分支覆盖     | +  | ++ | ++ | ++ |
| 1c | MC/DC 覆盖 | +  | +  | +  | ++ |

**一句话读法：**结构覆盖率是测试充分性的“后验度量”：语句覆盖 A/B ++，分支覆盖 B 起 ++，MC/DC 只在 D 是 ++。

**易错提示：**1a 语句覆盖在 C/D 降为 +，而分支覆盖在 B-D 为 ++；MC/DC 只有 D 为 ++，在 A-C 为 +。这张表给的是备选方法推荐度：它既不能单凭“只报语句

覆盖率”宣判项目不合规，也不能单凭 B 列的 + 宣称“标准不要求 MC/DC”；具体组合要回到相应要求、项目测试策略和选择理由。

D.3.9 软件集成（3 张）

D.3.10 Table 10 软件集成验证方法

要求条款 6-10.4.2 | 原文 6-Table 10（1-based 物理页 p34） | 8 行 × A/B/C/D = 32 格

| 条目 | 方法                | A  | B  | C  | D  |
|----|-------------------|----|----|----|----|
| 1a | 基于需求的测试（软件集成）     | ++ | ++ | ++ | ++ |
| 1b | 接口测试（软件集成）        | ++ | ++ | ++ | ++ |
| 1c | 故障注入测试（软件集成）      | +  | +  | ++ | ++ |
| 1d | 资源使用评估（软件集成）      | ++ | ++ | ++ | ++ |
| 1e | 模型-代码背靠背对比测试（适用时） | +  | +  | ++ | ++ |
| 1f | 控制流与数据流验证         | +  | +  | ++ | ++ |
| 1g | 静态代码分析（软件集成）      | ++ | ++ | ++ | ++ |
| 1h | 基于抽象解释的静态分析（软件集成） | +  | +  | +  | +  |

- 条目 1e 适用前提：仅在存在能够模拟软件功能的模型时适用（if applicable）。

**一句话读法：**软件集成验证八行中，基于需求的测试、接口测试、资源使用评估和静态代码分析全档 ++；故障注入、背靠背、控制流/数据流验证从 C 起为 ++。这是推荐度分布，不是标准预装的“四项底座”。

**易错提示：**1d 资源使用评估在本表全档 ++，而同名方法在单元层（Table 7 的 1m）只有 D 是 ++。这能说明两张表的推荐矩阵不同，不能仅凭矩阵断言差异由哪一种工程原因造成；选择时须按活动与验证对象取表。

D.3.11 Table 11 集成测试用例推导方法

要求条款 6-10.4.3 | 原文 6-Table 11 (1-based 物理页 p34) | 4 行 × A/B/C/D = 16 格

| 条目 | 方法               | A  | B  | C  | D  |
|----|------------------|----|----|----|----|
| 1a | 需求分析（集成测试用例）     | ++ | ++ | ++ | ++ |
| 1b | 等价类生成与分析（集成）     | +  | ++ | ++ | ++ |
| 1c | 边界值分析（集成）        | +  | ++ | ++ | ++ |
| 1d | 基于知识/经验的错误猜测（集成） | +  | +  | +  | +  |

**一句话读法：**集成测试用例推导与单元层（Table 8）矩阵相同：需求分析全 ++、等价类/边界值 B 起 ++、错误猜测恒 +；对象换成了集成后的组件交互。

**易错提示：**行名与 Table 8 几乎一致，但等价类的划分对象已从输入域变为交互场景与配置组合——复用单元层用例清单打勾，覆盖的其实是另一张表的语义。

D.3.12 Table 12 架构层结构覆盖率

要求条款 6-10.4.5 | 原文 6-Table 12 (1-based 物理页 p35) | 2 行 × A/B/C/D = 8 格

要求条款 10.4.5 依 §4.4 以括号标注 ASIL (A)/(B)：对 A/B 本表为建议而非要求。

| 条目 | 方法    | A | B | C  | D  |
|----|-------|---|---|----|----|
| 1a | 功能覆盖率 | + | + | ++ | ++ |
| 1b | 调用覆盖率 | + | + | ++ | ++ |

**一句话读法：**架构层的覆盖率不数语句数调用：功能覆盖率与调用覆盖率两行都是 C 起 ++。

**易错提示：**拿单元层的结构覆盖率数字顶替本表是常见错位——语句/分支/MC/DC 属于 Table 9，本表要的是“架构里每个功能、每条调用关系是否被测试演练过”。表头条款对 (A)/(B) 带括号：A/B 为建议档。

D.3.13 嵌入式软件测试（3 张）

D.3.14 Table 13 软件测试环境

要求条款 6-11.4.1 | 原文 6-Table 13（1-based 物理页 p37） | 3 行 × A/B/C/D = 12 格

| 条目 | 方法               | A  | B  | C  | D  |
|----|------------------|----|----|----|----|
| 1a | 硬 件 在 环<br>(HIL) | ++ | ++ | ++ | ++ |
| 1b | ECU 网络环<br>境     | ++ | ++ | ++ | ++ |
| 1c | 实车环境             | +  | +  | ++ | ++ |

**一句话读法：**嵌入式软件测试环境的推荐画像是：HIL 与 ECU 网络环境全档 ++，实车环境从 C 起为 ++。三种环境是备选条目，应按 §4.3 选择适当组合；表格没有规定必须按三者递进，也没有规定只能三选一。

**易错提示：**1c 实车环境在 A/B 是 + 而非 o：低 ASIL 也推荐上车验证，只是不强推。反向误读“A/B 不用上车”，会增加台架未暴露的标定类问题在较晚阶段才被发现的 风险。

D.3.15 Table 14 嵌入式软件测试方法

要求条款 6-11.4.2 | 原文 6-Table 14（1-based 物理页 p37） | 2 行 × A/B/C/D = 8 格

| 条目 | 方法                     | A  | B  | C  | D  |
|----|------------------------|----|----|----|----|
| 1a | 基于需求的测<br>试（嵌入式软<br>件） | ++ | ++ | ++ | ++ |
| 1b | 故障注入测试<br>（嵌入式软件）      | +  | +  | +  | ++ |

**一句话读法：**嵌入式软件测试只有两行：基于需求的测试全档 ++，故障注入在 A-C 为 +、到 D 才升为 ++。这是本表的推荐度分布，不额外规定两种方法在项目中的先后次序。

**易错提示：**1b 故障注入在 A-C 都是 +；它在单元层、软件集成层和嵌入式层的推荐度不同。一个层级的注入证据不会因方法同名而自动满足另一层级的验证目标；哪些层级适用、各层选择什么组合，仍要回到对应活动与项目范围，不能从三张表推出“一定三层全做”。



D.3.16 Table 15 嵌入式软件测试用例推导方法

要求条款 6-11.4.3 | 原文 6-Table 15 (1-based 物理页 p37) | 6 行 × A/B/C/D = 24 格

| 条目 | 方法            | A  | B  | C  | D  |
|----|---------------|----|----|----|----|
| 1a | 需求分析（嵌入式测试用例） | ++ | ++ | ++ | ++ |
| 1b | 等价类生成与分析（嵌入式） | +  | ++ | ++ | ++ |
| 1c | 边界值分析（嵌入式）    | +  | +  | ++ | ++ |
| 1d | 错误猜测（嵌入式）     | +  | +  | ++ | ++ |
| 1e | 功能依赖分析        | +  | +  | ++ | ++ |
| 1f | 运行使用用例分析      | +  | ++ | ++ | ++ |

**一句话读法：**嵌入式软件测试用例的六个来源：需求分析全档 ++，等价类与运行用例分析 B 起 ++，边界值、错误猜测、功能依赖分析 C 起 ++。

**易错提示：**1c 边界值分析在这里 C 才 ++，而在单元层与集成层 B 就 ++——层级越高越早强推的直觉在这一格失灵；同时 1f 运行使用用例分析 B 即 ++，最容易被当成“锦上添花”而漏选。

D.4 Part 8: TCL3 与 TCL2 的 2 张卡

这两张卡的入口不在表里。第一步先按 Part 8 §11.4.1 判断 Clause 11 是否适用：只有当安全生命周期活动或任务依赖软件工具正确运行，且相关工具输出在适用过程步骤中没有被检查或验证时，该工具才进入本条款要求。确认适用后，再描述具体用途并评估工具影响（TI）与错误检测能力（TD），据此得到 TCL；TCL1 不需要资格认证方法，TCL2 查 Table 5，TCL3 查 Table 4。两表方法同名而推荐度不同，活动语境见第 10 章。

D.4.1 工具资格认证方法（2 张）

D.4.2 Table 4 TCL3 资格认证方法

要求条款 8-11.4.6.1 | 原文 8-Table 4 (1-based 物理页 p40) | 4 行 × A/B/C/D = 16 格

| 条目 | 方法                                               | A  | B  | C  | D  |
|----|--------------------------------------------------|----|----|----|----|
| 1a | 增强使用置信 (§11.4.7)                                 | ++ | ++ | +  | +  |
| 1b | 工具开发过程评估 (§11.4.8)                               | ++ | ++ | +  | +  |
| 1c | 软件工具确认 (§11.4.9)                                 | +  | +  | ++ | ++ |
| 1d | 按安全标准开发 (如 ISO 26262/IEC 61508/EN 50128/DO-178C) | +  | +  | ++ | ++ |

**一句话读法：**TCL3 工具资格认证的四种备选方法中，A/B 档对增强使用置信和开发过程评估给 ++，C/D 档对软件工具确认和按安全标准开发给 ++。这只是矩阵呈现的推荐度换挡。

**易错提示：**进这张表前应先完成 §11.4.1 适用性判断，再按具体工具用途做 TI/TD→TCL 评估；跳过用途与适用范围直接选资格认证方法，所得结论没有明确对象。表内 1a 在 C/D 降为 +，说明使用经验仍被推荐，但推荐度低于 1c/1d。

D.4.3 Table 5 TCL2 资格认证方法

要求条款 8-11.4.6.1 | 原文 8-Table 5 (1-based 物理页 p40) | 4 行 × A/B/C/D = 16 格

| 条目 | 方法                 | A  | B  | C  | D  |
|----|--------------------|----|----|----|----|
| 1a | 增强使用置信 (§11.4.7)   | ++ | ++ | ++ | +  |
| 1b | 工具开发过程评估 (§11.4.8) | ++ | ++ | ++ | +  |
| 1c | 软件工具确认 (§11.4.9)   | +  | +  | +  | ++ |
| 1d | 按安全标准开发            | +  | +  | +  | +  |

**一句话读法：**TCL2 与 TCL3 使用同名四法，但各行推荐矩阵不同：增强使用置信与开发过程评估到 C 仍是 ++，软件工具确认只有 D 是 ++，按安全标准开发恒为 +。

**易错提示：**两张表的方法行同名，但推荐单元不同；若把 TCL2 的矩阵用于 TCL3，就会读错方法推荐度。先确认 TCL，再翻对应的表。

D.5 按场景跨表导航

一种使用方式是从“我要证明什么”出发。下面按验证目标反查候选卡；命中多张卡时，先判断对应活动和层级是否适用，再进入相应卡片。一个层级的证据不会因为方法同名就自动满足另一层级，但范围足够且追溯清楚的共同证据也不必机械复制。

| 我要证明什么       | 候选卡                                                            | 提示                               |
|--------------|----------------------------------------------------------------|----------------------------------|
| 测试用例来源充分     | 4-Table 3; 6-Table 8 / 11 / 15                                 | 按层取表：单元/软件集成/嵌入式/系统集成各有一张        |
| 安全需求被正确实现    | 4-Table 4（硬件-软件）/ 9（系统）/ 13（整车）；6-Table 14（嵌入式软件）              | 基于需求的测试在四张表里均为全档 ++ 候选方法         |
| 性能、精度与时序达标   | 4-Table 5 / 10 / 14                                            | 三张表同名方法档位不同，按层选                  |
| 接口正确实现       | 4-Table 6 / 11 / 15; 6-Table 7 条目 1k; 6-Table 10 条目 1b         | 静态一致性检查与动态接口测试观察不同证据，组合由项目论证     |
| 系统经得起折腾（稳健性） | 4-Table 8 / 12 / 16                                            | 系统层 EMC 行全档 ++，仍按备选条目规则选法        |
| 安全机制真的有效     | 4-Table 7; 6-Table 7 条目 1l; 6-Table 10 条目 1c; 6-Table 14 条目 1b | 故障注入在各层档位和验证对象不同，逐层判断适用性         |
| 测试测得够充分      | 6-Table 9（单元结构覆盖）; 6-Table 12（架构层覆盖）                           | 两张覆盖率表对象不同，数字不能互替                |
| 架构/单元设计写得可验证 | 6-Table 2 / 5(表示法); 6-Table 4（架构验证）                            | 形式化恒为 +，强推档在半形式化                 |
| 测试环境选得对      | 6-Table 13                                                     | HIL 与 ECU 网络全档 ++，实车 C 起 ++      |
| 开发工具可以信任     | 8-Table 4（TCL3）/ 5（TCL2）                                       | 先做 §11.4.1 适用性判断，再按用途做 TI/TD→TCL |

两条使用惯例：其一，同一方法在多层出现时，先比较各层验证对象，不能仅凭“在单元层做过”就给集成层记完成；若同一活动确实覆盖多层目标，应逐项给出追溯。

其二，导航表只指到卡，适用性、方法组合和证据充分性仍要回到 D.1、对应规范性条款与所在主章判断。

**一次完整的导航走查：**假设你在 EPS 项目里负责“扭矩合理性检查这个安全机制到底有没有用”的证据。从导航表相应行会找到四处故障注入候选：软件单元验证的 6-Table 7 条目 1l、软件集成的 6-Table 10 条目 1c、硬件-软件集成的 4-Table 7，以及系统层实现验证的 4-Table 9 条目 1b。正确动作不是立即安排“四层四轮”，而是先识别项目实际经历哪些活动、每层要验证的命题、已有测试的对象与覆盖范围，再分别按备选条目规则形成组合。若选择在多层使用故障注入，注入点、观测点和判据应对应各层目标；若一份测试覆盖多个目标，应把追溯写明。这个例子强调的是“同名方法不等于同一证据”，不是新增一条“任何层都不得缺席”的义务。

## D.6 人工注释：组合理由与适用性边界

当前**没有**覆盖 28 张表的 released Semantica package，也没有可执行的全表转录门禁。历史旧仓门禁曾比较 RDF 与 MinerU 表格提取件的部分已编码字段，但那是迁移来源证据，不是现行执行能力；它也从未证明本附录 Markdown 逐格相同，更不能证明任何项目决定正当。人工卡片需要分别处理三类判断并留痕：

**组合理由。**备选条目组里选了哪些方法、为什么这个组合或单一方法对本项目的验证目标足够。推荐度较高的方法应优先考虑，但选择记录不能停在“照抄 ++ 列”，还要把方法与对象、失败模式和判据联系起来。

**适用性与未选项说明。**本附录各表都是备选条目；只要所选组合或单一方法已有理由说明其满足相应要求，§4.3 不要求再为每个未选的 ++ 逐项寻找“同等替代”。但诸如“无可执行模型”这类适用性事实应记录清楚，“排期不够”只能说明资源问题，不能证明所选组合充分。若另用表外方法，也应把它纳入组合理由。

**结果充分性与放行授权。**验证活动依据预先规定的对象、判据与结果评价结论是否充分；确认评审、审核与功能安全评估再按各自对象发挥作用。最终放行由指定责任主体依据安全档案、适用确认措施和受控发布配置作出。方法表、验证责任人或任一确认措施都不会单独背书具体测试报告或自动产生放行结论。

一份可审查的方法选择记录可以长什么样？下面给出教学示例的骨架（内容为示例，非任何项目结论）：

| 字段    | 示例内容                                                                                                                        |
|-------|-----------------------------------------------------------------------------------------------------------------------------|
| 对象与层级 | 软件单元 SWU_TorquePlausibility, 单元验证 (6-Table 7/8/9)                                                                           |
| ASIL  | D (继承自 TSR_TorquePlausibility, 未分解)                                                                                         |
| 选用组合  | Table 7: 1c 审查、1d 半形式化验证、1f/1g 控制流与数据流分析、1h 静态分析、1j 基于需求的测试、1k 接口测试、1l 故障注入、1m 资源使用评估; Table 8: 1a+1b+1c; Table 9: 1a+1b+1c |
| 适用性   | Table 7 的 1n 模型-代码背靠背为 if applicable; 本示例无可执行单元模型, 因此记为不适用, 不列作“放弃的 ++”                                                     |
| 推荐度说明 | 上述组合覆盖当前适用的 ASIL D ++ 候选; 另选 Table 9 的 1a (+) 作为补充度量。此记录方式是教学选择, 不表示备选表必须全选 ++                                              |
| 组合理由  | 各方法所覆盖的失败模式、判据与证据位置见验证计划 VP-SWU-01; 是否充分仍待执行结果与评审确认                                                                         |
| 记录状态  | Candidate——组合已评审, 验证执行 Planned                                                                                              |

注意最后一行：状态字段是这份记录里最诚实的部分。这个教学示例停在 Candidate/Planned，不冒充执行完成；其他章与仓库对象应读取各自的实际状态，不能由本附录一概宣称都处于 Planned。卡片上的任何符号都不构成“已验证”的暗示。

## D.7 本体化实践：Semantica 全表包的待建合同

旧本体仓库曾有三份方法表 RDF 模块、来源锚点与转录检查；这些执行资产不再留在书旁，当前 Semantica registry 也尚未登记覆盖全 28 表的 released successor。现状只有可读出版链：编辑者维护本文件中的矩阵、“一句话读法”和“易错提示”。未来机器链应沿 MethodTableSet → MethodTable → MethodRecommendation 保存表、方法、ASIL 与推荐度，但必须作为 Semantica package 的显式 delta 建立，不能从旧路径临时加载。

历史转录曾结合原 PDF、pdftotext -layout 与 MinerU 表格复核；识别损坏处（如 Part 6 Table 7 续表把条目 1l 识别为 11）以校订留痕。当前更新纪律是先回到用户合法持有的原表核对，再人工同步本附录并逐卡复核。只有未来 Semantica package 同时登记原表坐标/哈希、28/123/492 exact inventory、正反例、oracle 和 Markdown 生成校验后，才能把“机器转录通过”或“重跑导出”写成已实现能力。

候选模型中的 recTable 属性值得单独一提：同名方法（如“故障注入测试”）在不同表可以是不同 VerificationMethod 个体，每个推荐单元再指向所属表。未来门禁



可据此检测部分跨表错配；在 package 尚未 released 的当前，这只是建模要求，不能报告为已经执行的保护，也不可能“结构上防死”所有 Markdown 抄写错误。

最后交代本附录与主章的分工。第 5、7、10 章提供活动语境，速查卡集中展示人工矩阵与易错读法。若卡片与原表出现冲突，以用户合法持有的正版标准文本和受控复核结论为裁决依据；若将来已经存在 Semantica package，还必须把差异作为新 revision 受控更新，而不能只改书或只改机器资产。